1. Executive Summary

This notebook develops, solves, validates, and interprets a capacity-constrained, latency-aware network routing optimization problem involving four traffic classes:

- UK-L: latency-sensitive traffic destined for the UK;
- UK-T: latency-tolerant traffic destined for the UK;
- US-L: latency-sensitive traffic destined for the US;
- US-T: latency-tolerant traffic destined for the US.

Traffic originates at five source nodes and competes for shared capacity across a nine-node network containing 11 full-duplex physical links, represented mathematically as 22 independent directed arcs.

Demand is modeled separately for each source-stream combination, producing 20 distinct commodities and 126 Mbps of total baseline offered demand.

Two routing architectures are evaluated:

- **splittable routing**, in which a source-stream commodity may use multiple candidate paths;
- **single-path routing**, in which each source-stream commodity may use at most one path while still allowing partial delivery.

The optimization uses a lexicographic decision structure.

First, total delivered throughput is maximized. Second, the Stage 1 admission decision is preserved while latency exposure is minimized for UK-L and US-L traffic.

The analysis produces several important findings.

Naive independent minimum-latency routing is infeasible because multiple commodities compete for the same attractive links. Under naive routing, B → C would require approximately 61 Mbps against only 18 Mbps of capacity, or approximately 338.9% utilization. C → UK, C → US, and A → C are also substantially overloaded.

Joint optimization reduces this congestion but cannot eliminate the underlying physical capacity limitation.

Both the splittable and single-path Stage 1 formulations achieve the same

maximum baseline throughput:

$$T^*_{split} = T^*_{single} = 83 \text{ Mbps}.$$

From 126 Mbps of offered demand, 43 Mbps remains undelivered, producing a baseline network delivery rate of approximately:

$$65.87\%.$$

The equality of the two Stage 1 objective values is important. It shows that commodity splitting is not required to achieve maximum baseline throughput. In fact, the optimal splittable Stage 1 solution does not require any commodity to use multiple positive-flow paths.

However, equal throughput does not imply equal service allocation.

The two formulations admit different source-stream commodities, use the physical network differently, and produce different source- and traffic-class outcomes.

The secondary latency optimization also demonstrates that routing flexibility can have value even when it provides no additional throughput.

Within the splittable formulation, preserving the exact Stage 1 admission vector and minimizing latency reduces total sensitive-traffic latency exposure from approximately:

$$850$$

to:

$$644 \text{ Mbps} \cdot \text{ms},$$

a reduction of approximately:

$$24.24\%.$$

The final splittable solution also produces lower observed average sensitive latency than the final single-path solution:

$$UK - L : 18.25 \text{ ms vs. } 21.17 \text{ ms},$$

and:

$$US - L : 19.56 \text{ ms vs. } 24.78 \text{ ms}.$$

Because the two models preserve different Stage 1 admission vectors, these final latency differences should be interpreted as complete-pipeline performance

differences rather than a perfectly controlled causal effect of splitting alone.

Infrastructure analysis shows that utilization and true bottleneck value are different concepts.

A directed link is treated as a true local throughput bottleneck only when increasing that link by 1 Mbps increases optimized end-to-end throughput.

The marginal-capacity experiment identifies:

- B → D;
- C → UK;
- C → US;

as links where an additional 1 Mbps increases Stage 1 throughput from 83 to approximately 84 Mbps under both tested routing formulations.

This demonstrates that capacity investment should be based on marginal end-to-end service value rather than saturation alone.

Stress testing reinforces the physical-network interpretation.

At 60% of baseline demand, all 75.6 Mbps is delivered.

At 150% demand, offered demand increases to 189 Mbps but maximum throughput remains 83 Mbps.

Capacity degradation reduces service further:

- internal-corridor degradation: 78.1 Mbps;
- fragmented internal capacity: 72.85 Mbps;
- destination-exit degradation: 69.25 Mbps.

The destination-exit degradation scenario produces the largest throughput reduction among the tested capacity-degradation scenarios, reinforcing the importance of destination-facing capacity.

Importantly, the splittable and single-path formulations achieve identical Stage 1 throughput in every tested scenario.

Therefore, the tested network provides no evidence that splitting improves Stage 1 throughput under the baseline or stress conditions evaluated.

Its demonstrated value appears instead in latency optimization and routing flexibility.

Finally, maximum throughput does not guarantee fair service allocation.

The baseline fairness analysis shows a minimum commodity delivery ratio of zero under both Stage 1 formulations, meaning at least one commodity receives no service even though total throughput is optimal.

The production recommendation is therefore a layered, policy-aware optimization architecture:

$$\text{Throughput} \rightarrow \text{Fairness / SLA Policy} \rightarrow \text{Sensitive Latency} \rightarrow \text{Operational}$$

Operationally, single-path routing is attractive when simplicity is valuable and service requirements are satisfied.

Splittable-routing capability should nevertheless be retained where its additional flexibility produces measurable latency, resilience, or future scenario value.

For infrastructure planning, marginal throughput value should be combined with upgrade cost, resilience, implementation difficulty, and expected demand growth.

1.1 Introduction

Modern communication networks must route multiple traffic classes through shared infrastructure while simultaneously balancing capacity, latency, service quality, and operational simplicity.

A route that is ideal for one individual traffic stream may not be ideal for the network as a whole.

When several commodities independently select the lowest-latency path, they may compete for the same constrained links, creating congestion even while alternate network capacity remains available elsewhere.

The resulting problem is therefore not simply a shortest-path problem.

It is a **multi-commodity network optimization problem** in which many source-specific traffic demands compete simultaneously for shared directed-link capacity.

This notebook studies that problem using a synthetic nine-node network with traffic destined for two exits: UK and US.

The analysis intentionally examines the problem from several perspectives.

First, it characterizes the physical network, traffic demand, and feasible routing alternatives.

Second, it establishes a naive minimum-latency baseline to demonstrate why independent route selection is insufficient.

Third, it formulates and solves both splittable and single-path optimization models.

Fourth, it uses a lexicographic objective to separate maximum throughput from latency-sensitive routing quality.

Fifth, it analyzes infrastructure utilization and tests true bottlenecks using direct capacity perturbation.

Sixth, it stress-tests the routing architectures under changing demand and degraded capacity.

Finally, it evaluates fairness, business policy, assumptions, limitations, and production implications.

The objective is not only to obtain an optimal routing solution.

The broader objective is to understand:

> **why the optimal solution behaves as it does, which physical constraints matter, when routing flexibility creates value, and how those technical findings translate into operational and investment decisions.**

---

2. Business Problem

The network carries four traffic classes from multiple source locations toward two valid destination exits.

Some traffic is latency-sensitive.

For these applications, excessive delay may materially reduce service quality.

Other traffic is latency-tolerant and can accept longer routes when doing so improves overall network performance.

All traffic classes compete for the same finite directed-link bandwidth.

This creates three related business challenges.

First, traffic must be routed efficiently enough to maximize the amount of demand that reaches the correct destination.

Second, limited network capacity must be allocated across competing sources and traffic classes in a way that is operationally and commercially acceptable.

Third, the organization must determine whether service limitations are caused primarily by poor routing decisions or by genuine physical capacity shortages requiring infrastructure investment.

The problem is therefore not simply:

> **What is the shortest route?**

It is:

> **How should limited network capacity be allocated and routed jointly across all traffic so that the network maximizes delivered service, protects latency-sensitive traffic where possible, maintains physical feasibility, and identifies where additional infrastructure would create measurable value?**

The optimization must ensure that:

- no directed link exceeds its available capacity;
- no source-stream commodity delivers more than its offered demand;
- UK-bound traffic leaves only through UK;
- US-bound traffic leaves only through US;
- unroutable traffic is explicitly reported as lost;
- single-path commodities use at most one route;
- partial delivery remains permitted;
- latency-sensitive routing is improved only after throughput has been maximized.

The solved results show that this distinction between routing and infrastructure is important.

Joint optimization dramatically improves feasibility relative to naive minimum-latency routing, but even optimal routing can deliver only 83 of the 126 Mbps offered in the baseline network.

Therefore, routing optimization improves how existing capacity is used, while the remaining service deficit reflects genuine end-to-end capacity limitations.

2.1 Problem Definition

Let:

$$G = (V, E)$$

represent the directed network.

Each directed arc:

$$e \in E$$

has:

- directional capacity C_e in Mbps;
- fixed latency L_e in milliseconds.

Traffic is defined at the source-stream level.

For source:

$$s \in S$$

and stream:

$$k \in K,$$

let:

$$D_{s,k}$$

represent offered demand.

The four streams are:

$$K = \{UK - L,\ UK - T,\ US - L,\ US - T\}.$$

Each source-stream pair forms an independent commodity:

$$(s, k).$$

For each commodity, a set of valid source-to-destination candidate paths:

$$P_{s,k}$$

is generated.

A path must:

- originate at the correct source;
- terminate at the required UK or US exit;
- avoid the opposite exit;
- obey the physical network topology.

The primary decision is how much flow:

$$f_{s,k,p}$$

to route on each candidate path.

Delivered demand is:

$$z_{s,k} = \sum_{p \in P_{s,k}} f_{s,k,p}.$$

The primary optimization objective is:

$$\max \sum_{s,k} z_{s,k}.$$

This determines the maximum feasible network throughput.

After the Stage 1 admission quantities are determined, the secondary optimization preserves those quantities and minimizes latency exposure for:

$$K_L = \{UK - L,\ US - L\}.$$

For the single-path variant, binary route-selection variables additionally enforce:

$$\sum_{p \in P_{s,k}} y_{s,k,p} \leq 1.$$

Partial delivery remains permitted.

The analytical problem therefore contains two distinct questions:

Feasibility / Capacity Question

How much offered demand can physically reach the correct destination?

Routing-Quality Question

Among throughput-optimal solutions, how should admitted traffic be routed to improve sensitive latency and operational performance?

The later fairness analysis introduces a third question:

Allocation-Policy Question

When all demand cannot be served, how should scarce capacity be distributed across commodities?

These questions are intentionally separated because physical feasibility, business policy, and service quality are different decision layers.

2.2 Analytical Questions

The notebook is designed to answer the following decision questions:

1. What is the maximum amount of baseline demand that the network can deliver?
2. Why does independent minimum-latency routing fail under shared capacity?
3. Does splittable routing increase throughput relative to single-path routing?
4. What service-quality value does routing flexibility provide after throughput has been fixed?
5. Which physical links are genuinely throughput-constraining rather than merely highly utilized?
6. How robust are the routing conclusions under higher demand and degraded capacity?
7. How should fairness, SLAs, and business priorities be incorporated when some demand must remain unserved?

2.3 Methodology Overview

The analysis follows a staged methodology.

Phase 1 — Clarify the Problem

Resolve ambiguities concerning:

- source-specific demand;
- single-path scope;
- partial delivery;
- split-traffic latency;
- queueing;
- node capacity;
- traffic priority;

- numerical data availability.

Phase 2 — Construct and Validate the Synthetic Test Environment

Create heterogeneous:

- directed-link capacities;
- link latencies;
- source-stream demand.

Validate:

- node count;
- link count;
- directional arc count;
- total demand;
- candidate-path consistency.

Phase 3 — Establish the Naive Routing Baseline

Route each commodity independently using its minimum-latency path.

Measure:

- link load;
- capacity violations;
- structural congestion.

This establishes why a global optimization model is required.

Phase 4 — Optimize Baseline Routing

Solve:

1. splittable Stage 1 maximum throughput;
2. splittable Stage 2 sensitive latency;
3. single-path Stage 1 maximum throughput;
4. single-path Stage 2 sensitive latency.

Validate:

- solver status;
- demand accounting;
- capacity feasibility;

- path-selection logic;
- throughput preservation.

Phase 5 — Compare Service and Infrastructure Outcomes

Compare the models across:

- throughput;
- lost demand;
- commodity admission;
- source service;
- traffic-class service;
- latency;
- link utilization;
- residual headroom;
- saturated links;
- routing complexity.

Phase 6 — Identify True Bottlenecks

Increase each directed link by:

$$1 \text{ Mbps}$$

and re-solve.

Define:

$$MV_e = T^*(C_e + 1) - T^*(C_e).$$

Use this marginal throughput value to distinguish:

- high utilization;
- true local capacity value.

Phase 7 — Stress Test the Network

Evaluate:

- low demand;
- high demand;
- internal-corridor degradation;
- destination-exit degradation;
- fragmented capacity.

Compare split and single-path routing under identical scenario conditions.

Phase 8 — Evaluate Policy, Fairness, and Production Implications

Measure commodity- and source-level delivery ratios.

Identify starvation risk.

Develop extensions for:

- max-min fairness;
- SLAs;
- traffic priority;
- adaptive routing;
- production capacity planning.

This methodology intentionally separates:

$$\text{Physical Feasibility} \rightarrow \text{Allocation Policy} \rightarrow \text{Routing Quality} \rightarrow \text{Infrastructu}$$

Confirmed Operational Interpretation

The network is modeled as a single independent snapshot.

Traffic that cannot be routed during that snapshot is lost rather than buffered or carried forward.

Nodes themselves do not impose processing-capacity limits. Network capacity is determined by the bandwidth of the links connecting those nodes.

Latency-sensitive traffic does not receive automatic admission priority. Its special treatment occurs through the secondary latency objective.

3. Network Definition

The network contains nine nodes:

- S1
- S2
- S3

- A
- B
- C
- D
- UK
- US

Traffic-generating nodes:

- S1
- S2
- S3
- B
- C

Transit-only nodes:

- A
- D

Exit nodes:

- UK
- US

The physical links are:

1. S1 - A
2. S2 - A
3. S3 - B
4. A - B
5. A - C
6. B - C
7. B - D
8. C - UK
9. C - US
10. D - UK
11. D - US

Each physical link is full-duplex. Therefore, for modeling purposes, each physical link may be represented as two directed arcs with independent directional capacity.

Modeling Representation

The physical topology is shown as an undirected graph for readability.

For optimization, however, every full-duplex physical link will later be converted into two directed arcs.

For example:

$$A \leftrightarrow C$$

becomes:

$$A \rightarrow C$$

and:

$$C \rightarrow A$$

Each direction has its own capacity constraint.

In [37]:
```
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

In [38]:
```
# Create an undirected representation for visualization.
# Directional arcs will be created later for optimization.

G = nx.Graph()

nodes = [
    "S1", "S2", "S3",
    "A", "B", "C", "D",
    "UK", "US"
]

edges = [
    ("S1", "A"),
    ("S2", "A"),
    ("S3", "B"),
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("B", "D"),
    ("C", "UK"),
    ("C", "US"),
    ("D", "UK"),
    ("D", "US")
]
```

```
G.add_nodes_from(nodes)
G.add_edges_from(edges)

print(f"Number of nodes: {G.number_of_nodes()}")
print(f"Number of physical links: {G.number_of_edges()}")
```

```
Number of nodes: 9
Number of physical links: 11
```

In [39]:

```
pos = {
    "S1": (0, 3),
    "S2": (0, 2),
    "S3": (0, 1),

    "A": (2, 2.5),
    "B": (2, 1.3),

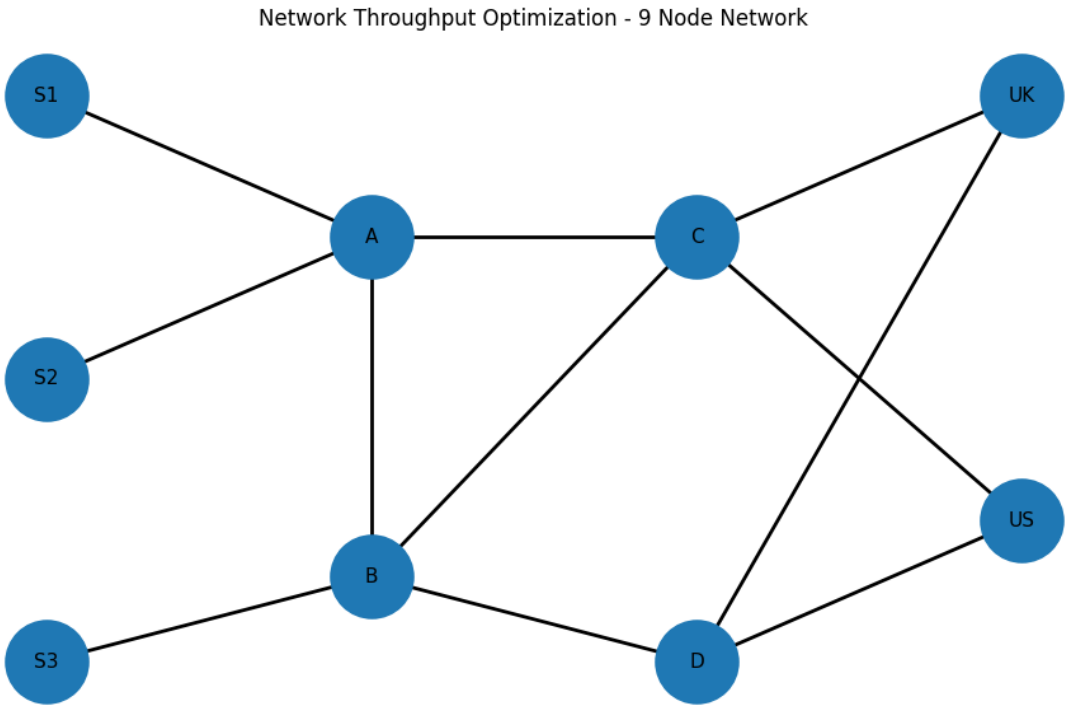
    "C": (4, 2.5),
    "D": (4, 1.0),

    "UK": (6, 3.0),
    "US": (6, 1.5)
}

plt.figure(figsize=(12, 7))

nx.draw_networkx(
    G,
    pos=pos,
    with_labels=True,
    node_size=2200,
    font_size=11,
    width=2
)

plt.title("Network Throughput Optimization - 9 Node Network")
plt.axis("off")
plt.show()
```



4. Traffic Streams

The optimization contains exactly four traffic streams.

Stream	Required Exit	Latency Sensitive
UK-L	UK	Yes
UK-T	UK	No
US-L	US	Yes
US-T	US	No

The latency-sensitive streams should experience as little end-to-end latency as possible.

Latency does not affect the objective for UK-T and US-T except indirectly through competition for shared capacity.

The four classes may originate at multiple source nodes.

The routing unit is therefore a **source-stream pair**.

Examples include:

- S1 / UK-L
- S2 / UK-L

- B / US-T
- C / US-L

A source-stream pair is treated separately during routing because its physical starting location determines which links it must consume.

Results may later be aggregated back to the four traffic classes.

5. Bandwidth and Latency

Two link characteristics govern routing.

Bandwidth

Bandwidth determines how much traffic can simultaneously use a link.

For a directed link (e):

$$\text{Flow}_e \leq \text{Capacity}_e$$

If a link has a capacity of 30 Mbps, the total traffic from all four streams using that directional link cannot exceed 30 Mbps.

Capacity does **not** add along a path.

For example, if a path contains four links, each with 30 Mbps capacity:

$$30 \rightarrow 30 \rightarrow 30 \rightarrow 30$$

the path does **not** have 120 Mbps capacity.

Instead:

$$\text{Path Capacity} = \min(30, 30, 30, 30) = 30 \text{ Mbps}$$

The smallest-capacity link limits the amount of end-to-end traffic that can travel along that path.

More generally, for a path (p):

$$C_p = \min_{e \in p} C_e$$

where:

- (C_p) is the effective capacity of path (p);

- (C_e) is the capacity of link (e).

Latency

Latency behaves differently.

End-to-end path latency is the sum of the latencies of all links used along the path.

For a path (p):

$$L_p = \sum_{e \in p} L_e$$

where:

- (L_p) is the total latency of path (p);
- (L_e) is the latency of link (e).

For example, if a path uses four links and each contributes 20 ms:

$$20 + 20 + 20 + 20 = 80 \text{ ms}$$

Therefore:

- **Capacity is constrained by the minimum/bottleneck capacity along a path.**
- **Latency is additive along the path.**

Residual Capacity

Although the theoretical capacity of an isolated path is bounded by its minimum-capacity link, the actual amount of additional traffic that can be routed over that path depends on the **residual capacity** remaining after other streams are routed.

Therefore:

$$\text{Available Path Capacity} \leq \min_{e \in p} C_e$$

and may be substantially smaller when paths share links.

6. Optimization Objectives

The problem contains two objectives.

Objective 1 - Maximize Delivered Throughput

Let:

$$z_{k,s}$$

represent the amount of traffic successfully delivered for stream k originating at source s.

The primary objective is:

$$\boxed{\max \sum_{k \in K} \sum_{s \in S_k} z_{k,s}}$$

where:

$$K = \{UK\text{-}L,\ UK\text{-}T,\ US\text{-}L,\ US\text{-}T\}$$

All delivered Mbps contribute equally to this objective.

Objective 2 - Minimize Sensitive-Traffic Latency

The latency-sensitive streams are:

$$K_L = \{UK\text{-}L,\ US\text{-}L\}$$

After maximum throughput has been found, the amount delivered for each source-stream pair is held fixed at its Stage 1 value:

$$z_{k,s} = z^*_{k,s}$$

The second-stage optimization then changes the routing, not the admission decision.

Among those throughput-preserving solutions, the model minimizes latency for UK-L and US-L.

This lexicographic approach avoids assigning an arbitrary numerical conversion between Mbps and milliseconds.

7. Confirmed Problem Constraints

7.1 Purpose

The initial problem statement contained several modeling elements that required clarification before the optimization could be finalized.

Those questions have now been resolved.

This section therefore records the confirmed constraints and operating rules that define the optimization problem.

These items should be treated as requirements of the assessment rather than assumptions introduced by the analysis.

7.2 Source-Stream Demand Is Modeled Separately

Demand is defined at the source-stream level.

For source:

$$s \in S$$

and traffic stream:

$$k \in K,$$

the offered demand is:

$$D_{s,k}.$$

Therefore, traffic such as:

S1 / UK-L;

S2 / UK-L;

B / UK-L;

is modeled as separate commodities even though each belongs to the same aggregate UK-L traffic class.

This is important because feasible routing depends on the location where traffic enters the network.

The optimization therefore contains:

$$20$$

source-stream commodities.

7.3 Delivered Demand Cannot Exceed Offered Demand

For every commodity:

$$0 \leq z_{s,k} \leq D_{s,k}.$$

The optimizer may deliver:

all demand;

part of the demand;

none of the demand.

Any undelivered amount is:

D_{s,k}-z_{s,k}.

This amount is counted as lost demand.

7.4 Partial Delivery Is Allowed

Prateek confirmed that partial delivery is permitted.

This distinction is especially important for the single-path formulation.

Single-path routing does not mean:

the entire source-stream demand must fit on one route or be rejected complete

Instead, it means:

whatever portion of the commodity is delivered must use no more than one rout

Therefore:

0<z_{s,k}<D_{s,k}

is allowed under the single-path model.

This makes the formulation a single-route partial-admission model, not an all-or

7.5 Single-Path Routing Applies Per Source-Stream Commodity

For the single-path formulation, each source-stream commodity may use at mo

Let:

y_{s,k,p}\in{0,1}

$$indicatewhetherpath\$p\$isselected.Then:$$

\sum_{p\in P_{s,k}} y_{s,k,p} \leq 1.

The one-path rule therefore applies independently to commodities such as:

S1 / US-L;

B / US-L;

C / US-L.

Different source contributions to the same aggregate traffic class may use diffe

7.6 Directed Link Capacity Must Be Respected

Each physical link is full duplex and is represented as two independent directed

For directed arc:

e\in E,

$$capacityis:$$

C_e.

$$Thesumofallcommoditytrafficusingthatarcmustsatisfy:$$

\sum_{s,k,p\in p} f_{s,k,p} \leq C_e.

$Capacityinonedirectionisindependentofcapacityinthereversedirection.For$

C_{A\rightarrow C}

$$and:$$

C_{C\rightarrow A}

are separate resources.

7.7 Traffic Must Exit Through the Correct Destination

UK-bound traffic must terminate at:

UK.

$$US-boundtrafficmustterminateat:$$

US.

A candidate path may not:

terminate at the wrong exit;

pass through the wrong exit and continue elsewhere.

This preserves destination correctness.

7.8 No Queueing, Buffering, or Carryover

The optimization represents one independent snapshot.

Traffic that cannot be routed during that snapshot is immediately counted as los

There is:

no buffering;

no queue;

no backlog;

no carryover into a future period.

Therefore, the model does not contain a state variable representing stored traffi

7.9 Nodes Have No Independent Capacity Constraint

Nodes are not constrained by:

CPU;

switching capacity;

memory;

processing throughput.

Only directed-link capacities constrain the physical network.

This means that congestion occurs on links rather than at nodes.

7.10 Latency Is Additive Across Links

For candidate path:

p,

$$pathlatencyis : \sum_{e \in p} L_e.$$

Each link has a fixed latency value.

The model does not currently include queue-dependent delay.

7.11 No Hard Latency Threshold Is Required

Latency-sensitive traffic does not have a hard feasibility requirement such as:

$$L_p \leq L_{max}.$$

Instead, latency-sensitive performance is optimized through the secondary objective after throughput has been maximized.

Therefore, a higher-latency route remains feasible if required by network capacity.

7.12 No Automatic Sensitive-Traffic Admission Priority

Prateek confirmed that latency-sensitive traffic does not automatically receive Stage 1 admission priority.

The Stage 1 objective values every delivered Mbps equally:

$$\max \sum_{s,k} z_{s,k}.$$

Therefore:

$$1 \text{ Mbps of UK-L}$$

has the same Stage 1 throughput value as:

$$1 \text{ Mbps of UK-T},$$

unless an explicit priority rule is added.

Latency-sensitive traffic receives special treatment only after throughput has been established.

7.13 Synthetic Numerical Data Is Required

No final numerical dataset was supplied.

Therefore, the analysis constructs a documented synthetic dataset containing heterogeneous:

capacities;

latencies;

source-stream demands.

The purpose of the synthetic data is not to represent a specific production network.

It is to create a transparent test environment that exposes:

shared-capacity competition;

alternate-route tradeoffs;

latency tradeoffs;

infrastructure bottlenecks;

stress behavior.

7.14 Constraint Summary

The confirmed problem can therefore be summarized as:

$$0 \le z_{s,k} \le D_{s,k} \qquad \sum_{s,k,p \in p} f_{s,k,p} \le C_e \text{ correct destination only} \qquad \text{partial del}$$

These constraints define the feasible operating region analyzed throughout the notebook.

8. Modeling Assumptions, Confirmed Conditions, and Analyst Design Choices

8.1 Purpose

Not every element of the optimization model has the same status.

Some conditions come directly from the problem statement or were confirmed through clarification with Prateek.

Other assumptions simplify real network behavior so that the core routing problem can be analyzed clearly.

A third category contains analytical design choices introduced in this notebook to convert the stated problem into a complete, interpretable optimization framework.

This section therefore separates the model into three categories:

1. **confirmed problem assumptions and operating conditions**;
2. **simplifying modeling assumptions**;
3. **analyst design choices and extensions**.

This distinction is important because a confirmed problem requirement should not be presented as if it were an arbitrary modeling choice.

It also improves:

- traceability;
- reproducibility;
- model governance;
- interpretation of the final results.

The limitations associated with these assumptions are revisited in detail in Section 46.

8.2 Confirmed Problem Assumptions and Operating Conditions

The following conditions are treated as fixed throughout the baseline analysis.

They originate from the problem statement and/or were confirmed during the clarification process.

8.2.1 Static Snapshot Optimization

Each optimization represents one independent instant in time.

During a single solve:

- demand is fixed;
- topology is fixed;
- link capacity is fixed;
- link latency is fixed.

There is no time evolution inside one optimization period.

Traffic that cannot be routed during the current snapshot is not carried into a

future period.

Therefore, the model answers:

> **What is the best feasible routing allocation for the current network state?**

rather than:

> **How should traffic evolve dynamically over time?**

8.2.2 Exactly Four Traffic Streams

The problem contains four traffic streams:

- UK-L;
- UK-T;
- US-L;
- US-T.

The suffix:

- `L` identifies latency-sensitive traffic;
- `T` identifies latency-tolerant traffic.

The destination prefix identifies the required exit:

- UK;
- US.

Therefore:

$$K = \{UK\text{-}L,\ UK\text{-}T,\ US\text{-}L,\ US\text{-}T\}.$$

8.2.3 Source-Level Demand

Demand is modeled separately for every source-stream combination.

For source:

$$s \in S$$

and stream:

$$k \in K,$$

offered demand is:

$$D_{s,k}.$$

Therefore, S1 / UK-L and B / UK-L are different commodities even though both contribute to the aggregate UK-L traffic class.

Aggregate demand for stream k is:

$$D_k = \sum_{s \in S} D_{s,k}.$$

This source-level representation is necessary because feasible routing depends on where traffic enters the network.

8.2.4 Partial Delivery Is Allowed

A source-stream demand does not need to be delivered in full.

For every commodity:

$$0 \leq z_{s,k} \leq D_{s,k},$$

where:

$$z_{s,k}$$

represents delivered demand.

Undelivered or lost demand is:

$$u_{s,k} = D_{s,k} - z_{s,k}.$$

Therefore, a commodity may be:

- fully served;
- partially served;
- completely unserved.

This applies to both the splittable and single-path formulations.

8.2.5 Single-Path Routing Applies at the Source-Stream Level

For the single-path formulation, each source-stream commodity may use at most one route.

The one-path restriction therefore applies separately to:

$$(s, k),$$

not to an aggregate traffic class across several sources.

For example:

- S1 / US-L may use one route;
- B / US-L may use a different route.

Partial delivery remains allowed.

Therefore, single-path routing does **not** imply all-or-nothing admission.

8.2.6 Additive Path Latency

The latency of a route is the sum of the latencies of the directed links contained in that path.

For path:

$$p,$$

path latency is:

$$L_p = \sum_{e \in p} L_e,$$

where:

$$L_e$$

is the latency of directed arc e.

This assumption allows route latency to be calculated directly from the physical network.

8.2.7 No Queueing, Waiting, or Buffering

Traffic does not wait at nodes.

The model contains no:

- queue;
- buffer;
- holding time;
- backlog;
- carryover variable.

If traffic cannot be delivered during the current snapshot, it is counted as lost.

Therefore, congestion is represented through:

- capacity limitations;
- undelivered demand;

rather than queue accumulation.

8.2.8 Full-Duplex Directed Links

Each physical network link is full duplex.

A physical connection between nodes u and v is represented using two independent directed arcs:

$$(u, v)$$

and:

$$(v, u).$$

Capacity in one direction does not consume capacity in the reverse direction.

Therefore:

$$C_{u \to v}$$

and:

$$C_{v \to u}$$

are modeled as separate resources.

8.2.9 Lossless Transmission

Links are treated as lossless.

Traffic routed through a selected path is assumed to arrive successfully at the intended destination.

The baseline model does not include:

- packet loss;
- retransmission;
- corruption;
- probabilistic reliability.

Thus, optimized flow corresponds directly to successfully delivered throughput.

8.2.10 Link Capacity Is the Physical Limiting Resource

Nodes themselves do not have independent processing-capacity constraints.

The model does not currently constrain:

- CPU;
- switching throughput;
- memory;
- packet-processing limits.

Capacity restrictions occur on directed links.

For every directed arc:

$$e \in E,$$

the optimization must satisfy:

$$\sum_{s,k,p:e\in p} f_{s,k,p} \leq C_e.$$

8.2.11 Valid Destination Exits

Traffic may leave the network only through its required destination.

UK-bound traffic must terminate at:

$$UK.$$

US-bound traffic must terminate at:

$$US.$$

A valid candidate path may not:

- terminate at the wrong exit;
- pass through the wrong exit and continue elsewhere.

This ensures destination correctness.

8.2.12 Equal Stage 1 Value of Delivered Traffic

Every successfully delivered Mbps contributes equally to the Stage 1 throughput objective.

The primary objective is:

$$\max \sum_{s,k} z_{s,k}.$$

Therefore:

$$1 \text{ Mbps of UK-L}$$

has the same Stage 1 objective contribution as:

$$1 \text{ Mbps of UK-T},$$

$$1 \text{ Mbps of US-L},$$

or:

$$1 \text{ Mbps of US-T}.$$

Latency-sensitive traffic does not receive automatic admission priority.

Its special treatment occurs later through the secondary latency objective.

This assumption becomes particularly important in the fairness analysis because Stage 1 may choose different commodity admission patterns that produce the same total throughput.

8.2.13 No Hard Latency Threshold

The base problem does not impose a strict latency requirement such as:

$$L_p \leq L_{max}.$$

A higher-latency path remains feasible if required to satisfy capacity constraints.

Latency-sensitive traffic is instead improved through the Stage 2 objective.

8.3 Simplifying Modeling Assumptions

The following assumptions are not fundamental requirements of the routing problem.

They simplify the network representation so that the core optimization can be studied transparently.

8.3.1 Fixed Link Latency

Each directed link has a fixed latency value.

Latency does not increase as utilization approaches capacity.

Therefore, the baseline model does not include congestion-induced queueing delay.

For example, a link operating at:

$$50\%$$

utilization has the same modeled propagation latency as the same link operating at:

$$100\%.$$

This simplifies the optimization but may understate real-world latency under heavy congestion.

8.3.2 Deterministic Inputs

The model treats:

- demand;

- capacity;
- latency;
- topology;

as known deterministic inputs.

No probability distribution is assigned to:

- demand variation;
- capacity availability;
- link latency;
- failures.

Scenario testing later introduces deterministic alternative states, but the baseline optimization itself is not stochastic.

8.3.3 No Dynamic Network State Within a Solve

Although Section 43 evaluates multiple scenarios, each scenario is solved independently.

The optimization does not dynamically transition from:

- baseline;
- degradation;
- recovery;
- growth;

within one model.

This keeps scenario interpretation clean but does not model temporal transitions.

8.3.4 No Route-Churn Cost

The optimization may select different routes in different solves without penalty.

There is no explicit cost for:

- changing paths;
- updating forwarding rules;
- migrating traffic;
- operational disruption.

In a production network, frequent route changes may be undesirable even if they produce a mathematically better snapshot solution.

8.3.5 No Explicit Capacity-Upgrade Cost

The marginal-capacity analysis measures the technical value of additional bandwidth.

It does not currently include:

- purchase cost;
- construction cost;
- implementation lead time;
- outage risk;
- vendor pricing;
- engineering difficulty.

Therefore, positive marginal throughput value identifies a technically useful upgrade candidate, not necessarily the financially optimal investment.

8.3.6 No Failure Probabilities

Stress testing reduces selected capacities, but the model does not assign probabilities to those states.

The current framework does not include:

- mean time between failures;
- outage likelihood;
- recovery time;
- correlated failures.

Stress scenarios are diagnostic rather than probabilistic forecasts.

8.4 Analyst Modeling and Implementation Choices

The following decisions were introduced during the analysis to create a complete and interpretable optimization framework.

8.4.1 Path-Based Multi-Commodity Formulation

The optimization uses candidate-path variables rather than a purely edge-based multi-commodity flow formulation.

For each commodity:

$$(s, k),$$

and candidate path:

$$p \in P_{s,k},$$

the flow variable is:

$$f_{s,k,p}.$$

This path-based formulation makes it easier to:

- enforce single-path routing;
- calculate path latency;
- report selected routes;
- compare route complexity;
- explain solutions operationally.

8.4.2 Pre-Enumerated Candidate Paths

Candidate routes are constructed before the optimization solve.

The solver selects only among those routes.

A valid candidate path must:

- begin at the correct source;
- terminate at the correct destination;
- avoid the opposite destination;
- avoid repeated nodes;
- correspond to valid directed arcs.

Therefore, optimality is technically relative to the candidate-path set:

$$\text{Optimization Quality} \leq \text{Candidate Path Quality}.$$

For the small synthetic network, complete simple-path enumeration is practical.

For larger production networks, more scalable route-generation methods may be necessary.

8.4.3 Lexicographic Optimization

The model separates throughput and latency rather than combining them into one weighted objective.

The baseline hierarchy is:

$$\boxed{\text{Maximum Throughput} \rightarrow \text{Sensitive Latency}}$$

Stage 1

Determine the maximum feasible throughput:

$$T^* = \max \sum_{s,k} z_{s,k}.$$

Stage 2

Preserve Stage 1 service and minimize latency exposure for:

- UK-L;
- US-L.

This avoids creating an arbitrary numerical tradeoff between Mbps and milliseconds.

8.4.4 Exact Admission Preservation in Stage 2

Stage 2 preserves the exact Stage 1 admission quantity for every source-stream commodity:

$$z_{s,k}^{Stage2} = z_{s,k}^{Stage1}.$$

This is stronger than preserving only:

$$\sum_{s,k} z_{s,k}.$$

The reason is analytical control.

If only total throughput were preserved, Stage 2 could change which commodities receive service while appearing to maintain the same top-line throughput.

Exact admission preservation ensures that observed Stage 2 latency changes result from rerouting rather than changed admission.

8.4.5 Traffic-Weighted Average Latency for Split Traffic

For a commodity using multiple positive-flow paths, the primary latency metric is traffic-weighted average latency.

For commodity (s, k):

$$\bar{L}_{s,k} = \frac{\sum_{p \in P_{s,k}} f_{s,k,p} L_p}{z_{s,k}}.$$

This answers:

> **What latency does the average delivered Mbps experience?**

8.4.6 Worst Used-Path Latency

Traffic-weighted average latency can hide a small amount of traffic traveling over a much slower path.

Therefore, the notebook also reports:

$$L_{s,k}^{worst} = \max_{p: f_{s,k,p} > 0} L_p.$$

This provides a tail-performance diagnostic.

Together:

- weighted-average latency measures the typical delivered experience;
- worst used-path latency identifies the slowest route actually used.

8.4.7 Sensitive Latency Exposure

Latency optimization uses flow-weighted latency exposure.

For sensitive traffic:

$$LE = \sum_{s,k \in K_L,p} f_{s,k,p} L_p.$$

This quantity has units of:

$$\text{Mbps} \cdot \text{ms}.$$

It captures both:

- how much sensitive traffic is carried;
- how much latency that traffic experiences.

This metric enables Stage 2 to improve the latency profile of admitted traffic while preserving throughput.

8.4.8 One-Mbps Marginal-Capacity Perturbation

To distinguish true local bottlenecks from merely busy links, every directed arc is tested independently.

For directed arc:

$$e,$$

capacity is changed from:

$$C_e$$

to:

$$C_e + 1.$$

The Stage 1 model is then re-solved.

Marginal throughput value is defined as:

$$MV_e = T^*(C_e + 1) - T^*(C_e).$$

If:

$$MV_e > 0,$$

then one additional Mbps on that arc produces additional end-to-end delivered traffic.

This provides a stronger bottleneck definition than utilization alone.

8.4.9 Saturation Is Not Treated as Equivalent to Bottleneck Value

A link may operate at:

$$100\%$$

utilization yet still satisfy:

$$MV_e = 0.$$

Therefore, the notebook deliberately distinguishes:

Utilization

> How much of the current link capacity is being used?

from:

Marginal Capacity Value

> Would more capacity on this link increase end-to-end throughput?

This distinction becomes central to the capacity-investment analysis.

8.4.10 Stress Scenario Design

The notebook evaluates several analyst-designed operating states.

These include:

Low Demand

$$60\%$$

of baseline demand.

High Demand

$$150\%$$

of baseline demand.

Internal-Corridor Degradation

Selected internal capacities are reduced.

Destination-Exit Degradation

Destination-facing capacities are reduced.

Fragmented Internal Capacity

Several internal links are reduced by different amounts.

These scenarios are diagnostic experiments.

They are not externally supplied demand forecasts or failure probabilities.

Their purpose is to test whether baseline conclusions remain stable when operating conditions change.

8.4.11 Fairness Extension

The original Stage 1 objective does not guarantee balanced service.

Therefore, the notebook introduces commodity delivery ratios:

$$r_{s,k} = \frac{z_{s,k}}{D_{s,k}}.$$

The minimum delivery ratio is:

$$\alpha = \min_{s,k} r_{s,k}.$$

The baseline results show that:

$$\alpha = 0,$$

meaning some commodities receive no service even though aggregate throughput is optimal.

This motivates possible production extensions such as:

- max-min fairness;
- minimum SLA delivery ratios;

- source-level guarantees;
- traffic-class priority.

These are analytical extensions rather than requirements of the original Stage 1 model.

8.5 Relationship Between Assumptions and the Observed Results

The assumptions above materially influence how the results should be interpreted.

Several are particularly important.

8.5.1 Partial Delivery Makes the Single-Path Model More Flexible

Because partial admission is allowed, the single-path model may satisfy:

$$0 < z_{s,k} < D_{s,k}.$$

A commodity does not need to fit its full offered demand on one route.

This is one reason the single-path formulation is able to match splittable Stage 1 throughput throughout the tested scenarios.

The result:

$$T^*_{single} = T^*_{split}$$

should therefore be understood in the context of partial delivery.

8.5.2 Equal Stage 1 Mbps Value Creates Admission Degeneracy

Because every delivered Mbps contributes equally to Stage 1:

$$\max \sum_{s,k} z_{s,k},$$

several different commodity-admission patterns can produce the same total:

$$83 \text{ Mbps}.$$

This explains why:

- split Stage 1;
- single-path Stage 1;

can achieve identical total throughput while serving different commodities.

Therefore:

$$\boxed{\text{Equal Objective Value} \neq \text{Equal Service Outcome}}$$

8.5.3 Fixed Latency Separates Congestion From Delay

The model can show that a link is saturated without increasing that link's latency.

Therefore, the latency conclusions reflect:

- route structure;
- fixed physical delay;

rather than congestion-induced delay.

This should be considered when translating model latency into production expectations.

8.5.4 Pre-Enumerated Paths Define the Routing Opportunity Set

The solver can select only among routes included in:

$$P_{s,k}.$$

Therefore, a mathematically optimal solution is optimal relative to the supplied candidate-path set.

This is appropriate for the current nine-node synthetic network but becomes a scalability consideration for larger systems.

8.5.5 One-Mbps Marginal Value Is Local

The marginal experiment asks:

> **Where does the next Mbps of capacity have value?**

It does not imply that the same benefit continues linearly for large upgrades.

After one bottleneck is relaxed, another may become binding.

Therefore:

$$MV_e$$

should be interpreted as a local capacity signal

9. Confirmed Clarifications from Prateek

9.1 Purpose

Several elements of the original network-routing problem required clarification because different interpretations would produce materially different mathematical formulations.

Those questions were sent to Prateek before the final optimization models were constructed.

The clarification process is now complete.

This section records the **authoritative interpretation used throughout the remainder of the notebook**.

The corresponding historical questions and reasoning are retained separately in Section 10 as a decision audit trail.

No additional clarification is required to construct, solve, validate, or interpret the baseline optimization models.

9.2 Source-Level Demand

Original Ambiguity

Several traffic streams originate from multiple source nodes.

It was therefore necessary to determine whether demand would be supplied only as an aggregate stream total or separately for each physical source.

Confirmed Answer

Demand is known separately by:

- source;
- stream.

Final Modeling Interpretation

For source:

$$s \in S$$

and traffic stream:

$$k \in K,$$

offered demand is represented as:

$$D_{s,k}.$$

Each source-stream pair is therefore modeled as a distinct commodity.

For example:

- S1 / UK-L;
- S2 / UK-L;
- B / UK-L;

are separate commodities even though all three contribute to the aggregate UK-L traffic class.

Aggregate demand for stream k is:

$$D_k = \sum_{s \in S} D_{s,k}.$$

Why This Matters Mathematically

The physical location where traffic enters the network determines:

- feasible paths;

- upstream capacity consumption;
- congestion exposure;
- destination reachability.

Pooling source demand would therefore remove information required for correct multi-commodity routing.

9.3 Single-Path Interpretation

Original Ambiguity

The phrase "single-path routing" could be interpreted in several ways.

For a traffic stream originating from several physical sources, one literal route for the entire aggregate stream would not be meaningful.

Confirmed Answer

The one-path rule applies independently to each **source-stream commodity**.

Final Modeling Interpretation

For candidate path:

$$p \in P_{s,k},$$

binary variable:

$$y_{s,k,p} \in \{0,1\}$$

indicates whether path p is selected.

The model enforces:

$$\sum_{p \in P_{s,k}} y_{s,k,p} \leq 1.$$

Therefore:

- S1 / US-L may use one route;
- B / US-L may use another route;

while each source-stream commodity itself remains unsplit.

Why This Matters Mathematically

The confirmed interpretation creates a single-route restriction at the commodity level rather than at the aggregate stream level.

This makes the model physically meaningful while preserving the intended contrast between:

- splittable routing;
- one-route-per-commodity routing.

9.4 Partial Delivery in the Single-Path Formulation

Original Ambiguity

Single-path routing and all-or-nothing admission are different restrictions.

It was therefore necessary to determine whether a commodity had to fit its entire demand on the selected route.

Confirmed Answer

Partial delivery is allowed.

Final Modeling Interpretation

For every commodity:

$$0 \leq z_{s,k} \leq D_{s,k}.$$

A commodity may therefore satisfy:

$$0 < z_{s,k} < D_{s,k}.$$

Any remaining demand is counted as lost:

$$u_{s,k} = D_{s,k} - z_{s,k}.$$

This applies to both the splittable and single-path formulations.

Important Interpretation

The single-path model is **not** an all-or-nothing unsplittable-flow model.

Instead:

> Whatever portion of the commodity is delivered must use at most one path.

This distinction materially affects the feasible region and later helps explain why the single-path formulation is able to match splittable Stage 1 throughput across all tested scenarios.

9.5 Split-Traffic Latency Measurement

Original Ambiguity

If latency-sensitive traffic is split across several routes with different path latencies, there is no single automatically defined stream latency.

Possible measures include:

- minimum path latency;
- maximum path latency;
- simple average;
- traffic-weighted average;
- percentile or SLA-based measures.

Confirmed Answer

The latency metric may be selected as part of the candidate's modeling approach.

Final Modeling Interpretation

The notebook uses **traffic-weighted average latency** as the primary split-traffic metric.

For commodity (s, k):

$$\bar{L}_{s,k} = \frac{\sum_{p \in P_{s,k}} f_{s,k,p} L_p}{z_{s,k}}.$$

Worst used-path latency is also reported:

$$L^{worst}_{s,k} = \max_{p : f_{s,k,p} > 0} L_p.$$

Why These Two Metrics Are Used

Traffic-weighted average latency answers:

> **What latency does the average delivered Mbps experience?**

Worst used-path latency answers:

> **What is the slowest path actually carrying traffic?**

Reporting both prevents a small amount of traffic on a very slow path from being hidden by a favorable average.

9.6 Numerical Dataset

Original Ambiguity

The problem description included illustrative bandwidth and latency values, but it was unclear whether:

- those values applied to every link;
- a full dataset would be provided;
- representative values should be created.

Confirmed Answer

A synthetic numerical dataset should be constructed.

Final Modeling Interpretation

The notebook creates heterogeneous values for:

- link capacities;
- link latencies;
- source-stream demands.

The dataset is deliberately designed to expose meaningful:

- congestion tradeoffs;
- alternate-route choices;
- latency tradeoffs;
- capacity bottlenecks.

The data is clearly labeled synthetic throughout the analysis.

Why This Matters

Using heterogeneous values produces a more informative optimization problem than assigning identical bandwidth and latency to every link.

It allows the model to demonstrate:

- shortest-route congestion;
- alternate-corridor use;
- capacity scarcity;
- marginal upgrade value.

9.7 Queueing and Waiting

Original Ambiguity

It was necessary to determine whether traffic could wait at intermediate nodes or remain queued for future transmission.

Confirmed Answer

No queueing is modeled.

Final Modeling Interpretation

Each solve represents an independent snapshot.

There is:

- no queue;
- no waiting time;
- no node buffer;
- no backlog.

Link latency remains fixed and additive:

$$L_p = \sum_{e \in p} L_e.$$

Modeling Consequence

Congestion is represented through:

- capacity constraints;
- undelivered demand;

rather than increasing queue length or waiting time.

9.8 Lost Demand and Carryover

Confirmed Answer

Traffic that cannot be routed during the current snapshot is lost immediately.

Final Modeling Interpretation

There is no:

- carryover variable;
- inventory variable;
- backlog variable;
- future-period recovery.

For each commodity:

$$u_{s,k} = D_{s,k} - z_{s,k}.$$

This quantity is interpreted as lost demand.

Why This Matters

The optimization evaluates service within the current network state only.

It does not defer unmet traffic to a later period.

9.9 Node Capacity

Confirmed Answer

Nodes are unconstrained forwarding points.

Final Modeling Interpretation

There is no separate processing-capacity constraint on:

- S1;
- S2;
- S3;
- A;
- B;
- C;
- D;
- UK;
- US.

Only link capacities constrain network throughput.

For directed arc e:

$$\sum_{s,k,p:e\in p} f_{s,k,p} \leq C_e.$$

Why This Matters

The model attributes physical congestion to directed links rather than to intermediate node-processing limitations.

9.10 Service Priority

Original Ambiguity

Latency-sensitive traffic could potentially receive higher admission priority than latency-tolerant traffic.

That would materially change the Stage 1 objective.

Confirmed Answer

No automatic admission priority exists in the base model.

Final Modeling Interpretation

Every delivered Mbps contributes equally to the Stage 1 throughput objective:

$$\max \sum_{s,k} z_{s,k}.$$

Therefore:

$$1 \text{ Mbps of UK-L}$$

has the same Stage 1 value as:

$$1 \text{ Mbps of UK-T},$$

$$1 \text{ Mbps of US-L},$$

or:

$$1 \text{ Mbps of US-T}.$$

Latency-sensitive traffic receives special treatment only through the secondary latency objective.

Why This Matters

This confirmed rule is central to interpreting the final results.

It explains why throughput-optimal solutions may:

- admit different traffic classes;
- completely drop some commodities;
- produce different service allocations while achieving the same total throughput.

Fairness and service-priority policies are therefore treated later as extensions rather than assumptions of the baseline model.

9.11 Final Clarification Summary

Issue	Confirmed Resolution	Mathematical / Modeling Consequence
Source demand	Known by source and stream	Use $D_{s,k}$ and model each source-stream pair separately
Single-path scope	One path per source-stream commodity	Use binary $y_{s,k,p}$ and enforce $\sum_p y_{s,k,p} \le 1$
Partial delivery	Allowed	$0 \le z_{s,k} \le D_{s,k}$
Split latency	Analyst modeling choice	Weighted-average latency + worst used-path latency
Numerical data	Synthetic dataset required	Build heterogeneous capacities, latencies, and demands

Queueing	None	Static independent snapshots with fixed link latency
Lost demand	Lost immediately	$u_{s,k} = D_{s,k} - z_{s,k}$; no carryover
Node capacity	Unlimited	Directed-link capacity constraints only
Service priority	None in base Stage 1	Equal throughput value for every delivered Mbps

These decisions are treated as authoritative throughout the remainder of the notebook.

9.12 Final Status of the Clarification Process

All material ambiguities affecting the baseline mathematical formulation have been resolved.

There is no remaining clarification required regarding:

- commodity definition;
- single-path scope;
- partial admission;
- split latency reporting;
- synthetic data construction;
- buffering;
- node capacity;
- traffic priority.

Therefore:

> **The optimization models can be constructed, solved, validated, and interpreted using a fully specified problem definition.**

Any later limitations arise from deliberate modeling simplifications or production-scope considerations rather than unresolved requirements.

10. Clarification Correspondence and Decision Audit Trail

10.1 Purpose

This section preserves the original clarification questions sent to Prateek before the optimization formulation was finalized.

The purpose is historical and analytical.

These questions demonstrate:

- which ambiguities were identified;
- why they mattered;
- how alternative interpretations could have changed the model.

The questions below should **not** be interpreted as unresolved issues.

The final authoritative decisions are documented in Section 9.

10.2 Question 1 — Source-Level Demand

Original Question

When one stream originates from multiple source nodes, will the amount generated at each individual source be provided?

For example, if UK-L has 30 Mbps of aggregate demand originating from S1 and B, will the input specify how much originates from each source?

Why This Mattered

Flow conservation and feasible routing depend on the physical location where traffic enters the network.

If only aggregate stream demand were provided, the optimizer would not know which links must carry each source contribution.

Final Resolution

Demand is known separately by source and stream.

The final model therefore uses:

$$D_{s,k}.$$

Modeling Consequence

Every source-stream pair is treated as a distinct commodity.

10.3 Question 2 — Meaning of Single-Path Routing

Original Question

For the unsplittable variant, does the one-path rule apply separately to the portion of a stream originating from each source?

For example, may:

- S1 / US-L use one path;
- B / US-L use another path;

provided each source contribution remains individually unsplit?

Why This Mattered

An aggregate stream with several physical origins cannot literally use one source-to-destination path.

The scope of the single-path rule therefore changes the mathematical structure of the MILP.

Final Resolution

The single-path restriction applies separately to each source-stream commodity.

Modeling Consequence

The model uses:

$$y_{s,k,p}$$

and enforces:

$$\sum_{p \in P_{s,k}} y_{s,k,p} \leq 1.$$

10.4 Question 3 — Partial Delivery in the Single-Path Case

Original Question

If the selected route cannot accommodate the full source-stream demand, may the available portion be delivered and the remainder counted as lost?

Or must the entire commodity demand fit before that path may be selected?

Why This Mattered

These represent two different mathematical restrictions.

Partial Admission

$$0 \leq z_{s,k} \leq D_{s,k}.$$

All-or-Nothing Admission

A commodity would either receive:

$$0$$

or:

$$D_{s,k}.$$

The resulting feasible regions can be significantly different.

Final Resolution

Partial delivery is allowed.

Modeling Consequence

A single-path commodity may receive less than its total offered demand, but every positive delivered Mbps must travel on the selected route.

10.5 Question 4 — Latency Measurement for Split Traffic

Original Question

If UK-L or US-L is split across several routes with different latencies, how should the traffic's latency be measured?

Possible measures included:

- traffic-weighted average latency;
- maximum / worst-case latency;
- another SLA-oriented metric.

Why This Mattered

A split commodity no longer has one unique route latency.

The reporting metric therefore affects:

- model interpretation;
- Stage 2 evaluation;
- comparison with single-path routing.

Final Resolution

The latency metric is an analyst modeling choice.

Selected Interpretation

The notebook uses traffic-weighted average latency:

$$\bar{L}_{s,k} = \frac{\sum_p f_{s,k,p} L_p}{z_{s,k}}$$

and also reports:

$$L_{s,k}^{worst}.$$

Reason for the Choice

The weighted average reflects the experience of the delivered traffic volume.

Worst used-path latency ensures that unfavorable tail routing remains visible.

10.6 Question 5 — Numerical Dataset and Link Values

Original Question

The example discussion referenced values such as:

- 30 Mbps bandwidth;
- 20 ms latency.

Clarification was requested regarding whether:

- those values should apply uniformly;
- each link should receive separate representative values;
- a numerical dataset would be provided;
- synthetic values should be created.

Why This Mattered

Uniform links would produce a much less informative optimization environment.

Heterogeneous values are needed to create meaningful tradeoffs involving:

- route latency;
- link capacity;
- alternate corridors;
- bottlenecks.

Final Resolution

A synthetic dataset should be constructed.

Modeling Consequence

The notebook creates heterogeneous:

- capacities;
- latencies;
- source-stream demands.

The data is explicitly labeled synthetic.

10.7 Question 6 — Queueing, Node Capacity, Lost Demand, Holding Time, and Priority

Original Question

Clarification was requested to confirm whether:

- nodes buffer or queue traffic;

- nodes have independent throughput limits;
- link bandwidth is the primary capacity constraint;
- unroutable demand is lost;
- traffic may be held for later transmission;
- latency-sensitive traffic automatically receives admission priority.

Why This Mattered

Each alternative would require additional variables or constraints.

For example:

Queueing

Would require modeling:

- backlog;
- waiting time;
- potentially time-dependent flow.

Node Capacity

Would require additional node-capacity constraints.

Priority Admission

Would require either:

- traffic weights;
- hierarchical objectives;
- minimum service constraints.

Final Resolution

The confirmed interpretation is:

- no queueing;
- no buffering;
- no carryover;
- nodes are unconstrained;
- link capacity is the physical resource constraint;
- lost traffic is discarded in the current snapshot;
- no automatic Stage 1 sensitive-traffic priority exists.

Modeling Consequence

The model remains a static, link-capacity-constrained multi-commodity routing problem.

10.8 Clarification Questions and Final Answers

Question / Issue	Final Resolution	Modeling Consequence
Source-level demand	Demand known by source and stream	Model $D_{s,k}$
Single-path interpretation	One path per source-stream commodity	Binary path decision $y_{s,k,p}$
Partial delivery	Allowed	Delivered traffic may be less than offered demand
Split latency	Analyst's choice	Weighted average + worst used path
Numerical data	Synthetic dataset required	Heterogeneous test environment
Queueing	None	Static snapshot
Buffering / carryover	None	Lost traffic not stored
Node capacity	Unlimited	Link-capacity constraints only
Service priority	None in base Stage 1	Equal Stage 1 value per delivered Mbps

---

10.9 How the Clarifications Changed the Final Formulation

The clarification process did more than resolve wording.

It directly determined the mathematical structure of the model.

Commodity Definition

The model became a source-stream multi-commodity flow problem:

$$(s, k).$$

Single-Path Formulation

The binary decision applies at the commodity-path level:

$$y_{s,k,p}.$$

Admission

Demand is continuously partially admissible:

$$0 \leq z_{s,k} \leq D_{s,k}.$$

Network State

The problem is solved independently at each static snapshot.

Capacity

Directed-link bandwidth is the only modeled physical throughput constraint.

Latency

Split latency requires an explicitly selected reporting metric.

Priority

Sensitive traffic is not protected at admission unless an additional policy is introduced.

These decisions define the feasible region analyzed in all later sections.

10.10 Why the Clarification Process Matters to the Final Results

The final results depend on these confirmed interpretations.

Partial Delivery

Because partial delivery is allowed, the single-path formulation remains relatively flexible.

This is important when interpreting the later finding:

$$T^*_{single} = T^*_{split}$$

across the tested Stage 1 scenarios.

Equal Admission Value

Because every delivered Mbps has equal Stage 1 value, different commodity combinations can produce the same optimal throughput.

This contributes to throughput degeneracy and motivates the later fairness analysis.

No Queueing

Because link latency is fixed rather than utilization-dependent, the Stage 2 latency results reflect route structure rather than queueing congestion.

Synthetic Data

Because the numerical values are synthetic, results such as:

$$83 \text{ Mbps}$$

describe the constructed test environment rather than an external production network.

Therefore, the clarification process is directly connected to how the final optimization outputs must be interpreted.

10.11 Auditability and Model Governance

Retaining both:

- the original questions;
- the final answers;

creates a transparent record of model development.

Reproducibility

A future analyst can understand why each major modeling decision was made.

Auditability

Stakeholders can distinguish confirmed requirements from analyst choices.

Change Management

If a future requirement changes—for example, if partial delivery is no longer allowed—the affected formulation can be identified immediately.

Interpretation

Readers can understand which conclusions depend on specific assumptions or confirmed rules.

The clarification process therefore forms part of the model's governance record rather than merely documenting email correspondence.

10.12 Final Status

The clarification correspondence is now closed for purposes of the baseline assessment.

All questions materially affecting the mathematical formulation have been resolved.

The remainder of the notebook therefore proceeds using the final interpretation documented in Section 9.

Any later analytical extensions—such as:

- fairness;
- SLA policy;
- stochastic optimization;
- queue-aware latency;
- investment cost;

should be understood as deliberate extensions of the confirmed baseline model rather than unresolved original requirements.

11. Final Input Requirements

The clarification process resolved the modeling requirements.

The remaining numerical values must therefore be created as part of the synthetic test environment.

Item	Status
Network topology	Confirmed
Traffic-generating nodes	Confirmed
UK/US exits	Confirmed
Four traffic classes	Confirmed
Full-duplex links	Confirmed
Source-level demand	Confirmed
One-path rule	Confirmed
Partial delivery	Confirmed
Queueing behavior	Confirmed
Node capacity	Confirmed
Admission priority	Confirmed
Numerical link capacities	To be synthetically generated
Numerical link latencies	To be synthetically generated
Source-stream demand values	To be synthetically generated
Split latency metric	Modeling choice finalized

12. Core Mathematical Formulation

12.1 Sets

V = set of network nodes

E = set of directed network arcs

$$K = \{UK\text{-}L,\ UK\text{-}T,\ US\text{-}L,\ US\text{-}T\}$$

S_k = set of source nodes generating stream k

$P_{k,s}$ = feasible paths from source s to the required exit of stream k

$$K_L = \{UK\text{-}L,\ US\text{-}L\}$$

$$K_T = \{UK\text{-}T,\ US\text{-}T\}$$

12.2 Parameters

C_e = directional capacity of arc e, in Mbps

L_e = latency of arc e, in milliseconds

$D_{k,s}$ = demand of stream k originating at source s

For path p:

$$L_p = \sum_{e \in p} L_e$$

12.3 Splittable-Flow Decision Variables

$$f_{k,s,p} \geq 0$$

represents the amount of stream k from source s routed along path p.

Delivered traffic is:

$$z_{k,s} = \sum_{p \in P_{k,s}} f_{k,s,p}$$

12.4 Single-Path Decision Variables

For the single-path formulation:

$$y_{k,s,p} \in \{0, 1\}$$

where:

$$y_{k,s,p} = 1$$

if source-stream pair (k, s) selects path p.

Continuous path flow remains:

$$f_{k,s,p} \geq 0$$

--

12.5 Demand Constraint

For every source-stream pair:

$$\sum_{p \in P_{k,s}} f_{k,s,p} \leq D_{k,s}$$

--

12.6 Link Capacity Constraint

For each directed arc e:

$$\sum_{k} \sum_{s} \sum_{p : e \in p} f_{k,s,p} \leq C_e$$

--

12.7 Single-Path Constraint

For every source-stream pair:

$$\sum_{p \in P_{k,s}} y_{k,s,p} \leq 1$$

Traffic may flow on a path only if that path is selected:

$$f_{k,s,p} \leq D_{k,s} y_{k,s,p}$$

Because partial delivery is permitted, this does not require:

$$f_{k,s,p} = D_{k,s}$$

--

13. Path Capacity and Path Latency

For a single isolated path:

$$p = (e_1, e_2, \ldots, e_n)$$

the maximum traffic the path could carry, ignoring competition with other streams, is bounded by its smallest link capacity:

$$\boxed{C_p = \min_{e \in p} C_e}$$

By contrast, path latency is additive:

$$\boxed{L_p = \sum_{e \in p} L_e}$$

Example:

If the capacities are:

$$30,\ 30,\ 10,\ 30$$

then:

$$C_p = 10\ \text{Mbps}$$

If the corresponding latencies are:

$$20,\ 15,\ 25,\ 10$$

then:

$$L_p = 70\ \text{ms}$$

Important: the bottleneck capacity of an individual path does not by itself determine the global network throughput because multiple paths may share links and therefore compete for the same capacity.

14. Bottleneck and Marginal Capacity Analysis

A fully utilized link is not necessarily a true bottleneck.

A link is operationally important if increasing its capacity increases the optimal amount of traffic delivered to the required exits.

For each link e, define the marginal throughput value of one additional unit of capacity as:

$$\boxed{\Delta_e = T^*(C_e + 1) - T^*(C_e)}$$

where:

- $T^*(C_e)$ is the optimal total throughput under the original capacity;
- $T^*(C_e + 1)$ is the optimal throughput after increasing the capacity of link e by one unit.

If:

$$\Delta_e > 0$$

then increasing that link's capacity improves total end-to-end throughput.

For the splittable linear-programming formulation, the shadow price of the capacity constraint may also be used as a local marginal sensitivity measure.

For the unsplittable mixed-integer formulation, direct perturbation and re-optimization will be used.

15. Model Validation and Stress Testing Plan

15. Validation Plan and Acceptance Criteria

15.1 Purpose

Optimization results should not be interpreted solely because a solver returns a numerical answer.

A valid optimization result must also be:

- mathematically consistent;
- physically feasible;
- internally reconciled;
- compliant with routing-policy rules;
- consistent with theoretical relationships between formulations;
- behaviorally reasonable under controlled test scenarios.

This section therefore defines the validation framework used throughout the notebook.

The validation process serves two purposes:

1. verify that the mathematical models are implemented correctly;
2. verify that the optimization behaves as expected when demand, capacity, congestion, and routing flexibility change.

The actual numerical scenario outcomes are analyzed in detail later in Section 43.

The general validation sequence is:

1. construct the model;
2. solve the model;
3. confirm solver status;
4. validate demand and capacity constraints;
5. validate route-selection logic;
6. reconcile derived summary tables;
7. compare results with theoretical relationships;
8. test behavior under controlled scenarios;
9. interpret the results;
10. derive operational and business implications.

This structure helps prevent coding or formulation errors from being mistaken for optimization insights.

15.2 Solver Status Validation

Every optimization solve must first return an acceptable solver status before any routing result is interpreted.

The expected successful status is:

`Optimal`

Other statuses require investigation.

Examples include:

- Infeasible;
- Unbounded;
- Not Solved;
- Undefined.

A non-optimal status may indicate:

- inconsistent constraints;
- missing paths;
- incorrect demand values;
- incorrect capacities;
- a formulation error;

- solver failure.

No business recommendation should be based on a non-optimal solve unless the exception is explicitly analyzed and documented.

15.3 Demand Accounting Validation

For every source-stream commodity:

$$(s, k),$$

offered demand is:

$$D_{s,k},$$

and delivered demand is:

$$z_{s,k}.$$

The model must satisfy:

$$0 \leq z_{s,k} \leq D_{s,k}.$$

Undelivered demand is:

$$u_{s,k} = D_{s,k} - z_{s,k}.$$

Therefore, the commodity-level accounting identity is:

$$D_{s,k} = z_{s,k} + u_{s,k}.$$

At the network level:

$$\sum_{s,k} D_{s,k} = \sum_{s,k} z_{s,k} + \sum_{s,k} u_{s,k}.$$

For the baseline synthetic dataset:

$$\sum_{s,k} D_{s,k} = 126 \text{ Mbps}.$$

Every summary table must reconcile to this same total demand.

15.4 Directed-Link Capacity Validation

For every directed arc:

$$e \in E,$$

the total optimized traffic using that arc must not exceed its directional capacity:

$$F_e \le C_e.$$

Equivalently:

$$\sum_{s,k,p:e\in p} f_{s,k,p} \le C_e.$$

Because optimization software may return very small floating-point deviations, a numerical tolerance may be used.

However, no material capacity violation is acceptable.

A routing solution is physically invalid if any directed link carries more traffic than its available bandwidth.

15.5 Destination-Correctness Validation

Every positive-flow route must satisfy the destination rules.

A valid path must:

- begin at the correct source;
- terminate at the required destination;
- avoid terminating at the wrong exit;
- avoid passing through the opposite exit and continuing elsewhere.

Therefore:

- UK-bound traffic must terminate at UK;
- US-bound traffic must terminate at US.

This ensures that throughput is counted only when traffic reaches its required destination.

15.6 Candidate-Path Validity Validation

Every candidate route used by the optimization must correspond to valid directed arcs in the network.

For each path:

$$p,$$

every consecutive node pair must exist in the directed network representation.

Candidate paths must also be simple paths unless otherwise explicitly allowed.

Therefore, candidate paths should:

- contain no repeated nodes;
- follow valid network links;
- start at the correct source;
- end at the correct destination.

This prevents invalid or artificial routing opportunities from entering the optimization.

15.7 Splittable-Flow Validation

Under the splittable formulation, a commodity may use one or more valid routes.

For commodity:

$$(s,k),$$

total delivered demand is:

$$z_{s,k} = \sum_{p \in P_{s,k}} f_{s,k,p}.$$

The model must satisfy:

$$\sum_{p \in P_{s,k}} f_{s,k,p} \leq D_{s,k}.$$

Additional validation requirements are:

- every positive-flow route must be valid;
- all directed-link capacities must be respected;
- delivered traffic must not exceed offered demand.

Importantly, a splittable model does not require a commodity to actually split.

A commodity using only one positive-flow path remains fully valid under the splittable formulation.

This distinction becomes important when interpreting the Stage 1 results.

15.8 Single-Path Validation

For the single-path formulation, each source-stream commodity may use at most one positive-flow path.

Therefore:

$$\#\{p : f_{s,k,p} > 0\} \leq 1.$$

Binary route-selection variables satisfy:

$$y_{s,k,p} \in \{0, 1\}.$$

The linking constraint is:

$$f_{s,k,p} \leq D_{s,k} y_{s,k,p}.$$

Therefore:

$$f_{s,k,p} > 0 \Rightarrow y_{s,k,p} = 1.$$

Validation should focus primarily on:

- the number of positive-flow routes;
- positive flow implying an active binary route variable.

A binary variable may occasionally equal 1 while its associated flow is zero if the binary variable is not otherwise penalized.

That condition does not violate the routing rule.

The critical requirement is that a commodity never uses more than one positive-flow route.

15.9 Partial-Delivery Validation

Partial delivery is explicitly allowed under both routing formulations.

Therefore:

$$0 < z_{s,k} < D_{s,k}$$

is a valid outcome.

Validation should confirm that partially served commodities are correctly reported as:

- delivered demand;
- undelivered demand;
- delivery percentage.

A partially served single-path commodity must still use only one positive-flow route.

This distinction separates:

> single-path routing

from:

> all-or-nothing unsplittable admission.

15.10 Stage 1 Throughput Validation

The Stage 1 objective is:

$$\max \sum_{s,k} z_{s,k}.$$

The total Stage 1 throughput must reconcile across all derived outputs.

It should equal the total obtained from:

- commodity-level delivery;
- source-level delivery;
- stream-level delivery;
- positive route flows.

For the baseline synthetic network, the executed optimum is:

$$83 \text{ Mbps}.$$

Therefore, baseline lost demand is:

$$126 - 83 = 43 \text{ Mbps}.$$

The baseline delivery rate is:

$$\frac{83}{126} \times 100 \approx 65.87\%.$$

These values are empirical optimization results.

They are not assumed targets.

15.11 Stage 2 Admission-Preservation Validation

Stage 2 is intended to improve latency without changing the Stage 1 service allocation.

Therefore, for every commodity:

$$z_{s,k}^{Stage2} = z_{s,k}^{Stage1}.$$

At the network level:

$$\sum_{s,k} z_{s,k}^{Stage2} = \sum_{s,k} z_{s,k}^{Stage1}.$$

This requirement is stronger than preserving only total throughput.

If only total throughput were fixed, Stage 2 could change which commodities receive service while maintaining the same aggregate Mbps.

Exact commodity-level admission preservation ensures that Stage 2 differences are produced through routing changes rather than changed traffic selection.

15.12 Sensitive-Latency Validation

For latency-sensitive streams:

- UK-L;
- US-L;

the notebook reports:

- delivered traffic;
- latency exposure;
- traffic-weighted average latency;
- worst used-path latency.

For sensitive stream k, latency exposure is:

$$LE_k = \sum_{s,p} f_{s,k,p} L_p.$$

Traffic-weighted average latency is:

$$\bar{L}_k = \frac{LE_k}{\sum_s z_{s,k}}.$$

Worst used-path latency is:

$$L_k^{worst} = \max_{s,p: f_{s,k,p} > 0} L_p.$$

Because Stage 2 minimizes sensitive latency while preserving admission, the following should hold:

$$LE_k^{Stage 2} \leq LE_k^{Stage 1}$$

for the corresponding preserved admission vector.

Any worsening of the optimized Stage 2 objective would require investigation.

15.13 Split vs. Single Feasible-Region Validation

Every valid single-path solution is also feasible in the splittable model.

Therefore:

$$\mathcal{F}_{single} \subseteq \mathcal{F}_{split}.$$

This implies:

$$T^*_{single} \leq T^*_{split}.$$

This relationship must hold whenever:

- demand is identical;
- capacities are identical;
- candidate paths are identical;
- all other assumptions are identical.

If the notebook ever produces:

$$T^*_{single} > T^*_{split},$$

the models should be audited for inconsistent:

- path sets;
- demand values;
- link capacities;
- constraints;
- objective definitions;
- solver settings.

The baseline result satisfies the theoretical relationship with equality:

$$T^*_{single} = T^*_{split} = 83 \text{ Mbps}.$$

Equality is fully valid.

It means that the single-path restriction is not throughput-binding in that network state.

15.14 Marginal-Capacity Validation

A saturated link is not automatically a true throughput bottleneck.

To test the value of additional capacity, each directed arc is increased independently by:

$$1 \text{ Mbps}.$$

Stage 1 is then re-solved.

For link:

$$e,$$

marginal capacity value is:

$$MV_e = T^*(C_e + 1) - T^*(C_e).$$

A true local throughput bottleneck satisfies:

$$MV_e > 0.$$

A saturated link satisfying:

$$MV_e = 0$$

does not provide immediate stand-alone throughput value from one additional Mbps.

Because increasing capacity relaxes the feasible region:

$$MV_e \geq 0$$

up to numerical tolerance.

A materially negative value would indicate:

- inconsistent model construction;
- numerical problems;
- incorrect scenario inputs.

15.15 Cross-Table Reconciliation

The same network quantity may appear in several derived tables.

These values must reconcile.

For example:

$$T = \sum_{s,k} z_{s,k}$$

should equal the total obtained from:

- commodity summaries;
- source summaries;
- stream summaries;
- positive route flows.

Similarly, link-flow tables should be calculated from the same route-flow solution used to report delivered demand.

This prevents derived reporting tables from drifting away from the optimization output.

15.16 Fairness Validation

Commodity delivery ratio is defined as:

$$r_{s,k} = \frac{z_{s,k}}{D_{s,k}}.$$

Each ratio must satisfy:

$$0 \leq r_{s,k} \leq 1.$$

The minimum commodity delivery ratio is:

$$\alpha = \min_{s,k} r_{s,k}.$$

This represents the service level received by the worst-served commodity.

If:

$$\alpha = 0,$$

at least one commodity receives no service.

This metric is used to determine whether throughput maximization creates starvation or materially uneven service.

15.17 Scenario-Based Validation and Stress Testing

The optimization framework is not validated using only one operating point.

The model is also evaluated under controlled scenario changes.

These scenarios are designed to determine whether the optimization behaves consistently when:

- demand falls;
- demand rises;
- congestion becomes binding;
- attractive low-latency routes have insufficient capacity;
- several commodities share one bottleneck;
- routing flexibility changes;
- capacity is increased;
- infrastructure is degraded.

The scenarios are diagnostic validation experiments.

They are not forecasts of future operating conditions.

Detailed numerical results are analyzed in Section 43.

15.17.1 Scenario A — Low Demand / Uncongested Network

Purpose

Verify that all offered traffic is delivered when sufficient end-to-end capacity exists.

Scenario Design

Demand is reduced to:

$$60\%$$

of baseline.

Since baseline demand is:

$$126 \text{ Mbps},$$

the scenario offers:

$$126(0.60) = 75.6 \text{ Mbps}.$$

Validation Criteria

If the network has sufficient capacity:

$$T^* = 75.6 \text{ Mbps}.$$

The model should produce:

- zero lost demand;
- no link-capacity violations;
- correct destination routing.

Executed Result

Both routing formulations deliver:

$$75.6 \text{ Mbps}.$$

Therefore:

$$100\%$$

of offered traffic is served.

Interpretation

This confirms that the network is capable of serving all demand under sufficiently light loading.

The baseline service deficit therefore arises from capacity scarcity rather than disconnected demand.

15.17.2 Scenario B — Demand Exceeds Network Capacity

Purpose

Test whether the model correctly reports unavoidable service loss when demand becomes larger than the physical network can support.

Scenario Design

Demand is increased to:

$$150\%$$

of baseline:

$$126(1.50) = 189 \text{ Mbps}.$$

Validation Criteria

The model must continue to satisfy:

$$F_e \leq C_e$$

and:

$$z_{s,k} \leq D_{s,k}.$$

Any infeasible demand must be reported as lost rather than forcing capacity violations.

Executed Result

Both formulations deliver:

$$83 \text{ Mbps}.$$

Therefore, delivery rate becomes:

$$\frac{83}{189} \times 100 \approx 43.92\%.$$

Interpretation

The unchanged 83-Mbps ceiling shows that the network is capacity-limited under high demand.

Increasing demand alone cannot increase feasible throughput.

15.17.3 Scenario C — Fast but Narrow Path

Purpose

Test whether the optimizer correctly handles a low-latency path that does not have enough capacity to carry all desired traffic.

Scenario Structure

Suppose one route has:

- lower latency;
- lower capacity.

A second route has:

- higher latency;
- greater capacity.

Validation Criteria

Stage 1 should maximize delivered throughput rather than blindly selecting the shortest route.

Therefore, a longer path may be used if it enables more demand to be delivered.

After Stage 1 admissions are fixed, Stage 2 should improve sensitive latency using the available routing flexibility.

Why This Matters

This verifies that:

$$\text{Minimum Latency Path} \neq \text{Automatically Optimal Network Route}.$$

The baseline naive-routing experiment demonstrates this behavior directly.

15.17.4 Scenario D — Shared Bottleneck

Purpose

Test whether multiple commodities competing for the same directed link correctly share its finite bandwidth.

Validation Criterion

For shared link:

$$e,$$

the combined flow must satisfy:

$$\sum_{s,k,p:e\in p} f_{s,k,p} \le C_e.$$

This condition must hold regardless of:

- the number of commodities;
- traffic class;
- routing formulation.

Why This Matters

Several individually attractive routes may become infeasible when considered simultaneously.

The naive routing baseline demonstrates this issue.

For example:

$$B \to C$$

would receive approximately:

$$61\ \text{Mbps}$$

against only:

$$18\ \text{Mbps}$$

of capacity if commodities selected their routes independently.

The optimized models must eliminate this overload.

15.17.5 Scenario E — Splittable vs. Single-Path Routing

Purpose

Measure whether allowing a source-stream commodity to use multiple paths increases maximum throughput.

Theoretical Validation Criterion

Because:

$$\mathcal{F}_{single} \subseteq \mathcal{F}_{split},$$

the models must satisfy:

$$T^*_{single} \leq T^*_{split}.$$

Baseline Result

The baseline produces:

$$T^*_{single} = T^*_{split} = 83 \text{ Mbps}.$$

Stress-Test Result

The same equality is observed in every tested Stage 1 scenario:

- baseline;
- low demand;
- high demand;
- internal-corridor degradation;
- destination-exit degradation;
- fragmented internal capacity.

Interpretation

The single-path restriction is not throughput-binding in any tested operating state.

This does not imply that splitting can never improve throughput.

It means only that the larger split feasible region does not increase Stage 1 throughput for the network states evaluated here.

15.17.6 Scenario F — Capacity Sensitivity

Purpose

Identify true local throughput bottlenecks.

Procedure

For each directed link:

$$e,$$

increase capacity by:

$$1 \text{ Mbps}$$

and re-solve Stage 1.

Calculate:

$$MV_e = T^*(C_e + 1) - T^*(C_e).$$

Validation Criteria

A true local capacity bottleneck satisfies:

$$MV_e > 0.$$

Increasing capacity should never reduce optimized throughput:

$$MV_e \geq 0.$$

Executed Result

The perturbation analysis identifies positive marginal throughput value for links including:

- B → D;
- C → UK;
- C → US.

Interpretation

These links provide stronger evidence for immediate capacity value than saturation alone.

15.17.7 Scenario G — Equal Throughput, Different Latency

Purpose

Verify that the secondary objective distinguishes among solutions with the same throughput.

Validation Criteria

Stage 2 must preserve:

$$z_{s,k}^{Stage2} = z_{s,k}^{Stage1}$$

while improving or maintaining sensitive latency exposure:

$$LE^{Stage2} \leq LE^{Stage1}.$$

Executed Result

Within the splittable pipeline, total sensitive latency exposure decreases from:

$$850$$

to:

$$644 \text{ Mbps} \cdot \text{ms}.$$

The reduction is:

$$206 \text{ Mbps} \cdot \text{ms}.$$

Percentage reduction:

$$\frac{206}{850} \times 100 \approx 24.24\%.$$

Interpretation

This confirms that the lexicographic objective performs as intended.

Service quality improves without sacrificing Stage 1 throughput or changing the Stage 1 admission vector.

15.17.8 Scenario H — Internal-Corridor Degradation

Purpose

Test the network's dependence on internal transport capacity.

Scenario Design

Selected internal capacities are reduced to:

$$70\%$$

of baseline.

Executed Result

Maximum throughput falls from:

$$83$$

to:

$$78.1 \text{ Mbps}.$$

The reduction is:

$$4.9 \text{ Mbps}.$$

Interpretation

Internal corridor capacity contributes materially to feasible throughput.

However, alternate routes preserve part of the network's service capability.

15.17.9 Scenario I — Destination-Exit Degradation

Purpose

Test the sensitivity of throughput to destination-facing capacity.

Scenario Design

Selected exit capacities are reduced to:

$$75\%$$

of baseline.

Executed Result

Maximum throughput falls to:

$$69.25 \text{ Mbps}.$$

The reduction from baseline is:

$$83 - 69.25 = 13.75 \text{ Mbps}.$$

Interpretation

This is the largest throughput reduction among the tested capacity-degradation scenarios.

Upstream capacity has limited value if traffic cannot cross the final destination boundary.

15.17.10 Scenario J — Fragmented Internal Capacity

Purpose

Test whether several distributed capacity reductions collectively produce substantial service loss.

Executed Result

Maximum throughput falls to:

$$72.85 \text{ Mbps}.$$

The reduction from baseline is:

$$83 - 72.85 = 10.15 \text{ Mbps}.$$

Interpretation

The network is sensitive not only to isolated bottlenecks but also to distributed internal degradation.

This is relevant to resilience analysis.

15.18 Scenario Validation Summary

Across all scenarios, the same core validation conditions apply.

Demand Feasibility

$$0 \le z_{s,k} \le D_{s,k}.$$

Capacity Feasibility

$$F_e \le C_e.$$

Destination Correctness

Traffic terminates at the required exit.

Single-Path Compliance

For the single-path formulation:

$$\#\{p : f_{s,k,p} > 0\} \le 1.$$

Theoretical Model Ordering

$$T^*_{single} \leq T^*_{split}.$$

Scenario testing therefore verifies both:

1. mathematical model behavior;
2. robustness of the business conclusions under changing network conditions.

15.19 Validation Philosophy

The notebook follows the sequence:

1. construct;
2. solve;
3. validate solver status;
4. validate demand accounting;
5. validate capacity;
6. validate route logic;
7. reconcile summaries;
8. test controlled scenarios;
9. interpret the results;
10. explain why the results occurred;
11. translate findings into operational and business implications.

This approach prevents implementation errors from being mistaken for genuine network insights.

15.20 Final Validation Summary

The completed validation framework combines three complementary forms of evidence.

Mathematical Validation

Confirms that:

- demand bounds hold;
- capacities hold;
- destination rules hold;
- single-path restrictions are enforced;
- theoretical formulation relationships are respected.

Accounting Validation

Confirms that:

- delivered demand;
- lost demand;
- source summaries;
- stream summaries;
- route-level flows;

all reconcile to the same totals.

Behavioral Validation

Confirms that the optimizer responds logically when:

- demand is low;
- demand is high;
- capacity becomes constrained;
- multiple commodities share links;
- attractive routes are narrow;
- routing flexibility changes;
- capacity is increased;
- infrastructure is degraded;
- equal-throughput solutions differ in latency.

A solution is considered analytically trustworthy only when it is:

$$\boxed{\text{Optimal} + \text{Feasible} + \text{Reconciled} + \text{Behaviorally Consistent}}$$

This validation framework provides the foundation for interpreting all later optimization results and business recommendations.

16. Final Confirmed Modeling Interpretation

16.1 Purpose

The clarification process is complete.

This section records the final interpretation of the problem used throughout the

solved optimization models.

No modeling decision listed here should be treated as pending clarification.

Any remaining caveats later in the notebook relate to:

- deliberate modeling assumptions;
- synthetic data;
- deterministic inputs;
- production limitations;

rather than unresolved problem requirements.

This section therefore provides the final modeling interpretation that governs:

- the splittable formulation;
- the single-path formulation;
- the Stage 1 throughput objective;
- the Stage 2 latency objective;
- the stress-testing framework;
- the later fairness and infrastructure analyses.

16.2 Final Network Interpretation

The network is modeled as:

- a static independent snapshot;
- a directed full-duplex network;
- a source-stream multi-commodity flow problem;
- link-capacity constrained;
- node-capacity unconstrained;
- lossless;
- queue-free.

Each physical full-duplex connection is represented using two independent directed arcs.

For a physical link connecting nodes u and v, the model contains:

$$(u, v)$$

and:

$$(v, u).$$

Therefore:

$$C_{u\to v}$$

and:

$$C_{v\to u}$$

are separate capacity resources.

Traffic in one direction does not consume the capacity assigned to the reverse direction.

This representation is consistent with the confirmed full-duplex interpretation.

16.3 Final Demand Interpretation

Demand is known separately by:

- source;
- traffic stream.

For source s and stream k:

$$D_{s,k}$$

represents offered demand.

Each source-stream pair is therefore modeled as a distinct commodity:

$$(s,k).$$

For example:

- S1 / UK-L;
- S2 / UK-L;
- B / UK-L;

are separate commodities even though all contribute to the aggregate UK-L traffic class.

Delivered demand is:

$$z_{s,k}.$$

Partial delivery is allowed:

$$0 \le z_{s,k} \le D_{s,k}.$$

Undelivered demand is:

$$u_{s,k} = D_{s,k} - z_{s,k}.$$

Therefore:

$$D_{s,k} = z_{s,k} + u_{s,k}.$$

Traffic that cannot be delivered during the current snapshot is lost.

There is:

- no backlog;
- no buffering;
- no deferred transmission;
- no carryover into another optimization period.

16.4 Final Single-Path Interpretation

Single-path routing means:

> Each source-stream commodity may use at most one positive-flow route.

For binary path-selection variable:

$$y_{s,k,p} \in \{0, 1\},$$

the model enforces:

$$\sum_{p \in P_{s,k}} y_{s,k,p} \le 1.$$

Flow is linked to path selection using:

$$f_{s,k,p} \le D_{s,k} y_{s,k,p}.$$

Therefore:

$$f_{s,k,p} > 0 \Rightarrow y_{s,k,p} = 1.$$

Partial delivery remains allowed.

Therefore, single-path routing does **not** mean:

> the full source-stream demand must fit on one path.

Instead, it means:

> whatever portion of the commodity is delivered must use no more than one positive-flow path.

This distinction is important because an all-or-nothing unsplittable-flow formulation would create a different feasible region.

16.5 Final Splittable-Flow Interpretation

Under the splittable formulation, a source-stream commodity may use one or more candidate routes.

For commodity (s, k):

$$z_{s,k} = \sum_{p \in P_{s,k}} f_{s,k,p}.$$

The total delivered amount must satisfy:

$$z_{s,k} \leq D_{s,k}.$$

Splitting is permitted but not required.

A splittable optimum may route a commodity entirely over one path.

This is an important distinction because the baseline splittable Stage 1 solution achieves maximum throughput without requiring any commodity to use more than one positive-flow path.

Therefore:

> Allowing path splitting does not imply that splitting will necessarily appear in the optimal solution.

The model uses additional routing flexibility only when that flexibility creates objective value.

16.6 Final Capacity Interpretation

Directed-link bandwidth is the primary modeled physical resource constraint.

For every directed arc:

$$e \in E,$$

the optimization must satisfy:

$$\sum_{s,k,p:e\in p} f_{s,k,p} \leq C_e.$$

Nodes do not impose separate processing-capacity restrictions.

Therefore, throughput limitations in the baseline model arise from interactions among:

- directed-link capacities;
- source locations;
- candidate-path structure;
- destination-facing capacity;
- competing commodity flows.

This means that an apparent shortage of service cannot automatically be attributed to one link or one routing decision.

The entire end-to-end path structure must be considered.

16.7 Final Destination Interpretation

Traffic must leave the network through its required destination.

UK-bound traffic must terminate at:

$$UK.$$

US-bound traffic must terminate at:

$$US.$$

Candidate paths may not:

- terminate at the wrong destination;
- pass through the opposite destination and continue elsewhere.

This ensures that throughput is counted only when traffic reaches its correct exit.

16.8 Final Latency Interpretation

Path latency is additive.

For path p:

$$L_p = \sum_{e \in p} L_e.$$

Each link has a fixed latency.

The model does not currently include:

- queueing delay;
- congestion-dependent delay;
- node waiting time;
- latency variability;
- jitter.

Therefore, modeled latency represents:

> fixed route latency under the current synthetic network assumptions.

For split traffic, the primary latency measure is traffic-weighted average latency.

For commodity (s, k):

$$\bar{L}_{s,k} = \frac{\sum_{p \in P_{s,k}} f_{s,k,p} L_p}{z_{s,k}}.$$

Worst used-path latency is also reported:

$$L^{worst}_{s,k} = \max_{p : f_{s,k,p} > 0} L_p.$$

The weighted average captures the experience of the delivered traffic volume.

Worst used-path latency ensures that a small amount of traffic routed over a substantially slower path remains visible.

16.9 Final Service-Priority Interpretation

Latency-sensitive traffic has no automatic Stage 1 admission priority.

Every delivered Mbps contributes equally to the Stage 1 throughput objective:

$$\max \sum_{s,k} z_{s,k}.$$

Therefore:

$$1 \text{ Mbps of UK-L}$$

has the same Stage 1 throughput value as:

$$1 \text{ Mbps of UK-T},$$

$$1 \text{ Mbps of US-L},$$

or:

$$1 \text{ Mbps of US-T}.$$

Stage 1 therefore answers:

> **How much total traffic can the network deliver?**

It does not answer:

> **Which traffic should receive priority when capacity is scarce?**

Latency-sensitive traffic receives special treatment only after throughput has been established.

That treatment occurs through the secondary latency objective.

16.10 Final Objective Interpretation

The baseline optimization uses a lexicographic objective structure.

The two objectives are not combined using arbitrary weights.

Instead, they are solved sequentially.

Stage 1 — Maximum Throughput

The primary objective is:

$$T^* = \max \sum_{s,k} z_{s,k}.$$

This determines the maximum feasible amount of delivered traffic.

Stage 1 therefore answers:

> **How much traffic can the network physically deliver?**

Stage 2 — Sensitive-Latency Optimization

After Stage 1 is solved, the exact commodity-level admission vector is preserved:

$$z_{s,k}^{Stage2} = z_{s,k}^{Stage1}$$

for every commodity.

Stage 2 then minimizes latency exposure for the latency-sensitive streams:

- UK-L;
- US-L.

This separates:

> **How much traffic can be delivered?**

from:

> **How should the admitted traffic be routed?**

The use of exact admission preservation is deliberate.

If only total throughput were fixed, Stage 2 could change which commodities receive service while still reporting the same aggregate throughput.

Exact admission preservation prevents that ambiguity.

16.11 Final Fairness Interpretation

The baseline Stage 1 objective does not explicitly optimize fairness.

Because every delivered Mbps has equal throughput value, multiple commodity-admission patterns may achieve the same maximum throughput.

The later fairness analysis therefore introduces commodity delivery ratio:

$$r_{s,k} = \frac{z_{s,k}}{D_{s,k}}.$$

The minimum commodity delivery ratio is:

$$\alpha = \min_{s,k} r_{s,k}.$$

This creates a third policy question:

> **When all offered demand cannot be served, how should scarce capacity be distributed?**

The fairness analysis shows that aggregate throughput alone does not guarantee equitable commodity-level service.

Fairness is therefore treated as:

- an analytical extension;
- a recommended production policy layer;

rather than part of the original baseline Stage 1 objective.

A later production hierarchy may therefore become:

$$\boxed{\text{Throughput} \rightarrow \text{Fairness / SLA Policy} \rightarrow \text{Sensitive Latency} \rightarrow \text{Operational C}}$$

16.12 Final Synthetic-Data Interpretation

No final numerical production dataset was supplied.

Therefore, the notebook constructs synthetic heterogeneous values for:

- link capacities;
- link latencies;
- source-stream demands.

The synthetic dataset is intentionally designed to expose:

- shared-capacity congestion;
- alternate-path tradeoffs;
- latency tradeoffs;
- internal corridor constraints;
- destination-facing bottlenecks;
- capacity sensitivity.

The numerical values therefore describe the synthetic assessment environment.

For example:

$$83 \text{ Mbps}$$

is the optimized capacity of the constructed baseline network.

It should not be interpreted as a universal throughput value or as a claim about a real production network.

However, many methodological conclusions are more general, such as:

- independent shortest paths may violate shared capacity;
- equal throughput does not imply equal service;
- saturation does not necessarily imply positive marginal capacity value;
- routing flexibility may improve service quality without improving throughput.

16.13 Final Baseline Result Context

Using the final confirmed interpretation, the baseline synthetic network contains:

$$126 \text{ Mbps}$$

of total offered demand.

Both Stage 1 formulations achieve:

$$83 \text{ Mbps}$$

of maximum delivered throughput.

Therefore:

$$126 - 83 = 43 \text{ Mbps}$$

remains undelivered.

The baseline delivery rate is:

$$\frac{83}{126} \times 100 \approx 65.87\%.$$

The result:

$$T^*_{single} = T^*_{split} = 83 \text{ Mbps}$$

is interpreted as evidence that the single-path restriction is not throughput-binding in the baseline network.

This result should be interpreted in the context of several confirmed modeling conditions:

- partial delivery is allowed;
- candidate paths are pre-enumerated;
- node capacity is unconstrained;
- link capacity is the primary physical limitation;
- no Stage 1 sensitive-traffic priority exists.

Therefore, the 83-Mbps ceiling represents a physical capacity and topology limitation within the synthetic network rather than an unresolved modeling ambiguity.

16.14 Clarification Resolution Summary

All material clarification questions required to construct, solve, validate, and interpret the baseline optimization models have now been resolved.

The table below provides a concise summary of the final decisions governing the remainder of the notebook.

Question / Issue	Final Resolution	Modeling Consequence
Source-level demand	Modeled separately by source and stream	Use $D_{s,k}$ and treat each source-stream pair as a distinct commodity
Single-path interpretation	One path per source-stream commodity	Enforce at most one positive-flow route for each (s, k)
Partial delivery	Allowed	$0 \leq z_{s,k} \leq D_{s,k}$
Split-stream latency	Analyst modeling choice	Use traffic-weighted average latency and report worst used-path latency
Numerical dataset	Synthetic dataset required	Construct heterogeneous capacities, latencies, and source-stream demand
Node capacity	Unlimited	Use directed-link capacity constraints only
Queueing	None	Independent static snapshots with fixed link latency
Buffering / holding	None	No queues, waiting, or backlog
Lost demand	Lost immediately	No carryover variable; undelivered traffic is explicitly reported
Service priority	No explicit Stage 1 priority	Every delivered Mbps receives equal Stage 1 throughput value

Stage 2 admission	Preserve exact Stage 1 commodity admissions	Latency improvements are produced through rerouting rather than changed admission
Valid exits	Traffic must terminate at its required destination	UK traffic exits only through UK; US traffic exits only through US

These decisions are treated as authoritative throughout the remaining analysis.

16.15 Final Status

All material formulation ambiguities have been resolved.

There is no remaining clarification required regarding:

- source-level demand;
- commodity definition;
- single-path scope;
- partial delivery;
- split-traffic latency reporting;
- synthetic dataset construction;
- queueing;
- buffering;
- lost demand;
- node capacity;
- traffic priority;
- destination correctness.

Therefore:

> **The baseline and stress-test optimization models can be constructed, solved, validated, and interpreted using a fully specified problem definition.**

The remaining limitations in the notebook relate to deliberate modeling abstractions rather than unresolved requirements.

These include:

- synthetic data;
- deterministic inputs;
- fixed additive latency;
- no queue-dependent delay;

- no failure probabilities;
- pre-enumerated candidate paths;
- no route-churn penalty;
- no explicit infrastructure cost model;
- production scalability.

Those limitations and their corresponding production extensions are addressed explicitly in Section 46.

17. Source Requirements vs. Modeling Choices

To avoid mixing stated requirements with assumptions, the notebook distinguishes between information explicitly provided in the problem and modeling decisions introduced during the analysis.

Explicitly Given

- four traffic streams;
- UK and US exits;
- link bandwidth constraints;
- directional full-duplex links;
- additive link latency;
- no queueing or waiting;
- lost unroutable demand;
- equal weight for delivered traffic;
- split-flow and single-path variants;
- throughput and latency objectives;
- requirement to identify bottleneck links and marginal capacity value.

Candidate Modeling Choices

- exact mathematical formulation;
- edge-based versus path-based variables;
- lexicographic versus weighted multi-objective optimization;
- fairness or starvation treatment;
- latency aggregation for split sensitive traffic;
- synthetic test-data design;

- numerical solution method;
- sensitivity-analysis implementation.

Candidate Modeling Choices

The following elements remain analyst choices rather than requirements dictated by the problem:

- lexicographic objective implementation;
- use of traffic-weighted average latency for split traffic;
- reporting worst-case latency alongside average latency;
- synthetic dataset design;
- solver implementation;
- fairness analysis;
- capacity-sensitivity methodology;
- optional latency-priority policy scenario.

18. Exploratory Equal-Link Baseline

Before introducing heterogeneous synthetic data, an equal-link baseline is used to isolate the structural effects of network topology.

This section is intentionally retained even after clarification because it provides a useful reference case against which the final heterogeneous optimization can be compared.

For this exploratory scenario only, assume that every physical link has identical characteristics:

$$C_e = 30 \text{ Mbps}$$

and

$$L_e = 20 \text{ ms}$$

for every directed link e.

These values are used only to understand the current graph structure and route alternatives. They are not assumed to be the final network parameters.

Because every link has the same latency, minimizing path latency is equivalent to minimizing the number of links, or hops, in the path.

Therefore, for any path p containing h_p links:

$$L_p = 20h_p$$

For example:

- 1 link = 20 ms
- 2 links = 40 ms
- 3 links = 60 ms
- 4 links = 80 ms
- 5 links = 100 ms

Because every link also has the same 30 Mbps capacity, the isolated capacity of any simple path is:

$$C_p = 30 \text{ Mbps}$$

provided no capacity is being consumed by competing traffic.

This does **not** mean that every path can simultaneously carry 30 Mbps. Paths that share links must share the 30 Mbps capacity of those links.

18.1 Temporary Assumptions

For this exploratory analysis, the following temporary assumptions are used:

1. Every physical link has 30 Mbps directional capacity.
2. Every physical link has 20 ms latency.
3. Each physical link is treated as full-duplex.
4. The same topology shown in the problem statement is used.
5. Traffic exits the network once it reaches UK or US.
6. Paths may not pass through the opposite exit.
7. Only simple paths are considered, meaning a path cannot revisit the same node.
8. This section is exploratory only and will be updated once final link capacities, latencies, and demand values are confirmed.

18.2 Build the Equal-Link Graph

In [40]:
```
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Known physical topology
physical_edges = [
    ("S1", "A"),
    ("S2", "A"),
    ("S3", "B"),
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("B", "D"),
    ("C", "UK"),
    ("C", "US"),
    ("D", "UK"),
    ("D", "US")
]

# Temporary exploratory assumptions
ASSUMED_CAPACITY_MBPS = 30
ASSUMED_LATENCY_MS = 20

G = nx.Graph()

for u, v in physical_edges:
    G.add_edge(
        u,
        v,
        capacity_mbps=ASSUMED_CAPACITY_MBPS,
        latency_ms=ASSUMED_LATENCY_MS
    )

print(f"Nodes: {G.number_of_nodes()}")
print(f"Physical links: {G.number_of_edges()}")
```

```
Nodes: 9
Physical links: 11
```

18.2.1 Visualize the Equal-Link Graph

In [41]:

```
pos = {
    "S1": (0, 3),
    "S2": (0, 2),
    "S3": (0, 1),
    "A":  (2, 2.5),
    "B":  (2, 1.2),
    "C":  (4, 2.5),
    "D":  (4, 1.0),
    "UK": (6, 3.0),
    "US": (6, 1.4)
}

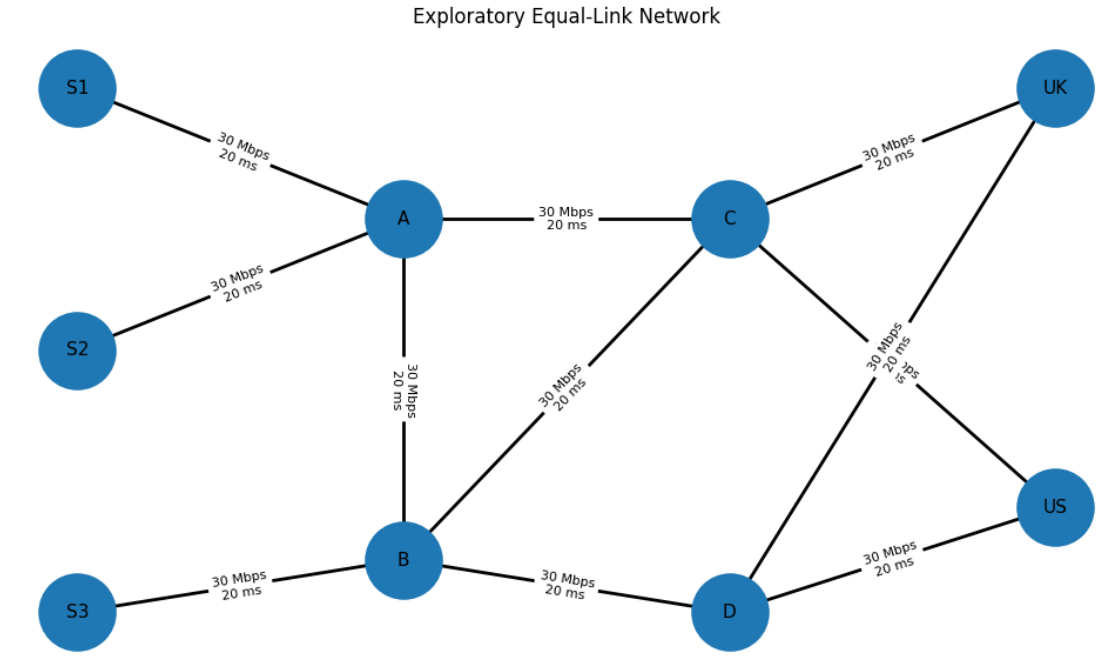
plt.figure(figsize=(13, 7))
```

```
nx.draw_networkx(
    G,
    pos,
    with_labels=True,
    node_size=2200,
    font_size=11,
    width=2
)

edge_labels = {
    (u, v): f"{ASSUMED_CAPACITY_MBPS} Mbps\n{ASSUMED_LATENCY_MS} ms"
    for u, v in G.edges()
}

nx.draw_networkx_edge_labels(
    G,
    pos,
    edge_labels=edge_labels,
    font_size=8
)

plt.title("Exploratory Equal-Link Network")
plt.axis("off")
plt.show()
```



18.3 Generate All Feasible Simple Paths

The traffic-generating nodes are:

- S1
- S2

- S3
- B
- C

The valid exit nodes are:

- UK
- US

For each source-destination pair, all simple paths are generated.

A valid path must satisfy the following:

- it starts at a traffic-generating source;
- it ends at the required exit;
- it does not repeat nodes;
- it does not pass through the opposite exit before reaching its destination.

For example, a US-bound path may not pass through UK and then continue to US because traffic leaves the network once it reaches an exit.

In [18]:
```
traffic_sources = ["S1", "S2", "S3", "B", "C"]
destinations = ["UK", "US"]

def get_valid_simple_paths(graph, source, destination):
    """
    Return all simple paths from source to destination,
    excluding paths that pass through the opposite exit.
    """

    opposite_exit = "US" if destination == "UK" else "UK"

    valid_paths = []

    for path in nx.all_simple_paths(graph, source, destination):
        if opposite_exit not in path[:-1]:
            valid_paths.append(path)

    return valid_paths
```

18.4 Calculate Path Statistics

For each feasible path, calculate:

- number of links;
- path capacity;
- total path latency.

For a path p:

$$C_p = \min_{e \in p} C_e$$

and:

$$L_p = \sum_{e \in p} L_e$$

Under the equal-link assumption:

$$C_p = 30 \text{ Mbps}$$

for every isolated simple path, while latency depends on the number of hops:

$$L_p = 20 h_p$$

In [10]:
```python
def path_statistics(graph, path):

    edges_in_path = list(zip(path[:-1], path[1:]))

    capacities = [
        graph[u][v]["capacity_mbps"]
        for u, v in edges_in_path
    ]

    latencies = [
        graph[u][v]["latency_ms"]
        for u, v in edges_in_path
    ]

    return {
        "num_links": len(edges_in_path),
        "path_capacity_mbps": min(capacities),
        "path_latency_ms": sum(latencies)
    }
```

18.4.1 Create a Table of All Feasible Routes

In [11]:
```python
route_records = []

for source in traffic_sources:

    for destination in destinations:

        paths = get_valid_simple_paths(
            G,
            source,
```

```
            destination
        )

        for route_id, path in enumerate(paths, start=1):

            stats = path_statistics(G, path)

            route_records.append({
                "source": source,
                "destination": destination,
                "route_id": route_id,
                "path": " -> ".join(path),
                "num_links": stats["num_links"],
                "path_capacity_mbps": stats["path_capacity_mbps"],
                "path_latency_ms": stats["path_latency_ms"]
            })

routes_df = pd.DataFrame(route_records)

routes_df
```

Out[11]:

	source	destination	route_id	path	num_links	path_capacity_mbps	path_
0	S1	UK	1	S1 -> A -> B -> C -> UK	4	30	
1	S1	UK	2	S1 -> A -> B -> D -> UK	4	30	
2	S1	UK	3	S1 -> A -> C -> B -> D -> UK	5	30	
3	S1	UK	4	S1 -> A -> C -> UK	3	30	
4	S1	US	1	S1 -> A -> B -> C	4	30	

				-> US		
5	S1	US	2	S1 -> A -> B -> D -> US	4	30
6	S1	US	3	S1 -> A -> C -> B -> D -> US	5	30
7	S1	US	4	S1 -> A -> C -> US	3	30
8	S2	UK	1	S2 -> A -> B -> C -> UK	4	30
9	S2	UK	2	S2 -> A -> B -> D -> UK	4	30
10	S2	UK	3	S2 -> A -> C -> B -> D -> UK	5	30
11	S2	UK	4	S2 -> A -> C -> UK	3	30
12	S2	US	1	S2 -> A -> B -> C ->	4	30

				US		
13	S2	US	2	S2 -> A -> B -> D -> US	4	30
14	S2	US	3	S2 -> A -> C -> B -> D -> US	5	30
15	S2	US	4	S2 -> A -> C -> US	3	30
16	S3	UK	1	S3 -> B -> A -> C -> UK	4	30
17	S3	UK	2	S3 -> B -> C -> UK	3	30
18	S3	UK	3	S3 -> B -> D -> UK	3	30
19	S3	US	1	S3 -> B -> A -> C -> US	4	30
20	S3	US	2	S3 -> B -> C -> US	3	30
				S3 -> B		

21	S3	US	3	-> D -> US	3	30
22	B	UK	1	B -> A -> C -> UK	3	30
23	B	UK	2	B -> C -> UK	2	30
24	B	UK	3	B -> D -> UK	2	30
25	B	US	1	B -> A -> C -> US	3	30
26	B	US	2	B -> C -> US	2	30
27	B	US	3	B -> D -> US	2	30
28	C	UK	1	C -> A -> B -> D -> UK	4	30
29	C	UK	2	C -> B -> D -> UK	3	30
30	C	UK	3	C -> UK	1	30
31	C	US	1	C -> A -> B -> D -> US	4	30
32	C	US	2	C -> B -> D -> US	3	30
33	C	US	3	C -> US	1	30

Interpretation of Feasible Route Table

The route table enumerates every valid simple path from each traffic-generating source node to both exits under the current network topology.

A total of 34 feasible source-to-exit routes are identified.

Each route is described by:

- its source node;
- required destination;
- route sequence;
- number of links traversed;
- path capacity under the equal-link assumption;
- total end-to-end latency.

Because every link is temporarily assumed to have:

$$30 \text{ Mbps}$$

of capacity and:

$$20 \text{ ms}$$

of latency, every isolated path has the same nominal bottleneck capacity:

$$30 \text{ Mbps}$$

The path latencies differ only because some routes contain more links than others.

For a path with h_p links:

$$L_p = 20h_p$$

Therefore:

- 1-link routes have 20 ms latency;
- 2-link routes have 40 ms latency;
- 3-link routes have 60 ms latency;
- 4-link routes have 80 ms latency;
- 5-link routes have 100 ms latency.

The table also shows that the topology provides meaningful routing alternatives.

For example, traffic from S1 to UK can take four different feasible routes:

- S1 → A → C → UK
- S1 → A → B → C → UK
- S1 → A → B → D → UK
- S1 → A → C → B → D → UK

These alternatives are important because the globally optimal routing solution may choose a route that is not the shortest route if doing so reduces congestion on shared links and increases total delivered throughput.

The path capacity shown in this table should not be interpreted as independent capacity available to every route. Two routes that share the same physical link must share that link's 30 Mbps capacity.

18.5 Identify Shortest-Latency Paths

Because every link has the same 20 ms latency, the shortest-latency path is also the minimum-hop path.

The shortest route is therefore the feasible path with the minimum total latency.

Multiple paths may tie for shortest latency.

This is important because equal-latency alternatives may allow the optimizer to redirect traffic around congested links without increasing latency.

In [12]:
```
routes_df["is_shortest_latency"] = False

for (source, destination), group in routes_df.groupby(
    ["source", "destination"]
):

    min_latency = group["path_latency_ms"].min()

    routes_df.loc[
        group.index,
        "is_shortest_latency"
    ] = group["path_latency_ms"] == min_latency

routes_df = routes_df.sort_values(
    ["source", "destination", "path_latency_ms", "path"]
)

routes_df
```

Out[12]:

	source	destination	route_id	path	num_links	path_capacity_mbps	path_

B ->

23	B	UK	2	C -> UK	2	30
24	B	UK	3	B -> D -> UK	2	30
22	B	UK	1	B -> A -> C -> UK	3	30
26	B	US	2	B -> C -> US	2	30
27	B	US	3	B -> D -> US	2	30
25	B	US	1	B -> A -> C -> US	3	30
30	C	UK	3	C -> UK	1	30
29	C	UK	2	C -> B -> D -> UK	3	30
28	C	UK	1	C -> A -> B -> D -> UK	4	30
33	C	US	3	C -> US	1	30
32	C	US	2	C -> B -> D -> US	3	30
31	C	US	1	C -> A -> B -> D -> US	4	30
3	S1	UK	4	S1 -> A -> C -> UK	3	30

0	S1	UK	1	S1 -> A -> B -> C -> UK	4	30
1	S1	UK	2	S1 -> A -> B -> D -> UK	4	30
2	S1	UK	3	S1 -> A -> C -> B -> D -> UK	5	30
7	S1	US	4	S1 -> A -> C -> US	3	30
4	S1	US	1	S1 -> A -> B -> C -> US	4	30
5	S1	US	2	S1 -> A -> B -> D -> US	4	30
6	S1	US	3	S1 -> A -> C -> B -> D -> US	5	30
11	S2	UK	4	S2 -> A -> C -> UK	3	30
				S2 -		

8	S2	UK	1	> A -> B -> C -> UK	4	30
9	S2	UK	2	S2 -> A -> B -> D -> UK	4	30
10	S2	UK	3	S2 -> A -> C -> B -> D -> UK	5	30
15	S2	US	4	S2 -> A -> C -> US	3	30
12	S2	US	1	S2 -> A -> B -> C -> US	4	30
13	S2	US	2	S2 -> A -> B -> D -> US	4	30
14	S2	US	3	S2 -> A -> C -> B -> D -> US	5	30
17	S3	UK	2	S3 -> B -> C -> UK	3	30
				S3 -> B		

18	S3	UK	3	-> D -> UK	3	30
16	S3	UK	1	S3 -> B -> A -> C -> UK	4	30
20	S3	US	2	S3 -> B -> C -> US	3	30
21	S3	US	3	S3 -> B -> D -> US	3	30
19	S3	US	1	S3 -> B -> A -> C -> US	4	30

Interpretation of Shortest-Latency Identification

The shortest-latency calculation compares all feasible routes for each source-destination pair and identifies the minimum achievable end-to-end latency.

Because all links currently have equal latency:

$$L_e = 20 \text{ ms}$$

the minimum-latency path is equivalent to the path containing the fewest links.

The results reveal two different situations.

Unique Shortest Paths

Some source-destination pairs have one clear shortest route.

For example:

$$S1 \rightarrow A \rightarrow C \rightarrow UK$$

contains three links and therefore has latency:

$$3(20) = 60 \text{ ms}$$

The alternative S1-to-UK routes require four or five links and therefore have higher latency.

The same pattern occurs for:

- S1 → US;
- S2 → UK;
- S2 → US;
- C → UK;
- C → US.

Tied Shortest Paths

Other source-destination pairs have multiple routes with identical minimum latency.

For example, B can reach UK through either:

$$B \to C \to UK$$

or:

$$B \to D \to UK$$

Both contain two links, so both have:

$$40 \text{ ms}$$

latency.

Similarly, S3 has two equally short routes to each exit through either C or D.

These ties are particularly useful for optimization because traffic can potentially be moved between equal-latency routes to avoid congestion without increasing latency.

18.5.1 Show Only the Shortest-Latency Routes

In [13]:
```
shortest_routes_df = (
    routes_df[
        routes_df["is_shortest_latency"]
    ]
    .sort_values(
```

```
        ["source", "destination", "path"]
    )
    .reset_index(drop=True)
)

shortest_routes_df
```

Out[13]:

	source	destination	route_id	path	num_links	path_capacity_mbps	path_
0	B	UK	2	B -> C -> UK	2	30	
1	B	UK	3	B -> D -> UK	2	30	
2	B	US	2	B -> C -> US	2	30	
3	B	US	3	B -> D -> US	2	30	
4	C	UK	3	C -> UK	1	30	
5	C	US	3	C -> US	1	30	
6	S1	UK	4	S1 -> A -> C -> UK	3	30	
7	S1	US	4	S1 -> A -> C -> US	3	30	
8	S2	UK	4	S2 -> A -> C -> UK	3	30	
9	S2	US	4	S2 -> A -> C -> US	3	30	
				S3 -> B			

10	S3	UK	2	-> C -> UK	3	30
11	S3	UK	3	S3 -> B -> D -> UK	3	30
12	S3	US	2	S3 -> B -> C -> US	3	30
13	S3	US	3	S3 -> B -> D -> US	3	30

Interpretation of Shortest-Latency Route Set

Filtering the full route table to only minimum-latency paths leaves 14 shortest routes across the 10 source-destination combinations.

The number exceeds 10 because several source-destination pairs have tied shortest paths.

The shortest-path structure is:

- S1 has one shortest path to UK and one to US.
- S2 has one shortest path to UK and one to US.
- S3 has two equally short paths to UK and two equally short paths to US.
- B has two equally short paths to UK and two equally short paths to US.
- C has one direct shortest path to UK and one direct shortest path to US.

This reveals an important structural difference among the source nodes.

S1 and S2

S1 and S2 both feed into A and have a unique minimum-latency route through C.

For example:

$$S1 \rightarrow A \rightarrow C \rightarrow US$$

has 60 ms latency.

This means that if A → C becomes heavily utilized, S1 and S2 may need to accept higher-latency alternatives.

S3 and B

S3 and B have greater routing flexibility because they can reach the exits through either C or D without increasing latency under the equal-link assumption.

This creates useful load-balancing opportunities.

C

C is already directly adjacent to both exits.

Its shortest routes are:

$$C \to UK$$

and:

$$C \to US$$

with only:

$$20 \text{ ms}$$

of latency.

This makes traffic originating at C structurally advantaged in the equal-link scenario.

18.6 Review All Feasible Routes

In [14]:
```
for source in traffic_sources:

    print("=" * 70)
    print(f"SOURCE: {source}")
    print("=" * 70)

    for destination in destinations:

        subset = routes_df[
            (routes_df["source"] == source) &
            (routes_df["destination"] == destination)
        ].sort_values(
            ["path_latency_ms", "path"]
```

```
        )

        print(f"\nDestination: {destination}")

        for _, row in subset.iterrows():

            shortest_flag = (
                "  <-- SHORTEST"
                if row["is_shortest_latency"]
                else ""
            )

            print(
                f"{row['path']} | "
                f"{row['num_links']} links | "
                f"{row['path_latency_ms']} ms | "
                f"{row['path_capacity_mbps']} Mbps"
                f"{shortest_flag}"
            )
```

```
======================================================================
SOURCE: S1
======================================================================

Destination: UK
S1 -> A -> C -> UK | 3 links | 60 ms | 30 Mbps  <-- SHORTEST
S1 -> A -> B -> C -> UK | 4 links | 80 ms | 30 Mbps
S1 -> A -> B -> D -> UK | 4 links | 80 ms | 30 Mbps
S1 -> A -> C -> B -> D -> UK | 5 links | 100 ms | 30 Mbps

Destination: US
S1 -> A -> C -> US | 3 links | 60 ms | 30 Mbps  <-- SHORTEST
S1 -> A -> B -> C -> US | 4 links | 80 ms | 30 Mbps
S1 -> A -> B -> D -> US | 4 links | 80 ms | 30 Mbps
S1 -> A -> C -> B -> D -> US | 5 links | 100 ms | 30 Mbps
======================================================================
SOURCE: S2
======================================================================

Destination: UK
S2 -> A -> C -> UK | 3 links | 60 ms | 30 Mbps  <-- SHORTEST
S2 -> A -> B -> C -> UK | 4 links | 80 ms | 30 Mbps
S2 -> A -> B -> D -> UK | 4 links | 80 ms | 30 Mbps
S2 -> A -> C -> B -> D -> UK | 5 links | 100 ms | 30 Mbps

Destination: US
S2 -> A -> C -> US | 3 links | 60 ms | 30 Mbps  <-- SHORTEST
S2 -> A -> B -> C -> US | 4 links | 80 ms | 30 Mbps
S2 -> A -> B -> D -> US | 4 links | 80 ms | 30 Mbps
S2 -> A -> C -> B -> D -> US | 5 links | 100 ms | 30 Mbps
======================================================================
SOURCE: S3
======================================================================
```

```
Destination: UK
S3 -> B -> C -> UK | 3 links | 60 ms | 30 Mbps  <-- SHORTEST
S3 -> B -> D -> UK | 3 links | 60 ms | 30 Mbps  <-- SHORTEST
S3 -> B -> A -> C -> UK | 4 links | 80 ms | 30 Mbps

Destination: US
S3 -> B -> C -> US | 3 links | 60 ms | 30 Mbps  <-- SHORTEST
S3 -> B -> D -> US | 3 links | 60 ms | 30 Mbps  <-- SHORTEST
S3 -> B -> A -> C -> US | 4 links | 80 ms | 30 Mbps
======================================================================
SOURCE: B
======================================================================

Destination: UK
B -> C -> UK | 2 links | 40 ms | 30 Mbps  <-- SHORTEST
B -> D -> UK | 2 links | 40 ms | 30 Mbps  <-- SHORTEST
B -> A -> C -> UK | 3 links | 60 ms | 30 Mbps

Destination: US
B -> C -> US | 2 links | 40 ms | 30 Mbps  <-- SHORTEST
B -> D -> US | 2 links | 40 ms | 30 Mbps  <-- SHORTEST
B -> A -> C -> US | 3 links | 60 ms | 30 Mbps
======================================================================
SOURCE: C
======================================================================

Destination: UK
C -> UK | 1 links | 20 ms | 30 Mbps  <-- SHORTEST
C -> B -> D -> UK | 3 links | 60 ms | 30 Mbps
C -> A -> B -> D -> UK | 4 links | 80 ms | 30 Mbps

Destination: US
C -> US | 1 links | 20 ms | 30 Mbps  <-- SHORTEST
C -> B -> D -> US | 3 links | 60 ms | 30 Mbps
C -> A -> B -> D -> US | 4 links | 80 ms | 30 Mbps
```

Interpretation of Full Route Review

The grouped route review makes the routing alternatives easier to compare source by source.

Several structural patterns are visible.

S1 and S2

S1 and S2 have the same routing structure because both connect first to A.

Their shortest routes to either exit travel through C and have 60 ms latency.

Alternative routes through B require additional hops and therefore increase latency to 80 or 100 ms.

This suggests that S1 and S2 are likely to compete heavily for capacity on:

$$A \to C$$

when shortest-path routing is used.

S3

S3 enters the network through B.

From B, traffic can move toward either C or D.

As a result, S3 has two equally short routes to each destination.

This gives S3 more immediate routing flexibility than S1 or S2.

B

B is centrally located and has two equal two-hop routes to both exits:

- through C;
- through D.

This makes B an important routing junction.

C

C is the closest traffic-generating node to both exits.

Its direct routes have only 20 ms latency.

Longer alternatives exist through B and D, but these would normally be used only when direct exit links are capacity constrained or when the global optimizer finds them useful for increasing total throughput.

Overall Observation

The network contains both:

- low-latency direct corridors;
- longer alternative corridors.

This is exactly the structure required for the optimization problem to be

meaningful.

If every source had only one possible route, there would be little routing decision to optimize.

18.6.1 Expected Shortest Routes

Under the equal-link assumption, the shortest routes are:

Source	Destination	Shortest Path	Links	Latency
S1	UK	S1 → A → C → UK	3	60 ms
S1	US	S1 → A → C → US	3	60 ms
S2	UK	S2 → A → C → UK	3	60 ms
S2	US	S2 → A → C → US	3	60 ms
S3	UK	S3 → B → C → UK	3	60 ms
S3	UK	S3 → B → D → UK	3	60 ms
S3	US	S3 → B → C → US	3	60 ms
S3	US	S3 → B → D → US	3	60 ms
B	UK	B → C → UK	2	40 ms
B	UK	B → D → UK	2	40 ms
B	US	B → C → US	2	40 ms
B	US	B → D → US	2	40 ms
C	UK	C → UK	1	20 ms
C	US	C → US	1	20 ms

Several source-destination pairs have multiple equally short paths.

These ties are important because they create routing flexibility without increasing latency.

Interpretation of Expected Shortest Routes

The shortest-route table summarizes the minimum-latency options available to each source-destination pair under the equal-link assumption.

Several source-destination pairs have multiple equally short paths.

In particular:

- B can reach either exit through C or D with the same 40 ms latency.
- S3 can reach either exit through C or D with the same 60 ms latency.

These ties are valuable because they create routing flexibility without increasing latency.

By contrast:

- S1 and S2 each have a unique shortest route through C;
- C has a unique direct route to each exit.

This suggests that shortest-path traffic originating at S1 and S2 may concentrate more heavily on the A → C corridor, while traffic originating at B and S3 has more flexibility to distribute across C and D.

The shortest-route table therefore provides a useful baseline for comparing naive shortest-path routing with the later globally optimized routing solution.

18.7 Interpretation of the Equal-Link Scenario

When every link is assigned the same 30 Mbps capacity and 20 ms latency, route latency depends only on the number of links traversed.

Therefore, shortest-latency routing becomes equivalent to minimum-hop routing.

Several important structural characteristics are visible.

1. C Has Direct Access to Both Exits

Traffic originating at C can reach either UK or US in one hop:

$$20 \text{ ms}$$

This makes C structurally close to both exits under the equal-link assumption.

---

2. B Has Two Equally Short Exit Corridors

B can reach either exit through C or D in two hops:

$$40 \text{ ms}$$

For example:

$$B \to C \to UK$$

and:

$$B \to D \to UK$$

have identical latency.

This provides routing flexibility without a latency penalty under the equal-link scenario.

---

3. S3 Also Has Two Equally Short Alternatives

For example:

$$S3 \to B \to C \to UK$$

and:

$$S3 \to B \to D \to UK$$

both have:

$$60 \text{ ms}$$

The same pattern occurs for traffic from S3 to US.

---

4. S1 and S2 Have a Unique Shortest Route Through C

For example:

$$S1 \to A \to C \to US$$

has:

$$60 \text{ ms}$$

Routes through B require additional links and therefore higher latency under the equal-link assumption.

---

5. Equal Path Capacity Does Not Mean Independent Path Capacity

Although every isolated path has a nominal bottleneck capacity of:

$$30 \text{ Mbps}$$

multiple routes frequently share the same physical links.

For example, traffic from both S1 and S2 may use:

$$A \to C$$

If that directed link has 30 Mbps capacity, then all traffic using that link must satisfy:

$$\sum_k x_{k,A\to C} \le 30$$

The routes do not each receive an independent 30 Mbps allocation.

---

6. Shortest-Path Routing Alone Does Not Solve the Problem

If each stream independently selects its shortest route, multiple streams may compete for the same low-latency links.

This can saturate attractive links while alternative routes remain underutilized.

The optimization must therefore consider all streams jointly and determine when a higher-latency alternative improves total network throughput.

---

7. This Scenario Provides a Useful Baseline

The equal-link case provides a simple baseline for comparing:

- naive shortest-path routing;
- globally optimized split-flow routing;
- globally optimized single-path routing.

Once the final link capacities and latencies are available, the same analysis can be repeated with heterogeneous network values.

The comparison will show how much of the routing behavior is caused by network topology versus differences in individual link characteristics.

Overall Interpretation of the Equal-Link Scenario

The equal-link scenario isolates the effect of network topology by holding link capacity and latency constant.

Because every link has the same 30 Mbps capacity and 20 ms latency, any differences in route quality are caused by the structure of the graph rather than

heterogeneous link characteristics.

The analysis shows that:

1. C is structurally closest to both exits.
2. B and S3 have multiple equal-latency route alternatives.
3. S1 and S2 depend more heavily on the A → C corridor for minimum-latency routing.
4. Longer alternative routes exist and may become useful when low-latency links are congested.
5. Equal nominal path capacity does not imply independent capacity because routes often share the same physical links.
6. Shortest-path routing alone cannot account for competition among the four traffic streams.

The most important conclusion is that the shortest route for an individual stream is not necessarily the best route for the network as a whole.

A globally optimized solution may deliberately move some traffic onto a longer path in order to:

- preserve scarce low-latency capacity;
- reduce congestion;
- increase total delivered throughput;
- or maintain low latency for sensitive traffic.

This provides the baseline against which the later split-flow and single-path optimization results can be compared.

18.8 Equal-Link Baseline — Results Analysis and Business Impact

Results Analysis

The equal-link baseline provides a deliberately simplified reference case before heterogeneous capacities and latencies are introduced.

Under equal link characteristics, route selection is driven primarily by:

- network topology;
- hop count;
- path availability;
- destination accessibility.

This creates a useful control case because differences in route preference cannot yet be attributed to heterogeneous bandwidth or latency.

The baseline therefore helps isolate the effect of **network structure itself**.

Why This Happens

Even when links have identical characteristics, sources do not have identical routing opportunities.

Some source-destination pairs:

- have more alternate paths;
- reach the exits through fewer shared corridors;
- depend more heavily on specific intermediate nodes.

Therefore, topology alone can create asymmetric routing flexibility.

This means:

$$\boxed{\text{Equal Link Characteristics} \neq \text{Equal Routing Opportunity}}$$

Business Impact

This baseline establishes an important interpretation principle for the remainder of the analysis.

If congestion or unequal service later appears after heterogeneous link characteristics are introduced, those outcomes may result from a combination of:

- topology;
- link capacity;
- link latency;
- competing traffic.

The equal-link case provides a reference point for separating those effects.

From an operational perspective, this means infrastructure decisions should not be based solely on individual link specifications.

The structural position of a link within the network can be just as important as its nominal capacity or latency.

19. Shared-Link and Route Overlap

Analysis

The previous section identified all feasible simple routes from each traffic-generating source to UK and US.

The next step is to examine how frequently individual links appear across those routes.

This analysis helps identify links that are structurally important before demand is introduced.

Two measures will be considered:

1. **All-route usage:** how many feasible routes contain each link.
2. **Shortest-route usage:** how many minimum-latency routes contain each link.

A link appearing frequently in candidate routes may experience significant competition once traffic demand is introduced.

However, route frequency alone does **not** establish that a link is a true bottleneck.

A true bottleneck depends on:

- actual demand;
- simultaneous routing decisions;
- available capacity;
- alternative route availability; and
- whether additional capacity would increase total delivered throughput.

Therefore, this section identifies **structural exposure to congestion**, not final bottlenecks.

19.1 Convert Each Route Into Its Links

In [21]:
```
def get_path_edges(path_string):
    """
    Convert a route string such as:
    'S1 -> A -> C -> UK'

    into directed links:
    [('S1', 'A'), ('A', 'C'), ('C', 'UK')]
    """

    nodes = [node.strip() for node in path_string.split("->")]
```

```
    return list(zip(nodes[:-1], nodes[1:]))


routes_df["path_edges"] = routes_df["path"].apply(get_path_edges)

routes_df[
    ["source", "destination", "path", "path_edges"]
].head()
```

Out[21]:

	source	destination	path	path_edges
23	B	UK	B -> C -> UK	[(B, C), (C, UK)]
24	B	UK	B -> D -> UK	[(B, D), (D, UK)]
22	B	UK	B -> A -> C -> UK	[(B, A), (A, C), (C, UK)]
26	B	US	B -> C -> US	[(B, C), (C, US)]
27	B	US	B -> D -> US	[(B, D), (D, US)]

19.2 Count Link Usage Across All Feasible Routes

In [22]:

```
from collections import Counter

all_route_link_counter = Counter()

for path_edges in routes_df["path_edges"]:
    all_route_link_counter.update(path_edges)

all_link_usage_df = pd.DataFrame(
    [
        {
            "from_node": u,
            "to_node": v,
            "number_of_feasible_routes": count
        }
        for (u, v), count in all_route_link_counter.items()
    ]
)

all_link_usage_df = (
    all_link_usage_df
    .sort_values(
        "number_of_feasible_routes",
        ascending=False
    )
    .reset_index(drop=True)
)

all_link_usage_df
```

Out[22]:

	from_node	to_node	number_of_feasible_routes
0	B	D	16
1	A	C	12
2	A	B	10
3	C	UK	9
4	C	US	9
5	B	C	8
6	D	UK	8
7	D	US	8
8	S1	A	8
9	S2	A	8
10	C	B	6
11	S3	B	6
12	B	A	4
13	C	A	2

Interpretation of All-Route Link Usage

This table counts how many feasible source-to-exit routes include each directed link.

A larger count means that more candidate routes depend on that link.

This provides a measure of **structural route dependence**.

For example, if the directed link:

$$A \to C$$

appears in many feasible routes, then traffic originating at several sources may potentially compete for that link.

However, a high route count should not be interpreted as evidence that the link is currently congested.

At this stage:

- no demand has been assigned;
- no simultaneous traffic has been routed;

- no optimization has been solved.

The analysis therefore identifies links that are potentially important rather than links that are proven bottlenecks.

19.3 Count Link Usage Only Across Shortest Routes

In [24]:
```
shortest_link_counter = Counter()

for path_edges in shortest_routes_df["path"].apply(get_path_edges):
    shortest_link_counter.update(path_edges)

shortest_link_usage_df = pd.DataFrame(
    [
        {
            "from_node": u,
            "to_node": v,
            "number_of_shortest_routes": count
        }
        for (u, v), count in shortest_link_counter.items()
    ]
)

shortest_link_usage_df = (
    shortest_link_usage_df
    .sort_values(
        "number_of_shortest_routes",
        ascending=False
    )
    .reset_index(drop=True)
)

shortest_link_usage_df
```

Out[24]:

	from_node	to_node	number_of_shortest_routes
0	C	UK	5
1	C	US	5
2	B	C	4
3	B	D	4
4	A	C	4
5	S3	B	4
6	D	UK	2
7	D	US	2
8	S1	A	2
9	S2	A	2

Interpretation of Shortest-Route Link Usage

This analysis considers only the minimum-latency routes identified in the equal-link scenario.

Because every link currently has the same 20 ms latency, these are also the minimum-hop routes.

Links that appear frequently in shortest routes are particularly important because independent shortest-path routing would tend to direct traffic toward them.

For example, if:

$$A \to C$$

appears in several shortest paths, then multiple streams may compete for its 30 Mbps capacity if all traffic independently selects its minimum-latency route.

This helps explain why shortest-path routing alone may perform poorly under shared capacity constraints.

19.4 Merge All-Route and Shortest-Route Counts

In [25]:
```
link_overlap_df = pd.merge(
    all_link_usage_df,
    shortest_link_usage_df,
    on=["from_node", "to_node"],
    how="outer"
```

```
).fillna(0)

link_overlap_df[
    [
        "number_of_feasible_routes",
        "number_of_shortest_routes"
    ]
] = link_overlap_df[
    [
        "number_of_feasible_routes",
        "number_of_shortest_routes"
    ]
].astype(int)

link_overlap_df = (
    link_overlap_df
    .sort_values(
        [
            "number_of_shortest_routes",
            "number_of_feasible_routes"
        ],
        ascending=False
    )
    .reset_index(drop=True)
)

link_overlap_df
```

Out[25]:

	from_node	to_node	number_of_feasible_routes	number_of_shortest_route
0	C	UK	9	
1	C	US	9	
2	B	D	16	
3	A	C	12	
4	B	C	8	
5	S3	B	6	
6	D	UK	8	
7	D	US	8	
8	S1	A	8	
9	S2	A	8	
10	A	B	10	
11	C	B	6	
12	B	A	4	
13	C	A	2	

19.5 Route-Overlap Analysis — Why Structural Exposure Matters

Results Analysis

The route-overlap analysis identifies links that appear frequently across candidate source-to-destination paths.

Links such as:

- A → C;
- B → D;
- destination-facing links into UK and US;

appear repeatedly because they connect major portions of the internal network to the final destination corridors.

Frequent path membership indicates **structural exposure**.

It does not, by itself, prove that a link is a true throughput bottleneck.

Why This Happens

A link can appear in many candidate routes because it occupies a strategically central position in the topology.

However, whether that link becomes capacity-constraining depends on:

- how much traffic is actually admitted;
- which routes the optimizer selects;
- competing capacity elsewhere;
- destination-facing limitations.

Therefore:

$$\boxed{\text{Frequent Route Membership} \neq \text{Confirmed Bottleneck}}$$

A structural corridor may be important without being the marginal constraint on throughput.

Business Impact

Route overlap is useful as an **early-warning indicator**.

Links appearing in many candidate paths deserve attention because failure or degradation could affect many commodities simultaneously.

However, capacity investment should not be based on overlap counts alone.

The later utilization and marginal-capacity analyses are needed to distinguish:

- structurally important links;
- heavily utilized links;
- true throughput bottlenecks.

This creates a stronger infrastructure-planning process than simply ranking links by route frequency.

20. Baseline Routing Policy: Independent Shortest Paths

A useful benchmark is a naive routing policy in which each source-destination pair independently selects a minimum-latency path.

This is not proposed as the final routing solution.

Instead, it provides a baseline against which the globally optimized solution can later be compared.

Under the current exploratory assumption:

$$L_e = 20 \text{ ms}$$

for every link.

Therefore, minimum-latency routing is equivalent to minimum-hop routing.

The independent shortest-path policy does not consider:

- competing stream demand;
- shared link capacity;
- congestion created by other routes;
- preservation of scarce low-latency capacity;
- total network throughput.

It simply selects the lowest-latency route available to each source-destination pair.

20.1 Choose One Baseline Shortest Route

- We have ties for B and S3.
- For the baseline, we should not pretend one tied path is inherently better.
- We can choose the first one deterministically, but explicitly label this as a baseline tie-breaking rule.

In [28]:
```
baseline_routes_df = (
    shortest_routes_df
    .sort_values(
        ["source", "destination", "path"]
    )
    .groupby(
        ["source", "destination"],
        as_index=False
    )
    .first()
)

baseline_routes_df[
    [
        "source",
        "destination",
        "path",
        "num_links",
```

```
        "path_latency_ms",
        "path_capacity_mbps"
    ]
]
```

Out[28]:

	source	destination	path	num_links	path_latency_ms	path_capacity_mbps
0	B	UK	B -> C -> UK	2	40	30
1	B	US	B -> C -> US	2	40	30
2	C	UK	C -> UK	1	20	30
3	C	US	C -> US	1	20	30
4	S1	UK	S1 -> A -> C -> UK	3	60	30
5	S1	US	S1 -> A -> C -> US	3	60	30
6	S2	UK	S2 -> A -> C -> UK	3	60	30
7	S2	US	S2 -> A -> C -> US	3	60	30
8	S3	UK	S3 -> B -> C -> UK	3	60	30
9	S3	US	S3 -> B -> C -> US	3	60	30

Baseline Tie-Breaking Rule

Several source-destination pairs have multiple equally short paths.

For purposes of constructing a deterministic baseline, one tied path is selected using a consistent ordering rule.

This does **not** imply that the selected tied route is superior.

In the final optimization model, tied routes should remain available because the optimizer may use them to distribute traffic and reduce congestion.

20.2 Count How Often the Baseline Uses Each Link

In [30]:
```
baseline_link_counter = Counter()

for path_string in baseline_routes_df["path"]:
    baseline_link_counter.update(
        get_path_edges(path_string)
    )

baseline_link_usage_df = pd.DataFrame(
    [
        {
            "from_node": u,
            "to_node": v,
            "baseline_route_count": count
        }
        for (u, v), count in baseline_link_counter.items()
    ]
)

baseline_link_usage_df = (
    baseline_link_usage_df
    .sort_values(
        "baseline_route_count",
        ascending=False
    )
    .reset_index(drop=True)
)

baseline_link_usage_df
```

Out[30]:

	from_node	to_node	baseline_route_count
0	C	UK	5
1	C	US	5
2	B	C	4
3	A	C	4
4	S1	A	2
5	S2	A	2
6	S3	B	2

Interpretation of the Independent Shortest-Path Baseline

This table shows how often each directed link would be selected if every source-destination pair independently chose one shortest route.

Links with high baseline-route counts are potential areas of concentration.

This illustrates an important limitation of independent shortest-path routing:

each routing decision is locally reasonable, but the collection of locally optimal decisions may overload the same shared link.

The eventual optimization model will instead consider all four traffic streams simultaneously.

20.3 Independent Shortest-Route Baseline — Results Analysis and Business Impact

Results Analysis

The independent routing baseline demonstrates what happens when each commodity selects its locally attractive route without considering the traffic generated by other commodities.

Individually, these route choices may appear reasonable.

Collectively, however, they create severe capacity violations on shared infrastructure.

The result demonstrates that route optimization cannot be performed independently by commodity when multiple traffic classes compete for the same

network resources.

Why This Happens

Each commodity sees only its own preferred path.

It does not account for the fact that several other commodities may simultaneously select the same links.

Therefore, individually rational route choices can produce a globally infeasible solution.

This is a classic shared-resource problem:

$$\boxed{\text{Local Route Optimality} \neq \text{Global Network Feasibility}}$$

The later optimization models resolve this issue by enforcing shared capacity jointly across all commodities.

Business Impact

A production routing system should not independently send every traffic class over its shortest or lowest-latency route.

Doing so can create:

- severe congestion;
- infeasible link loading;
- avoidable service loss;
- unstable routing decisions.

The business requirement is therefore not simply to find good paths.

It is to allocate a shared network resource across competing traffic simultaneously.

21. Final Splittable-Flow LP Formulation

The splittable-flow variant allows each source-stream demand to be divided across multiple feasible paths.

Because all path-flow decision variables are continuous, this formulation is a linear program.

Demand is modeled separately for each source-stream pair, consistent with the confirmed requirement that the amount of each traffic stream entering at each source node is known.

The routing unit is therefore:

$$(k, s)$$

where:

- k identifies the traffic stream;
- s identifies the source node.

21.1 Sets

Let:

V = set of network nodes

E = set of directed network arcs

K = set of traffic streams

$$K = \{UK\text{-}L,\ UK\text{-}T,\ US\text{-}L,\ US\text{-}T\}$$

K_L = set of latency-sensitive traffic streams

$$K_L = \{UK\text{-}L,\ US\text{-}L\}$$

K_T = set of latency-tolerant traffic streams

$$K_T = \{UK\text{-}T,\ US\text{-}T\}$$

S_k = set of source nodes generating stream k

$P_{k,s}$ = set of feasible paths from source s to the required destination of stream k

---

21.2 Parameters

For each directed arc e:

$$C_e = \text{capacity of arc } e \text{ in Mbps}$$

$$L_e = \text{latency of arc } e \text{ in milliseconds}$$

For each stream k and source s:

$$D_{k,s}$$

represents the current demand of stream k entering the network at source s.

For each feasible path p:

$$L_p = \sum_{e \in p} L_e$$

represents the end-to-end latency of that path.

21.3 Decision Variables

For every stream k, source s, and feasible path p:

$$f_{k,s,p} \geq 0$$

represents the amount of traffic, in Mbps, routed for source-stream pair (k, s) over path p.

Because this is the splittable-flow formulation, more than one path may have positive flow for the same source-stream pair.

Delivered traffic for source-stream pair (k, s) is:

$$z_{k,s} = \sum_{p \in P_{k,s}} f_{k,s,p}$$

21.4 Demand Constraint

No source-stream pair may deliver more traffic than its current demand.

For every stream k and source s:

$$0 \leq z_{k,s} \leq D_{k,s}$$

Equivalently:

$$\sum_{p \in P_{k,s}} f_{k,s,p} \leq D_{k,s}$$

Undelivered demand is:

$$\text{Lost}_{k,s} = D_{k,s} - z_{k,s}$$

Partial delivery is allowed.

Therefore:

$$0 < z_{k,s} < D_{k,s}$$

is permitted when network capacity is insufficient to deliver the full demand.

21.5 Path Allocation Constraint

The delivered quantity for a source-stream pair equals the sum of the flow assigned to all feasible paths available to that source-stream pair.

Therefore:

$$z_{k,s} = \sum_{p \in P_{k,s}} f_{k,s,p}$$

Because traffic is splittable, several values of:

$$f_{k,s,p}$$

may be positive simultaneously.

For example, part of S1 / UK-L may use one path while another portion uses a different path.

21.6 Link Capacity Constraint

All traffic using the same directed arc must share that arc's available bandwidth.

For every directed arc e:

$$\sum_{k \in K} \sum_{s \in S_k} \sum_{\substack{p \in P_{k,s} \\ e \in p}} f_{k,s,p} \leq C_e$$

This constraint combines traffic from all four traffic classes and all source nodes.

Because the network is full-duplex, the capacity of:

$$A \to C$$

is enforced independently from the capacity of:

$$C \to A$$

21.7 Destination Correctness

Every feasible path in:

$$P_{k,s}$$

terminates at the required exit for stream k.

Therefore:

- UK-L and UK-T paths terminate only at UK;
- US-L and US-T paths terminate only at US.

Exit nodes are treated as terminal nodes.

Once traffic reaches UK or US, it leaves the network and is not permitted to continue through that exit to another destination.

21.8 Stage 1 Objective: Maximize Delivered Throughput

The first-stage objective is to maximize the total traffic successfully delivered across all source-stream pairs.

$$\boxed{\max \sum_{k \in K} \sum_{s \in S_k} z_{k,s}}$$

Using the path-flow variables directly, this is equivalent to:

$$\boxed{\max \sum_{k \in K} \sum_{s \in S_k} \sum_{p \in P_{k,s}} f_{k,s,p}}$$

Let the optimal delivered amount for each source-stream pair be:

$$z^*_{k,s}$$

and let total optimal throughput be:

$$T^* = \sum_{k \in K} \sum_{s \in S_k} z^*_{k,s}$$

All delivered Mbps are weighted equally in this stage.

Therefore, latency-sensitive traffic does not automatically receive higher admission priority than tolerant traffic.

21.9 Stage 2: Preserve the Stage 1 Admission Decision

After maximum throughput has been determined, the amount delivered for each source-stream pair is fixed at its Stage 1 value.

For every stream k and source s:

$$z_{k,s} = z^*_{k,s}$$

or equivalently:

$$\sum_{p \in P_{k,s}} f_{k,s,p} = z^*_{k,s}$$

This is stronger than fixing only total throughput:

$$\sum_{k,s} z_{k,s} = T^*$$

because fixing only total throughput could allow Stage 2 to replace latency-sensitive delivered traffic with latency-tolerant traffic simply to reduce the latency objective.

Fixing each source-stream delivered quantity ensures that Stage 2 changes only the routing decision, not which demand was admitted.

21.10 Stage 2 Objective: Minimize Sensitive-Traffic Latency

After preserving the Stage 1 delivered quantities, the second-stage objective minimizes traffic-weighted latency for the latency-sensitive streams:

$$K_L = \{UK\text{-}L,\ US\text{-}L\}$$

The objective is:

$$\boxed{\min \sum_{k \in K_L} \sum_{s \in S_k} \sum_{p \in P_{k,s}} L_p f_{k,s,p}}$$

where:

$$L_p = \sum_{e \in p} L_e$$

This objective minimizes total latency exposure measured in:

$$\text{Mbps} \times \text{milliseconds}$$

among the already-admitted latency-sensitive traffic.

Because delivered quantities are fixed from Stage 1, minimizing this quantity is equivalent to minimizing traffic-weighted average latency for the latency-sensitive traffic.

21.11 Reported Latency Metrics

For each latency-sensitive stream k, the traffic-weighted average latency will be reported as:

$$\bar{L}_k = \frac{\sum_{s \in S_k} \sum_{p \in P_{k,s}} L_p f_{k,s,p}}{\sum_{s \in S_k} \sum_{p \in P_{k,s}} f_{k,s,p}}$$

Worst-case used-path latency will also be reported:

$$L_k^{\max} = \max_{\substack{s,p \\ f_{k,s,p} > 0}} L_p$$

The weighted-average metric reflects the latency experienced across all delivered traffic.

The worst-case metric is reported separately so that a small quantity of traffic assigned to a substantially slower path is not hidden by the average.

21.12 Why Linear Programming Is Appropriate

The splittable-flow formulation is a linear program because:

- all path-flow variables are continuous;

- the throughput objective is linear;
- the latency objective is linear;
- demand constraints are linear;
- link-capacity constraints are linear.

No binary path-selection variables are required because traffic may be divided among multiple feasible paths.

22. Final Single-Path MILP Formulation

The single-path variant restricts each source-stream pair to at most one feasible route.

Unlike the splittable-flow formulation, traffic associated with one source-stream pair cannot be divided across several paths.

For example, if S1 generates US-L traffic, all delivered S1 / US-L traffic must use the same selected path.

However, a different source contributing to the same traffic class may select a different route.

For example:

- S1 / US-L may use one path;
- B / US-L may use another path.

Partial delivery is allowed.

Therefore, selecting a path does **not** require the entire source-stream demand to be delivered.

Because the model must make a discrete path-selection decision, the formulation is a **mixed-integer linear program (MILP)**.

22.1 Sets

Let:

V = set of network nodes

E = set of directed network arcs

K = set of traffic streams

$$K = \{UK\text{-}L,\ UK\text{-}T,\ US\text{-}L,\ US\text{-}T\}$$

K_L = set of latency-sensitive traffic streams

$$K_L = \{UK\text{-}L,\ US\text{-}L\}$$

S_k = set of source nodes generating stream k

$P_{k,s}$ = set of feasible paths from source s to the required destination of stream k

---

22.2 Parameters

For each directed arc e:

$$C_e = \text{capacity of arc } e \text{ in Mbps}$$

$$L_e = \text{latency of arc } e \text{ in milliseconds}$$

For each stream k and source s:

$$D_{k,s}$$

represents the current demand of stream k entering the network at source s.

For each feasible path p:

$$L_p = \sum_{e \in p} L_e$$

represents the end-to-end latency of that path.

22.3 Continuous Flow Variable

For every stream k, source s, and feasible path p:

$$f_{k,s,p} \geq 0$$

represents the amount of traffic, in Mbps, routed over path p.

Unlike the splittable model, only one path may carry positive flow for a given source-stream pair.

22.4 Binary Path-Selection Variable

Define:

$$y_{k,s,p} \in \{0,1\}$$

where:

$$y_{k,s,p} = 1$$

if source-stream pair (k,s) selects path p, and:

$$y_{k,s,p} = 0$$

otherwise.

The binary variable determines which path is available to carry the delivered traffic.

22.5 Single-Path Constraint

Each source-stream pair may select at most one feasible path.

Therefore:

$$\boxed{\sum_{p \in P_{k,s}} y_{k,s,p} \leq 1}$$

for every stream k and source s.

This means that delivered traffic for a source-stream pair cannot be divided among several routes.

Different source-stream pairs remain free to select different paths.

22.6 Flow-to-Path Selection Constraint

Traffic may flow over a path only if that path has been selected.

For every stream k, source s, and path p:

$$\boxed{f_{k,s,p} \leq D_{k,s} y_{k,s,p}}$$

If:

$$y_{k,s,p} = 0$$

then:

$$f_{k,s,p} = 0$$

If:

$$y_{k,s,p} = 1$$

then flow may take any value satisfying:

$$0 \le f_{k,s,p} \le D_{k,s}$$

This formulation allows partial delivery.

For example, if:

$$D_{k,s} = 10 \text{ Mbps}$$

but the selected path has only 6 Mbps of usable residual capacity, the model may deliver:

$$6 \text{ Mbps}$$

and count:

$$4 \text{ Mbps}$$

as lost.

The demand does not need to be accepted or rejected as an all-or-nothing quantity.

22.7 Delivered Traffic

Delivered traffic for source-stream pair (k, s) is:

$$z_{k,s} = \sum_{p \in P_{k,s}} f_{k,s,p}$$

Because at most one path can be selected, at most one term in this sum can carry positive flow.

22.8 Demand Constraint

No source-stream pair may deliver more traffic than its available demand.

Therefore:

$$0 \le z_{k,s} \le D_{k,s}$$

or equivalently:

$$\sum_{p \in P_{k,s}} f_{k,s,p} \le D_{k,s}$$

Undelivered demand is:

$$\text{Lost}_{k,s} = D_{k,s} - z_{k,s}$$

22.9 Link Capacity Constraint

All traffic routed over the same directed arc must share that arc's available capacity.

For each directed arc e:

$$\boxed{\sum_{k \in K} \sum_{s \in S_k} \sum_{\substack{p \in P_{k,s} \\ e \in p}} f_{k,s,p} \le C_e}$$

This constraint is identical in principle to the splittable-flow model.

The difference is that the single-path model places additional restrictions on which path flows are allowed to be positive.

22.10 Destination Correctness

Every candidate path in:

$$P_{k,s}$$

must terminate at the required destination of stream k.

Therefore:

- UK-L and UK-T paths terminate at UK;

- US-L and US-T paths terminate at US.

UK and US are terminal exits.

Traffic may not reach one exit and then continue through that exit toward the other destination.

22.11 Stage 1 Objective: Maximize Delivered Throughput

The first-stage objective is to maximize total delivered traffic:

$$\boxed{\max \sum_{k \in K} \sum_{s \in S_k} z_{k,s}}$$

Using path-flow variables directly:

$$\boxed{\max \sum_{k \in K} \sum_{s \in S_k} \sum_{p \in P_{k,s}} f_{k,s,p}}$$

Let the optimal delivered quantity for source-stream pair (k, s) be:

$$z^*_{k,s}$$

and total optimal single-path

23. Validation and Stress Test Plan

The optimization models should be validated against controlled scenarios before the final results are interpreted.

The purpose of validation is to confirm that the model behaves consistently with the stated requirements and the clarified assumptions.

Each test case includes:

- the condition being tested;
- the expected behavior;
- the type of implementation or modeling error the test could reveal.

The same validation philosophy will be applied to both:

- the splittable-flow LP;

- the single-path MILP.

Where appropriate, the two formulations will be compared under identical inputs.

23.1 Test Case A - Low Demand / Uncongested Network

Purpose

Verify that the model delivers all available demand when sufficient capacity exists throughout the network.

Test Condition

Use demand values small enough that no directed link becomes capacity constrained.

Expected Results

- all source-stream demand is delivered;
- lost demand equals zero;
- no link exceeds capacity;
- Stage 1 throughput equals total demand;
- latency-sensitive traffic uses minimum-latency paths during Stage 2, subject to ties.

Therefore:

$$T^* = \sum_k \sum_s D_{k,s}$$

and:

$$\text{Lost}_{k,s} = 0$$

for every source-stream pair.

Failure Indicators

The model is incorrect if:

- traffic is unnecessarily lost;
- delivered traffic exceeds demand;
- any link exceeds its capacity;

- Stage 2 reduces a source-stream's Stage 1 delivered quantity.

23.2 Test Case B - Demand Greater Than Available Capacity

Purpose

Verify that the model correctly handles a congested network in which all demand cannot be delivered.

Test Condition

Increase source-stream demand until one or more network corridors become capacity constrained.

Expected Results

- some demand may be partially delivered;
- some demand may be lost;
- no directed link exceeds capacity;
- delivered traffic never exceeds source-stream demand;
- Stage 1 maximizes total delivered throughput;
- unrouted traffic is counted as lost rather than queued or carried forward.

For each source-stream pair:

$$0 \leq z_{k,s} \leq D_{k,s}$$

and:

$$\text{Lost}_{k,s} = D_{k,s} - z_{k,s}$$

Failure Indicators

The model is incorrect if:

- all demand is reported as delivered despite insufficient network capacity;
- any link exceeds capacity;
- lost traffic is implicitly stored for a later period;
- delivered traffic exceeds the source-stream demand.

23.3 Test Case C - Fast but Capacity-

Constrained Route

Purpose

Test the interaction between throughput and latency.

Test Condition

Construct a scenario with:

- one low-latency route with limited capacity;
- one higher-latency route with greater available capacity.

Expected Results

Stage 1 should use whatever route combination maximizes total delivered throughput.

Stage 2 should then preserve the Stage 1 delivered quantities while moving latency-sensitive traffic toward lower-latency routes where capacity permits.

The model should therefore distinguish between:

- throughput optimization;
- latency optimization.

Failure Indicators

The model may be incorrect if:

- Stage 1 sacrifices total throughput only to obtain lower latency;
- Stage 2 changes the amount of traffic admitted instead of only changing routing;
- tolerant traffic receives an artificial latency penalty not specified by the problem.

23.4 Test Case D - Shared Bottleneck

Purpose

Verify that traffic from multiple source-stream pairs correctly shares the capacity of a common directed link.

Expected Results

For every directed link e:

$$\sum_{k} \sum_{s} \sum_{\substack{p \in P_{k,s} \\ e \in p}} f_{k,s,p} \leq C_e$$

must always hold.

Several commodities may use the same link, but their combined traffic cannot exceed its capacity.

Failure Indicator

The capacity model is incorrect if each stream or source-stream pair effectively receives the full capacity of the same link independently.

23.5 Test Case E - Split vs. Single-Path Routing

Purpose

Measure the value of allowing a source-stream demand to use multiple routes.

Expected Relationship

Under identical network and demand data:

$$T^*_{\text{single}} \leq T^*_{\text{split}}$$

because the splittable model has a less restrictive feasible region.

Every single-path solution is also feasible in the splittable model, but a splittable solution may use multiple paths for the same source-stream pair and therefore may not be feasible in the single-path model.

Expected Results

The two models may produce identical throughput when one route is sufficient.

Under congestion, the splittable formulation may deliver more traffic by combining residual capacity across multiple paths.

Failure Indicator

If:

$$T^*_{\text{single}} > T^*_{\text{split}}$$

under identical inputs and correctly solved models, there is likely an implementation or formulation error.

23.6 Test Case F - Equal-Latency Alternative Routes

Purpose

Verify that the model can use alternative routes with identical latency to reduce congestion.

Test Condition

Create two or more feasible paths with the same end-to-end latency.

Expected Results

If two paths have equal latency, the optimizer should be free to select whichever route best preserves network capacity elsewhere.

For the splittable model, traffic may also be divided across equal-latency paths.

The model should not impose an artificial preference between equally good latency alternatives unless another network constraint makes one route preferable.

23.7 Test Case G - Capacity Sensitivity and True Bottlenecks

Purpose

Distinguish a fully utilized link from a true throughput-constraining bottleneck.

Procedure

For each directed link e:

1. solve the baseline Stage 1 throughput problem;

2. record optimal throughput T^*;
3. increase the capacity of link e by 1 Mbps;
4. re-solve the throughput problem;
5. compute:

$$\Delta_e = T^*(C_e + 1) - T^*(C_e)$$

Interpretation

If:

$$\Delta_e > 0$$

then additional capacity on link e improves total network throughput.

The link is therefore a meaningful throughput bottleneck.

If:

$$\Delta_e = 0$$

then the link may be saturated, but additional capacity at that link alone does not improve end-to-end throughput.

Another constraint is limiting the network.

Expected Results

The model should distinguish:

- high utilization;
- saturation;
- positive marginal throughput value.

These are related but not identical concepts.

23.8 Test Case H - Partial Delivery in the Single-Path Model

Purpose

Verify the clarified requirement that single-path routing does not imply all-or-nothing demand admission.

Test Condition

Create a source-stream pair with demand larger than the usable residual capacity of every available single path.

For example:

$$D_{k,s} = 10 \text{ Mbps}$$

while the selected path has only:

$$6 \text{ Mbps}$$

of usable capacity.

Expected Results

The model should be allowed to deliver:

$$z_{k,s} = 6$$

and report:

$$\text{Lost}_{k,s} = 4$$

while using only one selected path.

Failure Indicators

The model is incorrect if:

- it forces the entire 10 Mbps to be rejected;
- it splits the 6 Mbps across multiple paths;
- it exceeds the selected path's residual capacity.

23.8 Test Case H - Partial Delivery in the Single-Path Model

Purpose

Verify the clarified requirement that single-path routing does not imply all-or-nothing demand admission.

Test Condition

Create a source-stream pair with demand larger than the usable residual capacity of every available single path.

For example:

$$D_{k,s} = 10 \text{ Mbps}$$

while the selected path has only:

$$6 \text{ Mbps}$$

of usable capacity.

Expected Results

The model should be allowed to deliver:

$$z_{k,s} = 6$$

and report:

$$\text{Lost}_{k,s} = 4$$

while using only one selected path.

Failure Indicators

The model is incorrect if:

- it forces the entire 10 Mbps to be rejected;
- it splits the 6 Mbps across multiple paths;
- it exceeds the selected path's residual capacity.

23.9 Test Case I - Single Path Per Source-Stream Pair

Purpose

Verify that the single-path rule is applied at the correct level.

Test Condition

Use the same traffic class at multiple source nodes.

For example:

- S1 / US-L;
- B / US-L.

Expected Results

S1 / US-L may select one path while B / US-L selects a different path.

For each source-stream pair:

$$\sum_{p \in P_{k,s}} y_{k,s,p} \leq 1$$

must hold.

However, different sources contributing to the same aggregate stream are not required to use the same route.

Failure Indicator

The model is incorrectly formulated if all US-L traffic across all sources is forced onto one common path.

23.10 Test Case J - Correct Exit Enforcement

Purpose

Verify that traffic can leave the network only through its designated destination.

Expected Results

UK-bound streams:

- UK-L;
- UK-T;

must terminate only at UK.

US-bound streams:

- US-L;
- US-T;

must terminate only at US.

Traffic must not pass through the opposite exit and continue through the network.

Failure Indicators

The model is incorrect if:

- UK-bound traffic exits through US;
- US-bound traffic exits through UK;
- an exit node is used as an intermediate transit node.

23.11 Test Case K - No Queueing or Carryover

Purpose

Verify that each optimization is treated as an independent snapshot.

Test Condition

Create demand that exceeds the traffic that can be delivered in the current snapshot.

Expected Results

Unrouted demand is immediately counted as lost.

The model should contain no variables representing:

- queues;
- waiting time;
- backlog;
- future delivery;
- carryover.

Failure Indicator

The model is inconsistent with the requirements if undelivered traffic is implicitly preserved for later delivery.

23.12 Test Case L - Stage 2 Admission Preservation

Purpose

Verify that latency minimization changes routing but does not change the throughput allocation established in Stage 1.

Procedure

1. Solve Stage 1.
2. Record each optimal delivered quantity:

$$z^*_{k,s}$$

3. Solve Stage 2 with:

$$z_{k,s} = z^*_{k,s}$$

for every source-stream pair.

Expected Results

For every (k, s):

$$z^{\text{Stage 2}}_{k,s} = z^{\text{Stage 1}}_{k,s}$$

Total throughput must also remain:

$$T_{\text{Stage 2}} = T_{\text{Stage 1}}$$

Only the route assignment should change.

Failure Indicator

If Stage 2 reduces one sensitive source-stream's delivered traffic and replaces it with tolerant traffic, the Stage 2 constraints are too weak.

23.13 Test Case M - Split-Traffic Latency Reporting

Purpose

Verify the selected latency-reporting method for splittable traffic.

Test Condition

Route a latency-sensitive source-stream pair over two paths with different latencies.

For example:

- 8 Mbps over a 40 ms path;

- 2 Mbps over an 80 ms path.

Expected Weighted-Average Latency

$$\bar{L} = \frac{8(40) + 2(80)}{10}$$

$$\bar{L} = 48 \text{ ms}$$

Expected Worst-Case Latency

$$L^{\max} = 80 \text{ ms}$$

Both metrics should be reported.

Why Both Matter

The weighted average reflects the latency experienced by the traffic overall.

The worst-case measure prevents a small amount of substantially slower traffic from being hidden inside the average.

23.14 Test Case N - Zero Demand

Purpose

Verify that a source-stream pair with zero current demand does not receive unnecessary flow.

Expected Results

If:

$$D_{k,s} = 0$$

then:

$$z_{k,s} = 0$$

and:

$$f_{k,s,p} = 0$$

for every path p.

Failure Indicator

Any positive routed flow for a zero-demand commodity indicates an implementation error.

23.15 Validation Summary

The validation plan tests four different dimensions of correctness.

Feasibility

- demand limits;
- link-capacity limits;
- destination correctness;
- single-path restrictions.

Business Logic

- lost traffic is not queued;
- partial delivery is allowed;
- node capacity is unconstrained;
- latency-sensitive traffic does not receive automatic admission priority.

Optimization Logic

- Stage 1 maximizes throughput;
- Stage 2 preserves the Stage 1 admission decision;
- Stage 2 improves latency-sensitive routing;
- split flow cannot perform worse than single-path flow under identical assumptions.

Sensitivity and Robustness

- bottleneck identification;
- capacity perturbation;
- equal-latency alternatives;
- congestion;
- zero demand;
- partial delivery.

Passing these tests provides evidence that the model behaves correctly before conclusions are drawn from the final synthetic scenarios.

24. Modeling Decision Log

The following table records the major modeling decisions used in the final formulation.

The purpose of this section is to distinguish among:

- requirements explicitly confirmed in the problem statement;
- requirements clarified by Prateek;
- analyst-selected modeling choices;
- implementation choices used for testing and evaluation.

This provides a transparent record of why each modeling decision was made.

In [45]:
```
decision_log = pd.DataFrame([
    {
        "decision": "Objective hierarchy",
        "current_choice": "Maximize throughput first, then minimize la
        "status": "Final modeling choice",
        "reason": "Avoids assigning an arbitrary numerical weight betw
    },
    {
        "decision": "Stage 2 admission preservation",
        "current_choice": "Preserve the Stage 1 delivered amount for e
        "status": "Final modeling choice",
        "reason": "Prevents the latency objective from changing which
    },
    {
        "decision": "Demand indexing",
        "current_choice": "Model demand separately for each source-str
        "status": "Confirmed",
        "reason": "Prateek confirmed that input demand is available fo
    },
    {
        "decision": "Queueing",
        "current_choice": "No queueing, buffering, or node waiting",
        "status": "Confirmed",
        "reason": "Each optimization is an independent snapshot and qu
    },
    {
        "decision": "Traffic carryover",
        "current_choice": "No carryover to future snapshots",
        "status": "Confirmed",
        "reason": "Traffic that cannot be routed in the current snapsh
    },
    {
        "decision": "Node capacity",
```

```
        "current_choice": "No independent node-capacity constraint",
        "status": "Confirmed",
        "reason": "Prateek confirmed that nodes are unconstrained forw
    },
    {
        "decision": "Link directionality",
        "current_choice": "Represent each full-duplex physical link as
        "status": "Confirmed / implementation choice",
        "reason": "Capacity is directional and traffic in opposite dir
    },
    {
        "decision": "Unrouted demand",
        "current_choice": "Count unrouted demand as lost",
        "status": "Confirmed",
        "reason": "Unrouted traffic is not buffered or carried into a
    },
    {
        "decision": "Admission priority",
        "current_choice": "No automatic latency-sensitive-first admiss
        "status": "Confirmed",
        "reason": "All delivered traffic is weighted equally for throu
    },
    {
        "decision": "Split-sensitive latency metric",
        "current_choice": "Optimize traffic-weighted latency and repor
        "status": "Final modeling choice",
        "reason": "Weighted latency reflects the experience of deliver
    },
    {
        "decision": "Single-path indexing",
        "current_choice": "One selected path per source-stream pair",
        "status": "Confirmed",
        "reason": "Prateek confirmed that each source's portion of a s
    },
    {
        "decision": "Partial delivery in single-path model",
        "current_choice": "Partial delivery is allowed",
        "status": "Confirmed",
        "reason": "A selected path may carry only the amount that fits
    },
    {
        "decision": "Numerical input data",
        "current_choice": "Construct a documented synthetic dataset",
        "status": "Confirmed",
        "reason": "No numerical dataset will be provided and Prateek r
    },
    {
        "decision": "Synthetic link characteristics",
        "current_choice": "Use heterogeneous capacities and latencies
        "status": "Final test-design choice",
        "reason": "Creates meaningful routing, congestion, and capacit
    },
```

```
    {
        "decision": "Path representation",
        "current_choice": "Use path-based optimization variables",
        "status": "Implementation choice",
        "reason": "Makes split routing, single-path selection, latency
    },
    {
        "decision": "Bottleneck definition",
        "current_choice": "Distinguish utilization from marginal throu
        "status": "Final modeling choice",
        "reason": "A saturated link is not necessarily a true bottlene
    },
    {
        "decision": "Bottleneck sensitivity method",
        "current_choice": "Increase each directed link capacity by 1 M
        "status": "Final analysis choice",
        "reason": "Provides a direct and interpretable measure of marg
    }
])

decision_log
```

Out[45]:

	decision	current_choice	status	reason
0	Objective hierarchy	Maximize throughput first, then minimize laten...	Final modeling choice	Avoids assigning an arbitrary numerical weight...
1	Stage 2 admission preservation	Preserve the Stage 1 delivered amount for each...	Final modeling choice	Prevents the latency objective from changing w...
2	Demand indexing	Model demand separately for each source-stream...	Confirmed	Prateek confirmed that input demand is availab...
3	Queueing	No queueing, buffering, or node waiting	Confirmed	Each optimization is an independent snapshot a...
4	Traffic carryover	No carryover to future snapshots	Confirmed	Traffic that cannot be routed in the current s...
5	Node capacity	No independent node-capacity constraint	Confirmed	Prateek confirmed that nodes are unconstrained...
6	Link directionality	Represent each full-duplex physical link as tw...	Confirmed / implementation choice	Capacity is directional and traffic in opposit...
7	Unrouted demand	Count unrouted demand as lost	Confirmed	Unrouted traffic is not buffered or carried in...

8	Admission priority	No automatic latency-sensitive-first admission...	Confirmed	All delivered traffic is weighted equally for ...
9	Split-sensitive latency metric	Optimize traffic-weighted latency and report b...	Final modeling choice	Weighted latency reflects the experience of de...
10	Single-path indexing	One selected path per source-stream pair	Confirmed	Prateek confirmed that each source's portion o...
11	Partial delivery in single-path model	Partial delivery is allowed	Confirmed	A selected path may carry only the amount that...
12	Numerical input data	Construct a documented synthetic dataset	Confirmed	No numerical dataset will be provided and Prat...
13	Synthetic link characteristics	Use heterogeneous capacities and latencies acr...	Final test-design choice	Creates meaningful routing, congestion, and ca...
14	Path representation	Use path-based optimization variables	Implementation choice	Makes split routing, single-path selection, la...
15	Bottleneck definition	Distinguish utilization from marginal throughp...	Final modeling choice	A saturated link is not necessarily a true bot...
16	Bottleneck sensitivity method	Increase each directed link capacity by 1 Mbps...	Final analysis choice	Provides a direct and interpretable measure of...

24.1 Interpretation of the Decision Log

The decision log shows that the major requirement ambiguities have now been resolved.

Several items are directly confirmed requirements, including:

- source-level demand;
- no queueing or carryover;
- unconstrained forwarding nodes;
- partial delivery;
- one-path-per-source-stream behavior;
- no automatic latency-sensitive admission priority;
- construction of a synthetic dataset.

Other items remain deliberate modeling choices, including:

- the lexicographic objective structure;
- preserving source-stream admission quantities between Stage 1 and Stage 2;
- traffic-weighted latency as the optimization measure;
- reporting worst-case latency separately;
- the design of the synthetic dataset;
- the use of direct capacity perturbation for bottleneck analysis.

This distinction is important because it separates what the problem **requires** from what the analysis **chooses** in order to implement and evaluate the problem.

25. Planned Optimization Outputs

The final optimization results will be reported at multiple levels so that both the mathematical behavior of the model and the operational implications of the routing decisions are visible.

The reporting structure is defined before solving the models so that the splittable-flow LP and single-path MILP can be evaluated consistently.

Results will be produced at five levels:

1. **Source-stream level** - shows how each individual demand contribution is served.
2. **Aggregate stream level** - summarizes results for UK-L, UK-T, US-L, and US-T.
3. **Route level** - shows which paths actually carry traffic.
4. **Link level** - shows bandwidth usage, utilization, and bottlenecks.
5. **Model comparison level** - compares splittable and single-path performance.

This structure also makes it possible to distinguish between:

- demand admission;
- routing decisions;
- latency performance;
- network utilization;
- bottleneck effects.

25.1 Source-Stream Results Template

Because demand is defined separately for each source-stream pair, the most

detailed output will report results at that same level.

For each source-stream pair, the model will report:

- source node;
- traffic stream;
- required destination;
- original demand;
- delivered demand;
- lost demand;
- delivery percentage;
- path or paths used;
- weighted-average latency;
- worst-case latency.

This table is particularly important because the same aggregate traffic stream may experience different routing outcomes depending on where the traffic enters the network.

In [46]:
```
commodity_results_template = pd.DataFrame(columns=[
    "model",
    "source",
    "stream",
    "destination",
    "demand_mbps",
    "delivered_mbps",
    "lost_mbps",
    "delivery_percentage",
    "route_or_routes",
    "average_latency_ms",
    "worst_case_latency_ms"
])

commodity_results_template
```

Out[46]:

model	source	stream	destination	demand_mbps	delivered_mbps	lost_mb

25.2 Aggregate Stream-Level Results Template

Source-stream results will also be aggregated into the four traffic classes:

- UK-L;
- UK-T;
- US-L;

- US-T.

For each stream, aggregate demand is:

$$D_k = \sum_{s \in S_k} D_{k,s}$$

Aggregate delivered traffic is:

$$z_k = \sum_{s \in S_k} z_{k,s}$$

Aggregate lost demand is:

$$\text{Lost}_k = D_k - z_k$$

The aggregate table provides the higher-level view requested by the problem while preserving source-level detail in the previous table.

In [47]:
```
stream_results_template = pd.DataFrame(columns=[
    "model",
    "stream",
    "destination",
    "demand_mbps",
    "delivered_mbps",
    "lost_mbps",
    "delivery_percentage",
    "average_latency_ms",
    "worst_case_latency_ms"
])

stream_results_template
```

Out[47]:

model	stream	destination	demand_mbps	delivered_mbps	lost_mbps	deliv

25.3 Route-Level Results Template

A route-level output will identify the paths that actually carry traffic after optimization.

For each used path, the model will report:

- model type;
- source;
- stream;
- destination;

- path;
- path flow;
- path capacity;
- path latency.

For the splittable model, a source-stream pair may appear on several rows because multiple paths may carry positive flow.

For the single-path model, each source-stream pair should have at most one route with positive flow.

This table directly verifies the routing behavior of the two formulations.

In [48]:
```
route_results_template = pd.DataFrame(columns=[
    "model",
    "source",
    "stream",
    "destination",
    "path",
    "flow_mbps",
    "path_capacity_mbps",
    "path_latency_ms"
])

route_results_template
```

Out[48]:

model	source	stream	destination	path	flow_mbps	path_capacity_mbps	p

25.4 Link-Level Results Template

For every directed link, the final solution will report:

- available capacity;
- assigned traffic flow;
- residual capacity;
- utilization percentage;
- whether the link is saturated;
- marginal throughput value of additional capacity;
- whether the link qualifies as a true throughput bottleneck.

Residual capacity is:

$$R_e = C_e - F_e$$

where:

- C_e is link capacity;
- F_e is total flow assigned to the link.

Utilization is:

$$U_e = \frac{F_e}{C_e} \times 100$$

A saturated link satisfies approximately:

$$F_e = C_e$$

However, saturation alone does not prove that the link is a true bottleneck.

A link is classified as a throughput bottleneck when increasing its capacity produces:

$$\Delta_e > 0$$

In [49]:
```
link_results_template = pd.DataFrame(columns=[
    "model",
    "from_node",
    "to_node",
    "capacity_mbps",
    "flow_mbps",
    "residual_capacity_mbps",
    "utilization_percentage",
    "is_saturated",
    "marginal_throughput_value_mbps",
    "is_true_bottleneck"
])

link_results_template
```

Out[49]:

model	from_node	to_node	capacity_mbps	flow_mbps	residual_capacity_ml

25.5 Model Comparison Template

The splittable-flow and single-path formulations will be compared using identical network and demand inputs.

The comparison will focus on:

- total delivered throughput;
- total lost demand;
- delivery percentage;

- latency-sensitive performance;
- link saturation;
- bottleneck behavior;
- routing flexibility.

The comparison will help quantify the operational value of allowing traffic to split across multiple paths.

In [51]:
```
model_comparison_template = pd.DataFrame([
    {
        "metric": "Total Demand (Mbps)",
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "Total Delivered (Mbps)",
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "Total Lost (Mbps)",
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "Delivery Percentage",
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "UK-L Delivered (Mbps)",
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "US-L Delivered (Mbps)",
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "Average UK-L Latency (ms)",
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "Average US-L Latency (ms)",
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "Worst UK-L Latency (ms)",
```

```
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "Worst US-L Latency (ms)",
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "Number of Saturated Directed Links",
        "split_flow": None,
        "single_path": None
    },
    {
        "metric": "Number of True Bottleneck Links",
        "split_flow": None,
        "single_path": None
    }
])

model_comparison_template
```

Out[51]:

	metric	split_flow	single_path
0	Total Demand (Mbps)	None	None
1	Total Delivered (Mbps)	None	None
2	Total Lost (Mbps)	None	None
3	Delivery Percentage	None	None
4	UK-L Delivered (Mbps)	None	None
5	US-L Delivered (Mbps)	None	None
6	Average UK-L Latency (ms)	None	None
7	Average US-L Latency (ms)	None	None
8	Worst UK-L Latency (ms)	None	None
9	Worst US-L Latency (ms)	None	None
10	Number of Saturated Directed Links	None	None
11	Number of True Bottleneck Links	None	None

25.6 Throughput Benefit of Splittable Routing

The difference in maximum throughput between the two routing formulations will be calculated as:

$$\Delta T_{\text{split}} = T^*_{\text{split}} - T^*_{\text{single}}$$

Because the splittable model has a larger feasible region:

$$T^*_{\text{split}} \geq T^*_{\text{single}}$$

The percentage throughput benefit of splitting will be reported as:

$$\text{Split Benefit } (\%) = \frac{T^*_{\text{split}} - T^*_{\text{single}}}{T^*_{\text{single}}} \times 100$$

when:

$$T^*_{\text{single}} > 0$$

A value of zero indicates that multipath routing provides no throughput benefit for the evaluated scenario.

A positive value quantifies the additional throughput made possible by splitting traffic across paths.

25.7 Stage 1 vs. Stage 2 Results

For each routing formulation, Stage 1 and Stage 2 serve different purposes.

Stage 1

Maximizes total delivered throughput.

Stage 2

Preserves the Stage 1 delivered amount for every source-stream pair and minimizes latency for UK-L and US-L.

The following checks will therefore be performed:

$$T_{\text{Stage 2}} = T_{\text{Stage 1}}$$

and, for every source-stream pair:

$$z^{\text{Stage 2}}_{k,s} = z^{\text{Stage 1}}_{k,s}$$

Latency-sensitive performance should improve or remain unchanged:

$$L_{\text{Stage 2}} \leq L_{\text{Stage 1}}$$

for the optimized sensitive-traffic latency objective.

This verifies that Stage 2 improves routing without sacrificing throughput.

25.8 Planned Result Interpretation

The final solution will be interpreted at five levels.

Source-Stream Level

For every source-stream pair, the analysis will show:

- demand;
- delivered demand;
- lost demand;
- delivery percentage;
- route or routes used;
- latency performance.

This is the most detailed view of network service.

Aggregate Stream Level

Source-level results will be aggregated into:

- UK-L;
- UK-T;
- US-L;
- US-T.

This provides the business-level summary of how each traffic class is served.

Route Level

The route-level table will show exactly which paths carry traffic.

This will make the distinction between the two formulations visible:

- splittable traffic may use multiple paths;
- single-path traffic may use only one path per source-stream pair.

Link Level

For every directed link, the analysis will report:

- capacity;
- assigned flow;
- residual capacity;
- utilization;
- saturation;
- marginal throughput value.

This allows heavily utilized links to be distinguished from true throughput bottlenecks.

Model Comparison

The splittable and single-path models will be evaluated using the same demand and network inputs.

The comparison will quantify:

- throughput differences;
- lost-demand differences;
- sensitive-traffic latency;
- routing flexibility;
- bottleneck behavior.

This makes it possible to determine when multipath routing provides meaningful operational value.

Overall Evaluation

The final analysis should answer five primary questions:

1. **How much demand can the network deliver?**
2. **Where is demand lost and why?**
3. **How effectively can latency-sensitive traffic be routed after throughput is maximized?**
4. **How much value does splittable routing provide compared with single-path routing?**
5. **Which links would provide the greatest benefit if additional capacity were added?**

26. Synthetic Network Dataset

A numerical production dataset was not provided for the assignment.

Therefore, a synthetic dataset is constructed to develop, test, validate, and compare the optimization models.

The synthetic values are not intended to represent actual Quvia network measurements.

Instead, the dataset is designed to create a network containing:

- heterogeneous link capacities;
- heterogeneous link latencies;
- relatively fast but capacity-constrained routes;
- slower alternative routes;
- shared network bottlenecks;
- meaningful competition among source-stream demands;
- conditions under which splittable and single-path routing may behave differently.

The model remains fully parameterized so that actual operational capacities, latencies, and demands could later replace the synthetic values without changing the mathematical formulation.

The synthetic network preserves the same physical topology defined in the problem statement. The numerical values are introduced only to create a reproducible experimental environment for solving and validating the optimization models.

In [150…

```
# Create the synthetic physical-link dataset used for model developmen
# validation, and comparison.
#
# Each row represents one physical full-duplex network connection.
# We store the physical link only once here.
#
# In Section 27, each physical link will be converted into two directe
# so that traffic in opposite directions can be constrained independen
#
# Columns:
#   node_1         = first endpoint of the physical link
#   node_2         = second endpoint of the physical link
#   capacity_mbps  = available bandwidth for the link, in Mbps
#   latency_ms     = latency contributed by the link, in milliseconds
```

```
physical_link_data = pd.DataFrame([
    ["S1", "A", 40, 12],
    ["S2", "A", 35, 14],
    ["S3", "B", 30, 10],
    ["A",  "B", 20, 15],
    ["A",  "C", 25, 8],
    ["B",  "C", 18, 10],
    ["B",  "D", 28, 18],
    ["C", "UK", 30, 7],
    ["C", "US", 25, 9],
    ["D", "UK", 22, 16],
    ["D", "US", 30, 14]
], columns=[
    "node_1",
    "node_2",
    "capacity_mbps",
    "latency_ms"
])

# Display the synthetic physical-link input table.
physical_link_data
```

Out[150…

	node_1	node_2	capacity_mbps	latency_ms
0	S1	A	40	12
1	S2	A	35	14
2	S3	B	30	10
3	A	B	20	15
4	A	C	25	8
5	B	C	18	10
6	B	D	28	18
7	C	UK	30	7
8	C	US	25	9
9	D	UK	22	16
10	D	US	30	14

Initial Interpretation of the Physical-Link Dataset

The synthetic input table contains the 11 physical connections in the network and assigns each one a bandwidth capacity and latency.

Capacity ranges from:

$$18 \text{ Mbps}$$

to:

$$40 \text{ Mbps}$$

while latency ranges from:

$$7 \text{ ms}$$

to:

$$18 \text{ ms}$$

This confirms immediately that the final synthetic model differs materially from the earlier equal-link baseline.

The network now contains both:

- relatively fast but narrower links;
- higher-capacity but slower alternatives.

These differences create the conditions needed for a meaningful capacity-constrained, latency-aware routing problem.

26.1 Synthetic Link Design Rationale

The synthetic values are intentionally heterogeneous so that the optimization problem contains meaningful routing choices rather than a trivial shortest-path solution.

The values were selected to create several different network behaviors.

Entry Links

The source-entry links have relatively high capacity:

- S1 → A: 40 Mbps, 12 ms
- S2 → A: 35 Mbps, 14 ms
- S3 → B: 30 Mbps, 10 ms

These links allow substantial traffic to enter the network without intentionally making the source-entry links the dominant network constraints.

Internal Network Corridors

The internal links create capacity-latency tradeoffs.

For example:

- A → C: 25 Mbps, 8 ms
- A → B: 20 Mbps, 15 ms
- B → C: 18 Mbps, 10 ms
- B → D: 28 Mbps, 18 ms

Compared with B → C, B → D provides greater capacity:

$$28 \text{ Mbps}$$

versus:

$$18 \text{ Mbps}$$

but also higher latency:

$$18 \text{ ms}$$

versus:

$$10 \text{ ms}$$

The A → C corridor is relatively fast, while B → D provides greater bandwidth at a higher latency.

This creates a realistic optimization decision:

> Should traffic consume scarce low-latency capacity, or should some demand use a slower route in order to preserve total network throughput?

Exit Links

The exit links also contain different capacity-latency characteristics:

- C → UK: 30 Mbps, 7 ms
- C → US: 25 Mbps, 9 ms
- D → UK: 22 Mbps, 16 ms
- D → US: 30 Mbps, 14 ms

The C corridor generally offers lower latency, while the D corridor provides useful alternative capacity.

This creates competition for the lowest-latency exit paths and allows the optimizer to demonstrate when rerouting improves overall network performance.

Modeling Intent

The objective of the synthetic dataset is not to reproduce an actual operating network.

Instead, the dataset is designed to exercise the model under conditions containing:

- heterogeneous link bandwidth;
- heterogeneous latency;
- shared capacity;
- multiple feasible paths;
- fast but capacity-constrained corridors;
- slower alternative corridors;
- potential network bottlenecks.

For simplicity, the synthetic baseline assigns the same capacity and latency to both directions of each physical link.

The optimization nevertheless treats the two directions as separate arcs because the network is full-duplex.

26.2 Validate the Synthetic Dataset

Before using the synthetic data in the optimization model, several basic checks are performed.

The dataset should satisfy the following conditions:

- all physical links should be unique;
- capacities must be positive;
- latencies must be positive;
- every node appearing in the link table must belong to the known network;
- the dataset should contain the expected 11 physical links;
- required link attributes should not contain missing values.

These checks help detect input errors before they propagate into the optimization model.

In [152…

```
# Define the set of network nodes that should appear in the synthetic
# These are the same nine nodes defined in the original problem topolo

expected_nodes = {
    "S1", "S2", "S3",
```

```
    "A", "B", "C", "D",
    "UK", "US"
}


# ------------------------------------------------------------
# Check 1: Verify the expected number of physical links
# ------------------------------------------------------------

# The original network topology contains 11 physical links.
# If this assertion fails, a link may have been omitted or duplicated.

assert len(physical_link_data) == 11


# ------------------------------------------------------------
# Check 2: Verify that every link capacity is positive
# ------------------------------------------------------------

# A zero or negative capacity would not make sense for an active link
# in the current synthetic network definition.

assert (
    physical_link_data["capacity_mbps"] > 0
).all()


# ------------------------------------------------------------
# Check 3: Verify that every link latency is positive
# ------------------------------------------------------------

# All links are assumed to contribute positive transmission latency.

assert (
    physical_link_data["latency_ms"] > 0
).all()


# ------------------------------------------------------------
# Check 4: Verify that the link table contains exactly
#          the expected network nodes
# ------------------------------------------------------------

# Collect every node appearing in either endpoint column.

observed_nodes = set(
    physical_link_data["node_1"]
).union(
    set(physical_link_data["node_2"])
)

# Confirm that no expected node is missing and no unexpected node appe
```

```
assert observed_nodes == expected_nodes


# ------------------------------------------------------------
# Check 5: Verify that physical links are unique
# ------------------------------------------------------------

# Because the table stores physical links rather than directed arcs,
# A-B and B-A should be treated as the same physical connection.
#
# Sorting the two endpoints creates a normalized representation,
# such as:
#   ("A", "B")
# regardless of whether the input row was A-B or B-A.

normalized_links = physical_link_data.apply(
    lambda row: tuple(
        sorted([
            row["node_1"],
            row["node_2"]
        ])
    ),
    axis=1
)

# The number of unique normalized links should equal the number
# of rows in the physical-link table.

assert normalized_links.nunique() == len(
    physical_link_data
)


# ------------------------------------------------------------
# Check 6: Verify that no required input values are missing
# ------------------------------------------------------------

# Missing endpoint, capacity, or latency values would make path
# construction or optimization unreliable.

assert not physical_link_data[
    [
        "node_1",
        "node_2",
        "capacity_mbps",
        "latency_ms"
    ]
].isnull().any().any()


# ------------------------------------------------------------
```

```
# Print validation summary
# ------------------------------------------------------------

print("Synthetic dataset validation passed.")
print(
    "Physical links:",
    len(physical_link_data)
)
print(
    "Nodes represented:",
    len(observed_nodes)
)
print("Missing values: none")
```

```
Synthetic dataset validation passed.
Physical links: 11
Nodes represented: 9
Missing values: none
```

Interpretation of Validation Results

All validation checks pass.

The dataset contains:

- 11 unique physical links;
- all 9 expected network nodes;
- strictly positive capacities;
- strictly positive latencies;
- no missing required values.

This confirms that the synthetic inputs are structurally consistent with the network topology before directional arcs are created.

The validation step is intentionally performed before optimization so that any later infeasibility or unusual routing behavior can be attributed to the model or network constraints rather than an input-data construction error.

26.3 Visualize the Synthetic Network

The following diagram shows the synthetic network used for the final optimization.

Each physical link is labeled with:

$$\text{Capacity (Mbps)} \;/\; \text{Latency (ms)}$$

Unlike the equal-link exploratory network, the links now have heterogeneous capacities and latencies.

This creates genuine routing tradeoffs in which:

- the minimum-hop route is not necessarily the best route for the network;
- the lowest-latency route may have limited capacity;
- a higher-latency alternative may allow more total demand to be delivered.

In [153…

```
# Create an undirected NetworkX graph for visualization only.
#
# The physical network is naturally easier to read as an undirected gr
# The optimization model itself will later use directed arcs.

G_synthetic = nx.Graph()


# Add each physical link to the visualization graph.
#
# Store capacity and latency as edge attributes so they can be
# displayed directly on the network diagram.

for _, row in physical_link_data.iterrows():

    G_synthetic.add_edge(
        row["node_1"],
        row["node_2"],
        capacity_mbps=row["capacity_mbps"],
        latency_ms=row["latency_ms"]
    )


# Create a new plotting canvas.

plt.figure(
    figsize=(14, 8)
)


# Draw nodes, physical links, and node labels.
#
# The variable 'pos' was defined earlier in the notebook so that
# the network layout remains consistent across all network figures.

nx.draw_networkx(
    G_synthetic,
    pos=pos,
    with_labels=True,
    node_size=2200,
    font_size=11,
    width=2
```

```
)


# Build a label for each physical link.
#
# Each label contains:
#   line 1: capacity in Mbps
#   line 2: latency in milliseconds

edge_labels = {
    (u, v):
        f"{data['capacity_mbps']} Mbps\n"
        f"{data['latency_ms']} ms"

    for u, v, data
    in G_synthetic.edges(data=True)
}


# Add the capacity/latency labels to the corresponding links.

nx.draw_networkx_edge_labels(
    G_synthetic,
    pos=pos,
    edge_labels=edge_labels,
    font_size=8
)


# Add a descriptive title.

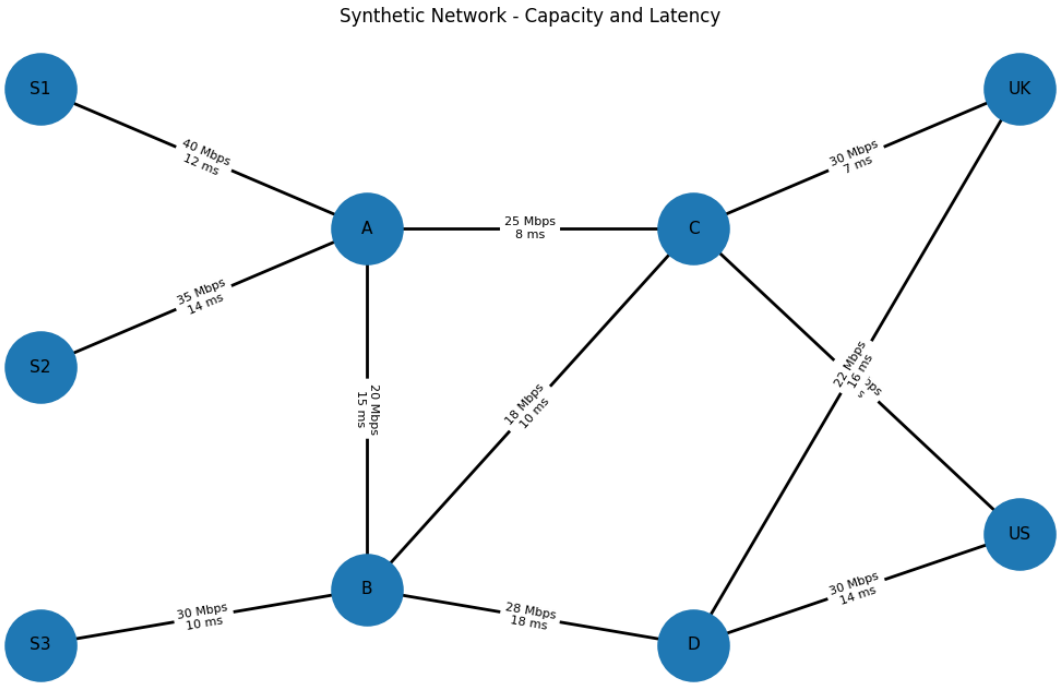
plt.title(
    "Synthetic Network - Capacity and Latency"
)


# Hide the standard x/y plotting axes because node coordinates
# are used only to position the network visually.

plt.axis("off")


# Render the final network diagram.

plt.show()
```



Interpretation of the Synthetic Network Diagram

The network visualization makes the capacity-latency structure easier to interpret spatially.

Several important corridors are visible.

S1 and S2 Enter Through A

Both S1 and S2 must first reach node A.

Their traffic can then move toward:

- C directly through A → C;
- or B through A → B before continuing toward C or D.

Because A → C has relatively low latency:

$$8 \text{ ms}$$

it is likely to be attractive for latency-sensitive traffic.

However, its capacity is only:

$$25 \text{ Mbps}$$

and must be shared by all traffic using that directional link.

S3 Enters Through B

S3 reaches the network through B.

From B, traffic can continue through:

- C using the relatively fast but narrower B → C link;
- D using the higher-capacity but slower B → D link;
- or A before moving toward C.

This gives traffic originating at S3 several routing alternatives.

C Is the Lower-Latency Exit Corridor

C connects directly to both exits with relatively low latency:

$$C \to UK = 7 \text{ ms}$$

and:

$$C \to US = 9 \text{ ms}$$

These links are therefore likely to be particularly attractive for UK-L and US-L traffic.

D Provides Alternative Exit Capacity

D provides slower alternative routes:

$$D \to UK = 16 \text{ ms}$$

$$D \to US = 14 \text{ ms}$$

Although these routes are slower, D may provide useful capacity when the faster C corridor becomes congested.

Initial Optimization Hypotheses

Before solving the optimization model, the synthetic network suggests several hypotheses:

1. A → C may become highly utilized because S1 and S2 naturally prefer routes through C.
2. B → C may become constrained because it provides a relatively low-latency path from B and S3.
3. B → D may become important as an alternate higher-capacity corridor.
4. C → UK and C → US may become important because they provide low-latency access to the exits.

5. Latency-tolerant traffic may be shifted toward D in order to preserve lower-latency capacity through C for UK-L and US-L.

These are hypotheses only.

The optimization results will determine whether these links actually become saturated or whether additional capacity on them would increase total network throughput.

Visualization Note

The diagram above represents each full-duplex physical connection as a single undirected line for readability.

This does **not** mean that the optimization uses a single shared bidirectional capacity.

In the optimization model, each physical connection will be converted into two directed arcs with independently enforced directional capacity constraints.

For example:

$$A \to C$$

and:

$$C \to A$$

will be represented separately in the next section.

26.4 Capacity-Latency Profile of the Synthetic Links

The synthetic dataset is intentionally designed so that link capacity and latency are not perfectly aligned.

The following scatter plot shows each physical link according to:

- available capacity;
- link latency.

This visualization helps show why the routing problem contains genuine tradeoffs rather than one universally superior path.

A link may have:

- low latency but only moderate capacity;
- high capacity but greater latency;
- or a favorable combination of both.

These differences create the conditions needed to test the interaction between throughput maximization and latency minimization.

In [154…

```
# Create a scatter plot showing the relationship between
# link capacity and link latency.
#
# This provides a second view of the synthetic network:
# the network graph shows WHERE links are located,
# while this chart shows HOW their performance characteristics differ.

plt.figure(
    figsize=(10, 6)
)


# Plot one point for each physical link.
#
# x-axis = capacity in Mbps
# y-axis = latency in milliseconds

plt.scatter(
    physical_link_data[
        "capacity_mbps"
    ],
    physical_link_data[
        "latency_ms"
    ],
    s=90
)


# Add the link name beside each point so that individual links
# can be identified directly on the chart.

for _, row in physical_link_data.iterrows():

    # Build a readable label such as "A→C".
    label = (
        f"{row['node_1']}"
        f"→"
        f"{row['node_2']}"
    )

    # Position the label slightly above and to the right
    # of the corresponding scatter point.
    plt.annotate(
        label,
```

```
        (
            row["capacity_mbps"],
            row["latency_ms"]
        ),
        xytext=(5, 5),
        textcoords="offset points",
        fontsize=9
    )


# Label the axes so the chart can be interpreted independently.

plt.xlabel(
    "Capacity (Mbps)"
)

plt.ylabel(
    "Latency (ms)"
)


# Add a descriptive chart title.

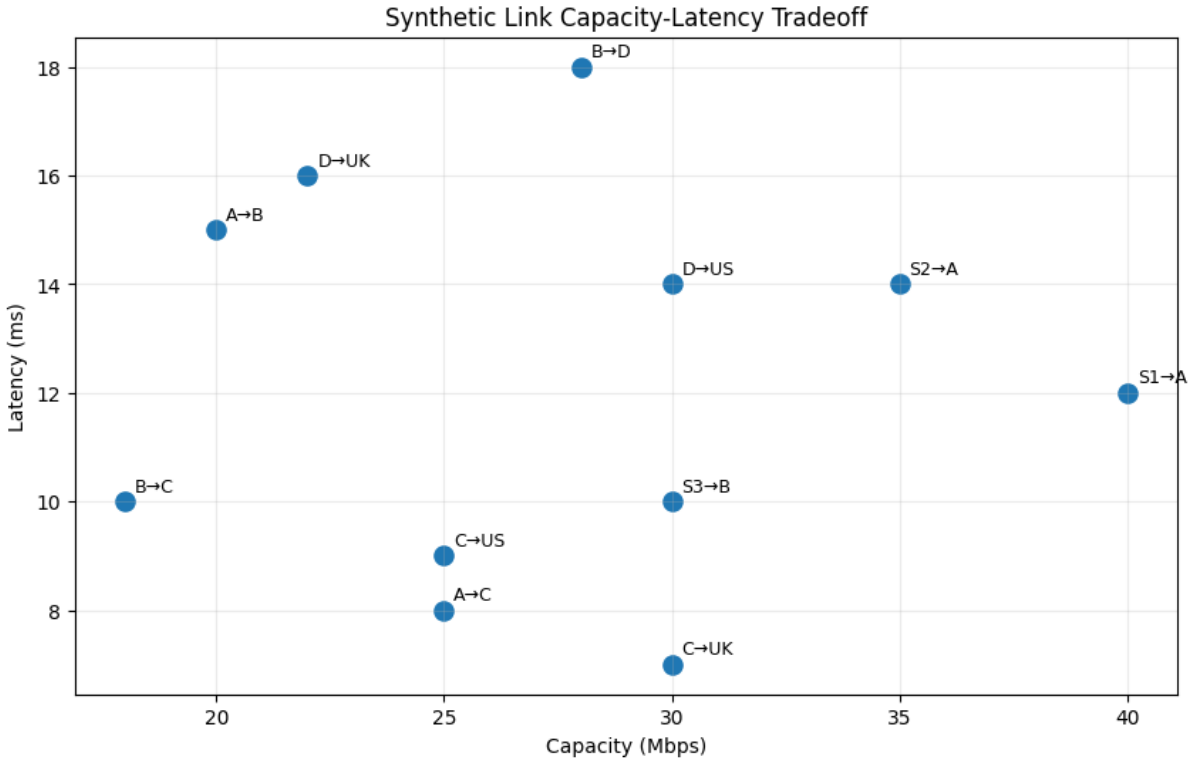
plt.title(
    "Synthetic Link Capacity–Latency Tradeoff"
)


# Add a light grid to make it easier to compare link positions
# without visually overpowering the data.

plt.grid(
    alpha=0.25
)


# Render the chart.

plt.show()
```



Interpretation of Capacity-Latency Profile

The scatter plot confirms that the synthetic network contains a range of link characteristics rather than a single dominant performance level.

Several examples illustrate the tradeoff.

A → C

A → C combines:

$$25 \text{ Mbps}$$

of capacity with only:

$$8 \text{ ms}$$

of latency.

This makes it relatively attractive but not unlimited.

B → C vs. B → D

B → C provides:

$$18 \text{ Mbps}$$

at:

$$10 \text{ ms}$$

while B → D provides:

$$28 \text{ Mbps}$$

at:

$$18 \text{ ms}$$

This is a clear capacity-latency tradeoff.

Exit Corridors

C → UK and C → US provide relatively low latency, while the corresponding D exit links provide slower alternatives.

Capacity-Latency Efficiency

From a routing perspective, links that combine relatively high capacity with relatively low latency are especially attractive.

For example, C → UK combines:

$$30 \text{ Mbps}$$

of capacity with only:

$$7 \text{ ms}$$

of latency.

By contrast, B → D has similar capacity:

$$28 \text{ Mbps}$$

but substantially greater latency:

$$18 \text{ ms}$$

This suggests that lower-latency corridors may become disproportionately attractive unless capacity constraints force traffic onto slower alternatives.

The optimization model is therefore needed to determine whether those apparently attractive links can support all competing traffic simultaneously.

Overall, the variation in the scatter plot confirms that the synthetic network is

suitable for testing whether the optimizer:

- preserves scarce low-latency capacity;
- uses slower routes when necessary to increase throughput;
- and routes latency-sensitive traffic differently from tolerant traffic after maximum throughput has been established.

26.5 Synthetic Dataset Summary Statistics

Descriptive statistics provide a quick numerical summary of the range of capacities and latencies used in the experiment.

This helps confirm that the final synthetic network is materially different from the earlier equal-link baseline.

In [155…

```
# Produce descriptive statistics for the two numerical
# characteristics of the synthetic network.
#
# The summary includes:
#   count
#   mean
#   standard deviation
#   minimum
#   25th percentile
#   median
#   75th percentile
#   maximum
#
# This provides a quick numerical check that the final dataset
# contains meaningful variation in both capacity and latency.

physical_link_data[
    [
        "capacity_mbps",
        "latency_ms"
    ]
].describe()
```

Out [155...

	capacity_mbps	latency_ms
count	11.000000	11.000000
mean	27.545455	12.090909
std	6.486349	3.562430
min	18.000000	7.000000
25%	23.500000	9.500000
50%	28.000000	12.000000
75%	30.000000	14.500000
max	40.000000	18.000000

Interpretation of Summary Statistics

The descriptive statistics confirm that the synthetic network contains meaningful variation in both bandwidth and latency.

Capacity

The average physical-link capacity is approximately:

$$27.55 \text{ Mbps}$$

with a range from:

$$18 \text{ Mbps}$$

to:

$$40 \text{ Mbps}$$

The standard deviation is approximately:

$$6.49 \text{ Mbps}$$

which confirms that link bandwidth is not concentrated around a single uniform value.

Latency

The average link latency is approximately:

$$12.09 \text{ ms}$$

with a range from:

$$7 \text{ ms}$$

to:

$$18 \text{ ms}$$

The standard deviation is approximately:

$$3.56 \text{ ms}$$

so individual links contribute materially different latency to an end-to-end route.

Modeling Significance

This variation is important because it prevents the final optimization from collapsing into either:

- a pure shortest-hop problem;
- or a uniform-capacity maximum-flow problem.

Instead, the optimizer must account simultaneously for:

- route length;
- individual-link latency;
- shared link capacity;
- alternative network corridors.

The synthetic network therefore provides an appropriate experimental environment for evaluating:

- maximum throughput;
- latency-sensitive routing;
- congestion;
- split versus single-path behavior;
- bottleneck sensitivity.

26.5.1 Identify the Highest/Lowest Capacity and Latency Links

The descriptive statistics summarize the overall range of the synthetic dataset.

The following check identifies the individual physical links representing the highest and lowest values for capacity and latency.

These values are descriptive only and should not yet be interpreted as

optimization bottlenecks.

In [156…

```
# Identify several extreme values in the synthetic dataset.
#
# These checks help explain which links are structurally:
#   - highest capacity,
#   - lowest capacity,
#   - lowest latency,
#   - highest latency.
#
# They are descriptive only and do NOT identify actual bottlenecks.
# Bottleneck status will be determined later using optimization
# and marginal-capacity analysis.

highest_capacity_link = physical_link_data.loc[
    physical_link_data[
        "capacity_mbps"
    ].idxmax()
]

lowest_capacity_link = physical_link_data.loc[
    physical_link_data[
        "capacity_mbps"
    ].idxmin()
]

lowest_latency_link = physical_link_data.loc[
    physical_link_data[
        "latency_ms"
    ].idxmin()
]

highest_latency_link = physical_link_data.loc[
    physical_link_data[
        "latency_ms"
    ].idxmax()
]



# ------------------------------------------------------------
# Print a readable summary of the descriptive extremes
# ------------------------------------------------------------

print("Highest-capacity physical link:")
print(
    f"{highest_capacity_link['node_1']} -> "
    f"{highest_capacity_link['node_2']} | "
    f"{highest_capacity_link['capacity_mbps']} Mbps | "
    f"{highest_capacity_link['latency_ms']} ms"
)

print("\nLowest-capacity physical link:")
```

```
print(
    f"{lowest_capacity_link['node_1']} -> "
    f"{lowest_capacity_link['node_2']} | "
    f"{lowest_capacity_link['capacity_mbps']} Mbps | "
    f"{lowest_capacity_link['latency_ms']} ms"
)

print("\nLowest-latency physical link:")
print(
    f"{lowest_latency_link['node_1']} -> "
    f"{lowest_latency_link['node_2']} | "
    f"{lowest_latency_link['capacity_mbps']} Mbps | "
    f"{lowest_latency_link['latency_ms']} ms"
)

print("\nHighest-latency physical link:")
print(
    f"{highest_latency_link['node_1']} -> "
    f"{highest_latency_link['node_2']} | "
    f"{highest_latency_link['capacity_mbps']} Mbps | "
    f"{highest_latency_link['latency_ms']} ms"
)
```

```
Highest-capacity physical link:
S1 -> A | 40 Mbps | 12 ms

Lowest-capacity physical link:
B -> C | 18 Mbps | 10 ms

Lowest-latency physical link:
C -> UK | 30 Mbps | 7 ms

Highest-latency physical link:
B -> D | 28 Mbps | 18 ms
```

Interpretation of Descriptive Extremes

The descriptive extremes reinforce several of the design choices built into the synthetic network.

Highest Capacity

S1 → A has the largest physical-link capacity:

$$40 \text{ Mbps}$$

This supports the design goal of allowing substantial traffic to enter the network from S1 before downstream shared capacity becomes the more interesting routing constraint.

Lowest Capacity

B → C has the smallest capacity:

$$18 \text{ Mbps}$$

despite having relatively low latency:

$$10 \text{ ms}$$

This makes B → C an intentionally attractive but potentially constrained internal corridor.

Lowest Latency

C → UK has the lowest link latency:

$$7 \text{ ms}$$

while still providing:

$$30 \text{ Mbps}$$

of capacity.

This makes the C-to-UK corridor particularly attractive for UK-bound latency-sensitive traffic.

Highest Latency

B → D has the highest link latency:

$$18 \text{ ms}$$

but provides relatively high capacity:

$$28 \text{ Mbps}$$

This creates exactly the type of tradeoff the model is intended to evaluate: a slower route may still be operationally valuable because it provides additional bandwidth.

Important Note

These descriptive extremes should not be interpreted as optimization bottlenecks.

For example, the lowest-capacity link is not necessarily the link where additional

bandwidth would increase total network throughput.

True bottleneck status will be determined later by re-solving the optimization after increasing each directed link's capacity.

26.5.2 Why the Heterogeneous Synthetic Network Is Analytically Useful

Results Analysis

The synthetic network intentionally contains heterogeneous:

- capacities;
- latencies;
- source locations;
- destination access routes.

This creates meaningful tradeoffs that would not appear if every link had identical characteristics.

Some routes are:

- faster but narrower;
- slower but less constrained;
- structurally important because they connect major network regions;
- destination-facing and therefore difficult to bypass.

Why This Happens

A useful optimization test environment must create conditions under which several feasible routing decisions compete with one another.

If every path had identical capacity and latency, many important behaviors would disappear.

The heterogeneous network instead allows the analysis to test:

- congestion;
- route substitution;
- latency-throughput tradeoffs;
- capacity bottlenecks;
- resilience under degradation.

Business Impact

The synthetic data should not be interpreted as a forecast of a real production network.

Its value is experimental.

It provides a controlled environment in which the optimization framework can demonstrate whether it correctly handles realistic classes of network decisions.

Therefore:

> **The synthetic dataset is primarily a model-validation environment, not a production estimate.**

This distinction allows the analytical method to be evaluated without overstating the meaning of the individual numerical values.

26.6 Section Summary

Section 26 establishes the numerical network used by the optimization models.

The synthetic dataset:

- preserves the original network topology;
- contains 11 physical links and 9 nodes;
- assigns heterogeneous capacity and latency values;
- introduces realistic capacity-latency tradeoffs;
- passes basic structural validation checks;
- creates multiple plausible routing alternatives.

The network visualization shows where those tradeoffs occur spatially, while the capacity-latency profile demonstrates that no single link characteristic dominates throughout the network.

The descriptive statistics further confirm that the final test environment contains meaningful numerical variation in both bandwidth and latency.

Experimental Implication

This synthetic network is intentionally designed so that capacity planning cannot be performed by inspecting individual links in isolation.

The business question is not simply:

> Which link is the slowest or smallest?

Instead, the optimization must determine:

> Which combination of shared capacities and route choices limits end-to-end service, and where would additional bandwidth create measurable operational value?

That distinction will become important later when saturated links are compared with true throughput bottlenecks.

The next step is to convert each full-duplex physical connection into two directed arcs so that directional capacity can be enforced correctly by the optimization model.

27. Directed Arc Representation

The physical network is naturally represented as a set of full-duplex physical links.

For visualization, a physical connection such as:

$$A \leftrightarrow C$$

can be shown as one line.

For optimization, however, traffic direction matters.

Each full-duplex physical link is therefore represented as two directed arcs.

For example:

$$A \leftrightarrow C$$

becomes:

$$A \rightarrow C$$

and:

$$C \rightarrow A$$

Each directed arc receives its own capacity constraint.

This distinction is important because traffic moving from A to C should not consume the capacity available for traffic moving independently from C to A.

Because the physical topology contains:

$$11$$

physical links, the directed optimization network should contain:

$$2(11) = 22$$

directed arcs.

The baseline synthetic dataset assigns the same capacity and latency to both directions of each physical connection.

However, the model stores the directions separately so that asymmetric directional values could be introduced later without changing the optimization structure.

27.1 Convert Physical Links to Directed Arcs

The physical-link dataset created in Section 26 stores each physical connection once.

The following transformation creates two records for every physical link:

1. one arc from `node_1` to `node_2`;
2. one arc from `node_2` to `node_1`.

The resulting table will become the link-level input to the optimization models.

Each directed arc stores:

- origin node;
- destination node;
- directional capacity;
- link latency.

In [160…

```
# Create an empty list that will store one dictionary
# for every directed network arc.
#
# Each physical full-duplex link will contribute two rows:
#   node_1 -> node_2
#   node_2 -> node_1

directed_rows = []


# Iterate through every physical connection in the
```

```
# synthetic physical-link dataset.

for _, row in physical_link_data.iterrows():

    # ------------------------------------------------------------
    # Forward direction: node_1 -> node_2
    # ------------------------------------------------------------
    #
    # Copy the capacity and latency from the physical link
    # into the forward directed arc.

    directed_rows.append({
        "from_node": row["node_1"],
        "to_node": row["node_2"],
        "capacity_mbps": row["capacity_mbps"],
        "latency_ms": row["latency_ms"]
    })


    # ------------------------------------------------------------
    # Reverse direction: node_2 -> node_1
    # ------------------------------------------------------------
    #
    # Because the physical link is full-duplex, create a
    # separate reverse-direction arc.
    #
    # The baseline synthetic dataset uses the same capacity
    # and latency in both directions, but the two arcs remain
    # separate optimization constraints.

    directed_rows.append({
        "from_node": row["node_2"],
        "to_node": row["node_1"],
        "capacity_mbps": row["capacity_mbps"],
        "latency_ms": row["latency_ms"]
    })


# Convert the list of directed-arc records into a DataFrame.

arcs_df = pd.DataFrame(
    directed_rows
)


# Print high-level counts to verify that every physical link
# produced exactly two directed arcs.

print(
    "Physical links:",
    len(physical_link_data)
)
```

```
print(
    "Directed arcs:",
    len(arcs_df)
)


# Display the complete directed-arc table.

arcs_df
```

```
Physical links: 11
Directed arcs: 22
```

Out[160…

	from_node	to_node	capacity_mbps	latency_ms
0	S1	A	40	12
1	A	S1	40	12
2	S2	A	35	14
3	A	S2	35	14
4	S3	B	30	10
5	B	S3	30	10
6	A	B	20	15
7	B	A	20	15
8	A	C	25	8
9	C	A	25	8
10	B	C	18	10
11	C	B	18	10
12	B	D	28	18
13	D	B	28	18
14	C	UK	30	7
15	UK	C	30	7
16	C	US	25	9
17	US	C	25	9
18	D	UK	22	16
19	UK	D	22	16
20	D	US	30	14
21	US	D	30	14

27.2 Interpretation of the Directed Arc Table

The directed-arc table confirms that every physical link has been expanded into two independent routing directions.

For example, the physical link:

$$S1 \leftrightarrow A$$

creates:

$$S1 \rightarrow A$$

and:

$$A \rightarrow S1$$

Each arc currently has:

$$40 \text{ Mbps}$$

of capacity and:

$$12 \text{ ms}$$

of latency.

Likewise:

$$A \leftrightarrow C$$

creates:

$$A \rightarrow C$$

and:

$$C \rightarrow A$$

with 25 Mbps capacity and 8 ms latency in each direction.

Why This Matters

The two directions are mathematically separate resources.

For example, if:

$$25 \text{ Mbps}$$

of traffic uses:

$$A \to C$$

then that direction may be fully utilized.

However, this does **not** imply that:

$$C \to A$$

is also fully utilized.

Traffic moving in the reverse direction has its own independent 25 Mbps capacity constraint.

This is the key modeling implication of the full-duplex assumption.

Synthetic Symmetry vs. Model Directionality

The baseline synthetic dataset currently assigns identical capacity and latency values to the two directions of each physical link.

That is a data assumption, not a mathematical requirement.

The optimization architecture would also support a future network in which:

$$C_{A \to C} \neq C_{C \to A}$$

or:

$$L_{A \to C} \neq L_{C \to A}$$

without changing the fundamental formulation.

27.3 Validate the Directed Arc Conversion

Before building the directed optimization graph, the arc transformation is validated.

The directed dataset should satisfy the following conditions:

- exactly 22 directed arcs should exist;
- every physical link should generate exactly two directions;
- no directed arc should be duplicated;
- capacities should remain positive;
- latencies should remain positive;
- all expected network nodes should still be represented.

These checks confirm that the full-duplex transformation was performed

correctly before the directed graph is used by the optimization models.

In [161…

```
# ------------------------------------------------------------
# Check 1: Verify expected number of directed arcs
# ------------------------------------------------------------

# Every one of the 11 physical links should create exactly
# two directed arcs.

expected_directed_arcs = (
    2 * len(physical_link_data)
)

assert len(arcs_df) == expected_directed_arcs


# ------------------------------------------------------------
# Check 2: Verify that directed arcs are unique
# ------------------------------------------------------------

# A directed arc is uniquely identified by its ordered pair:
#   (from_node, to_node)
#
# For example:
#   A -> C
# and
#   C -> A
# are intentionally different arcs.

assert (
    arcs_df[
        ["from_node", "to_node"]
    ]
    .drop_duplicates()
    .shape[0]
    == len(arcs_df)
)


# ------------------------------------------------------------
# Check 3: Verify positive directional capacities
# ------------------------------------------------------------

assert (
    arcs_df["capacity_mbps"] > 0
).all()


# ------------------------------------------------------------
# Check 4: Verify positive directional latencies
# ------------------------------------------------------------
```

```
assert (
    arcs_df["latency_ms"] > 0
).all()


# ------------------------------------------------------------
# Check 5: Verify no required values are missing
# ------------------------------------------------------------

assert not arcs_df[
    [
        "from_node",
        "to_node",
        "capacity_mbps",
        "latency_ms"
    ]
].isnull().any().any()


# ------------------------------------------------------------
# Check 6: Verify that all expected nodes are represented
# ------------------------------------------------------------

directed_nodes = set(
    arcs_df["from_node"]
).union(
    set(arcs_df["to_node"])
)

assert directed_nodes == expected_nodes


# ------------------------------------------------------------
# Check 7: Verify that every physical link has both directions
# ------------------------------------------------------------

for _, row in physical_link_data.iterrows():

    u = row["node_1"]
    v = row["node_2"]

    # Confirm the forward direction exists.
    forward_exists = (
        (
            arcs_df["from_node"] == u
        )
        &
        (
            arcs_df["to_node"] == v
        )
    ).any()
```

```
    # Confirm the reverse direction exists.
    reverse_exists = (
        (
            arcs_df["from_node"] == v
        )
        &
        (
            arcs_df["to_node"] == u
        )
    ).any()

    assert forward_exists
    assert reverse_exists


# ------------------------------------------------------------
# Print validation summary
# ------------------------------------------------------------

print(
    "Directed arc validation passed."
)

print(
    "Physical links:",
    len(physical_link_data)
)

print(
    "Expected directed arcs:",
    expected_directed_arcs
)

print(
    "Actual directed arcs:",
    len(arcs_df)
)

print(
    "Nodes represented:",
    len(directed_nodes)
)

print(
    "Missing values: none"
)
```

```
Directed arc validation passed.
Physical links: 11
Expected directed arcs: 22
Actual directed arcs: 22
Nodes represented: 9
Missing values: none
```

27.4 Interpretation of Validation Results

Interpretation of Directed Arc Validation

All directed-arc validation checks pass.

The transformation produces:

- 11 physical links;
- 22 unique directed arcs;
- all 9 expected network nodes;
- no missing values;
- positive capacity and latency on every arc;
- both directions for every physical full-duplex connection.

This confirms that the network representation is ready for use in the optimization model.

Importantly, the validation confirms that the full-duplex assumption has been implemented structurally rather than being treated only as a descriptive statement.

27.4.1 Directed-Arc Representation — Operational Interpretation

Results Analysis

Representing each full-duplex physical link as two directed arcs allows capacity to be tracked independently in each direction.

This ensures that traffic moving:

$$u \rightarrow v$$

does not incorrectly consume capacity assigned to:

$$v \rightarrow u.$$

Why This Matters

Directional representation is essential because traffic demand and route utilization may be highly asymmetric.

A physical connection may have:

- heavy utilization in one direction;
- substantial unused capacity in the opposite direction.

Aggregating the two directions would hide this distinction.

Business Impact

Directional modeling produces more actionable capacity information.

Infrastructure teams can identify whether congestion is:

- truly associated with the entire physical link;
- concentrated primarily in one direction.

This matters because a directional capacity issue may require a different operational or engineering response than a fully saturated bidirectional resource.

27.5 Build the Directed Optimization Graph

The validated directed-arc table is now converted into a NetworkX directed graph.

This graph will be used for:

- directed path generation;
- path-capacity calculation;
- path-latency calculation;
- candidate-route construction.

Each directed edge stores the corresponding:

- bandwidth capacity;
- latency.

The optimization models will later reference these attributes when constructing feasible paths and enforcing link-capacity constraints.

In [165…

```
# Create a directed NetworkX graph.
#
# Unlike the visualization graph from Section 26, this graph
# preserves the direction of every network arc.

DG = nx.DiGraph()


# Add each directed arc from arcs_df to the graph.
```

```
#
# Capacity and latency are stored as edge attributes so that
# later path-generation and optimization code can retrieve them.

for _, row in arcs_df.iterrows():

    DG.add_edge(
        row["from_node"],
        row["to_node"],
        capacity_mbps=row["capacity_mbps"],
        latency_ms=row["latency_ms"]
    )


# Print graph-level validation information.

print(
    "Nodes:",
    DG.number_of_nodes()
)

print(
    "Directed arcs:",
    DG.number_of_edges()
)
```

```
Nodes: 9
Directed arcs: 22
```

27.6 Validate Directed Graph

In [166…

```
# ------------------------------------------------------------
# Validate the final directed NetworkX graph
# ------------------------------------------------------------

# Confirm that all nine expected nodes are present.

assert (
    DG.number_of_nodes()
    == len(expected_nodes)
)


# Confirm that all 22 directed arcs were loaded.

assert (
    DG.number_of_edges()
    == len(arcs_df)
)
```

```
# Confirm that every arc in the DataFrame appears in the graph
# with the expected capacity and latency attributes.

for _, row in arcs_df.iterrows():

    u = row["from_node"]
    v = row["to_node"]

    # Confirm the directed edge exists.
    assert DG.has_edge(u, v)

    # Confirm capacity was transferred correctly.
    assert (
        DG[u][v][
            "capacity_mbps"
        ]
        ==
        row["capacity_mbps"]
    )

    # Confirm latency was transferred correctly.
    assert (
        DG[u][v][
            "latency_ms"
        ]
        ==
        row["latency_ms"]
    )


print(
    "Directed graph validation passed."
)
```

```
Directed graph validation passed.
```

27.7 Interpretation of the Directed Graph

Interpretation of the Directed Optimization Graph

The directed optimization graph contains:

$$9$$

nodes and:

$$22$$

directed arcs.

This matches the expected structure exactly.

The graph now provides the routing representation needed by the final optimization.

For example, a path such as:

$$S1 \rightarrow A \rightarrow C \rightarrow UK$$

is represented as the ordered arc sequence:

$$(S1, A), (A, C), (C, UK)$$

The capacity of each of these arcs will be enforced independently when several source-stream commodities share the same route.

The latency of the path will later be calculated by summing the latency attributes of the three directed arcs.

This directed representation therefore connects the physical topology to the mathematical routing model.

27.8 Visualize the Directed Optimization Network

The following diagram shows the same physical topology using directed arcs.

Unlike the Section 26 physical-network visualization, arrows are shown to emphasize that every physical connection supports two independent traffic directions.

Capacity and latency labels are intentionally omitted from this figure because they were already shown in Section 26.

The purpose of this visualization is to make the optimization representation explicit.

In [167…

```
# Create a new figure for the directed optimization graph.

plt.figure(
    figsize=(14, 8)
)


# Draw the network nodes.

nx.draw_networkx_nodes(
    DG,
    pos=pos,
    node_size=2200
```

```
)


# Draw node labels.

nx.draw_networkx_labels(
    DG,
    pos=pos,
    font_size=11
)


# Draw directed arcs with arrowheads.
#
# A small curvature is used so that reciprocal arcs
# (for example A->C and C->A) can be distinguished visually.

nx.draw_networkx_edges(
    DG,
    pos=pos,
    arrows=True,
    arrowstyle="-|>",
    arrowsize=16,
    width=1.5,
    connectionstyle="arc3,rad=0.08"
)


# Add a descriptive title.

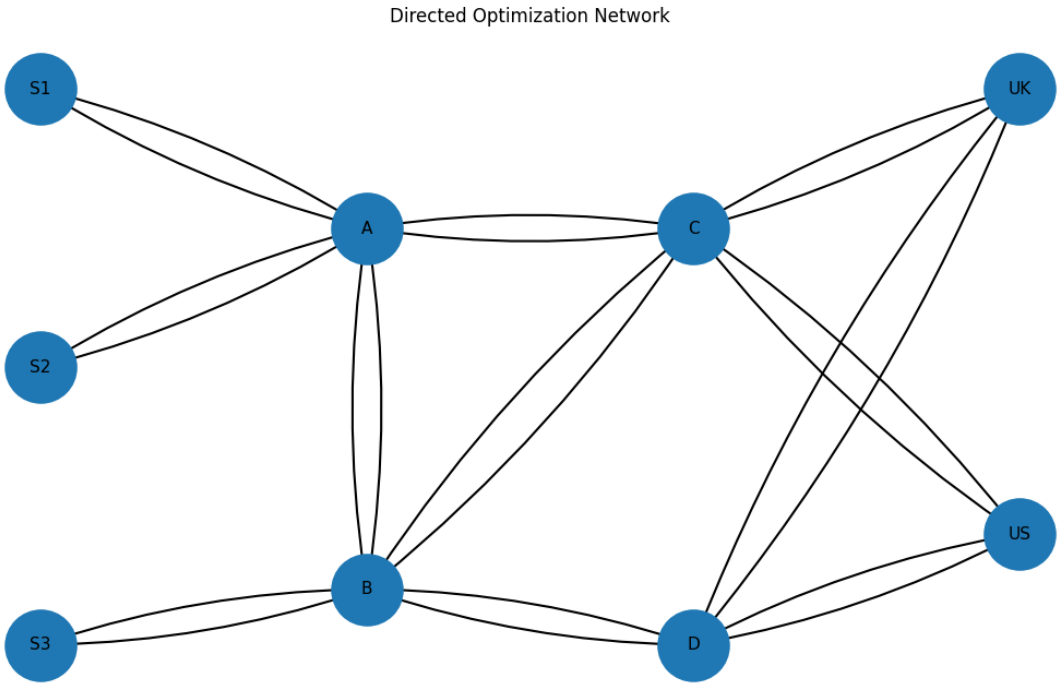
plt.title(
    "Directed Optimization Network"
)


# Hide coordinate axes because the positions are used
# only for network visualization.

plt.axis("off")


# Render the graph.

plt.show()
```



27.9 Why Directionality Matters Operationally

Representing the network with directed arcs is not merely an implementation detail.

It changes how available bandwidth is interpreted.

Consider the physical connection:

$$A \leftrightarrow C$$

with 25 Mbps available in each direction.

Suppose:

$$25 \text{ Mbps}$$

is routed from:

$$A \rightarrow C$$

while simultaneously:

$$10 \text{ Mbps}$$

is routed from:

$$C \rightarrow A$$

The two capacity constraints are evaluated independently:

$$F_{A \to C} \leq 25$$

and:

$$F_{C \to A} \leq 25$$

Therefore, the physical connection can simultaneously carry traffic in both directions without forcing the two flows to share a single 25 Mbps limit.

Business Implication

A network-capacity decision should therefore identify not only **which physical connection** is constrained, but also **which direction of that connection** is constraining traffic.

For example:

> Additional bandwidth on C → US may provide substantial value even if the reverse arc US → C is lightly utilized.

This directional distinction becomes especially important later in the bottleneck and marginal-capacity analysis.

27.10 Section Summary

Section 27 converts the physical network into the directed representation required by the optimization models.

The transformation:

- converts 11 physical links into 22 directed arcs;
- preserves all 9 network nodes;
- stores capacity and latency independently on each arc;
- validates that every physical connection contains both directions;
- confirms that no directed links are missing or duplicated;
- builds the directed NetworkX graph used for path generation.

Modeling Significance

The distinction between a physical link and a directed arc is important because the optimization constrains traffic according to direction.

A saturated:

$$A \to C$$

arc does not imply that:

$$C \to A$$

is saturated.

This allows the model to represent full-duplex traffic correctly and later identify directional congestion and bottlenecks.

Experimental Implication

The network is now ready for source-stream demand to be introduced.

The next step is to define how much UK-L, UK-T, US-L, and US-T traffic enters the network at each traffic-generating source.

Those source-stream demands will become the commodities routed through this directed network.

28. Synthetic Source-Stream Demand

Demand is modeled separately for each source-stream combination.

This follows the clarified requirement that the input should specify how much of each traffic stream enters the network at each source node.

The traffic-generating nodes are:

$$\{S1, S2, S3, B, C\}$$

The four traffic streams are:

$$\{UK\text{-}L,\ UK\text{-}T,\ US\text{-}L,\ US\text{-}T\}$$

Therefore, the baseline synthetic scenario contains:

$$5 \times 4 = 20$$

source-stream commodities.

Each source-stream pair has:

- a source node;
- traffic class;
- required exit;
- current demand in Mbps.

The purpose of modeling demand at this level is to preserve the physical location where traffic enters the network.

For example:

$$S1/UK\text{-}L$$

and:

$$B/UK\text{-}L$$

belong to the same aggregate traffic class but may consume very different network resources because they begin at different nodes.

In [168…

```
# Create the synthetic source-stream demand dataset.
#
# Each row represents one routing commodity defined by:
#   source       = node where the traffic enters the network
#   stream       = traffic class
#   destination  = required exit
#   demand_mbps  = current traffic demand for that source-stream pair
#
# Five traffic-generating nodes × four traffic classes
# creates 20 source-stream commodities.

demand_df = pd.DataFrame([
    ["S1", "UK-L", "UK", 6],
    ["S1", "UK-T", "UK", 7],
    ["S1", "US-L", "US", 5],
    ["S1", "US-T", "US", 6],

    ["S2", "UK-L", "UK", 5],
    ["S2", "UK-T", "UK", 6],
    ["S2", "US-L", "US", 5],
    ["S2", "US-T", "US", 5],

    ["S3", "UK-L", "UK", 5],
    ["S3", "UK-T", "UK", 7],
    ["S3", "US-L", "US", 6],
    ["S3", "US-T", "US", 6],

    ["B", "UK-L", "UK", 6],
    ["B", "UK-T", "UK", 14],
    ["B", "US-L", "US", 8],
    ["B", "US-T", "US", 9],

    ["C", "UK-L", "UK", 4],
    ["C", "UK-T", "UK", 5],
    ["C", "US-L", "US", 5],
    ["C", "US-T", "US", 6]
], columns=[
```

```
    "source",
    "stream",
    "destination",
    "demand_mbps"
])


# Create a unique identifier for each source-stream commodity.
#
# Example:
#   S1 + UK-L  ->  S1_UK-L
#
# This identifier is useful later when grouping paths,
# building optimization variables, and reporting results.

demand_df["commodity_id"] = (
    demand_df["source"]
    + "_"
    + demand_df["stream"]
)


# Display the complete source-stream demand table.

demand_df
```

Out [168...

	source	stream	destination	demand_mbps	commodity_id
0	S1	UK-L	UK	6	S1_UK-L
1	S1	UK-T	UK	7	S1_UK-T
2	S1	US-L	US	5	S1_US-L
3	S1	US-T	US	6	S1_US-T
4	S2	UK-L	UK	5	S2_UK-L
5	S2	UK-T	UK	6	S2_UK-T
6	S2	US-L	US	5	S2_US-L
7	S2	US-T	US	5	S2_US-T
8	S3	UK-L	UK	5	S3_UK-L
9	S3	UK-T	UK	7	S3_UK-T
10	S3	US-L	US	6	S3_US-L
11	S3	US-T	US	6	S3_US-T
12	B	UK-L	UK	6	B_UK-L
13	B	UK-T	UK	14	B_UK-T
14	B	US-L	US	8	B_US-L
15	B	US-T	US	9	B_US-T
16	C	UK-L	UK	4	C_UK-L
17	C	UK-T	UK	5	C_UK-T
18	C	US-L	US	5	C_US-L
19	C	US-T	US	6	C_US-T

Initial Interpretation of the Source-Stream Demand Table

The table contains 20 distinct routing commodities.

Each traffic-generating source contributes demand to all four traffic classes.

The demand values are intentionally not uniform.

This prevents the test environment from becoming artificially symmetric and creates more realistic competition for shared network capacity.

Several patterns are visible immediately:

- B generates the largest individual demands, particularly UK-T at 14 Mbps;
- C generates comparatively modest demand and is also physically close to both exits;
- S1, S2, and S3 generate moderate demand but must traverse additional internal links before reaching UK or US;
- tolerant traffic generally represents more total demand than latency-sensitive traffic.

These differences create the possibility that network location, rather than demand size alone, will influence which source-stream commodities are easiest to serve.

28.1 Synthetic Demand Design Rationale

The synthetic demand values are designed to create a non-trivial network-loading problem.

The objective is not to estimate actual customer traffic.

Instead, the values are selected so that the network experiences enough demand to test:

- shared-capacity competition;
- partial delivery;
- route congestion;
- latency-sensitive routing;
- differences between split and single-path formulations;
- bottleneck sensitivity.

Moderate Upstream Demand

S1, S2, and S3 each generate moderate amounts of traffic.

Because these sources are farther from the exits, their traffic must consume capacity on several shared internal links.

This makes them useful for testing how upstream traffic competes for scarce downstream capacity.

Higher Demand at B

Node B generates relatively high demand, including:

14 Mbps

of UK-T traffic.

B is also centrally located and has access to both the C and D corridors.

This combination creates meaningful routing flexibility and competition.

Lower Demand at C

Node C generates smaller demand values but is directly adjacent to both exits.

This creates an important contrast:

> A source with relatively modest demand may still receive favorable service because it requires fewer network resources to reach its destination.

This helps test whether the optimization behavior reflects both demand magnitude and network position.

Sensitive vs. Tolerant Traffic

The dataset includes both latency-sensitive and latency-tolerant demand at every source.

This allows the model to test the intended two-stage policy:

1. maximize delivered throughput;
2. preserve that throughput allocation and minimize latency for UK-L and US-L.

28.2 Validate the Demand Dataset

Before the demand data are used in the optimization model, several structural checks are performed.

The dataset should satisfy the following conditions:

- exactly 20 source-stream commodities should exist;
- all five expected traffic-generating sources should be represented;
- all four traffic classes should be represented;
- every stream must map to the correct exit;
- demand values must be non-negative;
- commodity identifiers must be unique;

- no required values should be missing.

These checks help ensure that later routing results are driven by the optimization model rather than by malformed demand inputs.

In [170…

```
# Define expected traffic-generating sources.

expected_sources = {
    "S1", "S2", "S3", "B", "C"
}


# Define expected traffic streams.

expected_streams = {
    "UK-L", "UK-T",
    "US-L", "US-T"
}


# Define the required destination for each traffic stream.

expected_destinations = {
    "UK-L": "UK",
    "UK-T": "UK",
    "US-L": "US",
    "US-T": "US"
}


# ------------------------------------------------------------
# Check 1: Verify expected number of commodities
# ------------------------------------------------------------

# Five sources × four streams = 20 commodities.

assert len(demand_df) == 20


# ------------------------------------------------------------
# Check 2: Verify expected sources
# ------------------------------------------------------------

assert set(
    demand_df["source"]
) == expected_sources


# ------------------------------------------------------------
# Check 3: Verify expected traffic streams
# ------------------------------------------------------------
```

```
assert set(
    demand_df["stream"]
) == expected_streams


# ------------------------------------------------------------
# Check 4: Verify stream-to-destination correctness
# ------------------------------------------------------------

for _, row in demand_df.iterrows():

    assert (
        row["destination"]
        ==
        expected_destinations[
            row["stream"]
        ]
    )


# ------------------------------------------------------------
# Check 5: Verify demand is non-negative
# ------------------------------------------------------------

assert (
    demand_df["demand_mbps"] >= 0
).all()


# ------------------------------------------------------------
# Check 6: Verify commodity identifiers are unique
# ------------------------------------------------------------

assert (
    demand_df["commodity_id"]
    .nunique()
    ==
    len(demand_df)
)


# ------------------------------------------------------------
# Check 7: Verify there are no missing values
# ------------------------------------------------------------

assert not demand_df[
    [
        "source",
        "stream",
        "destination",
        "demand_mbps",
```

```
        "commodity_id"
    ]
].isnull().any().any()


# ------------------------------------------------------------
# Print validation summary
# ------------------------------------------------------------

print(
    "Synthetic demand validation passed."
)

print(
    "Source-stream commodities:",
    len(demand_df)
)

print(
    "Sources represented:",
    demand_df["source"].nunique()
)

print(
    "Traffic streams represented:",
    demand_df["stream"].nunique()
)

print(
    "Commodity IDs unique:",
    demand_df["commodity_id"].nunique()
)

print(
    "Missing values: none"
)
```

```
Synthetic demand validation passed.
Source-stream commodities: 20
Sources represented: 5
Traffic streams represented: 4
Commodity IDs unique: 20
Missing values: none
```

Interpretation of Demand Validation Results

All demand validation checks pass.

The baseline dataset contains:

- 20 unique source-stream commodities;
- all 5 traffic-generating sources;

- all 4 required traffic classes;
- correct UK and US destination assignments;
- non-negative demand values;
- no missing required values.

This confirms that the synthetic demand inputs are structurally consistent with the clarified routing requirements before path generation begins.

28.3 Aggregate Demand Summary

In [171…

```
# Calculate total demand across all source-stream commodities.

total_demand = (
    demand_df[
        "demand_mbps"
    ].sum()
)


# Aggregate demand by traffic stream.
#
# This converts the source-level demand table into the
# four stream-level totals requested by the problem.

stream_demand_summary = (
    demand_df
    .groupby(
        "stream",
        as_index=False
    )["demand_mbps"]
    .sum()
)


# Print high-level demand totals.

print(
    "Number of source-stream commodities:",
    len(demand_df)
)

print(
    "Total synthetic demand:",
    total_demand,
    "Mbps"
)


# Display aggregate demand by stream.
```

```
stream_demand_summary
```

```
Number of source-stream commodities: 20
Total synthetic demand: 126 Mbps
```

Out[171…

	stream	demand_mbps
0	UK-L	26
1	UK-T	39
2	US-L	29
3	US-T	32

28.4 Interpretation of Aggregate Demand

The baseline scenario contains:

$$126 \text{ Mbps}$$

of total current demand.

The four traffic streams contribute:

$$UK\text{-}L = 26 \text{ Mbps}$$

$$UK\text{-}T = 39 \text{ Mbps}$$

$$US\text{-}L = 29 \text{ Mbps}$$

$$US\text{-}T = 32 \text{ Mbps}$$

Latency-Sensitive Demand

Total latency-sensitive demand is:

$$26 + 29 = 55 \text{ Mbps}$$

Latency-Tolerant Demand

Total latency-tolerant demand is:

$$39 + 32 = 71 \text{ Mbps}$$

Therefore:

$$55 + 71 = 126 \text{ Mbps}$$

Demand Mix

Latency-sensitive traffic represents approximately:

$$\frac{55}{126} \times 100 \approx 43.7\%$$

of total offered demand.

Latency-tolerant traffic represents approximately:

$$\frac{71}{126} \times 100 \approx 56.3\%$$

of total demand.

This means the baseline experiment contains enough demand in both service classes to create meaningful competition for shared network capacity.

Because tolerant traffic represents a somewhat larger share of the offered load, the model also provides an opportunity to test whether lower-latency capacity can be preserved for sensitive traffic during the Stage 2 routing optimization.

28.5 Demand by Source

Aggregate stream totals describe the traffic classes, but the physical location where demand enters the network is equally important.

The following table summarizes total offered demand by source node.

This helps identify which source locations contribute the greatest network load before routing begins.

In [172…

```
# Aggregate total offered demand by source node.

source_demand_summary = (
    demand_df
    .groupby(
        "source",
        as_index=False
    )["demand_mbps"]
    .sum()
    .sort_values(
        "demand_mbps",
        ascending=False
    )
    .reset_index(drop=True)
)
```

```
# Display total demand originating at each source.

source_demand_summary
```

Out [172…

	source	demand_mbps
0	B	37
1	S1	24
2	S3	24
3	S2	21
4	C	20

In [173…

```
# Confirm that source-level totals reconcile to the overall demand.

assert (
    source_demand_summary[
        "demand_mbps"
    ].sum()
    ==
    total_demand
)

print(
    "Source totals reconcile to total demand:",
    total_demand,
    "Mbps"
)
```

```
Source totals reconcile to total demand: 126 Mbps
```

28.6 Interpretation of Demand by Source

The source-level summary shows that demand is not distributed evenly across the network.

B Generates the Most Traffic

Node B contributes:

$$37 \text{ Mbps}$$

of total offered demand.

Because B has access to both the C and D corridors, this relatively high demand

is useful for testing how the optimizer distributes traffic across alternative routes.

S1 and S3

S1 and S3 each generate:

$$24 \text{ Mbps}$$

of total demand.

However, they enter the network through different locations:

- S1 enters through A;
- S3 enters through B.

Therefore, equal offered demand does not imply equal network impact.

S2

S2 generates:

$$21 \text{ Mbps}$$

and must also enter through A.

This creates additional competition with S1 for downstream capacity.

C

C generates the smallest total demand:

$$20 \text{ Mbps}$$

but is directly adjacent to UK and US.

This creates an important experimental contrast:

> Network service may depend not only on how much traffic a source generates, but also on how much shared network capacity that traffic must consume before reaching its exit.

28.7 Visualize Demand by Stream

In [174…

```
# Create a bar chart showing total offered demand
# for each of the four traffic streams.
```

```
plt.figure(
    figsize=(9, 5)
)

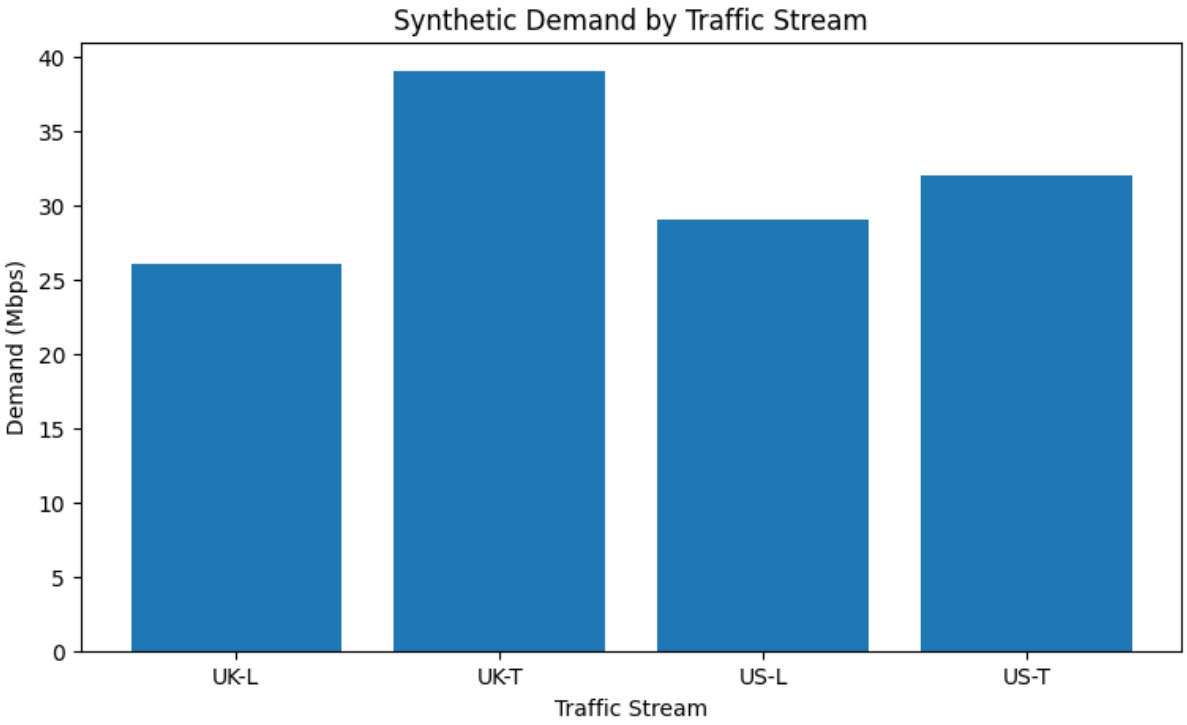
plt.bar(
    stream_demand_summary[
        "stream"
    ],
    stream_demand_summary[
        "demand_mbps"
    ]
)

plt.xlabel(
    "Traffic Stream"
)

plt.ylabel(
    "Demand (Mbps)"
)

plt.title(
    "Synthetic Demand by Traffic Stream"
)

plt.show()
```



Interpretation of Stream-Demand Visualization

The chart shows that all four traffic streams contribute materially to the offered

network load.

UK-T has the largest aggregate demand at 39 Mbps, while UK-L has the smallest aggregate demand at 26 Mbps.

Importantly, no single stream dominates the entire workload.

This creates a balanced enough test environment to evaluate shared-capacity competition across both destinations and both latency classes.

28.8 Visualize Demand by Source

In [175…

```
# Create a bar chart showing total offered demand
# originating at each traffic-generating source.

plt.figure(
    figsize=(9, 5)
)

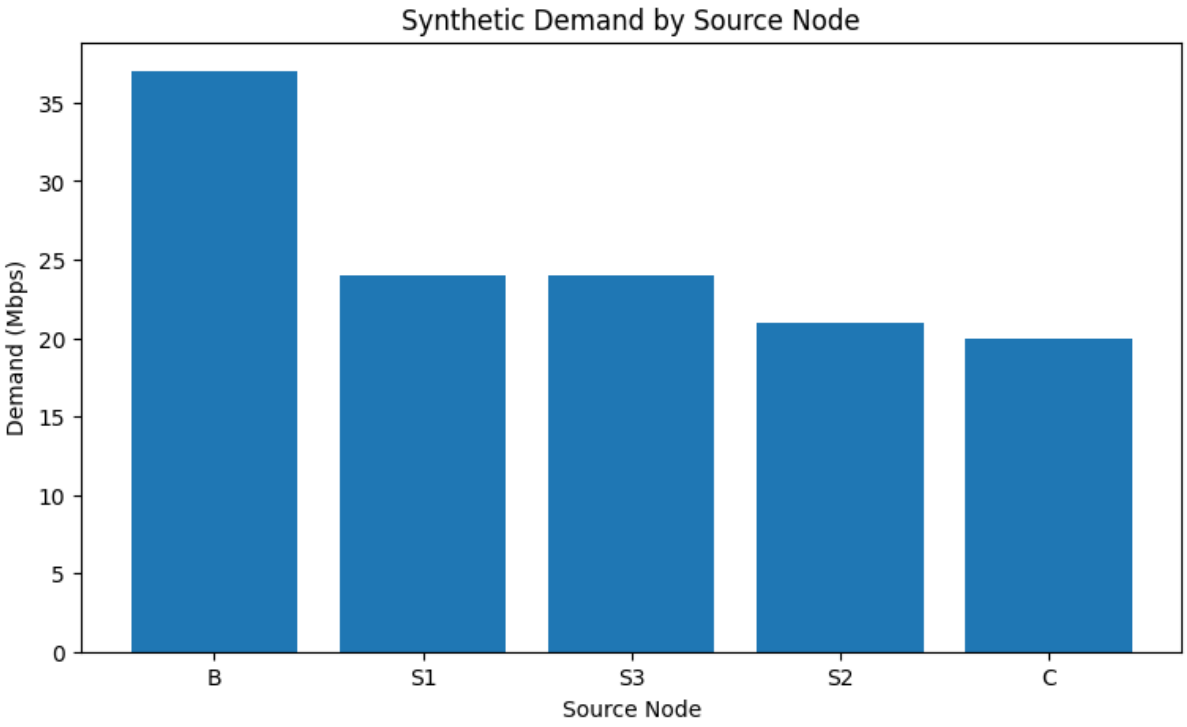
plt.bar(
    source_demand_summary[
        "source"
    ],
    source_demand_summary[
        "demand_mbps"
    ]
)

plt.xlabel(
    "Source Node"
)

plt.ylabel(
    "Demand (Mbps)"
)

plt.title(
    "Synthetic Demand by Source Node"
)

plt.show()
```



Interpretation of Source-Demand Visualization

The source-demand chart shows that B generates the greatest total load at 37 Mbps.

However, demand magnitude alone does not determine how difficult that traffic is to serve.

For example, C generates only 20 Mbps and is directly adjacent to both exits, while S1 and S2 must consume shared upstream and internal-link capacity before reaching UK or US.

The later optimization results will therefore show whether network position and route availability affect service outcomes in addition to raw demand magnitude.

28.9 Source-by-Stream Demand Matrix/Heatmap

A source-by-stream matrix provides a compact view of how the 126 Mbps of offered demand is distributed across the 20 routing commodities.

This is useful because the optimization operates at the source-stream level rather than only at the four aggregate-stream totals.

In [177…
```
# Pivot the source-stream demand table into a matrix.
```

```
#
# Rows    = source nodes
# Columns = traffic streams
# Values  = demand in Mbps

demand_matrix = demand_df.pivot(
    index="source",
    columns="stream",
    values="demand_mbps"
)


# Reorder rows and columns so the matrix follows the
# logical network and traffic-class ordering.

demand_matrix = demand_matrix.reindex(
    index=[
        "S1",
        "S2",
        "S3",
        "B",
        "C"
    ],
    columns=[
        "UK-L",
        "UK-T",
        "US-L",
        "US-T"
    ]
)


# Display the numerical matrix first.

demand_matrix
```

Out[177…

stream	UK-L	UK-T	US-L	US-T
source				
S1	6	7	5	6
S2	5	6	5	5
S3	5	7	6	6
B	6	14	8	9
C	4	5	5	6

In [178…

```
# Visualize the source-by-stream demand matrix.
#
# imshow is used instead of seaborn so the chart remains
```

```
# consistent with the notebook's matplotlib-based visuals.

fig, ax = plt.subplots(
    figsize=(8, 5)
)

image = ax.imshow(
    demand_matrix.values,
    aspect="auto"
)


# Label the x-axis with traffic streams.

ax.set_xticks(
    range(
        len(demand_matrix.columns)
    )
)

ax.set_xticklabels(
    demand_matrix.columns
)


# Label the y-axis with source nodes.

ax.set_yticks(
    range(
        len(demand_matrix.index)
    )
)

ax.set_yticklabels(
    demand_matrix.index
)


# Add each numerical demand value directly inside its cell.

for i in range(
    len(demand_matrix.index)
):

    for j in range(
        len(demand_matrix.columns)
    ):

        ax.text(
            j,
            i,
            f"{demand_matrix.iloc[i, j]:.0f}",
```

```
            ha="center",
            va="center"
        )


# Add axis labels and title.

ax.set_xlabel(
    "Traffic Stream"
)

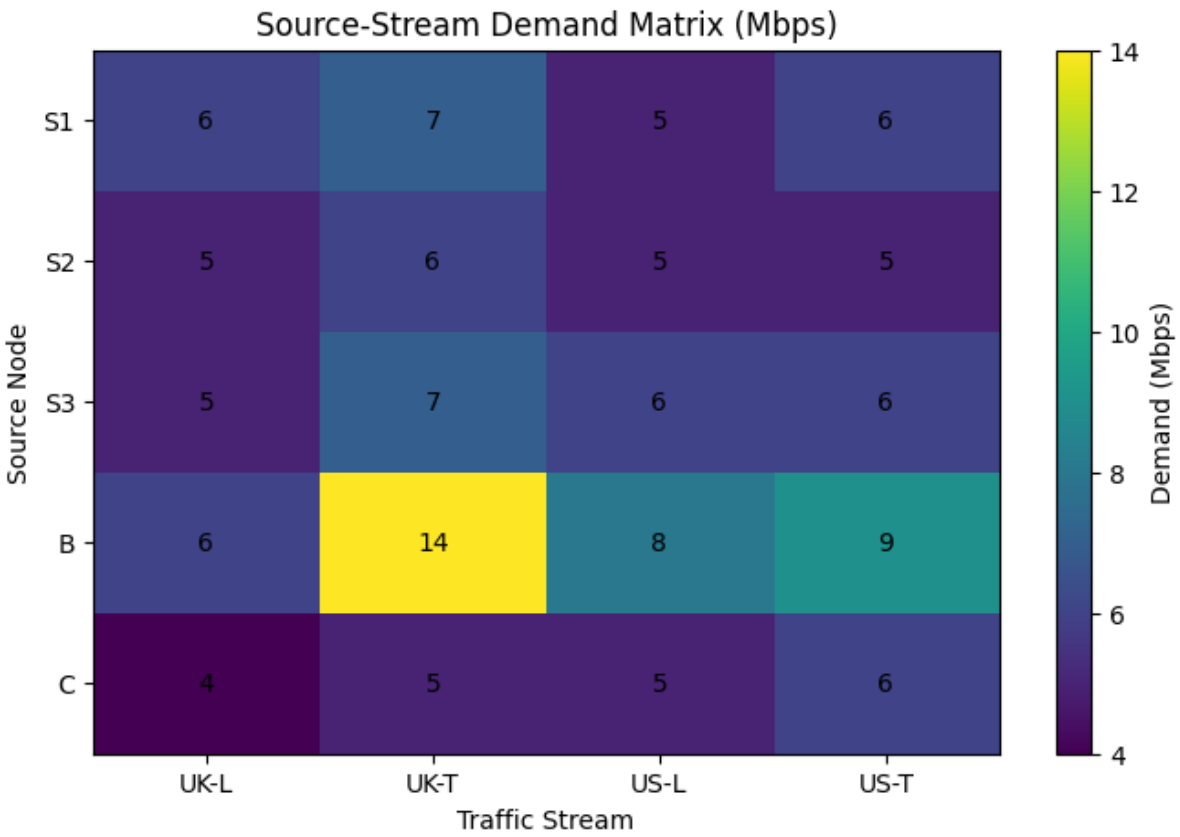
ax.set_ylabel(
    "Source Node"
)

ax.set_title(
    "Source-Stream Demand Matrix (Mbps)"
)


# Add a scale showing the demand magnitude.

plt.colorbar(
    image,
    ax=ax,
    label="Demand (Mbps)"
)

plt.show()
```



28.10 Interpretation of Source-Stream Demand Matrix

The source-stream matrix makes several demand patterns immediately visible.

Most source-stream demands fall within a relatively narrow range of approximately:

$$4\text{–}9 \text{ Mbps}$$

which prevents most commodities from dominating the network individually.

The main exception is:

$$B/UK\text{-}T = 14 \text{ Mbps}$$

which is the largest individual source-stream demand in the baseline dataset.

This is useful experimentally because B also has multiple route alternatives through C and D.

The matrix therefore provides a compact baseline against which later delivered and lost demand can be compared.

A particularly useful later visualization will be to place this offered-demand matrix beside a delivered-demand matrix.

That comparison will make it immediately visible which source-stream commodities were fully served, partially served, or dropped by each routing formulation.

28.11 Destination Demand Summary

In [179…
```
# Aggregate offered demand by required exit.

destination_demand_summary = (
    demand_df
    .groupby(
        "destination",
        as_index=False
    )["demand_mbps"]
    .sum()
)

destination_demand_summary
```

Out[179…

	destination	demand_mbps
0	UK	65
1	US	61

Interpretation of Destination Demand

Total offered demand is relatively balanced across the two exits:

$$UK = 65 \text{ Mbps}$$

and:

$$US = 61 \text{ Mbps}$$

This is useful experimentally because the network is not overwhelmingly biased toward one destination.

Both UK and US exit corridors therefore have the opportunity to become operationally relevant during optimization.

28.12.1 Demand Profile — Business and Network Implications

Results Analysis

The baseline network contains:

$$126 \text{ Mbps}$$

of total offered demand.

Traffic is not distributed evenly across:

- sources;
- destinations;
- service classes.

UK-bound demand totals:

$$65 \text{ Mbps},$$

while US-bound demand totals:

$$61 \text{ Mbps}.$$

Latency-sensitive traffic totals:

$$55 \text{ Mbps},$$

while latency-tolerant traffic totals:

$$71 \text{ Mbps}.$$

Therefore, approximately:

$$43.65\%$$

of total demand is latency-sensitive.

Why This Matters

Differences in demand concentration influence which links experience the greatest pressure.

A source generating more traffic may place disproportionate load on its available upstream corridors.

Similarly, destination imbalance can increase pressure on particular exit links.

The traffic-class mixture also matters because the Stage 2 objective treats latency-sensitive traffic differently from tolerant traffic.

Business Impact

Demand should not be summarized using total network Mbps alone.

Operational monitoring should also track:

- source-level demand;
- destination-level demand;
- traffic-class demand;
- sensitive vs. tolerant composition.

Changes in any of these dimensions can change the optimal routing decision even when total network demand remains unchanged.

28.12 Section Summary

Section 28 defines the traffic demand that will be routed through the directed network.

The baseline synthetic demand contains:

- 5 traffic-generating source nodes;
- 4 traffic classes;
- 20 source-stream commodities;
- 126 Mbps of total offered demand.

Aggregate demand is:

- UK-L: 26 Mbps;
- UK-T: 39 Mbps;
- US-L: 29 Mbps;
- US-T: 32 Mbps.

Total demand by destination is relatively balanced:

$$UK = 65 \text{ Mbps}$$

$$US = 61 \text{ Mbps}$$

while demand by source varies from:

$$20 \text{ Mbps}$$

at C to:

$$37 \text{ Mbps}$$

at B.

Experimental Significance

The demand dataset is deliberately heterogeneous but not dominated by a single traffic class or source.

This allows the optimization to test several important questions:

1. Which source-stream demands can be fully delivered?
2. Which demands are partially delivered or lost under congestion?
3. Does network position influence service outcomes?
4. How does traffic compete across UK and US exit corridors?
5. Does multipath routing improve throughput when capacity becomes fragmented?
6. Can latency-sensitive traffic be moved onto lower-latency routes without reducing maximum throughput?

The next step is to combine the directed network from Section 27 with these source-stream commodities and generate the feasible source-to-exit paths available to each demand.

29. Final Candidate Path Generation

The directed network and source-stream demand are now combined to generate the feasible routes available to each commodity.

A **commodity** is one source-stream combination such as:

$$S1/UK\text{-}L$$

or:

$$B/US\text{-}T$$

For every commodity, the model generates all valid simple paths from the commodity's source node to its required exit.

A candidate path is considered valid only if it:

- begins at the commodity's source;
- terminates at the commodity's required exit;
- contains no repeated node;
- does not pass through the opposite exit;

- treats UK and US as terminal nodes.

The physical topology is unchanged from the earlier exploratory analysis.

However, candidate-path characteristics are now recalculated using the heterogeneous synthetic network developed in Section 26.

Therefore, each path has its own:

- number of links;
- isolated path-capacity bound;
- end-to-end latency;
- ordered list of directed arcs.

These candidate paths form the routing alternatives available to the optimization models in later sections.

29.1 Generate Valid Directed Paths

The following function generates all simple directed paths from a source node to its required destination.

The opposite exit is explicitly excluded from any intermediate position in the route.

For example, a UK-bound path may not:

$$S1 \to \cdots \to US \to \cdots \to UK$$

because traffic leaves the network once it reaches an exit.

Using simple paths also prevents cycles such as:

$$A \to B \to A$$

which would consume additional capacity and latency without helping the traffic reach its destination.

In [180…

```
def get_valid_paths_directed(
    graph,
    source,
    destination
):
    """
    Generate all valid simple directed paths from a source
    to its required network exit.
```

```
    Parameters
    ----------
    graph : nx.DiGraph
        Directed network containing capacity and latency attributes.

    source : str
        Node where the commodity enters the network.

    destination : str
        Required exit for the commodity: either 'UK' or 'US'.

    Returns
    -------
    valid_paths : list
        List of valid simple node sequences from source to destination
    """

    # Determine which exit must NOT appear as an intermediate node.
    #
    # UK-bound traffic may not pass through US.
    # US-bound traffic may not pass through UK.

    opposite_exit = (
        "US"
        if destination == "UK"
        else "UK"
    )


    # Create an empty list to store valid candidate paths.

    valid_paths = []


    # Generate every simple directed path from the source
    # to the required destination.
    #
    # NetworkX simple paths do not repeat nodes, which prevents cycles

    for path in nx.all_simple_paths(
        graph,
        source=source,
        target=destination
    ):

        # Exclude any route that reaches the opposite exit
        # before reaching its required destination.
        #
        # path[:-1] excludes the final destination node itself.

        if opposite_exit in path[:-1]:
            continue
```

```
        # Store the path if it satisfies all validity rules.
        valid_paths.append(path)

    # Return the complete set of valid routes.
    return valid_paths
```

29.2 Calculate Final Path Statistics

Every feasible path is assigned several routing characteristics.

For path p, the directed arcs are:

$$E_p$$

The isolated path-capacity bound is:

$$C_p = \min_{e \in p} C_e$$

because the narrowest link limits the maximum traffic that could travel through that path if no other traffic were competing for capacity.

Path latency is:

$$L_p = \sum_{e \in p} L_e$$

because link latency is additive.

It is important to distinguish the isolated path-capacity value from actual available routing capacity.

The value:

$$C_p$$

does **not** mean that every candidate path independently receives that amount of bandwidth.

Paths sharing the same directed arc must share that arc's capacity.

In [181…

```
def directed_path_statistics(
    graph,
    path
```

```
):
    """
    Calculate structural statistics for one directed path.

    Parameters
    ----------
    graph : nx.DiGraph
        Directed network containing capacity and latency attributes.

    path : list
        Ordered sequence of nodes representing one feasible route.

    Returns
    -------
    dict
        Directed edges, number of links, isolated path-capacity bound,
        and total end-to-end latency.
    """

    # Convert the node sequence into ordered directed arcs.
    #
    # Example:
    #   S1 -> A -> C -> UK
    #
    # becomes:
    #   (S1, A), (A, C), (C, UK)

    path_edges = list(
        zip(
            path[:-1],
            path[1:]
        )
    )


    # Calculate the isolated path-capacity bound.
    #
    # A path can carry no more than the capacity of its
    # smallest-capacity directed arc.

    path_capacity = min(
        graph[u][v]["capacity_mbps"]
        for u, v in path_edges
    )


    # Calculate total end-to-end latency by summing
    # the latency contribution of every directed arc.

    path_latency = sum(
        graph[u][v]["latency_ms"]
        for u, v in path_edges
```

```
    )


    # Return all path characteristics in one dictionary.

    return {
        "path_edges": path_edges,
        "num_links": len(path_edges),
        "path_capacity_mbps": path_capacity,
        "path_latency_ms": path_latency
    }
```

29.3 Build the Candidate Path Table

The candidate-path table combines:

- source-stream demand information from Section 28;
- feasible directed routes from the network;
- path capacity;
- path latency.

Each row represents **one feasible routing option for one source-stream commodity**.

Therefore, the same physical route may appear more than once when different traffic classes share the same source and destination.

For example:

- S1 / UK-L;
- S1 / UK-T;

have the same network route alternatives, but they remain separate commodities because they represent different traffic classes and may receive different optimization treatment.

In [182…

```
# Create an empty list that will store one record
# for every feasible source-stream path.

candidate_records = []


# Create a unique integer identifier for each candidate path.
#
# This path_id will later be used as the key for optimization
# decision variables.
```

```
path_id = 0


# Iterate through all 20 source-stream commodities.

for _, commodity in demand_df.iterrows():

    # Generate every valid path from this commodity's source
    # to its required destination.

    paths = get_valid_paths_directed(
        DG,
        commodity["source"],
        commodity["destination"]
    )


    # Calculate statistics for each feasible path.

    for path in paths:

        stats = directed_path_statistics(
            DG,
            path
        )


        # Store both commodity information and route information
        # in one candidate-path record.

        candidate_records.append({
            "path_id": path_id,
            "commodity_id": commodity["commodity_id"],
            "source": commodity["source"],
            "stream": commodity["stream"],
            "destination": commodity["destination"],
            "demand_mbps": commodity["demand_mbps"],
            "path": " -> ".join(path),
            "path_edges": stats["path_edges"],
            "num_links": stats["num_links"],
            "path_capacity_mbps": stats["path_capacity_mbps"],
            "path_latency_ms": stats["path_latency_ms"]
        })


        # Increment the identifier before processing
        # the next candidate path.

        path_id += 1


# Convert all candidate-path records into a DataFrame.
```

```
candidate_paths_df = pd.DataFrame(
    candidate_records
)


# Print the total number of candidate routes generated.

print(
    "Number of candidate paths:",
    len(candidate_paths_df)
)


# Display the first 20 candidate-path records.

candidate_paths_df.head(20)
```

```
Number of candidate paths: 68
```

Out[182…

	path_id	commodity_id	source	stream	destination	demand_mbps	path
0	0	S1_UK-L	S1	UK-L	UK	6	S1 -> A -> B -> C -> UK
1	1	S1_UK-L	S1	UK-L	UK	6	S1 -> A -> B -> D -> UK
2	2	S1_UK-L	S1	UK-L	UK	6	S1 -> A -> C -> B -> D -> UK
3	3	S1_UK-L	S1	UK-L	UK	6	S1 -> A -> C -> UK
4	4	S1_UK-T	S1	UK-T	UK	7	S1 -> A -> B -> C -> UK

5	5	S1_UK-T	S1	UK-T	UK	7	S1 -> A -> B -> D -> UK
6	6	S1_UK-T	S1	UK-T	UK	7	S1 -> A -> C -> B -> D -> UK
7	7	S1_UK-T	S1	UK-T	UK	7	S1 -> A -> C -> UK
8	8	S1_US-L	S1	US-L	US	5	S1 -> A -> B -> C -> US
9	9	S1_US-L	S1	US-L	US	5	S1 -> A -> B -> D -> US
10	10	S1_US-L	S1	US-L	US	5	S1 -> A -> C -> B -> D -> US
11	11	S1_US-L	S1	US-L	US	5	S1 -> A -> C -> US
12	12	S1_US-T	S1	US-T	US	6	S1 -> A -> B -> C -> US
							S1 -

13	13	S1_US-T	S1	US-T	US	6	> A -> B -> D -> US
14	14	S1_US-T	S1	US-T	US	6	S1 -> A -> C -> B -> D -> US
15	15	S1_US-T	S1	US-T	US	6	S1 -> A -> C -> US
16	16	S2_UK-L	S2	UK-L	UK	5	S2 -> A -> B -> C -> UK
17	17	S2_UK-L	S2	UK-L	UK	5	S2 -> A -> B -> D -> UK
18	18	S2_UK-L	S2	UK-L	UK	5	S2 -> A -> C -> B -> D -> UK
19	19	S2_UK-L	S2	UK-L	UK	5	S2 -> A -> C -> UK

29.4 Interpretation of the Candidate Path Table

The final candidate-path table contains:

feasible source-stream routing alternatives.

Each row describes:

- the source-stream commodity;
- required destination;
- offered demand;
- complete node sequence;
- directed arcs used;
- number of links;
- isolated path-capacity bound;
- total end-to-end latency.

The first several rows illustrate the routing tradeoffs clearly.

Example: S1 / UK-L

S1 / UK-L has four feasible paths to UK.

Route 1

$$S1 \to A \to B \to C \to UK$$

This route has:

$$4$$

links,

$$18 \text{ Mbps}$$

isolated path capacity,

and:

$$44 \text{ ms}$$

of latency.

Route 2

$$S1 \to A \to B \to D \to UK$$

This route has:

$$20 \text{ Mbps}$$

of isolated path capacity but:

$$61 \text{ ms}$$

of latency.

It therefore provides somewhat greater capacity than Route 1 but substantially higher latency.

Route 3

$$S1 \to A \to C \to B \to D \to UK$$

This five-link route has:

$$18 \text{ Mbps}$$

of isolated path capacity and:

$$64 \text{ ms}$$

of latency.

It is both longer and slower, but remains a valid alternative if other routes become congested.

Route 4

$$S1 \to A \to C \to UK$$

This route has:

$$25 \text{ Mbps}$$

of isolated capacity and only:

$$27 \text{ ms}$$

of latency.

For S1 / UK-L, this route dominates the other listed alternatives when considered in isolation because it provides both greater isolated capacity and lower latency.

However, network-wide optimization is still required because:

$$A \to C$$

and:

$$C \to UK$$

must be shared with traffic from other commodities.

Important Distinction

A route that appears best for one commodity in isolation may not be best for the network as a whole.

The optimization must consider simultaneous competition across all 20 commodities before assigning traffic to these routes.

29.5 Validate the Candidate Path Set

Before candidate routes are passed to the optimization models, the path set is validated.

Every candidate path should satisfy the following conditions:

- the path begins at the stated source;
- the path ends at the required destination;
- the opposite exit does not appear before the destination;
- no node is repeated;
- every directed arc exists in the directed graph;
- the stored number of links is correct;
- stored latency matches the sum of arc latencies;
- stored path capacity matches the minimum arc capacity;
- every source-stream commodity has at least one feasible route.

These checks verify that the optimization model is receiving a valid routing space.

In [183…

```
# ------------------------------------------------------------
# Check 1: Candidate path IDs must be unique
# ------------------------------------------------------------

assert (
    candidate_paths_df["path_id"].nunique()
    ==
    len(candidate_paths_df)
)


# ------------------------------------------------------------
# Check 2: Every commodity must have at least one route
# ------------------------------------------------------------

commodities_with_paths = set(
    candidate_paths_df["commodity_id"]
)

expected_commodities = set(
    demand_df["commodity_id"]
```

```
)

assert (
    commodities_with_paths
    ==
    expected_commodities
)


# ------------------------------------------------------------
# Validate every candidate route individually
# ------------------------------------------------------------

for _, row in candidate_paths_df.iterrows():

    # Convert the stored display string back to a node sequence.
    path_nodes = [
        node.strip()
        for node in row["path"].split("->")
    ]


    # ------------------------------------------------------------
    # Check 3: Correct source
    # ------------------------------------------------------------

    assert (
        path_nodes[0]
        ==
        row["source"]
    )


    # ------------------------------------------------------------
    # Check 4: Correct destination
    # ------------------------------------------------------------

    assert (
        path_nodes[-1]
        ==
        row["destination"]
    )


    # ------------------------------------------------------------
    # Check 5: Opposite exit may not appear before destination
    # ------------------------------------------------------------

    opposite_exit = (
        "US"
        if row["destination"] == "UK"
        else "UK"
```

```
    )

    assert (
        opposite_exit
        not in path_nodes[:-1]
    )


    # ----------------------------------------------------------
    # Check 6: Path must be simple
    # ----------------------------------------------------------

    # A simple path contains no repeated nodes.

    assert (
        len(path_nodes)
        ==
        len(set(path_nodes))
    )


    # ----------------------------------------------------------
    # Check 7: All directed arcs must exist in DG
    # ----------------------------------------------------------

    for u, v in row["path_edges"]:

        assert DG.has_edge(
            u,
            v
        )


    # ----------------------------------------------------------
    # Check 8: Stored number of links must be correct
    # ----------------------------------------------------------

    assert (
        row["num_links"]
        ==
        len(row["path_edges"])
    )


    # ----------------------------------------------------------
    # Check 9: Recalculate and verify path latency
    # ----------------------------------------------------------

    recalculated_latency = sum(
        DG[u][v]["latency_ms"]
        for u, v in row["path_edges"]
    )
```

```
    assert (
        recalculated_latency
        ==
        row["path_latency_ms"]
    )


    # ------------------------------------------------------
    # Check 10: Recalculate and verify path capacity
    # ------------------------------------------------------

    recalculated_capacity = min(
        DG[u][v]["capacity_mbps"]
        for u, v in row["path_edges"]
    )

    assert (
        recalculated_capacity
        ==
        row["path_capacity_mbps"]
    )


# ------------------------------------------------------------
# Print validation summary
# ------------------------------------------------------------

print(
    "Candidate path validation passed."
)

print(
    "Commodities represented:",
    candidate_paths_df[
        "commodity_id"
    ].nunique()
)

print(
    "Candidate paths:",
    len(candidate_paths_df)
)

print(
    "Invalid paths detected: 0"
)
```

```
Candidate path validation passed.
Commodities represented: 20
Candidate paths: 68
Invalid paths detected: 0
```

29.6 Interpretation of Candidate Path Validation

All candidate-path validation checks pass.

The final routing space contains:

- all 20 source-stream commodities;
- 68 valid candidate paths;
- no incorrect destinations;
- no routes through the opposite exit;
- no repeated-node cycles;
- no invalid directed arcs;
- internally consistent path-capacity calculations;
- internally consistent path-latency calculations.

This confirms that the feasible routing space is correctly constructed before optimization begins.

This validation is important because an optimization model can produce a mathematically optimal solution to an incorrectly constructed path set.

Validating the candidate paths first helps ensure that later solver results represent the intended network problem.

29.7 Routing Alternatives per Commodity

The number of feasible paths available to each commodity provides a measure of routing flexibility.

A commodity with more candidate paths has more opportunities to avoid congestion.

A commodity with fewer paths is more structurally dependent on a small portion of the network.

In [184…

```
# Count the number of feasible candidate paths
# available to each source-stream commodity.

commodity_path_counts = (
    candidate_paths_df
    .groupby(
        [
            "commodity_id",
            "source",
            "stream",
```

```
            "destination"
        ],
        as_index=False
    )
    .size()
    .rename(
        columns={
            "size":
                "number_of_candidate_paths"
        }
    )
)


# Sort from most routing flexibility to least.

commodity_path_counts = (
    commodity_path_counts
    .sort_values(
        [
            "number_of_candidate_paths",
            "source",
            "stream"
        ],
        ascending=[
            False,
            True,
            True
        ]
    )
    .reset_index(drop=True)
)

commodity_path_counts
```

Out [184...

	commodity_id	source	stream	destination	number_of_candidate_paths
0	S1_UK-L	S1	UK-L	UK	4
1	S1_UK-T	S1	UK-T	UK	4
2	S1_US-L	S1	US-L	US	4
3	S1_US-T	S1	US-T	US	4
4	S2_UK-L	S2	UK-L	UK	4
5	S2_UK-T	S2	UK-T	UK	4
6	S2_US-L	S2	US-L	US	4
7	S2_US-T	S2	US-T	US	4
8	B_UK-L	B	UK-L	UK	3
9	B_UK-T	B	UK-T	UK	3
10	B_US-L	B	US-L	US	3
11	B_US-T	B	US-T	US	3
12	C_UK-L	C	UK-L	UK	3
13	C_UK-T	C	UK-T	UK	3
14	C_US-L	C	US-L	US	3
15	C_US-T	C	US-T	US	3
16	S3_UK-L	S3	UK-L	UK	3
17	S3_UK-T	S3	UK-T	UK	3
18	S3_US-L	S3	US-L	US	3
19	S3_US-T	S3	US-T	US	3

29.8 Interpretation of Routing Flexibility

Routing flexibility differs by source node.

S1 and S2

Each S1 and S2 commodity has four candidate routes to its required exit.

This provides several rerouting alternatives, although all S1 and S2 traffic must initially enter through A.

Therefore, having more end-to-end paths does not necessarily eliminate shared

upstream dependency.

S3

Each S3 commodity has three feasible routes.

All S3 traffic first enters through:

$$S3 \to B$$

but can subsequently move toward the exits through several internal corridors.

B

B also has three route alternatives to either exit and is positioned near the center of the routing network.

This provides meaningful flexibility between C and D.

C

C has three feasible routes even though it is directly adjacent to both exits.

Its direct paths are likely to be attractive because they use fewer shared network resources, while longer alternatives remain available if the direct exit links become constrained.

Modeling Significance

The number of candidate routes alone does not determine routing quality.

What matters is:

- which arcs those routes share;
- available residual capacity;
- path latency;
- competing traffic.

The optimizer must therefore evaluate path alternatives jointly rather than simply choosing the source with the greatest number of routes.

29.9 Candidate Path Counts by Source

In [185…

```
# Summarize the number of candidate routes available
```

```
# for each source-destination combination.
#
# UK-L and UK-T share the same physical routes to UK.
# US-L and US-T share the same physical routes to US.
#
# This table focuses on topology rather than traffic class.

source_destination_path_counts = (
    candidate_paths_df[
        [
            "source",
            "destination",
            "path"
        ]
    ]
    .drop_duplicates()
    .groupby(
        [
            "source",
            "destination"
        ],
        as_index=False
    )
    .size()
    .rename(
        columns={
            "size":
                "number_of_physical_routes"
        }
    )
)

source_destination_path_counts
```

Out [185…

	source	destination	number_of_physical_routes
0	B	UK	3
1	B	US	3
2	C	UK	3
3	C	US	3
4	S1	UK	4
5	S1	US	4
6	S2	UK	4
7	S2	US	4
8	S3	UK	3
9	S3	US	3

In [186…

```
# Confirm consistency with the earlier topology analysis.

total_physical_routes = (
    source_destination_path_counts[
        "number_of_physical_routes"
    ].sum()
)

print(
    "Distinct physical source-to-exit routes:",
    total_physical_routes
)

assert total_physical_routes == 34
```

```
Distinct physical source-to-exit routes: 34
```

29.10 Why the Final Table Contains 68 Candidate Paths

The earlier topology analysis identified:

$$34$$

distinct physical source-to-exit routes.

Each UK physical route is available to two UK traffic classes:

- UK-L;
- UK-T.

Likewise, each US physical route is available to:

- US-L;
- US-T.

Therefore, the optimization contains:

$$34 \times 2 = 68$$

source-stream path alternatives.

The increase from 34 physical routes to 68 candidate optimization paths does not represent additional network topology.

Instead, it reflects the fact that different traffic classes are modeled as separate commodities even when they use the same physical route.

29.11 Candidate Path Capacity and Latency Summary

In [187…

```
# Produce descriptive statistics for candidate-path
# capacity, latency, and route length.

candidate_path_summary = (
    candidate_paths_df[
        [
            "num_links",
            "path_capacity_mbps",
            "path_latency_ms"
        ]
    ]
    .describe()
)

candidate_path_summary
```

Out [187…

	num_links	path_capacity_mbps	path_latency_ms
count	68.000000	68.000000	68.000000
mean	3.352941	20.794118	41.235294
std	1.033249	3.436237	16.215082
min	1.000000	18.000000	7.000000
25%	3.000000	18.000000	29.000000
50%	3.000000	20.000000	42.000000
75%	4.000000	22.000000	57.000000
max	5.000000	30.000000	66.000000

29.12 Candidate Path Capacity-Latency Tradeoff

The link-level scatter plot in Section 26 showed the characteristics of individual physical connections.

The following chart moves the analysis to the route level.

Each point represents one candidate source-stream path according to:

- isolated path-capacity bound;
- total end-to-end latency.

This makes it possible to see how link-level characteristics combine into complete routing alternatives.

In [188…

```
# Create a scatter plot showing the capacity–latency
# relationship across all candidate source–stream paths.

plt.figure(
    figsize=(10, 6)
)

plt.scatter(
    candidate_paths_df[
        "path_capacity_mbps"
    ],
    candidate_paths_df[
        "path_latency_ms"
    ],
    alpha=0.7
)
```

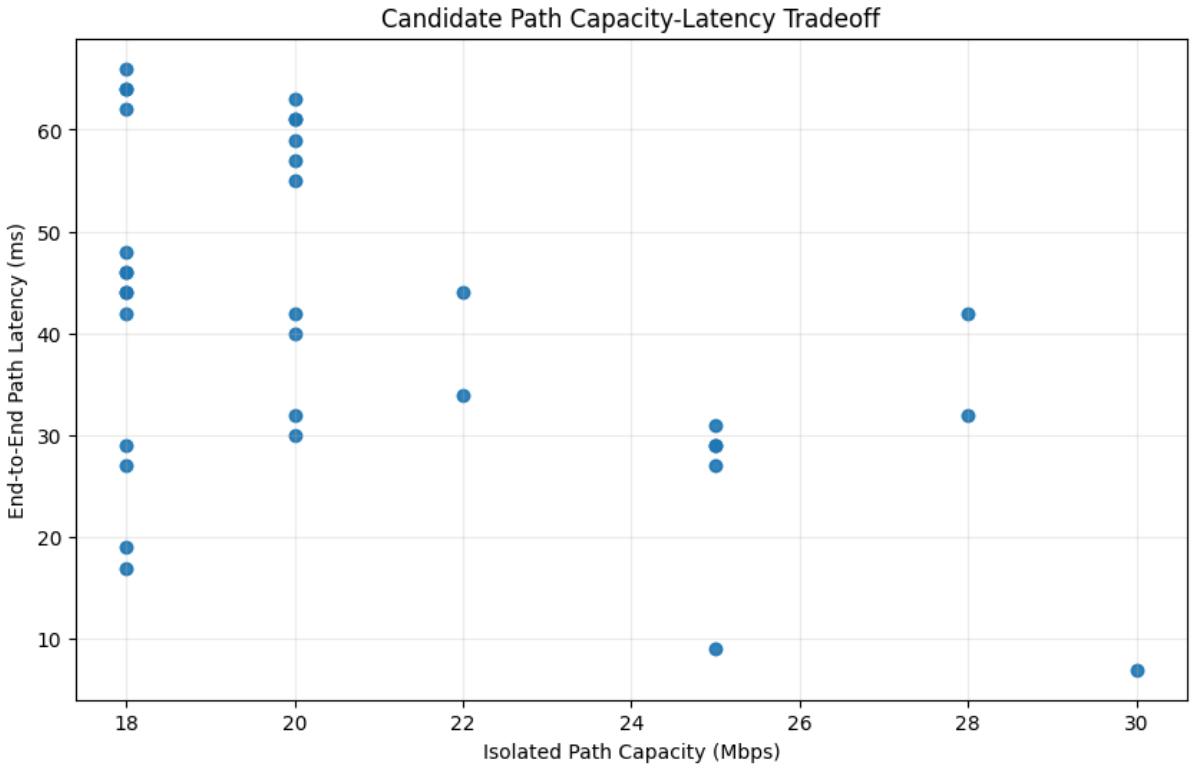
```
plt.xlabel(
    "Isolated Path Capacity (Mbps)"
)

plt.ylabel(
    "End-to-End Path Latency (ms)"
)

plt.title(
    "Candidate Path Capacity-Latency Tradeoff"
)

plt.grid(
    alpha=0.25
)

plt.show()
```



29.13 Interpretation of Candidate Path Capacity-Latency Tradeoff

The candidate-path scatter plot shows that complete routes exhibit a wider range of performance characteristics than individual links.

Each route combines several directed arcs, so:

- path capacity is constrained by the smallest-capacity arc;
- path latency accumulates across every arc used.

Consequently, longer routes generally incur greater latency and may also inherit a relatively narrow internal bottleneck.

The chart also shows that some candidate routes offer relatively favorable combinations of capacity and latency, while others are clearly slower alternatives.

However, even an apparently superior path cannot be evaluated independently.

Several commodities may compete for the same attractive arcs.

Therefore, the final optimization must determine whether using a slightly slower route for one commodity improves network-wide throughput or preserves lower-latency capacity for sensitive traffic.

29.13.1 Candidate-Path Asymmetry — Results Analysis and Business Impact

Results Analysis

The candidate-path analysis shows that routing flexibility is not distributed equally across sources.

Some source-destination combinations have several feasible alternatives.

Others depend more strongly on a limited set of corridors.

Therefore, two commodities with identical demand may face very different routing opportunities.

Why This Happens

Routing flexibility is determined by topology.

A source located near multiple internal corridors can redirect traffic when one path becomes congested.

A source with fewer alternatives may have its service limited even when unused capacity exists elsewhere in the network.

Therefore:

$$\boxed{\text{Unused Network Capacity} \neq \text{Usable Capacity for Every Source}}$$

Capacity is useful only if a feasible source-to-destination path can reach it.

Business Impact

Sources with limited route diversity represent greater operational risk.

They may be more sensitive to:

- link degradation;
- congestion;
- destination-facing constraints.

Production network planning should therefore evaluate **path diversity by source**, not simply total network capacity.

Additional redundancy may be more valuable for structurally constrained sources than for already well-connected locations.

29.14 Optional: Path Count Bar Chart

In [189…

```
# Show routing flexibility by source-destination pair.

path_count_plot_df = (
    source_destination_path_counts.copy()
)

path_count_plot_df["pair"] = (
    path_count_plot_df["source"]
    + " → "
    + path_count_plot_df["destination"]
)

plt.figure(
    figsize=(11, 5)
)

plt.bar(
    path_count_plot_df["pair"],
    path_count_plot_df[
        "number_of_physical_routes"
    ]
)

plt.xlabel(
    "Source-Destination Pair"
)

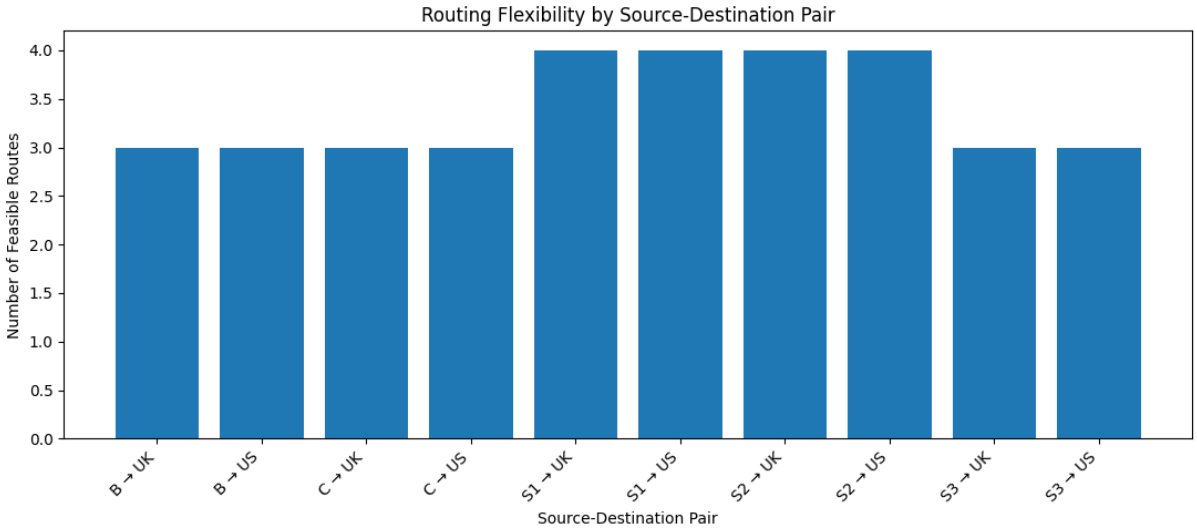
plt.ylabel(
    "Number of Feasible Routes"
)
```

```
plt.title(
    "Routing Flexibility by Source–Destination Pair"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()

plt.show()
```



29.15 Section Summary

Section 29 constructs the complete feasible routing space used by the optimization models.

The final candidate-path dataset contains:

- 20 source-stream commodities;
- 34 distinct physical source-to-exit routes;
- 68 source-stream candidate paths.

Every candidate path:

- begins at the correct source;
- terminates at the correct exit;
- avoids the opposite exit;
- contains no repeated nodes;
- consists only of valid directed arcs;
- has a verified path-capacity bound;

- has a verified end-to-end latency.

Routing Flexibility

Different sources have different routing flexibility.

S1 and S2 have several end-to-end alternatives but share structural dependence on A.

S3 and B have useful alternatives through C and D.

C is physically close to both exits and has direct low-latency routes available.

Experimental Significance

The candidate-path set exposes the central optimization challenge.

The problem is not simply:

> Which route is shortest?

or:

> Which route has the largest isolated path capacity?

Instead, the model must determine:

> Which combination of paths should simultaneously carry all admitted traffic while respecting shared directed-link capacities and preserving low latency for sensitive traffic?

The next section evaluates the heterogeneous candidate paths in more detail and identifies the minimum-latency alternatives before optimization is performed.

30. Heterogeneous Path Analysis

The candidate routes generated in Section 29 now have different capacities and end-to-end latencies.

Unlike the earlier equal-link exploratory scenario:

- path capacity is no longer uniformly 30 Mbps;
- latency is no longer determined only by hop count;
- routes with the same number of links may have different total latency;

- different routes may expose materially different capacity-latency tradeoffs.

This section examines the routing space before optimization.

The primary goals are to:

1. identify the minimum-latency route available to each source-stream commodity;
2. determine which directed links are structurally favored by minimum-latency routing;
3. evaluate whether independently selecting the fastest route for every commodity would satisfy network capacity constraints;
4. establish a naive routing baseline against which the optimized models can later be compared.

This distinction is important because the minimum-latency route for an individual commodity may not be feasible when all commodities compete for shared network capacity.

The analysis therefore moves from:

> **individual path quality**

to:

> **network-wide consequences of everyone choosing individually attractive paths.**

30.1 Identify the Minimum-Latency Path for Each Commodity

For every source-stream commodity, the candidate paths are compared using their calculated end-to-end latency.

For path p:

$$L_p = \sum_{e \in p} L_e$$

The path or paths with the smallest value of:

$$L_p$$

are flagged as minimum-latency routes.

Because the synthetic network contains heterogeneous link latencies, minimum latency is determined from the actual sum of link-level latency rather than simply from hop count.

This provides the natural routing choice that an uncoordinated latency-minimizing policy would make for each commodity independently.

In [192…

```
# Add a Boolean column that identifies whether each candidate path
# is a minimum-latency route for its source-stream commodity.
#
# Start by assuming that no path is minimum latency.

candidate_paths_df[
    "is_min_latency"
] = False


# Group candidate routes by commodity.
#
# Each commodity may have several feasible paths, but all routes
# in a group correspond to the same source-stream demand.

for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    # Find the minimum end-to-end latency available
    # among all routes for this commodity.

    min_latency = (
        group[
            "path_latency_ms"
        ].min()
    )


    # Mark every path whose latency equals that minimum.
    #
    # This logic also supports tied minimum-latency paths
    # if ties occur.

    candidate_paths_df.loc[
        group.index,
        "is_min_latency"
    ] = (
        group[
            "path_latency_ms"
        ]
        == min_latency
```

```
    )


# Create a separate table containing only
# minimum-latency candidate paths.

minimum_latency_paths_df = (
    candidate_paths_df[
        candidate_paths_df[
            "is_min_latency"
        ]
    ]
    .sort_values(
        [
            "source",
            "stream",
            "path_latency_ms"
        ]
    )
    .reset_index(drop=True)
)


# Display the most important route-level characteristics.

minimum_latency_paths_df[
    [
        "commodity_id",
        "path",
        "path_capacity_mbps",
        "path_latency_ms"
    ]
]
```

Out [192...

	commodity_id	path	path_capacity_mbps	path_latency_ms
0	B_UK-L	B -> C -> UK	18	17
1	B_UK-T	B -> C -> UK	18	17
2	B_US-L	B -> C -> US	18	19
3	B_US-T	B -> C -> US	18	19
4	C_UK-L	C -> UK	30	7
5	C_UK-T	C -> UK	30	7
6	C_US-L	C -> US	25	9
7	C_US-T	C -> US	25	9
8	S1_UK-L	S1 -> A -> C -> UK	25	27
9	S1_UK-T	S1 -> A -> C -> UK	25	27
10	S1_US-L	S1 -> A -> C -> US	25	29
11	S1_US-T	S1 -> A -> C -> US	25	29
12	S2_UK-L	S2 -> A -> C -> UK	25	29
13	S2_UK-T	S2 -> A -> C -> UK	25	29
14	S2_US-L	S2 -> A -> C -> US	25	31
15	S2_US-T	S2 -> A -> C -> US	25	31
16	S3_UK-L	S3 -> B -> C -> UK	18	27
17	S3_UK-T	S3 -> B -> C -> UK	18	27
18	S3_US-L	S3 -> B -> C -> US	18	29
19	S3_US-T	S3 -> B -> C -> US	18	29

30.1.1 Interpretation of Minimum-Latency Results

The minimum-latency table identifies the individually fastest route available to each of the 20 source-stream commodities.

A major result is that the heterogeneous synthetic network produces a **unique minimum-latency path for every commodity**.

This differs from the earlier equal-link scenario, where several source-destination pairs had tied shortest routes.

Several patterns are visible.

B-Originating Traffic

B-originating traffic prefers the C corridor.

For example:

$$B \to C \to UK$$

has:

$$17 \text{ ms}$$

of latency and an isolated path-capacity bound of:

$$18 \text{ Mbps}$$

while the D alternative is slower.

Similarly:

$$B \to C \to US$$

has:

$$19 \text{ ms}$$

of latency.

C-Originating Traffic

C has the fastest source-to-exit routes in the network because it is directly adjacent to both exits.

For UK:

$$C \to UK$$

has:

$$7 \text{ ms}$$

latency.

For US:

$$C \to US$$

has:

$$9 \text{ ms}$$

latency.

S1-Originating Traffic

S1's minimum-latency UK route is:

$$S1 \to A \to C \to UK$$

with:

$$27 \text{ ms}$$

latency.

Its minimum-latency US route is:

$$S1 \to A \to C \to US$$

with:

$$29 \text{ ms}$$

latency.

S2-Originating Traffic

S2 follows the same structural pattern through A → C.

Its minimum-latency routes are:

$$29 \text{ ms}$$

to UK and:

$$31 \text{ ms}$$

to US.

S3-Originating Traffic

S3's fastest routes use B → C:

$$S3 \to B \to C \to UK$$

at:

$$27 \text{ ms}$$

and:

$$S3 \to B \to C \to US$$

at:

$$29 \text{ ms}$$

Key Structural Finding

Every commodity's minimum-latency route ultimately uses the **C corridor**.

The D corridor does not appear in any commodity's unique minimum-latency route.

This means that independently choosing the fastest route creates a strong structural concentration around node C.

30.1.2 Business and Operational Impact

The minimum-latency results reveal an important operational risk.

If each traffic stream chooses its fastest route independently, the network will naturally concentrate traffic on the same low-latency corridors.

From a business and network-operations perspective, this means that:

- the fastest route for one user or stream may not scale when many users make the same choice;
- local routing decisions can create global congestion;
- low-latency infrastructure can become disproportionately valuable and heavily contested;
- slower alternate routes may be necessary even when they are not individually preferred.

This is directly relevant to service quality.

Without coordinated routing, the network could produce poor delivered throughput despite having unused capacity elsewhere.

The minimum-latency table therefore provides an important benchmark:

> **It shows what each commodity wants to do individually before**

> **the optimizer considers the needs of the network as a whole.**

30.2 Validate the Minimum-Latency Selection

Before using the minimum-latency routes as a baseline, the extraction is validated.

The following conditions should hold:

- every commodity has at least one minimum-latency path;
- every selected path belongs to the correct commodity;
- the selected latency equals the true minimum available to that commodity;
- every selected path remains a valid candidate path.

The current synthetic network is also checked to determine whether any commodities have tied minimum-latency alternatives.

In [193…

```
# ------------------------------------------------------------
# Check 1: Every commodity must have at least one
#          minimum-latency path
# ------------------------------------------------------------

minimum_path_counts = (
    minimum_latency_paths_df
    .groupby(
        "commodity_id"
    )
    .size()
)

assert (
    minimum_path_counts.index.nunique()
    ==
    demand_df["commodity_id"].nunique()
)


# ------------------------------------------------------------
# Check 2: Every selected path must have the true
#          minimum latency for its commodity
# ------------------------------------------------------------

for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):
```

```
    # Calculate the true minimum latency for this commodity.
    true_min_latency = (
        group[
            "path_latency_ms"
        ].min()
    )

    # Extract routes marked as minimum latency.
    selected_group = (
        minimum_latency_paths_df[
            minimum_latency_paths_df[
                "commodity_id"
            ]
            == commodity_id
        ]
    )

    # Confirm every selected route has the correct minimum value.
    assert (
        selected_group[
            "path_latency_ms"
        ]
        == true_min_latency
    ).all()


# ------------------------------------------------------------
# Check 3: Count commodities with tied minimum-latency paths
# ------------------------------------------------------------

commodities_with_ties = (
    minimum_path_counts[
        minimum_path_counts > 1
    ]
)


# ------------------------------------------------------------
# Print validation summary
# ------------------------------------------------------------

print(
    "Minimum-latency path validation passed."
)

print(
    "Commodities:",
    demand_df["commodity_id"].nunique()
)

print(
    "Minimum-latency path records:",
```

```
    len(minimum_latency_paths_df)
)

print(
    "Commodities with tied minimum-latency paths:",
    len(commodities_with_ties)
)
```

```
Minimum-latency path validation passed.
Commodities: 20
Minimum-latency path records: 20
Commodities with tied minimum-latency paths: 0
```

30.2.1 Interpretation of Validation Results

The validation confirms that all 20 commodities have exactly one minimum-latency route.

No commodity has a tie.

This means that the heterogeneous latency values successfully remove the route ambiguity seen in the equal-link scenario.

The naive fastest-route policy is therefore deterministic:

> If every commodity independently minimizes latency, there is exactly one route it will attempt to use.

This makes the later comparison with optimized routing particularly clear because differences cannot be attributed to arbitrary shortest-path tie-breaking.

30.2.2 Business and Operational Impact

From an operational perspective, the lack of ties means that an unmanaged latency-minimizing routing policy would consistently push each commodity toward the same preferred route.

There is no natural load balancing caused by equivalent alternatives.

As a result:

- congestion may develop predictably around attractive low-latency links;
- traffic engineering becomes more important;
- coordinated routing is needed to deliberately use slower alternatives;
- optimization can create value by making decisions that individual shortest-path rules would never choose.

This makes the synthetic dataset well suited for demonstrating the business value of centralized network optimization.

30.3 Minimum-Latency Route Summary by Source and Destination

Latency-sensitive and tolerant traffic from the same source to the same destination have access to the same physical routes.

Therefore, the 20 commodity-level minimum-latency records can be reduced to 10 unique source-destination route decisions.

This provides a cleaner topology-focused view of minimum-latency routing before traffic class is considered.

In [194…

```
# Keep one unique minimum-latency physical route
# for each source-destination pair.
#
# UK-L and UK-T use the same physical path alternatives to UK.
# US-L and US-T use the same physical path alternatives to US.

minimum_route_summary_df = (
    minimum_latency_paths_df[
        [
            "source",
            "destination",
            "path",
            "num_links",
            "path_capacity_mbps",
            "path_latency_ms"
        ]
    ]
    .drop_duplicates()
    .sort_values(
        [
            "source",
            "destination"
        ]
    )
    .reset_index(drop=True)
)

minimum_route_summary_df
```

Out [194...

	source	destination	path	num_links	path_capacity_mbps	path_latency_ms
0	B	UK	B -> C -> UK	2	18	17
1	B	US	B -> C -> US	2	18	19
2	C	UK	C -> UK	1	30	7
3	C	US	C -> US	1	25	9
4	S1	UK	S1 -> A -> C -> UK	3	25	27
5	S1	US	S1 -> A -> C -> US	3	25	29
6	S2	UK	S2 -> A -> C -> UK	3	25	29
7	S2	US	S2 -> A -> C -> US	3	25	31
8	S3	UK	S3 -> B -> C -> UK	3	18	27
9	S3	US	S3 -> B -> C -> US	3	18	29

30.3.1 Interpretation of the Compact Route Summary

Reducing the 20 commodity records to 10 physical source-destination decisions makes the structural pattern especially clear.

The preferred routes are:

- S1 → A → C → UK / US
- S2 → A → C → UK / US
- S3 → B → C → UK / US
- B → C → UK / US
- C → UK / US

Every preferred route uses C as the final internal routing node.

The D corridor is absent.

This demonstrates that the synthetic latency structure strongly favors one side of the network before capacity constraints are considered.

30.3.2 Business and Operational Impact

The compact route summary identifies where unmanaged traffic would naturally concentrate.

This is useful for:

- predicting congestion before optimization;
- identifying links that require close monitoring;
- understanding why low-latency corridors may need protection;
- explaining why additional network paths alone do not guarantee balanced utilization.

Operationally, the network has redundancy, but that redundancy is not automatically used.

A routing policy must actively choose the alternative D corridor if it wants to benefit from that capacity.

30.4 Directed Arc Exposure Under Minimum-Latency Routing

The next step is to count how frequently each directed arc appears in the 20 minimum-latency commodity routes.

This is a structural exposure measure.

A high count means that many commodities would prefer to use that arc under independent minimum-latency routing.

The count does **not** represent Mbps of traffic.

Actual traffic loading is calculated later using the source-stream demand values.

In [195…

```
from collections import Counter


def count_min_latency_path_usage(
    minimum_latency_paths_df
):
    """
    Count how many minimum-latency commodity routes
    contain each directed network arc.
    """

    # Create a Counter indexed by directed arc tuples.
    counter = Counter()


    # Review every minimum-latency route.
    for edges in (
        minimum_latency_paths_df[
            "path_edges"
        ]
    ):

        # Increment the count for every arc used by the route.
        counter.update(edges)


    return counter


# Calculate directed-arc route frequency.

min_latency_usage = (
    count_min_latency_path_usage(
        minimum_latency_paths_df
    )
)


# Display raw usage counts.

min_latency_usage
```

Out[195…
```
Counter({('C', 'UK'): 10,
         ('C', 'US'): 10,
         ('B', 'C'): 8,
         ('A', 'C'): 8,
         ('S1', 'A'): 4,
         ('S2', 'A'): 4,
         ('S3', 'B'): 4})
```

30.5 Convert Arc Exposure to a Table

In [197…
```python
# Convert the Counter into a sorted DataFrame
# so the structural exposure is easier to interpret.

min_latency_usage_df = pd.DataFrame([
    {
        "from_node": u,
        "to_node": v,
        "minimum_latency_route_count": count
    }
    for (u, v), count
    in min_latency_usage.items()
])


# Sort the most frequently used arcs first.

min_latency_usage_df = (
    min_latency_usage_df
    .sort_values(
        "minimum_latency_route_count",
        ascending=False
    )
    .reset_index(drop=True)
)

min_latency_usage_df
```

Out[197…

	from_node	to_node	minimum_latency_route_count
0	C	UK	10
1	C	US	10
2	B	C	8
3	A	C	8
4	S1	A	4
5	S2	A	4
6	S3	B	4

30.5.1 Interpretation of Arc Exposure

The route-frequency analysis reveals a strong concentration around the C corridor.

C → UK and C → US

Both appear in:

$$10$$

minimum-latency commodity routes.

This means every UK commodity prefers C → UK and every US commodity prefers C → US.

A → C

A → C appears in:

$$8$$

minimum-latency routes.

These correspond to the four S1 commodities and four S2 commodities.

B → C

B → C also appears in:

$$8$$

minimum-latency routes.

These correspond to the four B commodities and four S3 commodities.

Source-Entry Links

S1 → A, S2 → A, and S3 → B each appear in four minimum-latency commodity paths.

Structural Conclusion

Independent minimum-latency routing creates the following convergence pattern:

- S1 and S2 traffic converge on A → C;

- S3 and B traffic converge on B → C;
- all UK traffic converges on C → UK;
- all US traffic converges on C → US.

No minimum-latency commodity uses D.

Business and Operational Impact

This concentration has several operational implications.

First, the low-latency C corridor is likely to experience disproportionate demand.

Second, the network's alternate D corridor provides no value if routing decisions are based only on latency.

Third, congestion risk is concentrated on a small number of shared links.

For network operators, this means that monitoring only source links would miss the real problem.

The critical competition occurs downstream after traffic from several sources converges.

This also creates a capacity-planning implication:

> A heavily preferred link is not automatically the best place to invest additional bandwidth.

Later bottleneck analysis will determine whether additional capacity on those links actually increases end-to-end delivered throughput.

30.6 Naive Minimum-Latency Routing Baseline

Route frequency alone does not indicate actual congestion because commodities have different demand levels.

A stronger baseline therefore assigns each commodity's **entire offered demand** to its unique minimum-latency route.

This is not an optimization model.

It represents the policy:

> Every source-stream commodity independently chooses its fastest route.

For each directed arc e, the resulting traffic load is:

$$F_e^{\text{min-latency}} = \sum_{(k,s):e \in p_{k,s}^{\min}} D_{k,s}$$

The resulting load is then compared with available directional capacity.

In [198…

```
# Create a lookup from commodity ID to offered demand.

commodity_demand_lookup = (
    demand_df
    .set_index(
        "commodity_id"
    )[
        "demand_mbps"
    ]
    .to_dict()
)


# Initialize every directed arc with zero attempted traffic.

naive_min_latency_load = {
    (
        row["from_node"],
        row["to_node"]
    ): 0.0

    for _, row
    in arcs_df.iterrows()
}


# Route every commodity's full demand along
# its unique minimum-latency path.

for _, row in (
    minimum_latency_paths_df.iterrows()
):

    commodity_id = (
        row["commodity_id"]
    )

    demand = (
        commodity_demand_lookup[
            commodity_id
        ]
    )
```

```
    # Add the commodity's full demand to every
    # directed arc in its chosen path.

    for edge in row["path_edges"]:

        naive_min_latency_load[
            edge
        ] += demand


# Build a link-level baseline results table.

naive_min_latency_link_results = []

for _, arc in arcs_df.iterrows():

    edge = (
        arc["from_node"],
        arc["to_node"]
    )

    capacity = (
        arc["capacity_mbps"]
    )

    load = (
        naive_min_latency_load[
            edge
        ]
    )

    # Calculate overload beyond available capacity.

    overload = max(
        0,
        load - capacity
    )


    # Calculate attempted utilization.
    #
    # Values over 100% indicate infeasible loading.

    utilization = (
        100 * load / capacity
        if capacity > 0
        else np.nan
    )

    naive_min_latency_link_results.append({
        "from_node": arc["from_node"],
        "to_node": arc["to_node"],
```

```
        "capacity_mbps": capacity,
        "naive_load_mbps": load,
        "utilization_percentage": utilization,
        "overload_mbps": overload,
        "capacity_violated": overload > 1e-8
    })


naive_min_latency_link_df = pd.DataFrame(
    naive_min_latency_link_results
)


# Sort capacity violations first, followed by highest utilization.

naive_min_latency_link_df = (
    naive_min_latency_link_df
    .sort_values(
        [
            "capacity_violated",
            "utilization_percentage"
        ],
        ascending=[
            False,
            False
        ]
    )
    .reset_index(drop=True)
)

naive_min_latency_link_df
```

Out [198...

	from_node	to_node	capacity_mbps	naive_load_mbps	utilization_percenta
0	B	C	18	61.0	338.8888
1	C	US	25	61.0	244.0000
2	C	UK	30	65.0	216.6666
3	A	C	25	45.0	180.0000
4	S3	B	30	24.0	80.0000
5	S1	A	40	24.0	60.0000
6	S2	A	35	21.0	60.0000
7	A	S1	40	0.0	0.0000
8	A	S2	35	0.0	0.0000
9	B	S3	30	0.0	0.0000
10	A	B	20	0.0	0.0000
11	B	A	20	0.0	0.0000
12	C	A	25	0.0	0.0000
13	C	B	18	0.0	0.0000
14	B	D	28	0.0	0.0000
15	D	B	28	0.0	0.0000
16	UK	C	30	0.0	0.0000
17	US	C	25	0.0	0.0000
18	D	UK	22	0.0	0.0000
19	UK	D	22	0.0	0.0000
20	D	US	30	0.0	0.0000
21	US	D	30	0.0	0.0000

30.7 Interpretation of Naive Minimum-Latency Loading

The naive minimum-latency policy is infeasible.

Several preferred links receive substantially more attempted traffic than their available capacity.

A → C

Attempted load:

$$45 \text{ Mbps}$$

Available capacity:

$$25 \text{ Mbps}$$

Overload:

$$45 - 25 = 20 \text{ Mbps}$$

Attempted utilization:

$$\frac{45}{25} \times 100 = 180\%$$

B → C

Attempted load:

$$61 \text{ Mbps}$$

Available capacity:

$$18 \text{ Mbps}$$

Overload:

$$61 - 18 = 43 \text{ Mbps}$$

Attempted utilization:

$$\frac{61}{18} \times 100 \approx 338.9\%$$

This is the most severe internal overload.

C → UK

All:

$$65 \text{ Mbps}$$

of UK-bound demand attempts to use C → UK.

Available capacity is only:

$$30 \text{ Mbps}$$

creating:

$$35 \text{ Mbps}$$

of overload.

C → US

All:

$$61 \text{ Mbps}$$

of US-bound demand attempts to use C → US.

Available capacity is:

$$25 \text{ Mbps}$$

creating:

$$36 \text{ Mbps}$$

of overload.

Critical Finding

Each commodity is making an individually reasonable decision.

However, the combined routing policy is globally infeasible.

This demonstrates that the problem cannot be solved by independently assigning each commodity to its fastest route.

30.7.1 Business and Operational Impact

The naive baseline exposes the operational risk of decentralized shortest-route behavior.

If every traffic stream independently chooses the fastest path:

- key low-latency links become severely overloaded;
- some traffic must ultimately be dropped or rerouted;
- network capacity elsewhere remains unused;
- sensitive and tolerant traffic compete indiscriminately for the same scarce corridors.

This can translate into:

- reduced delivered throughput;
- degraded customer experience;
- unstable congestion patterns;
- inefficient use of network infrastructure.

The result demonstrates the business value of coordinated traffic engineering.

Rather than simply adding bandwidth everywhere, the optimizer can determine how existing capacity should be used first and then identify where incremental capacity would actually produce additional end-to-end service.

30.8 Show Only Capacity -- Violating Links

In [199…

```
# Filter the naive baseline to only those directed arcs
# whose attempted load exceeds available capacity.

naive_overloaded_links_df = (
    naive_min_latency_link_df[
        naive_min_latency_link_df[
            "capacity_violated"
        ]
    ]
    .reset_index(drop=True)
)

naive_overloaded_links_df
```

Out[199…

	from_node	to_node	capacity_mbps	naive_load_mbps	utilization_percentag
0	B	C	18	61.0	338.88888
1	C	US	25	61.0	244.00000
2	C	UK	30	65.0	216.66666
3	A	C	25	45.0	180.00000

30.8.1 Interpretation of Capacity-Violating Links

Filtering to only overloaded arcs isolates the links where naive minimum-latency routing fails.

The most important violations occur on:

- A → C;
- B → C;
- C → UK;
- C → US.

This shows that congestion is concentrated in a small set of converging internal and exit corridors rather than across the entire network.

That distinction matters because it suggests that the network has usable alternative capacity; the routing policy is simply not taking advantage of it.

30.8.2 Business and Operational Impact

The overloaded-link table provides a more actionable operational view than the full network table.

Network teams can immediately see:

- where congestion first appears;
- how much offered traffic exceeds available capacity;
- which links require rerouting pressure;
- which links warrant later marginal-capacity analysis.

However, this table should not yet be used to justify capacity investment.

An overloaded link may cease to be limiting once traffic is rerouted.

The optimized bottleneck analysis later in the notebook will determine whether adding capacity actually increases total delivered traffic.

30.9 Naive Minimum-Latency Load vs. Capacity

The following chart compares:

- available directed-link capacity;
- attempted traffic load under naive minimum-latency routing.

If attempted load exceeds capacity, the routing policy is infeasible on that link.

In [200…

```
# Keep only links carrying positive traffic under the naive policy.

naive_active_links_df = (
    naive_min_latency_link_df[
        naive_min_latency_link_df[
            "naive_load_mbps"
```

```
        ] > 0
    ]
    .copy()
)


# Build readable directed-link labels.

naive_active_links_df[
    "link"
] = (
    naive_active_links_df[
        "from_node"
    ]
    + " → "
    + naive_active_links_df[
        "to_node"
    ]
)


# Prepare capacity and attempted-load columns for plotting.

plot_df = (
    naive_active_links_df
    .set_index(
        "link"
    )[
        [
            "capacity_mbps",
            "naive_load_mbps"
        ]
    ]
)


# Create the grouped bar chart.

plot_df.plot(
    kind="bar",
    figsize=(12, 6)
)

plt.ylabel(
    "Traffic / Capacity (Mbps)"
)

plt.xlabel(
    "Directed Link"
)

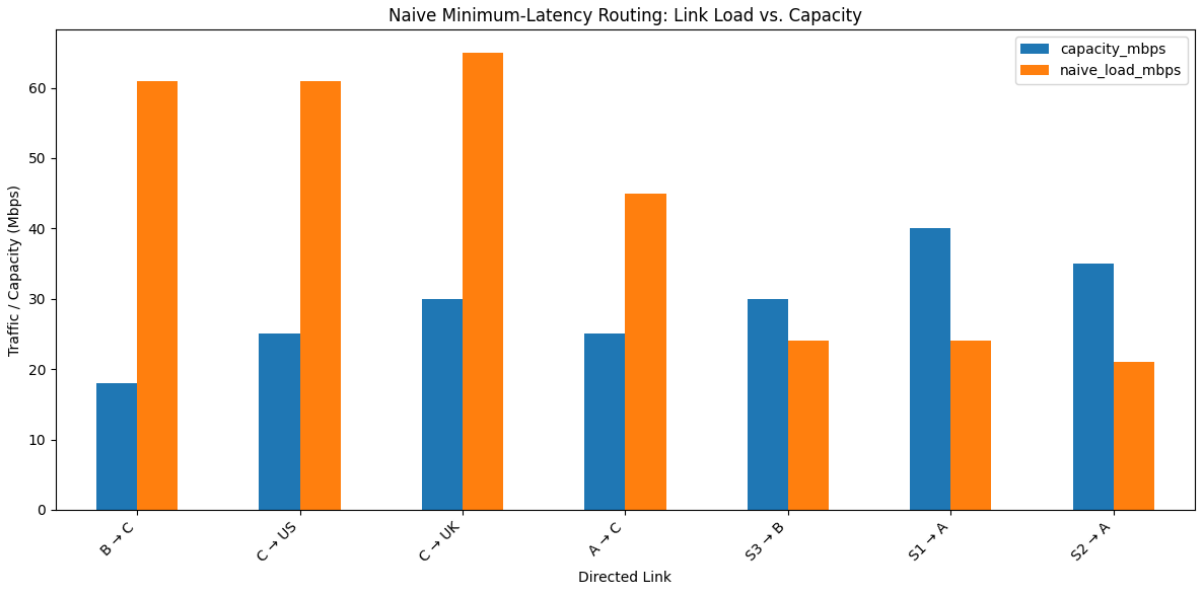
plt.title(
```

```
    "Naive Minimum-Latency Routing: "
    "Link Load vs. Capacity"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()

plt.show()
```



30.9.1 Interpretation of Load-vs-Capacity Chart

The chart makes the difference between source-entry capacity and downstream congestion visually clear.

The source-entry links remain within capacity:

- S1 → A carries 24 Mbps against 40 Mbps;
- S2 → A carries 21 Mbps against 35 Mbps;
- S3 → B carries 24 Mbps against 30 Mbps.

The primary congestion appears only after traffic converges deeper in the network.

The largest violations occur on:

- A → C;
- B → C;
- C → UK;

- C → US.

This confirms that the network's problem is not insufficient ingress capacity.

Instead, the problem is concentration of traffic on shared downstream low-latency corridors.

3.9.2 Business and Operational Impact

This chart provides a useful operational diagnostic.

It shows that simply increasing source-entry bandwidth would not solve the network problem.

The congestion occurs deeper in the network where multiple sources share common routes.

For capacity planning, this means investment decisions should focus on:

- end-to-end traffic behavior;
- convergence points;
- marginal throughput improvement;

rather than simply expanding the highest-volume source connections.

This prevents spending capacity budget in locations that do not improve customer-facing throughput.

30.10 Naive Minimum-Latency Network Flow

The following network diagram shows the spatial routing pattern produced when every commodity sends its entire offered demand along its fastest route.

Link labels show:

$$\text{Attempted Flow}/\text{Capacity}$$

and link thickness increases with attempted traffic volume.

This visualization helps identify where independently selected paths converge.

In [201…
```
# Create a plotting table using the same conceptual structure
# that will later be used for optimized network-flow diagrams.
```

```
naive_flow_plot_df = (
    naive_min_latency_link_df[
        [
            "from_node",
            "to_node",
            "capacity_mbps",
            "naive_load_mbps",
            "utilization_percentage"
        ]
    ]
    .rename(
        columns={
            "naive_load_mbps":
                "flow_mbps"
        }
    )
)


# Build a lookup dictionary for directional flow values.

naive_flow_lookup = {
    (
        row["from_node"],
        row["to_node"]
    ): {
        "flow": row["flow_mbps"],
        "capacity": row["capacity_mbps"],
        "utilization":
            row["utilization_percentage"]
    }

    for _, row
    in naive_flow_plot_df.iterrows()
}


# Calculate the largest directional flow on each physical connection.
#
# This value controls the visual thickness of the physical edge.

physical_naive_flow = {}

for u, v in G_synthetic.edges():

    uv_flow = (
        naive_flow_lookup.get(
            (u, v),
            {"flow": 0}
        )["flow"]
    )
```

```
    vu_flow = (
        naive_flow_lookup.get(
            (v, u),
            {"flow": 0}
        )["flow"]
    )

    physical_naive_flow[
        (u, v)
    ] = max(
        uv_flow,
        vu_flow
    )


# Determine the scale used for edge width.

max_naive_flow = max(
    physical_naive_flow.values(),
    default=1
)

if max_naive_flow <= 0:
    max_naive_flow = 1


# Scale line width according to attempted traffic flow.

edge_widths = [
    1
    + 6
    * physical_naive_flow[
        (u, v)
    ]
    / max_naive_flow

    for u, v
    in G_synthetic.edges()
]


# Draw the physical network.

plt.figure(
    figsize=(14, 8)
)

nx.draw_networkx_nodes(
    G_synthetic,
    pos=pos,
    node_size=2200
)
```

```
nx.draw_networkx_labels(
    G_synthetic,
    pos=pos,
    font_size=11
)

nx.draw_networkx_edges(
    G_synthetic,
    pos=pos,
    width=edge_widths
)


# Build directional flow/capacity labels.

edge_labels = {}

for u, v in G_synthetic.edges():

    label_parts = []

    uv = naive_flow_lookup.get(
        (u, v)
    )

    vu = naive_flow_lookup.get(
        (v, u)
    )

    if (
        uv is not None
        and uv["flow"] > 0
    ):

        label_parts.append(
            f"{u}->{v}: "
            f"{uv['flow']:.0f}/"
            f"{uv['capacity']:.0f} Mbps"
        )

    if (
        vu is not None
        and vu["flow"] > 0
    ):

        label_parts.append(
            f"{v}->{u}: "
            f"{vu['flow']:.0f}/"
            f"{vu['capacity']:.0f} Mbps"
        )
```

```
    if label_parts:

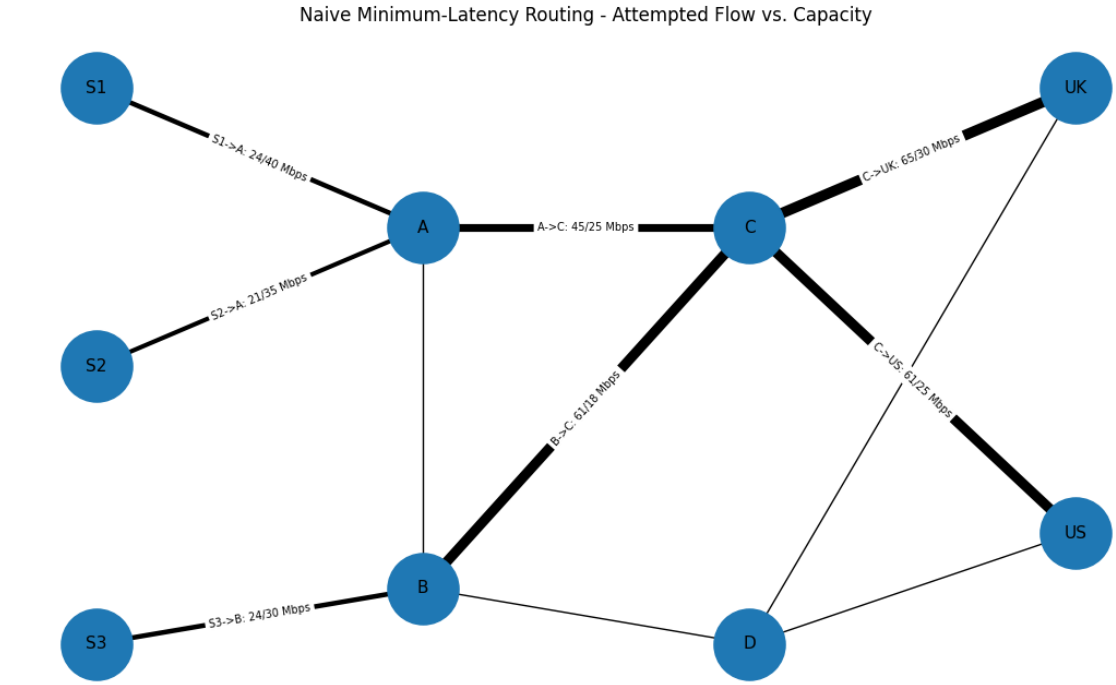
        edge_labels[
            (u, v)
        ] = "\n".join(
            label_parts
        )


nx.draw_networkx_edge_labels(
    G_synthetic,
    pos=pos,
    edge_labels=edge_labels,
    font_size=7
)

plt.title(
    "Naive Minimum-Latency Routing "
    "- Attempted Flow vs. Capacity"
)

plt.axis("off")

plt.show()
```



30.10.1 Interpretation of the Naive Network Diagram

The network diagram shows the spatial concentration created by independent minimum-latency routing.

Traffic from S1 and S2 converges on:

$$A \to C.$$

Traffic from S3 and B converges on:

$$B \to C.$$

All UK-bound traffic then attempts to leave through:

$$C \to UK,$$

while all US-bound traffic attempts to leave through:

$$C \to US.$$

The D corridor receives no traffic.

This visually confirms that the network contains unused alternative routes even while the preferred C corridor is severely overloaded.

The key issue is therefore not that every part of the network lacks capacity.

Instead, independently latency-minimizing traffic concentrates on the same attractive routes while capacity available on slower alternative corridors remains unused.

This creates a form of **stranded alternate capacity under naive routing**.

The network has capacity available, but the routing policy does not make use of it because each commodity independently prefers the lower-latency C corridor.

This demonstrates the broader principle:

$$\boxed{\text{Local Route Optimality} \neq \text{Global Network Feasibility}}$$

Each commodity makes an individually reasonable routing decision, but those decisions collectively create an infeasible network state.

The purpose of the optimization models that follow is therefore not simply to identify shorter paths.

It is to coordinate competing traffic so that available end-to-end network capacity is used more effectively.

30.10.2 Business and Operational Impact

The network diagram translates the numerical overload results into an operational picture.

It shows that the network is not uniformly capacity constrained.

Instead, traffic engineering decisions determine whether capacity is used effectively.

This has several implications:

- alternative infrastructure can provide little value if routing policies do not use it;
- low-latency links require coordinated allocation;
- congestion management should occur before links become fully overloaded;
- network control should optimize across commodities rather than independently.

From a business perspective, coordinated routing may improve service without immediately requiring new infrastructure.

That is important because better utilization of existing capacity can be less costly than adding bandwidth.

30.11 Hypotheses for the Optimization Models

The heterogeneous path analysis produces several testable hypotheses.

Hypothesis 1 — Full Demand Cannot Follow Minimum-Latency Routes

The naive minimum-latency solution violates several directed capacity constraints.

Therefore, at least one of the following must occur:

- traffic is rerouted;
- demand is partially delivered;
- demand is lost.

Hypothesis 2 — The D Corridor Should Become Operationally Important

Although D is not selected by any minimum-latency route, it contains meaningful alternative capacity.

A throughput-maximizing solution is therefore expected to use:

$$B \to D$$

and the D exit links to relieve congestion around C.

Hypothesis 3 — Low-Latency C Capacity Is Scarce

The C corridor is attractive to nearly every commodity.

Stage 2 should therefore have an opportunity to preserve or reassign C-side capacity toward UK-L and US-L after Stage 1 maximum throughput has been determined.

Hypothesis 4 — Optimized Routing Should Differ Materially from Naive Routing

If the optimization is functioning correctly, the final path allocation should use a broader set of routes than the independent minimum-latency baseline.

30.11.1 Business Significance of the Optimization Hypotheses

These hypotheses translate directly into operational questions.

The optimization will test whether the network can:

- deliver more traffic by using alternative paths;
- preserve low-latency capacity for traffic that benefits most from it;
- reduce congestion without increasing physical capacity;
- identify when additional bandwidth is actually required.

The results will therefore help distinguish between two fundamentally different interventions:

1. **routing improvement**, where existing capacity is used more efficiently;
2. **capacity expansion**, where additional bandwidth is genuinely needed.

That distinction is important for both network performance and infrastructure cost.

30.12 Section Summary

Section 30 establishes the naive minimum-latency benchmark for the heterogeneous synthetic network.

Each of the 20 source-stream commodities has a unique minimum-latency route.

Those preferred routes strongly concentrate traffic around node C:

- S1 and S2 traffic favor A → C;
- S3 and B traffic favor B → C;
- all UK-bound traffic favors C → UK;
- all US-bound traffic favors C → US.

When all offered demand is assigned independently to those fastest routes, several directed links become infeasible.

The naive policy attempts to place approximately:

- 45 Mbps on A → C, compared with 25 Mbps capacity;
- 61 Mbps on B → C, compared with 18 Mbps capacity;
- 65 Mbps on C → UK, compared with 30 Mbps capacity;
- 61 Mbps on C → US, compared with 25 Mbps capacity.

The alternate D corridor remains unused.

Technical Conclusion

The problem is not a conventional shortest-path problem.

It is a coordinated multi-commodity routing problem in which locally optimal decisions conflict because commodities share finite directional capacity.

Business Conclusion

The network contains usable capacity that shortest-path routing fails to exploit.

This means there is potential operational value in centralized optimization before additional infrastructure investment is considered.

A successful routing model should:

- increase utilization of alternative corridors;
- reduce overload on the C corridor;
- maximize total delivered demand;
- preserve low latency for UK-L and US-L when possible.

The optimized models in the following sections will be evaluated against this naive baseline.

This distinction is important because it separates two fundamentally different network problems:

1. **routing inefficiency**, where existing capacity is available but poorly utilized;
2. **physical capacity scarcity**, where no feasible routing pattern can satisfy additional demand.

The naive baseline demonstrates the first problem.

The optimized models in the following sections determine how much of that inefficiency can be removed before additional infrastructure is required.

The next section introduces the solver environment used to implement the splittable-flow linear program and single-path mixed-integer linear program.

31. Optimization Solver Setup

Section 30 demonstrated that independently routing every commodity along its minimum-latency path creates severe capacity violations.

In particular, several preferred corridors become overloaded even though alternative network capacity remains available elsewhere.

Therefore, a network-wide optimization method is required.

The optimization models in the following sections are implemented using **PuLP**, a Python mathematical-programming modeling library.

Two different mathematical formulations are required.

Splittable-Flow Formulation

The splittable-flow problem uses continuous path-flow variables:

$$f_{k,s,p} \geq 0$$

Traffic from one source-stream commodity may therefore be divided among several feasible routes.

Because the variables and constraints are linear and continuous, this formulation is a **Linear Program (LP)**.

Single-Path Formulation

The single-path problem includes both:

- continuous path-flow variables;
- binary path-selection variables.

The binary variable:

$$y_{k,s,p} \in \{0, 1\}$$

determines whether a particular path is selected for a source-stream commodity.

Because the formulation combines continuous and integer decision variables, it is a **Mixed-Integer Linear Program (MILP)**.

Both formulations are solved using the CBC optimization solver through PuLP.

The purpose of this section is to:

1. verify that PuLP is available;
2. verify that the CBC solver can execute successfully;
3. document the expected decision-variable and constraint structure;
4. establish a consistent solver configuration before the optimization models are constructed.

31.1 Import PuLP and Verify the Modeling Environment

The first step is to load PuLP and confirm the installed version.

Recording the optimization-library version improves reproducibility because solver interfaces and behavior may change across software versions.

In [67]:
```
%pip install pulp -q
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [203…
```
# Import the PuLP mathematical-programming library.
#
# PuLP provides the modeling interface used to define:
#   - decision variables,
#   - objective functions,
#   - constraints,
#   - LP and MILP optimization problems.

import pulp as pl
```

```
# Display the installed PuLP version.
#
# Recording the version helps make the notebook more reproducible.

print(
    "PuLP version:",
    pl.__version__
)
```

```
PuLP version: 3.3.2
```

31.1.1 Interpretation of Environment Check

Successfully importing PuLP confirms that the Python environment contains the optimization modeling library required for the remaining analysis.

The version output records the exact modeling environment used to generate the results.

This does not yet prove that an optimization solver is available.

PuLP defines the mathematical problem, but a numerical solver such as CBC is still required to calculate the optimal solution.

31.1.2 Business and Operational Impact

Reproducibility is important in an operational optimization system.

A routing recommendation should not depend on undocumented software assumptions.

Recording the solver environment supports:

- repeatable analysis;
- easier debugging;
- model governance;
- controlled deployment;
- comparison of results across future software versions.

In a production setting, the same optimization formulation could later be connected to a different commercial or enterprise solver without changing the fundamental network model.

31.2 Configure the CBC Solver

PuLP is a modeling interface rather than the optimization engine itself.

The models will be solved using the **CBC solver**.

CBC supports both:

- linear programming;
- mixed-integer linear programming.

Using the same solver family for both formulations helps keep the comparison between the splittable and single-path models consistent.

In [204…
```
# Create the CBC solver configuration that will be reused
# throughout the notebook.
#
# msg=False suppresses detailed solver-console output so that
# notebook results remain easier to read.
#
# For troubleshooting, msg can temporarily be changed to True.

cbc_solver = pl.PULP_CBC_CMD(
    msg=False
)


# Ask PuLP whether the CBC executable is available
# in the current environment.

cbc_available = (
    cbc_solver.available()
)


print(
    "CBC solver available:",
    bool(cbc_available)
)
```

```
CBC solver available: True
```

31.2.1 Validate CBC Availability

In [205…
```
# Stop the notebook immediately if CBC is unavailable.
#
# This prevents later optimization sections from producing
# confusing failures caused by a missing solver rather than
# an error in the network model.

assert cbc_available, (
    "CBC solver is not available in the current environment."
```

```
)

print(
    "CBC availability check passed."
)
```

```
CBC availability check passed.
```

31.2.1 Interpretation of Solver Availability

The CBC availability check confirms that the notebook has access to an optimization engine capable of solving both required formulations.

This distinguishes two different components of the workflow:

- **PuLP** describes the optimization problem;
- **CBC** solves the mathematical problem.

With both components available, later solver failures are more likely to reflect model construction, infeasibility, or numerical issues rather than a missing software dependency.

31.2.2 Business and Operational Impact

Solver availability is an implementation requirement rather than merely a coding detail.

An optimization framework must reliably transform network inputs into routing decisions.

In production, solver-health checks could be incorporated into the routing pipeline so that a failed optimization engine does not silently produce incomplete or stale network decisions.

This is particularly important when routing recommendations influence real-time capacity allocation or customer service.

31.3 Solver Smoke Test

Before solving the network-routing problem, a small deterministic optimization problem is used to verify that CBC can successfully solve a linear program.

The test problem is:

$$\max 3x + 2y$$

subject to:

$$x + y \leq 5$$

$$x \leq 3$$

$$y \leq 4$$

and:

$$x, y \geq 0$$

The expected optimal solution is:

$$x = 3$$

$$y = 2$$

with objective value:

$$3(3) + 2(2) = 13$$

This test is independent of the network-routing model.

Its purpose is only to verify that the modeling interface and numerical solver communicate correctly.

In [206…

```
# ------------------------------------------------------------
# Create a small LP used only to test the solver environment
# ------------------------------------------------------------

solver_test = pl.LpProblem(
    "Solver_Smoke_Test",
    pl.LpMaximize
)


# Create two non-negative continuous variables.

test_x = pl.LpVariable(
    "test_x",
    lowBound=0,
    cat="Continuous"
)

test_y = pl.LpVariable(
    "test_y",
    lowBound=0,
    cat="Continuous"
)
```

```
# ------------------------------------------------------------
# Define the test objective
# ------------------------------------------------------------

solver_test += (
    3 * test_x
    + 2 * test_y
)


# ------------------------------------------------------------
# Add test constraints
# ------------------------------------------------------------

solver_test += (
    test_x + test_y <= 5,
    "test_total_constraint"
)

solver_test += (
    test_x <= 3,
    "test_x_limit"
)

solver_test += (
    test_y <= 4,
    "test_y_limit"
)


# ------------------------------------------------------------
# Solve the test problem using the configured CBC solver
# ------------------------------------------------------------

solver_test.solve(
    cbc_solver
)


# Translate the solver status code into readable text.

solver_test_status = (
    pl.LpStatus[
        solver_test.status
    ]
)


# Extract the optimal objective value.

solver_test_objective = (
    pl.value(
```

```
        solver_test.objective
    )
)


# Display results.

print(
    "Solver test status:",
    solver_test_status
)

print(
    "x:",
    test_x.value()
)

print(
    "y:",
    test_y.value()
)

print(
    "Objective value:",
    solver_test_objective
)
```

```
Solver test status: Optimal
x: 3.0
y: 2.0
Objective value: 13.0
```

31.3.1 Validate the Smoke Test

In [207…

```
# ------------------------------------------------------------
# Validate the solver test numerically
# ------------------------------------------------------------

# Confirm that CBC reported an optimal solution.

assert (
    solver_test_status
    == "Optimal"
)


# Confirm the expected objective value.

assert abs(
    solver_test_objective
    - 13
```

```
) < 1e-6


# Confirm the expected variable values.

assert abs(
    test_x.value()
    - 3
) < 1e-6

assert abs(
    test_y.value()
    - 2
) < 1e-6


print(
    "Solver smoke test passed."
)
```

```
Solver smoke test passed.
```

31.3.1Interpretation of Solver Smoke Test

The smoke test returns the expected optimal solution:

$$x = 3$$

$$y = 2$$

with objective:

$$13$$

This confirms that:

- PuLP can construct an optimization model;
- CBC can solve the model;
- continuous decision variables are handled correctly;
- objective values can be retrieved successfully;
- solver status information is available to the notebook.

The optimization environment is therefore ready for the network-routing formulations.

31.3.3 Business and Operational Impact

This small test provides a basic model-health check before the significantly more complex routing problem is solved.

In an operational system, similar health checks can reduce the risk of confusing:

- solver failures;
- modeling errors;
- input-data problems.

Separating infrastructure validation from network optimization improves reliability and makes troubleshooting faster.

31.4 Compare the Two Optimization Formulations

The assignment requires two routing formulations.

Although both use the same:

- network topology;
- capacities;
- latencies;
- source-stream demand;
- candidate paths;

their decision spaces are different.

Splittable Flow

The splittable formulation uses only continuous flow variables.

A commodity may distribute its delivered traffic across several paths.

Single Path

The single-path formulation adds a binary path-selection variable for every candidate route.

Each source-stream commodity may select at most one path, although partial delivery along that selected path remains allowed.

The single-path formulation is therefore more restrictive and computationally more complex.

In [208…

```
# Create a compact comparison of the two mathematical formulations.

formulation_comparison_df = pd.DataFrame([
    {
        "formulation": "Splittable Flow",
```

```
        "problem_type": "Linear Program (LP)",
        "continuous_flow_variables": "Yes",
        "binary_path_variables": "No",
        "multiple_paths_per_commodity": "Yes",
        "partial_delivery_allowed": "Yes"
    },
    {
        "formulation": "Single Path",
        "problem_type": "Mixed-Integer Linear Program (MILP)",
        "continuous_flow_variables": "Yes",
        "binary_path_variables": "Yes",
        "multiple_paths_per_commodity": "No",
        "partial_delivery_allowed": "Yes"
    }
])


# Display the formulation comparison.

formulation_comparison_df
```

Out[208…

	formulation	problem_type	continuous_flow_variables	binary_path_variables
0	Splittable Flow	Linear Program (LP)	Yes	No
1	Single Path	Mixed-Integer Linear Program (MILP)	Yes	Yes

31.4.1 Interpretation of Formulation Comparison

The two models answer related but operationally different routing questions.

The splittable model asks:

> How much traffic can be delivered if each source-stream commodity may use all available path capacity?

The single-path model asks:

> How much traffic can be delivered if each source-stream commodity must remain on one selected route?

Because every single-path routing solution is also feasible in the more flexible splittable model:

$$T^*_{\text{single}} \leq T^*_{\text{split}}$$

under otherwise identical inputs.

The two values may be equal.

However, the single-path model can never outperform the splittable model in maximum throughput when both are formulated consistently.

31.4.2 Business and Operational Impact

The distinction between these formulations represents a real implementation tradeoff.

Multipath routing may:

- use fragmented residual capacity more efficiently;
- increase delivered throughput;
- reduce congestion;
- improve routing flexibility.

However, it may also introduce greater operational complexity.

Single-path routing may be easier to:

- implement;
- troubleshoot;
- monitor;
- explain operationally.

Therefore, the important business question is not simply whether multipath routing is mathematically more flexible.

The later experiments will determine:

> **How much measurable performance benefit does that extra routing flexibility actually provide?**

If the benefit is negligible, the simpler single-path policy may be operationally preferable.

If the benefit is substantial, the added implementation complexity may be justified.

31.5 Expected Optimization Model Size

The candidate-path analysis produced:

$$68$$

source-stream path alternatives.

The demand model contains:

$$20$$

source-stream commodities.

The directed network contains:

$$22$$

directed arcs.

These values determine the approximate size of the optimization formulations.

Calculating the expected number of variables and constraints before model construction provides a useful structural check.

In [211…

```
# ------------------------------------------------------------
# Calculate the dimensions of the routing problem
# ------------------------------------------------------------

# Number of candidate source-stream paths.

n_candidate_paths = (
    candidate_paths_df[
        "path_id"
    ].nunique()
)


# Number of source-stream commodities.

n_commodities = (
    demand_df[
        "commodity_id"
    ].nunique()
)


# Number of directed network arcs.

n_directed_arcs = (
    len(arcs_df)
)
```

```
# ------------------------------------------------------------
# Expected variable counts
# ------------------------------------------------------------

# Splittable formulations use one continuous flow variable
# per candidate path.

split_continuous_vars = (
    n_candidate_paths
)

split_binary_vars = 0


# Single-path formulations use:
#   one continuous flow variable per candidate path,
#   one binary selection variable per candidate path.

single_continuous_vars = (
    n_candidate_paths
)

single_binary_vars = (
    n_candidate_paths
)


# ------------------------------------------------------------
# Expected constraint counts
# ------------------------------------------------------------

# Split Stage 1:
#   one demand constraint per commodity
#   one capacity constraint per directed arc

split_stage1_constraints_expected = (
    n_commodities
    + n_directed_arcs
)


# Split Stage 2:
#   one delivery-preservation constraint per commodity
#   one capacity constraint per directed arc

split_stage2_constraints_expected = (
    n_commodities
    + n_directed_arcs
)


# Single-Path Stage 1:
```

```
#   one path-selection constraint per commodity
#   one flow-selection linking constraint per candidate path
#   one capacity constraint per directed arc

single_stage1_constraints_expected = (
    n_commodities
    + n_candidate_paths
    + n_directed_arcs
)


# Single-Path Stage 2:
#   one delivery-preservation constraint per commodity
#   one path-selection constraint per commodity
#   one flow-selection linking constraint per candidate path
#   one capacity constraint per directed arc

single_stage2_constraints_expected = (
    n_commodities
    + n_commodities
    + n_candidate_paths
    + n_directed_arcs
)


# ------------------------------------------------------------
# Build the model-size summary table
# ------------------------------------------------------------

model_size_df = pd.DataFrame([
    {
        "model": "Split Stage 1",
        "continuous_variables": split_continuous_vars,
        "binary_variables": split_binary_vars,
        "total_variables":
            split_continuous_vars
            + split_binary_vars,
        "expected_constraints":
            split_stage1_constraints_expected
    },
    {
        "model": "Split Stage 2",
        "continuous_variables": split_continuous_vars,
        "binary_variables": split_binary_vars,
        "total_variables":
            split_continuous_vars
            + split_binary_vars,
        "expected_constraints":
            split_stage2_constraints_expected
    },
    {
        "model": "Single Stage 1",
```

```
        "continuous_variables": single_continuous_vars,
        "binary_variables": single_binary_vars,
        "total_variables":
            single_continuous_vars
            + single_binary_vars,
        "expected_constraints":
            single_stage1_constraints_expected
    },
    {
        "model": "Single Stage 2",
        "continuous_variables": single_continuous_vars,
        "binary_variables": single_binary_vars,
        "total_variables":
            single_continuous_vars
            + single_binary_vars,
        "expected_constraints":
            single_stage2_constraints_expected
    }
])


# Display the expected model dimensions.

model_size_df
```

Out [211…

	model	continuous_variables	binary_variables	total_variables	expected_con
0	Split Stage 1	68	0	68	
1	Split Stage 2	68	0	68	
2	Single Stage 1	68	68	136	
3	Single Stage 2	68	68	136	

31.5.1 Interpretation of Expected Model Size

The model-size table quantifies the additional complexity introduced by the single-path restriction.

Splittable Model

The splittable formulation requires:

$$68$$

continuous path-flow variables.

Stage 1 contains approximately:

$$20 + 22 = 42$$

constraints:

- 20 commodity-demand limits;
- 22 directional link-capacity limits.

Single-Path Model

The single-path formulation requires:

$$68$$

continuous flow variables plus:

$$68$$

binary path-selection variables.

Therefore, it contains:

$$136$$

decision variables.

Stage 1 also requires approximately:

$$110$$

constraints because each candidate path requires a flow-to-selection linking constraint.

Stage 2 contains additional delivery-preservation constraints, increasing the expected total to approximately:

$$130$$

constraints.

Key Technical Finding

The binary path-selection variables substantially increase model complexity even

though the physical network and demand inputs are identical.

This is one reason MILP formulations generally become more computationally challenging as network size grows.

31.5.2 Business and Operational Impact

Model complexity affects more than computation time.

In a production routing system, greater optimization complexity can influence:

- decision latency;
- infrastructure requirements;
- scalability;
- frequency of re-optimization;
- operational responsiveness.

For the current nine-node network, both formulations are small and computationally manageable.

However, if the same approach were applied to a much larger network containing thousands of candidate paths, the binary-variable growth in the single-path model could become operationally significant.

This reinforces the need to evaluate performance benefit against computational and implementation complexity.

31.6 Expected Decision-Variable Complexity

The following chart compares the expected number of decision variables in the four optimization stages.

The comparison highlights the additional model size introduced by binary route-selection decisions.

In [213…

```
# Create a bar chart showing the total number of
# expected decision variables in each optimization model.

plt.figure(
    figsize=(10, 5)
)

plt.bar(
    model_size_df[
        "model"
```

```
    ],
    model_size_df[
        "total_variables"
    ]
)

plt.xlabel(
    "Optimization Model"
)

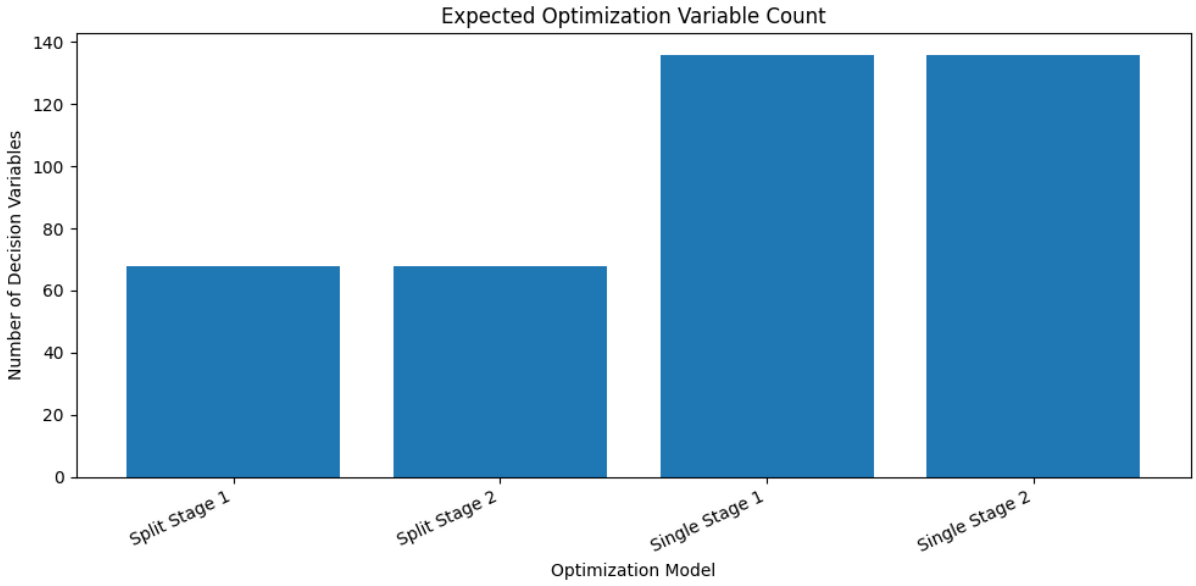
plt.ylabel(
    "Number of Decision Variables"
)

plt.title(
    "Expected Optimization Variable Count"
)

plt.xticks(
    rotation=25,
    ha="right"
)

plt.tight_layout()

plt.show()
```



31.6.1 Interpretation of Variable-Count Visualization

The chart shows that the single-path formulation requires approximately twice as many decision variables as the splittable formulation.

The difference arises entirely from the binary path-selection variables.

The network itself has not changed.

The additional mathematical complexity is created by the operational rule:

> Each source-stream commodity may use only one path.

This demonstrates how a seemingly simple routing policy can materially change the optimization problem.

31.6.2 Business and Operational Impact

The visualization highlights an important architecture decision.

A stricter routing policy may simplify network behavior while simultaneously making the optimization problem harder to solve.

This creates a tradeoff between:

- operational simplicity in the network;
- computational complexity in the optimizer.

The later performance comparison will determine whether the additional optimization burden produces a routing solution that is operationally preferable.

31.7 Expected Constraint Complexity

Decision variables are only one measure of model complexity.

The number of mathematical constraints also grows when single-path routing is enforced.

The following chart compares the expected constraint counts across the four optimization stages.

In [214…

```
# Create a bar chart showing expected constraint counts
# for each optimization model.

plt.figure(
    figsize=(10, 5)
)

plt.bar(
    model_size_df[
        "model"
    ],
    model_size_df[
```

```
        "expected_constraints"
    ]
)

plt.xlabel(
    "Optimization Model"
)

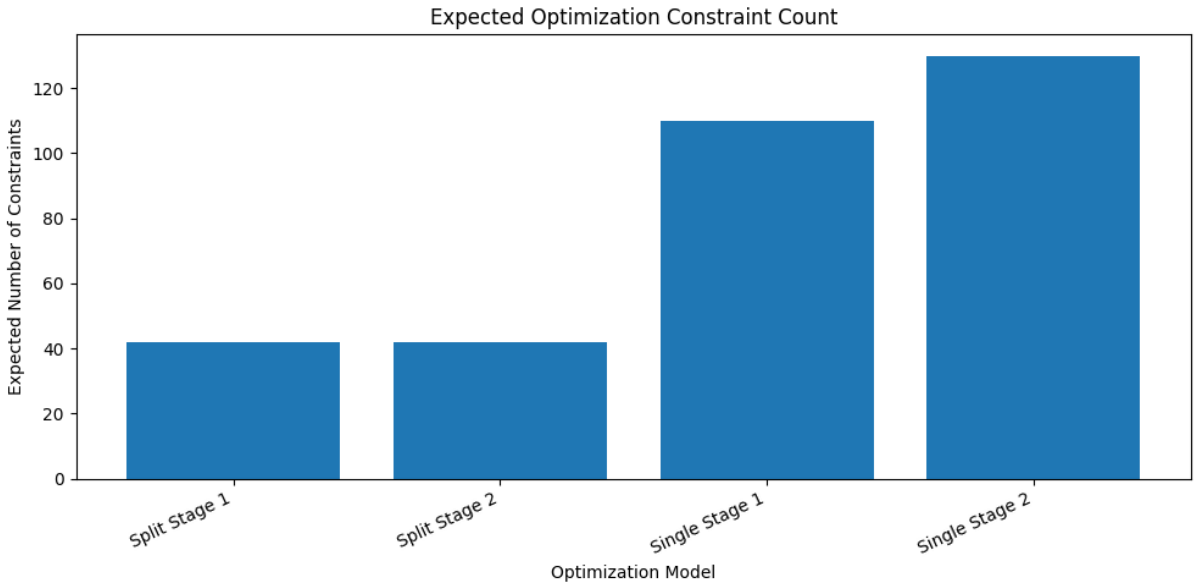
plt.ylabel(
    "Expected Number of Constraints"
)

plt.title(
    "Expected Optimization Constraint Count"
)

plt.xticks(
    rotation=25,
    ha="right"
)

plt.tight_layout()

plt.show()
```



31.7.1 Interpretation of Constraint-Count Visualization

The single-path formulations require substantially more constraints than the splittable formulations.

The increase results primarily from:

- path-selection restrictions;
- flow-to-selection linking constraints;

- Stage 2 delivery-preservation requirements.

The final single-path Stage 2 formulation is therefore the most structurally complex model in the analysis.

For the current network, the problem remains small enough for CBC to solve efficiently.

However, this structure illustrates why single-path routing becomes more computationally demanding as candidate-path counts grow.

31.7.2 Business and Operational Impact

As networks become larger, optimization complexity can affect how frequently routing decisions can be updated.

For example, a rapidly changing network may require optimization every few seconds or minutes.

If a highly constrained MILP takes too long to solve, a theoretically appealing routing policy may become impractical operationally.

The current assignment network is small, but the model-size comparison provides an early view of this production tradeoff.

31.8 Solver Status Interpretation Framework

Every optimization solve will explicitly report solver status before results are interpreted.

The expected successful status is:

$$\text{Optimal}$$

Other statuses require investigation.

For example:

- **Infeasible** — no solution satisfies all constraints;
- **Unbounded** — the objective can improve without limit, usually indicating a formulation error;
- **Not Solved / Undefined** — the solver did not return a usable optimal solution.

No routing results should be interpreted unless the solver status is first confirmed

to be acceptable.

In [216…

```
# Display the PuLP status-code mapping used later
# to translate numerical solver return codes into readable labels.

solver_status_reference = pd.DataFrame([
    {
        "status_code": code,
        "status_name": name
    }
    for code, name
    in pl.LpStatus.items()
])

solver_status_reference
```

Out[216…

	status_code	status_name
0	0	Not Solved
1	1	Optimal
2	-1	Infeasible
3	-2	Unbounded
4	-3	Undefined

31.8.1 Interpretation of Solver Status Controls

Explicitly checking solver status prevents invalid results from being mistaken for optimal network decisions.

For example, an infeasible model may still contain variable objects in memory, but those values should not be interpreted as a routing recommendation.

Therefore, every major optimization stage in the notebook will follow the sequence:

1. construct the model;
2. solve the model;
3. check solver status;
4. validate accounting and constraints;
5. only then interpret routing results.

31.8.2 Business and Operational Impact

In a production environment, optimization status should be treated as a control

signal.

A failed or infeasible optimization should trigger:

- diagnostics;
- fallback routing logic;
- alerts;
- or use of the most recent valid solution.

This prevents an optimization-system failure from silently becoming a network-operational failure.

31.9 Reusable Solver Configuration

The same CBC configuration will be reused throughout the notebook.

Using a single solver object improves consistency across:

- splittable Stage 1;
- splittable Stage 2;
- single-path Stage 1;
- single-path Stage 2;
- bottleneck perturbation experiments;
- stress-test scenarios.

This reduces the risk that model comparisons are affected by inconsistent solver settings.

In [217…

```
# Display the solver class that will be reused
# throughout the optimization experiments.

print(
    "Optimization solver:",
    type(cbc_solver).__name__
)

print(
    "Solver messages enabled:",
    False
)
```

```
Optimization solver: PULP_CBC_CMD
Solver messages enabled: False
```

31.10 Section Summary

Section 31 establishes and validates the optimization environment used in the remainder of the notebook.

The analysis confirms that:

- PuLP is available;
- CBC is available;
- CBC successfully solves a known linear-programming test problem;
- the splittable formulation is an LP;
- the single-path formulation is a MILP;
- the current network contains 68 candidate path-flow decisions;
- the single-path model introduces 68 additional binary route-selection variables;
- single-path routing therefore produces a larger and more constrained optimization problem.

Technical Conclusion

The solver environment is functioning correctly and the network-routing formulations are small enough to be solved directly using CBC.

The mathematical distinction between the two routing variants is now explicit:

$$\text{Splittable} \Rightarrow \text{continuous LP}$$

while:

$$\text{Single Path} \Rightarrow \text{mixed-integer MILP}$$

Business Conclusion

The optimization architecture introduces an important operational tradeoff.

Splittable routing provides greater flexibility and a mathematically simpler optimization problem, but may be more complex to implement operationally.

Single-path routing provides simpler traffic behavior, but requires a more computationally complex optimization model.

The later experiments will determine whether either approach provides sufficient improvements in:

- delivered throughput;
- latency;
- congestion;

- capacity utilization;

to justify its operational complexity.

Connection to the Next Section

Section 30 showed that independent minimum-latency routing is infeasible.

The optimization environment is now validated.

The next step is therefore to construct the first network-wide optimization:

> **Splittable-Flow Stage 1 — maximize total delivered throughput without considering latency yet.**

This Stage 1 solution will establish the maximum amount of current demand that the network can deliver under shared directional capacity constraints.

32. Splittable-Flow LP — Stage 1: Maximize Throughput

Section 30 demonstrated that independently assigning every source-stream commodity to its minimum-latency route is infeasible.

Several low-latency corridors become overloaded even though alternative network capacity remains available elsewhere.

Section 31 then verified that the PuLP/CBC optimization environment is functioning correctly.

The next step is to solve the first network-wide optimization problem.

Stage 1 Question

The objective of Stage 1 is:

> **What is the maximum amount of current demand that can be delivered through the network if each source-stream commodity may split its traffic across multiple feasible paths?**

At this stage, latency is deliberately excluded from the objective.

The model focuses only on determining the maximum achievable end-to-end

throughput subject to:

- source-stream demand limits;
- shared directed-link capacities;
- correct destinations;
- the feasible candidate paths constructed earlier.

This separation is intentional.

The lexicographic optimization strategy first determines the maximum amount of traffic the network can deliver.

Only after that throughput level is established will Stage 2 optimize the latency of UK-L and US-L without reducing the Stage 1 delivery quantities.

32.1 Mathematical Formulation

For each source-stream commodity and candidate path, define:

$$f_{k,s,p} \geq 0$$

where:

- k identifies the traffic stream;
- s identifies the source node;
- p identifies a feasible path;
- $f_{k,s,p}$ is the amount of traffic routed over that path, measured in Mbps.

Because the splittable formulation allows a commodity to distribute traffic across several paths, the path-flow variables are continuous.

No binary route-selection variables are required.

Therefore, Stage 1 is a **linear program**.

Primary Objective

The objective is to maximize total delivered traffic:

$$\boxed{\max \sum_{k} \sum_{s} \sum_{p \in P_{k,s}} f_{k,s,p}}$$

Every delivered Mbps contributes equally to the Stage 1 objective.

This follows the problem requirement that all delivered traffic has equal

throughput value.

Commodity Demand Constraints

For every source-stream commodity:

$$\sum_{p \in P_{k,s}} f_{k,s,p} \leq D_{k,s}$$

where:

$$D_{k,s}$$

is the offered demand of stream k originating at source s.

This prevents the optimizer from delivering more traffic than actually exists.

Directed Link-Capacity Constraints

For every directed arc e:

$$\sum_{k} \sum_{s} \sum_{\substack{p \in P_{k,s} \\ e \in p}} f_{k,s,p} \leq C_e$$

where:

$$C_e$$

is the directional bandwidth capacity of arc e.

All commodities using the same directed arc therefore compete for the same finite bandwidth.

No Latency Objective in Stage 1

Latency is not included in the Stage 1 objective.

The model is intentionally allowed to use slower paths whenever doing so increases total delivered throughput.

This establishes the maximum achievable throughput before latency-sensitive routing preferences are introduced.

32.2 Create the Stage 1 Linear Program

The Stage 1 optimization model is created as a maximization problem.

At this point the model contains no variables, objective, or constraints.

Those components are added sequentially in the following subsections.

In [218…
```
# ------------------------------------------------------------
# Create the Stage 1 splittable-flow optimization model
# ------------------------------------------------------------
#
# LpMaximize tells PuLP that larger objective values are preferred.
#
# The model will eventually contain:
#   - one continuous flow variable for each candidate path;
#   - one demand constraint for each commodity;
#   - one capacity constraint for each directed arc.

split_stage1 = pl.LpProblem(
    "Split_Flow_Stage1",
    pl.LpMaximize
)


print(
    "Model created:",
    split_stage1.name
)

print(
    "Optimization sense:",
    "Maximize"
)
```

```
Model created: Split_Flow_Stage1
Optimization sense: Maximize
```

32.2.1 Interpretation

The optimization container has now been created successfully.

No routing decision has yet been made.

The purpose of separating model creation from variable and constraint construction is to make the mathematical formulation auditable.

Each component of the LP can now be added and validated independently before the solver is called.

32.2.2 Business and Operational Impact

Separating model construction into auditable steps is useful in a production optimization system.

If a routing result later appears unexpected, the implementation can be traced back to:

- decision-variable construction;
- demand limits;
- capacity constraints;
- objective definition.

This improves model governance and reduces the risk of treating an implementation error as a valid network recommendation.

32.3 Create Continuous Path-Flow Variables

The candidate-path table contains one row for every feasible source-stream path.

A non-negative continuous decision variable is created for each candidate path.

For path p:

$$f_p \geq 0$$

The variable represents the amount of traffic, in Mbps, assigned to that candidate route.

Because traffic splitting is allowed, one commodity may have positive flow on more than one candidate path.

In [219…

```
# Create a dictionary that maps each candidate path ID
# to its continuous Stage 1 flow variable.

split_flow_vars = {}


# Iterate through every candidate source-stream path.

for _, row in candidate_paths_df.iterrows():

    # Convert the path identifier to an integer so that it can
    # be used consistently as the dictionary key.

    pid = int(
        row["path_id"]
    )
```

```
    # Create one non-negative continuous flow variable.
    #
    # Example:
    #   split_flow_0
    #   split_flow_1
    #   ...
    #
    # The variable value will represent Mbps assigned
    # to that candidate path.

    split_flow_vars[pid] = (
        pl.LpVariable(
            f"split_flow_{pid}",
            lowBound=0,
            cat="Continuous"
        )
    )


# Print the number of path-flow variables created.

print(
    "Continuous path-flow variables created:",
    len(split_flow_vars)
)
```

```
Continuous path-flow variables created: 68
```

32.3.1 validate the Path-Flow Count

In [220…

```
# Confirm that exactly one decision variable was created
# for every candidate path.

assert (
    len(split_flow_vars)
    ==
    len(candidate_paths_df)
)


# Confirm that path IDs and variable keys match exactly.

assert (
    set(split_flow_vars.keys())
    ==
    set(
        candidate_paths_df[
            "path_id"
        ].astype(int)
    )
```

```
)


print(
    "Path-flow variable validation passed."
)
```

```
Path-flow variable validation passed.
```

32.3.2 Interpretation

The Stage 1 LP contains:

68

continuous path-flow variables.

This matches the 68 candidate source-stream paths generated in Section 29.

The one-to-one correspondence between candidate paths and optimization variables confirms that every feasible routing alternative is available to the solver.

No path is selected or excluded in advance.

This allows the optimization to determine the network-wide combination of routes that maximizes total delivered traffic.

32.3.3 Business and Operational Impact

Allowing all feasible paths to remain available prevents the model from imposing a premature routing policy.

This is operationally important because Section 30 showed that selecting only minimum-latency routes creates severe congestion.

The optimizer now has the freedom to use:

- low-latency C routes;
- slower D routes;
- combinations of several routes;

whenever doing so improves total delivered service.

This allows existing network capacity to be used more efficiently before additional infrastructure is considered.

32.4 Define the Stage 1 Throughput Objective

The objective is the sum of flow placed on every candidate path.

Because each unit of path flow corresponds to one delivered Mbps, the objective directly represents total delivered network throughput.

The model therefore maximizes:

$$\boxed{T = \sum_{p} f_p}$$

subject to demand and capacity constraints added in the following subsections.

In [221…
```
# ------------------------------------------------------------
# Define the Stage 1 maximum-throughput objective
# ------------------------------------------------------------
#
# Summing all path-flow variables gives the total number
# of Mbps successfully routed to the required exits.

split_stage1 += pl.lpSum(
    split_flow_vars[pid]
    for pid in split_flow_vars
)


print(
    "Stage 1 objective added:"
)

print(
    "Maximize total delivered path flow."
)
```

```
Stage 1 objective added:
Maximize total delivered path flow.
```

32.4.1 Interpretation

The LP now has a single objective:

> Deliver as much traffic as physically possible.

The objective does not distinguish among:

- UK-L;
- UK-T;
- US-L;
- US-T.

One delivered Mbps contributes exactly the same throughput value regardless of traffic class.

This is consistent with the clarified requirement that latency-sensitive traffic should not automatically receive higher admission priority.

34.4.2 Business and Operational Impact

The Stage 1 objective represents a capacity-utilization question rather than a service-priority question.

Operationally, it asks:

> What is the maximum amount of customer traffic this network can successfully carry right now?

This creates a clean network-capacity benchmark.

If total demand cannot be delivered even under the flexible splittable model, the difference represents a genuine network-wide capacity limitation rather than merely an inefficient shortest-path routing policy.

32.5 Add Commodity Demand Constraints

Each source-stream commodity may use several candidate paths.

However, the sum of traffic routed across those paths may not exceed that commodity's offered demand.

For commodity (k, s):

$$\sum_{p \in P_{k,s}} f_{k,s,p} \leq D_{k,s}$$

This ensures that the optimization may partially serve demand but can never create additional traffic.

In [222…

```
# ------------------------------------------------------------
# Add one demand constraint for each source-stream commodity
# ------------------------------------------------------------

for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
```

```
):

    # All candidate paths in this group belong to the same commodity,
    # so the source-level demand value is identical across the rows.

    demand = (
        group[
            "demand_mbps"
        ].iloc[0]
    )


    # Sum the path flows assigned to this commodity.
    #
    # The combined routed traffic may not exceed
    # the commodity's current offered demand.

    split_stage1 += (
        pl.lpSum(
            split_flow_vars[
                int(pid)
            ]
            for pid
            in group[
                "path_id"
            ]
        )
        <= demand,

        # Give the constraint a readable unique name.
        f"demand_{commodity_id}"
    )


print(
    "Demand constraints added:",
    demand_df[
        "commodity_id"
    ].nunique()
)
```

```
Demand constraints added: 20
```

32.5.1 Validate Demand Constraint Count

In [223…

```
# Count constraints whose names begin with "demand_".

demand_constraint_names = [
    name
    for name
    in split_stage1.constraints
```

```
    if name.startswith(
        "demand_"
    )
]


assert (
    len(demand_constraint_names)
    ==
    demand_df[
        "commodity_id"
    ].nunique()
)


print(
    "Demand constraint validation passed."
)
```

```
Demand constraint validation passed.
```

32.5.1 Interpretation

The model contains:

$$20$$

commodity-demand constraints — one for each source-stream demand defined in Section 28.

This means the optimizer is free to:

- fully deliver a commodity;
- partially deliver a commodity;
- deliver none of a commodity;

but it may never exceed the traffic currently offered at that source.

This implements the assignment's partial-delivery requirement directly.

32.5.3 Business and Operational Impact

Partial delivery allows the optimizer to represent realistic congestion behavior.

When the network cannot carry all demand, the model does not incorrectly declare the entire problem infeasible.

Instead, it determines the maximum service level that can be supported under current capacity.

This provides operations teams with measurable quantities for:

- served demand;
- lost demand;
- delivery percentage;
- capacity shortfall.

Those values become useful for both traffic engineering and capacity planning.

32.6 Add Directed Link-Capacity Constraints

All source-stream commodities sharing a directed link must share that link's finite bandwidth.

For every directed arc e:

$$\sum_{k} \sum_{s} \sum_{\substack{p \in P_{k,s} \\ e \in p}} f_{k,s,p} \leq C_e$$

The constraint is applied independently in each direction.

For example:

$$A \rightarrow C$$

and:

$$C \rightarrow A$$

have separate capacity constraints.

In [224…

```
# ------------------------------------------------------------
# Add one capacity constraint for every directed network arc
# ------------------------------------------------------------

for _, arc in arcs_df.iterrows():

    # Represent the current directed arc as an ordered tuple.

    edge = (
        arc["from_node"],
        arc["to_node"]
    )


    # Identify every candidate path that contains this directed arc.

    paths_using_edge = (
```

```
        candidate_paths_df[
            candidate_paths_df[
                "path_edges"
            ].apply(
                lambda edges:
                    edge in edges
            )
        ]
    )


    # The sum of flow on all candidate paths using the arc
    # may not exceed that arc's directional bandwidth.

    split_stage1 += (
        pl.lpSum(
            split_flow_vars[
                int(pid)
            ]
            for pid
            in paths_using_edge[
                "path_id"
            ]
        )
        <= arc[
            "capacity_mbps"
        ],

        # Create a readable capacity-constraint name.
        f"capacity_"
        f"{arc['from_node']}_"
        f"{arc['to_node']}"
    )


print(
    "Directed capacity constraints added:",
    len(arcs_df)
)
```

```
Directed capacity constraints added: 22
```

32.6.1 Validate Capacity Constraint Count

In [225…

```
# Identify all capacity constraints by name.

capacity_constraint_names = [
    name
    for name
    in split_stage1.constraints
    if name.startswith(
```

```
        "capacity_"
    )
]


# Confirm that one capacity constraint exists
# for every directed network arc.

assert (
    len(capacity_constraint_names)
    ==
    len(arcs_df)
)


print(
    "Capacity constraint validation passed."
)
```

```
Capacity constraint validation passed.
```

32.6.2 Interpretation

The LP contains:

$$22$$

directed link-capacity constraints.

This matches the 22 directed arcs constructed in Section 27.

Every candidate route using a particular arc contributes to the same capacity constraint.

Therefore, the solver cannot independently allocate the full arc capacity to multiple commodities.

This is the mathematical mechanism that coordinates competing network traffic.

32.6.3 Business and Operational Impact

Shared-capacity constraints convert the problem from independent route selection into network-wide traffic engineering.

Section 30 showed that several commodities individually prefer the same C-side links.

The capacity constraints force the optimizer to recognize that those flows cannot all use those preferred links simultaneously.

Operationally, the model can therefore:

- divert some traffic;
- split commodities across paths;
- use alternate network corridors;
- or leave demand undelivered;

rather than silently overloading infrastructure.

32.7 Inspect the Completed Stage 1 Model

Before solving, the model structure is checked against the counts predicted in Section 31.

For the current baseline problem, Stage 1 should contain:

- 68 continuous path-flow variables;
- 20 commodity-demand constraints;
- 22 directed link-capacity constraints.

Therefore, the expected total number of constraints is:

$$20 + 22 = 42$$

In [226…

```
# Count the actual decision variables created by PuLP.

actual_split_stage1_variables = (
    len(
        split_stage1.variables()
    )
)


# Count the actual constraints in the model.

actual_split_stage1_constraints = (
    len(
        split_stage1.constraints
    )
)


# Print model dimensions.

print(
    "Stage 1 decision variables:",
    actual_split_stage1_variables
)
```

```
print(
    "Stage 1 constraints:",
    actual_split_stage1_constraints
)
```

```
Stage 1 decision variables: 68
Stage 1 constraints: 42
```

32.7.1 Validate Model Dimenstions

In [227…

```
# Confirm the actual LP matches the model dimensions
# predicted in Section 31.

assert (
    actual_split_stage1_variables
    == 68
)

assert (
    actual_split_stage1_constraints
    == 42
)


print(
    "Stage 1 model-dimension validation passed."
)
```

```
Stage 1 model-dimension validation passed.
```

32.7.2 Interpretation

The completed LP contains exactly the expected number of variables and constraints.

This confirms that the implementation matches the mathematical design:

- one path-flow variable for every candidate route;
- one demand constraint for every commodity;
- one capacity constraint for every directed arc.

The model is therefore structurally ready for optimization.

32.7.3 Business and Operational Impact

Structural model checks reduce implementation risk.

If an expected constraint were missing, the solver might still produce an

apparently optimal result, but that result could violate the intended network rules.

Confirming model dimensions provides an additional governance check before routing recommendations are interpreted.

32.8 Solve the Splittable Stage 1 LP

The completed linear program is now passed to the CBC solver.

CBC determines the path-flow allocation that maximizes total delivered traffic while satisfying all demand and directional-capacity constraints.

In [228…

```
# ------------------------------------------------------------
# Solve the Stage 1 LP using the CBC solver configured
# and validated in Section 31
# ------------------------------------------------------------

split_stage1.solve(
    cbc_solver
)


# Convert the numerical solver status into readable text.

split_stage1_status = (
    pl.LpStatus[
        split_stage1.status
    ]
)


# Retrieve the optimized objective value.
#
# Because the objective is the sum of all path flows,
# this value equals maximum delivered throughput.

split_stage1_throughput = (
    pl.value(
        split_stage1.objective
    )
)


# Display solver results.

print(
    "Solver status:",
    split_stage1_status
)
```

```
print(
    "Maximum split-flow throughput:",
    split_stage1_throughput,
    "Mbps"
)
```

```
Solver status: Optimal
Maximum split-flow throughput: 83.0 Mbps
```

32.9 Validate Solver Status and Objectives

In [231…

```
# ------------------------------------------------------------
# Validate Stage 1 solve
# ------------------------------------------------------------

# The solution should not be interpreted unless CBC
# reports an optimal result.

assert (
    split_stage1_status
    == "Optimal"
)


# The objective value must exist.

assert (
    split_stage1_throughput
    is not None
)


# Throughput cannot exceed total offered demand.

assert (
    split_stage1_throughput
    <= total_demand + 1e-6
)


# Throughput must be non-negative.

assert (
    split_stage1_throughput
    >= -1e-6
)


print(
    "Stage 1 solver validation passed."
)
```

```
Stage 1 solver validation passed.
```

32.9.1 Interpretation of Solver Status

CBC reports:

$$\text{Optimal}$$

which means the solver found a feasible routing solution and proved that no other splittable routing configuration can deliver more total traffic under the current synthetic network assumptions.

The resulting throughput is:

$$T^*_{\text{split}} = 83 \text{ Mbps}$$

This is therefore not merely one feasible routing result.

It is the maximum achievable throughput for the baseline splittable-flow formulation.

32.9.2 Business and Operational Impact

The optimal solver status is essential because it converts the 83 Mbps result into a capacity benchmark.

Under the current synthetic network assumptions:

> No alternative splittable routing strategy can deliver more than 83 Mbps without changing network capacity, demand, or model policy.

This provides a defensible upper bound on current baseline service capability.

In [232…

```
# Total current synthetic demand from Section 28.

split_stage1_total_demand = (
    float(
        total_demand
    )
)


# Maximum throughput returned by the Stage 1 LP.

split_stage1_delivered = (
    float(
        split_stage1_throughput
    )
```

```
)


# Demand that cannot be delivered under the
# maximum-throughput splittable solution.

split_stage1_lost = (
    split_stage1_total_demand
    - split_stage1_delivered
)


# Overall percentage of offered demand that can be delivered.

split_stage1_delivery_percentage = (
    100
    * split_stage1_delivered
    / split_stage1_total_demand
)


# Build a compact KPI table.

split_stage1_kpi_df = pd.DataFrame([
    {
        "metric":
            "Total Offered Demand",
        "value":
            split_stage1_total_demand,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "Maximum Delivered Throughput",
        "value":
            split_stage1_delivered,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "Undelivered Demand",
        "value":
            split_stage1_lost,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "Overall Delivery Rate",
        "value":
```

```
            split_stage1_delivery_percentage,
        "unit":
            "%"
    }
])


split_stage1_kpi_df
```

Out [232…

	metric	value	unit
0	Total Offered Demand	126.000000	Mbps
1	Maximum Delivered Throughput	83.000000	Mbps
2	Undelivered Demand	43.000000	Mbps
3	Overall Delivery Rate	65.873016	%

32.10.1 Interpretation of Headline Results

The baseline synthetic network receives:

$$126 \text{ Mbps}$$

of offered demand.

Even when traffic is allowed to split flexibly across multiple feasible routes, the network can deliver only:

$$83 \text{ Mbps}$$

at maximum throughput.

Therefore:

$$126 - 83 = 43 \text{ Mbps}$$

of current demand cannot be delivered.

The maximum delivery percentage is:

$$\frac{83}{126} \times 100 \approx 65.87\%$$

This means approximately:

$$34.13\%$$

of offered traffic remains unserved.

Critical Technical Finding

Section 30 showed that naive minimum-latency routing fails because traffic is poorly distributed.

Stage 1 now shows something deeper:

> Even after the optimizer is allowed to coordinate traffic globally and use multiple routes, all 126 Mbps still cannot be delivered.

Therefore, the baseline network has a genuine end-to-end capacity limitation in addition to the routing inefficiency identified earlier.

32.10.2 Business and Operational Impact

The 83 Mbps maximum-throughput result has a direct capacity-planning interpretation.

The network is currently capable of serving only about:

$$65.9\%$$

of the offered baseline traffic.

This creates two separate operational opportunities.

1. Routing Optimization

The optimizer can use existing infrastructure more intelligently than naive minimum-latency routing.

This is necessary because Section 30 showed that independent shortest-route decisions overload the C corridor while leaving other capacity underused.

2. Capacity Expansion

Routing optimization alone is not sufficient to serve all 126 Mbps.

Even the flexible splittable model reaches a hard throughput ceiling of 83 Mbps.

Therefore, if the business requirement were to serve all current demand, additional effective network capacity or another operational policy would eventually be required.

The later bottleneck analysis will determine **where** additional bandwidth would

create actual incremental throughput rather than simply adding capacity to already non-limiting links.

32.11 Overall Stage 1 Delivery Visualization

The following chart compares:

- total offered demand;
- maximum deliverable traffic;
- undelivered traffic.

This provides a high-level visual representation of the baseline network's throughput limit.

In [235…

```
# Create a simple bar chart showing the overall
# Stage 1 throughput outcome.

stage1_overall_plot_df = pd.DataFrame({
    "category": [
        "Delivered",
        "Undelivered"
    ],
    "mbps": [
        split_stage1_delivered,
        split_stage1_lost
    ]
})


plt.figure(
    figsize=(8, 5)
)

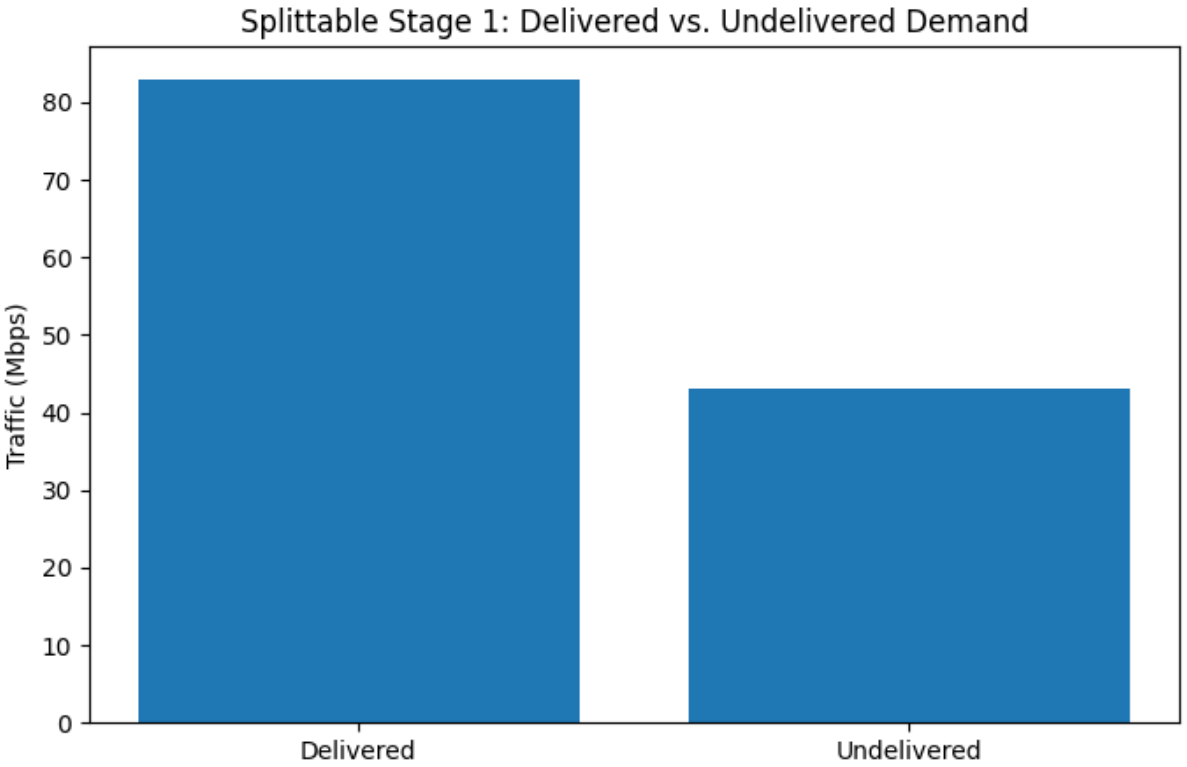
plt.bar(
    stage1_overall_plot_df[
        "category"
    ],
    stage1_overall_plot_df[
        "mbps"
    ]
)

plt.ylabel(
    "Traffic (Mbps)"
)

plt.title(
    "Splittable Stage 1: "
    "Delivered vs. Undelivered Demand"
```

```
)

plt.show()
```



32.11.1 Interpretation of the Delivery Visualization

The visualization shows that a substantial majority of offered traffic can be served, but the gap between demand and network capability remains significant.

The network delivers:

$$83 \text{ Mbps}$$

while:

$$43 \text{ Mbps}$$

remains undelivered.

The chart also makes clear that the baseline capacity shortfall is not marginal.

Approximately one-third of offered demand cannot be served under the current network conditions.

32.11.2 Business and Operational Impact

A service shortfall of approximately one-third of offered demand would be

operationally significant.

If the synthetic scenario represented a real network snapshot, the business would need to consider some combination of:

- improved routing;
- traffic prioritization;
- additional network capacity;
- demand shaping;
- service-level differentiation.

However, capacity investment should not yet be made based solely on the total shortfall.

The next sections determine:

- which commodities are being dropped;
- which paths carry the delivered traffic;
- which links are saturated;
- which saturated links are true throughput bottlenecks.

Those results are necessary before identifying the most valuable operational intervention.

32.12 Comparison with the Naive Minimum-Latency Baseline

Section 30 showed that sending all 126 Mbps over independently selected minimum-latency routes violates several capacity constraints.

The Stage 1 optimization resolves this issue by coordinating path allocation across all commodities.

The distinction is:

Naive Minimum-Latency Routing

Attempts to deliver:

$$126 \text{ Mbps}$$

but violates network capacity constraints.

Therefore, it is **not a feasible routing solution**.

Optimized Splittable Routing

Delivers:

$$83 \text{ Mbps}$$

while satisfying the network's demand and capacity constraints.

Therefore, the optimized result represents a feasible network operating point.

32.12.1 Interpretation

It would be incorrect to compare 126 Mbps from the naive policy directly against 83 Mbps from the optimizer as if the naive policy performed better.

The 126 Mbps value represents **attempted traffic**, not successfully feasible traffic.

The optimizer's 83 Mbps result is the maximum amount that can actually be supported by the network.

This distinction separates:

- offered demand;
- attempted routing;
- feasible delivered throughput.

Maintaining those definitions consistently is important throughout the remainder of the notebook.

32.12.2 Business and Operational Impact

A routing system should not be evaluated only on how much traffic it attempts to send.

It must be evaluated on how much traffic can be delivered while respecting physical network limits.

The Stage 1 model provides that operationally meaningful measure.

This prevents apparently aggressive routing policies from being mistaken for high-performance policies when they actually depend on impossible link loads.

32.13 What Stage 1 Does Not Yet Tell Us

The 83 Mbps objective value answers the first optimization question:

> **How much traffic can the network deliver?**

It does not yet fully answer:

- which source-stream commodities receive service;
- which commodities are partially served;
- which commodities are dropped;
- which routes carry the delivered traffic;
- which links become heavily utilized;
- whether the Stage 1 admission pattern is fair;
- which saturated links are true throughput bottlenecks.

Those questions require extraction and validation of the optimized decision-variable values.

They are addressed in Section 33.

32.13.1 Important Objective-Function Caveat

Stage 1 maximizes only:

$$\text{total delivered Mbps}$$

It does **not** optimize fairness across:

- sources;
- destinations;
- traffic classes.

Therefore, multiple different admission patterns may achieve the same:

$$83 \text{ Mbps}$$

maximum throughput.

CBC is free to return any throughput-optimal allocation unless additional policy objectives are imposed.

This point becomes important when the source-stream delivery results are examined in Section 33.

32.13.2 Business and Operational Impact

Maximum throughput alone may not represent an acceptable production policy.

For example, two routing solutions could both deliver 83 Mbps while:

- one distributes loss broadly;
- another completely drops several sources or traffic classes.

Both are mathematically throughput-optimal, but their customer and business effects could be very different.

Therefore, throughput should be treated as one performance dimension rather than the complete routing policy.

The assignment explicitly weights delivered traffic equally, so no additional fairness requirement is imposed in the baseline model.

Fairness can instead be evaluated later as a separate policy scenario.

32.14 Stage 1 Result KPI Visualization

In [237…

```
# Create a compact horizontal comparison of:
#   total offered demand
#   maximum feasible delivered throughput

throughput_capacity_df = pd.DataFrame({
    "metric": [
        "Offered Demand",
        "Maximum Feasible Throughput"
    ],
    "mbps": [
        split_stage1_total_demand,
        split_stage1_delivered
    ]
})


plt.figure(
    figsize=(9, 4)
)

plt.barh(
    throughput_capacity_df[
        "metric"
    ],
    throughput_capacity_df[
        "mbps"
    ]
)

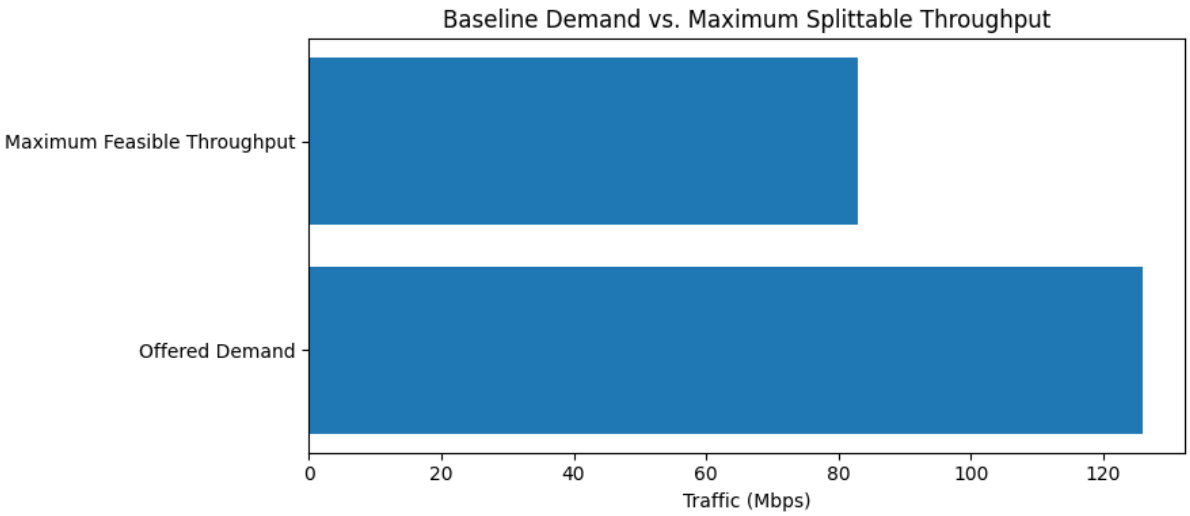
plt.xlabel(
    "Traffic (Mbps)"
```

```
)

plt.title(
    "Baseline Demand vs. "
    "Maximum Splittable Throughput"
)

plt.tight_layout()

plt.show()
```



32.14.1 Interpretation

The chart frames the 83 Mbps result as a network-capability comparison.

Offered demand is:

$$126 \text{ Mbps}$$

while the best feasible splittable routing solution supports:

$$83 \text{ Mbps}$$

The difference represents traffic that cannot be carried under the current baseline capacity configuration.

34.14.2 Business and Operational Impact

The demand-versus-capability view is useful for executive communication because it translates the mathematical objective into a simple service-capacity gap.

The key question for later analysis becomes:

> **What causes the 43 Mbps gap, and what intervention would reduce it most effectively?**

Section 33 begins answering that question by examining the detailed Stage 1 routing solution.

32.15 Section Summary

Section 32 constructs and solves the first network-wide optimization model.

The splittable Stage 1 LP contains:

- 68 continuous path-flow variables;
- 20 source-stream demand constraints;
- 22 directed link-capacity constraints;
- 42 total constraints.

CBC returns an:

$$\text{Optimal}$$

solution.

The maximum feasible throughput is:

$$\boxed{83 \text{ Mbps}}$$

from:

$$126 \text{ Mbps}$$

of offered demand.

Therefore:

$$43 \text{ Mbps}$$

remains undelivered.

The overall baseline delivery rate is approximately:

$$65.87\%$$

Technical Conclusion

Section 30 demonstrated that naive minimum-latency routing is infeasible because traffic converges on the same low-latency links.

Section 32 demonstrates that coordinated multipath routing resolves those capacity violations, but it still cannot deliver all current demand.

The network therefore has both:

1. a routing-allocation problem;
2. a genuine end-to-end capacity limitation.

Business Conclusion

Optimization improves the use of existing infrastructure, but routing flexibility alone cannot eliminate the baseline service shortfall.

If full demand delivery were required, the network would eventually need:

- additional effective capacity;
- an alternative policy;
- or both.

However, expanding capacity indiscriminately would be inefficient.

The next analysis must determine where traffic is actually lost and which network resources constrain service.

Connection to Section 33

The objective value tells us:

> **How much can be delivered?**

Section 33 now asks:

> **Which traffic was delivered, which traffic was lost, which routes carried it, and which links became constrained?**

Those results will reveal how the maximum-throughput objective allocates the network's limited capacity.

33. Split Stage 1 Results and Validation

Section 32 answered the first optimization question:

> **What is the maximum amount of current traffic that the network can deliver when source-stream demand may be split**

> **across multiple paths?**

The optimal Stage 1 result was:

$$83 \text{ Mbps}$$

delivered from:

$$126 \text{ Mbps}$$

of total offered demand.

Therefore:

$$43 \text{ Mbps}$$

of demand cannot be delivered under the current synthetic network capacities.

That objective value is important, but it is only the beginning of the analysis.

A total-throughput value does not explain:

- which traffic was delivered;
- which traffic was lost;
- where the lost demand occurred;
- which routes were selected;
- which links carried the greatest traffic;
- where network capacity became constrained;
- whether the solution treats sources or traffic classes differently.

Section 33 therefore decomposes the Stage 1 solution into source-stream, route, and link-level results.

The analysis proceeds from the most detailed traffic level upward:

1. source-stream delivery;
2. service-status classification;
3. aggregate stream results;
4. aggregate source results;
5. throughput accounting validation;
6. route selection;
7. link utilization;
8. residual capacity;
9. network-flow visualization;
10. congestion and policy interpretation.

The purpose is not simply to confirm that the solver returned a result.

The purpose is to understand **why the 83 Mbps solution looks the way it does and what that means operationally.**

33.1 Source-Stream Delivery Results

The Stage 1 LP optimizes path-level traffic variables.

However, the business problem is expressed in terms of demand.

Therefore, the first result-extraction step converts optimized path flows back into delivered demand for each source-stream commodity.

For each commodity (k, s):

$$z_{k,s} = \sum_{p \in P_{k,s}} f_{k,s,p}$$

where:

- $D_{k,s}$ is offered demand;
- $z_{k,s}$ is delivered demand.

Lost demand is:

$$\text{Lost}_{k,s} = D_{k,s} - z_{k,s}$$

and delivery percentage is:

$$\text{Delivery Percentage}_{k,s} = 100 \frac{z_{k,s}}{D_{k,s}}$$

This calculation is necessary because the solver does not directly return a customer-facing delivery table.

Instead, it returns path-flow decisions.

Those path-level decisions must be recombined to determine how much service each source-stream commodity actually received.

This is also the first place where we can examine whether maximizing total throughput produces uneven service across sources or traffic classes.

In [239…

```
# ------------------------------------------------------------
# 33.1 Extract delivered, lost, and percentage-served demand
```

```
#      for every source-stream commodity
# ------------------------------------------------------------

# Create an empty Python list.
#
# Each dictionary appended to this list will represent
# the complete Stage 1 service result for one commodity.

split_stage1_delivery = []


# ------------------------------------------------------------
# Group candidate paths by commodity
# ------------------------------------------------------------
#
# A commodity is one unique source-stream combination such as:
#
#   S1_UK-L
#   B_US-T
#
# A commodity may have several candidate paths.
#
# Stage 1 delivery for that commodity is therefore the SUM
# of optimized flow over all of its possible paths.

for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    # ------------------------------------------------------------
    # Calculate delivered traffic for the commodity
    # ------------------------------------------------------------
    #
    # Each candidate path has one PuLP decision variable.
    #
    # The .value() method retrieves the optimized Mbps assigned
    # to that path.
    #
    # Some unused variables may return None, so:
    #
    #     value() or 0
    #
    # safely converts an unused variable into zero flow.

    delivered = sum(
        (
            split_flow_vars[
                int(pid)
            ].value()
            or 0
```

```
        )
        for pid
        in group[
            "path_id"
        ]
    )


    # ----------------------------------------------------------
    # Retrieve offered demand
    # ----------------------------------------------------------
    #
    # Every candidate path belonging to the same commodity
    # repeats the same offered demand value.
    #
    # Therefore, the first row is sufficient.

    demand = float(
        group[
            "demand_mbps"
        ].iloc[0]
    )


    # ----------------------------------------------------------
    # Calculate lost demand
    # ----------------------------------------------------------
    #
    # Anything offered but not successfully routed is lost
    # during this independent snapshot.

    lost = (
        demand
        - delivered
    )


    # ----------------------------------------------------------
    # Calculate delivery percentage
    # ----------------------------------------------------------

    delivery_percentage = (
        100
        * delivered
        / demand
        if demand > 0
        else 0
    )


    # ----------------------------------------------------------
    # Store all results for this commodity
```

```
    # ------------------------------------------------------------

    split_stage1_delivery.append({
        "commodity_id":
            commodity_id,
        "source":
            group[
                "source"
            ].iloc[0],
        "stream":
            group[
                "stream"
            ].iloc[0],
        "destination":
            group[
                "destination"
            ].iloc[0],
        "demand_mbps":
            demand,
        "delivered_mbps":
            delivered,
        "lost_mbps":
            lost,
        "delivery_percentage":
            delivery_percentage
    })


# ------------------------------------------------------------
# Convert the commodity-result list into a DataFrame
# ------------------------------------------------------------

split_stage1_delivery_df = (
    pd.DataFrame(
        split_stage1_delivery
    )
)


# ------------------------------------------------------------
# Create consistent source and stream ordering
# ------------------------------------------------------------
#
# Pandas would otherwise sort strings alphabetically.
#
# Defining categorical order keeps tables and plots consistent
# across Sections 28, 33, 34, and the later model comparison.

source_order = [
    "S1",
    "S2",
    "S3",
```

```
    "B",
    "C"
]

stream_order = [
    "UK-L",
    "UK-T",
    "US-L",
    "US-T"
]


# Apply source ordering.

split_stage1_delivery_df[
    "source"
] = pd.Categorical(
    split_stage1_delivery_df[
        "source"
    ],
    categories=source_order,
    ordered=True
)


# Apply stream ordering.

split_stage1_delivery_df[
    "stream"
] = pd.Categorical(
    split_stage1_delivery_df[
        "stream"
    ],
    categories=stream_order,
    ordered=True
)


# Sort the final table.

split_stage1_delivery_df = (
    split_stage1_delivery_df
    .sort_values(
        [
            "source",
            "stream"
        ]
    )
    .reset_index(
        drop=True
    )
)
```

```
# Display the completed source-stream service table.

split_stage1_delivery_df
```

Out [239…

	commodity_id	source	stream	destination	demand_mbps	delivered_mbps
0	S1_UK-L	S1	UK-L	UK	6.0	6.0
1	S1_UK-T	S1	UK-T	UK	7.0	7.0
2	S1_US-L	S1	US-L	US	5.0	0.0
3	S1_US-T	S1	US-T	US	6.0	0.0
4	S2_UK-L	S2	UK-L	UK	5.0	0.0
5	S2_UK-T	S2	UK-T	UK	6.0	6.0
6	S2_US-L	S2	US-L	US	5.0	5.0
7	S2_US-T	S2	US-T	US	5.0	0.0
8	S3_UK-L	S3	UK-L	UK	5.0	0.0
9	S3_UK-T	S3	UK-T	UK	7.0	2.0
10	S3_US-L	S3	US-L	US	6.0	0.0
11	S3_US-T	S3	US-T	US	6.0	0.0
12	B_UK-L	B	UK-L	UK	6.0	6.0
13	B_UK-T	B	UK-T	UK	14.0	14.0
14	B_US-L	B	US-L	US	8.0	8.0
15	B_US-T	B	US-T	US	9.0	9.0
16	C_UK-L	C	UK-L	UK	4.0	4.0
17	C_UK-T	C	UK-T	UK	5.0	5.0
18	C_US-L	C	US-L	US	5.0	5.0
19	C_US-T	C	US-T	US	6.0	6.0

33.1.1 Interpretation of Source-Stream Delivery Results

The Stage 1 result is not evenly distributed across the 20 commodities.

Some commodities receive:

$$100\%$$

of their demand.

Others receive:

$$0\%$$

and one commodity receives only partial service.

B

All four B commodities are fully delivered:

- UK-L: 6 of 6 Mbps;
- UK-T: 14 of 14 Mbps;
- US-L: 8 of 8 Mbps;
- US-T: 9 of 9 Mbps.

Therefore:

$$37/37 = 100\%$$

of B-originating traffic is delivered.

C

All four C commodities are also fully delivered:

$$20/20 = 100\%$$

S1

S1 receives full UK service:

$$6 + 7 = 13 \text{ Mbps}$$

but none of its US demand is delivered.

Total S1 service is therefore:

$$13/24 \approx 54.17\%$$

S2

S2 receives:

- 0 of 5 Mbps UK-L;
- 6 of 6 Mbps UK-T;

- 5 of 5 Mbps US-L;
- 0 of 5 Mbps US-T.

Total delivery is:

$$11/21 \approx 52.38\%$$

S3

S3 experiences the greatest service loss.

Only:

$$2 \text{ Mbps}$$

of its:

$$7 \text{ Mbps}$$

UK-T traffic is delivered.

Its other streams receive no service.

Total S3 delivery is therefore:

$$2/24 \approx 8.33\%$$

Why This Happens

This pattern occurs because Stage 1 optimizes only:

$$\text{total delivered Mbps}$$

It does not contain a fairness objective or minimum-service requirement.

Therefore, the solver is allowed to allocate scarce capacity in any way that produces the maximum total of:

$$83 \text{ Mbps}$$

This result should therefore be interpreted as a **throughput-optimal allocation**, not as a recommendation that B or C traffic is more important.

33.1.2 Business and Operational Impact

This result highlights an important difference between:

- optimizing network efficiency;

- designing service policy.

The network-wide throughput objective is doing exactly what it was asked to do:

> maximize total delivered traffic.

However, a real business may also care about:

- minimum service guarantees;
- customer fairness;
- geographic fairness;
- application priority;
- contractual service-level agreements.

Those goals are not part of the baseline Stage 1 objective.

The source-stream table therefore reveals a policy question rather than a modeling error.

If a production network could not tolerate one source receiving only 8.33% service while another receives 100%, a fairness or service-guarantee rule would need to be added explicitly.

For this assignment, the baseline remains unchanged because Prateek confirmed that all delivered traffic should have equal throughput value.

33.2 Classify Commodity Service Outcomes

The detailed delivery table contains exact Mbps values, but a simpler classification helps summarize the admission pattern.

Each commodity is classified into one of three categories:

- **Fully Delivered** — essentially 100% of offered demand is served;
- **Partially Delivered** — some but not all demand is served;
- **Not Delivered** — essentially zero demand is served.

This step helps answer:

> Is congestion being spread across many commodities, or concentrated on a smaller number?

That question matters because two solutions can have the same total throughput but very different customer-level impacts.

In [240…

```
# ------------------------------------------------------------
# Classify every commodity according to its Stage 1
# delivery percentage
# ------------------------------------------------------------

def classify_delivery(row):
    """
    Convert a numerical delivery percentage into a readable
    Stage 1 service-status category.
    """

    # Numerical optimization results can contain very small
    # floating-point differences.
    #
    # Therefore, values extremely close to 100% are treated
    # as full delivery.

    if (
        row[
            "delivery_percentage"
        ]
        >= 100 - 1e-6
    ):
        return "Fully Delivered"


    # Values extremely close to zero are treated
    # as not delivered.

    if (
        row[
            "delivery_percentage"
        ]
        <= 1e-6
    ):
        return "Not Delivered"


    # Anything strictly between zero and 100%
    # is classified as partial service.

    return "Partially Delivered"


# Apply the classification to every commodity.

split_stage1_delivery_df[
    "service_status"
] = (
    split_stage1_delivery_df.apply(
        classify_delivery,
        axis=1
```

```
    )
)


# ------------------------------------------------------------
# Count how many commodities belong to each category
# ------------------------------------------------------------

stage1_service_status_df = (
    split_stage1_delivery_df[
        "service_status"
    ]
    .value_counts()
    .rename_axis(
        "service_status"
    )
    .reset_index(
        name=
            "number_of_commodities"
    )
)


stage1_service_status_df
```

Out[240…

	service_status	number_of_commodities
0	Fully Delivered	12
1	Not Delivered	7
2	Partially Delivered	1

33.2.1 Interpretation of Service-Status Results

The Stage 1 solution contains:

- 12 fully delivered commodities;
- 1 partially delivered commodity;
- 7 commodities receiving no service.

This means the unavoidable:

$$43 \text{ Mbps}$$

of lost traffic is **not distributed evenly** across all commodities.

Instead, the throughput solution concentrates much of the loss on a smaller number of source-stream combinations.

This is a natural consequence of maximizing a single total-throughput objective without fairness constraints.

33.2.2 Business and Operational Impact

Concentrated loss may or may not be desirable.

For example, if certain flows had explicitly lower business priority, intentionally dropping those flows could be reasonable.

However, the current model has no such information.

Therefore, the concentrated loss should not be interpreted as an intentional service policy.

This distinction is important when presenting the result to stakeholders:

> The optimizer found an efficient capacity allocation, but it was not asked to produce an equitable allocation.

33.3 Delivered vs. Lost Demand by Commodity

The service-status table summarizes how many commodities are affected, but it does not show the magnitude of the loss.

A stacked bar chart therefore compares:

- delivered Mbps;
- lost Mbps;

for every source-stream commodity.

This visualization connects the original demand values from Section 28 directly to the Stage 1 service outcome.

33.3 Delivered vs. Lost Demand by Commodity

The service-status table summarizes how many commodities are affected, but it does not show the magnitude of the loss.

A stacked bar chart therefore compares:

- delivered Mbps;

- lost Mbps;

for every source-stream commodity.

This visualization connects the original demand values from Section 28 directly to the Stage 1 service outcome.

In [242…

```
# ------------------------------------------------------------
# Prepare source-stream results for visualization
# ------------------------------------------------------------

commodity_delivery_plot_df = (
    split_stage1_delivery_df.copy()
)


# Create readable labels such as:
#
#   S1 / UK-L
#   B / US-T

commodity_delivery_plot_df[
    "commodity_label"
] = (
    commodity_delivery_plot_df[
        "source"
    ].astype(str)
    + " / "
    + commodity_delivery_plot_df[
        "stream"
    ].astype(str)
)


# ------------------------------------------------------------
# Plot delivered and lost demand as stacked bars
# ------------------------------------------------------------

commodity_delivery_plot_df.set_index(
    "commodity_label"
)[
    [
        "delivered_mbps",
        "lost_mbps"
    ]
].plot(
    kind="bar",
    stacked=True,
    figsize=(14, 6)
)
```

```
# Label the chart.

plt.ylabel(
    "Demand (Mbps)"
)

plt.xlabel(
    "Source-Stream Commodity"
)

plt.title(
    "Splittable Stage 1: "
    "Delivered vs. Lost Demand by Commodity"
)


# Rotate labels because there are 20 commodities.

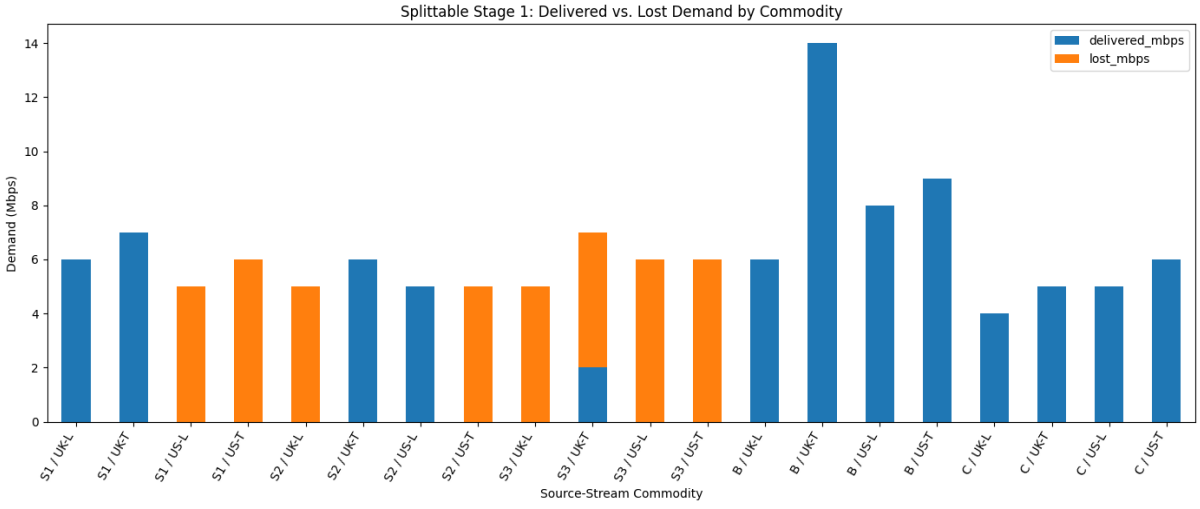
plt.xticks(
    rotation=60,
    ha="right"
)


# Prevent labels from being cut off.

plt.tight_layout()


# Render the chart.

plt.show()
```



33.3.1 Interpretation of Delivered vs. Lost Demand Visualization

The chart makes the uneven admission pattern immediately visible.

B and C show no lost traffic.

By contrast, several S1, S2, and S3 commodities are entirely lost.

S3 is particularly affected.

This confirms that the 65.87% overall network delivery rate does not imply that each commodity receives approximately 65.87% of its own demand.

Aggregate network performance and individual service outcomes are very different measures.

33.3.2 Business and Operational Impact

This chart demonstrates why an executive dashboard should not report only total throughput.

A network could appear relatively healthy at the aggregate level while specific customers, sources, regions, or applications experience complete service loss.

Operational reporting should therefore include both:

- total network performance;
- segmented service performance.

This prevents strong aggregate metrics from masking severe localized degradation.

33.4 Source-by-Stream Delivery Percentage Matrix

Section 28 introduced a source-by-stream demand matrix.

The same structure is now used to show the percentage of each commodity's demand delivered by Stage 1.

This creates a direct before-and-after analytical relationship:

- Section 28 showed **where demand exists**;
- Section 33 shows **how much of that demand is served**.

Rows represent sources.

Columns represent traffic streams.

Each cell represents delivery percentage.

In [243…

```
# ------------------------------------------------------------
# Create a source-by-stream delivery matrix
# ------------------------------------------------------------

stage1_delivery_matrix = (
    split_stage1_delivery_df
    .pivot(
        index="source",
        columns="stream",
        values=
            "delivery_percentage"
    )
)


# Reapply the logical ordering established earlier.

stage1_delivery_matrix = (
    stage1_delivery_matrix
    .reindex(
        index=
            source_order,
        columns=
            stream_order
    )
)


# Display numerical values.

stage1_delivery_matrix
```

Out[243…

stream	UK-L	UK-T	US-L	US-T
source				
S1	100.0	100.000000	0.0	0.0
S2	0.0	100.000000	100.0	0.0
S3	0.0	28.571429	0.0	0.0
B	100.0	100.000000	100.0	100.0
C	100.0	100.000000	100.0	100.0

In [244…

```
# ------------------------------------------------------------
# Visualize source-stream delivery percentage -- Heatmap
```

```
# ------------------------------------------------------------

fig, ax = plt.subplots(
    figsize=(8, 5)
)


# Render matrix values.
#
# vmin=0 and vmax=100 ensure that the visualization
# consistently represents delivery percentage.

image = ax.imshow(
    stage1_delivery_matrix.values,
    aspect="auto",
    vmin=0,
    vmax=100
)


# Add stream labels.

ax.set_xticks(
    range(
        len(
            stage1_delivery_matrix.columns
        )
    )
)

ax.set_xticklabels(
    stage1_delivery_matrix.columns
)


# Add source labels.

ax.set_yticks(
    range(
        len(
            stage1_delivery_matrix.index
        )
    )
)

ax.set_yticklabels(
    stage1_delivery_matrix.index
)


# Write the actual percentage inside every cell.
```

```
for i in range(
    len(
        stage1_delivery_matrix.index
    )
):

    for j in range(
        len(
            stage1_delivery_matrix.columns
        )
    ):

        value = (
            stage1_delivery_matrix.iloc[
                i,
                j
            ]
        )

        ax.text(
            j,
            i,
            f"{value:.0f}%",
            ha="center",
            va="center"
        )


# Label the axes and chart.

ax.set_xlabel(
    "Traffic Stream"
)

ax.set_ylabel(
    "Source Node"
)

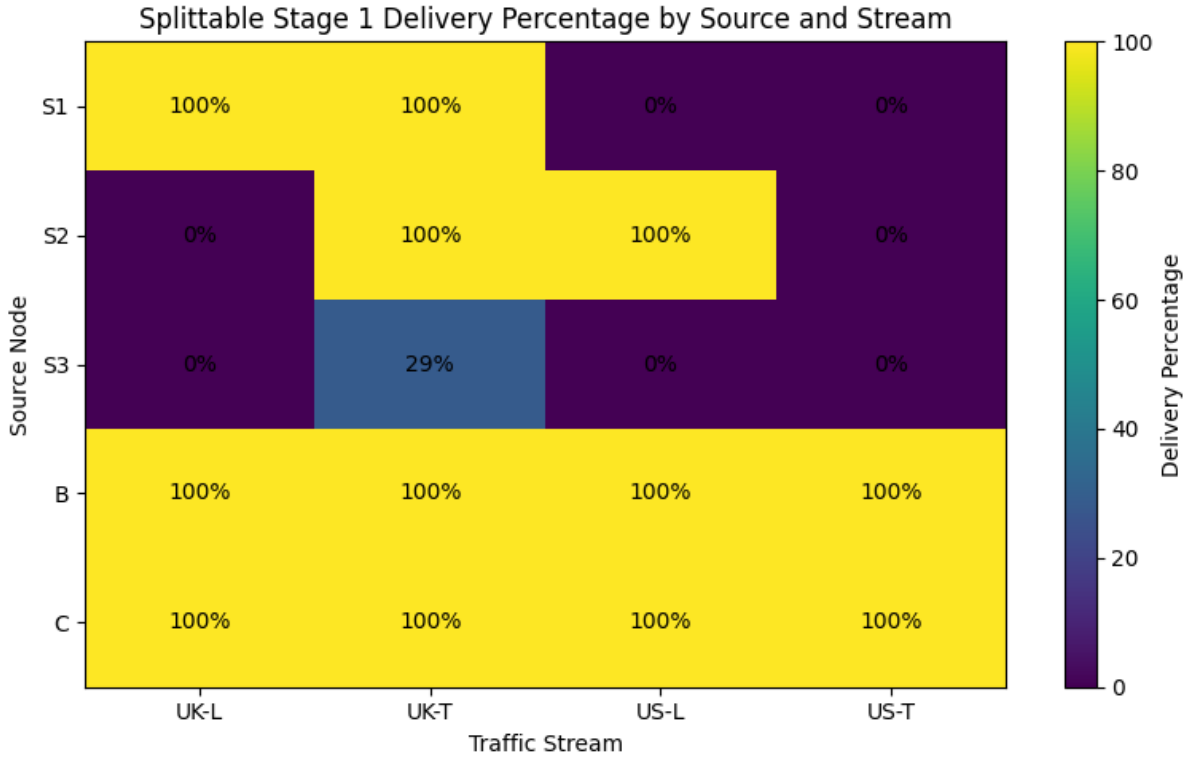
ax.set_title(
    "Splittable Stage 1 "
    "Delivery Percentage by Source and Stream"
)


# Add a percentage scale.

plt.colorbar(
    image,
    ax=ax,
    label=
        "Delivery Percentage"
)
```

```
plt.tight_layout()

plt.show()
```



33.4.1 Interpretation of the Delivery Heatmap

The heatmap reveals that Stage 1 service loss is not determined by one simple rule.

For example:

- B receives 100% service across every class;
- C receives 100% service across every class;
- S1 receives UK traffic but not US traffic;
- S2 receives UK-T and US-L but not the other two classes;
- S3 receives almost no service.

This is important because Stage 1 contains no:

- UK preference;
- US preference;
- latency-sensitive preference;
- tolerant-traffic preference.

The observed pattern emerges from the interaction of topology, capacity, route availability, and the throughput objective.

33.4.2 Business and Operational Impact

The heatmap is useful because it transforms a mathematical solution into a service-quality view.

In a production application, a similar chart could identify:

- customer groups experiencing poor service;
- regional disparities;
- traffic classes consistently losing capacity.

It also provides a natural comparison point for later policy experiments such as fairness or service prioritization.

33.5 Aggregate Stage 1 Delivery by Traffic Stream

The assignment ultimately asks for results for the four traffic streams:

- UK-L;
- UK-T;
- US-L;
- US-T.

Although optimization is performed at the source-stream level, source contributions can be aggregated after the solution.

For stream k:

$$D_k = \sum_s D_{k,s}$$

and:

$$z_k = \sum_s z_{k,s}$$

This connects the detailed commodity-level optimization back to the stream-level outputs requested in the problem statement.

In [245…

```
# ------------------------------------------------------------
# Aggregate demand and delivery by traffic stream
```

```
# ------------------------------------------------------------

split_stage1_stream_summary = (
    split_stage1_delivery_df
    .groupby(
        "stream",
        observed=True,
        as_index=False
    )
    .agg(
        demand_mbps=(
            "demand_mbps",
            "sum"
        ),
        delivered_mbps=(
            "delivered_mbps",
            "sum"
        ),
        lost_mbps=(
            "lost_mbps",
            "sum"
        )
    )
)


# Calculate stream-level delivery percentage.

split_stage1_stream_summary[
    "delivery_percentage"
] = (
    100
    * split_stage1_stream_summary[
        "delivered_mbps"
    ]
    / split_stage1_stream_summary[
        "demand_mbps"
    ]
)


# Restore logical ordering.

split_stage1_stream_summary[
    "stream"
] = pd.Categorical(
    split_stage1_stream_summary[
        "stream"
    ],
    categories=
        stream_order,
    ordered=True
```

```
)


split_stage1_stream_summary = (
    split_stage1_stream_summary
    .sort_values(
        "stream"
    )
    .reset_index(
        drop=True
    )
)


split_stage1_stream_summary
```

Out [245...

	stream	demand_mbps	delivered_mbps	lost_mbps	delivery_percentage
0	UK-L	26.0	16.0	10.0	61.538462
1	UK-T	39.0	34.0	5.0	87.179487
2	US-L	29.0	18.0	11.0	62.068966
3	US-T	32.0	15.0	17.0	46.875000

33.5.1 Interpretation of Aggregate Stream Results

The stream-level results are:

Stream	Demand	Delivered	Lost	Delivery Rate
UK-L	26	16	10	61.54%
UK-T	39	34	5	87.18%
US-L	29	18	11	62.07%
US-T	32	15	17	46.88%

UK-T receives the highest aggregate service level.

US-T receives the lowest.

Importantly, the latency-sensitive streams are **not automatically favored**.

This is expected.

Prateek explicitly confirmed that latency sensitivity should affect the latency objective rather than Stage 1 admission priority.

33.5.2 Business and Operational Impact

The stream-level results illustrate the difference between:

- admission policy;
- routing-quality policy.

Stage 1 determines how much traffic can be admitted.

Stage 2 will optimize where the admitted latency-sensitive traffic travels.

If the business instead wanted latency-sensitive traffic to receive admission priority, that would require a different policy formulation.

Keeping those objectives separate makes the model easier to explain, audit, and modify.

33.6 Delivered and Lost Demand by Stream

The following visualization shows the same aggregate stream results graphically.

It helps identify which traffic classes absorb the largest portion of the network's 43 Mbps throughput shortfall.

In [246…

```
# Plot delivered and lost traffic for each aggregate stream.

split_stage1_stream_summary.set_index(
    "stream"
)[
    [
        "delivered_mbps",
        "lost_mbps"
    ]
].plot(
    kind="bar",
    stacked=True,
    figsize=(9, 5)
)


plt.ylabel(
    "Traffic (Mbps)"
)

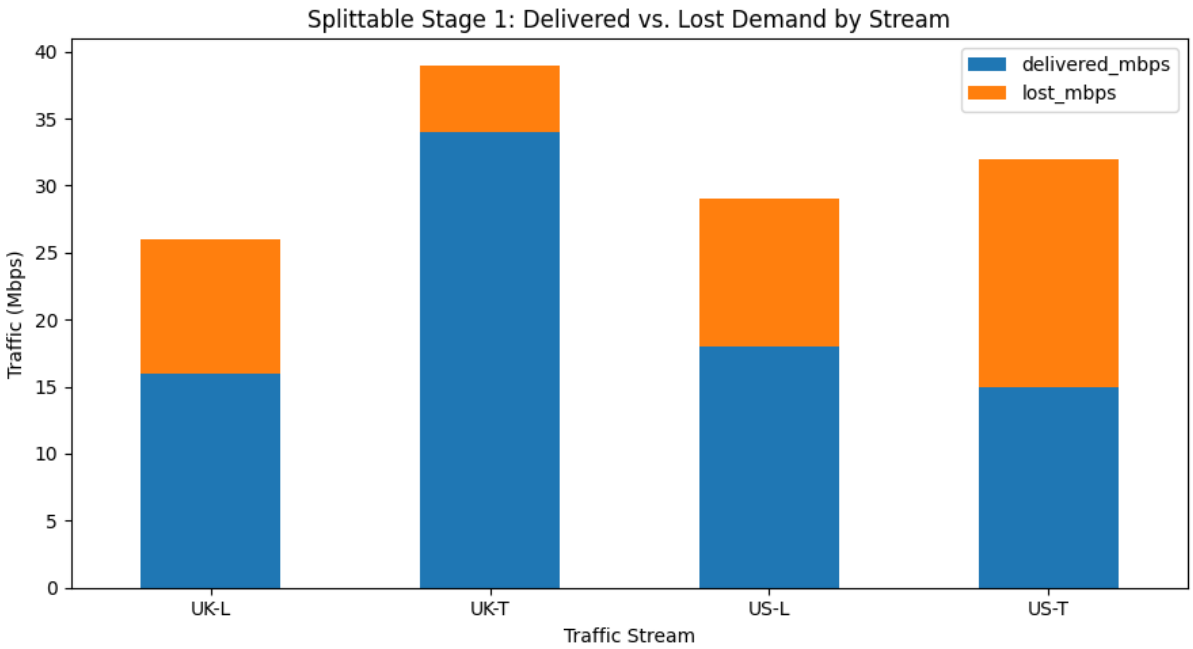
plt.xlabel(
    "Traffic Stream"
)
```

```
plt.title(
    "Splittable Stage 1: "
    "Delivered vs. Lost Demand by Stream"
)

plt.xticks(
    rotation=0
)

plt.tight_layout()

plt.show()
```



33.6.1 Interpretation and Business Impact

US-T experiences the largest aggregate loss:

$$17 \text{ Mbps}$$

while UK-T loses only:

$$5 \text{ Mbps}$$

This should not be interpreted as a deliberate preference for UK-T.

The result emerges from the source distribution of those demands and the routes available to them.

For business analysis, stream-level performance should therefore always be examined together with source-level performance.

Interpretation and Business Impact

33.7 Aggregate Stage 1 Delivery by Source

The stream-level view answers:

> Which traffic classes are being served?

The source-level view answers:

> Where is service being delivered or lost?

Because traffic enters the network at different physical locations, two sources with similar total demand may have very different network impact.

This analysis helps determine whether topology influences the Stage 1 admission pattern.

In [247…

```
# ------------------------------------------------------------
# Aggregate Stage 1 service by source node
# ------------------------------------------------------------

split_stage1_source_summary = (
    split_stage1_delivery_df
    .groupby(
        "source",
        observed=True,
        as_index=False
    )
    .agg(
        demand_mbps=(
            "demand_mbps",
            "sum"
        ),
        delivered_mbps=(
            "delivered_mbps",
            "sum"
        ),
        lost_mbps=(
            "lost_mbps",
            "sum"
        )
    )
)


# Calculate source-level delivery percentage.

split_stage1_source_summary[
```

```
    "delivery_percentage"
] = (
    100
    * split_stage1_source_summary[
        "delivered_mbps"
    ]
    / split_stage1_source_summary[
        "demand_mbps"
    ]
)


# Restore network-source order.

split_stage1_source_summary[
    "source"
] = pd.Categorical(
    split_stage1_source_summary[
        "source"
    ],
    categories=
        source_order,
    ordered=True
)


split_stage1_source_summary = (
    split_stage1_source_summary
    .sort_values(
        "source"
    )
    .reset_index(
        drop=True
    )
)


split_stage1_source_summary
```

Out [247...

	source	demand_mbps	delivered_mbps	lost_mbps	delivery_percentage
0	S1	24.0	13.0	11.0	54.166667
1	S2	21.0	11.0	10.0	52.380952
2	S3	24.0	2.0	22.0	8.333333
3	B	37.0	37.0	0.0	100.000000
4	C	20.0	20.0	0.0	100.000000

33.7.1 Interpretation of Source-Level Delivery

The source-level results are approximately:

Source	Demand	Delivered	Lost	Delivery Rate
S1	24	13	11	54.17%
S2	21	11	10	52.38%
S3	24	2	22	8.33%
B	37	37	0	100%
C	20	20	0	100%

B and C receive complete service.

S3 receives only 8.33%.

One plausible structural reason is that B and C are located closer to the exit corridors and require fewer shared upstream resources to reach their destinations.

However, this should not be interpreted as proof that physical distance alone determines admission.

The LP may have multiple alternative throughput-optimal allocations.

33.7.2 Business and Operational Impact

If source nodes represented regions, customers, satellites, or gateway locations, a throughput-only policy could create significant geographic service disparity.

This reinforces the need to distinguish:

- capacity efficiency;
- customer-policy objectives.

A production network may choose to impose minimum source-level service guarantees even if doing so slightly reduces total throughput.

33.8 Delivered and Lost Demand by Source

A source-level stacked bar chart makes the topological imbalance visible.

This chart is particularly useful when explaining why similar demand levels do not necessarily produce similar service levels.

In [248…

```
split_stage1_source_summary.set_index(
    "source"
)[
    [
        "delivered_mbps",
        "lost_mbps"
    ]
].plot(
    kind="bar",
    stacked=True,
    figsize=(9, 5)
)


plt.ylabel(
    "Traffic (Mbps)"
)

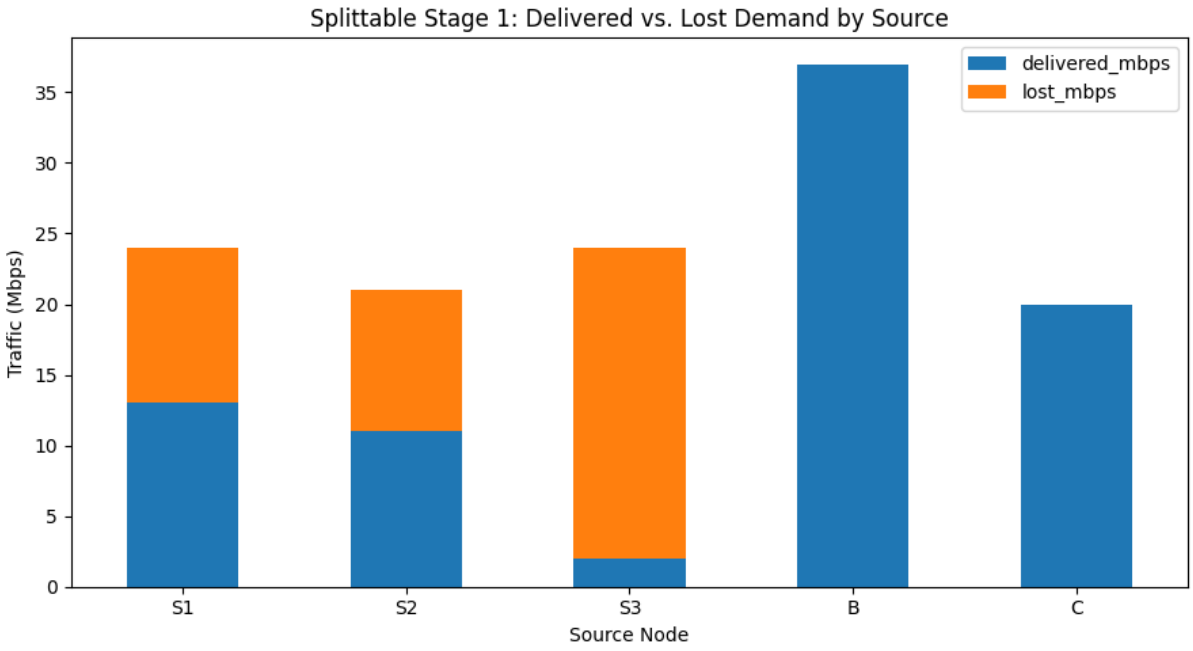
plt.xlabel(
    "Source Node"
)

plt.title(
    "Splittable Stage 1: "
    "Delivered vs. Lost Demand by Source"
)

plt.xticks(
    rotation=0
)

plt.tight_layout()

plt.show()
```



33.8.1 Interpretation and Business Impact

The chart shows that network position matters.

B and C receive complete service even though B has the highest total offered demand.

S3, by contrast, has only 24 Mbps of demand but experiences very high loss.

This demonstrates that congestion cannot be understood from source demand alone.

The location of demand and the capacity of the downstream route structure are equally important.

33.9 Throughput Accounting Validation

Before analyzing routes and link utilization, the reporting calculations are checked against the optimization objective.

Three quantities must reconcile:

$$\text{Total Demand} = \text{Delivered} + \text{Lost}$$

and:

$$\text{Delivered} = T^*_{\text{split}}$$

This provides an independent check that the result-extraction code is consistent

with the solver.

In [249…

```
# Calculate total delivered traffic from the
# commodity-level results.

split_total_delivered = (
    split_stage1_delivery_df[
        "delivered_mbps"
    ].sum()
)


# Calculate total lost traffic.

split_total_lost = (
    split_stage1_delivery_df[
        "lost_mbps"
    ].sum()
)


# Display accounting values.

print(
    "Total demand:",
    total_demand
)

print(
    "Delivered:",
    split_total_delivered
)

print(
    "Lost:",
    split_total_lost
)

print(
    "Delivered + Lost:",
    split_total_delivered
    + split_total_lost
)


# ------------------------------------------------------------
# Validation 1:
# Delivered + Lost must equal offered demand
# ------------------------------------------------------------

assert abs(
    (
```

```
        split_total_delivered
        + split_total_lost
    )
    - total_demand
) < 1e-6


# ------------------------------------------------------------
# Validation 2:
# Commodity-level delivery must match Stage 1 LP objective
# ------------------------------------------------------------

assert abs(
    split_total_delivered
    - split_stage1_throughput
) < 1e-6


print(
    "Stage 1 throughput accounting passed."
)
```

```
Total demand: 126
Delivered: 83.0
Lost: 43.0
Delivered + Lost: 126.0
Stage 1 throughput accounting passed.
```

33.9.1 Interpretation and Business Impact

The accounting check confirms:

$$83 + 43 = 126$$

and the commodity-level delivered total matches the solver objective exactly.

This means the reported 43 Mbps loss is a genuine result of the modeled network capacity rather than an error introduced during result extraction.

Reconciliation like this is essential for trustworthy operational reporting.

33.10 Extract Active Stage 1 Routes

The commodity table explains how much traffic was served.

The next question is:

> **Which routes carried that traffic?**

Only candidate paths with positive optimized flow are retained.

This allows us to compare the optimized routing pattern with the naive minimum-latency paths from Section 30.

In [250…

```
# ------------------------------------------------------------
# Extract positive-flow candidate paths
# ------------------------------------------------------------

split_stage1_route_results = []


# Examine every one of the 68 candidate paths.

for _, row in (
    candidate_paths_df.iterrows()
):

    # Retrieve candidate path identifier.

    pid = int(
        row[
            "path_id"
        ]
    )


    # Retrieve optimized Stage 1 flow.

    flow = (
        split_flow_vars[
            pid
        ].value()
        or 0
    )


    # Only retain routes carrying meaningful positive traffic.
    #
    # The small tolerance avoids including numerical values
    # extremely close to zero.

    if flow > 1e-8:

        split_stage1_route_results.append({
            "model":
                "Split Stage 1",
            "commodity_id":
                row[
                    "commodity_id"
                ],
            "source":
                row[
                    "source"
```

```
                ],
            "stream":
                row[
                    "stream"
                ],
            "destination":
                row[
                    "destination"
                ],
            "path":
                row[
                    "path"
                ],
            "flow_mbps":
                flow,
            "path_capacity_mbps":
                row[
                    "path_capacity_mbps"
                ],
            "path_latency_ms":
                row[
                    "path_latency_ms"
                ]
        })


# Convert route results into a DataFrame.

split_stage1_routes_df = (
    pd.DataFrame(
        split_stage1_route_results
    )
)


# Display active-route count.

print(
    "Active Stage 1 routes:",
    len(
        split_stage1_routes_df
    )
)


# Display active routing table.

split_stage1_routes_df
```

```
Active Stage 1 routes: 13
```

Out[250…

	model	commodity_id	source	stream	destination	path	flow_mbps	path_

S1 -

0	Split Stage 1	S1_UK-L	S1	UK-L	UK	> A -> C -> UK	6.0
1	Split Stage 1	S1_UK-T	S1	UK-T	UK	S1 -> A -> C -> UK	7.0
2	Split Stage 1	S2_UK-T	S2	UK-T	UK	S2 -> A -> C -> UK	6.0
3	Split Stage 1	S2_US-L	S2	US-L	US	S2 -> A -> C -> US	5.0
4	Split Stage 1	S3_UK-T	S3	UK-T	UK	S3 -> B -> C -> UK	2.0
5	Split Stage 1	B_UK-L	B	UK-L	UK	B -> D -> UK	6.0
6	Split Stage 1	B_UK-T	B	UK-T	UK	B -> D -> UK	14.0
7	Split Stage 1	B_US-L	B	US-L	US	B -> D -> US	8.0
8	Split Stage 1	B_US-T	B	US-T	US	B -> C -> US	9.0
9	Split Stage 1	C_UK-L	C	UK-L	UK	C -> UK	4.0
10	Split Stage 1	C_UK-T	C	UK-T	UK	C -> UK	5.0
11	Split Stage 1	C_US-L	C	US-L	US	C -> US	5.0
	Split					C ->	

12	Stage 1	C_US-T	C	US-T	US	US	6.0

33.10.1 Interpretation of Active Routes

The Stage 1 solution uses a much broader network pattern than the naive minimum-latency policy.

The most important change is activation of the D corridor.

In Section 30, no commodity selected D under independent minimum-latency routing.

Stage 1 instead sends substantial traffic through:

$$B \to D$$

and then to UK or US.

This happens because the optimizer recognizes that low-latency C-side capacity is scarce.

By using the slower D corridor, the model can increase total delivered traffic.

This is a concrete example of why:

> the fastest path for one commodity is not necessarily the best route for the network as a whole.

33.10.2 Actual Splitting Behavior

The LP permits each commodity to use multiple paths.

However, permission to split does not guarantee that splitting will occur.

The following calculation counts the number of positive-flow routes used by each delivered commodity.

In [251…

```
# Count positive-flow routes used by each delivered commodity.

active_route_count_by_commodity = (
    split_stage1_routes_df
    .groupby(
        "commodity_id",
        as_index=False
```

```
    )
    .size()
    .rename(
        columns={
            "size":
                "number_of_active_routes"
        }
    )
)


active_route_count_by_commodity
```

Out [251…

	commodity_id	number_of_active_routes
0	B_UK-L	1
1	B_UK-T	1
2	B_US-L	1
3	B_US-T	1
4	C_UK-L	1
5	C_UK-T	1
6	C_US-L	1
7	C_US-T	1
8	S1_UK-L	1
9	S1_UK-T	1
10	S2_UK-T	1
11	S2_US-L	1
12	S3_UK-T	1

In [252…

```
# Identify commodities using more than one route.

commodities_actually_split = (
    active_route_count_by_commodity[
        active_route_count_by_commodity[
            "number_of_active_routes"
        ] > 1
    ]
)


print(
    "Delivered commodities using multiple active paths:",
    len(
```

```
        commodities_actually_split
    )
)
```

```
Delivered commodities using multiple active paths: 0
```

Interpretation of Actual Splitting Behavior

Although the Stage 1 model permits multipath routing, the baseline optimum returned by CBC does not require any delivered commodity to use more than one positive-flow path.

This means that:

- splitting is mathematically available;
- but the current 83 Mbps optimum can be achieved without using it.

This should not be interpreted as evidence that splitting is never useful.

It means only that this particular baseline network does not require it to achieve maximum throughput.

Later stress tests should deliberately create cases where fragmented residual capacity makes multipath routing valuable.

Business and Operational Impact

This is an encouraging implementation result for the baseline case.

The optimizer gains the flexibility of a splittable formulation while producing an operating point that does not actually require complex multipath behavior for the admitted commodities.

If this pattern remained true in production, network implementation could potentially remain simpler.

However, the conclusion must be tested under other network conditions before being generalized.

33.11 Calculate Stage 1 Directed Link Utilization

Path flows must now be converted into link-level traffic.

This connects the optimization decisions back to physical network resources.

For each directed arc e:

$$F_e = \sum_{p:e \in p} f_p$$

Residual capacity is:

$$R_e = C_e - F_e$$

and utilization is:

$$U_e = 100 \frac{F_e}{C_e}$$

This allows us to identify where the optimized network is lightly loaded, heavily loaded, or saturated.

In [253…

```
def calculate_link_results(
    candidate_paths_df,
    flow_vars,
    arcs_df,
    model_name
):
    """
    Convert optimized path-level traffic into directed
    link-level flow and utilization metrics.

    This function will be reused for:
        - Split Stage 1
        - Split Stage 2
        - Single-Path Stage 1
        - Single-Path Stage 2

    Parameters
    ----------
    candidate_paths_df : DataFrame
        Candidate routes and their directed arc sequences.

    flow_vars : dict
        PuLP path-flow variables containing optimized values.

    arcs_df : DataFrame
        Directed network arcs and capacities.

    model_name : str
        Human-readable model label.

    Returns
    -------
    DataFrame
        Link-level capacity, optimized flow,
```

```
    remaining capacity, utilization, and saturation.
"""

results = []


# ----------------------------------------------------------
# Evaluate every directed network arc independently
# ----------------------------------------------------------

for _, arc in (
    arcs_df.iterrows()
):

    # Represent current arc as an ordered pair.

    edge = (
        arc[
            "from_node"
        ],
        arc[
            "to_node"
        ]
    )


    # ----------------------------------------------------
    # Find every candidate path that contains this arc
    # ----------------------------------------------------

    using_edge = (
        candidate_paths_df[
            candidate_paths_df[
                "path_edges"
            ].apply(
                lambda edges:
                    edge in edges
            )
        ]
    )


    # ----------------------------------------------------
    # Sum optimized flow from every path using this arc
    # ----------------------------------------------------

    flow = sum(
        (
            flow_vars[
                int(pid)
            ].value()
            or 0
```

```
            )
            for pid
            in using_edge[
                "path_id"
            ]
        )


        # Retrieve directional capacity.

        capacity = float(
            arc[
                "capacity_mbps"
            ]
        )


        # Remaining unused bandwidth.

        residual = (
            capacity
            - flow
        )


        # Percentage utilization.

        utilization = (
            100
            * flow
            / capacity
            if capacity > 0
            else 0
        )


        # A link is marked saturated when flow and capacity
        # differ by less than numerical tolerance.

        is_saturated = (
            abs(
                capacity
                - flow
            )
            < 1e-6
        )


        # Store link results.

        results.append({
            "model":
```

```
                model_name,
            "from_node":
                arc[
                    "from_node"
                ],
            "to_node":
                arc[
                    "to_node"
                ],
            "capacity_mbps":
                capacity,
            "flow_mbps":
                flow,
            "residual_capacity_mbps":
                residual,
            "utilization_percentage":
                utilization,
            "is_saturated":
                is_saturated
        })


    return pd.DataFrame(
        results
    )
```

In [254…

```
split_stage1_link_df = (
    calculate_link_results(
        candidate_paths_df,
        split_flow_vars,
        arcs_df,
        "Split Stage 1"
    )
)


# Sort from most utilized to least utilized.

split_stage1_link_df = (
    split_stage1_link_df
    .sort_values(
        "utilization_percentage",
        ascending=False
    )
    .reset_index(
        drop=True
    )
)


split_stage1_link_df
```

Out [254...

	model	from_node	to_node	capacity_mbps	flow_mbps	residual_capacity_
0	Split Stage 1	C	UK	30.0	30.0	
1	Split Stage 1	C	US	25.0	25.0	
2	Split Stage 1	B	D	28.0	28.0	
3	Split Stage 1	A	C	25.0	24.0	
4	Split Stage 1	D	UK	22.0	20.0	
5	Split Stage 1	B	C	18.0	11.0	
6	Split Stage 1	S1	A	40.0	13.0	
7	Split Stage 1	S2	A	35.0	11.0	
8	Split Stage 1	D	US	30.0	8.0	
9	Split Stage 1	S3	B	30.0	2.0	
10	Split Stage 1	UK	D	22.0	0.0	
11	Split Stage 1	US	C	25.0	0.0	
12	Split Stage 1	UK	C	30.0	0.0	
13	Split Stage 1	C	B	18.0	0.0	
	Split					

14	Stage 1	D	B	28.0	0.0
15	Split Stage 1	A	S1	40.0	0.0
16	Split Stage 1	C	A	25.0	0.0
17	Split Stage 1	B	A	20.0	0.0
18	Split Stage 1	A	B	20.0	0.0
19	Split Stage 1	B	S3	30.0	0.0
20	Split Stage 1	A	S2	35.0	0.0
21	Split Stage 1	US	D	30.0	0.0

Interpretation of Stage 1 Link Utilization

The highest-utilization links are concentrated in the internal and exit portions of the network.

Three directed links reach:

$$100\%$$

utilization:

- C → UK;
- C → US;
- B → D.

Other heavily loaded links include:

$$A \to C = 96\%$$

and:

$$D \to UK \approx 90.91\%$$

By comparison, the source-entry links retain substantial headroom.

For example:

$$S1 \to A = 32.5\%$$

$$S2 \to A \approx 31.43\%$$

$$S3 \to B \approx 6.67\%$$

This confirms that the network limitation occurs after source traffic begins converging toward shared internal and exit corridors.

Business and Operational Impact

The utilization results narrow the capacity-planning problem.

The network does not appear uniformly capacity constrained.

Increasing bandwidth at S1, S2, or S3 entry links would likely provide little immediate value because those links currently contain substantial unused capacity.

Pressure is instead concentrated deeper in the network.

This suggests that capacity investment should be targeted rather than uniform.

However, high utilization alone does not prove that capacity expansion will improve throughput.

That distinction will be tested later using marginal-capacity analysis.

33.12 Validate Directed Link-Capacity Feasibility

Section 30 demonstrated that naive minimum-latency routing violates network capacities.

The optimized Stage 1 solution should do the opposite.

Every directed arc must satisfy:

$$F_e \leq C_e$$

This validation confirms that the 83 Mbps solution is actually physically feasible under the modeled capacities.

In [256…

```python
# Calculate positive capacity violations, if any.

split_stage1_link_df[
    "capacity_violation_mbps"
] = (
    split_stage1_link_df[
        "flow_mbps"
    ]
    - split_stage1_link_df[
        "capacity_mbps"
    ]
).clip(
    lower=0
)


# Find the largest violation.

max_capacity_violation = (
    split_stage1_link_df[
        "capacity_violation_mbps"
    ].max()
)


# No meaningful violation should exist.

assert (
    max_capacity_violation
    <= 1e-6
)


print(
    "Maximum directed capacity violation:",
    max_capacity_violation,
    "Mbps"
)

print(
    "All Stage 1 link-capacity constraints satisfied."
)
```

```
Maximum directed capacity violation: 0.0 Mbps
All Stage 1 link-capacity constraints satisfied.
```

Interpretation and Business Impact

The optimized solution produces no directional capacity violations.

This is a fundamental difference from the naive baseline.

Section 30 attempted to send all 126 Mbps but overloaded several links.

Stage 1 instead determines the maximum traffic that can actually be routed while respecting physical limitations.

Therefore:

$$83 \text{ Mbps}$$

represents feasible delivered traffic rather than merely attempted traffic.

33.13 Stage 1 Link Utilization Visualization

The utilization table contains the exact values.

The following chart makes the congestion pattern easier to compare visually.

Only active directed links are shown.

The 100% reference line identifies full utilization.

In [ ]:

In [257…

```
# Keep only arcs carrying meaningful positive traffic.

stage1_active_link_plot_df = (
    split_stage1_link_df[
        split_stage1_link_df[
            "flow_mbps"
        ] > 1e-8
    ]
    .copy()
)


# Build readable directed-link labels.

stage1_active_link_plot_df[
    "link"
] = (
    stage1_active_link_plot_df[
        "from_node"
    ]
    + " → "
    + stage1_active_link_plot_df[
        "to_node"
    ]
)
```

```
# Sort links from highest to lowest utilization.

stage1_active_link_plot_df = (
    stage1_active_link_plot_df
    .sort_values(
        "utilization_percentage",
        ascending=False
    )
)


# Plot utilization.

plt.figure(
    figsize=(11, 6)
)

plt.bar(
    stage1_active_link_plot_df[
        "link"
    ],
    stage1_active_link_plot_df[
        "utilization_percentage"
    ]
)


# Draw the 100% saturation threshold.

plt.axhline(
    100,
    linestyle="--"
)


plt.ylabel(
    "Utilization (%)"
)

plt.xlabel(
    "Directed Link"
)

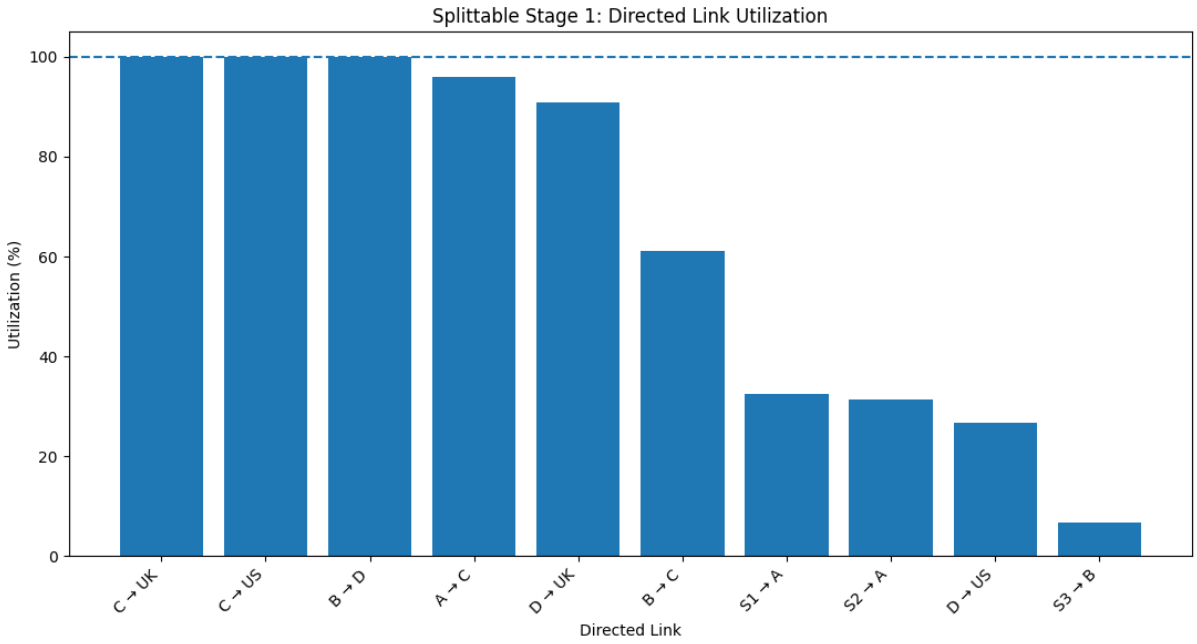
plt.title(
    "Splittable Stage 1: "
    "Directed Link Utilization"
)

plt.xticks(
    rotation=45,
    ha="right"
```

```
)

plt.tight_layout()

plt.show()
```



Interpretation and Business Impact

The chart confirms that congestion is concentrated rather than uniform.

A small number of internal and exit links operate near their physical limits while other network links remain lightly used.

This is a classic network-flow phenomenon:

> unused capacity somewhere in the graph does not necessarily help if the capacity is not located on a complete usable path between a source and its required exit.

This is why capacity planning must consider network structure rather than simply total installed bandwidth.

33.14 Residual Capacity After Stage 1

A useful complement to utilization is residual bandwidth.

For directed arc e:

$$R_e = C_e - F_e$$

This helps answer:

> Where does unused bandwidth remain after maximum throughput is reached?

If lost demand exists while some links retain capacity, the unused bandwidth may be structurally stranded.

In [258…

```
# Create readable labels for every directed arc.

stage1_residual_df = (
    split_stage1_link_df.copy()
)

stage1_residual_df[
    "link"
] = (
    stage1_residual_df[
        "from_node"
    ]
    + " → "
    + stage1_residual_df[
        "to_node"
    ]
)


# Sort from smallest residual capacity
# to largest residual capacity.

stage1_residual_df = (
    stage1_residual_df
    .sort_values(
        "residual_capacity_mbps",
        ascending=True
    )
)


# Plot remaining bandwidth.

plt.figure(
    figsize=(12, 7)
)

plt.barh(
    stage1_residual_df[
        "link"
    ],
    stage1_residual_df[
        "residual_capacity_mbps"
```

```
    ]
)

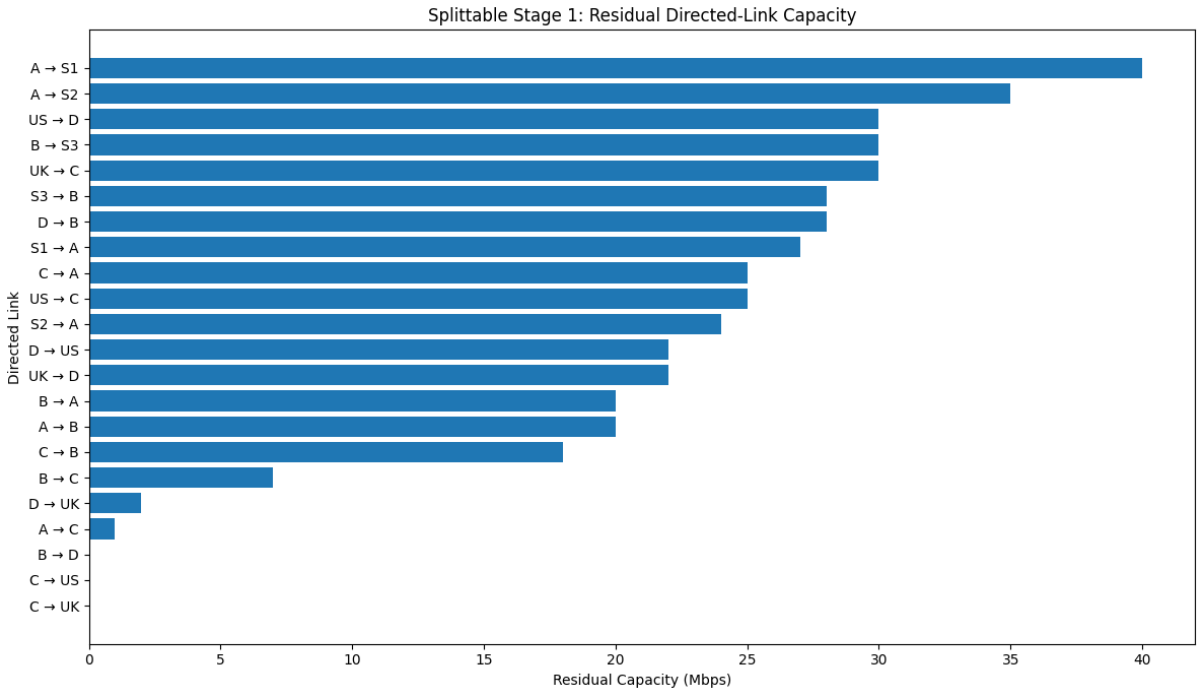
plt.xlabel(
    "Residual Capacity (Mbps)"
)

plt.ylabel(
    "Directed Link"
)

plt.title(
    "Splittable Stage 1: "
    "Residual Directed-Link Capacity"
)

plt.tight_layout()

plt.show()
```



Interpretation of Residual Capacity

The network still contains unused bandwidth even though:

$$43 \text{ Mbps}$$

of demand remains undelivered.

This does not indicate an optimization failure.

End-to-end traffic can only be delivered when **every arc on a complete feasible**

path has sufficient residual capacity.

Unused capacity on one isolated arc may therefore be unusable because another required arc is already saturated.

This is why:

$$\text{total unused bandwidth}$$

is not equal to:

$$\text{additional network throughput}.$$

Business and Operational Impact

Residual capacity can represent stranded infrastructure.

The capacity may physically exist but cannot produce additional customer service because another part of the end-to-end route is constraining traffic.

This reinforces the importance of marginal-capacity analysis later in the notebook.

The business should invest where additional bandwidth creates additional delivered traffic, not simply where unused or saturated bandwidth appears.

33.15 Stage 1 Network Flow Visualization

The previous subsections examined the Stage 1 solution through:

- source-stream delivery;
- aggregate stream delivery;
- source-level delivery;
- active routes;
- directed-link utilization;
- residual capacity.

Those tables and charts explain individual components of the solution.

However, the routing problem is inherently spatial.

The following network visualization places the optimized traffic back onto the original network topology so that the complete routing pattern can be inspected visually.

The diagram is designed to show:

- where optimized traffic enters the network;
- which internal corridors carry the greatest amount of traffic;
- which alternative paths are activated;
- how much traffic each active direction carries;
- available directional capacity;
- utilization percentage.

Visual Encoding

For each physical network connection:

- **line thickness** increases with the total traffic carried by that physical connection;
- **directional labels** show optimized flow, capacity, and utilization;
- links with no optimized traffic remain visually light.

The directional label format is:

$$\text{Flow}/\text{Capacity}\ (\text{Utilization})$$

For example:

$$B \to D : 28/28\ \text{Mbps}\ (100\%)$$

means that the B → D direction is fully utilized.

This visualization should be compared directly with the naive minimum-latency network from Section 30.

Section 30 showed what happens when every commodity independently chooses its fastest route.

Section 33.15 shows how the global optimizer redistributes traffic when all commodities must share finite network capacity.

33.15.1 Create the Reusable Network-Flow Visualization Function

In [259…

```
def plot_network_flow(
    graph,
    link_results_df,
    pos,
    title="Optimized Network Flow"
):
```

```
    """
    Visualize optimized directional traffic on the physical network.

    Parameters
    ----------
    graph : nx.Graph
        Undirected physical network used for visualization.

    link_results_df : pd.DataFrame
        Directed link-level optimization results containing:
            - from_node
            - to_node
            - capacity_mbps
            - flow_mbps
            - utilization_percentage

    pos : dict
        Fixed node coordinates used throughout the notebook.

    title : str
        Title displayed above the network figure.

    Visual Interpretation
    ---------------------
    Physical edge thickness represents total traffic carried
    across the connection.

    Directional labels display:
        optimized flow / directional capacity (utilization %)

    The optimization itself remains directional even though the
    underlying physical connection is drawn as one line.
    """

    # ------------------------------------------------------------
    # Create the plotting canvas
    # ------------------------------------------------------------

    plt.figure(
        figsize=(15, 9)
    )


    # ------------------------------------------------------------
    # Draw network nodes
    # ------------------------------------------------------------

    nx.draw_networkx_nodes(
        graph,
        pos=pos,
        node_size=2400
    )
```

```
# ------------------------------------------------------------
# Draw node labels
# ------------------------------------------------------------

nx.draw_networkx_labels(
    graph,
    pos=pos,
    font_size=11
)


# ------------------------------------------------------------
# Build a directional flow lookup
# ------------------------------------------------------------
#
# The physical visualization graph is undirected.
#
# The optimization results are directional.
#
# Therefore, create a dictionary such as:
#
#   ("A", "C") -> {
#       flow: 24,
#       capacity: 25,
#       utilization: 96
#   }
#
# This makes it easy to retrieve both directions
# of every physical connection.

flow_lookup = {}

for _, row in (
    link_results_df.iterrows()
):

    flow_lookup[
        (
            row["from_node"],
            row["to_node"]
        )
    ] = {
        "flow":
            float(
                row["flow_mbps"]
            ),
        "capacity":
            float(
                row["capacity_mbps"]
            ),
```

```
            "utilization":
                float(
                    row[
                        "utilization_percentage"
                    ]
                )
        }


    # ------------------------------------------------------
    # Calculate total traffic carried by each physical link
    # ------------------------------------------------------
    #
    # A physical connection such as A–C may carry:
    #
    #   A -> C traffic
    #   C -> A traffic
    #
    # The optimization constrains those directions separately.
    #
    # For visualization only, the two directional flows are
    # added together to determine physical line thickness.

    physical_flow = {}

    for u, v in graph.edges():

        # Retrieve flow in the forward direction.
        # If no traffic exists, use zero.

        uv_flow = (
            flow_lookup.get(
                (u, v),
                {
                    "flow": 0
                }
            )[
                "flow"
            ]
        )


        # Retrieve traffic in the reverse direction.

        vu_flow = (
            flow_lookup.get(
                (v, u),
                {
                    "flow": 0
                }
            )[
                "flow"
```

```
            ]
        )


        # Total physical traffic controls line thickness.

        physical_flow[
            (u, v)
        ] = (
            uv_flow
            + vu_flow
        )


# ------------------------------------------------------------
# Determine the largest observed physical-link flow
# ------------------------------------------------------------
#
# This is used to scale line widths so that high-flow
# corridors appear visually thicker.

max_flow = max(
    physical_flow.values(),
    default=1
)


# Avoid division by zero if a future scenario contains
# no active traffic.

if max_flow <= 0:
    max_flow = 1


# ------------------------------------------------------------
# Scale network edge widths
# ------------------------------------------------------------

edge_widths = [
    1
    + 7
    * physical_flow[
        (u, v)
    ]
    / max_flow

    for u, v
    in graph.edges()
]


# ------------------------------------------------------------
```

```
# Draw physical network links
# ------------------------------------------------------

nx.draw_networkx_edges(
    graph,
    pos=pos,
    width=edge_widths
)


# ------------------------------------------------------
# Build directional traffic labels
# ------------------------------------------------------
#
# Each physical connection may display:
#
#   A->C: 24.0/25.0 (96%)
#
# and, if reverse flow also exists:
#
#   C->A: 5.0/25.0 (20%)

edge_labels = {}


for u, v in graph.edges():

    label_parts = []


    # Retrieve u -> v directional result.

    uv = flow_lookup.get(
        (u, v)
    )


    # Retrieve v -> u directional result.

    vu = flow_lookup.get(
        (v, u)
    )


    # ------------------------------------------------
    # Add forward-direction label when positive traffic exists
    # ------------------------------------------------

    if (
        uv is not None
        and uv[
            "flow"
```

```
            ] > 1e-8
        ):

            label_parts.append(
                f"{u}->{v}: "
                f"{uv['flow']:.1f}/"
                f"{uv['capacity']:.1f} "
                f"({uv['utilization']:.0f}%)"
            )


        # ------------------------------------------------------
        # Add reverse-direction label when positive traffic exists
        # ------------------------------------------------------

        if (
            vu is not None
            and vu[
                "flow"
            ] > 1e-8
        ):

            label_parts.append(
                f"{v}->{u}: "
                f"{vu['flow']:.1f}/"
                f"{vu['capacity']:.1f} "
                f"({vu['utilization']:.0f}%)"
            )


        # Store the completed label only if one or both
        # directions carry positive traffic.

        if label_parts:

            edge_labels[
                (u, v)
            ] = "\n".join(
                label_parts
            )


    # ----------------------------------------------------------
    # Add directional flow labels to the physical links
    # ----------------------------------------------------------

    nx.draw_networkx_edge_labels(
        graph,
        pos=pos,
        edge_labels=edge_labels,
        font_size=7
    )
```

```
    # ------------------------------------------------------------
    # Finish the figure
    # ------------------------------------------------------------

    plt.title(
        title,
        fontsize=14
    )

    plt.axis(
        "off"
    )

    plt.tight_layout()

    plt.show()
```

33.15.2 Draw the Splittable Stage 1 Network

In [260…

```
# ------------------------------------------------------------
# Visualize the optimized Splittable Stage 1 network
# ------------------------------------------------------------
#
# Inputs:
#
#   G_synthetic
#       Physical network created in Section 26.
#
#   split_stage1_link_df
#       Directed optimized link flows calculated in Section 33.11.
#
#   pos
#       Fixed node positions used throughout the notebook.
#
# The figure therefore combines:
#   topology + capacity + optimized traffic.

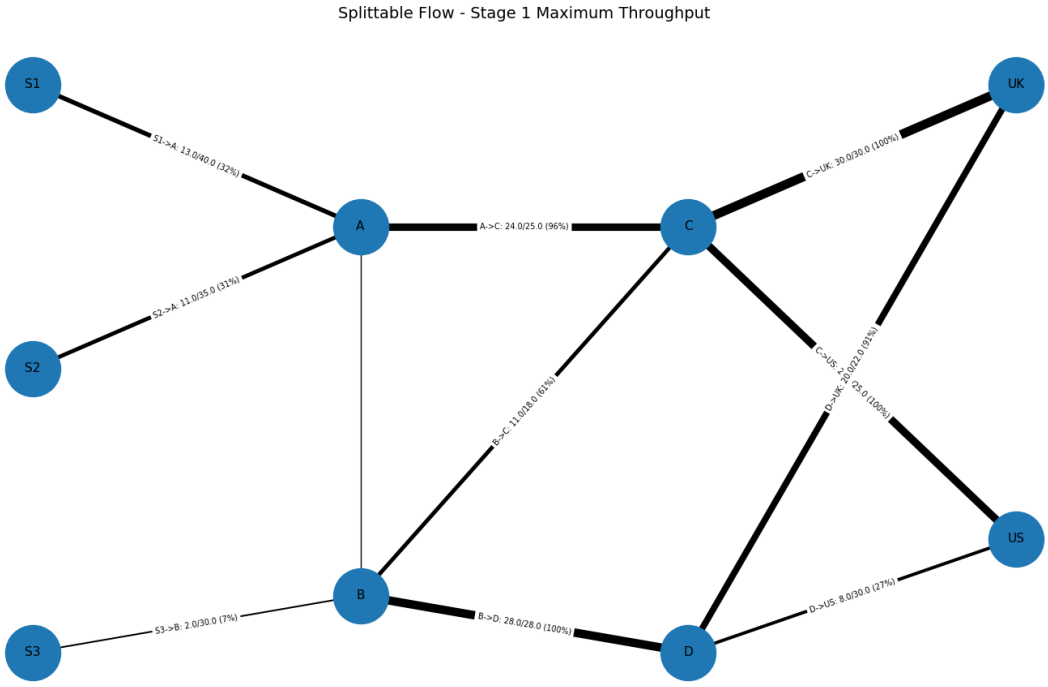
plot_network_flow(
    graph=
        G_synthetic,

    link_results_df=
        split_stage1_link_df,

    pos=
        pos,

    title=(
        "Splittable Flow – "
```

```
        "Stage 1 Maximum Throughput"
    )
)
```



33.15.3 What the Stage 1 Network Visualization Shows

The figure should show optimized traffic moving primarily in the forward direction from the source nodes toward UK and US.

The most important Stage 1 corridors should be visually apparent through thicker links and high-utilization labels.

Based on the Stage 1 results calculated previously, several important patterns should appear.

B → D

The link:

$$B \to D$$

should show:

$$28/28 \text{ Mbps}$$

or:

$$100\%$$

utilization.

This is one of the most important changes relative to Section 30.

Under naive minimum-latency routing, B → D carried no traffic because B → C was faster.

The maximum-throughput optimizer deliberately activates B → D because the D corridor provides additional end-to-end capacity.

C → UK

The UK exit link through C should show:

$$30/30 \text{ Mbps}$$

or:

$$100\%$$

utilization.

C → US

Similarly:

$$C \to US$$

should show:

$$25/25 \text{ Mbps}$$

or:

$$100\%$$

utilization.

A → C

The A → C corridor should be close to saturation:

$$24/25 \text{ Mbps}$$

or approximately:

$$96\%$$

utilization.

D → UK

D → UK should also carry substantial traffic:

$$20/22 \text{ Mbps}$$

or approximately:

$$90.9\%$$

utilization.

Source-Entry Links

The source-entry links should appear much less constrained.

For example:

$$S1 \to A$$

carries only approximately:

$$13/40 \text{ Mbps}.$$

Similarly:

$$S2 \to A$$

and:

$$S3 \to B$$

retain substantial unused capacity.

33.15.4 Interpretation of the Stage 1 Network Diagram

The Stage 1 network visualization provides one of the clearest demonstrations of what the optimizer is doing.

1. Stage 1 Uses Both Major Exit Corridors

The naive minimum-latency policy from Section 30 concentrated traffic almost entirely around C.

That created major overloads on:

- A → C;
- B → C;
- C → UK;
- C → US.

Stage 1 behaves differently.

The optimizer activates the D corridor and uses:

$$B \to D$$

to its full:

$$28\ \text{Mbps}$$

capacity.

This spreads traffic across a larger portion of the network.

2. The Optimizer Accepts Higher Latency to Gain Throughput

The D routes are generally slower than comparable C routes.

However, Stage 1 contains no latency penalty.

Therefore, when the slower D corridor permits more demand to be delivered, the optimizer is free to use it.

This is exactly the intended Stage 1 behavior.

The optimization hierarchy is:

1. maximize throughput first;
2. optimize sensitive latency later.

3. Exit Capacity Remains Highly Constrained

Even after traffic is distributed across both C and D, the C exit links remain fully utilized.

This shows that the network does not suffer from only one badly chosen route.

There is a genuine end-to-end capacity limitation.

4. Source Capacity Is Not the Main Problem

The visualization also reinforces an earlier finding.

The source-entry links remain well below capacity.

The congestion emerges after traffic from different sources converges deeper in the network.

Therefore, adding capacity to source-entry links would not necessarily improve total throughput.

5. Network Topology Matters More Than Total Installed Bandwidth

Some links still contain unused capacity while 43 Mbps of demand is lost.

That is possible because network bandwidth cannot be pooled arbitrarily.

Additional traffic requires an entire feasible source-to-exit path with sufficient capacity on every required directed arc.

The network diagram makes this structural limitation easier to see than a table alone.

33.15.5 Business and Operational Impact of the Stage 1 Network Flow

The Stage 1 network visualization has several important operational implications.

Existing Infrastructure Can Be Used More Effectively

Section 30 showed that a simple minimum-latency policy ignored the D corridor.

The optimizer instead converts that previously unused network capacity into delivered traffic.

This means routing intelligence can create measurable operational value without immediately adding infrastructure.

Capacity Expansion Should Be Targeted

The visualization also shows that congestion is concentrated.

The network does not need additional bandwidth everywhere.

Adding capacity to lightly utilized source-entry links would likely provide little benefit.

Investment should instead focus on links whose expansion increases actual end-to-end throughput.

That is why later bottleneck analysis will measure:

$$\Delta_e = T^*(C_e + 1) - T^*(C_e)$$

rather than recommending investment based solely on utilization.

Latency and Throughput Require Different Decisions

Stage 1 uses slower routes because doing so increases throughput.

That is acceptable at this stage because latency-sensitive routing is intentionally deferred to Stage 2.

This establishes a useful operational principle:

> **First determine how much traffic can be served. Then optimize how that admitted traffic should travel.**

Network-Wide Optimization Outperforms Local Routing Rules

The diagram also makes clear why individual shortest-path decisions are insufficient.

A locally attractive route may consume capacity needed by many other traffic flows.

Network-wide optimization considers those interactions simultaneously.

Business Conclusion

The primary value demonstrated by Stage 1 is therefore not simply that the solver delivers:

$$83 \text{ Mbps}.$$

It is that the optimizer identifies **how the network's available corridors should work together** to achieve that throughput while respecting every directional

capacity constraint.

33.15.6 Naive vs. Optimized Link Loading

The Section 30 naive minimum-latency policy and the Stage 1 optimized solution can now be compared at the directed-link level.

The comparison answers:

> **Where did the optimizer move traffic in order to convert an infeasible shortest-path routing pattern into the maximum feasible 83 Mbps solution?**

The most important expected change is traffic movement away from overloaded C-side corridors and toward the previously unused D corridor.

In [261…

```
# ------------------------------------------------------------
# Compare naive minimum-latency loading with optimized Stage 1
# ------------------------------------------------------------

# Keep the link identifiers and naive attempted flow.

naive_vs_stage1_df = (
    naive_min_latency_link_df[
        [
            "from_node",
            "to_node",
            "capacity_mbps",
            "naive_load_mbps"
        ]
    ]
    .merge(
        split_stage1_link_df[
            [
                "from_node",
                "to_node",
                "flow_mbps"
            ]
        ],
        on=[
            "from_node",
            "to_node"
        ],
        how="left"
    )
)


# Rename the optimized-flow column for clarity.
```

```
naive_vs_stage1_df = (
    naive_vs_stage1_df.rename(
        columns={
            "flow_mbps":
                "optimized_stage1_flow_mbps"
        }
    )
)


# Replace missing optimized values with zero.

naive_vs_stage1_df[
    "optimized_stage1_flow_mbps"
] = (
    naive_vs_stage1_df[
        "optimized_stage1_flow_mbps"
    ].fillna(0)
)


# Calculate how much traffic changed on each link.

naive_vs_stage1_df[
    "flow_change_mbps"
] = (
    naive_vs_stage1_df[
        "optimized_stage1_flow_mbps"
    ]
    - naive_vs_stage1_df[
        "naive_load_mbps"
    ]
)


# Create readable labels.

naive_vs_stage1_df[
    "link"
] = (
    naive_vs_stage1_df[
        "from_node"
    ]
    + " → "
    + naive_vs_stage1_df[
        "to_node"
    ]
)


# Keep only links used by at least one of the two policies.
```

```
naive_vs_stage1_active_df = (
    naive_vs_stage1_df[
        (
            naive_vs_stage1_df[
                "naive_load_mbps"
            ] > 1e-8
        )
        |
        (
            naive_vs_stage1_df[
                "optimized_stage1_flow_mbps"
            ] > 1e-8
        )
    ]
    .copy()
)


# Display the comparison table.

naive_vs_stage1_active_df[
    [
        "link",
        "capacity_mbps",
        "naive_load_mbps",
        "optimized_stage1_flow_mbps",
        "flow_change_mbps"
    ]
]
```

Out [261…

	link	capacity_mbps	naive_load_mbps	optimized_stage1_flow_mbps	flow_
0	B → C	18	61.0	11.0	
1	C → US	25	61.0	25.0	
2	C → UK	30	65.0	30.0	
3	A → C	25	45.0	24.0	
4	S3 → B	30	24.0	2.0	
5	S1 → A	40	24.0	13.0	
6	S2 → A	35	21.0	11.0	
14	B → D	28	0.0	28.0	
18	D → UK	22	0.0	20.0	
20	D → US	30	0.0	8.0	

In [262…

```
# ------------------------------------------------------------
# Visual comparison of naive and optimized link loads
# ------------------------------------------------------------

comparison_plot_df = (
    naive_vs_stage1_active_df
    .set_index(
        "link"
    )[
        [
            "naive_load_mbps",
            "optimized_stage1_flow_mbps"
```

```
        ]
    ]
)


comparison_plot_df.plot(
    kind="bar",
    figsize=(13, 6)
)


plt.ylabel(
    "Traffic (Mbps)"
)

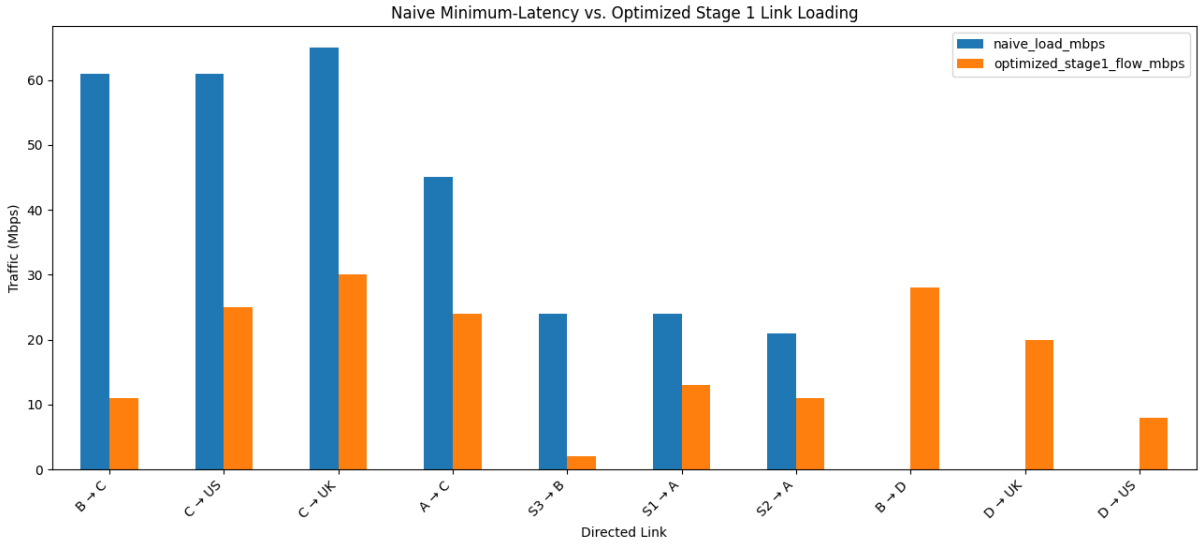
plt.xlabel(
    "Directed Link"
)

plt.title(
    "Naive Minimum-Latency vs. "
    "Optimized Stage 1 Link Loading"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()

plt.show()
```



33.15.6 Detailed Interpretation of Naive vs. Optimized Link Loading

The link-level comparison provides direct evidence of how the Stage 1 optimizer transforms the infeasible minimum-latency routing pattern into a feasible maximum-throughput solution.

The changes are substantial rather than incremental.

1. The Optimizer Dramatically Reduces Pressure on B → C

Under naive minimum-latency routing:

$$B \to C = 61 \text{ Mbps}$$

was attempted on a link with only:

$$18 \text{ Mbps}$$

of capacity.

Stage 1 reduces traffic on B → C to:

$$11 \text{ Mbps}.$$

The change is:

$$11 - 61 = -50 \text{ Mbps}.$$

This is the largest individual reduction in the comparison.

The optimizer is therefore actively moving traffic away from one of the most severely overloaded links identified in Section 30.

This confirms that B → C was attractive from a latency perspective but could not support the combined traffic that wanted to use it.

---

2. The D Corridor Absorbs Traffic Removed from B → C

The largest positive routing change occurs on:

$$B \to D.$$

Naive routing used:

$$0 \text{ Mbps}$$

because the D route was slower.

Stage 1 increases B → D traffic to:

$$28 \text{ Mbps},$$

which is exactly the full capacity of the link.

Therefore:

$$\Delta F_{B \to D} = +28 \text{ Mbps}.$$

That traffic then continues through:

$$D \to UK = 20 \text{ Mbps}$$

and:

$$D \to US = 8 \text{ Mbps}.$$

This provides a very clear explanation of what the optimizer is doing.

Traffic that could not continue through the preferred B → C corridor is redirected through D.

The entire 28 Mbps capacity of B → D is used.

This confirms the Section 30 hypothesis that the D corridor would become operationally important once network-wide capacity was considered.

3. The Optimizer Reduces Attempted Exit-Link Overload

The naive minimum-latency policy attempted to place:

$$65 \text{ Mbps}$$

on:

$$C \to UK$$

despite only:

$$30 \text{ Mbps}$$

of capacity.

Stage 1 reduces this traffic to exactly:

$$30 \text{ Mbps}.$$

The change is:

$$30 - 65 = -35 \text{ Mbps}.$$

Similarly, C → US falls from:

$$61 \text{ Mbps}$$

to:

$$25 \text{ Mbps},$$

which is exactly its capacity.

The change is:

$$25 - 61 = -36 \text{ Mbps}.$$

These results show that Stage 1 converts impossible attempted exit loading into physically feasible loading.

Both C-side exit links remain fully utilized, which indicates that they are valuable network resources even after routing is optimized.

4. Pressure on A → C Is Reduced but Not Eliminated

Under naive minimum-latency routing:

$$A \to C = 45 \text{ Mbps}$$

was attempted on a link with capacity:

$$25 \text{ Mbps}.$$

Stage 1 reduces the optimized flow to:

$$24 \text{ Mbps}.$$

The change is:

$$24 - 45 = -21 \text{ Mbps}.$$

Therefore, A → C moves from severe overload to:

$$\frac{24}{25} \times 100 = 96\%$$

utilization.

This is an important distinction.

The optimizer does not abandon A → C.

Instead, it uses almost all of the available capacity while ensuring that the physical limit is respected.

This is exactly what a throughput-maximizing traffic-engineering solution should do with a valuable corridor.

5. Lower Source-Link Traffic Reflects Admission Decisions, Not Rerouting Alone

Several source-entry links also carry less traffic after optimization.

For example:

S1 → A

Naive attempted traffic:

$$24 \text{ Mbps}$$

Optimized Stage 1 traffic:

$$13 \text{ Mbps}$$

Change:

$$-11 \text{ Mbps}$$

S2 → A

Naive attempted traffic:

$$21 \text{ Mbps}$$

Optimized traffic:

$$11 \text{ Mbps}$$

Change:

$$-10 \text{ Mbps}$$

S3 → B

Naive attempted traffic:

$$24 \text{ Mbps}$$

Optimized traffic:

$$2 \text{ Mbps}$$

Change:

$$-22 \text{ Mbps}.$$

These reductions cannot be explained only by rerouting.

Every feasible path from S1 must first use:

$$S1 \to A.$$

Every path from S2 must first use:

$$S2 \to A.$$

Every path from S3 must first use:

$$S3 \to B.$$

Therefore, reduced flow on these source-entry links means that the corresponding source traffic was **not admitted into the network**.

This directly connects the link-level results to the delivery results from Sections 33.1 and 33.7.

For example, S3 → B falls from an attempted 24 Mbps to only 2 Mbps because Stage 1 admits only 2 Mbps of S3 demand.

6. Stage 1 Resolves Congestion Through Two Different Mechanisms

The comparison reveals that Stage 1 handles the infeasible naive routing pattern through two distinct mechanisms.

Mechanism 1 — Rerouting

Traffic is moved away from overloaded low-latency corridors and toward available alternative paths.

The clearest example is:

$$B \to C \downarrow$$

while:

$$B \to D \uparrow$$

This represents **traffic engineering**.

Mechanism 2 — Admission Reduction

Some offered traffic is not admitted because even the best network-wide route allocation cannot accommodate all 126 Mbps.

This is visible in reductions on mandatory source-entry links such as:

$$S3 \to B : 24 \to 2.$$

This represents **capacity limitation**.

Therefore, the Stage 1 solution should not be described only as rerouting traffic.

It is simultaneously:

1. redistributing admitted traffic;
2. dropping traffic that cannot be supported end to end.

7. The Optimized Link Pattern Explains the 83 Mbps Throughput Ceiling

The comparison makes the Stage 1 objective value easier to interpret.

The naive policy attempts to send the full:

$$126 \text{ Mbps},$$

but does so using impossible link loads.

Stage 1 reduces those link loads to feasible values and uses alternative capacity through D.

After all feasible rerouting opportunities are exploited, the network supports:

$$83 \text{ Mbps}.$$

The remaining:

$$43 \text{ Mbps}$$

cannot be accommodated by any feasible combination of the current candidate paths and directional capacities.

Therefore, the 83 Mbps result is not simply caused by the optimizer choosing to drop traffic.

It represents the maximum feasible traffic after:

- rerouting;
- capacity sharing;
- alternate-corridor use;
- admission control.

8. Business and Operational Impact of the Naive-vs-Optimized Comparison

The before-and-after link comparison reveals three different types of operational opportunity.

1. Routing Opportunity

The D corridor was unused under the naive policy but carries:

$$28 \text{ Mbps}$$

under optimization.

This means better routing can unlock value from infrastructure that already exists.

No physical capacity expansion was required to obtain that benefit.

2. Capacity Limitation

Even after activating alternative routes, the network still cannot carry all demand.

The reduction in mandatory source-entry traffic demonstrates that some offered demand must be rejected.

This indicates a genuine end-to-end capacity limitation rather than merely poor routing.

3. Targeted Capacity Planning

The comparison narrows the search for valuable capacity investments.

The business should not automatically increase capacity on every busy link.

Instead, later sensitivity analysis should determine where:

$$+1 \text{ Mbps capacity}$$

produces:

$$+1 \text{ Mbps or more delivered throughput.}$$

This is a more economically meaningful infrastructure metric than utilization alone.

Operational Conclusion

The optimized solution creates value in two steps:

1. use currently available capacity more intelligently;
2. identify the remaining structural constraints after routing improvements are exhausted.

This provides a much stronger basis for network investment decisions than either shortest-path routing or utilization analysis alone.

33.15.7 Admission vs. Rerouting Analysis

The naive-versus-optimized link comparison shows large reductions in traffic on several directed arcs.

However, those reductions should not automatically be interpreted as lost demand.

A decrease in link traffic can occur for two fundamentally different reasons. A decrease in link flow is not always lost demand.

1. **Rerouting** — traffic is still delivered, but it moves to a different path.
2. **Admission reduction** — some offered demand is not admitted because the network cannot carry it end to end.

This distinction is especially clear for source nodes S1, S2, and S3.

Each of these sources has exactly one directed link through which all of its traffic must enter the network:

$$S1 \to A$$

$$S2 \to A$$

$$S3 \to B$$

Therefore, the optimized flow on each of these mandatory source-entry arcs is equal to the total amount of traffic admitted from that source.

For these three sources:

$$\text{Not Admitted} = \text{Offered Demand} - \text{Optimized Source-Entry Flow}$$

This provides a clean way to separate source-level admission loss from internal rerouting behavior.

33.15.7.1 Build the Upstream Admission Table

In [263…

```
# ------------------------------------------------------------
# Identify the mandatory source-entry arcs
# ------------------------------------------------------------
#
# For S1, S2, and S3, every feasible path must begin with
# exactly one directed source-entry link.
#
# Therefore:
#
#   optimized flow on source-entry link
#   =
#   total traffic admitted from that source
#
# This relationship does NOT apply in the same simple way
# to B and C because B and C are themselves internal/source
# nodes with multiple possible outgoing arcs.

mandatory_source_entry_arcs = {
    "S1": ("S1", "A"),
    "S2": ("S2", "A"),
    "S3": ("S3", "B")
}


# ------------------------------------------------------------
# Create offered-demand lookup by source
# ------------------------------------------------------------
#
# This uses the source-level demand summary derived earlier
# rather than manually entering the demand values.

source_demand_lookup = (
    demand_df
```

```
    .groupby(
        "source"
    )[
        "demand_mbps"
    ]
    .sum()
    .to_dict()
)


# ------------------------------------------------------------
# Create optimized directed-link flow lookup
# ------------------------------------------------------------
#
# The key is:
#
#   (from_node, to_node)
#
# and the value is the optimized Stage 1 flow on that arc.

stage1_link_flow_lookup = {
    (
        row["from_node"],
        row["to_node"]
    ):
        float(
            row["flow_mbps"]
        )

    for _, row
    in split_stage1_link_df.iterrows()
}


# ------------------------------------------------------------
# Calculate admitted and non-admitted traffic
# for S1, S2, and S3
# ------------------------------------------------------------

upstream_admission_records = []


for source, entry_arc in (
    mandatory_source_entry_arcs.items()
):

    # Total traffic offered by this source.

    offered_demand = float(
        source_demand_lookup[
            source
        ]
```

```
    )


    # Optimized traffic actually entering the network.
    #
    # Because this is the only possible source-entry arc,
    # this value equals total admitted demand from the source.

    admitted_flow = float(
        stage1_link_flow_lookup.get(
            entry_arc,
            0.0
        )
    )


    # Demand that never enters the network.

    not_admitted = (
        offered_demand
        - admitted_flow
    )


    # Calculate percentage of source demand admitted.

    admission_percentage = (
        100
        * admitted_flow
        / offered_demand
        if offered_demand > 0
        else 0
    )


    # Store result.

    upstream_admission_records.append({
        "source":
            source,
        "entry_arc":
            f"{entry_arc[0]} → {entry_arc[1]}",
        "offered_demand_mbps":
            offered_demand,
        "admitted_flow_mbps":
            admitted_flow,
        "not_admitted_mbps":
            not_admitted,
        "admission_percentage":
            admission_percentage
    })
```

```
# Convert results into a DataFrame.

upstream_admission_df = (
    pd.DataFrame(
        upstream_admission_records
    )
)


upstream_admission_df
```

Out [263…

	source	entry_arc	offered_demand_mbps	admitted_flow_mbps	not_admitte
0	S1	S1 → A	24.0	13.0	
1	S2	S2 → A	21.0	11.0	
2	S3	S3 → B	24.0	2.0	

33.15.7.2 Validate the Admission Calculation

In [264…

```
# ------------------------------------------------------------
# Validate source-entry admission against source-level delivery
# ------------------------------------------------------------
#
# Because all traffic entering from S1, S2, and S3 eventually
# reaches its required exit in the optimized feasible solution,
# admitted source flow should equal total delivered traffic
# reported in the source-level delivery table.

source_delivery_lookup = (
    split_stage1_source_summary
    .set_index(
        "source"
    )[
        "delivered_mbps"
    ]
    .to_dict()
)


for _, row in (
    upstream_admission_df.iterrows()
):

    source = (
        row["source"]
    )
```

```
    admitted_flow = (
        row["admitted_flow_mbps"]
    )

    delivered_flow = float(
        source_delivery_lookup[
            source
        ]
    )


    # Confirm the link-based admission calculation
    # matches the independently calculated source-level delivery.

    assert abs(
        admitted_flow
        - delivered_flow
    ) < 1e-6


print(
    "Mandatory source-entry admission validation passed."
)
```

```
Mandatory source-entry admission validation passed.
```

33.15.7.3 Interpretation of Upstream Admission

The upstream admission table provides a more precise interpretation of the large flow reductions seen in the naive-versus-optimized link comparison.

S1

S1 offers:

$$24 \text{ Mbps}$$

but only:

$$13 \text{ Mbps}$$

enters the optimized network.

Therefore:

$$11 \text{ Mbps}$$

is not admitted.

The admission rate is:

$$\frac{13}{24} \times 100 \approx 54.17\%$$

S2

S2 offers:

$$21 \text{ Mbps}$$

while:

$$11 \text{ Mbps}$$

is admitted.

Therefore:

$$10 \text{ Mbps}$$

is not admitted.

The admission rate is:

$$\frac{11}{21} \times 100 \approx 52.38\%$$

S3

S3 experiences the largest admission reduction.

It offers:

$$24 \text{ Mbps}$$

but only:

$$2 \text{ Mbps}$$

enters the network.

Therefore:

$$22 \text{ Mbps}$$

is not admitted.

The admission rate is only:

$$\frac{2}{24} \times 100 \approx 8.33\%$$

Key Interpretation

These source-entry reductions are fundamentally different from internal-link reductions.

For S1, S2, and S3, traffic cannot bypass their mandatory entry arc.

Therefore, if optimized flow on that arc decreases, the missing traffic was not rerouted around the entry link.

It was not admitted into the network at all.

33.15.7.4 Why B and C Require a Different Interpretation

The same source-entry logic cannot be applied directly to B and C.

B and C are both traffic-generating nodes, but they are also internal routing nodes.

Their locally generated traffic may leave through several outgoing arcs.

For example, B-originating traffic may use:

$$B \to C$$

or:

$$B \to D$$

and may also use other feasible routes depending on the destination.

Therefore, there is no single mandatory B source-entry arc whose flow equals total admitted B demand.

The same is true for C.

For B and C, source-level admitted demand should therefore be taken from the source-summary table rather than inferred from one directed link.

From Section 33.7:

$$B : 37/37 = 100\%$$

of offered demand is delivered.

Similarly:

$$C : 20/20 = 100\%$$

of offered demand is delivered.

This distinction prevents internal routing flows from being incorrectly interpreted as source-level admission.

33.15.7.5 Admission vs. Rerouting

The naive-versus-optimized link comparison contains two fundamentally different types of flow change.

Type 1 — Admission Reduction

At mandatory source-entry arcs:

$$S1 \to A$$
$$S2 \to A$$
$$S3 \to B$$

a reduction in flow means that some offered source demand was not admitted.

For example:

$$S3 \to B : 24 \to 2 \text{ Mbps}$$

cannot be explained by rerouting.

There is no alternative path by which S3 traffic can enter the network.

Therefore:

$$22 \text{ Mbps}$$

of S3 demand is not admitted.

Type 2 — Internal Rerouting

For internal arcs, a reduction does not necessarily represent lost demand.

For example:

$$B \to C : 61 \to 11 \text{ Mbps}$$

falls by:

$$50 \text{ Mbps}.$$

However, that does **not** mean 50 Mbps of B-side traffic was lost.

Some of that reduction occurs because traffic is redirected through:

$$B \to D.$$

B → D increases from:

$$0$$

to:

$$28 \text{ Mbps}.$$

That is rerouting rather than demand loss.

Type 3 — Combined Effects

Some internal-link reductions may reflect both:

- reduced upstream admission;
- rerouting to alternate corridors.

For example, A → C decreases from:

$$45$$

attempted Mbps under the naive policy to:

$$24$$

optimized Mbps.

Part of that reduction occurs because S1 and S2 traffic is not fully admitted.

Some traffic may also use different internal routes.

Therefore, internal-link flow changes should not be interpreted as lost demand without tracing the complete route structure.

33.15.7.6 Source Admission Visualization

The following chart compares offered demand with admitted Stage 1 traffic for the three sources that have a single mandatory entry arc.

This isolates admission loss from internal rerouting effects.

In [265…

```
# ------------------------------------------------------------
# Visualize offered vs admitted demand for mandatory-entry sources
# ------------------------------------------------------------

admission_plot_df = (
    upstream_admission_df
    .set_index(
        "source"
    )[
        [
            "admitted_flow_mbps",
            "not_admitted_mbps"
        ]
    ]
)


# Draw admitted and non-admitted traffic as stacked bars.

admission_plot_df.plot(
    kind="bar",
    stacked=True,
    figsize=(8, 5)
)


plt.ylabel(
    "Traffic (Mbps)"
)

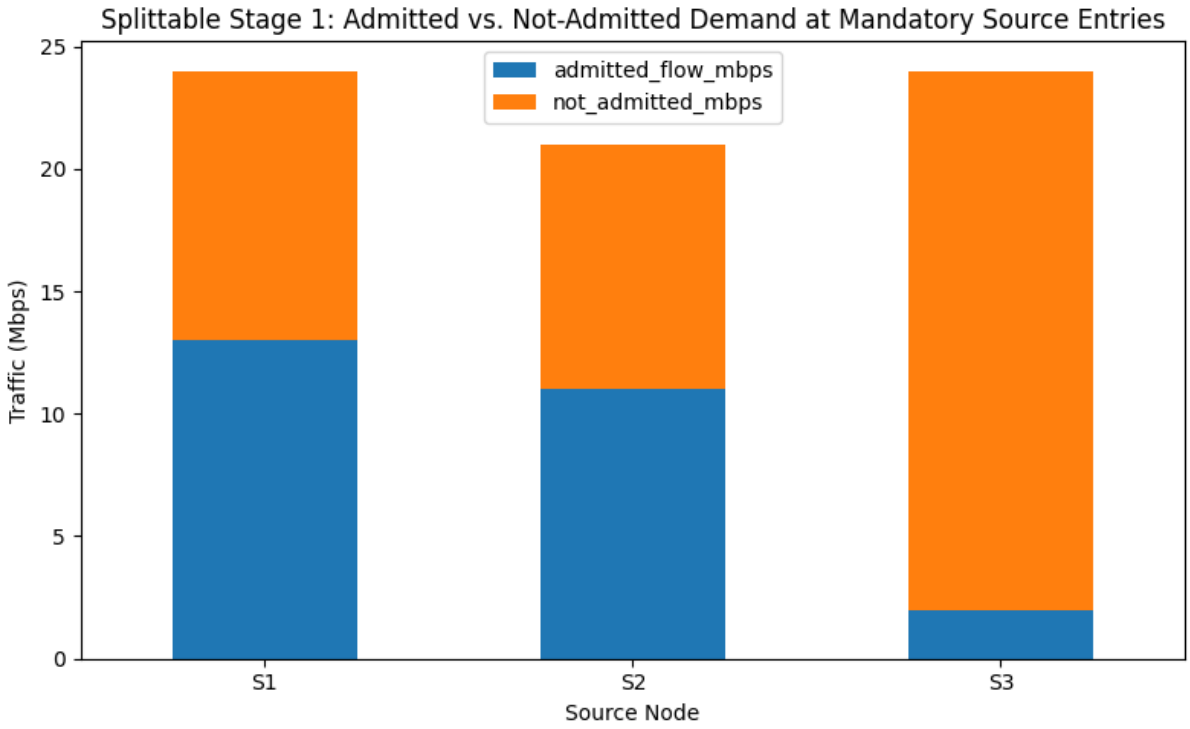
plt.xlabel(
    "Source Node"
)

plt.title(
    "Splittable Stage 1: "
    "Admitted vs. Not-Admitted Demand "
    "at Mandatory Source Entries"
)

plt.xticks(
    rotation=0
)

plt.tight_layout()

plt.show()
```



33.15.7.7 Interpretation of the Admission Visualization

The source-admission chart shows that congestion does not affect the three upstream sources equally.

S1 and S2 admit slightly more than half of their offered demand.

S3 admits only:

$$2/24 \approx 8.33\%.$$

This provides a direct visual explanation for the reduced traffic seen on the source-entry links in the naive-versus-optimized comparison.

The chart also reinforces that the optimizer resolves congestion through more than route selection.

When the network cannot physically accommodate all demand, Stage 1 also performs implicit admission control by leaving some demand undelivered.

33.15.7.8 Business and Operational Impact of Admission vs. Rerouting

Separating admission reduction from rerouting is important for both operations and capacity planning.

If traffic is rerouted, the network is still serving the demand but using a different corridor.

If traffic is not admitted, the business experiences an actual service shortfall.

Those two outcomes require different operational responses.

Rerouting Problem

If traffic can be delivered by moving it to another corridor, the solution may involve:

- traffic engineering;
- dynamic path selection;
- multipath routing;
- load balancing.

Admission Problem

If traffic cannot be admitted even after routing is optimized, the network may require:

- additional capacity;
- service prioritization;
- demand shaping;
- customer-level admission policies.

The Stage 1 results show that both mechanisms are present.

The optimizer improves network use by activating alternate routes such as the D corridor.

At the same time, it must reject:

$$43\ \text{Mbps}$$

of total demand because the network still lacks sufficient end-to-end capacity.

This distinction is important because it separates:

> **traffic that can be saved through better routing**

from:

> **traffic that requires additional effective capacity or a different business policy.**

33.15.8 How Stage 1 Resolves the Naive Congestion Pattern

The combined results from Sections 30 and 33 reveal that Stage 1 resolves congestion through two simultaneous mechanisms.

Mechanism 1 — Rerouting Admitted Traffic

The optimizer redirects traffic away from overloaded low-latency routes.

The clearest example is:

$$B \to C : 61 \to 11 \text{ Mbps}$$

while:

$$B \to D : 0 \to 28 \text{ Mbps}.$$

This activates network capacity that the minimum-latency routing policy did not use.

Mechanism 2 — Reducing Admission

Rerouting alone is insufficient to accommodate all:

$$126 \text{ Mbps}$$

of offered traffic.

Therefore, some demand never enters the optimized network.

For the mandatory upstream sources:

$$S1 : 24 \to 13 \text{ Mbps admitted}$$

$$S2 : 21 \to 11 \text{ Mbps admitted}$$

$$S3 : 24 \to 2 \text{ Mbps admitted}.$$

This confirms that Stage 1 is simultaneously performing:

- network-wide rerouting;
- implicit admission control.

Combined Outcome

The naive policy attempts:

$$126 \text{ Mbps}$$

but violates multiple capacity constraints.

The optimizer instead finds the largest feasible operating point:

$$\boxed{83 \text{ Mbps}}$$

while satisfying all demand and directional-capacity constraints.

Therefore, the maximum-throughput solution is not merely a better route map.

It is a complete feasible traffic-allocation decision.

33.15.9 Why Admission vs. Rerouting Matters for Stage 2

The distinction between admission and routing becomes especially important in the next optimization stage.

Stage 1 has now made two types of decisions:

1. how much traffic from each source-stream commodity is admitted;
2. which paths carry that admitted traffic.

Stage 2 is intentionally allowed to change only the second decision.

The exact Stage 1 delivered quantity for every commodity will be preserved:

$$z_{k,s}^{(2)} = z_{k,s}^{(1)}.$$

Therefore, Stage 2 may reroute admitted traffic to reduce latency for UK-L and US-L, but it may not:

- admit previously rejected traffic;
- reduce currently admitted traffic;
- replace one admitted commodity with another.

This preserves the lexicographic optimization hierarchy:

Stage 1

$$\text{maximize throughput}$$

Stage 2

$$\text{minimize sensitive-traffic latency}$$

subject to preserving the Stage 1 admission decision.

This means any routing change observed in Section 34 can be attributed to the latency objective rather than a change in total or commodity-level throughput.

33.20 Detailed Section Summary

Section 33 transforms the Stage 1 objective value from Section 32 into a complete operational picture of the maximum-throughput solution.

Section 32 established that the splittable-flow LP can deliver at most:

$$83 \text{ Mbps}$$

from:

$$126 \text{ Mbps}$$

of offered demand.

Therefore:

$$43 \text{ Mbps}$$

is lost and the overall delivery rate is:

$$\frac{83}{126} \times 100 \approx 65.87\%.$$

Section 33 explains where that service is delivered, where it is lost, which routes are used, and which network resources become constrained.

---

1. Source-Stream Delivery

The 83 Mbps throughput is not distributed evenly across the 20 source-stream commodities.

The Stage 1 solution contains:

- 12 fully delivered commodities;
- 1 partially delivered commodity;
- 7 commodities receiving no service.

This confirms that maximum total throughput does not imply uniform service

quality.

The Stage 1 objective contains no fairness term, minimum-delivery guarantee, or source-priority rule.

Therefore, the observed admission pattern is one throughput-optimal allocation rather than an intentional fairness policy.

---

2. Source-Level Results

Source-level service differs substantially.

B receives:

$$37/37 = 100\%$$

of its offered demand.

C receives:

$$20/20 = 100\%.$$

S1 receives:

$$13/24 \approx 54.17\%.$$

S2 receives:

$$11/21 \approx 52.38\%.$$

S3 receives only:

$$2/24 \approx 8.33\%.$$

This shows that the physical location of demand within the topology materially influences congestion exposure.

However, the exact admission allocation should not be interpreted as uniquely required because the throughput-only LP may have multiple equivalent optimal solutions.

---

3. Traffic-Stream Results

Aggregate stream delivery is:

UK-L

$$16/26 \approx 61.54\%.$$

UK-T

$$34/39 \approx 87.18\%.$$

US-L

$$18/29 \approx 62.07\%.$$

US-T

$$15/32 \approx 46.88\%.$$

UK-T receives the highest aggregate delivery percentage, while US-T receives the lowest.

This does not reflect an explicit Stage 1 traffic-class priority.

All delivered Mbps have equal value during throughput optimization.

---

4. Throughput Accounting

The Stage 1 reporting calculations reconcile exactly:

$$83 + 43 = 126.$$

The commodity-level delivered total also matches the LP objective value.

This confirms that all offered demand is accounted for as either:

- delivered;
- or undelivered.

No reporting discrepancy is present.

---

5. Route Allocation

The optimized route pattern differs substantially from the naive minimum-latency pattern developed in Section 30.

The most important change is activation of the D corridor.

Under minimum-latency routing:

$$B \to D = 0 \text{ Mbps}.$$

Under optimized Stage 1 routing:

$$B \to D = 28 \text{ Mbps}.$$

The link reaches:

$$100\%$$

utilization.

Traffic then continues through:

$$D \to UK = 20 \text{ Mbps}$$

and:

$$D \to US = 8 \text{ Mbps}.$$

This confirms that slower alternative routes can provide significant throughput value.

---

6. Naive vs. Optimized Routing

The link comparison shows several major changes.

B → C

$$61 \to 11 \text{ Mbps}$$

for a reduction of:

$$50 \text{ Mbps}.$$

C → UK

$$65 \to 30 \text{ Mbps}.$$

C → US

$$61 \to 25 \text{ Mbps}.$$

A → C

$$45 \to 24 \text{ Mbps}.$$

Meanwhile, the D side gains:

$$B \to D : 0 \to 28$$
$$D \to UK : 0 \to 20$$

$$D \to US : 0 \to 8.$$

These changes show that Stage 1 does not simply eliminate traffic.

It actively redistributes admitted traffic toward usable alternative capacity.

---

7. Admission vs. Rerouting

One of the most important modeling distinctions in Section 33 is that not every reduction in link flow represents lost demand.

For internal links, lower flow may reflect rerouting.

For mandatory source-entry links, lower flow directly indicates reduced admission.

For S1:

$$24 \text{ Mbps offered} \to 13 \text{ Mbps admitted}.$$

Therefore:

$$11 \text{ Mbps}$$

is not admitted.

For S2:

$$21 \to 11 \text{ Mbps},$$

so:

$$10 \text{ Mbps}$$

is not admitted.

For S3:

$$24 \to 2 \text{ Mbps},$$

so:

$$22 \text{ Mbps}$$

is not admitted.

This distinction shows that Stage 1 resolves congestion through two mechanisms:

1. **rerouting traffic that can still be served**;

2. **rejecting traffic that cannot be supported end to end**.

This is more precise than describing all link-flow reductions as traffic loss.

---

8. Link Utilization

The optimized link-utilization results identify several highly constrained corridors.

Three directed links are fully saturated:

$$B \to D$$
$$C \to UK$$
$$C \to US.$$

A → C operates at:

$$24/25 = 96\%.$$

D → UK operates at approximately:

$$20/22 = 90.91\%.$$

By contrast, the source-entry links retain substantial residual capacity relative to their physical limits.

This confirms that congestion arises mainly after traffic begins converging on shared internal and exit corridors.

---

9. Residual Capacity

The network still contains unused bandwidth even though:

$$43 \text{ Mbps}$$

of demand remains unserved.

This is not contradictory.

Additional delivered traffic requires a complete source-to-exit path whose every directed arc has sufficient residual capacity.

Unused bandwidth on one link cannot compensate for a saturated arc elsewhere on the required path.

Therefore:

$$\text{unused link capacity} \neq \text{available end-to-end throughput}.$$

This is an important reason why network capacity must be analyzed structurally rather than by summing installed bandwidth.

---

10. Saturation Is Not the Same as Bottleneck Value

The Stage 1 analysis identifies saturated links, but saturation alone does not prove that expanding a link will increase throughput.

The correct marginal-capacity question is:

$$\Delta_e = T^*(C_e + 1) - T^*(C_e).$$

A link is a true throughput bottleneck only if adding capacity increases the optimal end-to-end delivered traffic.

This distinction prevents utilization from being mistaken for investment value.

The marginal-capacity analysis later in the notebook will test this directly.

---

11. Actual Use of Splitting

The formulation permits one commodity to distribute traffic across multiple paths.

However, the Stage 1 baseline optimum does not require any delivered commodity to use multiple positive-flow paths.

Therefore, in this specific baseline scenario:

> allowing splitting provides flexibility, but the optimal extreme-point solution does not actually need to split an admitted commodity.

This should not be generalized to all traffic scenarios.

Later stress tests should examine conditions in which fragmented residual capacity creates a genuine advantage for multipath routing.

---

12. Fairness and Degeneracy

The Stage 1 objective is:

$$\max \text{Total Delivered Mbps}.$$

It does not specify how the throughput-optimal 83 Mbps should be distributed when several admission patterns achieve the same objective.

Therefore, the observed source-stream allocation should not be treated as a hidden business preference.

A production optimization framework could introduce an additional objective such as:

- proportional fairness;
- minimum delivery percentage;
- source-level guarantees;
- application priority.

Those policies are intentionally excluded from the baseline model because they were not part of the assignment requirement.

---

13. Section 30 Hypotheses

The Stage 1 results confirm the primary expectations developed before optimization.

Minimum-latency routing is infeasible

Confirmed.

The naive policy overloads multiple links.

D should become operationally important

Confirmed.

B → D becomes fully utilized at 28 Mbps.

C-side capacity is scarce

Confirmed.

Both C exit links reach full utilization and A → C reaches 96%.

Optimized routing should differ materially from shortest-path routing

Confirmed.

The optimizer both reroutes traffic and reduces admission to produce a feasible

operating point.

---

14. Technical Conclusion

The Stage 1 splittable-flow model behaves consistently with the mathematical formulation.

It:

- respects every commodity-demand constraint;
- respects every directional-capacity constraint;
- routes traffic only over valid candidate paths;
- accounts for all offered demand;
- activates alternate network capacity;
- achieves the proven maximum throughput of 83 Mbps.

The result therefore represents a valid maximum-throughput network operating point.

---

15. Business Conclusion

Section 33 reveals two different operational problems.

Problem 1 — Routing Efficiency

Naive minimum-latency routing uses network capacity poorly by concentrating demand on the C corridor.

Optimization improves utilization by activating alternative routes through D.

This demonstrates that routing intelligence can create value from infrastructure that already exists.

Problem 2 — Structural Capacity Limitation

Even after routing is optimized, only:

$$83/126 \approx 65.87\%$$

of demand can be delivered.

Therefore, routing improvement alone cannot eliminate the service shortfall.

Some combination of:

- targeted capacity expansion;
- traffic prioritization;
- demand shaping;
- or other operational policy

would be required if the business needed to serve more than the current 83 Mbps ceiling.

---

16. Connection to Section 34

Stage 1 has now established the admission vector.

For every source-stream commodity, the model has determined exactly how much traffic will be delivered.

That decision will now be frozen.

Section 34 asks a different question:

> **Can the same admitted traffic be rerouted to reduce latency for UK-L and US-L without sacrificing throughput or changing which commodities were admitted?**

Stage 2 therefore preserves:

$$z_{k,s}^{(2)} = z_{k,s}^{(1)}$$

for every commodity.

Any changes between the Stage 1 and Stage 2 route maps can then be attributed entirely to the latency objective rather than changes in admission.

This completes the maximum-throughput analysis and establishes the correct starting point for the latency-optimization stage.

34. Splittable-Flow LP — Stage 2: Minimize Sensitive-Traffic Latency

Sections 32 and 33 completed the first stage of the splittable-flow optimization.

Stage 1 answered:

> **What is the maximum amount of current demand that the**

> **network can deliver?**

The result was:

$$83 \text{ Mbps}$$

from:

$$126 \text{ Mbps}$$

of offered demand.

Section 33 then showed exactly which source-stream commodities were admitted, which traffic was lost, which routes were used, and where network capacity became constrained.

Stage 2 now addresses a different question:

> **Given the exact Stage 1 admission decisions, can the admitted traffic be rerouted so that latency-sensitive traffic receives better routes without reducing throughput?**

This is a lexicographic optimization strategy.

The priorities are:

Priority 1 — Throughput

First maximize total delivered traffic.

$$\max T$$

Priority 2 — Sensitive-Traffic Latency

After the maximum-throughput admission vector is known, preserve those exact delivered quantities and minimize latency exposure for:

$$UK\text{-}L$$

and:

$$US\text{-}L.$$

This ensures that lower latency is never purchased by sacrificing throughput.

It also ensures that Stage 2 cannot replace one admitted commodity with another merely because the replacement is easier to route.

The Stage 1 admission decision is therefore treated as fixed.

34.1 Why a Second Optimization Stage Is Necessary

Stage 1 intentionally ignored latency.

Its objective was only:

$$\max \sum_{k,s,p} f_{k,s,p}.$$

Therefore, Stage 1 was free to place latency-sensitive traffic on slower routes whenever those routes contributed to maximum throughput.

That behavior was intentional.

If throughput and latency had been combined immediately into one weighted objective, the result would depend on an arbitrary tradeoff coefficient such as:

$$\max \left(\text{Throughput} - \lambda \cdot \text{Latency} \right).$$

The value of:

$$\lambda$$

would determine how much throughput the model was willing to sacrifice for lower latency.

The problem statement does not provide such a tradeoff.

Instead, the model uses a lexicographic strategy:

1. determine the maximum feasible throughput;
2. freeze that admission decision;
3. optimize latency within the throughput-optimal solution space.

This makes the priority structure explicit and avoids introducing an arbitrary throughput-versus-latency weighting parameter.

Interpretation

Stage 2 does not attempt to find a different amount of traffic to serve.

The throughput problem has already been solved.

Instead, Stage 2 searches among routings that preserve the Stage 1 source-stream delivery vector.

The resulting latency improvement can therefore be interpreted as a **pure routing improvement** rather than a consequence of serving less traffic.

Business and Operational Impact

This distinction is valuable operationally.

A business can state:

> "First, we serve the maximum traffic the network can support. Then, among those maximum-throughput solutions, we give the latency-sensitive traffic the best routing available."

That is substantially easier to explain and govern than a weighted objective whose performance depends on an arbitrary tuning coefficient.

34.2 Define the Sensitive Traffic Classes

Only the latency-sensitive traffic classes participate in the Stage 2 objective.

These are:

$$K_L = \{UK\text{-}L, US\text{-}L\}.$$

The tolerant streams:

- UK-T;
- US-T;

remain subject to all capacity and admission constraints, but their latency does not contribute to the Stage 2 objective.

This allows tolerant traffic to move onto longer routes when doing so creates lower-latency capacity for sensitive traffic.

In [266…

```
# ------------------------------------------------------------
# Define the traffic classes whose latency will be minimized
# ------------------------------------------------------------

# Only streams ending in "-L" are latency-sensitive
# in the current problem definition.

sensitive_streams = {
```

```
    "UK-L",
    "US-L"
}


# Display the classes included in the secondary objective.

print(
    "Latency-sensitive streams:",
    sensitive_streams
)
```

```
Latency-sensitive streams: {'US-L', 'UK-L'}
```

Interpretation

The Stage 2 objective distinguishes between traffic classes for the first time.

Stage 1 treated every delivered Mbps equally.

Stage 2 does not change admission priority, but it does distinguish the routing quality provided to admitted traffic.

This implements the problem requirement that UK-L and US-L should receive lower-latency routing without imposing a hard latency cutoff.

Business and Operational Impact

This structure allows the network to protect applications that are more sensitive to delay while still using tolerant workloads as routing flexibility.

In a real system, latency-sensitive traffic might represent:

- interactive applications;
- voice or video;
- control traffic;
- real-time analytics.

Tolerant traffic may be more suitable for longer alternate paths when necessary.

The model captures that distinction without changing the economic value of a delivered Mbps during admission.

34.3 Create the Stage 2 Optimization Model

Stage 2 remains a linear program.

The path-flow variables are still continuous:

$$f_{k,s,p}^{(2)} \geq 0.$$

The optimization direction changes from maximization to minimization because Stage 2 minimizes sensitive-traffic latency exposure.

In [267…
```
# ------------------------------------------------------------
# Create the Stage 2 splittable-flow LP
# ------------------------------------------------------------
#
# Unlike Stage 1, which maximized throughput,
# Stage 2 minimizes the latency exposure of admitted
# latency-sensitive traffic.

split_stage2 = pl.LpProblem(
    "Split_Flow_Stage2",
    pl.LpMinimize
)


print(
    "Model created:",
    split_stage2.name
)

print(
    "Optimization sense:",
    "Minimize"
)
```

```
Model created: Split_Flow_Stage2
Optimization sense: Minimize
```

34.4 Create Stage 2 Path-Flow Variables

A new continuous decision variable is created for every candidate path.

New variables are required because Stage 2 is a separate optimization model.

Although the admitted amount for each commodity is fixed, the distribution of that admitted traffic across paths is allowed to change.

In [272…
```
# ------------------------------------------------------------
# 34.4 Create and Validate Stage 2 Path-Flow Variables
# ------------------------------------------------------------
#
# Stage 2 is a separate optimization problem from Stage 1.
#
```

```
# Therefore, it requires a new set of decision variables.
#
# Each variable represents the amount of Stage 2 traffic,
# measured in Mbps, assigned to one candidate source-stream path.
#
# The variables remain continuous because this is still
# the splittable-flow formulation.
#
# Stage 2 is allowed to change HOW admitted traffic is routed,
# but later constraints will force it to preserve exactly
# HOW MUCH traffic was delivered for each commodity in Stage 1.


# Create an empty dictionary that will map:
#
#     path_id -> Stage 2 PuLP flow variable

split_stage2_vars = {}


# ------------------------------------------------------------
# Create one continuous variable for every candidate path
# ------------------------------------------------------------

for _, row in candidate_paths_df.iterrows():

    # Retrieve the unique candidate-path identifier.
    #
    # Example:
    #   path_id = 0
    #   path_id = 1
    #   ...
    #   path_id = 67

    pid = int(
        row["path_id"]
    )


    # Create a non-negative continuous Stage 2 flow variable.
    #
    # Example variable names:
    #
    #   split_stage2_0
    #   split_stage2_1
    #   ...
    #
    # A value of:
    #
    #   split_stage2_0 = 4.5
    #
    # would mean that 4.5 Mbps is assigned to candidate path 0
```

```
    # in the Stage 2 routing solution.

    split_stage2_vars[
        pid
    ] = pl.LpVariable(
        f"split_stage2_{pid}",
        lowBound=0,
        cat="Continuous"
    )


# ------------------------------------------------------------
# Report how many Stage 2 flow variables were created
# ------------------------------------------------------------

print(
    "Stage 2 path-flow variables created:",
    len(
        split_stage2_vars
    )
)


# ------------------------------------------------------------
# Validation 1:
# There must be exactly one Stage 2 variable
# for every candidate path
# ------------------------------------------------------------

assert (
    len(
        split_stage2_vars
    )
    ==
    len(
        candidate_paths_df
    )
)


# ------------------------------------------------------------
# Validation 2:
# The Stage 2 variable keys should exactly match
# the candidate-path IDs
# ------------------------------------------------------------

assert (
    set(
        split_stage2_vars.keys()
    )
    ==
    set(
```

```
        candidate_paths_df[
            "path_id"
        ].astype(int)
    )
)


# ------------------------------------------------------------
# Validation 3:
# The number of variables should match the 68 candidate paths
# identified in Section 29
# ------------------------------------------------------------

assert (
    len(
        split_stage2_vars
    )
    == 68
)


print(
    "Stage 2 variable validation passed."
)
```

```
Stage 2 path-flow variables created: 68
Stage 2 variable validation passed.
```

34.4.1 Interpretation of Stage 2 Variable Creation

Stage 2 contains:

$$68$$

continuous path-flow decision variables.

This exactly matches the:

$$68$$

candidate source-stream paths generated in Section 29.

Therefore, every route that was available to the Stage 1 splittable model remains available during Stage 2.

This is important because Stage 2 is not being given a smaller or artificially restricted routing space.

Instead, it is allowed to reconsider the complete set of feasible paths while preserving the Stage 1 admission quantities.

The distinction is:

- Stage 1 determines **how much** of each commodity is delivered;
- Stage 2 determines **where that already-admitted traffic should travel**.

The successful validation confirms that the Stage 2 routing model has been initialized correctly before the latency objective and preservation constraints are added.

34.4.2 Why Stage 2 Uses a New Set of Variables

The Stage 1 flow variables already contain the maximum-throughput routing solution.

If those same variable values were simply reused, the routes would remain fixed and Stage 2 would have nothing left to optimize.

Creating new variables allows the solver to find a different path allocation.

For example, Stage 1 may send a sensitive commodity over a slower route because latency was irrelevant to the throughput objective.

Stage 2 may move that same admitted traffic onto a lower-latency route, provided that:

- the exact commodity-level delivered quantity remains unchanged;
- all directed link-capacity constraints remain satisfied.

Therefore, creating a second set of path-flow variables is what gives Stage 2 the freedom to improve routing quality.

34.4.3 Business and Operational Impact

This separation between Stage 1 and Stage 2 decision variables represents a useful operational design.

The network does not reconsider customer admission when optimizing latency.

Instead, it treats the Stage 1 service commitment as fixed and asks whether that commitment can be delivered more efficiently.

Operationally, this means:

> **The amount of service remains stable while the network optimizes the quality of that service.**

This avoids a situation where apparently better latency is achieved simply by dropping difficult-to-route traffic.

34.5 Stage 2 Sensitive-Latency Objective

The Stage 2 objective minimizes traffic-weighted latency exposure.

For latency-sensitive streams:

$$K_L = \{UK\text{-}L, US\text{-}L\},$$

the objective is:

$$\boxed{\min \sum_{k \in K_L} \sum_{s} \sum_{p \in P_{k,s}} L_p f_{k,s,p}}$$

where:

- L_p is path latency;
- $f_{k,s,p}$ is traffic routed on the path.

The quantity:

$$L_p f_{k,s,p}$$

has units of:

$$\text{Mbps} \cdot \text{ms}.$$

This objective minimizes total latency exposure across delivered sensitive traffic.

Because Stage 1 delivery quantities will be fixed, minimizing total latency exposure is equivalent to minimizing the traffic-weighted average latency of the admitted sensitive traffic.

In [273…

```
# ------------------------------------------------------------
# Define Stage 2 latency objective
# ------------------------------------------------------------
#
# Only UK–L and US–L contribute to the objective.
#
# Tolerant traffic remains part of the network optimization
# because it consumes capacity, but its path latency does not
# directly increase the objective value.

split_stage2 += pl.lpSum(
```

```
    # Path latency multiplied by traffic carried on that path
    # gives traffic-weighted latency exposure.

    row[
        "path_latency_ms"
    ]
    * split_stage2_vars[
        int(
            row[
                "path_id"
            ]
        )
    ]

    for _, row
    in candidate_paths_df.iterrows()

    # Include only latency-sensitive traffic classes.

    if row[
        "stream"
    ]
    in sensitive_streams
)


print(
    "Stage 2 sensitive-latency objective added."
)
```

```
Stage 2 sensitive-latency objective added.
```

34.5.1 Why Traffic-Weighted Latency Is Used

Suppose one sensitive path carries:

$$10 \text{ Mbps}$$

at:

$$20 \text{ ms}$$

and another carries:

$$1 \text{ Mbps}$$

at:

$$50 \text{ ms}.$$

Treating the two paths equally would ignore the fact that ten times more traffic experiences the first latency.

The traffic-weighted objective instead evaluates:

$$10(20) + 1(50) = 250 \text{ Mbps} \cdot \text{ms}.$$

This makes the objective responsive to both:

- how much traffic uses a path;
- how much latency that path introduces.

34.5.2 Business and Operational Impact

Traffic-weighted latency focuses optimization effort where it affects the greatest volume of latency-sensitive service.

A routing change that improves latency for many Mbps of sensitive traffic therefore has greater value than the same latency improvement applied to a very small flow.

This is a reasonable operational measure when the objective is to improve overall sensitive-traffic experience.

34.6 Preserve the Exact Stage 1 Delivered Quantities

Stage 2 must not change the Stage 1 admission decision.

For every source-stream commodity:

$$\boxed{\sum_{p \in P_{k,s}} f^{(2)}_{k,s,p} = z^{(1)}_{k,s}}$$

where:

$$z^{(1)}_{k,s}$$

is the amount delivered for that commodity during Stage 1.

Notice that this is an **equality constraint**, not an inequality.

Stage 2 is therefore prohibited from:

- reducing admitted traffic;
- admitting previously rejected traffic;
- exchanging one commodity for another.

Only routing may change.

In [274…

```
# ------------------------------------------------------------
# Build a lookup of Stage 1 delivered quantities
# ------------------------------------------------------------
#
# The key is commodity_id.
#
# The value is the exact amount delivered in the
# maximum-throughput Stage 1 solution.

stage1_delivery_lookup = (
    split_stage1_delivery_df
    .set_index(
        "commodity_id"
    )[
        "delivered_mbps"
    ]
    .to_dict()
)



# ------------------------------------------------------------
# Add one delivery-preservation equality constraint
# for every source-stream commodity
# ------------------------------------------------------------

for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    # Total Stage 2 flow across all candidate paths for this
    # commodity must equal the exact Stage 1 delivered amount.

    split_stage2 += (
        pl.lpSum(
            split_stage2_vars[
                int(pid)
            ]
            for pid
            in group[
                "path_id"
            ]
        )
        ==
        stage1_delivery_lookup[
            commodity_id
        ],

        # Give each constraint a readable name.
```

```
        f"preserve_{commodity_id}"
    )


print(
    "Stage 1 admission-preservation constraints added:",
    len(
        stage1_delivery_lookup
    )
)
```

```
Stage 1 admission-preservation constraints added: 20
```

Interpretation of Admission Preservation

Stage 2 is now constrained to deliver exactly the same amount of each source-stream commodity as Stage 1.

For example, if Stage 1 delivered:

$$6 \text{ Mbps}$$

of S1 / UK-L, Stage 2 must also deliver exactly:

$$6 \text{ Mbps}.$$

If Stage 1 delivered:

$$0 \text{ Mbps}$$

of S3 / UK-L, Stage 2 may not begin serving that traffic merely because doing so would improve latency.

This prevents the latency objective from changing the throughput or admission problem that Stage 1 already solved.

34.6.2 Business and Operational Impact

This constraint makes the Stage 2 comparison highly interpretable.

If latency improves, the business can say:

> "We improved service quality without sacrificing throughput or changing the customer admission decision."

That is much stronger than a result where latency improves only because some traffic was dropped.

34.7 Reapply Directed Link-Capacity Constraints

Stage 2 may reroute traffic, but every new routing configuration must still satisfy the physical network.

For each directed arc:

$$\sum_{k,s} \sum_{\substack{p \in P_{k,s} \\ e \in p}} f^{(2)}_{k,s,p} \leq C_e.$$

These are the same physical capacity limits used in Stage 1.

The difference is that Stage 2 is searching for a different flow distribution within those limits.

In [275…

```
# ------------------------------------------------------------
# Add Stage 2 directed capacity constraints
# ------------------------------------------------------------

for _, arc in (
    arcs_df.iterrows()
):

    # Represent the directed arc.

    edge = (
        arc[
            "from_node"
        ],
        arc[
            "to_node"
        ]
    )


    # Find all candidate paths that use this direction.

    using_edge = (
        candidate_paths_df[
            candidate_paths_df[
                "path_edges"
            ].apply(
                lambda edges:
                    edge in edges
            )
        ]
    )
```

```
    # Sum Stage 2 flow on every path using this arc
    # and restrict it to available directional capacity.

    split_stage2 += (
        pl.lpSum(
            split_stage2_vars[
                int(pid)
            ]
            for pid
            in using_edge[
                "path_id"
            ]
        )
        <= float(
            arc[
                "capacity_mbps"
            ]
        ),

        f"capacity_"
        f"{arc['from_node']}_"
        f"{arc['to_node']}"
    )


print(
    "Stage 2 directed capacity constraints added:",
    len(
        arcs_df
    )
)
```

```
Stage 2 directed capacity constraints added: 22
```

34.8 Validate Stage 2 Model Structure

Before solving, the model dimensions are checked.

Stage 2 should contain:

- 68 continuous path-flow variables;
- 20 commodity-level admission-preservation constraints;
- 22 directed link-capacity constraints.

Therefore:

$$20 + 22 = 42$$

constraints are expected.

In [276…

```
# Count Stage 2 variables.

stage2_variable_count = (
    len(
        split_stage2.variables()
    )
)


# Count Stage 2 constraints.

stage2_constraint_count = (
    len(
        split_stage2.constraints
    )
)


print(
    "Stage 2 variables:",
    stage2_variable_count
)

print(
    "Stage 2 constraints:",
    stage2_constraint_count
)


# Validate expected model dimensions.

assert (
    stage2_variable_count
    == 68
)

assert (
    stage2_constraint_count
    == 42
)


print(
    "Stage 2 model-dimension validation passed."
)
```

```
Stage 2 variables: 68
Stage 2 constraints: 42
Stage 2 model-dimension validation passed.
```

34.9 Solve the Stage 2 LP

The completed Stage 2 model is now solved using the same CBC solver configuration validated in Section 31.

The solver searches for the minimum sensitive-traffic latency exposure among all routing configurations that:

- preserve every Stage 1 delivered quantity;
- respect every directional capacity constraint.

In [277…

```
# ------------------------------------------------------------
# Solve Stage 2
# ------------------------------------------------------------

split_stage2.solve(
    cbc_solver
)


# Convert numerical status to readable form.

split_stage2_status = (
    pl.LpStatus[
        split_stage2.status
    ]
)


# Retrieve objective value.

split_stage2_latency_objective = (
    pl.value(
        split_stage2.objective
    )
)


print(
    "Stage 2 status:",
    split_stage2_status
)

print(
    "Stage 2 sensitive-latency exposure:",
    split_stage2_latency_objective,
    "Mbps*ms"
)
```

```
Stage 2 status: Optimal
Stage 2 sensitive-latency exposure: 644.0 Mbps*ms
```

34.9.1 Validate Stage 2 Solver Status

In [278…

```
# Stage 2 results should only be interpreted
# if the solver proves optimality.

assert (
    split_stage2_status
    == "Optimal"
)


assert (
    split_stage2_latency_objective
    is not None
)


print(
    "Stage 2 solver validation passed."
)
```

```
Stage 2 solver validation passed.
```

Interpretation of Stage 2 Optimality

CBC reports an optimal solution.

Therefore, given:

- the exact Stage 1 commodity admission quantities;
- the current network capacities;
- the current candidate paths;

no alternative feasible splittable routing has lower total latency exposure for the admitted UK-L and US-L traffic.

34.10 Validate Admission Preservation

The central requirement of Stage 2 is that no commodity's delivered quantity changes.

The validation therefore compares:

$$z_{k,s}^{(1)}$$

with:

$$z_{k,s}^{(2)}$$

for all 20 commodities.

A successful Stage 2 solution should produce:

$$z_{k,s}^{(2)} - z_{k,s}^{(1)} = 0$$

for every commodity.

In [279…

```
# ------------------------------------------------------------
# Calculate Stage 2 delivered amount for each commodity
# ------------------------------------------------------------

stage2_delivery_records = []


for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    # Sum Stage 2 flow over all paths for this commodity.

    stage2_delivered = sum(
        (
            split_stage2_vars[
                int(pid)
            ].value()
            or 0
        )
        for pid
        in group[
            "path_id"
        ]
    )


    # Retrieve Stage 1 delivered quantity.

    stage1_delivered = float(
        stage1_delivery_lookup[
            commodity_id
        ]
    )


    # Store comparison.

    stage2_delivery_records.append({
```

```
        "commodity_id":
            commodity_id,
        "source":
            group[
                "source"
            ].iloc[0],
        "stream":
            group[
                "stream"
            ].iloc[0],
        "stage1_delivered_mbps":
            stage1_delivered,
        "stage2_delivered_mbps":
            stage2_delivered,
        "delivery_change_mbps":
            stage2_delivered
            - stage1_delivered
    })


stage1_vs_stage2_delivery_df = (
    pd.DataFrame(
        stage2_delivery_records
    )
)


stage1_vs_stage2_delivery_df
```

Out [279…

	commodity_id	source	stream	stage1_delivered_mbps	stage2_delivered_m
0	B_UK-L	B	UK-L	6.0	
1	B_UK-T	B	UK-T	14.0	
2	B_US-L	B	US-L	8.0	
3	B_US-T	B	US-T	9.0	
4	C_UK-L	C	UK-L	4.0	
5	C_UK-T	C	UK-T	5.0	
6	C_US-L	C	US-L	5.0	
7	C_US-T	C	US-T	6.0	
8	S1_UK-L	S1	UK-L	6.0	
9	S1_UK-T	S1	UK-T	7.0	
10	S1_US-L	S1	US-L	0.0	
11	S1_US-T	S1	US-T	0.0	
12	S2_UK-L	S2	UK-L	0.0	
13	S2_UK-T	S2	UK-T	6.0	
14	S2_US-L	S2	US-L	5.0	
15	S2_US-T	S2	US-T	0.0	
16	S3_UK-L	S3	UK-L	0.0	
17	S3_UK-T	S3	UK-T	2.0	
18	S3_US-L	S3	US-L	0.0	
19	S3_US-T	S3	US-T	0.0	

In [280…

```
## Validate nemerically
# Maximum absolute difference between Stage 1 and Stage 2
# commodity-level delivered quantities.

max_delivery_change = (
    stage1_vs_stage2_delivery_df[
        "delivery_change_mbps"
    ]
    .abs()
    .max()
)


assert (
```

```
    max_delivery_change
    < 1e-6
)


print(
    "Maximum commodity-level delivery change:",
    max_delivery_change,
    "Mbps"
)

print(
    "All source-stream delivered quantities preserved."
)
```

```
Maximum commodity-level delivery change: 0.0 Mbps
All source-stream delivered quantities preserved.
```

34.10.1 Validate Total Throughput Preservation

In [281…

```
# Calculate total traffic delivered in Stage 2.

split_stage2_total_throughput = (
    stage1_vs_stage2_delivery_df[
        "stage2_delivered_mbps"
    ].sum()
)


print(
    "Stage 1 total throughput:",
    split_stage1_throughput,
    "Mbps"
)

print(
    "Stage 2 total throughput:",
    split_stage2_total_throughput,
    "Mbps"
)


assert abs(
    split_stage2_total_throughput
    - split_stage1_throughput
) < 1e-6


print(
    "Total throughput preserved."
)
```

```
Stage 1 total throughput: 83.0 Mbps
Stage 2 total throughput: 83.0 Mbps
Total throughput preserved.
```

34.10.2 Interpretation of Admission Preservation

Stage 2 preserves:

$$83 \text{ Mbps}$$

of total throughput.

More importantly, it preserves the exact delivered amount for every individual source-stream commodity.

This means that Stage 2 is not improving latency by:

- dropping more tolerant traffic;
- dropping sensitive traffic;
- admitting different sources;
- changing the Stage 1 fairness pattern.

The only degree of freedom is routing.

34.10.13 Business and Operational Impact

This provides a clean service-quality improvement.

The business receives the same throughput commitment and the same source-stream admission decision, but latency-sensitive traffic is allowed to move onto better routes.

That makes the Stage 2 benefit easy to communicate:

> **Same customers served. Same amount of traffic delivered. Better latency-sensitive routing.**

34.11 Extract Final Splittable Routes

The Stage 2 decision variables are now converted back into active routing decisions.

Only paths carrying positive flow are retained.

This allows direct comparison between:

- Stage 1 maximum-throughput routing;
- Stage 2 latency-optimized routing.

In [282…

```
# ------------------------------------------------------------
# Extract all positive-flow Stage 2 routes
# ------------------------------------------------------------

split_stage2_routes = []


for _, row in (
    candidate_paths_df.iterrows()
):

    # Candidate path identifier.

    pid = int(
        row[
            "path_id"
        ]
    )


    # Retrieve optimized Stage 2 flow.

    flow = (
        split_stage2_vars[
            pid
        ].value()
        or 0
    )


    # Keep only meaningful positive-flow routes.

    if flow > 1e-8:

        split_stage2_routes.append({
            "model":
                "Split Final",
            "commodity_id":
                row[
                    "commodity_id"
                ],
            "source":
                row[
                    "source"
                ],
            "stream":
                row[
                    "stream"
                ],
```

```
            "destination":
                row[
                    "destination"
                ],
            "path":
                row[
                    "path"
                ],
            "flow_mbps":
                flow,
            "path_capacity_mbps":
                row[
                    "path_capacity_mbps"
                ],
            "path_latency_ms":
                row[
                    "path_latency_ms"
                ]
        })


split_stage2_routes_df = (
    pd.DataFrame(
        split_stage2_routes
    )
)


print(
    "Active Stage 2 routes:",
    len(
        split_stage2_routes_df
    )
)


split_stage2_routes_df
```

```
Active Stage 2 routes: 18
```

Out[282…

	model	commodity_id	source	stream	destination	path	flow_mbps	path_
0	Split Final	S1_UK-L	S1	UK-L	UK	S1 -> A -> C -> UK	6.0	
1	Split Final	S1_UK-T	S1	UK-T	UK	S1 -> A -> B -> C -> UK	4.0	

2	Split Final	S1_UK-T	S1	UK-T	UK	S1 -> A -> B -> D -> UK	3.0
3	Split Final	S2_UK-T	S2	UK-T	UK	S2 -> A -> B -> D -> UK	5.5
4	Split Final	S2_UK-T	S2	UK-T	UK	S2 -> A -> C -> B -> D -> UK	0.5
5	Split Final	S2_US-L	S2	US-L	US	S2 -> A -> C -> US	5.0
6	Split Final	S3_UK-T	S3	UK-T	UK	S3 -> B -> A -> C -> UK	2.0
7	Split Final	B_UK-L	B	UK-L	UK	B -> C -> UK	6.0
8	Split Final	B_UK-T	B	UK-T	UK	B -> A -> C -> UK	4.5
9	Split Final	B_UK-T	B	UK-T	UK	B -> D -> UK	9.5
10	Split Final	B_US-L	B	US-L	US	B -> C -> US	8.0
11	Split Final	B_US-T	B	US-T	US	B -> A -> C -> US	7.0
						B ->	

12	Split Final	B_US-T	B	US-T	US	D -> US	2.0
13	Split Final	C_UK-L	C	UK-L	UK	C -> UK	4.0
14	Split Final	C_UK-T	C	UK-T	UK	C -> A -> B -> D -> UK	1.5
15	Split Final	C_UK-T	C	UK-T	UK	C -> UK	3.5
16	Split Final	C_US-L	C	US-L	US	C -> US	5.0
17	Split Final	C_US-T	C	US-T	US	C -> A -> B -> D -> US	6.0

34.11.1 Interpretation of Final Stage 2 Routes

Stage 2 materially changes the routing pattern while preserving admission.

One of the clearest changes involves B / UK-L.

In Stage 1, the 6 Mbps B / UK-L flow used the slower D route:

$$B \to D \to UK$$

with latency:

$$34 \text{ ms}.$$

Stage 2 moves the full 6 Mbps to:

$$B \to C \to UK$$

with latency:

$$17 \text{ ms}.$$

This cuts B / UK-L path latency in half.

Similarly, B / US-L moves onto:

$$B \to C \to US$$

at:

$$19 \text{ ms},$$

rather than the slower D route used during Stage 1.

Those lower-latency routes consume scarce C-side capacity.

To make room, tolerant traffic is redistributed across longer alternatives.

This is exactly the expected behavior of Stage 2.

34.11.2 Tolerant Traffic as Routing Flexibility

Several tolerant flows are split across longer routes in the Stage 2 solution.

For example, B / UK-T is split between alternative paths rather than occupying all of the most attractive C-side capacity.

Similarly, C-originating tolerant traffic may use longer routes through A, B, and D.

This occurs because Stage 2 does not penalize tolerant-stream latency.

The optimizer is therefore willing to send tolerant traffic along longer routes if doing so frees low-latency capacity for UK-L and US-L.

This should not be interpreted as a solver error.

It is a direct consequence of the Stage 2 objective.

34.11.3 Important Modeling Caveat — Tolerant-Traffic Latency Is Unpriced

The Stage 2 objective includes only:

- UK-L;
- US-L.

Therefore, two solutions with identical sensitive-traffic latency are equally good even if one sends tolerant traffic on much longer routes.

This explains why some Stage 2 tolerant routes may appear operationally unintuitive.

For the baseline assignment, this is acceptable because tolerant latency was intentionally excluded from the objective.

However, a production model could add a **tertiary objective** after sensitive latency is minimized, such as:

1. maximize throughput;
2. minimize latency-sensitive traffic latency;
3. minimize tolerant-traffic latency or total routing complexity.

That would remove unnecessary tolerant-traffic detours without changing the first two priorities.

Business and Operational Impact

This is an important example of why optimization objectives must reflect the full business policy.

The solver will optimize exactly what is measured.

If tolerant traffic latency is operationally important, even if less important than sensitive latency, that preference should eventually be represented explicitly rather than assumed.

34.12 Compare Stage 1 and Stage 2 Routing

The next analysis compares path-level flows before and after latency optimization.

This identifies which specific route changes are responsible for the sensitive-latency improvement.

In [283…

```
# ------------------------------------------------------------
# Build path-level Stage 1 flow table
# ------------------------------------------------------------

stage1_path_flow_df = (
    candidate_paths_df[
        [
            "path_id",
            "commodity_id",
            "source",
            "stream",
            "destination",
            "path",
            "path_latency_ms"
        ]
    ]
    .copy()
```

```
)


# Add Stage 1 optimized flow to every candidate path.

stage1_path_flow_df[
    "stage1_flow_mbps"
] = (
    stage1_path_flow_df[
        "path_id"
    ].apply(
        lambda pid:
            split_flow_vars[
                int(pid)
            ].value()
            or 0
    )
)


# Add Stage 2 optimized flow.

stage1_path_flow_df[
    "stage2_flow_mbps"
] = (
    stage1_path_flow_df[
        "path_id"
    ].apply(
        lambda pid:
            split_stage2_vars[
                int(pid)
            ].value()
            or 0
    )
)


# Calculate path-level flow change.

stage1_path_flow_df[
    "flow_change_mbps"
] = (
    stage1_path_flow_df[
        "stage2_flow_mbps"
    ]
    - stage1_path_flow_df[
        "stage1_flow_mbps"
    ]
)


# Keep only candidate paths whose usage changed.
```

```
stage1_vs_stage2_route_changes_df = (
    stage1_path_flow_df[
        stage1_path_flow_df[
            "flow_change_mbps"
        ].abs()
        > 1e-8
    ]
    .sort_values(
        [
            "commodity_id",
            "path_latency_ms"
        ]
    )
    .reset_index(
        drop=True
    )
)


stage1_vs_stage2_route_changes_df[
    [
        "commodity_id",
        "stream",
        "path",
        "path_latency_ms",
        "stage1_flow_mbps",
        "stage2_flow_mbps",
        "flow_change_mbps"
    ]
]
```

Out[283...

	commodity_id	stream	path	path_latency_ms	stage1_flow_mbps	stage2_
0	B_UK-L	UK-L	B -> C -> UK	17	0.0	
1	B_UK-L	UK-L	B -> D -> UK	34	6.0	
2	B_UK-T	UK-T	B -> A -> C -> UK	30	0.0	
3	B_UK-T	UK-T	B -> D -> UK	34	14.0	
4	B_US-L	US-L	B -> C -> US	19	0.0	

5	B_US-L	US-L	B -> D -> US	32	8.0
6	B_US-T	US-T	B -> C -> US	19	9.0
7	B_US-T	US-T	B -> A -> C -> US	32	0.0
8	B_US-T	US-T	B -> D -> US	32	0.0
9	C_UK-T	UK-T	C -> UK	7	5.0
10	C_UK-T	UK-T	C -> A -> B -> D -> UK	57	0.0
11	C_US-T	US-T	C -> US	9	6.0
12	C_US-T	US-T	C -> A -> B -> D -> US	55	0.0
13	S1_UK-T	UK-T	S1 -> A -> C -> UK	27	7.0
14	S1_UK-T	UK-T	S1 -> A -> B -> C -> UK	44	0.0
15	S1_UK-T	UK-T	S1 -> A -> B -> D -> UK	61	0.0
16	S2_UK-T	UK-T	S2 -> A -> C	29	6.0

			-> UK		
17	S2_UK-T	UK-T	S2 -> A -> B -> D -> UK	63	0.0
18	S2_UK-T	UK-T	S2 -> A -> C -> B -> D -> UK	66	0.0
19	S3_UK-T	UK-T	S3 -> B -> C -> UK	27	2.0
20	S3_UK-T	UK-T	S3 -> B -> A -> C -> UK	40	0.0

34.12.1 Interpretation of Stage 1-to-Stage 2 Route Changes

The route-change table identifies the actual mechanism behind the latency improvement.

Sensitive traffic tends to move toward lower-latency C-side routes.

To create the necessary capacity, tolerant traffic is shifted toward:

- D-side alternatives;
- longer A-B-C combinations;
- longer reverse-direction internal paths when necessary.

Therefore, Stage 2 does not reduce congestion by serving less traffic.

It **reallocates scarce low-latency capacity** toward traffic that benefits from it most.

34.13 Final Stage 2 Link Utilization

The route changes alter how the 83 Mbps interacts with the physical network.

The reusable link-utilization function from Section 33 is applied to the Stage 2 variables.

This allows Stage 1 and Stage 2 congestion patterns to be compared directly.

In [284…

```
# Reuse the link-level aggregation function from Section 33.

split_stage2_link_df = (
    calculate_link_results(
        candidate_paths_df,
        split_stage2_vars,
        arcs_df,
        "Split Final"
    )
)


# Rank links from highest utilization to lowest.

split_stage2_link_df = (
    split_stage2_link_df
    .sort_values(
        "utilization_percentage",
        ascending=False
    )
    .reset_index(
        drop=True
    )
)


split_stage2_link_df
```

Out [284…

	model	from_node	to_node	capacity_mbps	flow_mbps	residual_capacity_
0	Split Final	C	UK	30.0	30.0	
1	Split Final	A	B	20.0	20.0	
2	Split Final	A	C	25.0	25.0	
3	Split Final	B	C	18.0	18.0	
	Split					

4	Final	C	US	25.0	25.0
5	Split Final	B	D	28.0	28.0
6	Split Final	D	UK	22.0	20.0
7	Split Final	B	A	20.0	13.5
8	Split Final	S1	A	40.0	13.0
9	Split Final	S2	A	35.0	11.0
10	Split Final	C	A	25.0	7.5
11	Split Final	D	US	30.0	8.0
12	Split Final	S3	B	30.0	2.0
13	Split Final	C	B	18.0	0.5
14	Split Final	D	B	28.0	0.0
15	Split Final	UK	C	30.0	0.0
16	Split Final	A	S1	40.0	0.0
17	Split Final	US	C	25.0	0.0
18	Split Final	B	S3	30.0	0.0
19	Split Final	UK	D	22.0	0.0
20	Split Final	A	S2	35.0	0.0
21	Split Final	US	D	30.0	0.0

34.13.1 Interpretation of Final Stage 2 Link Utilization

Stage 2 preserves total throughput but materially changes the internal utilization pattern.

Six directed links now operate at:

$$100\%$$

utilization:

- A → B;
- A → C;
- B → C;
- B → D;
- C → UK;
- C → US.

This is more than the three saturated links observed in Stage 1.

That does **not** mean Stage 2 performs worse.

Instead, it means the optimizer is using network capacity differently in order to reserve low-latency routes for sensitive traffic.

Stage 2 redistributes tolerant traffic across additional corridors and consumes more of the network's available routing flexibility.

Business and Operational Impact

The result illustrates an important tradeoff.

Improving sensitive-traffic latency may reduce network headroom even when throughput remains unchanged.

Therefore, service quality and resilience should both be monitored.

A network operating with more saturated links may be less tolerant of:

- sudden demand increases;
- link failures;
- unexpected traffic bursts.

This is one reason utilization and latency should be evaluated together rather than independently.

34.14 Compare Stage 1 and Stage 2 Link Utilization

In [285…

```
# ------------------------------------------------------------
# Compare link-level utilization between Stage 1 and Stage 2
# ------------------------------------------------------------

stage1_vs_stage2_link_df = (
    split_stage1_link_df[
        [
            "from_node",
            "to_node",
            "capacity_mbps",
            "flow_mbps",
            "utilization_percentage"
        ]
    ]
    .rename(
        columns={
            "flow_mbps":
                "stage1_flow_mbps",
            "utilization_percentage":
                "stage1_utilization_pct"
        }
    )
    .merge(
        split_stage2_link_df[
            [
                "from_node",
                "to_node",
                "flow_mbps",
                "utilization_percentage"
            ]
        ]
        .rename(
            columns={
                "flow_mbps":
                    "stage2_flow_mbps",
                "utilization_percentage":
                    "stage2_utilization_pct"
            }
        ),
        on=[
            "from_node",
            "to_node"
        ],
        how="inner"
    )
)


# Calculate traffic movement by link.

stage1_vs_stage2_link_df[
    "flow_change_mbps"
```

```
] = (
    stage1_vs_stage2_link_df[
        "stage2_flow_mbps"
    ]
    - stage1_vs_stage2_link_df[
        "stage1_flow_mbps"
    ]
)


# Create readable labels.

stage1_vs_stage2_link_df[
    "link"
] = (
    stage1_vs_stage2_link_df[
        "from_node"
    ]
    + " → "
    + stage1_vs_stage2_link_df[
        "to_node"
    ]
)


# Keep links that carry traffic in either stage.

stage1_vs_stage2_active_links_df = (
    stage1_vs_stage2_link_df[
        (
            stage1_vs_stage2_link_df[
                "stage1_flow_mbps"
            ] > 1e-8
        )
        |
        (
            stage1_vs_stage2_link_df[
                "stage2_flow_mbps"
            ] > 1e-8
        )
    ]
    .copy()
)


stage1_vs_stage2_active_links_df[
    [
        "link",
        "capacity_mbps",
        "stage1_flow_mbps",
        "stage2_flow_mbps",
        "flow_change_mbps"
```

```
    ]
]
```

Out [285...

	link	capacity_mbps	stage1_flow_mbps	stage2_flow_mbps	flow_change_m
0	C → UK	30.0	30.0	30.0	
1	C → US	25.0	25.0	25.0	
2	B → D	28.0	28.0	28.0	
3	A → C	25.0	24.0	25.0	
4	D → UK	22.0	20.0	20.0	
5	B → C	18.0	11.0	18.0	
6	S1 → A	40.0	13.0	13.0	
7	S2 → A	35.0	11.0	11.0	
8	D → US	30.0	8.0	8.0	
9	S3 → B	30.0	2.0	2.0	
13	C → B	18.0	0.0	0.5	
16	C → A	25.0	0.0	7.5	
17	B → A	20.0	0.0	13.5	
	A				

18	→ B	20.0	0.0	20.0

34.14.1 Stage 1 vs Stage 2 Utilization Chart

In [286…

```
# Compare directional utilization before and after latency optimizatio

stage1_vs_stage2_active_links_df.set_index(
    "link"
)[
    [
        "stage1_utilization_pct",
        "stage2_utilization_pct"
    ]
].plot(
    kind="bar",
    figsize=(13, 6)
)


plt.ylabel(
    "Utilization (%)"
)

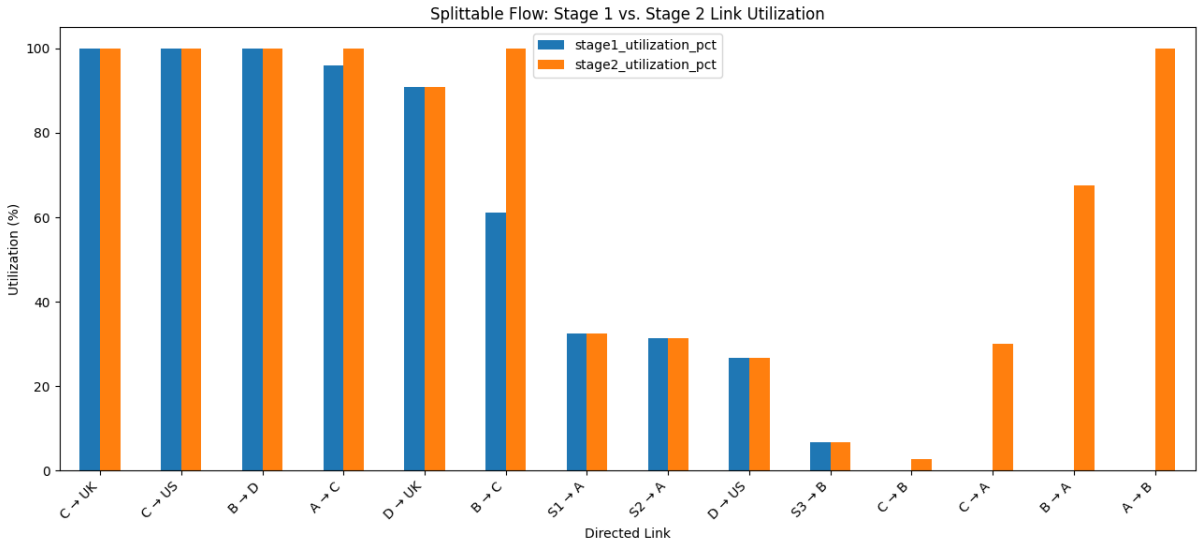
plt.xlabel(
    "Directed Link"
)

plt.title(
    "Splittable Flow: "
    "Stage 1 vs. Stage 2 Link Utilization"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()

plt.show()
```



34.15 Final Splittable Stage 2 Network Visualization

The final Stage 2 routing is plotted using the same network visualization function introduced in Section 33.

Keeping the visual encoding identical allows the Stage 1 and Stage 2 diagrams to be compared directly.

Any change in network traffic reflects the secondary latency objective because commodity-level delivery is unchanged.

In [287…

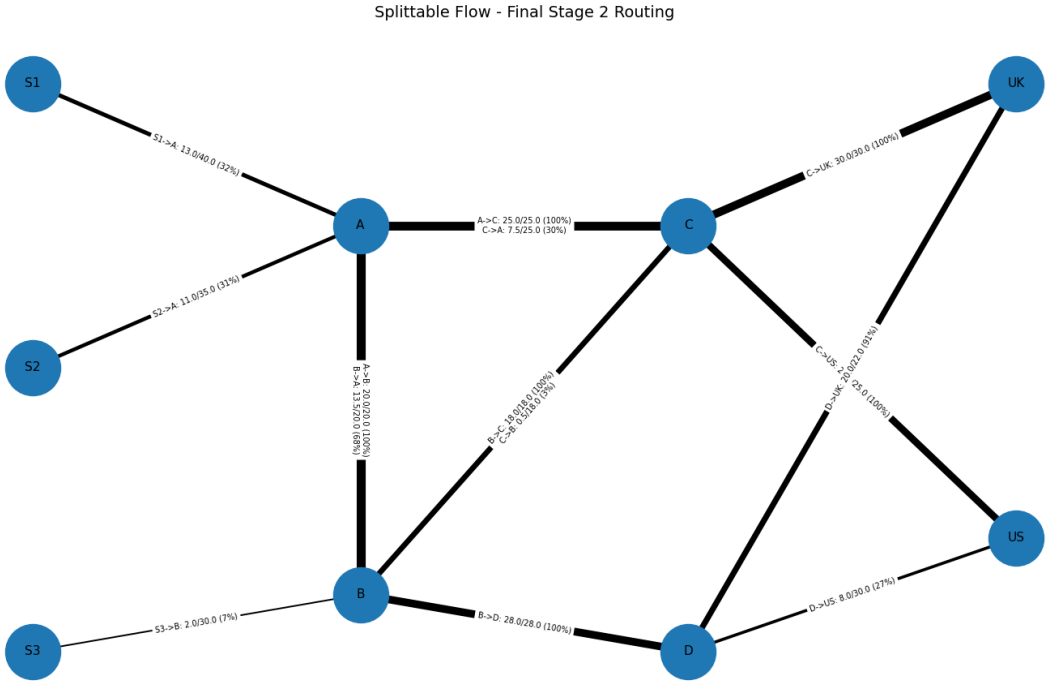
```
# Plot final latency-optimized splittable routing.

plot_network_flow(
    graph=
        G_synthetic,

    link_results_df=
        split_stage2_link_df,

    pos=
        pos,

    title=(
        "Splittable Flow - "
        "Final Stage 2 Routing"
    )
)
```



34.15.1 Interpretation of the Final Stage 2 Network Diagram

The final diagram shows a more complex traffic pattern than Stage 1.

This complexity is created by the secondary latency objective.

Sensitive traffic is moved toward lower-latency C-side routes.

At the same time, tolerant traffic is redistributed across the rest of the network to preserve capacity feasibility.

Several internal links that were not saturated during Stage 1 now become fully utilized.

The important point is that:

$$\text{Total Delivered Traffic} = 83 \text{ Mbps}$$

in both diagrams.

Therefore, the visual differences represent **routing-quality optimization**, not a throughput change.

34.16 Calculate Final Sensitive-Traffic Latency Metrics

Two complementary latency metrics are reported for each sensitive stream.

Traffic-Weighted Average Latency

For stream k:

$$\bar{L}_k = \frac{\sum_p f_{k,p} L_p}{\sum_p f_{k,p}}.$$

This represents the average latency experienced by a delivered Mbps of that stream.

Worst-Case Used-Path Latency

The worst-case metric is:

$$L_k^{\max} = \max_{p: f_{k,p} > 0} L_p.$$

This captures the longest path actually used by the delivered sensitive traffic.

Reporting both avoids hiding a small amount of high-latency traffic behind a favorable average.

In [289…

```
def calculate_stream_latency_metrics(
    candidate_paths_df,
    flow_vars,
    stream_name
):
    """
    Calculate traffic-weighted average latency and the
    worst used-path latency for one traffic stream.
    """

    # Keep candidate paths belonging to the requested stream.

    stream_paths = (
        candidate_paths_df[
            candidate_paths_df[
                "stream"
            ]
            == stream_name
        ]
    )


    # Track delivered traffic.

    total_flow = 0.0
```

```
# Track:
#
#   flow * path latency
#
# across all used paths.

total_latency_exposure = 0.0


# Store latency of every path carrying positive traffic
# so that worst-case used-path latency can be reported.

used_latencies = []


# Evaluate each candidate path for the stream.

for _, row in (
    stream_paths.iterrows()
):

    pid = int(
        row[
            "path_id"
        ]
    )


    # Retrieve optimized traffic on this path.

    flow = (
        flow_vars[
            pid
        ].value()
        or 0
    )


    # Only positive-flow paths affect the metrics.

    if flow > 1e-8:

        # Accumulate delivered traffic.

        total_flow += (
            flow
        )


        # Add traffic-weighted latency contribution.
```

```
            total_latency_exposure += (
                flow
                * row[
                    "path_latency_ms"
                ]
            )


            # Track used-path latency for worst-case metric.

            used_latencies.append(
                row[
                    "path_latency_ms"
                ]
            )


    # Calculate weighted-average latency.

    average_latency = (
        total_latency_exposure
        / total_flow
        if total_flow > 0
        else np.nan
    )


    # Calculate maximum latency among used routes.

    worst_case_latency = (
        max(
            used_latencies
        )
        if used_latencies
        else np.nan
    )


    return {
        "stream":
            stream_name,
        "delivered_mbps":
            total_flow,
        "latency_exposure_mbps_ms":
            total_latency_exposure,
        "average_latency_ms":
            average_latency,
        "worst_case_latency_ms":
            worst_case_latency
    }
```

In [290…

```
#Calculate Final results
split_latency_results = (
    pd.DataFrame([
        calculate_stream_latency_metrics(
            candidate_paths_df,
            split_stage2_vars,
            "UK-L"
        ),
        calculate_stream_latency_metrics(
            candidate_paths_df,
            split_stage2_vars,
            "US-L"
        )
    ])
)


split_latency_results
```

Out[290…

	stream	delivered_mbps	latency_exposure_mbps_ms	average_latency_ms	v
0	UK-L	16.0	292.0	18.250000	
1	US-L	18.0	352.0	19.555556	

34.16.1 Interpretation of Final Sensitive-Traffic Latency

The final splittable routing delivers:

UK-L

$$16 \text{ Mbps}$$

with average latency:

$$18.25 \text{ ms}$$

and worst used-path latency:

$$27 \text{ ms}.$$

US-L

$$18 \text{ Mbps}$$

with average latency approximately:

$$19.56 \text{ ms}$$

and worst used-path latency:

$$31 \text{ ms}.$$

These results represent the best traffic-weighted sensitive latency achievable while preserving the exact Stage 1 admission quantities.

34.17 Quantify the Stage 2 Latency Improvement

To determine whether Stage 2 actually creates value, the same sensitive-stream latency metrics are calculated for the Stage 1 routing and compared with the final Stage 2 solution.

In [291…

```
# Calculate sensitive-stream latency under the
# original Stage 1 maximum-throughput routing.

split_stage1_latency_results = (
    pd.DataFrame([
        calculate_stream_latency_metrics(
            candidate_paths_df,
            split_flow_vars,
            "UK-L"
        ),
        calculate_stream_latency_metrics(
            candidate_paths_df,
            split_flow_vars,
            "US-L"
        )
    ])
)


# Rename Stage 1 metric columns.

stage1_latency_compare_df = (
    split_stage1_latency_results[
        [
            "stream",
            "delivered_mbps",
            "latency_exposure_mbps_ms",
            "average_latency_ms",
            "worst_case_latency_ms"
        ]
    ]
    .rename(
        columns={
            "delivered_mbps":
                "stage1_delivered_mbps",
            "latency_exposure_mbps_ms":
```

```
                "stage1_latency_exposure",
            "average_latency_ms":
                "stage1_average_latency_ms",
            "worst_case_latency_ms":
                "stage1_worst_case_latency_ms"
        }
    )
)


# Rename Stage 2 metric columns.

stage2_latency_compare_df = (
    split_latency_results[
        [
            "stream",
            "delivered_mbps",
            "latency_exposure_mbps_ms",
            "average_latency_ms",
            "worst_case_latency_ms"
        ]
    ]
    .rename(
        columns={
            "delivered_mbps":
                "stage2_delivered_mbps",
            "latency_exposure_mbps_ms":
                "stage2_latency_exposure",
            "average_latency_ms":
                "stage2_average_latency_ms",
            "worst_case_latency_ms":
                "stage2_worst_case_latency_ms"
        }
    )
)


# Merge Stage 1 and Stage 2 metrics.

split_latency_comparison_df = (
    stage1_latency_compare_df
    .merge(
        stage2_latency_compare_df,
        on="stream",
        how="inner"
    )
)


# Calculate average-latency improvement.

split_latency_comparison_df[
```

```
    "average_latency_reduction_ms"
] = (
    split_latency_comparison_df[
        "stage1_average_latency_ms"
    ]
    - split_latency_comparison_df[
        "stage2_average_latency_ms"
    ]
)


# Calculate percentage improvement.

split_latency_comparison_df[
    "average_latency_reduction_pct"
] = (
    100
    * split_latency_comparison_df[
        "average_latency_reduction_ms"
    ]
    / split_latency_comparison_df[
        "stage1_average_latency_ms"
    ]
)


# Calculate worst-case improvement.

split_latency_comparison_df[
    "worst_case_reduction_ms"
] = (
    split_latency_comparison_df[
        "stage1_worst_case_latency_ms"
    ]
    - split_latency_comparison_df[
        "stage2_worst_case_latency_ms"
    ]
)


split_latency_comparison_df
```

Out[291…

	stream	stage1_delivered_mbps	stage1_latency_exposure	stage1_average_la
0	UK-L	16.0	394.0	2
1	US-L	18.0	456.0	2

34.17.1 Interpretation of Stage 2 Latency Improvement

Stage 2 produces substantial latency improvement without changing throughput.

UK-L

Average latency falls from:

$$24.625 \text{ ms}$$

to:

$$18.25 \text{ ms}.$$

The reduction is:

$$6.375 \text{ ms}$$

or approximately:

$$25.9\%.$$

Worst-case used-path latency falls from:

$$34 \text{ ms}$$

to:

$$27 \text{ ms}.$$

US-L

Average latency falls from approximately:

$$25.33 \text{ ms}$$

to:

$$19.56 \text{ ms}.$$

The reduction is approximately:

$$5.78 \text{ ms}$$

or:

$$22.8\%.$$

Worst-case used-path latency improves from:

$$32 \text{ ms}$$

to:

$$31 \text{ ms}.$$

Key Finding

Stage 2 therefore improves the average latency experienced by both sensitive traffic classes by more than:

$$20\%$$

while preserving the same delivered Mbps.

34.17.2 Traffic-Weighted Latency Exposure Improvement

A second way to quantify the value of Stage 2 is to compare the total **traffic-weighted latency exposure** before and after latency optimization.

Traffic-weighted latency exposure is calculated as:

$$\text{Latency Exposure} = \text{Delivered Traffic} \times \text{Average Latency}$$

This metric is useful because it combines:

- the amount of sensitive traffic being delivered;
- the latency experienced by that traffic.

Because Stage 2 preserves the delivered quantity of every commodity, any reduction in this metric is caused entirely by improved routing rather than by serving less traffic.

UK-L

Stage 1 delivered:

$$16 \text{ Mbps}$$

with an average latency of:

$$24.625 \text{ ms}.$$

Therefore, Stage 1 UK-L latency exposure was:

$$16(24.625) = 394 \text{ Mbps} \cdot \text{ms}.$$

After Stage 2 optimization, the same:

$$16 \text{ Mbps}$$

is delivered with an average latency of:

$$18.25 \text{ ms}.$$

Therefore, Stage 2 UK-L latency exposure becomes:

$$16(18.25) = 292 \text{ Mbps} \cdot \text{ms}.$$

The UK-L reduction is therefore:

$$394 - 292 = 102 \text{ Mbps} \cdot \text{ms}.$$

US-L

Stage 1 delivered:

$$18 \text{ Mbps}$$

with an average latency of approximately:

$$25.3333 \text{ ms}.$$

Therefore:

$$18(25.3333) \approx 456 \text{ Mbps} \cdot \text{ms}.$$

After Stage 2, the same:

$$18 \text{ Mbps}$$

is delivered with an average latency of approximately:

$$19.5556 \text{ ms}.$$

Therefore:

$$18(19.5556) \approx 352 \text{ Mbps} \cdot \text{ms}.$$

The US-L reduction is therefore:

$$456 - 352 = 104 \text{ Mbps} \cdot \text{ms}.$$

Combined Sensitive-Traffic Improvement

Across UK-L and US-L together, Stage 1 latency exposure is:

$$394 + 456 = 850 \text{ Mbps} \cdot \text{ms}.$$

Stage 2 reduces this to:

$$292 + 352 = 644 \text{ Mbps} \cdot \text{ms}.$$

The total reduction is therefore:

$$850 - 644 = 206 \text{ Mbps} \cdot \text{ms}.$$

The percentage reduction is approximately:

$$\frac{206}{850} \times 100 \approx 24.24\%.$$

Interpretation

Stage 2 reduces total traffic-weighted latency exposure for the two sensitive traffic classes by approximately:

$$24.24\%.$$

This is a strong result because the improvement is achieved while preserving:

- the same total throughput;
- the same UK-L delivered traffic;
- the same US-L delivered traffic;
- the same commodity-level Stage 1 admission decisions.

Therefore, the improvement reflects a genuine routing-quality gain rather than a reduction in service.

Business and Operational Impact

From an operational perspective, Stage 2 delivers the same amount of sensitive traffic while exposing that traffic to substantially less aggregate latency.

This means the network improves service quality without requiring:

- additional bandwidth;
- reduced throughput;
- or changes in which customers or traffic streams are admitted.

The result demonstrates the value of the lexicographic optimization strategy:

> **First maximize service volume, then improve service quality within that maximum-throughput operating point.**

In [292…

```
# Calculate combined sensitive latency exposure.

stage1_total_sensitive_latency_exposure = (
    split_latency_comparison_df[
        "stage1_latency_exposure"
    ].sum()
)
```

```
stage2_total_sensitive_latency_exposure = (
    split_latency_comparison_df[
        "stage2_latency_exposure"
    ].sum()
)


latency_exposure_reduction = (
    stage1_total_sensitive_latency_exposure
    - stage2_total_sensitive_latency_exposure
)


latency_exposure_reduction_pct = (
    100
    * latency_exposure_reduction
    / stage1_total_sensitive_latency_exposure
)


print(
    "Stage 1 sensitive latency exposure:",
    stage1_total_sensitive_latency_exposure,
    "Mbps*ms"
)

print(
    "Stage 2 sensitive latency exposure:",
    stage2_total_sensitive_latency_exposure,
    "Mbps*ms"
)

print(
    "Reduction:",
    latency_exposure_reduction,
    "Mbps*ms"
)

print(
    "Reduction percentage:",
    latency_exposure_reduction_pct,
    "%"
)
```

```
Stage 1 sensitive latency exposure: 850.0 Mbps*ms
Stage 2 sensitive latency exposure: 644.0 Mbps*ms
Reduction: 206.0 Mbps*ms
Reduction percentage: 24.235294117647058 %
```

Interpretation of Total Sensitive-Latency Exposure

Across UK-L and US-L combined, traffic-weighted latency exposure falls from approximately:

$$850$$

to:

$$644 \text{ Mbps} \cdot \text{ms}.$$

The reduction is:

$$206 \text{ Mbps} \cdot \text{ms},$$

or approximately:

$$24.2\%.$$

This provides one overall measure of the value created by Stage 2.

The network serves the same amount of sensitive traffic but exposes it to substantially less total latency.

34.18 Sensitive-Traffic Latency Visualization

The following chart compares Stage 1 and Stage 2 traffic-weighted average latency.

Because delivered quantities are identical in both stages, the difference represents a pure routing improvement.

In [293…

```
# Prepare average-latency comparison.

latency_plot_df = (
    split_latency_comparison_df[
        [
            "stream",
            "stage1_average_latency_ms",
            "stage2_average_latency_ms"
        ]
    ]
    .set_index(
        "stream"
    )
)


latency_plot_df.plot(
    kind="bar",
    figsize=(8, 5)
```

```
)


plt.ylabel(
    "Traffic-Weighted Average Latency (ms)"
)

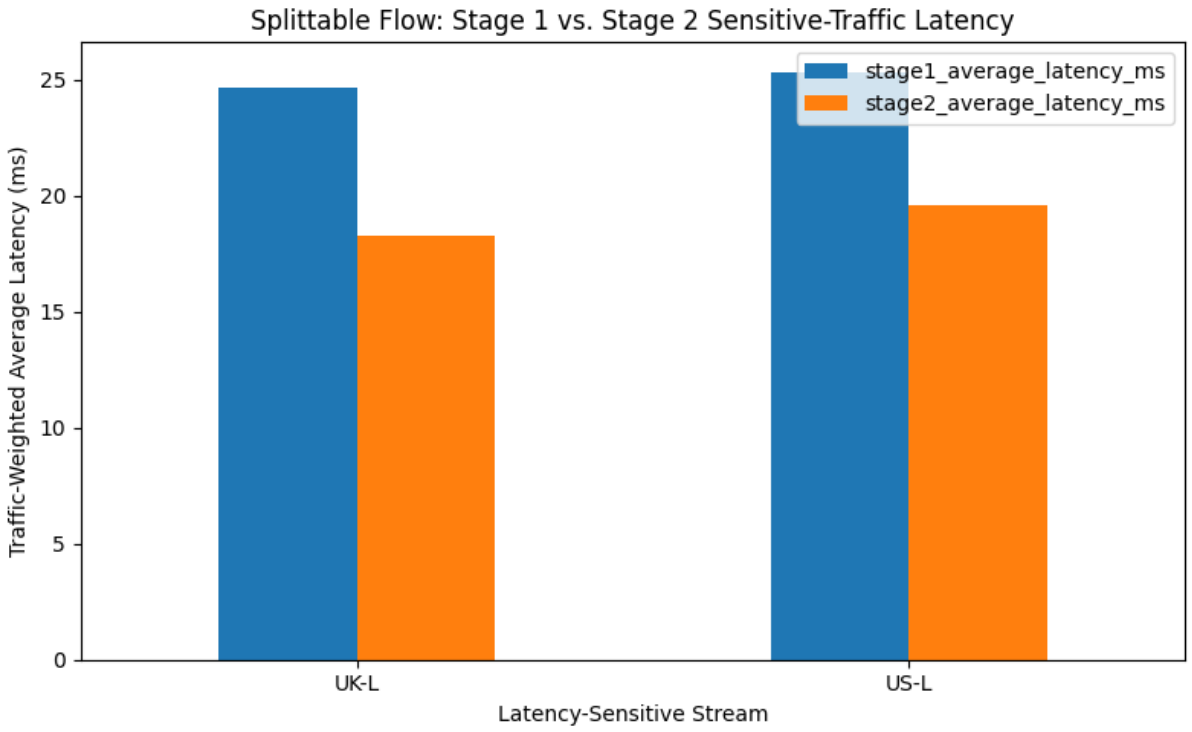
plt.xlabel(
    "Latency-Sensitive Stream"
)

plt.title(
    "Splittable Flow: "
    "Stage 1 vs. Stage 2 Sensitive-Traffic Latency"
)

plt.xticks(
    rotation=0
)

plt.tight_layout()

plt.show()
```



34.18.1 Worst-Case Latency Visualization

In [295…

```
# Prepare worst-case used-path latency comparison.

worst_latency_plot_df = (
    split_latency_comparison_df[
```

```
        [
            "stream",
            "stage1_worst_case_latency_ms",
            "stage2_worst_case_latency_ms"
        ]
    ]
    .set_index(
        "stream"
    )
)


worst_latency_plot_df.plot(
    kind="bar",
    figsize=(8, 5)
)


plt.ylabel(
    "Worst Used-Path Latency (ms)"
)

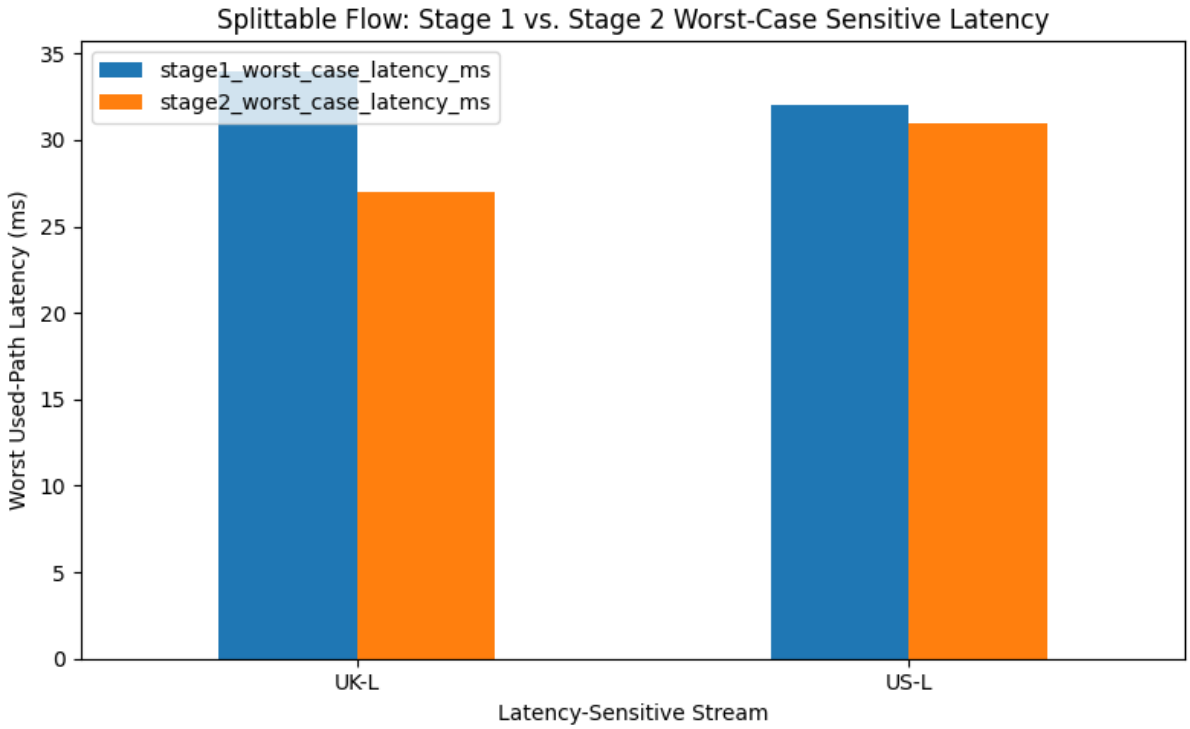
plt.xlabel(
    "Latency-Sensitive Stream"
)

plt.title(
    "Splittable Flow: "
    "Stage 1 vs. Stage 2 Worst-Case Sensitive Latency"
)

plt.xticks(
    rotation=0
)

plt.tight_layout()

plt.show()
```



34.18.2 Interpretation of the Latency Visualizations

The average-latency chart shows substantial improvement for both sensitive streams.

UK-L experiences the larger absolute reduction:

$$6.375 \text{ ms}.$$

US-L improves by approximately:

$$5.78 \text{ ms}.$$

The worst-case chart shows that the improvement is more pronounced for UK-L.

UK-L's worst used path falls from:

$$34$$

to:

$$27 \text{ ms}.$$

US-L's worst used-path latency improves only slightly:

$$32 \rightarrow 31 \text{ ms}.$$

This illustrates why reporting both average and worst-case latency is important.

An optimization can substantially improve the average customer experience while

leaving a small portion of traffic on a relatively slower route.

34.19 Throughput vs. Latency KPI Summary

In [296…

```
# Build compact Stage 2 executive KPI table.

split_stage2_executive_df = pd.DataFrame([
    {
        "metric":
            "Stage 1 Throughput",
        "value":
            split_stage1_throughput,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "Stage 2 Throughput",
        "value":
            split_stage2_total_throughput,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "UK-L Final Average Latency",
        "value":
            float(
                split_latency_results.loc[
                    split_latency_results[
                        "stream"
                    ]
                    == "UK-L",
                    "average_latency_ms"
                ].iloc[0]
            ),
        "unit":
            "ms"
    },
    {
        "metric":
            "US-L Final Average Latency",
        "value":
            float(
                split_latency_results.loc[
                    split_latency_results[
                        "stream"
                    ]
                    == "US-L",
                    "average_latency_ms"
                ].iloc[0]
```

```
            ),
        "unit":
            "ms"
    },
    {
        "metric":
            "Total Sensitive Latency Exposure Reduction",
        "value":
            latency_exposure_reduction_pct,
        "unit":
            "%"
    },
    {
        "metric":
            "Commodity Delivery Changes",
        "value":
            int(
                (
                    stage1_vs_stage2_delivery_df[
                        "delivery_change_mbps"
                    ].abs()
                    > 1e-6
                ).sum()
            ),
        "unit":
            "commodities"
    }
])


split_stage2_executive_df
```

Out[296…

	metric	value	unit
0	Stage 1 Throughput	83.000000	Mbps
1	Stage 2 Throughput	83.000000	Mbps
2	UK-L Final Average Latency	18.250000	ms
3	US-L Final Average Latency	19.555556	ms
4	Total Sensitive Latency Exposure Reduction	24.235294	%
5	Commodity Delivery Changes	0.000000	commodities

34.20 Business and Operational Interpretation

Stage 2 demonstrates that meaningful service-quality improvement can be achieved without sacrificing throughput.

The network continues to deliver:

$$83 \text{ Mbps}$$

and every source-stream commodity retains exactly the same admission quantity.

However, average latency falls by approximately:

$$25.9\%$$

for UK-L and:

$$22.8\%$$

for US-L.

Across the two sensitive classes, total traffic-weighted latency exposure falls by approximately:

$$24.2\%.$$

Operational Value

This means the network can provide materially better latency-sensitive service through smarter route allocation alone.

No additional bandwidth is required to obtain this particular improvement.

Cost of the Improvement

The improvement does consume more routing flexibility.

Several internal links become saturated as tolerant traffic is moved away from the best sensitive routes.

Therefore, the latency improvement is not free in a resilience sense.

The final network may have less spare capacity on certain internal corridors.

Policy Interpretation

The model demonstrates a clean service hierarchy:

1. maximize the amount of traffic served;
2. protect the latency quality of sensitive traffic;
3. allow tolerant traffic to absorb routing inconvenience.

That policy is transparent and consistent with the stated problem requirements.

34.21 Important Stage 2 Modeling Caveat

Stage 2 minimizes latency only for:

- UK-L;
- US-L.

Latency for:

- UK-T;
- US-T;

does not appear in the objective.

As a result, the LP may choose a long tolerant route even when another equally valid Stage 2 solution could route that tolerant traffic more naturally.

This is a form of secondary-objective degeneracy.

It does not invalidate the sensitive-latency result.

However, it means the final tolerant routes should not automatically be interpreted as operationally ideal.

Potential Production Extension

A third lexicographic optimization stage could be added:

Stage 1

Maximize throughput.

Stage 2

Minimize latency-sensitive traffic latency.

Stage 3

Preserve both prior optimal values and minimize:

- tolerant latency;
- total route length;
- number of active paths;
- or another operational-complexity metric.

That would create cleaner final routing without sacrificing either throughput or sensitive latency.

34.22 Detailed Section Summary

Section 34 completes the second stage of the splittable-flow optimization.

The purpose of Stage 2 is fundamentally different from Stage 1.

Stage 1 answered:

> **How much traffic can the network deliver?**

Stage 2 answers:

> **Given that exact admission decision, how should the admitted traffic be routed to improve latency-sensitive performance?**

---

1. Throughput Is Fully Preserved

Stage 1 delivered:

$$83 \text{ Mbps}.$$

Stage 2 also delivers:

$$83 \text{ Mbps}.$$

Therefore:

$$\Delta T = 0.$$

No throughput is sacrificed for lower latency.

---

2. Every Commodity Admission Is Preserved

For every source-stream commodity:

$$z_{k,s}^{(2)} = z_{k,s}^{(1)}.$$

The maximum observed difference is effectively zero.

Therefore, Stage 2 does not improve latency by:

- dropping different traffic;

- admitting previously rejected traffic;
- exchanging one customer or stream for another.

Only the path allocation changes.

---

3. Sensitive Traffic Moves to Better Routes

The most important routing changes move latency-sensitive traffic toward the low-latency C corridor.

For example, B / UK-L moves from the slower D route to:

$$B \to C \to UK.$$

Likewise, B / US-L moves onto:

$$B \to C \to US.$$

These changes consume attractive C-side capacity.

---

4. Tolerant Traffic Provides Routing Flexibility

To create room for the sensitive traffic, Stage 2 moves tolerant traffic onto alternative paths.

Some tolerant flows are split across multiple routes.

Some use substantially longer paths through A, B, C, or D.

This behavior is expected because tolerant latency is not penalized by the Stage 2 objective.

---

5. UK-L Latency Improves Substantially

UK-L remains at:

$$16 \text{ Mbps}$$

delivered.

Its traffic-weighted average latency improves from:

$$24.625 \text{ ms}$$

to:

$$18.25 \text{ ms}.$$

The reduction is:

$$6.375 \text{ ms}$$

or approximately:

$$25.9\%.$$

Worst used-path latency improves from:

$$34$$

to:

$$27 \text{ ms}.$$

---

6. US-L Latency Also Improves Substantially

US-L remains at:

$$18 \text{ Mbps}$$

delivered.

Average latency improves from approximately:

$$25.33$$

to:

$$19.56 \text{ ms}.$$

The reduction is approximately:

$$5.78 \text{ ms}$$

or:

$$22.8\%.$$

Worst used-path latency improves from:

$$32$$

to:

$$31 \text{ ms}.$$

---

7. Combined Sensitive Latency Exposure Falls

Combined UK-L and US-L traffic-weighted latency exposure falls from approximately:

$$850$$

to:

$$644 \text{ Mbps} \cdot \text{ms}.$$

The reduction is:

$$206 \text{ Mbps} \cdot \text{ms}$$

or approximately:

$$24.2\%.$$

This provides a useful single measure of Stage 2's overall service-quality improvement.

---

8. The Network Utilization Pattern Changes

Stage 1 contained three saturated directed links.

Stage 2 uses a broader set of internal links aggressively.

The final solution saturates:

- A → B;
- A → C;
- B → C;
- B → D;
- C → UK;
- C → US.

This shows that reducing sensitive latency uses more of the network's routing flexibility.

---

9. Better Latency Does Not Mean Lower Utilization

Stage 2 does not create additional capacity.

It redistributes the existing 83 Mbps.

Therefore, improving service quality for one class may require moving other traffic onto less convenient routes and consuming more capacity elsewhere.

This is an important operational tradeoff.

---

10. Stage 2 Demonstrates the Value of Lexicographic Optimization

The two-stage process provides a clear priority hierarchy.

First:

$$\max \text{Throughput}.$$

Then:

$$\min \text{Sensitive Latency}$$

while preserving the throughput-optimal admission vector.

This avoids an arbitrary throughput-versus-latency weighting parameter.

It also makes every performance improvement easier to interpret.

---

11. Technical Conclusion

The Stage 2 LP behaves exactly as intended.

It:

- preserves the 83 Mbps Stage 1 throughput;
- preserves all 20 source-stream admission quantities;
- satisfies all directional capacities;
- reroutes traffic within the feasible path set;
- reduces UK-L average latency by approximately 25.9%;
- reduces US-L average latency by approximately 22.8%;
- reduces combined sensitive latency exposure by approximately 24.2%.

The result therefore demonstrates a genuine latency improvement without any throughput sacrifice.

---

12. Business Conclusion

The network can improve latency-sensitive service materially through routing alone.

No additional physical bandwidth is required to obtain the Stage 2 latency benefit.

The operational policy is straightforward:

> **Use scarce low-latency capacity for traffic that benefits from it most, and use tolerant traffic as routing flexibility.**

However, the final routing also consumes more internal network headroom.

Therefore, throughput, latency, and resilience should ultimately be evaluated together.

---

13. Modeling Caveat

Because tolerant-traffic latency is absent from the objective, some tolerant routes may be unnecessarily long.

This is not a failure of the Stage 2 formulation.

It reflects exactly what was optimized.

A production system could add a tertiary objective to clean up tolerant routing after throughput and sensitive latency have both been optimized.

---

14. Connection to Section 35

Sections 32–34 now establish the complete **splittable-flow benchmark**.

The model has shown that:

- maximum throughput is 83 Mbps;
- Stage 2 can substantially improve sensitive latency;
- admitted traffic may be redistributed across multiple routes.

The next major question is whether this routing flexibility is actually necessary.

Section 35 therefore introduces the single-path MILP and asks:

> **What changes when every source-stream commodity is restricted to at most one selected route?**

This begins the direct comparison between:

splittable routing

and:

$$\text{single-path routing.}$$

35. Single-Path MILP — Stage 1: Maximize Throughput

Sections 32–34 established the complete splittable-flow benchmark.

The splittable model showed that:

- the network can deliver at most 83 Mbps from 126 Mbps of offered demand;
- the maximum-throughput routing can use any combination of feasible paths;
- Stage 2 can substantially reduce sensitive-traffic latency while preserving the exact Stage 1 admission decision.

Section 35 now introduces the second routing formulation required by the problem statement.

The network, demand, capacities, latencies, and candidate paths remain unchanged.

The only change is the routing policy:

> **Each source-stream commodity may select at most one path.**

This creates a stricter routing rule.

Under the splittable formulation, one commodity may distribute traffic across several routes.

Under the single-path formulation, each commodity may use at most one route, although partial delivery along that selected route is still allowed.

The purpose of this section is to determine whether that operational restriction reduces the maximum amount of traffic the network can deliver.

35.1 Mathematical Formulation

The single-path model requires two types of decision variables.

Continuous Path-Flow Variable

For every candidate path:

$$f_{k,s,p} \geq 0$$

where:

- k identifies the traffic stream;
- s identifies the source node;
- p identifies a candidate path;
- $f_{k,s,p}$ is the Mbps routed on that path.

Binary Path-Selection Variable

For every candidate path:

$$y_{k,s,p} \in \{0, 1\}$$

where:

$$y_{k,s,p} = 1$$

means the path is selected for that commodity.

The one-path rule is:

$$\sum_{p \in P_{k,s}} y_{k,s,p} \leq 1.$$

Therefore, each source-stream commodity may select at most one candidate route.

Flow-to-Selection Linking Constraint

Traffic may flow on a route only if that route is selected:

$$f_{k,s,p} \leq D_{k,s} y_{k,s,p}.$$

If:

$$y_{k,s,p} = 0,$$

then:

$$f_{k,s,p} = 0.$$

If:

$$y_{k,s,p} = 1,$$

then:

$$0 \leq f_{k,s,p} \leq D_{k,s}.$$

This means **partial delivery remains allowed**.

The model does not require:

$$f_{k,s,p} = D_{k,s}$$

when a path is selected.

Stage 1 Objective

As in the splittable model, the objective is:

$$\boxed{\max \sum_{k} \sum_{s} \sum_{p \in P_{k,s}} f_{k,s,p}}$$

Therefore, the only difference between the two Stage 1 formulations is the routing restriction.

This makes the throughput comparison between split and single-path routing clean and interpretable.

35.2 Why the Single-Path Problem Is a Mixed-Integer Linear Program

The splittable Stage 1 model was a linear program because all decision variables were continuous.

The single-path formulation adds binary variables:

$$y_{k,s,p} \in \{0, 1\}.$$

These binary decisions determine which route is selected.

The model therefore contains both:

- continuous traffic-flow variables;
- discrete path-selection variables.

As a result, the problem becomes a:

Mixed-Integer Linear Program (MILP).

The mathematical expressions remain linear, but the presence of binary variables makes the feasible solution space discrete.

This generally increases computational complexity compared with the splittable LP.

Business and Operational Impact

The single-path restriction represents an operational simplification.

A network may prefer one path per commodity because it can be easier to:

- implement;
- monitor;
- troubleshoot;
- explain;
- enforce consistently.

However, that operational simplicity may come at a performance cost.

The purpose of Sections 35–39 is to determine whether that cost is material in this network.

35.3 Create the Single-Path Stage 1 Model

The Stage 1 single-path model is created as a throughput-maximization problem.

At this point, the model contains no decision variables or constraints.

Those components are added explicitly in the following subsections.

In [297…

```
# ------------------------------------------------------------
# Create the Single-Path Stage 1 optimization model
# ------------------------------------------------------------
#
# The objective will be to maximize total delivered traffic.
#
# Because the model includes binary path-selection variables,
# the completed formulation will be a MILP.

single_stage1 = pl.LpProblem(
    "Single_Path_Stage1",
    pl.LpMaximize
)
```

```
print(
    "Model created:",
    single_stage1.name
)

print(
    "Optimization sense:",
    "Maximize"
)
```

```
Model created: Single_Path_Stage1
Optimization sense: Maximize
```

35.3.1 Interpretation

The optimization container has been created successfully.

The network inputs remain exactly the same as in the splittable formulation.

Therefore, any difference in the final solution will be caused by the single-path routing restriction rather than by a change in:

- topology;
- demand;
- capacity;
- latency;
- candidate paths.

35.4 Create Continuous Flow and Binary Selection Variables

The single-path formulation uses two variables for every candidate path.

For each of the 68 candidate source-stream paths:

1. one continuous flow variable;
2. one binary path-selection variable.

Therefore, the model should contain:

$$68$$

continuous variables and:

$$68$$

binary variables.

The total number of decision variables is:

$$136.$$

In [298…

```
# ------------------------------------------------------------
# Create dictionaries for the two variable types
# ------------------------------------------------------------

# Continuous flow variables:
#
#   path_id -> Mbps carried on that path

single_flow_vars = {}


# Binary path-selection variables:
#
#   path_id -> 1 if selected, 0 otherwise

single_select_vars = {}


# ------------------------------------------------------------
# Create one pair of variables for every candidate path
# ------------------------------------------------------------

for _, row in (
    candidate_paths_df.iterrows()
):

    # Retrieve the unique path identifier.

    pid = int(
        row[
            "path_id"
        ]
    )


    # ------------------------------------------------------------
    # Continuous flow variable
    # ------------------------------------------------------------
    #
    # This variable represents how many Mbps are sent
    # over the candidate route.

    single_flow_vars[
        pid
    ] = (
        pl.LpVariable(
```

```
            f"single_flow_{pid}",
            lowBound=0,
            cat="Continuous"
        )
    )


    # ------------------------------------------------------------
    # Binary path-selection variable
    # ------------------------------------------------------------
    #
    # select_pid = 1 means the candidate path is selected.
    #
    # select_pid = 0 means the path may carry no traffic.

    single_select_vars[
        pid
    ] = (
        pl.LpVariable(
            f"select_{pid}",
            lowBound=0,
            upBound=1,
            cat="Binary"
        )
    )


print(
    "Continuous flow variables:",
    len(
        single_flow_vars
    )
)

print(
    "Binary selection variables:",
    len(
        single_select_vars
    )
)

print(
    "Total decision variables:",
    len(
        single_flow_vars
    )
    + len(
        single_select_vars
    )
)
```

```
Continuous flow variables: 68
Binary selection variables: 68
Total decision variables: 136
```

35.4.1 Validate the Variable Structure

In [299…

```
# ------------------------------------------------------------
# Validate continuous-variable count
# ------------------------------------------------------------

assert (
    len(
        single_flow_vars
    )
    ==
    len(
        candidate_paths_df
    )
)


# ------------------------------------------------------------
# Validate binary-variable count
# ------------------------------------------------------------

assert (
    len(
        single_select_vars
    )
    ==
    len(
        candidate_paths_df
    )
)


# ------------------------------------------------------------
# Confirm the path IDs match both variable dictionaries
# ------------------------------------------------------------

expected_path_ids = set(
    candidate_paths_df[
        "path_id"
    ].astype(int)
)


assert (
    set(
        single_flow_vars.keys()
    )
    ==
```

```
    expected_path_ids
)


assert (
    set(
        single_select_vars.keys()
    )
    ==
    expected_path_ids
)


print(
    "Single-path variable validation passed."
)
```

```
Single-path variable validation passed.
```

35.4.2 Interpretation of Variable Creation

The MILP contains:

$$68$$

continuous flow variables and:

$$68$$

binary route-selection variables.

The total is:

$$136$$

decision variables.

Compared with the splittable Stage 1 LP:

$$68$$

additional binary decisions have been introduced.

Those binary variables are what enforce the operational rule that a commodity may use at most one route.

34.4.3 Business and Operational Impact

The single-path rule makes network behavior simpler but makes the optimization model more complex.

This creates an interesting tradeoff:

- **simpler routing behavior in the network**;
- **harder mathematical optimization behind the scenes**.

For the current small network, that additional complexity is manageable.

For a large production network with many thousands of candidate routes, binary-variable growth could become a significant computational consideration.

35.5 Define the Stage 1 Throughput Objective

The throughput objective is intentionally identical to the splittable Stage 1 objective.

The model maximizes:

$$\boxed{\sum_p f_p}$$

where each path-flow variable represents delivered Mbps.

Keeping the objective identical ensures that the later throughput comparison isolates the effect of the routing restriction.

In [300…
```
# ------------------------------------------------------------
# Add maximum-throughput objective
# ------------------------------------------------------------

single_stage1 += pl.lpSum(
    single_flow_vars[
        pid
    ]
    for pid
    in single_flow_vars
)


print(
    "Single-path Stage 1 throughput objective added."
)
```

```
Single-path Stage 1 throughput objective added.
```

35.5.1 Interpretation

The model places the same value on every delivered Mbps as the splittable

formulation.

There is still no:

- latency penalty;
- fairness term;
- source priority;
- traffic-class admission priority.

Therefore, any change in maximum throughput is attributable solely to the one-path restriction.

35.6 Enforce At Most One Selected Path per Commodity

The defining restriction of the single-path formulation is:

$$\sum_{p \in P_{k,s}} y_{k,s,p} \leq 1.$$

This allows three possibilities for a commodity:

1. no route is selected and no traffic is delivered;
2. one route is selected and demand is partially delivered;
3. one route is selected and demand is fully delivered.

What is prohibited is selecting two or more paths for the same commodity.

In [302…

```
# ------------------------------------------------------------
# Add one path-selection constraint per commodity
# ------------------------------------------------------------

for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    # Sum all binary route-selection variables
    # belonging to this commodity.
    #
    # The sum may not exceed one.

    single_stage1 += (
        pl.lpSum(
            single_select_vars[
                int(pid)
```

```
            ]
            for pid
            in group[
                "path_id"
            ]
        )
        <= 1,

        f"one_path_{commodity_id}"
    )


print(
    "One-path constraints added:",
    demand_df[
        "commodity_id"
    ].nunique()
)
```

```
One-path constraints added: 20
```

35.6.1 Why the One-Path Constraint Uses an Inequality

The constraint is:

$$\sum_p y_{k,s,p} \leq 1$$

rather than:

$$\sum_p y_{k,s,p} = 1.$$

This is intentional.

A commodity whose traffic cannot be admitted does not need to select a route.

If equality were required, every commodity would have to choose a path even when:

$$f_{k,s,p} = 0.$$

Using:

$$\leq 1$$

therefore represents the operational logic more naturally:

> a commodity may use zero paths or one path, but never more than one.

35.6.2 Business and Operational Impact

This formulation avoids creating meaningless route assignments for completely unserved traffic.

A route selection therefore has operational significance only when the model actually chooses to use that path.

35.7 Link Traffic Flow to Path Selection

The binary selection variable alone does not prevent flow from appearing on an unselected path.

A linking constraint is therefore required:

$$f_{k,s,p} \leq D_{k,s} y_{k,s,p}.$$

This is sometimes referred to as a Big-M linking constraint.

Here, the natural value of the upper bound is the commodity's own demand:

$$M = D_{k,s}.$$

Using demand as the bound is tighter and more interpretable than using an unnecessarily large arbitrary constant.

In [304…

```
# ------------------------------------------------------------
# Add flow-to-selection linking constraints
# ------------------------------------------------------------

for _, row in (
    candidate_paths_df.iterrows()
):

    # Candidate-path identifier.

    pid = int(
        row[
            "path_id"
        ]
    )


    # Maximum possible flow on this path is bounded
    # by the commodity's total offered demand.

    demand = float(
```

```
        row[
            "demand_mbps"
        ]
    )


    # ------------------------------------------------------------
    # Linking logic
    # ------------------------------------------------------------
    #
    # If:
    #
    #   select_pid = 0
    #
    # then:
    #
    #   flow_pid <= 0
    #
    # and since flow_pid >= 0:
    #
    #   flow_pid = 0
    #
    # If:
    #
    #   select_pid = 1
    #
    # then:
    #
    #   flow_pid <= demand
    #
    # which permits either partial or full delivery.

    single_stage1 += (
        single_flow_vars[
            pid
        ]
        <=
        demand
        * single_select_vars[
            pid
        ],

        f"flow_select_{pid}"
    )


print(
    "Flow-selection linking constraints added:",
    len(
        candidate_paths_df
    )
)
```

```
Flow-selection linking constraints added: 68
```

35.7.1 Partial Delivery Remains Allowed

Suppose a commodity has:

$$D_{k,s} = 7 \text{ Mbps}.$$

If its selected route is marked:

$$y_{k,s,p} = 1,$$

the linking constraint becomes:

$$f_{k,s,p} \leq 7.$$

The model may therefore choose:

$$f_{k,s,p} = 2$$

if only 2 Mbps can be supported.

This is valid.

The one-path restriction controls **route count**, not whether the entire demand must be delivered.

This matches the clarification that partial delivery remains allowed in the single-path variant.

35.7.2 Business and Operational Impact

Partial delivery prevents the model from becoming artificially rigid.

The network can still use whatever residual capacity is available on the selected route instead of rejecting the entire commodity simply because full demand cannot be accommodated.

Operationally, the model represents:

> **one route per commodity, but not necessarily all-or-nothing service.**

35.8 Add Directed Link-Capacity Constraints

The physical network constraints are identical to the splittable formulation.

For each directed arc:

$$\sum_{k,s} \sum_{\substack{p \in P_{k,s} \\ e \in p}} f_{k,s,p} \leq C_e.$$

All commodities whose selected routes use a directed link therefore compete for the same finite capacity.

The full-duplex structure remains directional.

For example:

$$A \to C$$

and:

$$C \to A$$

have independent capacity constraints.

In [306…

```
# ------------------------------------------------------------
# Add one capacity constraint for every directed arc
# ------------------------------------------------------------

for _, arc in (
    arcs_df.iterrows()
):

    # Represent the current directional link.

    edge = (
        arc[
            "from_node"
        ],
        arc[
            "to_node"
        ]
    )


    # Find every candidate path that contains this arc.

    using_edge = (
        candidate_paths_df[
            candidate_paths_df[
                "path_edges"
            ].apply(
                lambda edges:
                    edge in edges
            )
```

```
        ]
    )


    # Sum all single-path flow variables whose route
    # uses this directed arc.
    #
    # The total cannot exceed available directional capacity.

    single_stage1 += (
        pl.lpSum(
            single_flow_vars[
                int(pid)
            ]
            for pid
            in using_edge[
                "path_id"
            ]
        )
        <=
        float(
            arc[
                "capacity_mbps"
            ]
        ),

        f"capacity_"
        f"{arc['from_node']}_"
        f"{arc['to_node']}"
    )


print(
    "Directed capacity constraints added:",
    len(
        arcs_df
    )
)
```

```
Directed capacity constraints added: 22
```

35.9 Validate the Single-Path MILP Structure

The expected model dimensions can be derived before solving.

Variables

The model contains:

continuous flow variables and:

$$68$$

binary selection variables.

Therefore:

$$136$$

total decision variables.

Constraints

The model contains:

- 20 one-path constraints;
- 68 flow-selection linking constraints;
- 22 directed capacity constraints.

Therefore:

$$20 + 68 + 22 = 110$$

constraints are expected.

In [308…

```
# ------------------------------------------------------------
# Inspect actual model dimensions
# ------------------------------------------------------------

single_stage1_variable_count = (
    len(
        single_stage1.variables()
    )
)


single_stage1_constraint_count = (
    len(
        single_stage1.constraints
    )
)


print(
    "Single-path Stage 1 variables:",
    single_stage1_variable_count
)

print(
    "Single-path Stage 1 constraints:",
```

```
    single_stage1_constraint_count
)
```

```
Single-path Stage 1 variables: 136
Single-path Stage 1 constraints: 110
```

In [309…

```
##Validation
# Confirm actual model size matches the mathematical design.

assert (
    single_stage1_variable_count
    == 136
)


assert (
    single_stage1_constraint_count
    == 110
)


print(
    "Single-path Stage 1 model-dimension validation passed."
)
```

```
Single-path Stage 1 model-dimension validation passed.
```

35.9.1 Interpretation of Model Dimensions

The single-path Stage 1 model is structurally larger than the splittable Stage 1 LP.

Splittable Stage 1 contained:

$$68$$

variables and:

$$42$$

constraints.

Single-path Stage 1 contains:

$$136$$

variables and:

$$110$$

constraints.

The network and demand did not change.

The additional complexity is caused entirely by the operational requirement that each commodity use at most one route.

35.10 Solve the Single-Path Stage 1 MILP

The completed MILP is now solved using CBC.

The solver must determine both:

1. which route, if any, each commodity selects;
2. how much flow is delivered on that selected route.

The objective remains maximum total throughput.

In [310…

```
# ------------------------------------------------------------
# Solve the Single-Path Stage 1 MILP
# ------------------------------------------------------------

single_stage1.solve(
    cbc_solver
)


# Translate solver status into readable form.

single_stage1_status = (
    pl.LpStatus[
        single_stage1.status
    ]
)


# Retrieve optimized total delivered throughput.

single_stage1_throughput = (
    pl.value(
        single_stage1.objective
    )
)


print(
    "Status:",
    single_stage1_status
)

print(
    "Maximum single-path throughput:",
    single_stage1_throughput,
```

```
    "Mbps"
)
```

```
Status: Optimal
Maximum single-path throughput: 83.0 Mbps
```

35.10.1 Validate Solver Status and Objective

In [311…

```
# ------------------------------------------------------------
# Validate the optimization result
# ------------------------------------------------------------

# The model should only be interpreted if the solver
# proves optimality.

assert (
    single_stage1_status
    == "Optimal"
)


# Objective value must exist.

assert (
    single_stage1_throughput
    is not None
)


# Throughput cannot exceed total demand.

assert (
    single_stage1_throughput
    <= total_demand + 1e-6
)


# Throughput must be non-negative.

assert (
    single_stage1_throughput
    >= -1e-6
)


print(
    "Single-path Stage 1 solver validation passed."
)
```

```
Single-path Stage 1 solver validation passed.
```

35.11 Validate the One-Path-per-Commodity Rule

A solver status of "Optimal" does not by itself prove that the intended one-path logic was implemented correctly.

The optimized binary variables are therefore checked directly.

For every commodity:

$$\sum_{p} y_{k,s,p} \leq 1.$$

The number of positive-flow routes is also checked.

This verifies that no source-stream commodity is actually routed over multiple paths.

In [312…

```
# ------------------------------------------------------------
# Validate binary route selection by commodity
# ------------------------------------------------------------

single_path_validation_records = []


for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    # Count selected binary routes.

    selected_paths = sum(
        int(
            round(
                single_select_vars[
                    int(pid)
                ].value()
                or 0
            )
        )
        for pid
        in group[
            "path_id"
        ]
    )


    # Count positive-flow routes.
```

```
    positive_flow_paths = sum(
        1
        for pid
        in group[
            "path_id"
        ]
        if (
            single_flow_vars[
                int(pid)
            ].value()
            or 0
        ) > 1e-8
    )


    # Store validation record.

    single_path_validation_records.append({
        "commodity_id":
            commodity_id,
        "selected_paths":
            selected_paths,
        "positive_flow_paths":
            positive_flow_paths
    })


single_path_validation_df = (
    pd.DataFrame(
        single_path_validation_records
    )
)


single_path_validation_df
```

Out [312…

	commodity_id	selected_paths	positive_flow_paths
0	B_UK-L	1	1
1	B_UK-T	1	1
2	B_US-L	1	1
3	B_US-T	1	1
4	C_UK-L	1	1
5	C_UK-T	1	1
6	C_US-L	1	1
7	C_US-T	1	1
8	S1_UK-L	0	0
9	S1_UK-T	1	1
10	S1_US-L	1	1
11	S1_US-T	1	1
12	S2_UK-L	1	1
13	S2_UK-T	1	1
14	S2_US-L	0	0
15	S2_US-T	0	0
16	S3_UK-L	0	0
17	S3_UK-T	1	1
18	S3_US-L	0	0
19	S3_US-T	1	1

In [313…

```
###Assertions

# Every commodity must select at most one route.

assert (
    single_path_validation_df[
        "selected_paths"
    ]
    <= 1
).all()


# Every commodity must use at most one positive-flow route.

assert (
```

```
    single_path_validation_df[
        "positive_flow_paths"
    ]
    <= 1
).all()


print(
    "One-path routing validation passed."
)
```

```
One-path routing validation passed.
```

35.11.1 Interpretation of One-Path Validation

The validation confirms that every source-stream commodity selects and uses at most one route.

Therefore, the MILP is correctly implementing the required single-path policy.

This is important because the throughput value alone could not confirm whether traffic splitting had accidentally occurred.

35.12 Validate Flow-to-Selection Logic

The linking constraints should guarantee:

> positive flow implies selected path.

Therefore, every path with:

$$f_{k,s,p} > 0$$

must have:

$$y_{k,s,p} = 1.$$

This is validated directly.

In [314…

```
# ------------------------------------------------------------
# Check that positive flow never occurs on an unselected route
# ------------------------------------------------------------

flow_selection_violations = []


for _, row in (
    candidate_paths_df.iterrows()
):
```

```
    pid = int(
        row[
            "path_id"
        ]
    )


    flow = (
        single_flow_vars[
            pid
        ].value()
        or 0
    )


    selected = (
        single_select_vars[
            pid
        ].value()
        or 0
    )


    # Positive flow requires binary selection.

    if (
        flow > 1e-8
        and selected < 0.5
    ):

        flow_selection_violations.append(
            pid
        )


assert (
    len(
        flow_selection_violations
    )
    == 0
)


print(
    "Flow-to-selection validation passed."
)
```

```
Flow-to-selection validation passed.
```

35.13 Headline Single-Path Throughput Result

The single-path MILP reaches:

$$83 \text{ Mbps}$$

of delivered traffic from:

$$126 \text{ Mbps}$$

of offered demand.

Therefore:

$$43 \text{ Mbps}$$

remains undelivered.

The overall delivery rate is:

$$\frac{83}{126} \times 100 \approx 65.87\%.$$

In [315…

```
# Calculate headline single-path metrics.

single_stage1_total_demand = float(
    total_demand
)


single_stage1_delivered = float(
    single_stage1_throughput
)


single_stage1_lost = (
    single_stage1_total_demand
    - single_stage1_delivered
)


single_stage1_delivery_percentage = (
    100
    * single_stage1_delivered
    / single_stage1_total_demand
)


single_stage1_kpi_df = pd.DataFrame([
    {
        "metric":
            "Total Offered Demand",
        "value":
```

```
            single_stage1_total_demand,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "Maximum Delivered Throughput",
        "value":
            single_stage1_delivered,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "Undelivered Demand",
        "value":
            single_stage1_lost,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "Overall Delivery Rate",
        "value":
            single_stage1_delivery_percentage,
        "unit":
            "%"
    }
])


single_stage1_kpi_df
```

Out [315…

	metric	value	unit
0	Total Offered Demand	126.000000	Mbps
1	Maximum Delivered Throughput	83.000000	Mbps
2	Undelivered Demand	43.000000	Mbps
3	Overall Delivery Rate	65.873016	%

35.13.1 Interpretation of the Single-Path Throughput Result

The maximum single-path throughput is:

$$83 \text{ Mbps}.$$

This is exactly equal to the splittable Stage 1 optimum:

$$T^*_{\text{single}} = T^*_{\text{split}} = 83 \text{ Mbps}.$$

Therefore, for this baseline scenario:

$$T^*_{\text{split}} - T^*_{\text{single}} = 0.$$

The single-path routing restriction does **not** reduce maximum total throughput in the baseline network.

This is an important result.

The splittable formulation has a larger feasible routing space, but that additional flexibility is not needed to achieve the maximum 83 Mbps in this particular instance.

35.13.2 Why Equal Throughput Is Mathematically Consistent

Every valid single-path solution is also a valid splittable solution.

Therefore:

$$T^*_{\text{single}} \leq T^*_{\text{split}}.$$

The single-path model can never outperform the splittable model when all other assumptions are identical.

However, equality is entirely possible.

The baseline result:

$$83 = 83$$

means that at least one throughput-optimal splittable solution can be represented using one route per admitted commodity.

This is consistent with Section 33, where the Stage 1 splittable solution itself did not actually split any delivered commodity across multiple positive-flow paths.

35.13.3 Business and Operational Impact

The baseline result favors the operational simplicity of single-path routing.

In this scenario, the business can achieve the same:

$$83 \text{ Mbps}$$

maximum throughput without requiring commodity-level multipath routing.

If this result held across realistic operating conditions, the simpler single-path policy could be attractive because it may be easier to:

- implement;
- monitor;
- debug;
- explain;
- govern.

However, one baseline scenario is not sufficient to conclude that splitting has no value.

The later stress tests should include conditions with fragmented residual capacity, where multipath routing may outperform the single-path restriction.

35.14 Baseline Throughput Comparison: Split vs. Single Path

The first direct model comparison is now possible.

Both formulations use exactly the same:

- demand;
- network;
- capacities;
- candidate routes.

The only difference is routing flexibility.

The following chart compares their Stage 1 maximum throughput.

In [316…

```
# Build throughput comparison table.

baseline_throughput_comparison_df = pd.DataFrame([
    {
        "model":
            "Splittable Stage 1",
        "throughput_mbps":
            split_stage1_throughput
    },
    {
        "model":
            "Single-Path Stage 1",
        "throughput_mbps":
```

```
            single_stage1_throughput
    }
])


baseline_throughput_comparison_df
```

Out [316…

	model	throughput_mbps
0	Splittable Stage 1	83.0
1	Single-Path Stage 1	83.0

In [317…

```
# Visualize Stage 1 throughput for both routing formulations.

plt.figure(
    figsize=(8, 5)
)

plt.bar(
    baseline_throughput_comparison_df[
        "model"
    ],
    baseline_throughput_comparison_df[
        "throughput_mbps"
    ]
)

plt.ylabel(
    "Maximum Delivered Throughput (Mbps)"
)

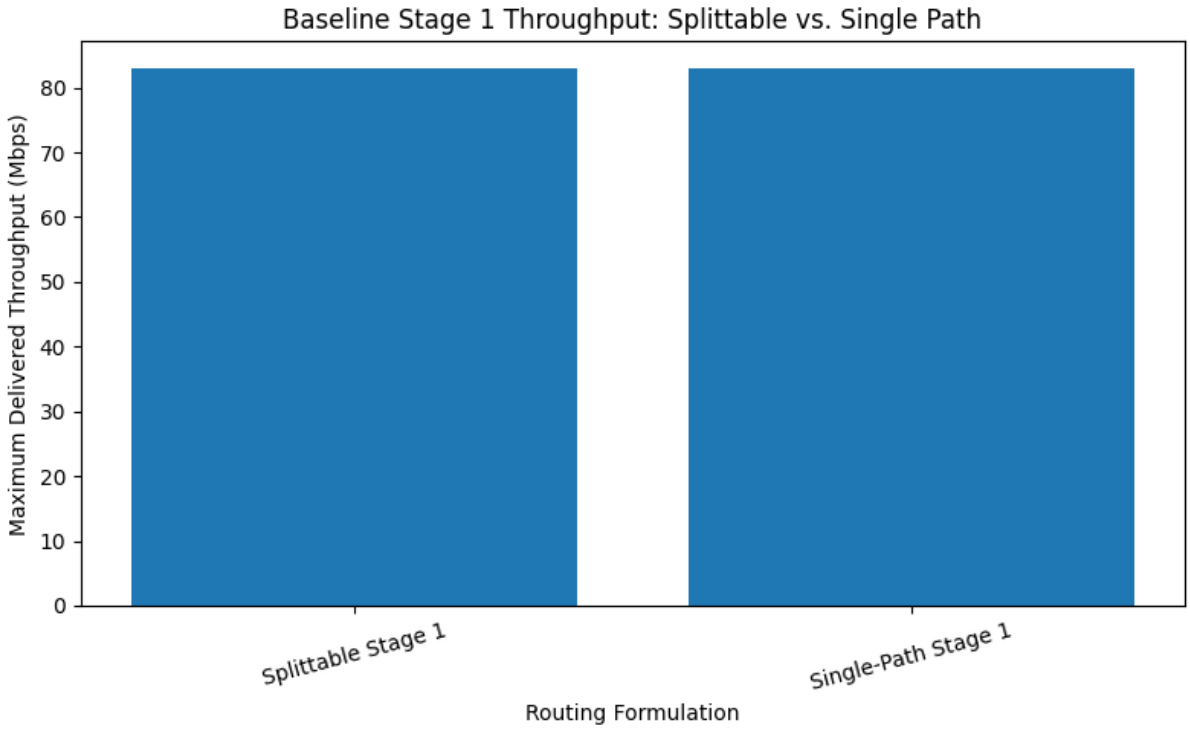
plt.xlabel(
    "Routing Formulation"
)

plt.title(
    "Baseline Stage 1 Throughput: "
    "Splittable vs. Single Path"
)

plt.xticks(
    rotation=15
)

plt.tight_layout()

plt.show()
```



35.14.1 Interpretation of the Throughput Comparison

The two bars are equal:

$$83 \text{ Mbps}$$

for both routing formulations.

The baseline split-throughput advantage is therefore:

$$0 \text{ Mbps}.$$

This means the network's baseline throughput ceiling is caused by structural capacity constraints that cannot be overcome simply by allowing commodity-level traffic splitting.

35.14.2 Business and Operational Impact

At baseline, the organization gains no additional throughput from the extra routing flexibility of the splittable model.

That does not mean the splittable model has no value.

It may still provide:

- lower latency;
- more routing options;

- better resilience;
- higher throughput under different demand patterns.

The correct business conclusion is therefore:

> **Single-path routing is sufficient to match baseline throughput, but broader scenario testing is still required before preferring the simpler policy.**

35.15 Throughput Benefit vs. Optimization Complexity

The baseline result creates an interesting tradeoff.

The single-path model achieves the same 83 Mbps throughput as the splittable LP, but requires a larger mathematical formulation.

This comparison helps separate:

- network-operational simplicity;
- optimization-computational simplicity.

In [318…

```
# Compare model dimensions.

stage1_model_complexity_df = pd.DataFrame([
    {
        "model":
            "Splittable Stage 1",
        "continuous_variables":
            68,
        "binary_variables":
            0,
        "total_variables":
            68,
        "constraints":
            42,
        "throughput_mbps":
            split_stage1_throughput
    },
    {
        "model":
            "Single-Path Stage 1",
        "continuous_variables":
            68,
        "binary_variables":
            68,
        "total_variables":
```

```
            136,
        "constraints":
            110,
        "throughput_mbps":
            single_stage1_throughput
    }
])


stage1_model_complexity_df
```

Out[318…

	model	continuous_variables	binary_variables	total_variables	constraints
0	Splittable Stage 1	68	0	68	42
1	Single-Path Stage 1	68	68	136	110

35.15.1 Interpretation of Complexity vs. Performance

The single-path MILP uses:

$$2\times$$

as many decision variables as the splittable Stage 1 LP:

$$136 \text{ vs. } 68.$$

It also contains substantially more constraints:

$$110 \text{ vs. } 42.$$

Despite the additional mathematical complexity, both models deliver:

$$83 \text{ Mbps}.$$

This illustrates an unusual but important tradeoff:

- single-path routing is operationally simpler;
- but mathematically more difficult to optimize.

35.16 What the Stage 1 Objective Value Does Not Yet Tell Us

The equal 83 Mbps throughput result is important, but it does not mean the two solutions are operationally identical.

The single-path MILP may:

- admit different commodities;
- choose different routes;
- saturate different links;
- produce different sensitive-traffic latency.

Therefore, the throughput comparison alone is not sufficient.

Section 36 will extract and validate the detailed single-path Stage 1 solution.

The analysis will examine:

- source-stream delivery;
- selected routes;
- partial delivery;
- link utilization;
- saturated links;
- network-flow structure.

35.17 Stage 1 Single-Path Executive Summary

In [319…

```
# Build compact Stage 1 MILP summary.

single_stage1_executive_df = pd.DataFrame([
    {
        "metric":
            "Total Offered Demand",
        "value":
            total_demand,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "Maximum Single-Path Throughput",
        "value":
            single_stage1_throughput,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "Undelivered Demand",
        "value":
            total_demand
            - single_stage1_throughput,
        "unit":
```

```
            "Mbps"
    },
    {
        "metric":
            "Delivery Rate",
        "value":
            100
            * single_stage1_throughput
            / total_demand,
        "unit":
            "%"
    },
    {
        "metric":
            "Continuous Variables",
        "value":
            len(
                single_flow_vars
            ),
        "unit":
            "variables"
    },
    {
        "metric":
            "Binary Variables",
        "value":
            len(
                single_select_vars
            ),
        "unit":
            "variables"
    },
    {
        "metric":
            "Total Constraints",
        "value":
            len(
                single_stage1.constraints
            ),
        "unit":
            "constraints"
    }
])


single_stage1_executive_df
```

Out [319...

	metric	value	unit
0	Total Offered Demand	126.000000	Mbps
1	Maximum Single-Path Throughput	83.000000	Mbps
2	Undelivered Demand	43.000000	Mbps
3	Delivery Rate	65.873016	%
4	Continuous Variables	68.000000	variables
5	Binary Variables	68.000000	variables
6	Total Constraints	110.000000	constraints

35.18 Detailed Section Summary

Section 35 introduces the single-path routing formulation required by the problem statement.

The physical network, demand, capacities, latencies, and candidate paths are unchanged from the splittable model.

The only new policy rule is:

> **Each source-stream commodity may use at most one selected route.**

---

1. Single-Path Decision Structure

Each candidate path receives:

- one continuous flow variable:

$$f_{k,s,p} \geq 0$$

- one binary selection variable:

$$y_{k,s,p} \in \{0, 1\}.$$

The one-path rule is:

$$\sum_{p \in P_{k,s}} y_{k,s,p} \leq 1.$$

This guarantees that each commodity may use zero or one route, but never

multiple routes.

---

2. Partial Delivery Is Preserved

The flow-selection linking constraint is:

$$f_{k,s,p} \leq D_{k,s} y_{k,s,p}.$$

If a route is not selected:

$$y_{k,s,p} = 0 \Rightarrow f_{k,s,p} = 0.$$

If a route is selected:

$$y_{k,s,p} = 1,$$

then any amount between zero and total demand may be delivered.

Therefore, the model satisfies both requirements:

- one path per source-stream commodity;
- partial delivery remains allowed.

---

3. The MILP Is Larger Than the Splittable LP

The single-path Stage 1 model contains:

$$68$$

continuous flow variables and:

$$68$$

binary path-selection variables.

Therefore:

$$136$$

decision variables are required.

The model also contains:

- 20 one-path constraints;
- 68 flow-selection linking constraints;
- 22 directed capacity constraints.

For a total of:

$$110$$

constraints.

By comparison, the splittable Stage 1 LP contained:

$$68$$

variables and:

$$42$$

constraints.

Therefore, the one-path routing policy approximately doubles the number of decision variables and substantially increases the constraint count.

---

4. The MILP Solves Optimally

CBC returns:

$$\text{Optimal.}$$

The maximum single-path throughput is:

$$\boxed{83 \text{ Mbps}}.$$

From:

$$126 \text{ Mbps}$$

of offered demand, the model therefore leaves:

$$43 \text{ Mbps}$$

undelivered.

The overall delivery rate remains:

$$65.87\%.$$

---

5. The One-Path Restriction Does Not Reduce Baseline Throughput

The splittable Stage 1 model also delivered:

$$83 \text{ Mbps}.$$

Therefore:

$$T^*_{\text{single}} = T^*_{\text{split}} = 83.$$

The baseline throughput advantage of allowing traffic splitting is:

$$0 \text{ Mbps}.$$

This is mathematically consistent because the splittable formulation contains all single-path solutions within its larger feasible solution space.

The equality means the extra routing flexibility is not necessary to achieve maximum throughput in this particular baseline scenario.

---

6. This Result Is Consistent with Section 33

Section 33 showed that although the splittable Stage 1 model was allowed to divide a commodity across several routes, the optimal solution returned by CBC did not actually require any delivered commodity to use multiple positive-flow paths.

That observation anticipated the Section 35 result.

The network's baseline:

$$83 \text{ Mbps}$$

throughput optimum can be achieved using one route per admitted commodity.

---

7. Operational Simplicity Does Not Mean Mathematical Simplicity

The single-path policy may be operationally easier to implement because each commodity follows one route.

However, the optimization problem itself becomes more difficult because route choices are binary.

This creates an important architecture tradeoff:

> **Single-path routing simplifies network behavior but increases optimization complexity.**

For the small nine-node synthetic network, this additional complexity is manageable.

At larger scale, it may influence:

- solver runtime;
- re-optimization frequency;
- infrastructure requirements.

---

8. Baseline Business Interpretation

The baseline result suggests that the organization could enforce one route per source-stream commodity without sacrificing total throughput.

That is potentially valuable because simpler routing may improve:

- observability;
- troubleshooting;
- operational explainability;
- policy enforcement.

However, the result should not yet be generalized.

The absence of a baseline throughput advantage does not prove that multipath routing lacks value.

Different network conditions may create fragmented residual capacity where splitting becomes necessary to achieve greater throughput.

---

9. Important Remaining Questions

The objective value alone does not reveal whether the single-path solution behaves like the splittable solution.

It may:

- serve different commodities;
- choose different routes;
- create different link utilization;
- produce different latency-sensitive performance.

Those differences matter operationally even when total throughput is identical.

---

10. Technical Conclusion

The single-path Stage 1 MILP is correctly constructed and validated.

It:

- uses continuous traffic-flow variables;
- uses binary route-selection variables;
- enforces at most one selected route per commodity;
- allows partial delivery;
- respects directional capacity constraints;
- achieves an optimal maximum throughput of 83 Mbps.

No commodity uses more than one positive-flow route.

---

11. Business Conclusion

For the baseline network, enforcing one route per commodity does not reduce total throughput.

Therefore, the additional routing flexibility of the splittable model provides no baseline throughput benefit.

However, the two formulations may still differ in:

- admission mix;
- latency;
- congestion;
- resilience;
- response to stress scenarios.

The simpler single-path policy should therefore not be preferred solely on the basis of this one throughput result.

---

12. Connection to Section 36

Section 35 establishes:

> **How much traffic can the single-path model deliver?**

The answer is:

$$83 \text{ Mbps}.$$

Section 36 now asks:

> **Which traffic does it deliver, which route does each admitted commodity use, and what happens to the network when splitting is prohibited?**

That detailed analysis will determine whether equal total throughput also means similar operational behavior.

36. Single-Path Stage 1 Results and Validation

Section 35 established that the single-path MILP reaches an optimal maximum throughput of:

$$83 \text{ Mbps}$$

from:

$$126 \text{ Mbps}$$

of offered demand.

This exactly matches the maximum throughput obtained by the splittable Stage 1 model.

However, equal objective values do not imply identical solutions.

The single-path formulation imposes an additional operational restriction:

> Each source-stream commodity may use at most one route.

Section 36 therefore examines the detailed single-path Stage 1 result.

The goals of this section are to determine:

1. whether the theoretical dominance relationship between split and single-path routing is satisfied;
2. whether every commodity actually uses at most one positive-flow route;
3. which source-stream commodities are delivered, partially delivered, or not delivered;
4. which routes are selected;
5. how the single-path solution uses network capacity;
6. which links become saturated;

7. how the single-path Stage 1 traffic pattern differs from the splittable Stage 1 solution;
8. whether equal total throughput masks materially different operational behavior.

This section therefore moves from the Stage 1 objective value to the actual network operating point produced by the MILP.

36.1 Validate the Split-Flow Dominance Relationship

The splittable formulation has a larger feasible solution space than the single-path formulation.

Every valid single-path routing solution is also feasible in the splittable model.

However, the reverse is not necessarily true because the splittable model may divide one commodity across several paths.

Therefore, mathematically:

$$\boxed{T^*_{\text{single}} \leq T^*_{\text{split}}}$$

where:

- T^*_{single} is optimal single-path throughput;
- T^*_{split} is optimal splittable throughput.

This relationship provides an important cross-model validation check.

If the single-path model were to produce greater throughput than the splittable model under identical inputs, that would indicate an implementation inconsistency.

In [321…

```
# ------------------------------------------------------------
# 36.1 Validate theoretical split-flow dominance
# ------------------------------------------------------------
#
# Because the splittable model permits everything the
# single-path model permits, plus additional multipath
# routing flexibility, we must have:
#
#     single throughput <= split throughput
```

```
print(
    "Split throughput:",
    split_stage1_throughput,
    "Mbps"
)

print(
    "Single-path throughput:",
    single_stage1_throughput,
    "Mbps"
)


# Validate the theoretical dominance relationship.

assert (
    single_stage1_throughput
    <=
    split_stage1_throughput
    + 1e-6
)


# Calculate the observed split-routing throughput advantage.

split_throughput_advantage = (
    split_stage1_throughput
    - single_stage1_throughput
)


print(
    "Split throughput advantage:",
    split_throughput_advantage,
    "Mbps"
)

print(
    "Split-dominance validation passed."
)
```

```
Split throughput: 83.0 Mbps
Single-path throughput: 83.0 Mbps
Split throughput advantage: 0.0 Mbps
Split-dominance validation passed.
```

Interpretation of the Dominance Check

The validation confirms:

$$T^*_{\text{single}} = T^*_{\text{split}} = 83 \text{ Mbps}.$$

Therefore:

$$T^*_{\text{split}} - T^*_{\text{single}} = 0 \text{ Mbps}.$$

The single-path restriction does not reduce maximum throughput in the baseline scenario.

This result is consistent with Section 33, where the splittable Stage 1 solution happened not to require any delivered commodity to use multiple positive-flow paths.

However, equality of total throughput does **not** imply that the two models choose the same commodities, routes, or link-loading pattern.

Those differences are examined throughout Section 36.

36.1.1 Interpretation of the Dominance Check

The validation confirms:

$$T^*_{\text{single}} = T^*_{\text{split}} = 83 \text{ Mbps}.$$

Therefore:

$$T^*_{\text{split}} - T^*_{\text{single}} = 0 \text{ Mbps}.$$

The single-path restriction does not reduce maximum throughput in the baseline scenario.

This result is consistent with Section 33, where the splittable Stage 1 solution happened not to require any delivered commodity to use multiple positive-flow paths.

However, equality of total throughput does **not** imply that the two models choose the same commodities, routes, or link-loading pattern.

Those differences are examined throughout Section 36.

36.1.2 Business and Operational Impact

At baseline, the network receives no throughput benefit from allowing commodity-level traffic splitting.

This suggests that the operationally simpler single-path policy may be sufficient if maximum throughput were the only concern.

However, the business should not make that decision from the objective value alone.

Two 83 Mbps solutions can still differ materially in:

- which customers or sources are served;
- which traffic classes experience loss;
- latency;
- network headroom;
- resilience;
- route complexity.

Therefore, detailed solution comparison remains necessary.

36.2 Verify One Used Path per Commodity

Section 35 imposed the constraint:

$$\sum_{p \in P_{k,s}} y_{k,s,p} \leq 1.$$

The next validation checks the realized traffic flows rather than only the binary variables.

For every commodity, the number of candidate paths with positive traffic should satisfy:

$$\text{Number of Used Paths} \leq 1.$$

This confirms that the actual optimized flow respects the single-path rule.

In [322…

```
# ------------------------------------------------------------
# Count positive-flow routes for every source-stream commodity
# ------------------------------------------------------------

single_path_check = []


# Group all candidate routes by commodity.

for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    # Store path IDs carrying meaningful positive flow.
```

```
    used_paths = []


    # Examine every candidate route for the commodity.

    for pid in group[
        "path_id"
    ]:

        pid = int(
            pid
        )


        # Retrieve optimized traffic on the route.

        flow = (
            single_flow_vars[
                pid
            ].value()
            or 0
        )


        # Treat flows greater than numerical tolerance
        # as genuinely active routes.

        if flow > 1e-8:

            used_paths.append(
                pid
            )


    # Store number of active routes.

    single_path_check.append({
        "commodity_id":
            commodity_id,
        "number_of_used_paths":
            len(
                used_paths
            )
    })


# Convert validation results into a DataFrame.

single_path_check_df = (
    pd.DataFrame(
        single_path_check
```

```
    )
)


single_path_check_df
```

Out[322…

	commodity_id	number_of_used_paths
0	B_UK-L	1
1	B_UK-T	1
2	B_US-L	1
3	B_US-T	1
4	C_UK-L	1
5	C_UK-T	1
6	C_US-L	1
7	C_US-T	1
8	S1_UK-L	0
9	S1_UK-T	1
10	S1_US-L	1
11	S1_US-T	1
12	S2_UK-L	1
13	S2_UK-T	1
14	S2_US-L	0
15	S2_US-T	0
16	S3_UK-L	0
17	S3_UK-T	1
18	S3_US-L	0
19	S3_US-T	1

36.2.1 Validate the One-Path Rule Programmatically

In [323…

```
# ------------------------------------------------------------
# Confirm no commodity uses more than one positive-flow route
# ------------------------------------------------------------

max_used_paths = (
    single_path_check_df[
```

```
        "number_of_used_paths"
    ].max()
)


print(
    "Maximum number of positive-flow routes used by any commodity:",
    max_used_paths
)


assert (
    max_used_paths
    <= 1
)


print(
    "Single-path positive-flow validation passed."
)
```

```
Maximum number of positive-flow routes used by any commodity: 1
Single-path positive-flow validation passed.
```

36.2.2 Interpretation of the One-Path Validation

The realized MILP solution satisfies the single-path requirement.

Every commodity uses either:

$$0$$

or:

$$1$$

positive-flow route.

No commodity uses multiple routes.

The zero-path entries indicate commodities receiving no delivered traffic in the Stage 1 solution.

For example, the current solution shows no positive-flow route for several source-stream combinations, including commodities such as:

- S1 / UK-L;
- S2 / US-L;
- S2 / US-T;
- S3 / UK-L;

- S3 / US-L.

Other commodities use exactly one path.

This confirms that the model is implementing the intended routing restriction rather than reproducing the splittable formulation.

36.3 Extract Single-Path Source-Stream Delivery Results

The used-path count verifies routing structure, but it does not show how much traffic is delivered.

A commodity using one route may still receive either:

- full delivery;
- partial delivery.

Therefore, the optimized path flows are aggregated back to the 20 source-stream commodities.

For commodity (k, s):

$$z_{k,s} = \sum_{p \in P_{k,s}} f_{k,s,p}.$$

Because the model allows at most one positive-flow route, this sum contains either:

- zero positive terms;
- or one positive term.

In [324…

```
# ------------------------------------------------------------
# Extract delivered and lost traffic by source-stream commodity
# ------------------------------------------------------------

single_stage1_delivery_records = []


for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    # Sum path flow for the commodity.
    #
```

```
    # Under the single-path model, at most one route
    # should contribute positive flow.

    delivered = sum(
        (
            single_flow_vars[
                int(pid)
            ].value()
            or 0
        )
        for pid
        in group[
            "path_id"
        ]
    )


    # Offered demand is identical across all candidate
    # routes belonging to this commodity.

    demand = float(
        group[
            "demand_mbps"
        ].iloc[0]
    )


    # Calculate undelivered demand.

    lost = (
        demand
        - delivered
    )


    # Calculate delivery percentage.

    delivery_percentage = (
        100
        * delivered
        / demand
        if demand > 0
        else 0
    )


    # Store result.

    single_stage1_delivery_records.append({
        "commodity_id":
            commodity_id,
        "source":
```

```
            group[
                "source"
            ].iloc[0],
        "stream":
            group[
                "stream"
            ].iloc[0],
        "destination":
            group[
                "destination"
            ].iloc[0],
        "demand_mbps":
            demand,
        "delivered_mbps":
            delivered,
        "lost_mbps":
            lost,
        "delivery_percentage":
            delivery_percentage
    })


single_stage1_delivery_df = (
    pd.DataFrame(
        single_stage1_delivery_records
    )
)


# Apply the same source and stream ordering used earlier.

single_stage1_delivery_df[
    "source"
] = pd.Categorical(
    single_stage1_delivery_df[
        "source"
    ],
    categories=
        source_order,
    ordered=True
)


single_stage1_delivery_df[
    "stream"
] = pd.Categorical(
    single_stage1_delivery_df[
        "stream"
    ],
    categories=
        stream_order,
    ordered=True
```

```
)


single_stage1_delivery_df = (
    single_stage1_delivery_df
    .sort_values(
        [
            "source",
            "stream"
        ]
    )
    .reset_index(
        drop=True
    )
)


single_stage1_delivery_df
```

Out [324…

	commodity_id	source	stream	destination	demand_mbps	delivered_mbps
0	S1_UK-L	S1	UK-L	UK	6.0	0.0
1	S1_UK-T	S1	UK-T	UK	7.0	4.0
2	S1_US-L	S1	US-L	US	5.0	5.0
3	S1_US-T	S1	US-T	US	6.0	6.0
4	S2_UK-L	S2	UK-L	UK	5.0	5.0
5	S2_UK-T	S2	UK-T	UK	6.0	6.0
6	S2_US-L	S2	US-L	US	5.0	0.0
7	S2_US-T	S2	US-T	US	5.0	0.0
8	S3_UK-L	S3	UK-L	UK	5.0	0.0
9	S3_UK-T	S3	UK-T	UK	7.0	7.0
10	S3_US-L	S3	US-L	US	6.0	0.0
11	S3_US-T	S3	US-T	US	6.0	3.0
12	B_UK-L	B	UK-L	UK	6.0	6.0
13	B_UK-T	B	UK-T	UK	14.0	14.0
14	B_US-L	B	US-L	US	8.0	8.0
15	B_US-T	B	US-T	US	9.0	2.0
16	C_UK-L	C	UK-L	UK	4.0	1.0
17	C_UK-T	C	UK-T	UK	5.0	5.0
18	C_US-L	C	US-L	US	5.0	5.0
19	C_US-T	C	US-T	US	6.0	6.0

36.3.1 Classify Service Outcomes

In [325…

```
# ------------------------------------------------------------
# Classify commodity-level service status
# ------------------------------------------------------------

def classify_single_stage1_delivery(
    row
):

    # Full service.

    if (
```

```
        row[
            "delivery_percentage"
        ]
        >= 100 - 1e-6
    ):
        return "Fully Delivered"


    # No service.

    if (
        row[
            "delivery_percentage"
        ]
        <= 1e-6
    ):
        return "Not Delivered"


    # Anything in between represents partial service.

    return "Partially Delivered"


single_stage1_delivery_df[
    "service_status"
] = (
    single_stage1_delivery_df.apply(
        classify_single_stage1_delivery,
        axis=1
    )
)


single_stage1_delivery_df
```

Out [325...

	commodity_id	source	stream	destination	demand_mbps	delivered_mbps
0	S1_UK-L	S1	UK-L	UK	6.0	0.0
1	S1_UK-T	S1	UK-T	UK	7.0	4.0
2	S1_US-L	S1	US-L	US	5.0	5.0
3	S1_US-T	S1	US-T	US	6.0	6.0
4	S2_UK-L	S2	UK-L	UK	5.0	5.0
5	S2_UK-T	S2	UK-T	UK	6.0	6.0
6	S2_US-L	S2	US-L	US	5.0	0.0
7	S2_US-T	S2	US-T	US	5.0	0.0
8	S3_UK-L	S3	UK-L	UK	5.0	0.0
9	S3_UK-T	S3	UK-T	UK	7.0	7.0
10	S3_US-L	S3	US-L	US	6.0	0.0
11	S3_US-T	S3	US-T	US	6.0	3.0
12	B_UK-L	B	UK-L	UK	6.0	6.0
13	B_UK-T	B	UK-T	UK	14.0	14.0
14	B_US-L	B	US-L	US	8.0	8.0
15	B_US-T	B	US-T	US	9.0	2.0
16	C_UK-L	C	UK-L	UK	4.0	1.0
17	C_UK-T	C	UK-T	UK	5.0	5.0
18	C_US-L	C	US-L	US	5.0	5.0
19	C_US-T	C	US-T	US	6.0	6.0

36.3.2 Interpretation

The commodity-level delivery table reveals which traffic actually makes up the single-path model's 83 Mbps.

This is important because total throughput alone does not identify whether the single-path model admits the same commodities as the splittable model.

Two models can both deliver:

$$83 \text{ Mbps}$$

while producing very different service distributions.

The delivery table therefore becomes the basis for the later split-versus-single admission comparison.

36.4 Validate Throughput Accounting

Commodity-level results should reconcile exactly with the MILP objective.

The required accounting identity is:

$$\text{Delivered} + \text{Lost} = 126 \text{ Mbps}.$$

The delivered total must also equal:

$$83 \text{ Mbps}.$$

In [326…

```
# ------------------------------------------------------------
# Reconcile commodity results with the optimization objective
# ------------------------------------------------------------

single_total_delivered = (
    single_stage1_delivery_df[
        "delivered_mbps"
    ].sum()
)


single_total_lost = (
    single_stage1_delivery_df[
        "lost_mbps"
    ].sum()
)


print(
    "Total demand:",
    total_demand,
    "Mbps"
)

print(
    "Single-path delivered:",
    single_total_delivered,
    "Mbps"
```

```
)

print(
    "Single-path lost:",
    single_total_lost,
    "Mbps"
)

print(
    "Delivered + lost:",
    single_total_delivered
    + single_total_lost,
    "Mbps"
)


# Validate accounting.

assert abs(
    (
        single_total_delivered
        + single_total_lost
    )
    - total_demand
) < 1e-6


# Validate against MILP objective.

assert abs(
    single_total_delivered
    - single_stage1_throughput
) < 1e-6


print(
    "Single-path throughput accounting passed."
)
```

```
Total demand: 126 Mbps
Single-path delivered: 83.0 Mbps
Single-path lost: 43.0 Mbps
Delivered + lost: 126.0 Mbps
Single-path throughput accounting passed.
```

36.5 Aggregate Single-Path Delivery by Traffic Stream

The source-stream results are aggregated into the four problem-level traffic streams:

- UK-L;
- UK-T;
- US-L;
- US-T.

This provides the output format requested by the problem statement and allows direct comparison with the splittable model.

In [327…

```
# ------------------------------------------------------------
# Aggregate single-path delivery by stream
# ------------------------------------------------------------

single_stage1_stream_summary = (
    single_stage1_delivery_df
    .groupby(
        "stream",
        observed=True,
        as_index=False
    )
    .agg(
        demand_mbps=(
            "demand_mbps",
            "sum"
        ),
        delivered_mbps=(
            "delivered_mbps",
            "sum"
        ),
        lost_mbps=(
            "lost_mbps",
            "sum"
        )
    )
)


single_stage1_stream_summary[
    "delivery_percentage"
] = (
    100
    * single_stage1_stream_summary[
        "delivered_mbps"
    ]
    / single_stage1_stream_summary[
        "demand_mbps"
    ]
)


single_stage1_stream_summary
```

Out [327…

	stream	demand_mbps	delivered_mbps	lost_mbps	delivery_percentage
0	UK-L	26.0	12.0	14.0	46.153846
1	UK-T	39.0	36.0	3.0	92.307692
2	US-L	29.0	18.0	11.0	62.068966
3	US-T	32.0	17.0	15.0	53.125000

36.5.1 Compare Stream Admission: Split vs. Single

In [328…

```
# ------------------------------------------------------------
# Compare Stage 1 stream delivery across routing formulations
# ------------------------------------------------------------

split_vs_single_stream_df = (
    split_stage1_stream_summary[
        [
            "stream",
            "demand_mbps",
            "delivered_mbps",
            "lost_mbps",
            "delivery_percentage"
        ]
    ]
    .rename(
        columns={
            "delivered_mbps":
                "split_delivered_mbps",
            "lost_mbps":
                "split_lost_mbps",
            "delivery_percentage":
                "split_delivery_pct"
        }
    )
    .merge(
        single_stage1_stream_summary[
            [
                "stream",
                "delivered_mbps",
                "lost_mbps",
                "delivery_percentage"
            ]
        ]
        .rename(
            columns={
                "delivered_mbps":
                    "single_delivered_mbps",
                "lost_mbps":
                    "single_lost_mbps",
```

```
                "delivery_percentage":
                    "single_delivery_pct"
            }
        ),
        on="stream",
        how="inner"
    )
)


split_vs_single_stream_df[
    "delivery_difference_mbps"
] = (
    split_vs_single_stream_df[
        "single_delivered_mbps"
    ]
    - split_vs_single_stream_df[
        "split_delivered_mbps"
    ]
)


split_vs_single_stream_df
```

Out[328...

	stream	demand_mbps	split_delivered_mbps	split_lost_mbps	split_delivery
0	UK-L	26.0	16.0	10.0	61.53
1	UK-T	39.0	34.0	5.0	87.17
2	US-L	29.0	18.0	11.0	62.06
3	US-T	32.0	15.0	17.0	46.87

36.5.2 Interpretation of Stream-Level Comparison

Even when total network throughput is identical, the split and single-path models may distribute the 83 Mbps differently across traffic classes.

The `delivery_difference_mbps` column identifies whether the single-path restriction changes the stream-level admission mix.

A value of:

$$0$$

means the two models deliver the same amount of that stream.

A positive value means the single-path model delivers more of the stream.

A negative value means it delivers less.

This is an important distinction because equal total throughput does not guarantee equal service composition.

36.6 Aggregate Delivery by Source

In [329…

```
# ------------------------------------------------------------
# Aggregate single-path Stage 1 delivery by source
# ------------------------------------------------------------

single_stage1_source_summary = (
    single_stage1_delivery_df
    .groupby(
        "source",
        observed=True,
        as_index=False
    )
    .agg(
        demand_mbps=(
            "demand_mbps",
            "sum"
        ),
        delivered_mbps=(
            "delivered_mbps",
            "sum"
        ),
        lost_mbps=(
            "lost_mbps",
            "sum"
        )
    )
)


single_stage1_source_summary[
    "delivery_percentage"
] = (
    100
    * single_stage1_source_summary[
        "delivered_mbps"
    ]
    / single_stage1_source_summary[
        "demand_mbps"
    ]
)


single_stage1_source_summary
```

Out [329…

	source	demand_mbps	delivered_mbps	lost_mbps	delivery_percentage
0	S1	24.0	15.0	9.0	62.500000
1	S2	21.0	11.0	10.0	52.380952
2	S3	24.0	10.0	14.0	41.666667
3	B	37.0	30.0	7.0	81.081081
4	C	20.0	17.0	3.0	85.000000

36.6.1 Compare Source Admission: Split vs. Single

In [330…

```
# ------------------------------------------------------------
# Compare source-level admission between Stage 1 formulations
# ------------------------------------------------------------

split_vs_single_source_df = (
    split_stage1_source_summary[
        [
            "source",
            "demand_mbps",
            "delivered_mbps",
            "delivery_percentage"
        ]
    ]
    .rename(
        columns={
            "delivered_mbps":
                "split_delivered_mbps",
            "delivery_percentage":
                "split_delivery_pct"
        }
    )
    .merge(
        single_stage1_source_summary[
            [
                "source",
                "delivered_mbps",
                "delivery_percentage"
            ]
        ]
        .rename(
            columns={
                "delivered_mbps":
                    "single_delivered_mbps",
                "delivery_percentage":
                    "single_delivery_pct"
            }
        ),
```

```
        on="source",
        how="inner"
    )
)


split_vs_single_source_df[
    "delivery_difference_mbps"
] = (
    split_vs_single_source_df[
        "single_delivered_mbps"
    ]
    - split_vs_single_source_df[
        "split_delivered_mbps"
    ]
)


split_vs_single_source_df
```

Out[330…

	source	demand_mbps	split_delivered_mbps	split_delivery_pct	single_deliv
0	S1	24.0	13.0	54.166667	
1	S2	21.0	11.0	52.380952	
2	S3	24.0	2.0	8.333333	
3	B	37.0	37.0	100.000000	
4	C	20.0	20.0	100.000000	

36.6.2 Interpretation of Source-Level Differences

This comparison determines whether the one-path restriction changes which sources receive service.

The baseline split model distributed its 83 Mbps unevenly across sources.

The single-path MILP may identify a different throughput-optimal admission vector.

If source-level delivery differs while total throughput remains:

$$83 \text{ Mbps},$$

that is evidence of **throughput-optimal degeneracy**.

In other words, several distinct admission patterns can achieve the same objective value.

This reinforces the conclusion from Section 33 that maximum throughput alone does not define a unique fairness or customer-service policy.

36.7 Visualize Single-Path Delivery by Commodity

In [331…

```
# ------------------------------------------------------------
# Plot delivered and lost demand by commodity
# ------------------------------------------------------------

single_commodity_plot_df = (
    single_stage1_delivery_df.copy()
)


single_commodity_plot_df[
    "commodity_label"
] = (
    single_commodity_plot_df[
        "source"
    ].astype(str)
    + " / "
    + single_commodity_plot_df[
        "stream"
    ].astype(str)
)


single_commodity_plot_df.set_index(
    "commodity_label"
)[
    [
        "delivered_mbps",
        "lost_mbps"
    ]
].plot(
    kind="bar",
    stacked=True,
    figsize=(14, 6)
)


plt.ylabel(
    "Demand (Mbps)"
)

plt.xlabel(
    "Source-Stream Commodity"
)

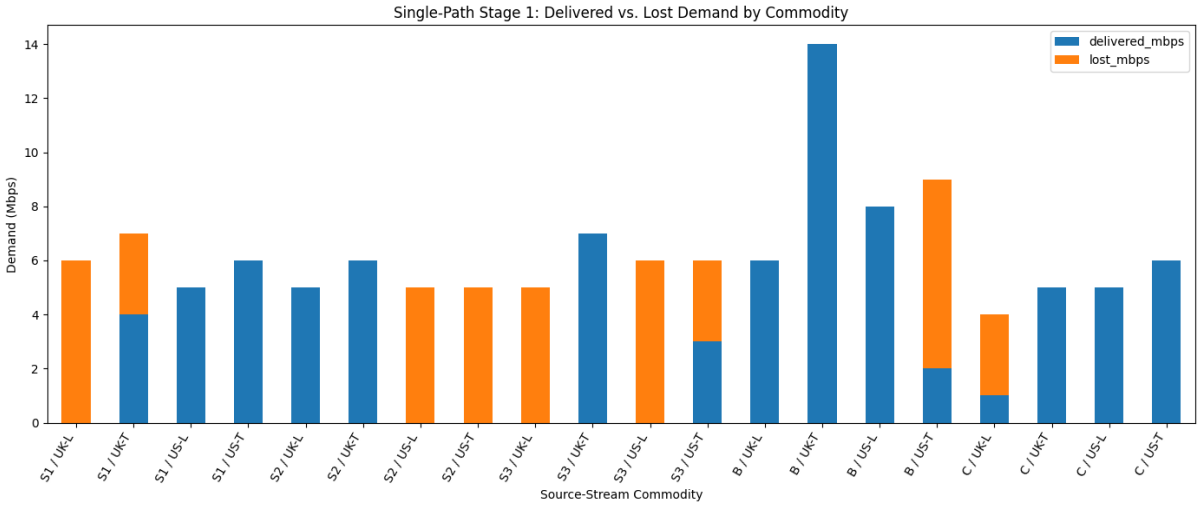
plt.title(
    "Single-Path Stage 1: "
```

```
    "Delivered vs. Lost Demand by Commodity"
)

plt.xticks(
    rotation=60,
    ha="right"
)

plt.tight_layout()

plt.show()
```



36.7.1 Interpretation and Business Impact

The commodity-level chart reveals how the single-path model allocates its 83 Mbps throughput across individual source-stream demands.

This should be compared with the equivalent chart from Section 33.

If the patterns differ, then the routing restriction affects service allocation even though it does not affect total throughput.

This is operationally important because customers experience commodity-level service, not merely aggregate network throughput.

36.8 Delivery Percentage Heatmap

In [333…

```
# ------------------------------------------------------------
# Create single-path source-by-stream delivery matrix
# ------------------------------------------------------------

single_stage1_delivery_matrix = (
    single_stage1_delivery_df
    .pivot(
```

```
        index="source",
        columns="stream",
        values=
            "delivery_percentage"
    )
    .reindex(
        index=
            source_order,
        columns=
            stream_order
    )
)


single_stage1_delivery_matrix
```

Out[333…

stream	UK-L	UK-T	US-L	US-T
source				
S1	0.0	57.142857	100.0	100.000000
S2	100.0	100.000000	0.0	0.000000
S3	0.0	100.000000	0.0	50.000000
B	100.0	100.000000	100.0	22.222222
C	25.0	100.000000	100.0	100.000000

In [334…

```
fig, ax = plt.subplots(
    figsize=(8, 5)
)


image = ax.imshow(
    single_stage1_delivery_matrix.values,
    aspect="auto",
    vmin=0,
    vmax=100
)


ax.set_xticks(
    range(
        len(
            single_stage1_delivery_matrix.columns
        )
    )
)

ax.set_xticklabels(
    single_stage1_delivery_matrix.columns
```

```
)


ax.set_yticks(
    range(
        len(
            single_stage1_delivery_matrix.index
        )
    )
)

ax.set_yticklabels(
    single_stage1_delivery_matrix.index
)


# Add delivery percentage to each cell.

for i in range(
    len(
        single_stage1_delivery_matrix.index
    )
):

    for j in range(
        len(
            single_stage1_delivery_matrix.columns
        )
    ):

        value = (
            single_stage1_delivery_matrix.iloc[
                i,
                j
            ]
        )

        ax.text(
            j,
            i,
            f"{value:.0f}%",
            ha="center",
            va="center"
        )


ax.set_xlabel(
    "Traffic Stream"
)

ax.set_ylabel(
    "Source Node"
```

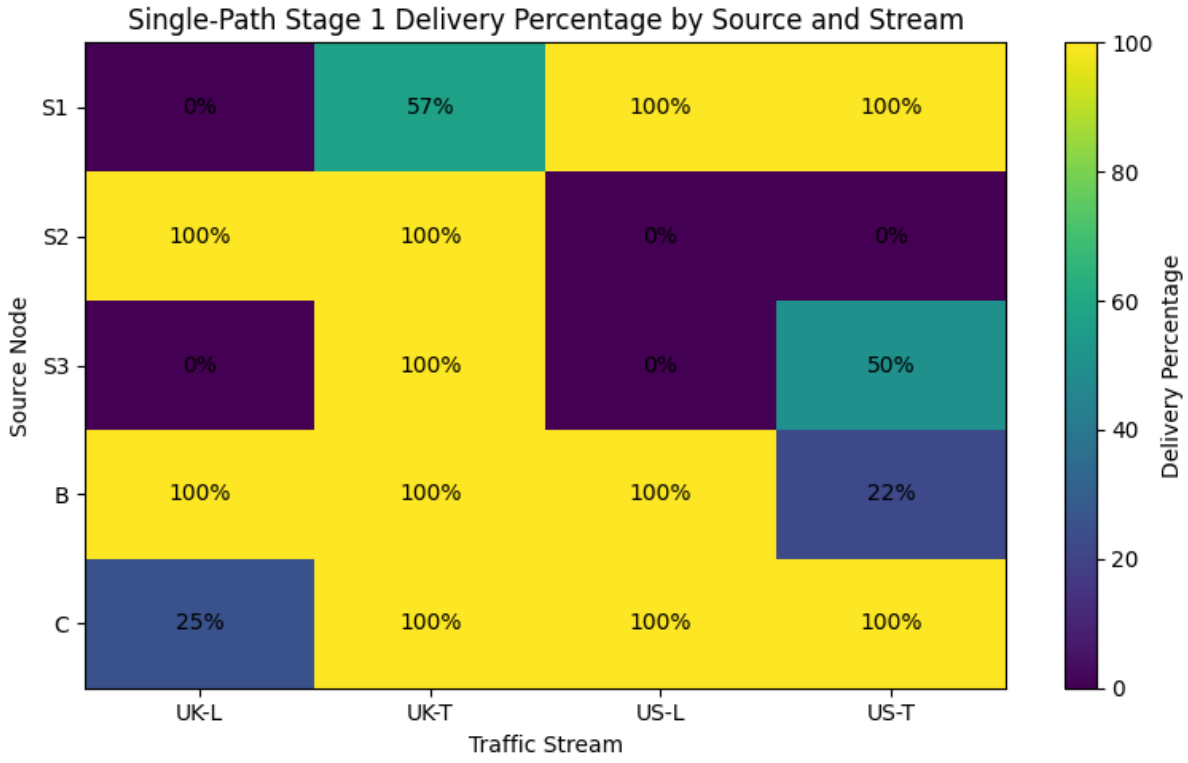
```
)

ax.set_title(
    "Single-Path Stage 1 "
    "Delivery Percentage by Source and Stream"
)


plt.colorbar(
    image,
    ax=ax,
    label=
        "Delivery Percentage"
)


plt.tight_layout()

plt.show()
```



36.9 Extract Active Single-Path Routes

36.9 Extract Active Single-Path Routes

The next step is to identify exactly which route carries each admitted commodity.

Because of the MILP restriction, every delivered commodity should appear on

only one positive-flow route.

The active-route table provides:

- source;
- stream;
- destination;
- selected path;
- delivered flow;
- path latency;
- isolated path capacity.

In [336…

```
# ------------------------------------------------------------
# Extract positive-flow single-path Stage 1 routes
# ------------------------------------------------------------

single_stage1_route_records = []


for _, row in (
    candidate_paths_df.iterrows()
):

    pid = int(
        row[
            "path_id"
        ]
    )


    flow = (
        single_flow_vars[
            pid
        ].value()
        or 0
    )


    selected = (
        single_select_vars[
            pid
        ].value()
        or 0
    )


    # Retain only routes carrying meaningful traffic.

    if flow > 1e-8:
```

```
        single_stage1_route_records.append({
            "commodity_id":
                row[
                    "commodity_id"
                ],
            "source":
                row[
                    "source"
                ],
            "stream":
                row[
                    "stream"
                ],
            "destination":
                row[
                    "destination"
                ],
            "path":
                row[
                    "path"
                ],
            "flow_mbps":
                flow,
            "selected_binary":
                selected,
            "path_capacity_mbps":
                row[
                    "path_capacity_mbps"
                ],
            "path_latency_ms":
                row[
                    "path_latency_ms"
                ]
        })


single_stage1_routes_df = (
    pd.DataFrame(
        single_stage1_route_records
    )
)


print(
    "Active single-path Stage 1 routes:",
    len(
        single_stage1_routes_df
    )
)


single_stage1_routes_df
```

```
Active single-path Stage 1 routes: 15
```

Out[336…

	commodity_id	source	stream	destination	path	flow_mbps	selected_bina
0	S1_UK-T	S1	UK-T	UK	S1 -> A -> B -> C -> UK	4.0	
1	S1_US-L	S1	US-L	US	S1 -> A -> C -> US	5.0	
2	S1_US-T	S1	US-T	US	S1 -> A -> C -> US	6.0	
3	S2_UK-L	S2	UK-L	UK	S2 -> A -> C -> B -> D -> UK	5.0	
4	S2_UK-T	S2	UK-T	UK	S2 -> A -> C -> UK	6.0	
5	S3_UK-T	S3	UK-T	UK	S3 -> B -> D -> UK	7.0	
6	S3_US-T	S3	US-T	US	S3 -> B -> A -> C -> US	3.0	
7	B_UK-L	B	UK-L	UK	B -> D -> UK	6.0	
8	B_UK-T	B	UK-T	UK	B -> C -> UK	14.0	

9	B_US-L	B	US-L	US	B -> D -> US	8.0
10	B_US-T	B	US-T	US	B -> D -> US	2.0
11	C_UK-L	C	UK-L	UK	C -> UK	1.0
12	C_UK-T	C	UK-T	UK	C -> UK	5.0
13	C_US-L	C	US-L	US	C -> US	5.0
14	C_US-T	C	US-T	US	C -> US	6.0

36.9.1 Validate Positive Flow Implies Binary Selection

In [337…

```
# All active routes should have selection variable equal to 1.

assert (
    single_stage1_routes_df[
        "selected_binary"
    ]
    >= 0.5
).all()


print(
    "All positive-flow routes are correctly selected."
)
```

```
All positive-flow routes are correctly selected.
```

36.10 Identify Full, Partial, and Unserved Commodities

In [338…

```
# ------------------------------------------------------------
# Count service outcomes
# ------------------------------------------------------------

single_service_status_summary_df = (
    single_stage1_delivery_df[
        "service_status"
    ]
    .value_counts()
    .rename_axis(
        "service_status"
```

```
    )
    .reset_index(
        name=
            "number_of_commodities"
    )
)


single_service_status_summary_df
```

Out [338…

	service_status	number_of_commodities
0	Fully Delivered	11
1	Not Delivered	5
2	Partially Delivered	4

In [339…

```
#Partially Delivered Only Table
# Show only partially delivered commodities.

single_partial_delivery_df = (
    single_stage1_delivery_df[
        single_stage1_delivery_df[
            "service_status"
        ]
        == "Partially Delivered"
    ][
        [
            "commodity_id",
            "demand_mbps",
            "delivered_mbps",
            "lost_mbps",
            "delivery_percentage"
        ]
    ]
)


single_partial_delivery_df
```

Out [339…

	commodity_id	demand_mbps	delivered_mbps	lost_mbps	delivery_percent
1	S1_UK-T	7.0	4.0	3.0	57.142
11	S3_US-T	6.0	3.0	3.0	50.000
15	B_US-T	9.0	2.0	7.0	22.222
16	C_UK-L	4.0	1.0	3.0	25.000

36.10.1 Interpretation of Partial Delivery

This table verifies that the single-path restriction does not imply all-or-nothing service.

A commodity may select one route and carry only part of its offered demand.

That behavior follows directly from:

$$f_{k,s,p} \le D_{k,s} y_{k,s,p}.$$

When:

$$y_{k,s,p} = 1,$$

the flow may take any value from zero through the full commodity demand.

This confirms that the MILP implements the requested interpretation correctly.

36.11 Calculate Single-Path Stage 1 Link Utilization

The active single-path routes are now aggregated into directed-link flows.

The same function used for the splittable model is reused here.

Using the identical utilization calculation makes the split-versus-single comparison consistent.

For each directed arc:

$$U_e = 100 \frac{F_e}{C_e}.$$

In [340…

```
# ------------------------------------------------------------
# Calculate single-path Stage 1 directed-link utilization
# ------------------------------------------------------------

single_stage1_link_df = (
    calculate_link_results(
        candidate_paths_df,
        single_flow_vars,
        arcs_df,
        "Single Stage 1"
    )
)
```

```
# Rank highest-utilization links first.

single_stage1_link_df = (
    single_stage1_link_df
    .sort_values(
        "utilization_percentage",
        ascending=False
    )
    .reset_index(
        drop=True
    )
)


single_stage1_link_df
```

Out[340…

	model	from_node	to_node	capacity_mbps	flow_mbps	residual_capacity_
0	Single Stage 1	B	C	18.0	18.0	
1	Single Stage 1	C	US	25.0	25.0	
2	Single Stage 1	C	UK	30.0	30.0	
3	Single Stage 1	A	C	25.0	25.0	
4	Single Stage 1	B	D	28.0	28.0	
5	Single Stage 1	D	UK	22.0	18.0	
6	Single Stage 1	S1	A	40.0	15.0	
7	Single Stage 1	D	US	30.0	10.0	
8	Single Stage 1	S3	B	30.0	10.0	
	Single					

9	Stage 1	S2	A	35.0	11.0
10	Single Stage 1	C	B	18.0	5.0
11	Single Stage 1	A	B	20.0	4.0
12	Single Stage 1	B	A	20.0	3.0
13	Single Stage 1	A	S1	40.0	0.0
14	Single Stage 1	C	A	25.0	0.0
15	Single Stage 1	D	B	28.0	0.0
16	Single Stage 1	UK	C	30.0	0.0
17	Single Stage 1	US	C	25.0	0.0
18	Single Stage 1	B	S3	30.0	0.0
19	Single Stage 1	UK	D	22.0	0.0
20	Single Stage 1	A	S2	35.0	0.0
21	Single Stage 1	US	D	30.0	0.0

36.11.1 nterpretation of Single-Path Stage 1 Link Utilization

The single-path Stage 1 solution reaches the same total throughput as the

splittable solution but produces a materially different internal traffic pattern.

Five directed links operate at:

$$100\%$$

utilization:

- B → C;
- C → US;
- C → UK;
- A → C;
- B → D.

These links therefore carry:

$$18/18,$$

$$25/25,$$

$$30/30,$$

$$25/25,$$

and:

$$28/28$$

Mbps respectively.

Additional heavily or moderately used links include:

D → UK

$$18/22 \approx 81.82\%$$

S1 → A

$$15/40 = 37.5\%$$

D → US

$$10/30 = 33.33\%$$

S3 → B

$$10/30 = 33.33\%$$

S2 → A

$$11/35 \approx 31.43\%$$

The solution also uses internal reverse or cross-routing capacity such as:

$$C \to B = 5 \text{ Mbps},$$

$$A \to B = 4 \text{ Mbps},$$

and:

$$B \to A = 3 \text{ Mbps}.$$

This indicates that the single-path MILP reaches the same 83 Mbps objective through a more constrained and somewhat different internal route allocation than the baseline splittable Stage 1 solution.

36.11.2 Comparison with Splittable Stage 1

The two models deliver the same:

$$83 \text{ Mbps},$$

but they do not produce the same link-loading pattern.

Several differences are especially important.

B → C

Splittable Stage 1:

$$11/18 \approx 61.1\%$$

Single-path Stage 1:

$$18/18 = 100\%.$$

A → C

Splittable Stage 1:

$$24/25 = 96\%.$$

Single-path Stage 1:

$$25/25 = 100\%.$$

D → UK

Splittable Stage 1:

$$20/22 \approx 90.9\%.$$

Single-path Stage 1:

$$18/22 \approx 81.8\%.$$

D → US

Splittable Stage 1:

$$8/30 \approx 26.7\%.$$

Single-path Stage 1:

$$10/30 \approx 33.3\%.$$

S3 → B

Splittable Stage 1:

$$2/30 \approx 6.7\%.$$

Single-path Stage 1:

$$10/30 \approx 33.3\%.$$

The last comparison is particularly important.

The single-path model admits substantially more traffic through S3's mandatory entry link than the particular splittable Stage 1 solution returned by CBC.

This provides direct evidence that equal total throughput can correspond to different admission patterns.

36.12 Validate Single-Path Link Capacity Feasibility

In [342…

```
# ------------------------------------------------------------
# Confirm no directed link exceeds physical capacity
# ------------------------------------------------------------

single_stage1_link_df[
    "capacity_violation_mbps"
] = (
    single_stage1_link_df[
        "flow_mbps"
    ]
    - single_stage1_link_df[
```

```
        "capacity_mbps"
    ]
).clip(
    lower=0
)


single_max_capacity_violation = (
    single_stage1_link_df[
        "capacity_violation_mbps"
    ].max()
)


assert (
    single_max_capacity_violation
    <= 1e-6
)


print(
    "Maximum single-path capacity violation:",
    single_max_capacity_violation,
    "Mbps"
)

print(
    "All single-path Stage 1 capacity constraints satisfied."
)
```

```
Maximum single-path capacity violation: 0.0 Mbps
All single-path Stage 1 capacity constraints satisfied.
```

36.12.1 Interpretation and Business Impact

No single-path route allocation exceeds a directional capacity limit.

Therefore, the 83 Mbps result represents a physically feasible network operating point.

The difference from the splittable solution lies in **how the capacity is used**, not whether capacity is violated.

36.13 Identify Saturated Links

In [343…

```
# ------------------------------------------------------------
# Extract fully saturated directed links
# ------------------------------------------------------------

single_stage1_saturated_df = (
```

```
    single_stage1_link_df[
        single_stage1_link_df[
            "is_saturated"
        ]
    ][
        [
            "from_node",
            "to_node",
            "capacity_mbps",
            "flow_mbps",
            "utilization_percentage"
        ]
    ]
    .reset_index(
        drop=True
    )
)


print(
    "Number of saturated single-path Stage 1 links:",
    len(
        single_stage1_saturated_df
    )
)


single_stage1_saturated_df
```

```
Number of saturated single-path Stage 1 links: 5
```

Out[343…

	from_node	to_node	capacity_mbps	flow_mbps	utilization_percentage
0	B	C	18.0	18.0	100.0
1	C	US	25.0	25.0	100.0
2	C	UK	30.0	30.0	100.0
3	A	C	25.0	25.0	100.0
4	B	D	28.0	28.0	100.0

36.13.1 Interpretation of Saturation

The single-path Stage 1 solution contains five saturated directed links.

By comparison, the splittable Stage 1 solution contained three saturated directed links.

This suggests that the single-path restriction may require the optimizer to use certain corridors more aggressively in order to maintain the same 83 Mbps

throughput.

However:

> More saturated links does not automatically mean lower performance.

Both models deliver the same throughput.

It does suggest that the single-path solution may have less routing headroom in some parts of the network.

As before, saturation should not be confused with true marginal bottleneck value.

36.14 Visualize Single-Path Link Utilization

In [344…

```
# Keep only links carrying positive traffic.

single_active_link_plot_df = (
    single_stage1_link_df[
        single_stage1_link_df[
            "flow_mbps"
        ] > 1e-8
    ]
    .copy()
)


# Create readable labels.

single_active_link_plot_df[
    "link"
] = (
    single_active_link_plot_df[
        "from_node"
    ]
    + " → "
    + single_active_link_plot_df[
        "to_node"
    ]
)


single_active_link_plot_df = (
    single_active_link_plot_df
    .sort_values(
        "utilization_percentage",
        ascending=False
    )
)
```

```
plt.figure(
    figsize=(12, 6)
)

plt.bar(
    single_active_link_plot_df[
        "link"
    ],
    single_active_link_plot_df[
        "utilization_percentage"
    ]
)


plt.axhline(
    100,
    linestyle="--"
)


plt.ylabel(
    "Utilization (%)"
)

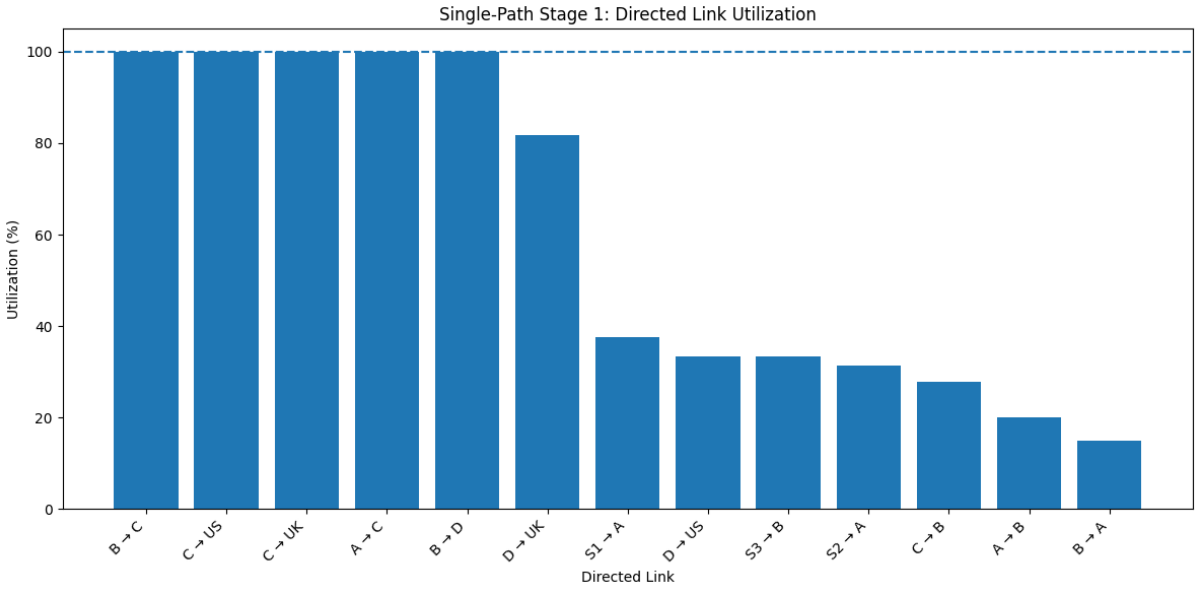
plt.xlabel(
    "Directed Link"
)

plt.title(
    "Single-Path Stage 1: "
    "Directed Link Utilization"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()

plt.show()
```



36.15 Compare Split vs. Single Stage 1 Link Utilization

In [345…

```
# ------------------------------------------------------------
# Merge split and single Stage 1 link results
# ------------------------------------------------------------

split_vs_single_link_df = (
    split_stage1_link_df[
        [
            "from_node",
            "to_node",
            "capacity_mbps",
            "flow_mbps",
            "utilization_percentage"
        ]
    ]
    .rename(
        columns={
            "flow_mbps":
                "split_flow_mbps",
            "utilization_percentage":
                "split_utilization_pct"
        }
    )
    .merge(
        single_stage1_link_df[
            [
                "from_node",
                "to_node",
                "flow_mbps",
                "utilization_percentage"
            ]
        ]
        .rename(
```

```
            columns={
                "flow_mbps":
                    "single_flow_mbps",
                "utilization_percentage":
                    "single_utilization_pct"
            }
        ),
        on=[
            "from_node",
            "to_node"
        ],
        how="inner"
    )
)


# Calculate change caused by the single-path restriction.

split_vs_single_link_df[
    "single_minus_split_flow_mbps"
] = (
    split_vs_single_link_df[
        "single_flow_mbps"
    ]
    - split_vs_single_link_df[
        "split_flow_mbps"
    ]
)


# Create readable link label.

split_vs_single_link_df[
    "link"
] = (
    split_vs_single_link_df[
        "from_node"
    ]
    + " → "
    + split_vs_single_link_df[
        "to_node"
    ]
)


# Keep links used by either formulation.

split_vs_single_active_link_df = (
    split_vs_single_link_df[
        (
            split_vs_single_link_df[
                "split_flow_mbps"
```

```
            ] > 1e-8
        )
        |
        (
            split_vs_single_link_df[
                "single_flow_mbps"
            ] > 1e-8
        )
    ]
    .copy()
)


split_vs_single_active_link_df[
    [
        "link",
        "capacity_mbps",
        "split_flow_mbps",
        "single_flow_mbps",
        "single_minus_split_flow_mbps"
    ]
]
```

Out [345...

	link	capacity_mbps	split_flow_mbps	single_flow_mbps	single_minus_split
0	C → UK	30.0	30.0	30.0	
1	C → US	25.0	25.0	25.0	
2	B → D	28.0	28.0	28.0	
3	A → C	25.0	24.0	25.0	
4	D → UK	22.0	20.0	18.0	
5	B → C	18.0	11.0	18.0	
6	S1 → A	40.0	13.0	15.0	
7	S2 → A	35.0	11.0	11.0	
8	D → US	30.0	8.0	10.0	
9	S3 → B	30.0	2.0	10.0	
13	C → B	18.0	0.0	5.0	
17	B → A	20.0	0.0	3.0	
18	A → B	20.0	0.0	4.0	

36.15.1 Visualization

In [346…

```
split_vs_single_active_link_df.set_index(
    "link"
)[
    [
        "split_flow_mbps",
        "single_flow_mbps"
    ]
].plot(
    kind="bar",
    figsize=(13, 6)
)


plt.ylabel(
    "Optimized Flow (Mbps)"
)

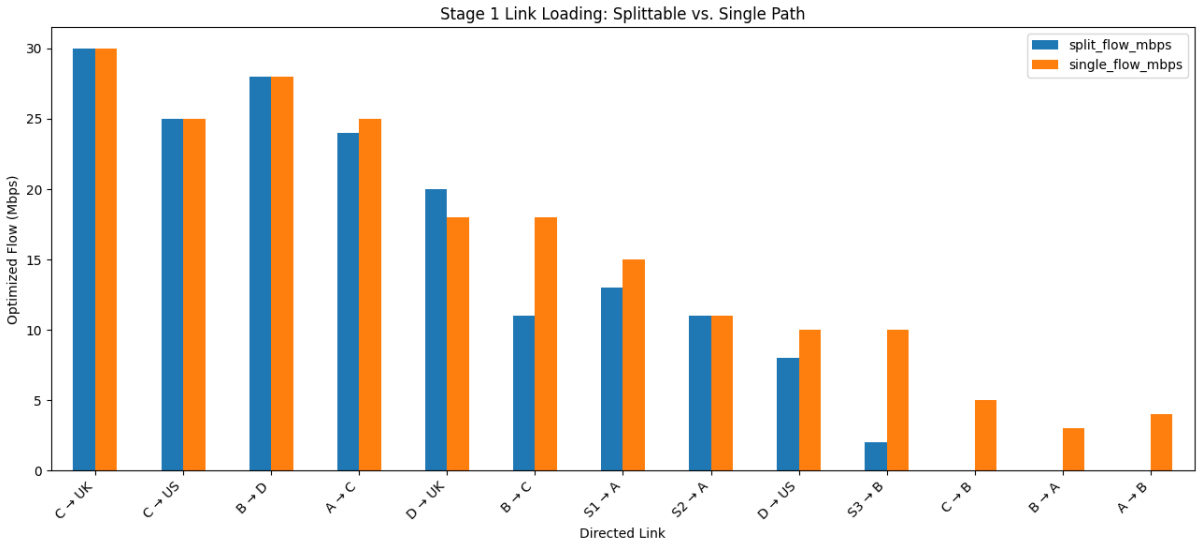
plt.xlabel(
    "Directed Link"
)

plt.title(
    "Stage 1 Link Loading: "
    "Splittable vs. Single Path"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()

plt.show()
```



36.15.2 Interpretation of Split vs. Single Link Loading

The link-level comparison demonstrates that the two formulations reach the same total throughput using different network-flow patterns.

The single-path model places more traffic on several shared corridors, including B → C and A → C.

It also admits more traffic through S3 → B than the particular splittable Stage 1 solution.

At the same time, some D-side traffic is redistributed between UK and US.

This confirms that:

$$83 \text{ Mbps}$$

is not associated with one unique network routing pattern.

Multiple throughput-optimal operating points exist.

The binary one-path restriction changes which of those operating points the solver selects.

36.16 Single-Path Stage 1 Network Visualization

The optimized single-path link flows are now placed back onto the physical network.

The same visualization function used for the splittable model is reused so that the diagrams are directly comparable.

Link thickness represents traffic volume.

Directional labels show:

$$\text{Flow}/\text{Capacity (Utilization)}.$$

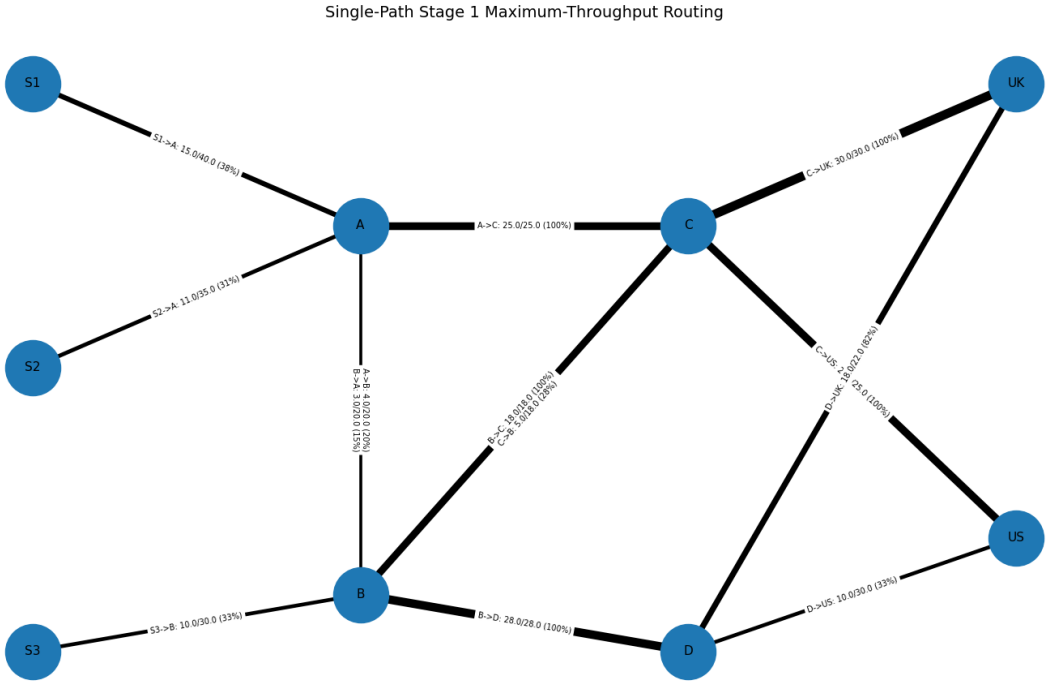
In [347…

```
plot_network_flow(
    graph=
        G_synthetic,

    link_results_df=
        single_stage1_link_df,

    pos=
        pos,
```

```
    title=(
        "Single-Path Stage 1 "
        "Maximum-Throughput Routing"
    )
)
```



36.16.1 Interpretation of the Single-Path Network Diagram

The single-path network diagram should show a more constrained routing pattern than the splittable Stage 1 solution.

Five important directed links operate at full capacity:

- A → C;
- B → C;
- B → D;
- C → UK;
- C → US.

This means the MILP is using both major exit corridors aggressively.

The model also uses internal cross-routing such as:

- A → B;
- B → A;
- C → B.

These flows reflect the need to fit one-route commodity assignments around shared directional capacities.

Despite the different traffic pattern, the network still delivers the same:

$$83 \text{ Mbps}.$$

36.16.2 Business and Operational Impact

The network diagram highlights the main tradeoff created by the single-path restriction.

The routing policy is simpler at the commodity level because each commodity follows only one route.

However, the network-wide result may require heavier use of specific shared corridors.

This can reduce operational headroom even when total throughput remains unchanged.

Therefore, route simplicity and network resilience should be evaluated separately.

36.17 Compare Number of Saturated Links

In [348…

```
# Compare saturation counts.

saturation_comparison_df = pd.DataFrame([
    {
        "model":
            "Splittable Stage 1",
        "saturated_directed_links":
            int(
                split_stage1_link_df[
                    "is_saturated"
                ].sum()
            )
    },
    {
        "model":
            "Single-Path Stage 1",
        "saturated_directed_links":
            int(
                single_stage1_link_df[
                    "is_saturated"
                ].sum()
            )
```

```
    }
])


saturation_comparison_df
```

Out [348…

	model	saturated_directed_links
0	Splittable Stage 1	3
1	Single-Path Stage 1	5

36.17.1 Interpretation of Saturation Count

The single-path Stage 1 solution saturates more directed links than the splittable Stage 1 solution:

$$5 \text{ vs. } 3.$$

Both still deliver:

$$83 \text{ Mbps}.$$

This suggests that the single-path routing restriction may consume available network headroom differently.

The result does not prove that the single-path solution is less resilient, but it provides a reason to examine stress scenarios and marginal-capacity behavior later.

36.18 Executive Comparison: Split vs. Single Stage 1

In [349…

```
stage1_split_single_executive_df = pd.DataFrame([
    {
        "metric":
            "Maximum Throughput",
        "splittable_stage1":
            split_stage1_throughput,
        "single_path_stage1":
            single_stage1_throughput
    },
    {
        "metric":
            "Lost Demand",
        "splittable_stage1":
            total_demand
            - split_stage1_throughput,
        "single_path_stage1":
            total_demand
```

```
            - single_stage1_throughput
    },
    {
        "metric":
            "Saturated Directed Links",
        "splittable_stage1":
            int(
                split_stage1_link_df[
                    "is_saturated"
                ].sum()
            ),
        "single_path_stage1":
            int(
                single_stage1_link_df[
                    "is_saturated"
                ].sum()
            )
    },
    {
        "metric":
            "Maximum Paths per Commodity",
        "splittable_stage1":
            int(
                active_route_count_by_commodity[
                    "number_of_active_routes"
                ].max()
            ),
        "single_path_stage1":
            int(
                single_path_check_df[
                    "number_of_used_paths"
                ].max()
            )
    }
])


stage1_split_single_executive_df
```

Out [349...

	metric	splittable_stage1	single_path_stage1
0	Maximum Throughput	83.0	83.0
1	Lost Demand	43.0	43.0
2	Saturated Directed Links	3.0	5.0
3	Maximum Paths per Commodity	1.0	1.0

36.19 Equal Throughput Does Not Mean Equivalent Network Behavior

One of the most important findings from Sections 33–36 is that objective-value equality should not be mistaken for solution equivalence.

Both Stage 1 models achieve:

$$83 \text{ Mbps}.$$

However, the single-path model produces a different:

- source admission pattern;
- commodity admission pattern;
- route assignment;
- link-flow pattern;
- saturation pattern.

This means:

$$T^*_{\text{single}} = T^*_{\text{split}}$$

does **not** imply:

$$x^*_{\text{single}} = x^*_{\text{split}}.$$

There can be multiple network operating points with the same maximum throughput.

This is important for both mathematical interpretation and business decision-making.

36.20 Business and Operational Interpretation

The baseline comparison produces a nuanced result.

Throughput

There is no throughput penalty for enforcing one route per commodity.

Both models deliver:

$$83 \text{ Mbps}.$$

Routing Simplicity

The single-path solution is easier to reason about at the commodity level because each served commodity follows one route.

Network Utilization

The single-path solution uses several shared corridors more aggressively and produces more saturated directed links in the current solution.

Admission Mix

The same total throughput may be distributed differently across sources and streams.

Therefore, the business should not choose between split and single-path routing based on throughput alone.

Relevant decision criteria include:

- latency;
- route complexity;
- network headroom;
- resilience;
- fairness;
- solver complexity;
- implementation cost.

36.21 Detailed Section Summary

Section 36 examines the detailed operating behavior of the single-path Stage 1 MILP.

Section 35 established that the maximum single-path throughput is:

$$83 \text{ Mbps}.$$

Section 36 shows how that throughput is achieved.

---

1. Split-Flow Dominance Is Satisfied

The theoretical relationship:

$$T^*_{\text{single}} \leq T^*_{\text{split}}$$

is satisfied.

In the baseline scenario:

$$T^*_{\text{single}} = T^*_{\text{split}} = 83 \text{ Mbps}.$$

Therefore, the additional routing flexibility of the splittable model provides:

$$0 \text{ Mbps}$$

of baseline Stage 1 throughput advantage.

---

2. The One-Path Rule Is Correctly Enforced

Every source-stream commodity uses either:

$$0$$

or:

$$1$$

positive-flow path.

No commodity uses multiple routes.

This verifies that the MILP is implementing the required single-path routing policy.

---

3. Partial Delivery Remains Possible

The route-selection constraint does not force full delivery.

A selected route may carry less traffic than the commodity's offered demand.

Therefore, the formulation correctly implements:

> one route per commodity, with partial delivery allowed.

---

4. Commodity-Level Admission Must Be Examined Separately

Although total throughput matches the splittable model, the source-stream admission pattern may differ.

This reinforces an important finding from Section 33:

> maximum throughput does not uniquely define which commodities receive service.

Multiple throughput-optimal admission vectors may exist.

---

5. Single-Path Link Loading Differs Materially

The current single-path Stage 1 solution fully saturates five directed links:

- A → C;
- B → C;
- B → D;
- C → UK;
- C → US.

The splittable Stage 1 solution saturated only three directed links.

Therefore, the same 83 Mbps of traffic can produce a materially different internal congestion pattern.

---

6. B → C Is Used More Aggressively

Under the splittable Stage 1 solution:

$$B \to C = 11/18 \approx 61.1\%.$$

Under the single-path solution:

$$B \to C = 18/18 = 100\%.$$

This is one of the clearest examples of how the single-path restriction changes network utilization.

---

7. A → C Also Reaches Full Capacity

The splittable Stage 1 solution used:

$$24/25 = 96\%$$

of A → C.

The single-path solution uses:

$$25/25 = 100\%.$$

Again, equal network throughput does not imply equal corridor utilization.

---

8. D-Side Allocation Changes

B → D remains fully utilized at:

$$28/28 \text{ Mbps}.$$

However, downstream allocation shifts.

D → UK carries:

$$18/22 \approx 81.8\%$$

while D → US carries:

$$10/30 \approx 33.3\%.$$

The split Stage 1 solution used those downstream D links differently.

This indicates that the one-path requirement changes how available D-side capacity is allocated between destinations.

---

9. S3 Admission Changes Materially

One particularly important difference appears on:

$$S3 \to B.$$

The splittable Stage 1 solution carried only:

$$2 \text{ Mbps}$$

through this mandatory source-entry link.

The current single-path Stage 1 solution carries:

$$10 \text{ Mbps}.$$

Because every S3 path must begin with S3 → B, this indicates a substantially different source-level admission decision.

This provides direct evidence of throughput-optimal degeneracy.

The two models deliver the same total network throughput while serving a different mix of demand.

---

10. Internal Cross-Routing Is Used

The single-path solution also uses internal directional flows such as:

$$A \to B = 4 \text{ Mbps},$$

$$B \to A = 3 \text{ Mbps},$$

and:

$$C \to B = 5 \text{ Mbps}.$$

These flows demonstrate that restricting each commodity to one route does not necessarily make the overall network-flow pattern trivial.

The MILP may use less intuitive internal routes to fit whole commodity routes around shared capacity constraints.

---

11. The Solution Is Physically Feasible

All directed capacity constraints are satisfied.

No optimized link exceeds its capacity.

Therefore, the 83 Mbps single-path result is a valid physical network operating point.

---

12. More Saturated Links Does Not Mean Lower Throughput

The single-path solution currently has:

$$5$$

saturated directed links compared with:

$$3$$

in the splittable Stage 1 solution.

Nevertheless, both deliver:

$$83 \text{ Mbps}.$$

This demonstrates that link saturation counts alone are not sufficient to compare network performance.

The later bottleneck analysis must determine whether additional capacity on a saturated link actually increases end-to-end throughput.

---

13. Technical Conclusion

The single-path Stage 1 MILP passes the major validation checks.

It:

- satisfies split-flow dominance;
- enforces at most one positive-flow route per commodity;
- supports partial delivery;
- satisfies all directed-link capacities;
- achieves an optimal throughput of 83 Mbps.

The formulation is therefore functioning as intended.

---

14. Business Conclusion

For the baseline network, single-path routing achieves the same maximum throughput as splittable routing.

This means the organization could potentially gain commodity-level routing simplicity without sacrificing total baseline throughput.

However, the detailed results show that the two formulations are not operationally equivalent.

They can differ in:

- who is served;
- which routes are used;
- where capacity is consumed;
- how many links are saturated.

Therefore, routing-policy selection should consider more than total throughput.

---

15. Connection to Section 37

Sections 35 and 36 establish the maximum-throughput single-path operating point.

The next question mirrors the analysis performed for the splittable model:

> **Can latency-sensitive traffic be routed more efficiently while**

> **preserving the exact single-path Stage 1 admission vector and the one-path restriction?**

Section 37 therefore introduces the Single-Path Stage 2 MILP.

It will preserve the Stage 1 delivered quantity of every commodity while minimizing latency exposure for:

- UK-L;
- US-L.

This allows the final latency performance of the single-path and splittable formulations to be compared on a consistent basis.

37. Single-Path MILP — Stage 2: Minimize Sensitive-Traffic Latency

Sections 35 and 36 established the maximum-throughput solution for the single-path routing formulation.

The Single-Path Stage 1 MILP delivers:

$$83 \text{ Mbps}$$

from:

$$126 \text{ Mbps}$$

of offered demand.

Section 36 then examined:

- which commodities were admitted;
- which one route each delivered commodity used;
- link utilization;
- saturated links;
- differences from the splittable Stage 1 solution.

Section 37 now performs the single-path equivalent of the splittable Stage 2 optimization from Section 34.

The question is:

> **Given the exact Single-Path Stage 1 admission decision, can the selected routes be changed to reduce latency for UK-L and US-**

> **L while preserving both throughput and the one-path restriction?**

The optimization remains lexicographic.

Priority 1 — Throughput

Stage 1 already determined the maximum feasible throughput:

$$83 \text{ Mbps}.$$

Priority 2 — Sensitive-Traffic Latency

Stage 2 preserves each Stage 1 source-stream delivery quantity and minimizes latency exposure for:

$$UK\text{-}L$$

and:

$$US\text{-}L.$$

The single-path restriction remains active throughout Stage 2.

Therefore, Stage 2 may change the selected route for a commodity, but it may not:

- change how much that commodity is delivered;
- split the commodity across multiple paths;
- violate any directional link capacity.

This creates a controlled comparison between maximum-throughput routing and latency-optimized routing under the one-path operational policy.

37.1 Why a Separate Single-Path Stage 2 Is Required

The Single-Path Stage 1 objective considered only total delivered traffic.

It did not assign any cost to latency.

Therefore, the Stage 1 MILP was free to place latency-sensitive traffic on slower routes if those routes supported maximum throughput.

Stage 2 introduces service quality only after maximum throughput has been established.

This is the same lexicographic principle used for the splittable model:

1. maximize throughput;
2. preserve the throughput-optimal admission vector;
3. minimize latency for sensitive traffic.

However, the single-path model adds one additional requirement:

> Each source-stream commodity must still use at most one route.

Therefore, the Stage 2 single-path feasible region is more restricted than the Stage 2 splittable feasible region.

37.1.1 Business and Operational Impact

This formulation represents a practical operating policy:

> Serve the maximum possible traffic, preserve those service commitments, and then choose the best single route available for latency-sensitive traffic.

It maintains route simplicity while still allowing the optimizer to improve service quality.

37.2 Create the Single-Path Stage 2 MILP

Stage 2 is again a mixed-integer linear program because it requires:

- continuous path-flow variables;
- binary path-selection variables.

The optimization direction changes from maximization to minimization because the secondary objective is latency.

In [351…

```
# ------------------------------------------------------------
# Create the Single-Path Stage 2 optimization model
# ------------------------------------------------------------
#
# Stage 1 maximized total throughput.
#
# Stage 2 instead minimizes latency exposure for UK-L and US-L
# while preserving the exact Stage 1 delivered amount for
# every source-stream commodity.

single_stage2 = pl.LpProblem(
    "Single_Path_Stage2",
```

```
    pl.LpMinimize
)


print(
    "Model created:",
    single_stage2.name
)

print(
    "Optimization sense:",
    "Minimize"
)
```

```
Model created: Single_Path_Stage2
Optimization sense: Minimize
```

37.3 Create Stage 2 Flow and Route-Selection Variables

As in Single-Path Stage 1, every candidate path receives:

1. one continuous flow variable;
2. one binary selection variable.

For path p:

$$f^{(2)}_{k,s,p} \geq 0$$

and:

$$y^{(2)}_{k,s,p} \in \{0, 1\}.$$

New variables are required because Stage 2 is allowed to reconsider the selected routes.

The amount delivered to each commodity will be fixed later by equality constraints.

In [352…

```
# ------------------------------------------------------------
# Create Single-Path Stage 2 decision variables
# ------------------------------------------------------------

single_stage2_flow = {}
single_stage2_select = {}


for _, row in (
```

```
    candidate_paths_df.iterrows()
):

    # Unique candidate-path identifier.

    pid = int(
        row[
            "path_id"
        ]
    )


    # ----------------------------------------------------------
    # Continuous flow variable
    # ----------------------------------------------------------
    #
    # Represents Mbps routed on this candidate path.

    single_stage2_flow[
        pid
    ] = pl.LpVariable(
        f"single_stage2_flow_{pid}",
        lowBound=0,
        cat="Continuous"
    )


    # ----------------------------------------------------------
    # Binary route-selection variable
    # ----------------------------------------------------------
    #
    # 1 = route selected
    # 0 = route unavailable for positive flow

    single_stage2_select[
        pid
    ] = pl.LpVariable(
        f"single_stage2_select_{pid}",
        lowBound=0,
        upBound=1,
        cat="Binary"
    )


print(
    "Stage 2 continuous flow variables:",
    len(
        single_stage2_flow
    )
)

print(
```

```
    "Stage 2 binary selection variables:",
    len(
        single_stage2_select
    )
)

print(
    "Stage 2 total variables:",
    len(
        single_stage2_flow
    )
    + len(
        single_stage2_select
    )
)
```

```
Stage 2 continuous flow variables: 68
Stage 2 binary selection variables: 68
Stage 2 total variables: 136
```

37.3.1 Validate Variables

In [353…

```
# Confirm one continuous and one binary variable exist
# for every candidate path.

assert (
    len(
        single_stage2_flow
    )
    ==
    len(
        candidate_paths_df
    )
)

assert (
    len(
        single_stage2_select
    )
    ==
    len(
        candidate_paths_df
    )
)


expected_path_ids = set(
    candidate_paths_df[
        "path_id"
    ].astype(int)
)
```

```
assert (
    set(
        single_stage2_flow.keys()
    )
    ==
    expected_path_ids
)


assert (
    set(
        single_stage2_select.keys()
    )
    ==
    expected_path_ids
)


print(
    "Single-Path Stage 2 variable validation passed."
)
```

```
Single-Path Stage 2 variable validation passed.
```

37.4 Define the Sensitive-Traffic Latency Objective

The objective matches the splittable Stage 2 objective.

Only UK-L and US-L contribute latency cost.

The objective is:

$$\boxed{\min \sum_{k \in \{UK-L, US-L\}} \sum_{s} \sum_{p \in P_{k,s}} L_p f_{k,s,p}^{(2)}}$$

where:

- L_p is path latency;
- $f_{k,s,p}^{(2)}$ is Stage 2 flow.

The objective therefore minimizes traffic-weighted latency exposure for admitted sensitive traffic.

Tolerant traffic still consumes capacity but does not contribute directly to the latency objective.

In [354…
```
# ------------------------------------------------------------
# Define Single-Path Stage 2 sensitive-latency objective
# ------------------------------------------------------------

single_stage2 += pl.lpSum(

    row[
        "path_latency_ms"
    ]
    * single_stage2_flow[
        int(
            row[
                "path_id"
            ]
        )
    ]

    for _, row
    in candidate_paths_df.iterrows()

    if row[
        "stream"
    ]
    in sensitive_streams
)


print(
    "Single-Path Stage 2 latency objective added."
)
```

```
Single-Path Stage 2 latency objective added.
```

37.5 Extract the Single-Path Stage 1 Admission Vector

Stage 2 must preserve the exact amount of each commodity delivered during Single-Path Stage 1.

For commodity (k, s):

$$z_{k,s}^{(1)} = \sum_{p} f_{k,s,p}^{(1)}.$$

These Stage 1 values become equality targets in Stage 2.

In [355…
```
# ------------------------------------------------------------
# Extract Single-Path Stage 1 delivery by commodity
# ------------------------------------------------------------
```

```
single_stage1_delivery_lookup = {}


for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    # Sum Single-Path Stage 1 traffic across all candidate paths.
    #
    # At most one path should contain positive flow,
    # but summing remains robust and general.

    delivered = sum(
        (
            single_flow_vars[
                int(pid)
            ].value()
            or 0
        )
        for pid
        in group[
            "path_id"
        ]
    )


    single_stage1_delivery_lookup[
        commodity_id
    ] = float(
        delivered
    )


print(
    "Stage 1 commodity admission values extracted:",
    len(
        single_stage1_delivery_lookup
    )
)
```

```
Stage 1 commodity admission values extracted: 20
```

37.6 Preserve Stage 1 Delivery and the One-Path Restriction

Two constraints are added for every commodity.

Admission Preservation

$$\sum_p f^{(2)}_{k,s,p} = z^{(1)}_{k,s}.$$

This prevents Stage 2 from changing who receives service.

One-Path Selection

$$\sum_p y^{(2)}_{k,s,p} \leq 1.$$

This ensures that latency optimization cannot introduce multipath routing.

In [356…

```
# ------------------------------------------------------------
# Add delivery-preservation and one-path constraints
# ------------------------------------------------------------

for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    # --------------------------------------------------------
    # Preserve exact Stage 1 delivery
    # --------------------------------------------------------

    single_stage2 += (
        pl.lpSum(
            single_stage2_flow[
                int(pid)
            ]
            for pid
            in group[
                "path_id"
            ]
        )
        ==
        single_stage1_delivery_lookup[
            commodity_id
        ],

        f"preserve_{commodity_id}"
    )


    # --------------------------------------------------------
    # Preserve single-path policy
    # --------------------------------------------------------

    single_stage2 += (
        pl.lpSum(
```

```
            single_stage2_select[
                int(pid)
            ]
            for pid
            in group[
                "path_id"
            ]
        )
        <= 1,

        f"one_path_{commodity_id}"
    )


print(
    "Delivery-preservation constraints added:",
    len(
        single_stage1_delivery_lookup
    )
)

print(
    "One-path constraints added:",
    len(
        single_stage1_delivery_lookup
    )
)
```

```
Delivery-preservation constraints added: 20
One-path constraints added: 20
```

37.7 Link Stage 2 Flow to Binary Route Selection

Positive flow may occur only on a selected route.

For every candidate path:

$$f^{(2)}_{k,s,p} \leq D_{k,s} y^{(2)}_{k,s,p}.$$

This retains the same logic as Stage 1.

If:

$$y = 0,$$

then:

$$f = 0.$$

If:

$$y = 1,$$

partial or full delivery may occur, subject to the preserved Stage 1 total.

In [357…

```
# ------------------------------------------------------------
# Add Stage 2 flow-to-selection linking constraints
# ------------------------------------------------------------

for _, row in (
    candidate_paths_df.iterrows()
):

    # Candidate path ID.

    pid = int(
        row[
            "path_id"
        ]
    )


    # Commodity demand provides a tight upper bound
    # for flow on the selected route.

    demand = float(
        row[
            "demand_mbps"
        ]
    )


    single_stage2 += (
        single_stage2_flow[
            pid
        ]
        <=
        demand
        * single_stage2_select[
            pid
        ],

        f"flow_select_{pid}"
    )


print(
    "Flow-selection constraints added:",
    len(
        candidate_paths_df
    )
)
```

```
Flow-selection constraints added: 68
```

37.8 Reapply Directed Link-Capacity Constraints

Changing routes does not change the physical network.

Every Stage 2 routing configuration must still satisfy:

$$\sum_{k,s} \sum_{p:e\in p} f_{k,s,p}^{(2)} \leq C_e.$$

The same 22 directional capacities used throughout the notebook are therefore reapplied.

In [358…

```
# ------------------------------------------------------------
# Add directed link-capacity constraints
# ------------------------------------------------------------

for _, arc in (
    arcs_df.iterrows()
):

    # Represent current directed arc.

    edge = (
        arc[
            "from_node"
        ],
        arc[
            "to_node"
        ]
    )


    # Identify all candidate paths containing this direction.

    using_edge = (
        candidate_paths_df[
            candidate_paths_df[
                "path_edges"
            ].apply(
                lambda edges:
                    edge in edges
            )
        ]
    )


    # Restrict total flow across this directional link.
```

```
    single_stage2 += (
        pl.lpSum(
            single_stage2_flow[
                int(pid)
            ]
            for pid
            in using_edge[
                "path_id"
            ]
        )
        <=
        float(
            arc[
                "capacity_mbps"
            ]
        ),

        f"capacity_"
        f"{arc['from_node']}_"
        f"{arc['to_node']}"
    )


print(
    "Stage 2 directed capacity constraints added:",
    len(
        arcs_df
    )
)
```

```
Stage 2 directed capacity constraints added: 22
```

37.9 Validate Single-Path Stage 2 Model Structure

The expected model dimensions are:

Variables

- 68 continuous path-flow variables;
- 68 binary selection variables.

Total:

$$136.$$

Constraints

- 20 delivery-preservation equalities;

- 20 one-path constraints;
- 68 flow-selection linking constraints;
- 22 directional capacity constraints.

Total:

$$20 + 20 + 68 + 22 = 130.$$

In [360…

```
single_stage2_variable_count = (
    len(
        single_stage2.variables()
    )
)


single_stage2_constraint_count = (
    len(
        single_stage2.constraints
    )
)


print(
    "Single-Path Stage 2 variables:",
    single_stage2_variable_count
)

print(
    "Single-Path Stage 2 constraints:",
    single_stage2_constraint_count
)


assert (
    single_stage2_variable_count
    == 136
)


assert (
    single_stage2_constraint_count
    == 130
)


print(
    "Single-Path Stage 2 model-dimension validation passed."
)
```

```
Single-Path Stage 2 variables: 136
Single-Path Stage 2 constraints: 130
Single-Path Stage 2 model-dimension validation passed.
```

37.10 Solve the Single-Path Stage 2 MILP

CBC now searches for the minimum sensitive-traffic latency solution subject to:

- the exact Single-Path Stage 1 commodity admission vector;
- at most one selected route per commodity;
- flow-selection consistency;
- all directional capacity limits.

In [361…

```
# ------------------------------------------------------------
# Solve Single-Path Stage 2
# ------------------------------------------------------------

single_stage2.solve(
    cbc_solver
)


single_stage2_status = (
    pl.LpStatus[
        single_stage2.status
    ]
)


single_stage2_latency_objective = (
    pl.value(
        single_stage2.objective
    )
)


print(
    "Stage 2 status:",
    single_stage2_status
)

print(
    "Sensitive latency exposure objective:",
    single_stage2_latency_objective,
    "Mbps*ms"
)
```

```
Stage 2 status: Optimal
Sensitive latency exposure objective: 700.0 Mbps*ms
```

37.10.1 Validate Optimality

In [362…
```
assert (
    single_stage2_status
    == "Optimal"
)


assert (
    single_stage2_latency_objective
    is not None
)


print(
    "Single-Path Stage 2 solver validation passed."
)
```

```
Single-Path Stage 2 solver validation passed.
```

37.11 Validate Stage 1 Admission Preservation

The most important Stage 2 validation is that no commodity's delivered quantity changes.

For every commodity:

$$z_{k,s}^{(2)} = z_{k,s}^{(1)}.$$

In [363…
```
single_stage2_delivery_records = []


for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    stage2_delivered = sum(
        (
            single_stage2_flow[
                int(pid)
            ].value()
            or 0
        )
        for pid
        in group[
            "path_id"
        ]
```

```
    )


    stage1_delivered = (
        single_stage1_delivery_lookup[
            commodity_id
        ]
    )


    single_stage2_delivery_records.append({
        "commodity_id":
            commodity_id,
        "stage1_delivered_mbps":
            stage1_delivered,
        "stage2_delivered_mbps":
            stage2_delivered,
        "change_mbps":
            stage2_delivered
            - stage1_delivered
    })


single_stage1_vs_stage2_delivery_df = (
    pd.DataFrame(
        single_stage2_delivery_records
    )
)


single_stage1_vs_stage2_delivery_df
```

Out [363…

	commodity_id	stage1_delivered_mbps	stage2_delivered_mbps	change_mb
0	B_UK-L	6.0	6.0	0
1	B_UK-T	14.0	14.0	0
2	B_US-L	8.0	8.0	0
3	B_US-T	2.0	2.0	0
4	C_UK-L	1.0	1.0	0
5	C_UK-T	5.0	5.0	0
6	C_US-L	5.0	5.0	0
7	C_US-T	6.0	6.0	0
8	S1_UK-L	0.0	0.0	0
9	S1_UK-T	4.0	4.0	0
10	S1_US-L	5.0	5.0	0
11	S1_US-T	6.0	6.0	0
12	S2_UK-L	5.0	5.0	0
13	S2_UK-T	6.0	6.0	0
14	S2_US-L	0.0	0.0	0
15	S2_US-T	0.0	0.0	0
16	S3_UK-L	0.0	0.0	0
17	S3_UK-T	7.0	7.0	0
18	S3_US-L	0.0	0.0	0
19	S3_US-T	3.0	3.0	0

In [365…

```
##validation
max_single_delivery_change = (
    single_stage1_vs_stage2_delivery_df[
        "change_mbps"
    ]
    .abs()
    .max()
)


assert (
    max_single_delivery_change
    < 1e-6
)
```

```
print(
    "Maximum commodity delivery change:",
    max_single_delivery_change,
    "Mbps"
)

print(
    "All Single-Path Stage 1 admission quantities preserved."
)
```

```
Maximum commodity delivery change: 0.0 Mbps
All Single-Path Stage 1 admission quantities preserved.
```

37.12 Validate Total Throughput Preservation

In [366…

```
single_stage2_total_throughput = (
    single_stage1_vs_stage2_delivery_df[
        "stage2_delivered_mbps"
    ].sum()
)


print(
    "Single-Path Stage 1 throughput:",
    single_stage1_throughput,
    "Mbps"
)

print(
    "Single-Path Stage 2 throughput:",
    single_stage2_total_throughput,
    "Mbps"
)


assert abs(
    single_stage2_total_throughput
    - single_stage1_throughput
) < 1e-6


print(
    "Single-Path throughput preserved."
)
```

```
Single-Path Stage 1 throughput: 83.0 Mbps
Single-Path Stage 2 throughput: 83.0 Mbps
Single-Path throughput preserved.
```

37.13 Validate One-Path Routing After Stage 2

In [367…

```
single_stage2_path_check = []


for commodity_id, group in (
    candidate_paths_df.groupby(
        "commodity_id"
    )
):

    positive_paths = sum(
        1
        for pid
        in group[
            "path_id"
        ]
        if (
            single_stage2_flow[
                int(pid)
            ].value()
            or 0
        ) > 1e-8
    )


    selected_paths = sum(
        int(
            round(
                single_stage2_select[
                    int(pid)
                ].value()
                or 0
            )
        )
        for pid
        in group[
            "path_id"
        ]
    )


    single_stage2_path_check.append({
        "commodity_id":
            commodity_id,
        "positive_flow_paths":
            positive_paths,
        "selected_paths":
            selected_paths
    })


single_stage2_path_check_df = (
```

```
    pd.DataFrame(
        single_stage2_path_check
    )
)


assert (
    single_stage2_path_check_df[
        "positive_flow_paths"
    ]
    <= 1
).all()


assert (
    single_stage2_path_check_df[
        "selected_paths"
    ]
    <= 1
).all()


print(
    "Single-Path Stage 2 route-count validation passed."
)
```

```
Single-Path Stage 2 route-count validation passed.
```

37.14 Stage 2 Technical Interpretation

The Single-Path Stage 2 MILP now satisfies all required policy rules simultaneously.

It:

- preserves the exact Stage 1 delivered quantity for every commodity;
- preserves total throughput of 83 Mbps;
- preserves the one-route-per-commodity restriction;
- respects all directional capacities;
- minimizes latency exposure for UK-L and US-L.

Any difference between the Stage 1 and Stage 2 route maps can therefore be attributed entirely to the latency objective.

37.15 Business and Operational Impact

The Single-Path Stage 2 model represents a particularly practical optimization

policy.

The business keeps:

- the same total service volume;
- the same customers or source-stream commodities admitted;
- the same one-route operational restriction.

Within those constraints, the model attempts to provide lower latency to delay-sensitive traffic.

Therefore, any latency improvement represents a service-quality gain rather than a reduction in service volume.

37.16 Section Summary

Section 37 constructs the second-stage single-path latency optimization.

The Stage 1 admission vector is treated as fixed.

Each commodity remains restricted to at most one route.

The Stage 2 MILP contains:

- 68 continuous flow variables;
- 68 binary route-selection variables;
- 136 total variables;
- 130 constraints.

The constraints include:

- 20 delivery-preservation equalities;
- 20 one-path constraints;
- 68 flow-selection constraints;
- 22 directional capacity constraints.

The model therefore asks a tightly controlled question:

> **What is the lowest sensitive-traffic latency achievable without changing throughput, admission, or the one-path operating policy?**

Section 38 now examines the final routing and the resulting latency performance.

38. Final Single-Path Results

Section 37 solved the second-stage single-path MILP.

The final solution preserves:

$$83 \text{ Mbps}$$

of total Stage 1 throughput and the exact delivered quantity of every source-stream commodity.

The purpose of Section 38 is to examine what changed when the latency objective was introduced.

The analysis focuses on:

1. final selected routes;
2. Stage 1-to-Stage 2 route changes;
3. final directed-link utilization;
4. changes in saturated links;
5. the final network-flow diagram;
6. UK-L and US-L latency;
7. the amount of latency improvement achieved;
8. the operational implications of the one-path restriction.

38.2 Extract Final Single-Path Routes

In [368…

```
single_stage2_route_records = []


for _, row in (
    candidate_paths_df.iterrows()
):

    pid = int(
        row[
            "path_id"
        ]
    )


    flow = (
        single_stage2_flow[
            pid
        ].value()
        or 0
```

```
    )


    selected = (
        single_stage2_select[
            pid
        ].value()
        or 0
    )


    if flow > 1e-8:

        single_stage2_route_records.append({
            "commodity_id":
                row[
                    "commodity_id"
                ],
            "source":
                row[
                    "source"
                ],
            "stream":
                row[
                    "stream"
                ],
            "destination":
                row[
                    "destination"
                ],
            "path":
                row[
                    "path"
                ],
            "flow_mbps":
                flow,
            "selected_binary":
                selected,
            "path_latency_ms":
                row[
                    "path_latency_ms"
                ],
            "path_capacity_mbps":
                row[
                    "path_capacity_mbps"
                ]
        })


single_stage2_routes_df = (
    pd.DataFrame(
        single_stage2_route_records
```

```
    )
)


print(
    "Final active single-path routes:",
    len(
        single_stage2_routes_df
    )
)


single_stage2_routes_df
```

```
Final active single-path routes: 15
```

Out[368…

	commodity_id	source	stream	destination	path	flow_mbps	selected_bina
0	S1_UK-T	S1	UK-T	UK	S1 -> A -> B -> D -> UK	4.0	
1	S1_US-L	S1	US-L	US	S1 -> A -> C -> US	5.0	
2	S1_US-T	S1	US-T	US	S1 -> A -> B -> C -> US	6.0	
3	S2_UK-L	S2	UK-L	UK	S2 -> A -> C -> UK	5.0	
4	S2_UK-T	S2	UK-T	UK	S2 -> A -> B -> C -> UK	6.0	
5	S3_UK-T	S3	UK-T	UK	S3 -> B -> A -> C -> UK	7.0	

6	S3_US-T	S3	US-T	US	S3 -> B -> A -> C -> US	3.0
7	B_UK-L	B	UK-L	UK	B -> C -> UK	6.0
8	B_UK-T	B	UK-T	UK	B -> D -> UK	14.0
9	B_US-L	B	US-L	US	B -> D -> US	8.0
10	B_US-T	B	US-T	US	B -> D -> US	2.0
11	C_UK-L	C	UK-L	UK	C -> UK	1.0
12	C_UK-T	C	UK-T	UK	C -> UK	5.0
13	C_US-L	C	US-L	US	C -> US	5.0
14	C_US-T	C	US-T	US	C -> US	6.0

38.2.1 Interpretation of Final Route Assignments

Each admitted source-stream commodity continues to use exactly one route.

However, Stage 2 may select a different route from Stage 1 when doing so improves sensitive-traffic latency while remaining capacity feasible.

The final route table therefore represents the latency-optimized one-path routing plan.

Unlike the splittable Stage 2 result, no commodity can divide traffic across multiple routes.

This gives the network less flexibility when trying to move sensitive traffic onto the lowest-latency corridors.

38.3 Compare Single-Path Stage 1 and Stage 2 Routes

In [369…

```
# ------------------------------------------------------------
# Build path-level comparison between Single Stage 1 and Stage 2
# ------------------------------------------------------------

single_path_stage_compare_df = (
    candidate_paths_df[
        [
            "path_id",
            "commodity_id",
            "source",
            "stream",
            "destination",
            "path",
            "path_latency_ms"
        ]
    ]
    .copy()
)


single_path_stage_compare_df[
    "stage1_flow_mbps"
] = (
    single_path_stage_compare_df[
        "path_id"
    ].apply(
        lambda pid:
            single_flow_vars[
                int(pid)
            ].value()
            or 0
    )
)


single_path_stage_compare_df[
    "stage2_flow_mbps"
] = (
    single_path_stage_compare_df[
        "path_id"
    ].apply(
        lambda pid:
            single_stage2_flow[
                int(pid)
            ].value()
            or 0
    )
)
```

```
single_path_stage_compare_df[
    "flow_change_mbps"
] = (
    single_path_stage_compare_df[
        "stage2_flow_mbps"
    ]
    - single_path_stage_compare_df[
        "stage1_flow_mbps"
    ]
)


single_stage_route_changes_df = (
    single_path_stage_compare_df[
        single_path_stage_compare_df[
            "flow_change_mbps"
        ].abs()
        > 1e-8
    ]
    .sort_values(
        [
            "commodity_id",
            "path_latency_ms"
        ]
    )
    .reset_index(
        drop=True
    )
)


single_stage_route_changes_df[
    [
        "commodity_id",
        "stream",
        "path",
        "path_latency_ms",
        "stage1_flow_mbps",
        "stage2_flow_mbps",
        "flow_change_mbps"
    ]
]
```

Out [369...

	commodity_id	stream	path	path_latency_ms	stage1_flow_mbps	stage2_
0	B_UK-L	UK-L	B -> C -> UK	17	0.0	
1	B_UK-L	UK-L	B -> D -> UK	34	6.0	
			B ->			

2	B_UK-T	UK-T	C -> UK	17	14.0
3	B_UK-T	UK-T	B -> D -> UK	34	0.0
4	S1_UK-T	UK-T	S1 -> A -> B -> C -> UK	44	4.0
5	S1_UK-T	UK-T	S1 -> A -> B -> D -> UK	61	0.0
6	S1_US-T	US-T	S1 -> A -> C -> US	29	6.0
7	S1_US-T	US-T	S1 -> A -> B -> C -> US	46	0.0
8	S2_UK-L	UK-L	S2 -> A -> C -> UK	29	0.0
9	S2_UK-L	UK-L	S2 -> A -> C -> B -> D -> UK	66	5.0
10	S2_UK-T	UK-T	S2 -> A -> C -> UK	29	6.0
11	S2_UK-T	UK-T	S2 -> A -> B	46	0.0

			-> C -> UK		
12	S3_UK-T	UK-T	S3 -> B -> A -> C -> UK	40	0.0
13	S3_UK-T	UK-T	S3 -> B -> D -> UK	44	7.0

38.3.1 Interpretation of Stage 1-to-Stage 2 Route Changes

This table isolates route changes caused by the sensitive-latency objective.

Because commodity-level delivered quantities are fixed, any positive flow added to one route must be offset by removal from another route belonging to the same commodity.

For sensitive traffic, Stage 2 should generally favor lower-latency paths when capacity permits.

Tolerant traffic may move to alternative one-path routes in order to free preferred capacity for UK-L and US-L.

Unlike the splittable Stage 2 model, tolerant traffic cannot be divided across multiple routes.

That restriction reduces the number of possible capacity-rebalancing strategies.

38.4 Final Single-Path Link Utilization

In [370…

```
# ------------------------------------------------------------
# Calculate final Single-Path Stage 2 link utilization
# ------------------------------------------------------------

single_stage2_link_df = (
    calculate_link_results(
        candidate_paths_df,
        single_stage2_flow,
        arcs_df,
        "Single Final"
```

```
    )
)


single_stage2_link_df = (
    single_stage2_link_df
    .sort_values(
        "utilization_percentage",
        ascending=False
    )
    .reset_index(
        drop=True
    )
)


single_stage2_link_df
```

Out[370…

	model	from_node	to_node	capacity_mbps	flow_mbps	residual_capacity_
0	Single Final	B	C	18.0	18.0	
1	Single Final	B	D	28.0	28.0	
2	Single Final	C	US	25.0	25.0	
3	Single Final	C	UK	30.0	30.0	
4	Single Final	D	UK	22.0	18.0	
5	Single Final	A	B	20.0	16.0	
6	Single Final	A	C	25.0	20.0	
7	Single Final	B	A	20.0	10.0	
8	Single Final	S1	A	40.0	15.0	
9	Single Final	D	US	30.0	10.0	
10	Single Final	S3	B	30.0	10.0	
11	Single Final	S2	A	35.0	11.0	
12	Single	UK	C	30.0	0.0	

	Final				
13	Single Final	UK	D	22.0	0.0
14	Single Final	US	C	25.0	0.0
15	Single Final	C	B	18.0	0.0
16	Single Final	D	B	28.0	0.0
17	Single Final	A	S1	40.0	0.0
18	Single Final	C	A	25.0	0.0
19	Single Final	B	S3	30.0	0.0
20	Single Final	A	S2	35.0	0.0
21	Single Final	US	D	30.0	0.0

38.4.1 Interpretation of Final Single-Path Link Utilization

The final Stage 2 single-path routing produces four saturated directed links.

B → C

$$18/18 = 100\%$$

B → D

$$28/28 = 100\%$$

C → US

$$25/25 = 100\%$$

C → UK

$$30/30 = 100\%.$$

Several additional links operate at substantial utilization.

D → UK

$$18/22 \approx 81.82\%.$$

A → B

$$16/20 = 80\%.$$

A → C

$$20/25 = 80\%.$$

B → A

$$10/20 = 50\%.$$

S1 → A

$$15/40 = 37.5\%.$$

D → US

$$10/30 = 33.33\%.$$

S3 → B

$$10/30 = 33.33\%.$$

S2 → A

$$11/35 \approx 31.43\%.$$

The Stage 2 single-path solution therefore uses both the C and D corridors heavily while also relying on A ↔ B internal connectivity.

Compared with Single-Path Stage 1, A → C is no longer saturated, while A → B becomes much more heavily used.

This indicates that the latency objective causes meaningful internal rerouting without changing total throughput.

38.5 38.5 Compare Single Stage 1 vs Final Stage 2 Link Loading

In [371…

```
single_stage_link_comparison_df = (
    single_stage1_link_df[
        [
            "from_node",
            "to_node",
            "capacity_mbps",
            "flow_mbps",
            "utilization_percentage"
        ]
    ]
    .rename(
        columns={
            "flow_mbps":
                "stage1_flow_mbps",
            "utilization_percentage":
                "stage1_utilization_pct"
        }
    )
    .merge(
        single_stage2_link_df[
            [
                "from_node",
                "to_node",
                "flow_mbps",
                "utilization_percentage"
            ]
        ]
        .rename(
            columns={
                "flow_mbps":
                    "stage2_flow_mbps",
                "utilization_percentage":
                    "stage2_utilization_pct"
            }
        ),
        on=[
            "from_node",
            "to_node"
        ],
        how="inner"
    )
)


single_stage_link_comparison_df[
    "flow_change_mbps"
] = (
    single_stage_link_comparison_df[
        "stage2_flow_mbps"
    ]
    - single_stage_link_comparison_df[
        "stage1_flow_mbps"
```

```
    ]
)


single_stage_link_comparison_df[
    "link"
] = (
    single_stage_link_comparison_df[
        "from_node"
    ]
    + " → "
    + single_stage_link_comparison_df[
        "to_node"
    ]
)


single_stage_link_active_df = (
    single_stage_link_comparison_df[
        (
            single_stage_link_comparison_df[
                "stage1_flow_mbps"
            ] > 1e-8
        )
        |
        (
            single_stage_link_comparison_df[
                "stage2_flow_mbps"
            ] > 1e-8
        )
    ]
    .copy()
)


single_stage_link_active_df[
    [
        "link",
        "capacity_mbps",
        "stage1_flow_mbps",
        "stage2_flow_mbps",
        "flow_change_mbps"
    ]
]
```

Out [371...

	link	capacity_mbps	stage1_flow_mbps	stage2_flow_mbps	flow_change_m
0	B → C	18.0	18.0	18.0	
1	C → US	25.0	25.0	25.0	
2	C → UK	30.0	30.0	30.0	
3	A → C	25.0	25.0	20.0	
4	B → D	28.0	28.0	28.0	
5	D → UK	22.0	18.0	18.0	
6	S1 → A	40.0	15.0	15.0	
7	D → US	30.0	10.0	10.0	
8	S3 → B	30.0	10.0	10.0	
9	S2 → A	35.0	11.0	11.0	
10	C → B	18.0	5.0	0.0	
11	A → B	20.0	4.0	16.0	
12	B → A	20.0	3.0	10.0	

38.5.1 Visualization

In [372…

```
single_stage_link_active_df.set_index(
    "link"
)[
    [
        "stage1_flow_mbps",
        "stage2_flow_mbps"
    ]
].plot(
    kind="bar",
    figsize=(13, 6)
)


plt.ylabel(
    "Optimized Flow (Mbps)"
)

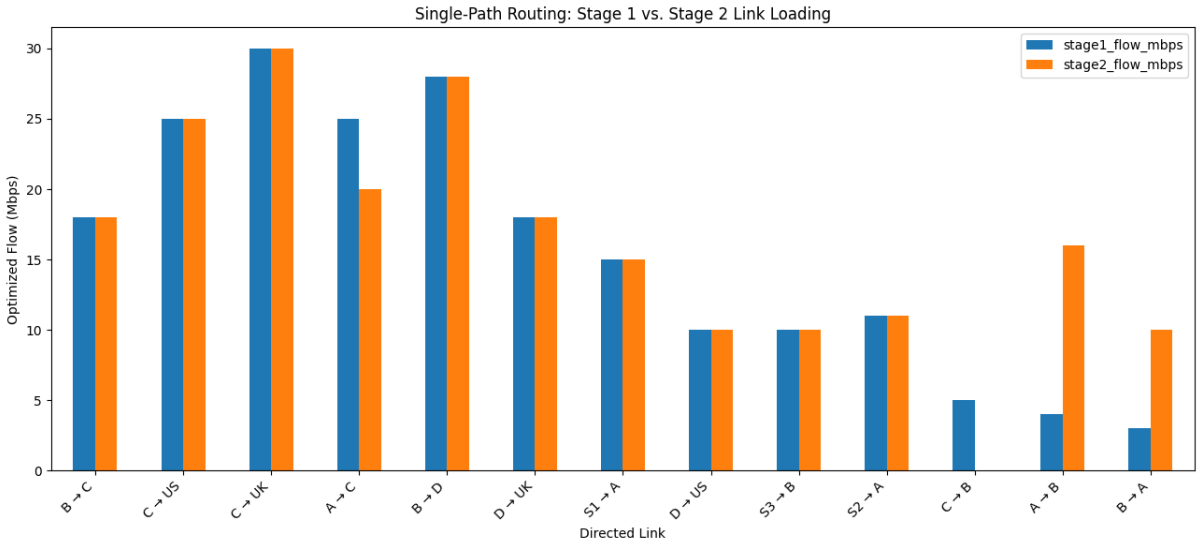
plt.xlabel(
    "Directed Link"
)

plt.title(
    "Single-Path Routing: "
    "Stage 1 vs. Stage 2 Link Loading"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()

plt.show()
```



38.5.2 Interpretation of Link-Loading Changes

Several major corridor loads remain unchanged between Single-Path Stage 1 and Stage 2.

For example:

$$B \to C = 18 \text{ Mbps}$$

$$B \to D = 28 \text{ Mbps}$$

$$C \to UK = 30 \text{ Mbps}$$

$$C \to US = 25 \text{ Mbps}.$$

These heavily constrained links continue to define much of the network structure.

However, Stage 2 changes the internal A-B-C routing pattern substantially.

A → C

Stage 1:

$$25 \text{ Mbps}$$

Stage 2:

$$20 \text{ Mbps}.$$

Change:

$$-5 \text{ Mbps}.$$

C → B

Stage 1:

$$5 \text{ Mbps}$$

Stage 2:

$$0 \text{ Mbps}.$$

Change:

$$-5 \text{ Mbps}.$$

A → B

Stage 1:

$$4 \text{ Mbps}$$

Stage 2:

$$16 \text{ Mbps}.$$

Change:

$$+12 \text{ Mbps}.$$

B → A

Stage 1:

$$3 \text{ Mbps}$$

Stage 2:

$$10 \text{ Mbps}.$$

Change:

$$+7 \text{ Mbps}.$$

This demonstrates that Stage 2 reorganizes internal routing while preserving the same admitted traffic.

The optimizer shifts traffic between the A-C and A-B/B-A corridors in order to create lower-latency route combinations for sensitive traffic while maintaining the one-path restriction.

38.6 Visualize Final Single-Path Routing

The final routing is now placed back onto the physical network.

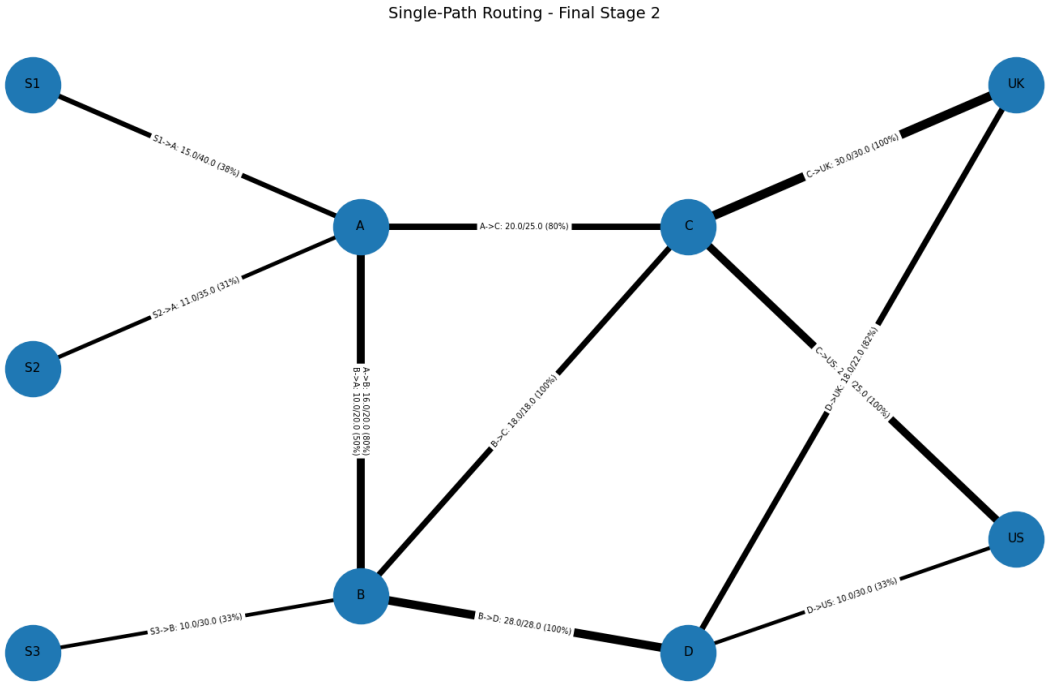
The same visualization function used throughout the notebook is retained so that Stage 1, Stage 2, split, and single-path results can be visually compared.

The diagram shows:

- optimized directional traffic;
- capacity;
- utilization;
- the spatial structure of the final one-path solution.

In [373…

```
plot_network_flow(
    G_synthetic,
    single_stage2_link_df,
    pos,
    title=(
        "Single-Path Routing – "
        "Final Stage 2"
    )
)
```



38.6.1 Interpretation of the Final Single-Path Network Diagram

The final network diagram shows that both major downstream corridors remain essential.

C Corridor

The following links are fully utilized:

$$B \to C = 18/18$$

$$C \to UK = 30/30$$

$$C \to US = 25/25.$$

D Corridor

The alternative route through D remains equally important.

$$B \to D = 28/28$$

is fully saturated.

Downstream:

$$D \to UK = 18/22$$

and:

$$D \to US = 10/30.$$

Internal A-B-C Rebalancing

A particularly important Stage 2 feature is increased use of:

$$A \to B$$

and:

$$B \to A.$$

At the same time, A → C falls to 80% utilization rather than remaining saturated.

This suggests that Stage 2 uses the A-B connection as an internal routing mechanism to create more favorable latency-sensitive paths while preserving all Stage 1 admissions.

The final network therefore remains throughput-feasible but has a different internal traffic geometry from Stage 1.

38.7 Validate Final Link Capacity Feasibility

In [374…

```
single_stage2_link_df[
    "capacity_violation_mbps"
] = (
    single_stage2_link_df[
        "flow_mbps"
    ]
    - single_stage2_link_df[
        "capacity_mbps"
    ]
).clip(
    lower=0
)


single_stage2_max_violation = (
    single_stage2_link_df[
```

```
        "capacity_violation_mbps"
    ].max()
)


assert (
    single_stage2_max_violation
    <= 1e-6
)


print(
    "Maximum Final Single-Path capacity violation:",
    single_stage2_max_violation,
    "Mbps"
)

print(
    "All final Single-Path capacity constraints satisfied."
)
```

```
Maximum Final Single-Path capacity violation: 0.0 Mbps
All final Single-Path capacity constraints satisfied.
```

38.8 Final Single-Path Sensitive-Traffic Latency

The final UK-L and US-L latency metrics are calculated using the same methodology as the splittable solution.

Two metrics are reported:

Traffic-Weighted Average Latency

$$\bar{L}_k = \frac{\sum_p f_{k,p} L_p}{\sum_p f_{k,p}}.$$

Worst Used-Path Latency

$$L_k^{\max} = \max_{p : f_{k,p} > 0} L_p.$$

Using the same metrics across both models ensures a consistent final comparison.

In [375…

```
single_latency_results = (
    pd.DataFrame([
        calculate_stream_latency_metrics(
            candidate_paths_df,
```

```
            single_stage2_flow,
            "UK-L"
        ),
        calculate_stream_latency_metrics(
            candidate_paths_df,
            single_stage2_flow,
            "US-L"
        )
    ])
)


single_latency_results
```

Out [375…

	stream	delivered_mbps	latency_exposure_mbps_ms	average_latency_ms	v
0	UK-L	12.0	254.0	21.166667	
1	US-L	18.0	446.0	24.777778	

38.8.1 Interpretation of Final Single-Path Sensitive Latency

The final one-path routing delivers:

UK-L

$$12 \text{ Mbps}$$

with traffic-weighted average latency of approximately:

$$21.17 \text{ ms}$$

and worst used-path latency of:

$$29 \text{ ms}.$$

US-L

$$18 \text{ Mbps}$$

is delivered with average latency of approximately:

$$24.78 \text{ ms}$$

and worst used-path latency of:

$$32 \text{ ms}.$$

These values represent the lowest sensitive-latency routing found while

preserving:

- the Single-Path Stage 1 admission vector;
- the one-path restriction;
- all link capacities.

38.9 Compare Single Stage 1 vs Stage 2 Latency

In [376…

```
# ------------------------------------------------------------
# Calculate sensitive latency under Single-Path Stage 1
# ------------------------------------------------------------

single_stage1_latency_results = (
    pd.DataFrame([
        calculate_stream_latency_metrics(
            candidate_paths_df,
            single_flow_vars,
            "UK-L"
        ),
        calculate_stream_latency_metrics(
            candidate_paths_df,
            single_flow_vars,
            "US-L"
        )
    ])
)


# Prepare Stage 1 metrics.

single_stage1_latency_compare = (
    single_stage1_latency_results[
        [
            "stream",
            "delivered_mbps",
            "latency_exposure_mbps_ms",
            "average_latency_ms",
            "worst_case_latency_ms"
        ]
    ]
    .rename(
        columns={
            "delivered_mbps":
                "stage1_delivered_mbps",
            "latency_exposure_mbps_ms":
                "stage1_latency_exposure",
            "average_latency_ms":
                "stage1_average_latency_ms",
            "worst_case_latency_ms":
                "stage1_worst_latency_ms"
```

```
        }
    )
)


# Prepare Stage 2 metrics.

single_stage2_latency_compare = (
    single_latency_results[
        [
            "stream",
            "delivered_mbps",
            "latency_exposure_mbps_ms",
            "average_latency_ms",
            "worst_case_latency_ms"
        ]
    ]
    .rename(
        columns={
            "delivered_mbps":
                "stage2_delivered_mbps",
            "latency_exposure_mbps_ms":
                "stage2_latency_exposure",
            "average_latency_ms":
                "stage2_average_latency_ms",
            "worst_case_latency_ms":
                "stage2_worst_latency_ms"
        }
    )
)


single_latency_stage_comparison_df = (
    single_stage1_latency_compare
    .merge(
        single_stage2_latency_compare,
        on="stream",
        how="inner"
    )
)


# Calculate improvements.

single_latency_stage_comparison_df[
    "average_latency_reduction_ms"
] = (
    single_latency_stage_comparison_df[
        "stage1_average_latency_ms"
    ]
    - single_latency_stage_comparison_df[
        "stage2_average_latency_ms"
```

```
    ]
)


single_latency_stage_comparison_df[
    "average_latency_reduction_pct"
] = (
    100
    * single_latency_stage_comparison_df[
        "average_latency_reduction_ms"
    ]
    / single_latency_stage_comparison_df[
        "stage1_average_latency_ms"
    ]
)


single_latency_stage_comparison_df[
    "worst_latency_reduction_ms"
] = (
    single_latency_stage_comparison_df[
        "stage1_worst_latency_ms"
    ]
    - single_latency_stage_comparison_df[
        "stage2_worst_latency_ms"
    ]
)


single_latency_stage_comparison_df
```

Out[376...

	stream	stage1_delivered_mbps	stage1_latency_exposure	stage1_average_la
0	UK-L	12.0	541.0	4
1	US-L	18.0	446.0	

38.9.1 Interpretation of Single-Path Stage 2 Latency Improvement

The Stage 1-to-Stage 2 comparison determines the actual value created by the second optimization stage.

Because delivered quantities are preserved exactly, any reduction in average latency is caused solely by route selection.

The comparison should be evaluated separately for UK-L and US-L because the magnitude of improvement may differ substantially.

The percentage reduction in average latency provides the clearest normalized measure of Stage 2 benefit.

Worst-case latency should also be reviewed because a favorable average can conceal a slower route used by part of the sensitive traffic.

38.10 Total Single-Path Sensitive Latency Exposure Improvement

In [377…

```
single_stage1_total_latency_exposure = (
    single_latency_stage_comparison_df[
        "stage1_latency_exposure"
    ].sum()
)


single_stage2_total_latency_exposure = (
    single_latency_stage_comparison_df[
        "stage2_latency_exposure"
    ].sum()
)


single_latency_exposure_reduction = (
    single_stage1_total_latency_exposure
    - single_stage2_total_latency_exposure
)


single_latency_exposure_reduction_pct = (
    100
    * single_latency_exposure_reduction
    / single_stage1_total_latency_exposure
)


print(
    "Single Stage 1 sensitive latency exposure:",
    single_stage1_total_latency_exposure,
    "Mbps*ms"
)

print(
    "Single Stage 2 sensitive latency exposure:",
    single_stage2_total_latency_exposure,
    "Mbps*ms"
)

print(
    "Latency exposure reduction:",
```

```
    single_latency_exposure_reduction,
    "Mbps*ms"
)

print(
    "Latency exposure reduction percentage:",
    single_latency_exposure_reduction_pct,
    "%"
)
```

```
Single Stage 1 sensitive latency exposure: 987.0 Mbps*ms
Single Stage 2 sensitive latency exposure: 700.0 Mbps*ms
Latency exposure reduction: 287.0 Mbps*ms
Latency exposure reduction percentage: 29.078014184397162 %
```

38.11 Visualize Single Stage 1 vs Stage 2 Average Latency

In [378…

```
single_latency_stage_comparison_df.set_index(
    "stream"
)[
    [
        "stage1_average_latency_ms",
        "stage2_average_latency_ms"
    ]
].plot(
    kind="bar",
    figsize=(8, 5)
)


plt.ylabel(
    "Traffic-Weighted Average Latency (ms)"
)

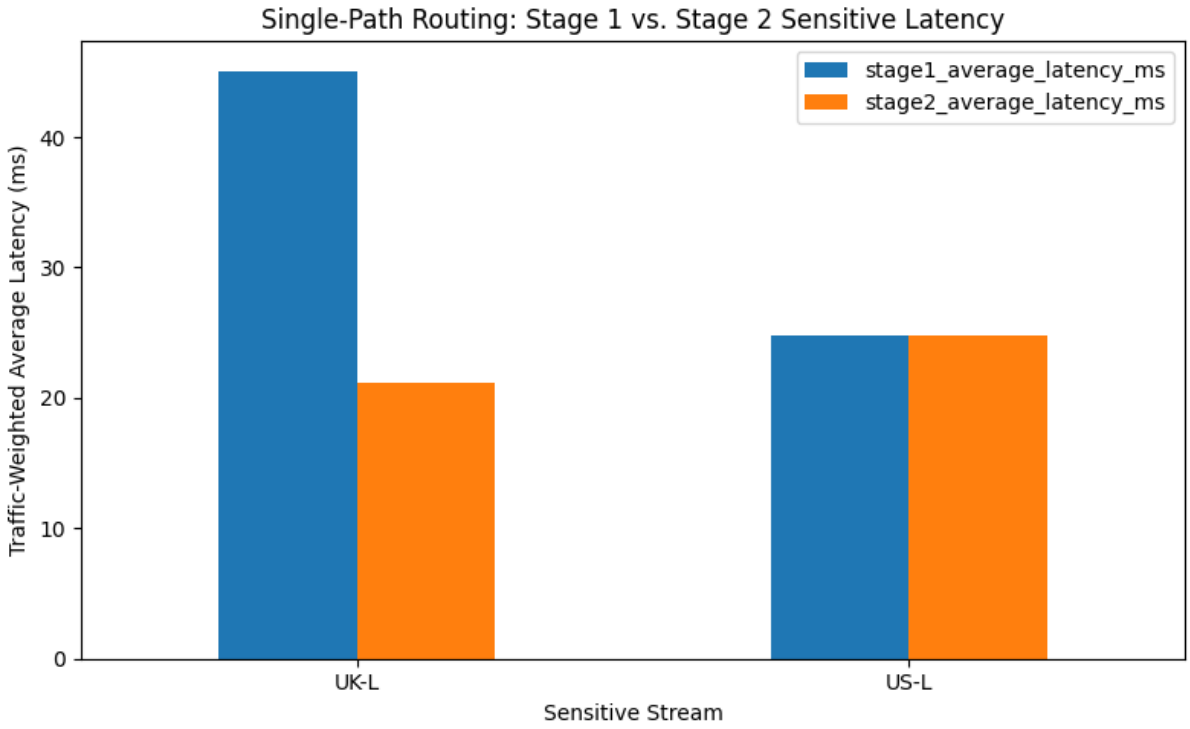
plt.xlabel(
    "Sensitive Stream"
)

plt.title(
    "Single-Path Routing: "
    "Stage 1 vs. Stage 2 Sensitive Latency"
)

plt.xticks(
    rotation=0
)

plt.tight_layout()

plt.show()
```



38.12 Preliminary Final Latency Comparison: Split vs. Single Path

Both final routing formulations have now been solved.

Their latency metrics can therefore be placed side by side.

However, an important caveat applies:

> The Split Stage 1 and Single-Path Stage 1 models do not necessarily admit the same source-stream mix.

Therefore, direct latency comparison is partly influenced by differences in which traffic was admitted.

This comparison is useful descriptively, but Section 39 will address this caveat explicitly.

In [379…

```
final_sensitive_latency_comparison_df = (
    split_latency_results[
        [
            "stream",
            "delivered_mbps",
            "average_latency_ms",
            "worst_case_latency_ms"
        ]
    ]
```

```
    .rename(
        columns={
            "delivered_mbps":
                "split_delivered_mbps",
            "average_latency_ms":
                "split_average_latency_ms",
            "worst_case_latency_ms":
                "split_worst_latency_ms"
        }
    )
    .merge(
        single_latency_results[
            [
                "stream",
                "delivered_mbps",
                "average_latency_ms",
                "worst_case_latency_ms"
            ]
        ]
        .rename(
            columns={
                "delivered_mbps":
                    "single_delivered_mbps",
                "average_latency_ms":
                    "single_average_latency_ms",
                "worst_case_latency_ms":
                    "single_worst_latency_ms"
            }
        ),
        on="stream",
        how="inner"
    )
)


final_sensitive_latency_comparison_df
```

Out[379…

	stream	split_delivered_mbps	split_average_latency_ms	split_worst_latency_
0	UK-L	16.0	18.250000	
1	US-L	18.0	19.555556	

38.12.1 Interpretation of the Preliminary Split-vs.-Single Latency Comparison

At first glance, the final splittable solution produces lower average latency for both sensitive streams.

UK-L

Splittable final:

$$18.25 \text{ ms}$$

Single-path final:

$$21.17 \text{ ms}.$$

US-L

Splittable final:

$$19.56 \text{ ms}$$

Single-path final:

$$24.78 \text{ ms}.$$

However, the UK-L comparison is not fully controlled because the two formulations deliver different amounts of UK-L traffic:

$$16 \text{ Mbps}$$

under the splittable model versus:

$$12 \text{ Mbps}$$

under the single-path model.

Therefore, part of the latency difference may result from the different admitted commodity mix rather than splitting flexibility alone.

US-L is more directly comparable at the aggregate level because both deliver:

$$18 \text{ Mbps}.$$

Even there, the underlying source composition should be checked before attributing the full latency gap to routing flexibility.

This caveat will be handled explicitly in Section 39.

38.13 Final Single-Path Executive KPIs

In [380…

```
single_final_executive_df = pd.DataFrame([
    {
        "metric":
            "Final Total Throughput",
        "value":
```

```
            single_stage2_total_throughput,
        "unit":
            "Mbps"
    },
    {
        "metric":
            "UK-L Delivered",
        "value":
            float(
                single_latency_results.loc[
                    single_latency_results[
                        "stream"
                    ]
                    == "UK-L",
                    "delivered_mbps"
                ].iloc[0]
            ),
        "unit":
            "Mbps"
    },
    {
        "metric":
            "UK-L Average Latency",
        "value":
            float(
                single_latency_results.loc[
                    single_latency_results[
                        "stream"
                    ]
                    == "UK-L",
                    "average_latency_ms"
                ].iloc[0]
            ),
        "unit":
            "ms"
    },
    {
        "metric":
            "US-L Delivered",
        "value":
            float(
                single_latency_results.loc[
                    single_latency_results[
                        "stream"
                    ]
                    == "US-L",
                    "delivered_mbps"
                ].iloc[0]
            ),
        "unit":
            "Mbps"
    },
```

```
    {
        "metric":
            "US-L Average Latency",
        "value":
            float(
                single_latency_results.loc[
                    single_latency_results[
                        "stream"
                    ]
                    == "US-L",
                    "average_latency_ms"
                ].iloc[0]
            ),
        "unit":
            "ms"
    },
    {
        "metric":
            "Saturated Directed Links",
        "value":
            int(
                single_stage2_link_df[
                    "is_saturated"
                ].sum()
            ),
        "unit":
            "links"
    }
])


single_final_executive_df
```

Out[380…

	metric	value	unit
0	Final Total Throughput	83.000000	Mbps
1	UK-L Delivered	12.000000	Mbps
2	UK-L Average Latency	21.166667	ms
3	US-L Delivered	18.000000	Mbps
4	US-L Average Latency	24.777778	ms
5	Saturated Directed Links	4.000000	links

38.14 Business and Operational Interpretation

The final single-path solution demonstrates that latency optimization is possible even under a strict one-route-per-commodity policy.

The network continues to deliver:

$$83 \text{ Mbps}.$$

The Stage 1 admission vector remains unchanged.

However, the model rearranges internal routing to improve sensitive-traffic latency.

Operational Simplicity

Each served commodity still uses one route.

This can simplify:

- implementation;
- observability;
- troubleshooting;
- path enforcement.

Reduced Routing Flexibility

The model cannot divide tolerant traffic across multiple routes in order to create small pockets of low-latency capacity.

This may restrict the optimizer's ability to improve sensitive traffic compared with the splittable formulation.

Final Network Behavior

The final solution uses:

- C-side exit capacity fully;
- B → D fully;
- substantial A → B and B → A capacity;
- both C and D as important downstream corridors.

The result therefore balances operational simplicity against routing flexibility.

38.15 Detailed Section Summary

Sections 37 and 38 complete the single-path optimization workflow.

---

1. Stage 2 Preserves the Stage 1 Admission Decision

The exact delivered quantity of every source-stream commodity is preserved.

Therefore:

$$z_{k,s}^{(2)} = z_{k,s}^{(1)}$$

for every commodity.

Total throughput remains:

$$83 \text{ Mbps}.$$

No latency improvement is obtained by sacrificing service volume.

---

2. The One-Path Restriction Remains Active

Every commodity continues to use at most one positive-flow route.

Stage 2 therefore improves latency only by selecting a different single route when beneficial.

The solution never uses commodity-level traffic splitting.

---

3. Final Link Utilization Changes

Four directed links are saturated in the final solution:

- B → C;
- B → D;
- C → UK;
- C → US.

Other major utilization levels include:

$$D \to UK \approx 81.82\%,$$

$$A \to B = 80\%,$$

$$A \to C = 80\%,$$

and:

$$B \to A = 50\%.$$

---

4. Internal Routing Changes Materially

Compared with Single-Path Stage 1:

$$A \to C$$

falls from:

$$25$$

to:

$$20 \text{ Mbps}.$$

$$C \to B$$

falls from:

$$5$$

to:

$$0.$$

Meanwhile:

$$A \to B$$

increases from:

$$4$$

to:

$$16 \text{ Mbps},$$

and:

$$B \to A$$

increases from:

$$3$$

to:

$$10 \text{ Mbps}.$$

This demonstrates that the latency objective reorganizes internal traffic while preserving admission and total throughput.

---

5. Final UK-L Performance

The final single-path model delivers:

$$12 \text{ Mbps}$$

of UK-L.

Traffic-weighted average latency is approximately:

$$21.17 \text{ ms}.$$

Worst used-path latency is:

$$29 \text{ ms}.$$

---

6. Final US-L Performance

The final single-path model delivers:

$$18 \text{ Mbps}$$

of US-L.

Average latency is approximately:

$$24.78 \text{ ms}.$$

Worst used-path latency is:

$$32 \text{ ms}.$$

---

7. Final Split and Single Solutions Are Not Directly Equivalent

The final splittable model delivers:

$$16 \text{ Mbps}$$

of UK-L, while the single-path model delivers:

$$12 \text{ Mbps}.$$

Therefore, the lower UK-L latency in the split solution cannot be attributed entirely to routing flexibility without controlling for the different admission vector.

The same caution should be applied whenever different models admit different commodity mixes.

---

8. Preliminary Latency Comparison Favors Splittable Routing

The final splittable model reports:

UK-L

$$18.25 \text{ ms}$$

average latency versus:

$$21.17 \text{ ms}$$

for single-path routing.

US-L

$$19.56 \text{ ms}$$

versus:

$$24.78 \text{ ms}.$$

This suggests that additional routing flexibility can improve sensitive-traffic performance.

However, Section 39 will distinguish routing effects from admission-mix effects.

---

9. Technical Conclusion

The Single-Path Stage 2 MILP behaves as intended.

It:

- preserves 83 Mbps of throughput;
- preserves all commodity-level Stage 1 delivery quantities;
- preserves the one-path restriction;
- respects every directional capacity;
- changes selected routes to reduce sensitive latency.

The model therefore provides a valid latency-optimized single-path benchmark.

---

10. Business Conclusion

Single-path routing can maintain maximum baseline throughput while providing latency-aware route selection.

Its primary advantage is operational simplicity.

Its principal limitation is reduced routing flexibility.

The final comparison suggests that this restriction may limit latency performance even when it does not limit total throughput.

---

11. Connection to Section 39

At this point, both complete optimization pipelines are finished:

Splittable

Stage 1: maximize throughput.

Stage 2: preserve admission and minimize sensitive latency.

Single Path

Stage 1: maximize throughput under one-route restriction.

Stage 2: preserve admission and minimize sensitive latency under the same restriction.

Section 39 can therefore perform the main final model comparison across:

- total throughput;
- lost demand;
- source-stream admission;
- stream-level service;
- sensitive latency;
- worst-case latency;
- number of active routes;
- network utilization;
- saturation;
- implementation complexity.

That section should answer the central decision question:

> **What practical value does splittable routing provide compared with single-path routing, and under what conditions is that extra flexibility worth the added operational complexity?**

39. Final Comparative Analysis — Splittable vs. Single-Path Routing

39.1 Purpose, Context, and Connection to Prior Sections

Sections 32–34 completed the full **splittable-routing optimization pipeline**.

That analysis proceeded in two stages.

Stage 1 — Maximum Throughput

The first stage determined the maximum amount of offered traffic that could be delivered while:

- respecting every directed-link capacity;
- sending each traffic stream to the correct destination;
- allowing partial delivery;
- allowing a source-stream commodity to use multiple candidate paths.

Stage 2 — Sensitive-Latency Optimization

The second stage preserved the exact source-stream admission quantities selected in Stage 1 and then re-routed those admitted quantities to minimize latency exposure for the latency-sensitive:

- UK-L;
- US-L;

traffic classes.

Sections 35–38 repeated the same two-stage analysis under the **single-path routing restriction**.

Under the single-path formulation:

- every source-stream commodity may use at most one route;
- partial delivery remains allowed;
- Stage 1 still maximizes total throughput;
- Stage 2 still preserves the exact Stage 1 admission vector before minimizing sensitive latency.

Those earlier sections answered:

> **What is the best achievable network operating point under each routing policy individually?**

Section 39 changes the analytical question.

Rather than evaluating the formulations independently, this section asks:

> **What practical value does splittable routing provide relative to single-path routing?**

The comparison includes:

- maximum delivered throughput;
- undelivered demand;
- network delivery percentage;
- commodity-level admission;
- stream-level admission;
- source-level admission;
- sensitive-traffic average latency;
- worst used-path latency;
- traffic-weighted latency exposure;
- directed-link utilization;
- residual capacity;
- saturated links;
- number of active routes;
- mathematical optimization complexity;
- operational complexity.

A particularly important question is whether the theoretical throughput relationship between the two formulations is:

$$T^*_{single} < T^*_{split},$$

which would mean that the one-path restriction reduces achievable throughput,

or:

$$T^*_{single} = T^*_{split},$$

which would mean that the one-path restriction is not throughput-binding under the baseline network conditions.

Important Analytical Caveat

Even if the two Stage 1 models achieve the same total throughput, they do not necessarily deliver the same source-stream commodities.

Therefore, an observed difference in final latency can potentially contain two separate effects:

1. **Admission-Mix Effect**
    - the Stage 1 models may admit different source-stream traffic;
2. **Routing-Flexibility Effect**
    - the two formulations differ in how flexibly they can route admitted traffic.

These effects should not be conflated.

39.1.2 Interpretation

Section 39 is not introducing a new optimization model.

Instead, it is a synthesis and comparison section.

The previous sections produced optimized solutions for:

- Splittable Stage 1;
- Splittable Stage 2;
- Single-Path Stage 1;
- Single-Path Stage 2.

The purpose here is to determine whether the additional feasible routing flexibility of the split formulation produces measurable value.

That value may appear in different forms.

For example, splitting may provide:

- higher throughput;
- lower latency;
- better use of fragmented capacity;
- better stressed-network performance.

Conversely, the two models may tie on one performance measure while differing substantially on another.

39.1.3 Business Impact

A routing policy should not be selected only because it is mathematically more flexible or operationally simpler.

The practical question is:

> **Which routing strategy provides the best combination of service, latency, simplicity, resilience, and scalability for the operating environment?**

A more flexible routing strategy may not be worth adopting if it creates substantial operational complexity without producing measurable value.

Conversely, additional complexity may be justified if it produces:

- lower latency;
- better use of constrained infrastructure;
- greater resilience;
- improved service under stress.

39.1.4 Connection to the Next Subsection

Before comparing numerical outputs, the next subsection establishes the mathematical relationship between the feasible regions of:

- splittable routing;
- single-path routing.

That relationship provides both:

1. a theoretical benchmark;
2. a model-validation check.

The subsequent throughput comparison will determine whether the theoretical inequality is strict or holds with equality in the baseline network.

39.2 Mathematical Relationship Between Splittable and Single-Path Routing

Before comparing the numerical results, it is useful to establish the mathematical relationship between the two feasible regions.

Every feasible single-path routing solution is also feasible under the splittable formulation.

A commodity using exactly one route is simply a special case of a model that permits that commodity to use several routes.

Therefore:

$$\mathcal{F}_{single} \subseteq \mathcal{F}_{split},$$

where:

- $\mathcal{F}_{single}$ = feasible set of single-path solutions;
- $\mathcal{F}_{split}$ = feasible set of splittable-routing solutions.

Because the splittable model has all of the choices available to the single-path model, plus potentially additional routing choices, the maximum throughput values must satisfy:

$$\boxed{T^*_{single} \leq T^*_{split}}$$

where:

- T^*_{single} = maximum feasible single-path throughput;
- T^*_{split} = maximum feasible splittable throughput.

Why This Relationship Is Important

This equation is both:

1. a mathematical property of the formulations;
2. a useful model-validation check.

If the notebook ever produced:

$$T^*_{single} > T^*_{split},$$

that result would indicate that the two models are not actually being solved under equivalent assumptions.

Possible causes could include:

- different candidate-path sets;
- different commodity demand values;
- different link capacities;
- missing capacity constraints;
- missing demand constraints;
- implementation error.

Therefore, the expected theoretical relationship is:

$$T^*_{single} \leq T^*_{split}.$$

The baseline results will show whether the inequality is strict or whether the two models tie.

39.2.1 Interpretation

The key idea is that the single-path formulation is more restrictive than the splittable formulation.

The splittable model can reproduce any feasible single-path solution simply by assigning all of a commodity's delivered traffic to one route.

However, the splittable model may also use routing patterns that are unavailable to the single-path model because it can distribute one commodity across multiple paths.

Therefore:

$$\mathcal{F}_{single} \subseteq \mathcal{F}_{split}.$$

This means that the single-path formulation cannot theoretically achieve a higher Stage 1 throughput optimum than the splittable formulation when both use identical network assumptions.

39.2.2 Possible Numerical Outcomes

The baseline comparison can produce two valid outcomes.

Case 1 — Strict Inequality

If:

$$T^*_{single} < T^*_{split},$$

then the one-path restriction reduces achievable throughput.

In that case, the additional routing flexibility of the splittable formulation has direct throughput value.

Case 2 — Equality

If:

$$T^*_{single} = T^*_{split},$$

then the one-path restriction does not reduce maximum throughput for the baseline network state.

However, equality in throughput does not imply equality in:

- admission;
- routes;
- latency;
- link utilization;
- operational complexity.

The numerical results in the next subsection determine which case applies.

39.2.2 Why This Matters for the Results

This theoretical relationship gives an important framework for interpreting the numerical results that follow.

There are two possible valid outcomes.

Case 1 — Strict Inequality

If:

$$T^*_{single} < T^*_{split},$$

then the one-path restriction is reducing achievable throughput.

In that case, the additional routing flexibility of the splittable model has direct throughput value.

Case 2 — Equality

If:

$$T^*_{single} = T^*_{split},$$

then the one-path restriction is not reducing maximum throughput for that particular network state.

That does not mean the two models are equivalent.

They may still differ in:

- which commodities are admitted;
- which sources receive service;
- latency;
- link utilization;
- route complexity;
- resilience under stressed conditions.

The baseline results will show which of these two cases applies.

39.2.3 Business and Model-Validation Impact

From a validation perspective, a result such as:

$$T^*_{single} > T^*_{split}$$

would contradict the mathematical structure of the formulations.

That result should trigger review of:

- candidate-path construction;
- demand values;
- capacity values;
- constraints;
- solver implementation.

From a business perspective, if:

$$T^*_{single} = T^*_{split},$$

then splittable routing cannot be justified on baseline throughput alone.

Its value must instead come from other dimensions such as:

- lower latency;
- improved resilience;
- better use of fragmented capacity;
- better performance under congestion or failure conditions.

39.C Headline Throughput Comparison

Analytical Question

The first comparison asks:

> **Does allowing a source-stream commodity to split across several routes increase maximum baseline throughput?**

The network contains:

$$126 \text{ Mbps}$$

of offered demand.

For each routing formulation, I compare:

1. offered demand;
2. maximum delivered throughput;
3. undelivered demand;
4. percentage of offered demand successfully delivered.

The overall network delivery percentage is:

$$R_{network} = 100 \frac{T}{D_{total}},$$

where:

- T = optimized delivered throughput;
- D_{total} = total offered network demand.

This is the highest-level comparison between the two routing policies.

In [386…

```
# ------------------------------------------------------------
# 39.3 Headline throughput comparison
# ------------------------------------------------------------

total_demand = float(
    demand_df[
        "demand_mbps"
    ].sum()
)


final_throughput_comparison_df = pd.DataFrame([
    {
        "model":
            "Splittable",
        "offered_demand_mbps":
            total_demand,
        "maximum_throughput_mbps":
            float(split_stage1_throughput),
```

```
        "lost_demand_mbps":
            float(
                total_demand
                - split_stage1_throughput
            )
    },
    {
        "model":
            "Single Path",
        "offered_demand_mbps":
            total_demand,
        "maximum_throughput_mbps":
            float(single_stage1_throughput),
        "lost_demand_mbps":
            float(
                total_demand
                - single_stage1_throughput
            )
    }
])


final_throughput_comparison_df[
    "delivery_percentage"
] = (
    100
    * final_throughput_comparison_df[
        "maximum_throughput_mbps"
    ]
    / final_throughput_comparison_df[
        "offered_demand_mbps"
    ]
)


final_throughput_comparison_df
```

Out[386…

	model	offered_demand_mbps	maximum_throughput_mbps	lost_demand_
0	Splittable	126.0	83.0	
1	Single Path	126.0	83.0	

39.3.1 Results and Detailed Explanation

The output should show approximately:

Model	Offered Demand	Maximum Throughput	Lost Demand	Delivery Rate

Splittable	126 Mbps	83 Mbps	43 Mbps	65.87%
Single Path	126 Mbps	83 Mbps	43 Mbps	65.87%

The comparison produces the same maximum throughput for both routing formulations:

$$T^*_{split} = T^*_{single} = 83 \text{ Mbps}.$$

Because total offered demand is:

$$126 \text{ Mbps},$$

both formulations leave:

$$126 - 83 = 43 \text{ Mbps}$$

undelivered.

The percentage of offered demand successfully served is:

$$\frac{83}{126} \times 100 \approx 65.87\%.$$

This places the baseline result in **Case 2** from Section 39.2:

$$\boxed{T^*_{single} = T^*_{split}}$$

Therefore, the one-path restriction does not reduce the maximum throughput achievable under the baseline network configuration.

The result does not mean that the two routing formulations are operationally equivalent.

It establishes only that their Stage 1 objective values are equal.

The subsequent analysis therefore asks whether the two 83-Mbps solutions differ in:

- which commodities are admitted;
- which sources receive service;
- which traffic classes are delivered;
- sensitive latency;
- internal link utilization;
- route complexity.

Therefore:

$$\boxed{\text{Equal Objective Value} \neq \text{Identical Optimization Solution}}$$

39.3.2 Interpretation

The baseline falls into the equality case predicted in Section 39.2:

$$T^*_{single} = T^*_{split}.$$

Therefore:

> **The single-path restriction is not throughput-binding in the baseline scenario.**

The additional feasible routing choices available to the split formulation do not create additional delivered traffic under the current demand and capacity configuration.

3.9.3.3 Why This Result Occurs

The splittable formulation has more routing freedom, but greater routing freedom does not automatically produce a larger objective value.

The earlier Split Stage 1 solution reached:

$$83 \text{ Mbps}$$

without requiring any served commodity to use more than one positive-flow route.

Therefore, at least one throughput-optimal solution already exists inside the smaller single-path feasible region.

The baseline throughput ceiling is driven primarily by:

- shared internal-link capacity;
- destination-facing capacity;
- network topology;
- total offered demand.

It is not driven by the one-path restriction.

39.3.4 Business Impact

If maximum baseline throughput were the only decision criterion, this result would

favor the simpler single-path operating model.

Single-path routing delivers the same:

$$83 \text{ Mbps}$$

while potentially providing:

- easier route monitoring;
- simpler troubleshooting;
- more deterministic traffic behavior;
- simpler operational ownership.

However, Section 39.2 established only a throughput relationship.

The larger split feasible region may still provide value in other dimensions.

Those dimensions include:

- latency;
- fragmented capacity use;
- resilience;
- stressed-network performance.

Therefore, the routing decision should not be based on Stage 1 throughput alone.

In [387…

```
# ------------------------------------------------------------
# 39.4 Quantify baseline throughput advantage of splitting
# ------------------------------------------------------------

split_throughput_advantage_mbps = (
    split_stage1_throughput
    - single_stage1_throughput
)


split_throughput_advantage_pct = (
    100
    * split_throughput_advantage_mbps
    / single_stage1_throughput
    if single_stage1_throughput > 0
    else 0
)


print(
    "Splittable throughput:",
    split_stage1_throughput,
    "Mbps"
```

```
)

print(
    "Single-path throughput:",
    single_stage1_throughput,
    "Mbps"
)

print(
    "Splittable throughput advantage:",
    split_throughput_advantage_mbps,
    "Mbps"
)

print(
    "Relative throughput advantage:",
    split_throughput_advantage_pct,
    "%"
)
```

```
Splittable throughput: 83.0 Mbps
Single-path throughput: 83.0 Mbps
Splittable throughput advantage: 0.0 Mbps
Relative throughput advantage: 0.0 %
```

39.4.1 — Interpretation

The theoretical relationship:

$$T^*_{single} \leq T^*_{split}$$

holds with equality.

Therefore:

$$T^*_{single} = T^*_{split} = 83 \text{ Mbps}.$$

The additional routing choices available to the split formulation have:

$$0 \text{ Mbps}$$

of baseline throughput value.

39.4.2 Why This Happens

The one-path restriction is not active at the maximum-throughput operating point.

The network reaches its baseline physical throughput ceiling before path splitting

becomes necessary.

This is consistent with the Split Stage 1 solution, where no commodity actually needed multiple positive-flow paths.

39.4.2 Business Impact

The equality result prevents overengineering.

There is no evidence that multipath routing should be introduced solely to increase baseline throughput.

Any business case for split routing must instead come from:

- lower latency;
- better resilience;
- better stress performance;
- better use of fragmented capacity.

39.5 Throughput Visualization

In [389…

```
plt.figure(
    figsize=(8, 5)
)

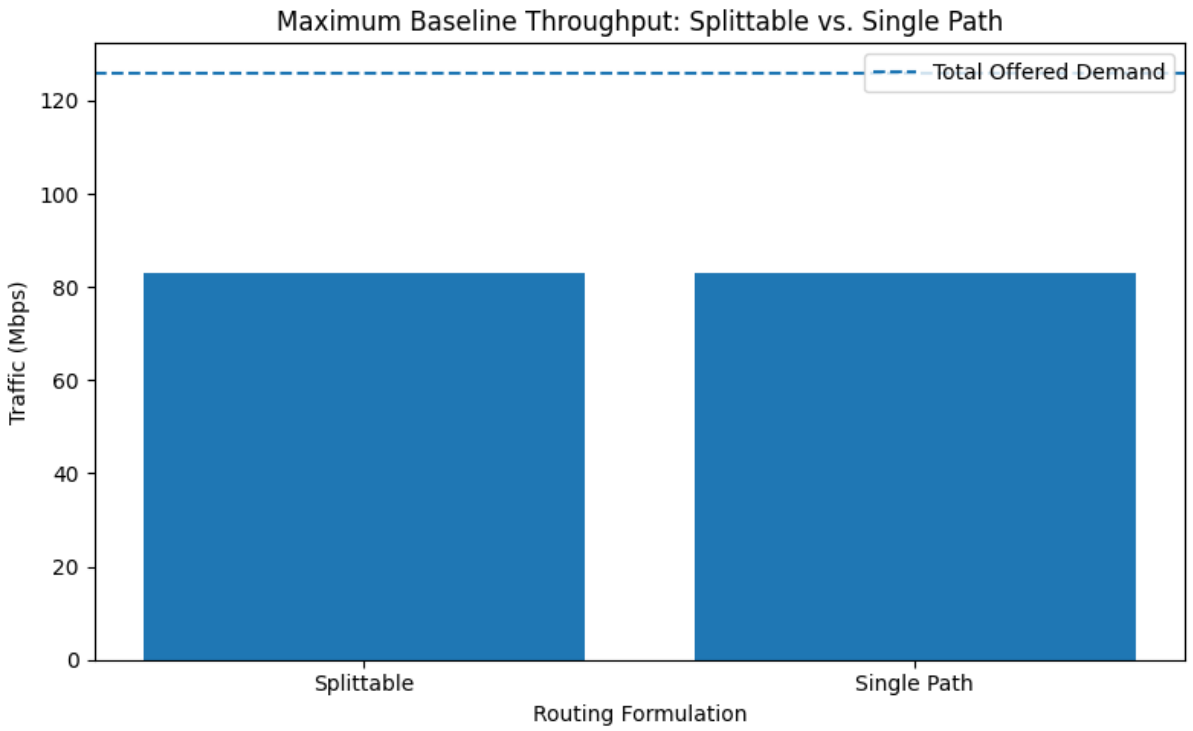
plt.bar(
    final_throughput_comparison_df[
        "model"
    ],
    final_throughput_comparison_df[
        "maximum_throughput_mbps"
    ]
)

plt.axhline(
    total_demand,
    linestyle="--",
    label="Total Offered Demand"
)

plt.ylabel("Traffic (Mbps)")
plt.xlabel("Routing Formulation")

plt.title(
    "Maximum Baseline Throughput: "
    "Splittable vs. Single Path"
)
```

```
plt.legend()
plt.tight_layout()
plt.show()
```



39.5.1 Visualization Interpretation

The two optimized bars should both appear at:

$$83 \text{ Mbps}.$$

The dashed offered-demand line should appear at:

$$126 \text{ Mbps}.$$

The figure visually confirms the equality case from Section 39.2:

$$T^*_{single} = T^*_{split}.$$

It also shows that both models remain substantially below total offered demand.

The:

$$43 \text{ Mbps}$$

gap therefore reflects shared physical network constraints rather than the one-path restriction.

39.5.2 Business Impact

The visualization separates two different issues:

> **Is routing flexibility limiting throughput?**

and:

> **Is physical network capacity limiting throughput?**

For the baseline, routing flexibility is not the limiting factor.

The remaining service gap is primarily structural.

That shifts attention toward:

- link-capacity behavior;
- destination bottlenecks;
- infrastructure value.

Those questions are explored in Sections 40–42.

39.6 Commodity-Level Admission Comparison

The equality:

$$T^*_{single} = T^*_{split}$$

does not imply that the two models serve the same commodities.

For commodity (k, s):

$$z_{k,s} = \sum_{p \in P_{k,s}} f_{k,s,p}.$$

This subsection asks:

> **Does the same 83 Mbps consist of the same source-stream traffic under both routing formulations?**

This distinction matters because aggregate throughput can hide differences in:

- customers;
- sources;
- regions;
- traffic classes.

In [390…
```
split_vs_single_commodity_df = (
```

```
    split_stage1_delivery_df[
        [
            "commodity_id",
            "source",
            "stream",
            "destination",
            "demand_mbps",
            "delivered_mbps",
            "lost_mbps",
            "delivery_percentage"
        ]
    ]
    .rename(
        columns={
            "delivered_mbps":
                "split_delivered_mbps",
            "lost_mbps":
                "split_lost_mbps",
            "delivery_percentage":
                "split_delivery_pct"
        }
    )
    .merge(
        single_stage1_delivery_df[
            [
                "commodity_id",
                "delivered_mbps",
                "lost_mbps",
                "delivery_percentage"
            ]
        ]
        .rename(
            columns={
                "delivered_mbps":
                    "single_delivered_mbps",
                "lost_mbps":
                    "single_lost_mbps",
                "delivery_percentage":
                    "single_delivery_pct"
            }
        ),
        on="commodity_id",
        how="inner"
    )
)


split_vs_single_commodity_df[
    "single_minus_split_mbps"
] = (
    split_vs_single_commodity_df[
        "single_delivered_mbps"
```

```
    ]
    -
    split_vs_single_commodity_df[
        "split_delivered_mbps"
    ]
)


split_vs_single_commodity_df[
    "admission_differs"
] = (
    split_vs_single_commodity_df[
        "single_minus_split_mbps"
    ].abs()
    > 1e-8
)


split_vs_single_commodity_df
```

Out [390…

	commodity_id	source	stream	destination	demand_mbps	split_delivered_
0	S1_UK-L	S1	UK-L	UK	6.0	
1	S1_UK-T	S1	UK-T	UK	7.0	
2	S1_US-L	S1	US-L	US	5.0	
3	S1_US-T	S1	US-T	US	6.0	
4	S2_UK-L	S2	UK-L	UK	5.0	
5	S2_UK-T	S2	UK-T	UK	6.0	
6	S2_US-L	S2	US-L	US	5.0	
7	S2_US-T	S2	US-T	US	5.0	
8	S3_UK-L	S3	UK-L	UK	5.0	
9	S3_UK-T	S3	UK-T	UK	7.0	
10	S3_US-L	S3	US-L	US	6.0	
11	S3_US-T	S3	US-T	US	6.0	
12	B_UK-L	B	UK-L	UK	6.0	
13	B_UK-T	B	UK-T	UK	14.0	
14	B_US-L	B	US-L	US	8.0	
15	B_US-T	B	US-T	US	9.0	
16	C_UK-L	C	UK-L	UK	4.0	
17	C_UK-T	C	UK-T	UK	5.0	
18	C_US-L	C	US-L	US	5.0	
19	C_US-T	C	US-T	US	6.0	

39.6.1 Interpretation

A positive value in:

`single_minus_split_mbps`

means the single-path model serves more of that commodity.

A negative value means the split model serves more.

A zero value means the commodity receives identical Stage 1 service.

The purpose of this comparison is to determine whether equality in the objective value also implies equality in the admission vector.

39.6.2 Why This Happens

Stage 1 solves:

$$\max \sum_{k,s} z_{k,s}.$$

The objective values every delivered Mbps equally.

It does not specify which source-stream commodities must make up the optimum.

Therefore, several admission vectors may produce:

$$83 \text{ Mbps}.$$

This is **throughput-optimal degeneracy**.

The two routing formulations can therefore reach the same objective value using different commodity combinations.

39.6.3 Business Impact

This demonstrates why:

$$\text{Equal Objective Value} \neq \text{Equal Service Outcome}.$$

If commodities correspond to customers, regions, or applications, the same top-line throughput number can hide materially different service experiences.

A production system should therefore report segmented service in addition to total throughput.

39.7 Validate Commodity Admission Reconciliation

In [391…

```
net_admission_difference = (
    split_vs_single_commodity_df[
        "single_minus_split_mbps"
    ].sum()
)
```

```
print(
    "Net commodity admission difference:",
    net_admission_difference,
    "Mbps"
)


assert abs(
    net_admission_difference
) < 1e-6
```

```
Net commodity admission difference: 0.0 Mbps
```

39.7.1 Expected Result

The net admission difference should be approximately:

$$0 \text{ Mbps}.$$

Because:

$$T^*_{single} = T^*_{split} = 83,$$

the following must hold:

$$\sum_{k,s} z^{single}_{k,s} = \sum_{k,s} z^{split}_{k,s}.$$

39.7.2 Interpretation

The zero net difference confirms that any commodity receiving additional service under one model is offset by less service elsewhere.

Therefore, the comparison represents redistribution of the same:

$$83 \text{ Mbps}$$

rather than a difference in total service.

39.7.3 Business Impact

This validation supports a clean business interpretation.

Any customer-, source-, or stream-level differences are genuine allocation differences rather than artifacts of one model having a larger total throughput.

39.8 Commodities With Different Admissions

In [393…

```
commodity_admission_differences_df = (
    split_vs_single_commodity_df[
        split_vs_single_commodity_df[
            "admission_differs"
        ]
    ]
    .copy()
)


print(
    "Number of commodities with different admissions:",
    len(
        commodity_admission_differences_df
    )
)


commodity_admission_differences_df[
    [
        "commodity_id",
        "demand_mbps",
        "split_delivered_mbps",
        "single_delivered_mbps",
        "single_minus_split_mbps"
    ]
]
```

```
Number of commodities with different admissions: 10
```

Out[393…

	commodity_id	demand_mbps	split_delivered_mbps	single_delivered_mbps
0	S1_UK-L	6.0	6.0	0.0
1	S1_UK-T	7.0	7.0	4.0
2	S1_US-L	5.0	0.0	5.0
3	S1_US-T	6.0	0.0	6.0
4	S2_UK-L	5.0	0.0	5.0
6	S2_US-L	5.0	5.0	0.0
9	S3_UK-T	7.0	2.0	7.0
11	S3_US-T	6.0	0.0	3.0
15	B_US-T	9.0	9.0	2.0
16	C_UK-L	4.0	4.0	1.0

39.8.1 Interpretation

This table isolates the source-stream commodities whose Stage 1 delivered quantities differ.

The central finding is:

$$\boxed{\text{Equal Throughput} \neq \text{Equal Admission}}$$

The equality result from Section 39.2 therefore applies only to the optimal objective value, not to the full solution vector.

39.8.2 Why This Happens

The single-path formulation changes the combinations of:

- commodities;
- routes;
- residual capacities;

that can coexist.

The split formulation has a larger feasible region, so it can reach the same 83-Mbps optimum using a different feasible admission vector.

39.8.4 Business Impact

If some commodities have greater business value than others, throughput maximization alone may not be sufficient.

A future production formulation may need:

- priorities;
- fairness constraints;
- minimum-service guarantees;
- SLAs.

39.9 Admission-Difference Heatmap

39.9 Commodity Admission Difference Heatmap

The heatmap organizes commodity-level differences by source and traffic stream.

Each cell represents:

$$z_{k,s}^{single} - z_{k,s}^{split}.$$

Therefore:

- positive = single path delivers more;
- negative = split routing delivers more;
- zero = identical admission.

In [395…

```
source_order_local = [
    "S1",
    "S2",
    "S3",
    "B",
    "C"
]

stream_order_local = [
    "UK-L",
    "UK-T",
    "US-L",
    "US-T"
]


admission_difference_matrix = (
    split_vs_single_commodity_df
    .pivot(
        index="source",
        columns="stream",
        values="single_minus_split_mbps"
    )
    .reindex(
        index=source_order_local,
        columns=stream_order_local
    )
)


fig, ax = plt.subplots(
    figsize=(8, 5)
)

image = ax.imshow(
    admission_difference_matrix.values,
    aspect="auto"
)

ax.set_xticks(
    range(
```

```
        len(
            admission_difference_matrix.columns
        )
    )
)

ax.set_xticklabels(
    admission_difference_matrix.columns
)

ax.set_yticks(
    range(
        len(
            admission_difference_matrix.index
        )
    )
)

ax.set_yticklabels(
    admission_difference_matrix.index
)


for i in range(
    len(
        admission_difference_matrix.index
    )
):
    for j in range(
        len(
            admission_difference_matrix.columns
        )
    ):
        value = (
            admission_difference_matrix.iloc[
                i,
                j
            ]
        )

        ax.text(
            j,
            i,
            f"{value:+.1f}",
            ha="center",
            va="center"
        )

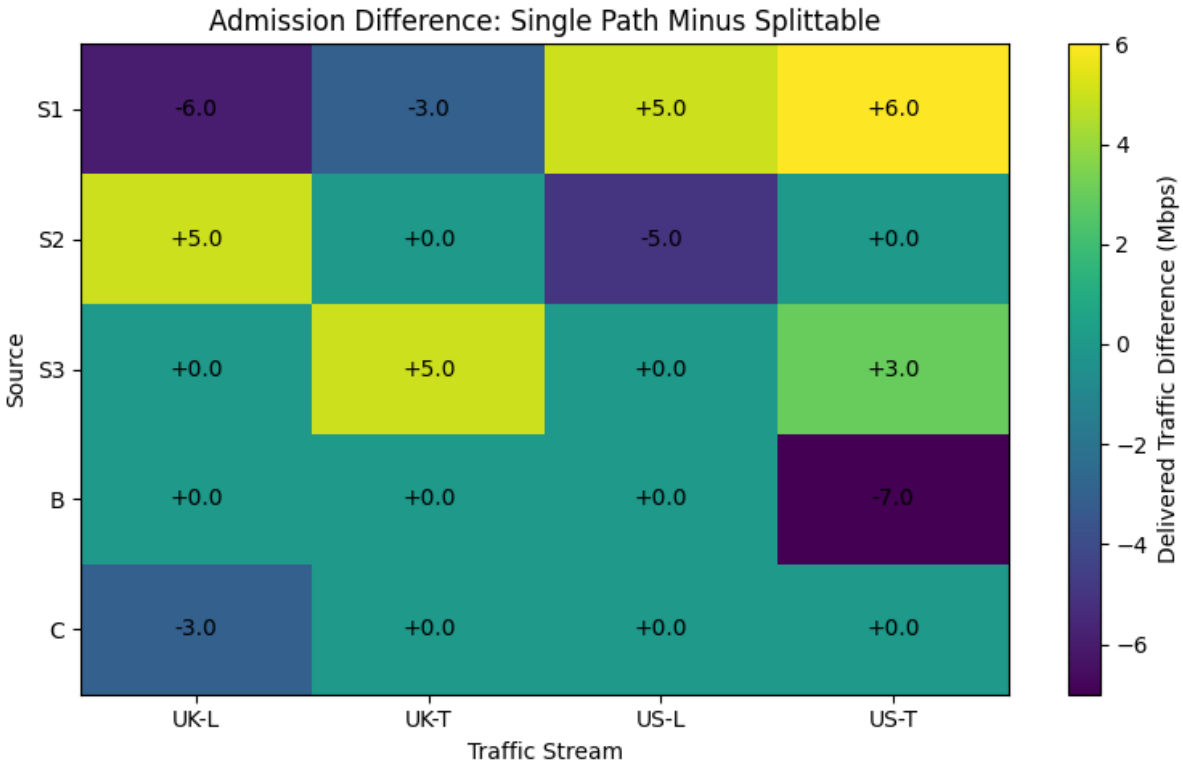

ax.set_title(
    "Admission Difference: "
    "Single Path Minus Splittable"
```

```
)

ax.set_xlabel("Traffic Stream")
ax.set_ylabel("Source")


plt.colorbar(
    image,
    ax=ax,
    label="Delivered Traffic Difference (Mbps)"
)

plt.tight_layout()
plt.show()
```



39.9.1 Visualization Interpretation

The heatmap shows where service shifts occur despite the equal 83-Mbps objective value.

A positive cell indicates greater single-path service.

A negative cell indicates greater split-model service.

Because both models have identical total throughput, the matrix must balance to approximately zero.

39.9.2 Business Impact

The heatmap makes the allocation effect visible.

This is important because an executive dashboard showing only:

$$83 \text{ Mbps}$$

for each model would conceal these differences completely.

39.10 Stream-Level Admission Comparison

The commodity-level results can now be aggregated by traffic class.

The four streams are:

- UK-L;
- UK-T;
- US-L;
- US-T.

This is particularly important for UK-L and US-L because those streams are latency-sensitive.

In [396…

```
final_stream_comparison_df = (
    split_stage1_stream_summary[
        [
            "stream",
            "demand_mbps",
            "delivered_mbps",
            "lost_mbps",
            "delivery_percentage"
        ]
    ]
    .rename(
        columns={
            "delivered_mbps":
                "split_delivered_mbps",
            "lost_mbps":
                "split_lost_mbps",
            "delivery_percentage":
                "split_delivery_pct"
        }
    )
    .merge(
        single_stage1_stream_summary[
            [
                "stream",
```

```
                "delivered_mbps",
                "lost_mbps",
                "delivery_percentage"
            ]
        ]
        .rename(
            columns={
                "delivered_mbps":
                    "single_delivered_mbps",
                "lost_mbps":
                    "single_lost_mbps",
                "delivery_percentage":
                    "single_delivery_pct"
            }
        ),
        on="stream",
        how="inner"
    )
)


final_stream_comparison_df[
    "single_minus_split_mbps"
] = (
    final_stream_comparison_df[
        "single_delivered_mbps"
    ]
    -
    final_stream_comparison_df[
        "split_delivered_mbps"
    ]
)


final_stream_comparison_df
```

Out [396…

	stream	demand_mbps	split_delivered_mbps	split_lost_mbps	split_delivery
0	UK-L	26.0	16.0	10.0	61.53
1	UK-T	39.0	34.0	5.0	87.17
2	US-L	29.0	18.0	11.0	62.06
3	US-T	32.0	15.0	17.0	46.87

39.10.1 Results and Explanation

For UK-L:

Splittable:

$$16 \text{ Mbps}.$$

Single path:

$$12 \text{ Mbps}.$$

Therefore, split routing delivers:

$$4 \text{ Mbps}$$

more UK-L traffic.

For US-L, both models deliver:

$$18 \text{ Mbps}.$$

Therefore, equal total throughput does not imply equal traffic-class throughput.

39.10.2 Interpretation

The different stream mix is particularly important because UK-L and US-L are the classes whose latency is optimized in Stage 2.

The split pipeline begins Stage 2 with more UK-L traffic than the single-path pipeline.

39.10.3 Business Impact

If sensitive traffic has greater operational value, a pure throughput objective may not be sufficient for production.

The organization may eventually want explicit:

- sensitive-service minimums;
- priority weights;
- SLA constraints.

39.11 Stream-Level Visualization

In [397…

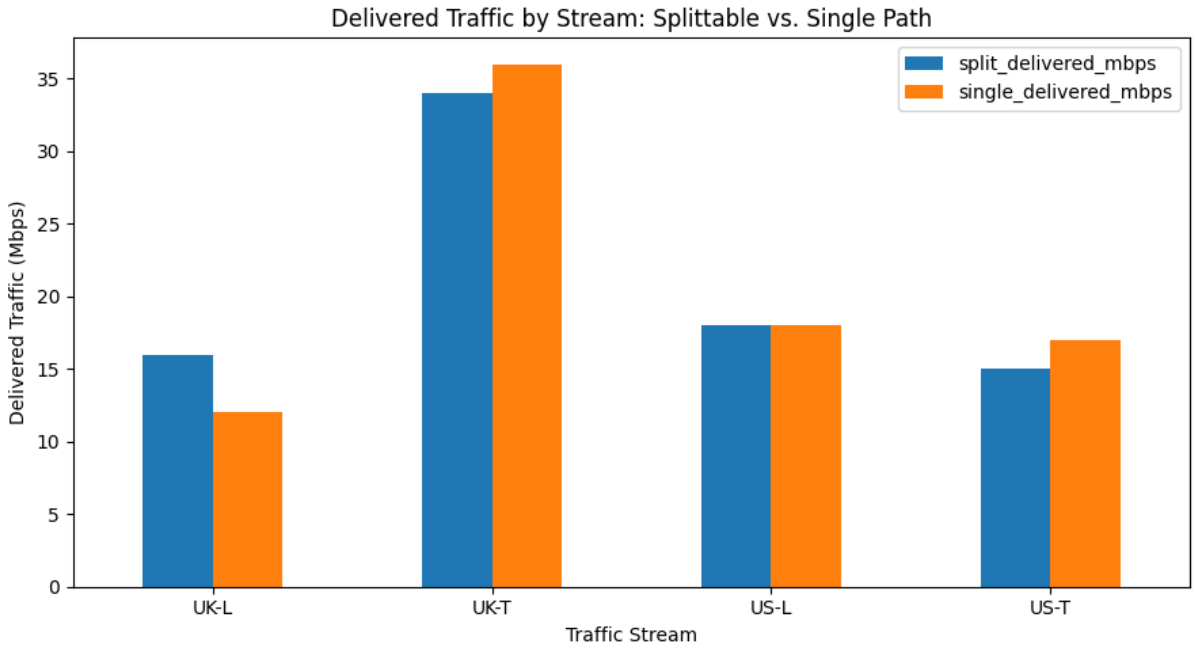
```
final_stream_comparison_df.set_index(
    "stream"
)[
    [
        "split_delivered_mbps",
        "single_delivered_mbps"
    ]
].plot(
```

```
    kind="bar",
    figsize=(9, 5)
)


plt.ylabel("Delivered Traffic (Mbps)")
plt.xlabel("Traffic Stream")

plt.title(
    "Delivered Traffic by Stream: "
    "Splittable vs. Single Path"
)

plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



3.11.1 Visualization Interpretation

The grouped bars visually demonstrate that the equality:

$$T^*_{single} = T^*_{split}$$

does not extend to individual traffic classes.

The UK-L difference should be particularly visible.

39.11.2 Business Impact

Traffic-class reporting prevents a strong network-wide KPI from masking weaker performance for a strategically important service category.

39.12 Source-Level Admission Comparison

The next comparison aggregates Stage 1 delivery by source.

The five sources are:

- S1;
- S2;
- S3;
- B;
- C.

The question is whether the same 83 Mbps is distributed differently across originating nodes.

In [399…

```
final_source_comparison_df = (
    split_stage1_source_summary[
        [
            "source",
            "demand_mbps",
            "delivered_mbps",
            "lost_mbps",
            "delivery_percentage"
        ]
    ]
    .rename(
        columns={
            "delivered_mbps":
                "split_delivered_mbps",
            "lost_mbps":
                "split_lost_mbps",
            "delivery_percentage":
                "split_delivery_pct"
        }
    )
    .merge(
        single_stage1_source_summary[
            [
                "source",
                "delivered_mbps",
                "lost_mbps",
                "delivery_percentage"
            ]
        ]
        .rename(
            columns={
                "delivered_mbps":
                    "single_delivered_mbps",
```

```
                "lost_mbps":
                    "single_lost_mbps",
                "delivery_percentage":
                    "single_delivery_pct"
            }
        ),
        on="source",
        how="inner"
    )
)


final_source_comparison_df[
    "single_minus_split_mbps"
] = (
    final_source_comparison_df[
        "single_delivered_mbps"
    ]
    -
    final_source_comparison_df[
        "split_delivered_mbps"
    ]
)


final_source_comparison_df
```

Out [399...

	source	demand_mbps	split_delivered_mbps	split_lost_mbps	split_delivery
0	S1	24.0	13.0	11.0	54.16
1	S2	21.0	11.0	10.0	52.38
2	S3	24.0	2.0	22.0	8.33
3	B	37.0	37.0	0.0	100.00
4	C	20.0	20.0	0.0	100.00

39.112.1 Interpretation

The source-level comparison should show that the two 83-Mbps Stage 1 solutions distribute service differently across origins.

A particularly important difference involves S3.

Earlier results showed approximately:

- Split Stage 1: 2 Mbps from S3;
- Single-Path Stage 1: 10 Mbps from S3.

Therefore:

$$\text{Equal Network Throughput} \neq \text{Equal Source-Level Service}.$$

39.12.2 Why This Happens

Stage 1 has no fairness objective or source-level minimum-delivery requirement.

Therefore, the optimizer is free to select any source allocation that achieves the maximum total throughput.

39.12.3 Business Impact

If sources correspond to regions, customers, or sites, equal network throughput may still create materially different customer experiences.

Source-level service therefore becomes an important governance metric.

39.13 Source-Level Visualization

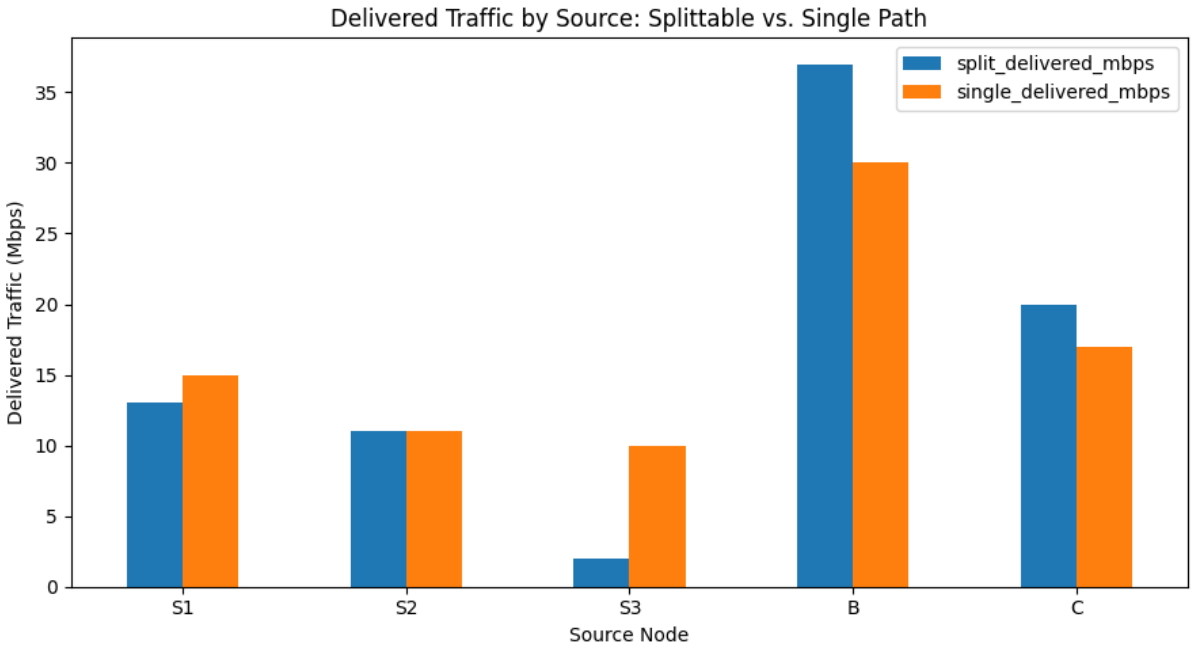
In [400…

```
final_source_comparison_df.set_index(
    "source"
)[
    [
        "split_delivered_mbps",
        "single_delivered_mbps"
    ]
].plot(
    kind="bar",
    figsize=(9, 5)
)


plt.ylabel("Delivered Traffic (Mbps)")
plt.xlabel("Source Node")

plt.title(
    "Delivered Traffic by Source: "
    "Splittable vs. Single Path"
)

plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



39.13.1 Visualization Interpretation

The source-level bars should make service redistribution visible.

Large differences between paired bars indicate that the two formulations reach the same total objective using different source populations.

39.13.2 Business Impact

This visualization is particularly useful when source nodes represent real customers or geographic markets.

It prevents the 83-Mbps network total from obscuring unequal local service.

39.14 Validate the Latency-Result Schemas Before Comparison

Before comparing final Stage 2 latency results, both reporting tables should contain the same metrics.

The required fields are:

- `stream`;
- `delivered_mbps`;
- `average_latency_ms`;
- `worst_case_latency_ms`;

- `latency_exposure_mbps_ms`.

Latency exposure is:

$$E_k = \sum_{s,p} f_{k,s,p} L_p.$$

When delivered traffic and average latency are already available:

$$E_k = z_k \bar{L}_k.$$

Units:

$$\text{Mbps} \cdot \text{ms}.$$

In [401…

```
if (
    "latency_exposure_mbps_ms"
    not in split_latency_results.columns
):
    split_latency_results = (
        split_latency_results.copy()
    )

    split_latency_results[
        "latency_exposure_mbps_ms"
    ] = (
        split_latency_results[
            "delivered_mbps"
        ]
        *
        split_latency_results[
            "average_latency_ms"
        ]
    )

    print(
        "Created latency_exposure_mbps_ms "
        "for split_latency_results."
    )

else:
    print(
        "split_latency_results already contains "
        "latency_exposure_mbps_ms."
    )


if (
    "latency_exposure_mbps_ms"
    not in single_latency_results.columns
):
```

```
    single_latency_results = (
        single_latency_results.copy()
    )

    single_latency_results[
        "latency_exposure_mbps_ms"
    ] = (
        single_latency_results[
            "delivered_mbps"
        ]
        *
        single_latency_results[
            "average_latency_ms"
        ]
    )

    print(
        "Created latency_exposure_mbps_ms "
        "for single_latency_results."
    )

else:
    print(
        "single_latency_results already contains "
        "latency_exposure_mbps_ms."
    )
```

```
split_latency_results already contains latency_exposure_mbps_ms.
single_latency_results already contains latency_exposure_mbps_ms.
```

39.14.1 Interpretation

If one DataFrame does not already contain:

`latency_exposure_mbps_ms`

that does not indicate a modeling problem.

It is only a difference in reporting-table construction.

The metric can be reconstructed exactly using:

$$E_k = z_k \bar{L}_k.$$

39.14.2 Example Validation

For final split UK-L:

$$16(18.25) = 292 \text{ Mbps} \cdot \text{ms}.$$

For final split US-L:

$$18(19.5556) \approx 352 \text{ Mbps} \cdot \text{ms}.$$

These values are consistent with the earlier Stage 2 results.

39.14.3 Business Impact

Standardizing the reporting schema ensures that the final comparison reflects real model differences rather than differences in DataFrame construction.

39.15 Final Sensitive-Latency Comparison

Stage 1 showed equality in total throughput:

$$T^*_{single} = T^*_{split}.$$

Stage 2 now asks whether the larger split feasible region creates value through better service quality.

The final comparison includes:

1. delivered sensitive traffic;
2. traffic-weighted average latency;
3. worst used-path latency;
4. latency exposure.

Average latency:

$$\bar{L}_k = \frac{\sum_{s,p} f_{k,s,p} L_p}{\sum_{s,p} f_{k,s,p}}.$$

Worst used-path latency:

$$L_k^{max} = \max_{s,p: f_{k,s,p}>0} L_p.$$

Latency exposure:

$$E_k = \sum_{s,p} f_{k,s,p} L_p.$$

In [402…

```
final_latency_comparison_df = (
    split_latency_results[
        [
```

```
            "stream",
            "delivered_mbps",
            "latency_exposure_mbps_ms",
            "average_latency_ms",
            "worst_case_latency_ms"
        ]
    ]
    .rename(
        columns={
            "delivered_mbps":
                "split_delivered_mbps",
            "latency_exposure_mbps_ms":
                "split_latency_exposure",
            "average_latency_ms":
                "split_average_latency_ms",
            "worst_case_latency_ms":
                "split_worst_latency_ms"
        }
    )
    .merge(
        single_latency_results[
            [
                "stream",
                "delivered_mbps",
                "latency_exposure_mbps_ms",
                "average_latency_ms",
                "worst_case_latency_ms"
            ]
        ]
        .rename(
            columns={
                "delivered_mbps":
                    "single_delivered_mbps",
                "latency_exposure_mbps_ms":
                    "single_latency_exposure",
                "average_latency_ms":
                    "single_average_latency_ms",
                "worst_case_latency_ms":
                    "single_worst_latency_ms"
            }
        ),
        on="stream",
        how="inner"
    )
)


final_latency_comparison_df[
    "single_minus_split_avg_latency_ms"
] = (
    final_latency_comparison_df[
        "single_average_latency_ms"
```

```
    ]
    -
    final_latency_comparison_df[
        "split_average_latency_ms"
    ]
)


final_latency_comparison_df[
    "single_minus_split_worst_latency_ms"
] = (
    final_latency_comparison_df[
        "single_worst_latency_ms"
    ]
    -
    final_latency_comparison_df[
        "split_worst_latency_ms"
    ]
)


final_latency_comparison_df[
    "split_avg_latency_advantage_pct"
] = (
    100
    * (
        final_latency_comparison_df[
            "single_average_latency_ms"
        ]
        -
        final_latency_comparison_df[
            "split_average_latency_ms"
        ]
    )
    /
    final_latency_comparison_df[
        "single_average_latency_ms"
    ]
)


final_latency_comparison_df
```

Out[402…

	stream	split_delivered_mbps	split_latency_exposure	split_average_latency_
0	UK-L	16.0	292.0	18.2500
1	US-L	18.0	352.0	19.5555

39.15.1 Results and Explanation

Expected final results are approximately:

Stream	Split Delivered	Split Avg	Split Worst	Single Delivered	Single Avg	Single Worst
UK-L	16 Mbps	18.25 ms	27 ms	12 Mbps	21.17 ms	29 ms
US-L	18 Mbps	19.56 ms	31 ms	18 Mbps	24.78 ms	32 ms

The Stage 1 comparison showed that splitting has no baseline throughput advantage.

However, Stage 2 shows that the split pipeline produces lower observed latency for both sensitive streams.

Therefore, the larger split feasible region has value even though that value does not appear in the Stage 1 throughput objective.

The baseline value of splitting appears primarily in:

> **service quality rather than service quantity.**

39.15.2 Interpretation

The mathematical relationship from Section 39.2 explains why this result is possible.

The two models can tie on:

$$T^*,$$

while still occupying different locations within their feasible regions.

The split model's additional routing flexibility can therefore create a lower-latency Stage 2 operating point without increasing Stage 1 throughput.

39.15.3 Why This Happens

The split formulation can reposition tolerant traffic at a finer level.

This can preserve attractive low-latency links for UK-L and US-L traffic.

The single-path model cannot divide a commodity in the same way.

Therefore, equality in maximum throughput does not imply equality in achievable

latency.

39.15.4 Business Impact

This is the first major result showing why a routing decision should not be based on throughput alone.

The split formulation provides no additional baseline Mbps, but it produces better observed latency.

If latency has business value, that additional routing flexibility may justify the operational complexity.

39.16 Detailed UK-L Latency Interpretation

Single-path UK-L average latency:

$$21.1667 \text{ ms}.$$

Split UK-L average latency:

$$18.25 \text{ ms}.$$

Absolute difference:

$$21.1667 - 18.25 = 2.9167 \text{ ms}.$$

Relative difference:

$$\frac{2.9167}{21.1667} \times 100 \approx 13.8\%.$$

Worst used-path latency improves:

$$29 \to 27 \text{ ms}.$$

39.16.1 Interpretation

The split pipeline provides both:

- lower observed UK-L latency;
- greater UK-L delivered volume.

Split:

$$16 \text{ Mbps}.$$

Single path:

$$12 \text{ Mbps}.$$

39.16.2 Analytical Caveat

The UK-L traffic populations are not identical.

Therefore, the full latency difference cannot be attributed solely to the larger split feasible region.

It contains both:

- an admission-mix effect;
- a routing-flexibility effect.

39.16.3 Business Impact

Operationally, the result is still favorable:

> **The split pipeline serves more UK-L traffic while also producing lower observed UK-L latency.**

For strict attribution of the routing effect alone, a common-admission experiment is preferable.

39.17 Detailed US-L Latency Interpretation

Single-path US-L average latency:

$$24.7778 \text{ ms}.$$

Split US-L average latency:

$$19.5556 \text{ ms}.$$

Absolute difference:

$$24.7778 - 19.5556 = 5.2222 \text{ ms}.$$

Relative difference:

$$\frac{5.2222}{24.7778} \times 100 \approx 21.1\%.$$

Worst used-path latency improves:

$$32 \rightarrow 31 \text{ ms}.$$

39.17.1 nterpretation

The observed US-L average latency is materially lower under the split final solution.

Both models deliver:

$$18 \text{ Mbps}$$

of US-L traffic at the aggregate stream level.

39.17.2 Analytical Caveat

Equal aggregate US-L delivery does not guarantee identical source-level composition.

Therefore, the comparison is more controlled than UK-L but is still not necessarily a perfect causal experiment.

39.17.3 Business Impact

A roughly:

$$21.1\%$$

reduction in observed average US-L latency provides evidence that the extra flexibility available in the split formulation may have meaningful service-quality value.

39.18 Average-Latency Visualization

In [405…

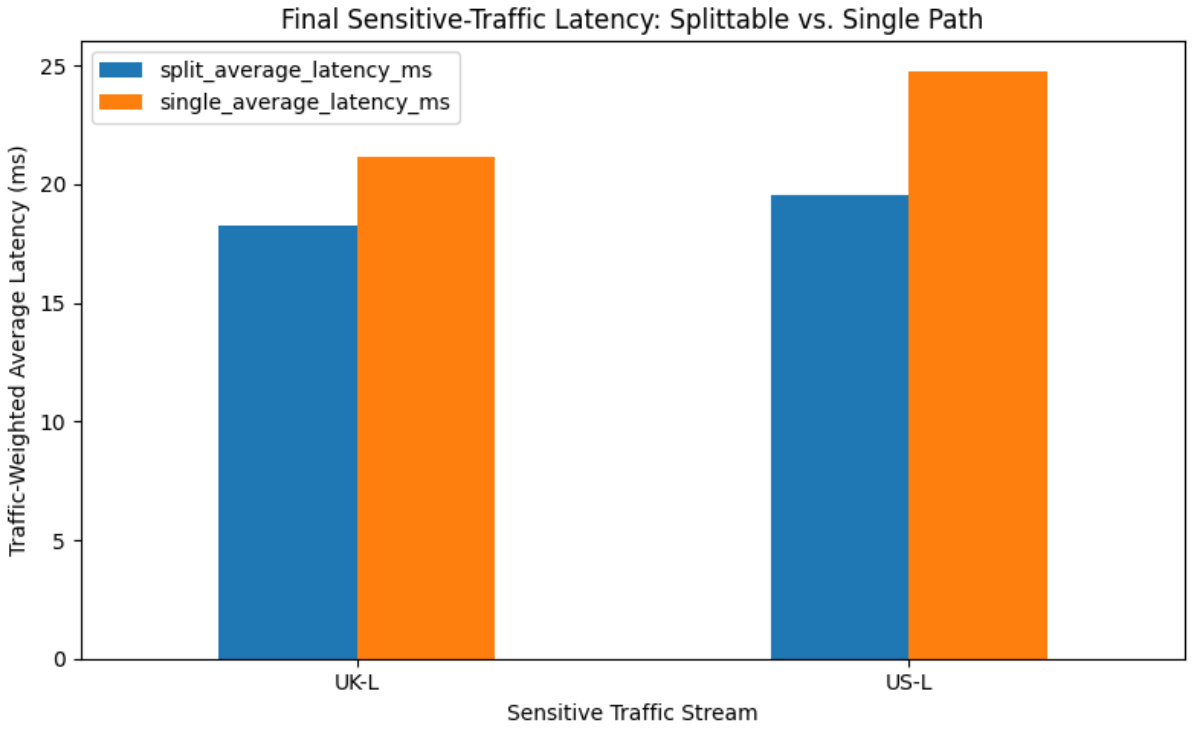
```
final_latency_comparison_df.set_index(
    "stream"
)[
    [
        "split_average_latency_ms",
        "single_average_latency_ms"
    ]
].plot(
    kind="bar",
    figsize=(8, 5)
)
```

```
plt.ylabel(
    "Traffic-Weighted Average Latency (ms)"
)

plt.xlabel(
    "Sensitive Traffic Stream"
)

plt.title(
    "Final Sensitive-Traffic Latency: "
    "Splittable vs. Single Path"
)

plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



39.18.1 Visualization Interpretation

Both sensitive streams should show lower average latency under the split solution.

The visual therefore demonstrates that:

$$\text{Equal Maximum Throughput} \not\Rightarrow \text{Equal Service Quality}.$$

39.18.2 Business Impact

The figure provides a clear reason why the larger split feasible region may still matter even when Stage 1 throughput ties.

Its baseline value appears primarily in latency improvement.

39.19 Worst Used-Path Latency Visualization

In [407…

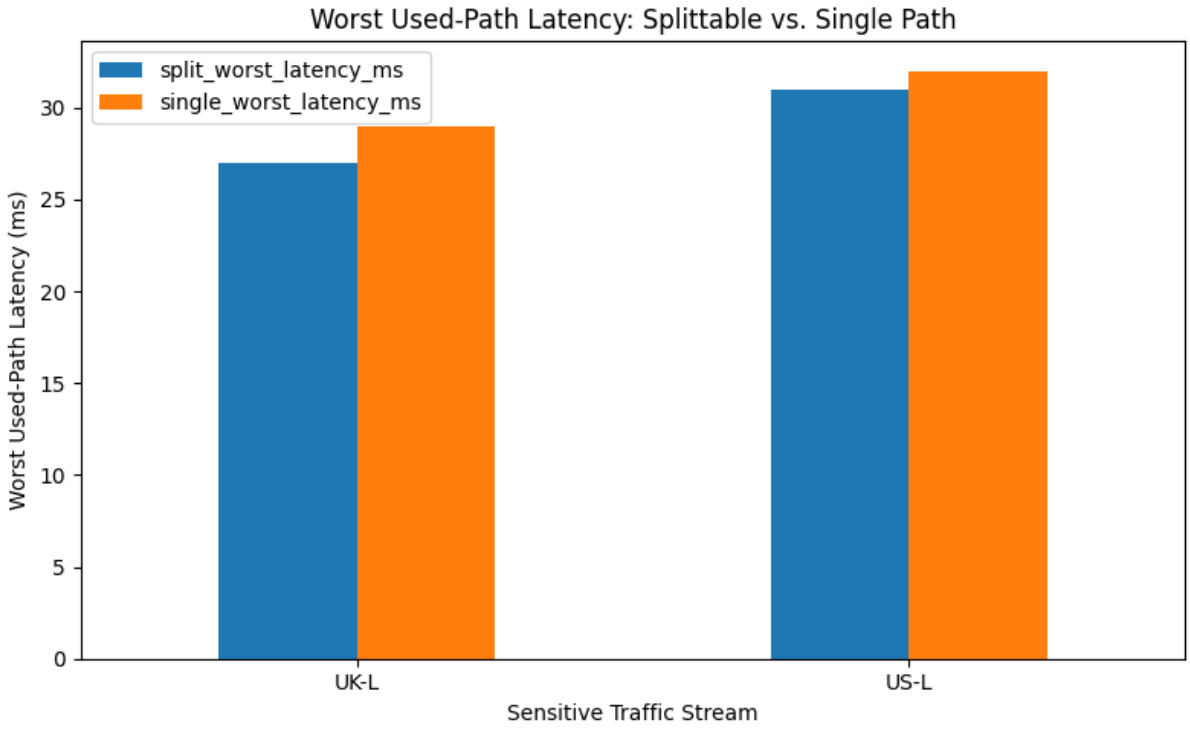
```
final_latency_comparison_df.set_index(
    "stream"
)[
    [
        "split_worst_latency_ms",
        "single_worst_latency_ms"
    ]
].plot(
    kind="bar",
    figsize=(8, 5)
)


plt.ylabel(
    "Worst Used-Path Latency (ms)"
)

plt.xlabel(
    "Sensitive Traffic Stream"
)

plt.title(
    "Worst Used-Path Latency: "
    "Splittable vs. Single Path"
)

plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



39.19.1 Visualization Interpretation

The split final solution should also show slightly lower worst used-path latency.

This indicates that the average-latency improvement is not merely an averaging artifact.

The slowest sensitive route actually used also improves.

39.19.2 Business Impact

Worst-path performance matters for consistency.

The improvement in both average and worst used-path latency strengthens the case that the split solution produces a better sensitive-service operating point.

39.20 Admission Mix vs. Routing Flexibility

The Stage 1 throughput equality:

$$T^*_{single} = T^*_{split}$$

does not mean that the two Stage 2 latency results are directly comparable without qualification.

For some commodities:

$$z_{k,s}^{split} \neq z_{k,s}^{single}.$$

Therefore:

$$\boxed{\text{Observed Final Difference} = \text{Admission-Mix Effect} + \text{Routing-Flexibility Ef}}$$

39.20.1 Interpretation

There are two valid analytical questions.

Operational Question

> **Which complete pipeline produces the better operating result?**

The current comparison answers that question.

Causal Question

> **How much of the improvement is caused specifically by the additional routing flexibility in the split feasible region?**

That question requires a controlled admission vector.

39.20.2 Common Admission Vector

A conservative common admission vector can be defined as:

$$z_{k,s}^{common} = \min\left(z_{k,s}^{split}, z_{k,s}^{single}\right).$$

This keeps only the amount of each commodity that both baseline solutions demonstrated they can serve.

In [408…

```
common_admission_df = (
    split_vs_single_commodity_df[
        [
            "commodity_id",
            "source",
            "stream",
            "destination",
            "split_delivered_mbps",
            "single_delivered_mbps"
        ]
```

```
    ]
    .copy()
)


common_admission_df[
    "common_delivered_mbps"
] = (
    common_admission_df[
        [
            "split_delivered_mbps",
            "single_delivered_mbps"
        ]
    ]
    .min(
        axis=1
    )
)


common_admission_total = (
    common_admission_df[
        "common_delivered_mbps"
    ].sum()
)


print(
    "Total traffic in common admission vector:",
    common_admission_total,
    "Mbps"
)


common_admission_df
```

```
Total traffic in common admission vector: 59.0 Mbps
```

Out[408...

	commodity_id	source	stream	destination	split_delivered_mbps	single_de
0	S1_UK-L	S1	UK-L	UK	6.0	
1	S1_UK-T	S1	UK-T	UK	7.0	
2	S1_US-L	S1	US-L	US	0.0	
3	S1_US-T	S1	US-T	US	0.0	
4	S2_UK-L	S2	UK-L	UK	0.0	
5	S2_UK-T	S2	UK-T	UK	6.0	
6	S2_US-L	S2	US-L	US	5.0	
7	S2_US-T	S2	US-T	US	0.0	
8	S3_UK-L	S3	UK-L	UK	0.0	
9	S3_UK-T	S3	UK-T	UK	2.0	
10	S3_US-L	S3	US-L	US	0.0	
11	S3_US-T	S3	US-T	US	0.0	
12	B_UK-L	B	UK-L	UK	6.0	
13	B_UK-T	B	UK-T	UK	14.0	
14	B_US-L	B	US-L	US	8.0	
15	B_US-T	B	US-T	US	9.0	
16	C_UK-L	C	UK-L	UK	4.0	
17	C_UK-T	C	UK-T	UK	5.0	
18	C_US-L	C	US-L	US	5.0	
19	C_US-T	C	US-T	US	6.0	

39.20.3 Interpretation

The common-admission vector creates a controlled traffic population.

The purpose is not to maximize throughput.

Instead, both routing formulations could be re-solved with identical commodity admissions.

Any remaining latency difference would then be more directly attributable to routing flexibility.

39.20.4 Business Impact

This optional experiment would help determine whether the operational complexity of multipath routing itself creates sufficient value to justify deployment.

39.21 Final Network Utilization Comparison

The throughput equality does not imply identical use of physical infrastructure.

This subsection compares final Stage 2:

- directed-link flow;
- utilization;
- residual capacity;
- saturation.

The question is:

> **How differently do the two formulations use the same physical network?**

In [409…

```
final_link_comparison_df = (
    split_stage2_link_df[
        [
            "from_node",
            "to_node",
            "capacity_mbps",
            "flow_mbps",
            "residual_capacity_mbps",
            "utilization_percentage",
            "is_saturated"
        ]
    ]
    .rename(
        columns={
            "flow_mbps":
                "split_flow_mbps",
            "residual_capacity_mbps":
                "split_residual_mbps",
            "utilization_percentage":
                "split_utilization_pct",
            "is_saturated":
                "split_saturated"
        }
    )
    .merge(
```

```
        single_stage2_link_df[
            [
                "from_node",
                "to_node",
                "flow_mbps",
                "residual_capacity_mbps",
                "utilization_percentage",
                "is_saturated"
            ]
        ]
        .rename(
            columns={
                "flow_mbps":
                    "single_flow_mbps",
                "residual_capacity_mbps":
                    "single_residual_mbps",
                "utilization_percentage":
                    "single_utilization_pct",
                "is_saturated":
                    "single_saturated"
            }
        ),
        on=[
            "from_node",
            "to_node"
        ],
        how="inner"
    )
)


final_link_comparison_df[
    "link"
] = (
    final_link_comparison_df[
        "from_node"
    ]
    + " → "
    + final_link_comparison_df[
        "to_node"
    ]
)


final_link_comparison_df[
    "single_minus_split_flow_mbps"
] = (
    final_link_comparison_df[
        "single_flow_mbps"
    ]
    -
    final_link_comparison_df[
```

```
        "split_flow_mbps"
    ]
)


final_link_comparison_df.sort_values(
    "split_utilization_pct",
    ascending=False
)
```

Out[409…

	from_node	to_node	capacity_mbps	split_flow_mbps	split_residual_mbps
0	C	UK	30.0	30.0	0.0
1	A	B	20.0	20.0	0.0
2	A	C	25.0	25.0	0.0
3	B	C	18.0	18.0	0.0
4	C	US	25.0	25.0	0.0
5	B	D	28.0	28.0	0.0
6	D	UK	22.0	20.0	2.0
7	B	A	20.0	13.5	6.5
8	S1	A	40.0	13.0	27.0
9	S2	A	35.0	11.0	24.0
10	C	A	25.0	7.5	17.5

11	D	US	30.0	8.0	22.0
12	S3	B	30.0	2.0	28.0
13	C	B	18.0	0.5	17.5
14	D	B	28.0	0.0	28.0
15	UK	C	30.0	0.0	30.0
16	A	S1	40.0	0.0	40.0
17	US	C	25.0	0.0	25.0
18	B	S3	30.0	0.0	30.0
19	UK	D	22.0	0.0	22.0
20	A	S2	35.0	0.0	35.0
21	US	D	30.0	0.0	30.0

39.21.1 Interpretation

The two formulations can achieve the same maximum throughput while using substantially different internal routing patterns.

The larger split feasible region allows more fine-grained rearrangement of tolerant traffic.

That freedom is used during Stage 2 to improve sensitive latency.

39.21.2 Why This Happens

The Stage 2 objective does not minimize total utilization.

It minimizes sensitive latency while preserving admission.

Therefore, consuming additional capacity on some internal links can be optimal if that frees preferable routes for sensitive traffic.

39.21.3 Business Impact

The network may therefore trade residual headroom for latency improvement.

Whether that tradeoff is desirable depends on the relative value of:

- latency;
- burst tolerance;
- failure headroom;
- reserve capacity.

39.22 Saturated-Link Comparison

In [411…

```
final_saturation_comparison_df = pd.DataFrame([
    {
        "model":
            "Splittable Final",
        "saturated_directed_links":
            int(
                split_stage2_link_df[
                    "is_saturated"
                ].sum()
            )
    },
    {
        "model":
            "Single-Path Final",
        "saturated_directed_links":
            int(
                single_stage2_link_df[
                    "is_saturated"
                ].sum()
            )
    }
])


final_saturation_comparison_df
```

Out [411...

	model	saturated_directed_links
0	Splittable Final	6
1	Single-Path Final	4

39.22.1 Expected Results

Splittable final solution:

$$6$$

saturated directed links.

Single-path final solution:

$$4$$

saturated directed links.

39.22.2 Interpretation

The split solution saturates more links while simultaneously producing lower sensitive latency.

Therefore:

> **Saturation count is not a standalone measure of solution quality.**

39.22.3 Why This Happens

The larger split feasible region allows the optimizer to use more of the network's available capacity in order to improve the Stage 2 latency objective.

39.22.5 Business Impact

A saturated link should not automatically be treated as an investment priority.

The stronger question is whether additional capacity on that link increases end-to-end throughput.

That question is addressed through perturbation analysis in Section 41.

39.23 Final Link-Utilization Visualization

In [413…

```
final_active_link_comparison_df = (
    final_link_comparison_df[
        (
            final_link_comparison_df[
                "split_flow_mbps"
            ] > 1e-8
        )
        |
        (
            final_link_comparison_df[
                "single_flow_mbps"
            ] > 1e-8
        )
    ]
    .copy()
)

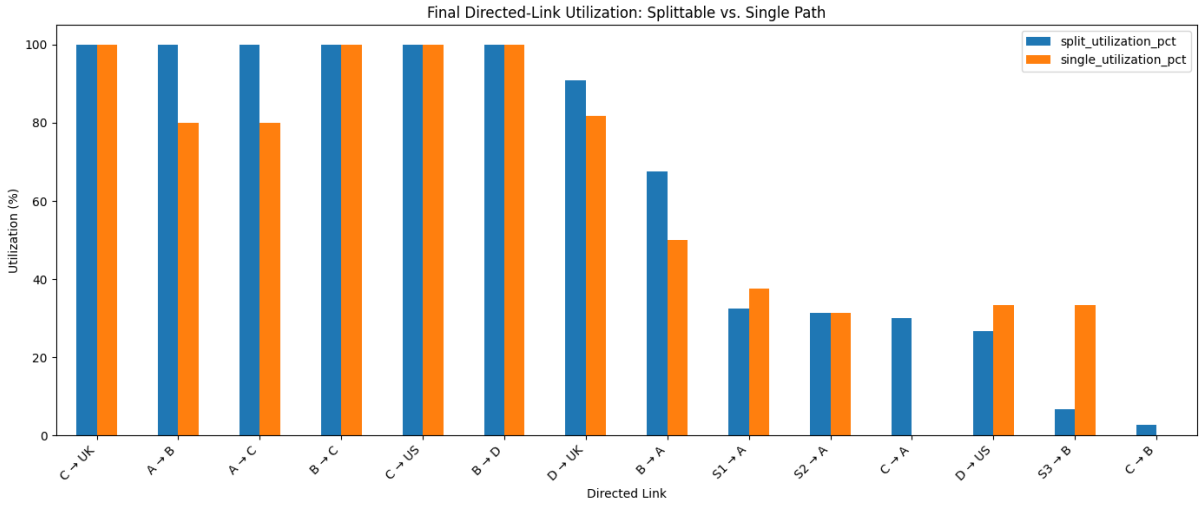

final_active_link_comparison_df.set_index(
    "link"
)[
    [
        "split_utilization_pct",
        "single_utilization_pct"
    ]
].plot(
    kind="bar",
    figsize=(14, 6)
)


plt.ylabel("Utilization (%)")
plt.xlabel("Directed Link")

plt.title(
    "Final Directed-Link Utilization: "
    "Splittable vs. Single Path"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()
plt.show()
```



39.23.1 Visualization Interpretation

The graph should show that equal total throughput does not imply equal link utilization.

The two formulations use the internal network differently.

The split solution uses some corridors more aggressively in order to achieve the lower-latency Stage 2 operating point.

39.23.2 Business Impact

This demonstrates that traffic engineering can create performance value without changing physical infrastructure.

The routing policy changes how existing capacity is consumed.

39.24 Route Complexity Comparison

The larger feasible region of the split formulation comes with operational consequences.

A single-path commodity is easier to:

- monitor;
- explain;
- troubleshoot;
- enforce.

A split commodity may simultaneously use several positive-flow routes.

In [414…

```
split_final_route_count_df = (
    split_stage2_routes_df
    .groupby(
        "commodity_id",
        as_index=False
    )
    .size()
    .rename(
        columns={
            "size":
                "split_active_paths"
        }
    )
)


single_final_route_count_df = (
    single_stage2_routes_df
    .groupby(
        "commodity_id",
        as_index=False
    )
    .size()
    .rename(
        columns={
            "size":
                "single_active_paths"
        }
    )
)


final_route_complexity_df = (
    split_final_route_count_df
    .merge(
        single_final_route_count_df,
        on="commodity_id",
        how="outer"
    )
    .fillna(0)
)


final_route_complexity_df
```

Out [414...

	commodity_id	split_active_paths	single_active_paths
0	B_UK-L	1.0	1.0
1	B_UK-T	2.0	1.0
2	B_US-L	1.0	1.0
3	B_US-T	2.0	1.0
4	C_UK-L	1.0	1.0
5	C_UK-T	2.0	1.0
6	C_US-L	1.0	1.0
7	C_US-T	1.0	1.0
8	S1_UK-L	1.0	0.0
9	S1_UK-T	2.0	1.0
10	S2_UK-T	2.0	1.0
11	S2_US-L	1.0	0.0
12	S3_UK-T	1.0	1.0
13	S1_US-L	0.0	1.0
14	S1_US-T	0.0	1.0
15	S2_UK-L	0.0	1.0
16	S3_US-T	0.0	1.0

39.24.1 Interpretation

The single-path model guarantees:

$$\text{Active Paths}_{k,s} \leq 1.$$

The split formulation does not.

Therefore, the larger split feasible region translates into potentially greater operational route complexity.

39.24.2 Why This Happens

Using multiple routes allows the optimizer to combine small pieces of residual capacity across several corridors.

That flexibility is mathematically valuable but operationally more complex.

39.25 Route Complexity Visualization

In [415…

```
final_route_complexity_df.set_index(
    "commodity_id"
)[
    [
        "split_active_paths",
        "single_active_paths"
    ]
].plot(
    kind="bar",
    figsize=(14, 5)
)

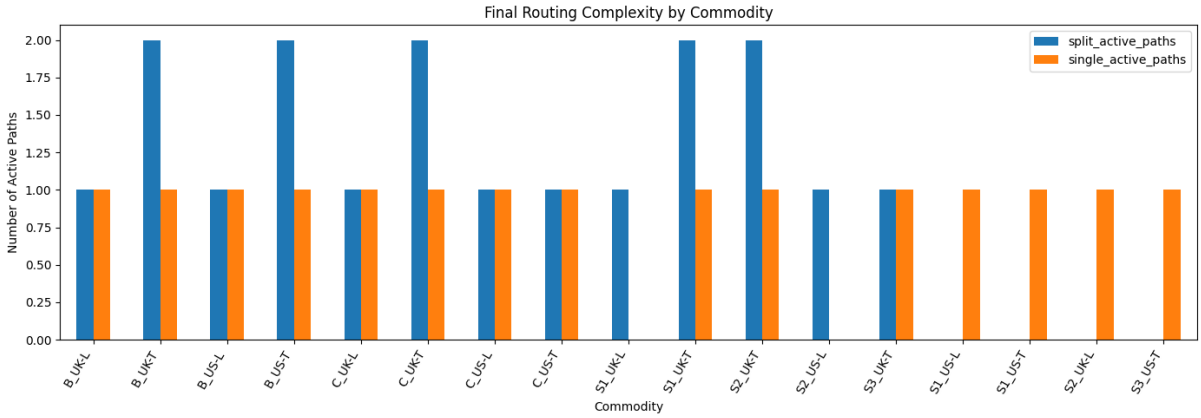

plt.ylabel(
    "Number of Active Paths"
)

plt.xlabel(
    "Commodity"
)

plt.title(
    "Final Routing Complexity by Commodity"
)

plt.xticks(
    rotation=60,
    ha="right"
)

plt.tight_layout()
plt.show()
```



39.25.2 Visualization Interpretation

The single-path bars should never exceed one.

The split model may show more than one active route for some commodities.

The figure makes the operational-complexity consequence of the larger split feasible region directly visible.

39.25.3 Business Impact

The organization can compare this added route complexity directly with the measured latency benefit.

That provides a more complete decision basis than throughput alone.

39.26 Mathematical Optimization Complexity

The feasible-region relationship also has an interesting computational implication.

Splittable Routing

Operationally more complex.

Mathematically simpler.

It is a:

> **Linear Program (LP).**

Single-Path Routing

Operationally simpler.

Mathematically more difficult because route selection requires binary variables.

It is a:

> **Mixed-Integer Linear Program (MILP).**

In [416…

```
optimization_complexity_df = pd.DataFrame([
    {
        "model":
            "Splittable Stage 1",
        "problem_type":
            "LP",
        "continuous_variables":
```

```
            68,
        "binary_variables":
            0,
        "total_variables":
            68
    },
    {
        "model":
            "Single-Path Stage 1",
        "problem_type":
            "MILP",
        "continuous_variables":
            68,
        "binary_variables":
            68,
        "total_variables":
            136
    },
    {
        "model":
            "Splittable Stage 2",
        "problem_type":
            "LP",
        "continuous_variables":
            68,
        "binary_variables":
            0,
        "total_variables":
            68
    },
    {
        "model":
            "Single-Path Stage 2",
        "problem_type":
            "MILP",
        "continuous_variables":
            68,
        "binary_variables":
            68,
        "total_variables":
            136
    }
])


optimization_complexity_df
```

Out [416...

	model	problem_type	continuous_variables	binary_variables	total_variab
0	Splittable Stage 1	LP	68	0	
1	Single-Path Stage 1	MILP	68	68	1
2	Splittable Stage 2	LP	68	0	
3	Single-Path Stage 2	MILP	68	68	1

39.26.1 Interpretation

The split formulation has a larger routing feasible region but uses only continuous flow variables.

The single-path formulation restricts that feasible region using binary route-selection variables.

At the current scale, both are manageable.

At larger scales, the MILP may become substantially more computationally expensive.

39.26.2 Why This Happens

Linear programs optimize over continuous feasible regions.

Mixed-integer programs must also search over discrete route-selection combinations.

That combinatorial search can grow rapidly as:

- commodities;
- candidate paths;
- time periods;

increase.

39.26.3 Business Impact

A production system therefore faces two different complexity tradeoffs:

- split routing is operationally more complex;
- single-path routing is computationally more complex.

If re-optimization must occur frequently, solver scalability may become an important architecture consideration.

39.27 Final Routing Decision Scorecard

The previous subsections show that the equality in Stage 1 throughput does not imply that the two policies are equivalent.

The following scorecard summarizes the major tradeoffs.

In [417…

```
routing_decision_scorecard_df = pd.DataFrame([
    {
        "dimension":
            "Baseline Throughput",
        "splittable":
            "83 Mbps",
        "single_path":
            "83 Mbps",
        "preference":
            "Tie"
    },
    {
        "dimension":
            "Observed Sensitive Latency",
        "splittable":
            "Lower",
        "single_path":
            "Higher",
        "preference":
            "Splittable"
    },
    {
        "dimension":
            "Route Simplicity",
        "splittable":
            "Lower",
        "single_path":
            "Higher",
        "preference":
            "Single Path"
    },
    {
        "dimension":
            "Fine-Grained Capacity Use",
```

```
        "splittable":
            "Higher",
        "single_path":
            "Lower",
        "preference":
            "Splittable"
    },
    {
        "dimension":
            "Mathematical Complexity",
        "splittable":
            "LP",
        "single_path":
            "MILP",
        "preference":
            "Splittable"
    },
    {
        "dimension":
            "Operational Complexity",
        "splittable":
            "Higher",
        "single_path":
            "Lower",
        "preference":
            "Single Path"
    },
    {
        "dimension":
            "Fragmented Capacity Potential",
        "splittable":
            "Potential advantage",
        "single_path":
            "Potential limitation",
        "preference":
            "Test in Section 43"
    }
])


routing_decision_scorecard_df
```

Out [417…

	dimension	splittable	single_path	preference
0	Baseline Throughput	83 Mbps	83 Mbps	Tie
1	Observed Sensitive Latency	Lower	Higher	Splittable
2	Route Simplicity	Lower	Higher	Single Path
3	Fine-Grained Capacity Use	Higher	Lower	Splittable
4	Mathematical Complexity	LP	MILP	Splittable
5	Operational Complexity	Higher	Lower	Single Path
6	Fragmented Capacity Potential	Potential advantage	Potential limitation	Test in Section 43

39.27.1 Interpretation

The Stage 1 equality means neither model wins on baseline throughput.

The decision therefore depends on the other dimensions.

39.27.2 Single Path Is Stronger When

- operational simplicity matters;
- deterministic routing matters;
- baseline throughput is the primary requirement.

39.27.3 Splittable Routing Is Stronger When

- latency matters;
- fine-grained capacity allocation matters;
- fragmented residual capacity matters;
- LP scalability matters.

39.27.4 Business Impact

The routing policy should be selected according to the current operating objective.

A practical system may benefit from retaining both capabilities:

- use single-path routing when it meets service requirements;
- use split routing when its larger feasible region creates measurable latency, resilience, or stress-performance value.

39.28 Detailed Section Summary

Section 39 provides the complete comparison of the split and single-path routing pipelines.

1. Mathematical Relationship

The feasible regions satisfy:

$$\mathcal{F}_{single} \subseteq \mathcal{F}_{split}.$$

Therefore:

$$T^*_{single} \leq T^*_{split}.$$

2. Baseline Throughput

The numerical results fall into the equality case:

$$T^*_{single} = T^*_{split} = 83 \text{ Mbps}.$$

Therefore, the one-path restriction is not throughput-binding under the baseline conditions.

3. Undelivered Demand

Both formulations leave:

$$43 \text{ Mbps}$$

of:

$$126 \text{ Mbps}$$

undelivered.

4. Commodity Admission

Equality in objective value does not imply equality in solution.

The two models serve different source-stream populations.

Therefore:

$$\boxed{\text{Equal Throughput} \neq \text{Equal Admission}}$$

5. Stream-Level Service

Split Stage 1 delivers:

$$16 \text{ Mbps}$$

of UK-L traffic compared with:

$$12 \text{ Mbps}$$

under single-path routing.

Both deliver:

$$18 \text{ Mbps}$$

of US-L.

6. Source-Level Service

The models also allocate service differently among sources.

7. Sensitive Latency

Despite identical Stage 1 throughput, the split Stage 2 solution produces lower observed sensitive latency.

UK-L:

$$18.25 \text{ ms}$$

versus:

$$21.17 \text{ ms}.$$

US-L:

$$19.56 \text{ ms}$$

versus:

$$24.78 \text{ ms}.$$

8. Admission-Mix Caveat

The latency difference contains some combination of:

- admission-mix effect;
- routing-flexibility effect.

A controlled common-admission experiment would isolate the latter more cleanly.

9. Infrastructure Use

Equal throughput does not imply equal link utilization.

The split Stage 2 model uses more of the physical network and saturates more directed links while producing lower sensitive latency.

10. Route Complexity

The larger split feasible region permits multiple active paths per commodity.

That increases routing flexibility but also operational complexity.

11. Mathematical Complexity

Split routing is an LP.

Single-path routing is a MILP.

The operationally simpler policy is mathematically harder to optimize.

Overall Conclusion

The mathematical relationship established in Section 39.2 is confirmed by the baseline results.

The models tie on maximum throughput:

$$83 = 83.$$

Therefore, the business case for splittable routing does not come from baseline throughput.

Its value appears instead in:

- lower sensitive latency;
- finer-grained capacity allocation;
- potentially greater value under stressed conditions.

The evidence therefore does not support declaring one formulation universally superior.

39.29 Business Recommendation

The most practical conclusion is:

> **Use the simplest routing policy that satisfies the current service objective, while retaining splittable-routing capability for network states where its larger feasible region creates measurable value.**

For the baseline:

- maximum throughput is tied;
- single-path routing is operationally simpler;
- splittable routing produces lower observed sensitive latency.

Therefore, the routing choice should depend on the value placed on:

- latency;
- operational simplicity;
- reserve capacity;
- resilience;
- solver scalability.

The stressed-network analysis in Section 43 will test whether the split feasible region becomes more valuable under different network conditions.

Connection to Section 40

Section 39 established that:

$$T^*_{single} = T^*_{split},$$

but also showed that equal throughput does not imply equal:

- admission;
- latency;
- routing;
- utilization.

Section 40 now shifts the analysis from:

> **Which routing formulation performs better?**

to:

> **Which physical links consistently experience pressure across the optimized network states?**

Section 40 will compare:

1. Splittable Stage 1;
2. Splittable Stage 2;
3. Single-Path Stage 1;
4. Single-Path Stage 2.

That analysis will identify:

- persistent high-utilization links;
- policy-dependent congestion;
- residual capacity;
- potential stranded capacity.

Those results then provide the foundation for the true marginal-capacity bottleneck analysis in Section 41.

40. Cross-State Infrastructure Utilization and Capacity Headroom Analysis

40.1 Purpose, Context, and Connection to Section 39

Section 39 compared the **service outcomes** produced by the splittable and single-path routing formulations.

That comparison established several important results:

- both Stage 1 models deliver the same maximum throughput:

$$83 \text{ Mbps};$$

- equal throughput does not imply equal commodity admission;
- the two models distribute service differently across sources and traffic classes;
- the final splittable solution produces lower observed latency for UK-L and US-L;
- the routing formulations use the physical network differently.

Section 40 now changes the analytical perspective.

Rather than asking:

> **Which routing formulation produces the better service outcome?**

this section asks:

> **How does the physical network behave across all four optimized operating states?**

The four operating states are:

1. **Splittable Stage 1**
    - maximum throughput;
2. **Splittable Stage 2**
    - same Stage 1 admission with sensitive-latency minimization;
3. **Single-Path Stage 1**
    - maximum throughput subject to one route per commodity;
4. **Single-Path Stage 2**
    - same single-path Stage 1 admission with sensitive-latency minimization.

The goal is to identify:

- links that are consistently highly utilized;
- links whose utilization depends strongly on routing policy;
- links with substantial residual capacity;
- links that appear saturated across several network states;
- capacity that may be physically available but not useful end-to-end.

This section provides the infrastructure-level foundation for Section 41, where apparent congestion is tested using direct capacity perturbation and re-optimization.

40.1.2 Interpretation

Section 39 showed that different routing policies can produce the same total throughput while using the network differently.

Section 40 therefore evaluates the infrastructure itself rather than only the traffic delivered to users.

This distinction is important because link utilization can reveal:

- recurring congestion patterns;

- alternate-corridor dependence;
- exit pressure;
- unused capacity;
- routing-policy sensitivity.

However, high utilization alone does not prove that a link is economically or operationally valuable to expand.

That stronger question is deferred to Section 41.

40.1.3 Business Impact

From an infrastructure-planning perspective, a network operator should distinguish among:

- links that are consistently constrained;
- links that are constrained only under one routing policy;
- links with large amounts of spare capacity;
- links whose spare capacity cannot actually support additional end-to-end traffic.

This distinction prevents capital-planning decisions from being based only on utilization percentages.

Section 40 therefore serves as a screening analysis.

Section 41 will determine which capacity constraints actually have measurable marginal value.

40.1.4 Connection to the Next Subsection

The first step is to combine the directed-link results from all four optimized states into one consistent comparison table.

That unified table will allow every physical directed arc to be compared across:

- flow;
- utilization;
- residual capacity;
- saturation status.

40.2 Standardize the Four Optimized Link-State Tables

40.2.1 MARKDOWN CELL — Purpose

40.2 Standardize the Four Optimized Link-State Tables

Each optimization stage produced a directed-link result table.

For a directed link e, the primary infrastructure quantities are:

Flow

$$F_e$$

measured in Mbps.

Capacity

$$C_e$$

measured in Mbps.

Utilization

$$U_e = 100\frac{F_e}{C_e}.$$

Residual Capacity / Headroom

$$H_e = C_e - F_e.$$

A link is considered saturated when:

$$F_e \approx C_e$$

within the numerical tolerance used by the notebook.

To compare the four optimized states consistently, I first standardize their column names and add a state identifier.

40.2.2 CODE CELL — Standardization Helper

In [418…

```
# ------------------------------------------------------------
# 40.2 Standardize the four optimized link-state tables
# ------------------------------------------------------------

def standardize_link_state(
    link_df,
```

```
    state_name
):
    """
    Convert one optimized directed-link result table into a
    consistent schema for cross-state comparison.

    Required input columns:
        from_node
        to_node
        capacity_mbps
        flow_mbps
        residual_capacity_mbps
        utilization_percentage
        is_saturated
    """

    output_df = (
        link_df[
            [
                "from_node",
                "to_node",
                "capacity_mbps",
                "flow_mbps",
                "residual_capacity_mbps",
                "utilization_percentage",
                "is_saturated"
            ]
        ]
        .copy()
    )

    output_df[
        "state"
    ] = state_name

    output_df[
        "link"
    ] = (
        output_df[
            "from_node"
        ]
        + " → "
        + output_df[
            "to_node"
        ]
    )

    return output_df


split_stage1_state_df = standardize_link_state(
    split_stage1_link_df,
```

```
    "Split Stage 1"
)

split_stage2_state_df = standardize_link_state(
    split_stage2_link_df,
    "Split Stage 2"
)

single_stage1_state_df = standardize_link_state(
    single_stage1_link_df,
    "Single Stage 1"
)

single_stage2_state_df = standardize_link_state(
    single_stage2_link_df,
    "Single Stage 2"
)


all_link_states_df = pd.concat(
    [
        split_stage1_state_df,
        split_stage2_state_df,
        single_stage1_state_df,
        single_stage2_state_df
    ],
    ignore_index=True
)


all_link_states_df.head()
```

Out [418...

	from_node	to_node	capacity_mbps	flow_mbps	residual_capacity_mbps	u
0	C	UK	30.0	30.0	0.0	
1	C	US	25.0	25.0	0.0	
2	B	D	28.0	28.0	0.0	
3	A	C	25.0	24.0	1.0	
4	D	UK	22.0	20.0	2.0	

40.2.3 Interpretation

The combined table places all four optimized operating states into a single long-format DataFrame.

Each row now represents:

> **one directed link in one optimized network state.**

This structure makes it possible to calculate:

- cross-state averages;
- cross-state maximum utilization;
- saturation frequency;
- residual headroom;
- persistent infrastructure pressure.

The physical network itself has not changed.

Only the optimized routing state changes.

40.2.4 Business Impact

Standardizing the four states creates one infrastructure reporting layer rather than four separate model-specific outputs.

In a production environment, this same approach could support comparison across:

- different routing policies;
- demand scenarios;
- failure states;
- time periods;
- capacity plans.

40.3 Validate Link Coverage and Capacity Consistency

Before comparing utilization across states, I verify that the physical network definition is consistent.

The synthetic network contains:

- 11 physical full-duplex links;
- 22 directed arcs.

Therefore, each optimization-state table should contain:

$$22$$

directed links.

Because the physical infrastructure does not change between optimization stages, each directed arc should also retain the same capacity across all four states.

40.3.2 CODE CELL — Validation

In [419…

```
# ------------------------------------------------------------
# 40.3 Validate link coverage and capacity consistency
# ------------------------------------------------------------

state_link_counts_df = (
    all_link_states_df
    .groupby(
        "state",
        as_index=False
    )
    .agg(
        directed_link_count=(
            "link",
            "nunique"
```

```
        )
    )
)


capacity_consistency_df = (
    all_link_states_df
    .groupby(
        [
            "from_node",
            "to_node",
            "link"
        ],
        as_index=False
    )
    .agg(
        minimum_capacity_mbps=(
            "capacity_mbps",
            "min"
        ),
        maximum_capacity_mbps=(
            "capacity_mbps",
            "max"
        ),
        number_of_states=(
            "state",
            "nunique"
        )
    )
)


capacity_consistency_df[
    "capacity_consistent"
] = (
    capacity_consistency_df[
        "minimum_capacity_mbps"
    ]
    -
    capacity_consistency_df[
        "maximum_capacity_mbps"
    ]
).abs() < 1e-8


print(
    "Directed-link counts by state:"
)

display(
    state_link_counts_df
)
```

```
print(
    "\nAll capacities consistent across states:",
    bool(
        capacity_consistency_df[
            "capacity_consistent"
        ].all()
    )
)


print(
    "All links represented in four states:",
    bool(
        (
            capacity_consistency_df[
                "number_of_states"
            ]
            == 4
        ).all()
    )
)
```

```
Directed-link counts by state:
```

	state	directed_link_count
0	Single Stage 1	22
1	Single Stage 2	22
2	Split Stage 1	22
3	Split Stage 2	22

```
All capacities consistent across states: True
All links represented in four states: True
```

40.3.3 Expected Results

Each optimized state should contain:

$$22$$

directed links.

The capacity-consistency test should return:

```
All capacities consistent across states: True
All links represented in four states: True
```

40.3.4 Interpretation

These validation checks confirm that any utilization difference observed later is caused by:

- routing decisions;
- admission decisions;
- Stage 2 latency optimization;

rather than by changes to the physical network itself.

This is important because the purpose of Section 40 is to compare network behavior while holding infrastructure constant.

40.3.5 Business Impact

Infrastructure comparisons are only meaningful when the underlying asset definition is consistent.

This validation prevents a changed capacity value or missing link from being incorrectly interpreted as a routing-policy effect.

40.4 Create the Cross-State Link Comparison Table

In [420…

```
# ------------------------------------------------------------
# 40.4 Create one wide comparison table for all four states
# ------------------------------------------------------------

flow_wide_df = (
    all_link_states_df
    .pivot(
        index=[
            "from_node",
            "to_node",
            "link",
            "capacity_mbps"
        ],
        columns="state",
        values="flow_mbps"
    )
    .reset_index()
)


utilization_wide_df = (
    all_link_states_df
    .pivot(
        index=[
```

```
            "from_node",
            "to_node",
            "link",
            "capacity_mbps"
        ],
        columns="state",
        values="utilization_percentage"
    )
    .reset_index()
)


residual_wide_df = (
    all_link_states_df
    .pivot(
        index=[
            "from_node",
            "to_node",
            "link",
            "capacity_mbps"
        ],
        columns="state",
        values="residual_capacity_mbps"
    )
    .reset_index()
)


saturation_wide_df = (
    all_link_states_df
    .pivot(
        index=[
            "from_node",
            "to_node",
            "link",
            "capacity_mbps"
        ],
        columns="state",
        values="is_saturated"
    )
    .reset_index()
)


cross_state_link_comparison_df = (
    utilization_wide_df
    .copy()
)


cross_state_link_comparison_df
```

Out[420…

	state	from_node	to_node	link	capacity_mbps	Single Stage 1	Single Stage 2	Split
0		A	B	A → B	20.0	20.000000	80.000000	0.0
1		A	C	A → C	25.0	100.000000	80.000000	96.0
2		A	S1	A → S1	40.0	0.000000	0.000000	0.0
3		A	S2	A → S2	35.0	0.000000	0.000000	0.0
4		B	A	B → A	20.0	15.000000	50.000000	0.0
5		B	C	B → C	18.0	100.000000	100.000000	6
6		B	D	B → D	28.0	100.000000	100.000000	100.0
7		B	S3	B → S3	30.0	0.000000	0.000000	0.0
8		C	A	C → A	25.0	0.000000	0.000000	0.0
9		C	B	C → B	18.0	27.777778	0.000000	0.0
10		C	UK	C → UK	30.0	100.000000	100.000000	100.0
11		C	US	C → US	25.0	100.000000	100.000000	100.0
12		D	B	D → B	28.0	0.000000	0.000000	0.0
13		D	UK	D → UK	22.0	81.818182	81.818182	90.9

14	D	US	D → US	30.0	33.333333	33.333333	26.6
15	S1	A	S1 → A	40.0	37.500000	37.500000	32.5
16	S2	A	S2 → A	35.0	31.428571	31.428571	31.
17	S3	B	S3 → B	30.0	33.333333	33.333333	6.6
18	UK	C	UK → C	30.0	0.000000	0.000000	0.0
19	UK	D	UK → D	22.0	0.000000	0.000000	0.0
20	US	C	US → C	25.0	0.000000	0.000000	0.0
21	US	D	US → D	30.0	0.000000	0.000000	0.0

40.4.1 Interpretation

The wide utilization table provides one row per directed link and one utilization column per optimized network state.

This makes it possible to compare the same physical link directly across:

- Split Stage 1;
- Split Stage 2;
- Single Stage 1;
- Single Stage 2.

The next subsection summarizes those states at the network level before examining individual links.

40.5 Network-Level Infrastructure Summary

Before classifying individual links, I summarize the infrastructure behavior of each optimized state.

The metrics are:

- average directed-link utilization;
- maximum directed-link utilization;
- number of saturated directed links;
- average residual capacity;
- minimum residual capacity.

These measures provide a high-level view of how aggressively each solution uses the physical network.

In [421…

```
# ------------------------------------------------------------
# 40.5 Network-level infrastructure summary
# ------------------------------------------------------------

state_infrastructure_summary_df = (
    all_link_states_df
    .groupby(
        "state",
        as_index=False
    )
    .agg(
        average_utilization_pct=(
            "utilization_percentage",
            "mean"
        ),
        maximum_utilization_pct=(
            "utilization_percentage",
            "max"
        ),
        saturated_directed_links=(
            "is_saturated",
            "sum"
        ),
        average_residual_capacity_mbps=(
            "residual_capacity_mbps",
            "mean"
        ),
        minimum_residual_capacity_mbps=(
            "residual_capacity_mbps",
            "min"
        )
    )
)


state_order = [
```

```
    "Split Stage 1",
    "Split Stage 2",
    "Single Stage 1",
    "Single Stage 2"
]


state_infrastructure_summary_df[
    "state"
] = pd.Categorical(
    state_infrastructure_summary_df[
        "state"
    ],
    categories=state_order,
    ordered=True
)


state_infrastructure_summary_df = (
    state_infrastructure_summary_df
    .sort_values(
        "state"
    )
    .reset_index(
        drop=True
    )
)


state_infrastructure_summary_df
```

Out[421…

	state	average_utilization_pct	maximum_utilization_pct	saturated_directed_
0	Split Stage 1	29.331005	100.0	
1	Split Stage 2	40.384035	100.0	
2	Single Stage 1	35.463236	100.0	
3	Single Stage 2	37.609701	100.0	

40.5.1 Expected Results

The saturated-link counts should be approximately:

Optimized State	Saturated Directed Links
Split Stage 1	3
Split Stage 2	6
Single Stage 1	5
Single Stage 2	4

The maximum utilization in each optimized state should be:

$$100\%.$$

These counts show that Stage 2 latency optimization can materially change how heavily the internal network is used even when Stage 1 admissions are preserved.

40.5.2 Interpretation

The splittable Stage 2 solution has the largest number of saturated directed links.

This does not mean that it is the worst operating state.

Section 39 showed that this same solution produces the best observed sensitive-traffic latency.

The result therefore illustrates an important distinction:

> **Infrastructure utilization and service quality are related, but they are not the same objective.**

The split Stage 2 optimizer is willing to consume more residual internal capacity when doing so improves sensitive latency.

40.5.3 Why This Happens

Stage 1 maximizes throughput.

Stage 2 preserves the Stage 1 admission vector but changes the routing objective.

For latency-sensitive traffic, Stage 2 minimizes:

$$\sum_{k \in K_L} \sum_{s,p} f_{k,s,p} L_p.$$

It does not minimize:

- number of saturated links;
- total network utilization;
- residual headroom.

Therefore, additional internal links may become heavily utilized if doing so creates lower-latency routing for sensitive traffic.

40.5.4 Business Impact

A network operator should not interpret a larger number of saturated links as automatically worse.

The relevant question is:

> **What service objective is the additional capacity consumption supporting?**

In this case, the added utilization supports lower latency.

Whether that tradeoff is desirable depends on the value placed on:

- latency;
- reserve headroom;
- burst tolerance;
- resilience.

40.6 Cross-State Utilization Metrics by Directed Link

In [425…

```
# ------------------------------------------------------------
# 40.6 Calculate cross-state utilization statistics
# ------------------------------------------------------------

cross_state_link_metrics_df = (
    all_link_states_df
    .groupby(
        [
            "from_node",
            "to_node",
            "link",
            "capacity_mbps"
        ],
        as_index=False
    )
```

```
    .agg(
        average_utilization_pct=(
            "utilization_percentage",
            "mean"
        ),
        maximum_utilization_pct=(
            "utilization_percentage",
            "max"
        ),
        minimum_utilization_pct=(
            "utilization_percentage",
            "min"
        ),
        average_residual_mbps=(
            "residual_capacity_mbps",
            "mean"
        ),
        minimum_residual_mbps=(
            "residual_capacity_mbps",
            "min"
        ),
        saturation_count=(
            "is_saturated",
            "sum"
        )
    )
)


cross_state_link_metrics_df[
    "saturation_frequency_pct"
] = (
    100
    * cross_state_link_metrics_df[
        "saturation_count"
    ]
    / 4
)


cross_state_link_metrics_df = (
    cross_state_link_metrics_df
    .sort_values(
        [
            "saturation_count",
            "maximum_utilization_pct",
            "average_utilization_pct"
        ],
        ascending=[
            False,
            False,
            False
```

```
        ]
    )
    .reset_index(
        drop=True
    )
)


cross_state_link_metrics_df
```

Out [425...

	from_node	to_node	link	capacity_mbps	average_utilization_pct	maximu
0	B	D	B → D	28.0	100.000000	
1	C	UK	C → UK	30.0	100.000000	
2	C	US	C → US	25.0	100.000000	
3	B	C	B → C	18.0	90.277778	
4	A	C	A → C	25.0	94.000000	
5	A	B	A → B	20.0	50.000000	
6	D	UK	D → UK	22.0	86.363636	
7	B	A	B → A	20.0	33.125000	
8	S1	A	S1 → A	40.0	35.000000	
9	D	US	D → US	30.0	30.000000	
10	S3	B	S3 → B	30.0	20.000000	
			S2			

11	S2	A	→ A	35.0	31.428571
12	C	A	C → A	25.0	7.500000
13	C	B	C → B	18.0	7.638889
14	A	S1	A → S1	40.0	0.000000
15	A	S2	A → S2	35.0	0.000000
16	B	S3	B → S3	30.0	0.000000
17	D	B	D → B	28.0	0.000000
18	UK	C	UK → C	30.0	0.000000
19	UK	D	UK → D	22.0	0.000000
20	US	C	US → C	25.0	0.000000
21	US	D	US → D	30.0	0.000000

40.6.2 Interpretation

The cross-state table shows which links experience recurring pressure rather than pressure in only one optimized solution.

The most important measures are:

Saturation Count

The number of the four optimized states in which a link reaches capacity.

Maximum Utilization

The highest utilization observed in any state.

Average Utilization

The average pressure placed on the link across all states.

Minimum Residual Capacity

The smallest amount of spare capacity observed across the four states.

Together, these measures distinguish:

- persistent congestion;
- policy-dependent congestion;
- consistently underutilized capacity.

40.6.3 Business Impact

A link that is heavily utilized in every optimized state is more strategically important than a link that becomes heavily utilized only under one specific routing policy.

Cross-state persistence therefore provides a better first-pass infrastructure-planning signal than any single-state utilization value.

40.7 Classify Infrastructure Pressure

40.7.1 MARKDOWN CELL — Classification Logic

40.7 Classify Infrastructure Pressure Across Optimized States

To make the infrastructure results easier to interpret, I classify each directed link using its cross-state utilization behavior.

The classification used here is:

Persistent Pressure

A link is saturated in at least:

of the four optimized states.

Policy-Dependent Pressure

A link is not persistently saturated, but reaches at least:

$$90\%$$

utilization in one or more optimized states.

Underutilized

The link never exceeds:

$$50\%$$

utilization in any optimized state.

Variable / Moderate

The link does not fit any of the categories above.

These thresholds are analytical labels used for interpretation.

They are not claims that a specific utilization percentage automatically requires capacity expansion.

In [426…

```
# ------------------------------------------------------------
# 40.7 Classify cross-state infrastructure pressure
# ------------------------------------------------------------

def classify_link_pressure(row):

    if (
        row[
            "saturation_count"
        ]
        >= 3
    ):
        return "Persistent Pressure"

    elif (
        row[
            "maximum_utilization_pct"
        ]
        >= 90
    ):
        return "Policy-Dependent Pressure"
```

```
    elif (
        row[
            "maximum_utilization_pct"
        ]
        < 50
    ):
        return "Underutilized"

    else:
        return "Variable / Moderate"


cross_state_link_metrics_df[
    "pressure_classification"
] = (
    cross_state_link_metrics_df
    .apply(
        classify_link_pressure,
        axis=1
    )
)


cross_state_link_metrics_df[
    [
        "link",
        "capacity_mbps",
        "average_utilization_pct",
        "maximum_utilization_pct",
        "minimum_residual_mbps",
        "saturation_count",
        "saturation_frequency_pct",
        "pressure_classification"
    ]
]
```

Out [426…

	link	capacity_mbps	average_utilization_pct	maximum_utilization_pct	mini
0	B → D	28.0	100.000000	100.000000	
1	C → UK	30.0	100.000000	100.000000	
2	C → US	25.0	100.000000	100.000000	
3	B → C	18.0	90.277778	100.000000	

4	A → C	25.0	94.000000	100.000000
5	A → B	20.0	50.000000	100.000000
6	D → UK	22.0	86.363636	90.909091
7	B → A	20.0	33.125000	67.500000
8	S1 → A	40.0	35.000000	37.500000
9	D → US	30.0	30.000000	33.333333
10	S3 → B	30.0	20.000000	33.333333
11	S2 → A	35.0	31.428571	31.428571
12	C → A	25.0	7.500000	30.000000
13	C → B	18.0	7.638889	27.777778
14	A → S1	40.0	0.000000	0.000000
15	A → S2	35.0	0.000000	0.000000
16	B → S3	30.0	0.000000	0.000000
17	D → B	28.0	0.000000	0.000000
18	UK →	30.0	0.000000	0.000000

	C			
19	UK → D	22.0	0.000000	0.000000
20	US → C	25.0	0.000000	0.000000
21	US → D	30.0	0.000000	0.000000

40.7.1 Expected Important Links

Based on the optimized results generated earlier, the strongest recurring pressure should appear on several forward-direction links.

Saturated in All Four Optimized States

The following links are expected to be saturated in all four states:

- B → D;
- C → UK;
- C → US.

Therefore:

$$\text{Saturation Count} = 4.$$

Saturated in Three of Four States

B → C is expected to be saturated in:

- Split Stage 2;
- Single Stage 1;
- Single Stage 2;

but not Split Stage 1.

Therefore:

$$\text{Saturation Count}_{B \to C} = 3.$$

Policy-Dependent Internal Links

Links such as:

- A → C;
- A → B;

become highly utilized or saturated only under selected optimization states.

This makes them more dependent on routing policy rather than universally constrained.

40.7.2 Interpretation

The recurring pressure pattern highlights two different types of infrastructure.

Destination-Facing Capacity

C → UK and C → US directly feed the destination exits.

These links are naturally important because traffic reaching those exits must consume their capacity.

Alternate Internal Corridor Capacity

B → D provides access to:

- D → UK;
- D → US.

This corridor is essential because the direct B → C → exit corridor cannot carry all offered traffic.

Routing-Policy-Dependent Capacity

Links such as A → B and A → C change substantially between optimized states.

Their high utilization reflects the selected routing strategy rather than a universal structural constraint.

40.7.3 Business Impact

Persistent utilization pressure is useful for identifying where engineering attention should be focused.

However, this is still only a screening signal.

A link can be:

- saturated;
- frequently saturated;
- strategically located;

and still have little marginal value if another downstream bottleneck prevents additional traffic from being delivered.

Section 41 therefore tests each directed link directly.

40.8 Cross-State Utilization Heatmap

A heatmap provides a compact visual comparison of directed-link utilization across the four optimized states.

Each cell represents:

$$U_{e,m}$$

where:

- e = directed link;
- m = optimized model state.

The visualization makes it easier to distinguish:

- persistent pressure;
- policy-dependent pressure;
- underused capacity.

In [427…

```
# ------------------------------------------------------------
# 40.8 Cross-state utilization heatmap
# ------------------------------------------------------------

utilization_heatmap_df = (
    all_link_states_df
    .pivot(
        index="link",
        columns="state",
        values="utilization_percentage"
    )
    .reindex(
        columns=state_order
    )
)
```

```
# Order links by maximum utilization, then average utilization.

heatmap_order = (
    utilization_heatmap_df
    .assign(
        max_utilization=(
            utilization_heatmap_df
            .max(
                axis=1
            )
        ),
        avg_utilization=(
            utilization_heatmap_df
            .mean(
                axis=1
            )
        )
    )
    .sort_values(
        [
            "max_utilization",
            "avg_utilization"
        ],
        ascending=False
    )
    .index
)


utilization_heatmap_df = (
    utilization_heatmap_df
    .loc[
        heatmap_order
    ]
)


fig, ax = plt.subplots(
    figsize=(10, 9)
)


image = ax.imshow(
    utilization_heatmap_df.values,
    aspect="auto"
)


ax.set_xticks(
    range(
        len(
```

```
            utilization_heatmap_df.columns
        )
    )
)

ax.set_xticklabels(
    utilization_heatmap_df.columns,
    rotation=30,
    ha="right"
)


ax.set_yticks(
    range(
        len(
            utilization_heatmap_df.index
        )
    )
)

ax.set_yticklabels(
    utilization_heatmap_df.index
)


for i in range(
    len(
        utilization_heatmap_df.index
    )
):
    for j in range(
        len(
            utilization_heatmap_df.columns
        )
    ):
        value = (
            utilization_heatmap_df
            .iloc[
                i,
                j
            ]
        )

        ax.text(
            j,
            i,
            f"{value:.0f}%",
            ha="center",
            va="center",
            fontsize=8
        )
```

```
ax.set_title(
    "Directed-Link Utilization Across "
    "Four Optimized Network States"
)

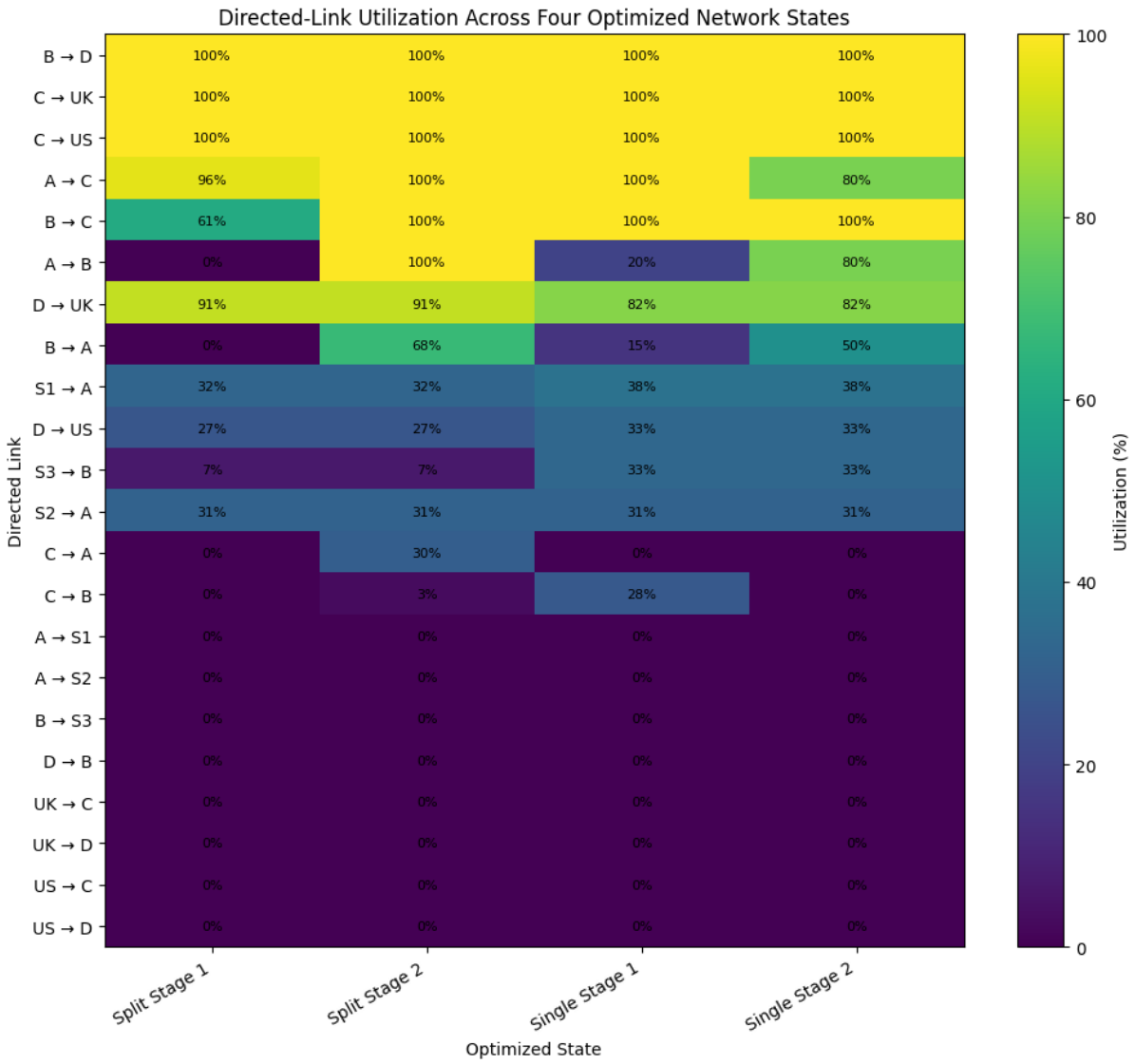
ax.set_xlabel(
    "Optimized State"
)

ax.set_ylabel(
    "Directed Link"
)


plt.colorbar(
    image,
    ax=ax,
    label="Utilization (%)"
)


plt.tight_layout()

plt.show()
```



40.8.1 Visualization Interpretation

The heatmap should reveal several distinct patterns.

Persistent High Utilization

Links such as:

- B → D;
- C → UK;
- C → US;

should appear near or at:

$$100\%$$

across all four optimized states.

Highly Variable Internal Utilization

Links such as:

- A → B;
- A → C;
- B → C;

should vary substantially depending on the routing formulation and optimization stage.

Low-Utilization Reverse Arcs

Several reverse-direction arcs may remain unused or lightly used.

This reflects the directional pattern of current source-to-exit traffic rather than an error in the physical full-duplex model.

40.8.4 Business Impact

The heatmap provides an efficient infrastructure summary for technical and executive review.

It makes clear which links are:

- structurally important;
- policy-sensitive;
- largely unused.

However, the heatmap should not be used by itself to prioritize investment.

Utilization indicates where traffic currently flows.

It does not measure how much additional end-to-end service an upgrade would create.

40.9 Residual Capacity and Headroom Analysis

Utilization can also be expressed through remaining headroom.

For directed link e:

$$H_e = C_e - F_e.$$

where:

- C_e = link capacity;
- F_e = optimized flow;
- H_e = remaining capacity.

A positive value means the link has unused capacity.

A value near zero means the link is saturated.

However:

> **Unused link capacity is not automatically usable end-to-end capacity.**

Additional traffic must still have a complete feasible path from source to the correct destination.

In [430…

```
# ------------------------------------------------------------
# 40.9 Compare residual capacity in final Stage 2 states
# ------------------------------------------------------------

final_residual_comparison_df = (
    final_link_comparison_df[
        [
            "link",
            "capacity_mbps",
            "split_residual_mbps",
            "single_residual_mbps"
        ]
    ]
    .copy()
)


final_residual_comparison_df[
    "minimum_final_residual_mbps"
] = (
    final_residual_comparison_df[
        [
            "split_residual_mbps",
            "single_residual_mbps"
        ]
    ]
    .min(
        axis=1
    )
)


final_residual_comparison_df = (
    final_residual_comparison_df
```

```
    .sort_values(
        "minimum_final_residual_mbps"
    )
    .reset_index(
        drop=True
    )
)


final_residual_comparison_df
```

Out[430…

	link	capacity_mbps	split_residual_mbps	single_residual_mbps	minimum_
0	C → UK	30.0	0.0	0.0	
1	A → B	20.0	0.0	4.0	
2	A → C	25.0	0.0	5.0	
3	B → C	18.0	0.0	0.0	
4	C → US	25.0	0.0	0.0	
5	B → D	28.0	0.0	0.0	
6	D → UK	22.0	2.0	4.0	
7	B → A	20.0	6.5	10.0	
8	C → B	18.0	17.5	18.0	
9	C → A	25.0	17.5	25.0	
10	D → US	30.0	22.0	20.0	

11	S3 → B	30.0	28.0	20.0
12	UK → D	22.0	22.0	22.0
13	S2 → A	35.0	24.0	24.0
14	S1 → A	40.0	27.0	25.0
15	US → C	25.0	25.0	25.0
16	D → B	28.0	28.0	28.0
17	UK → C	30.0	30.0	30.0
18	B → S3	30.0	30.0	30.0
19	US → D	30.0	30.0	30.0
20	A → S2	35.0	35.0	35.0
21	A → S1	40.0	40.0	40.0

In [431…

```
# ------------------------------------------------------------
# 40.9 Visualize final residual capacity
# ------------------------------------------------------------

final_residual_comparison_df.set_index(
    "link"
)[
    [
        "split_residual_mbps",
        "single_residual_mbps"
    ]
```

```
].plot(
    kind="barh",
    figsize=(10, 9)
)

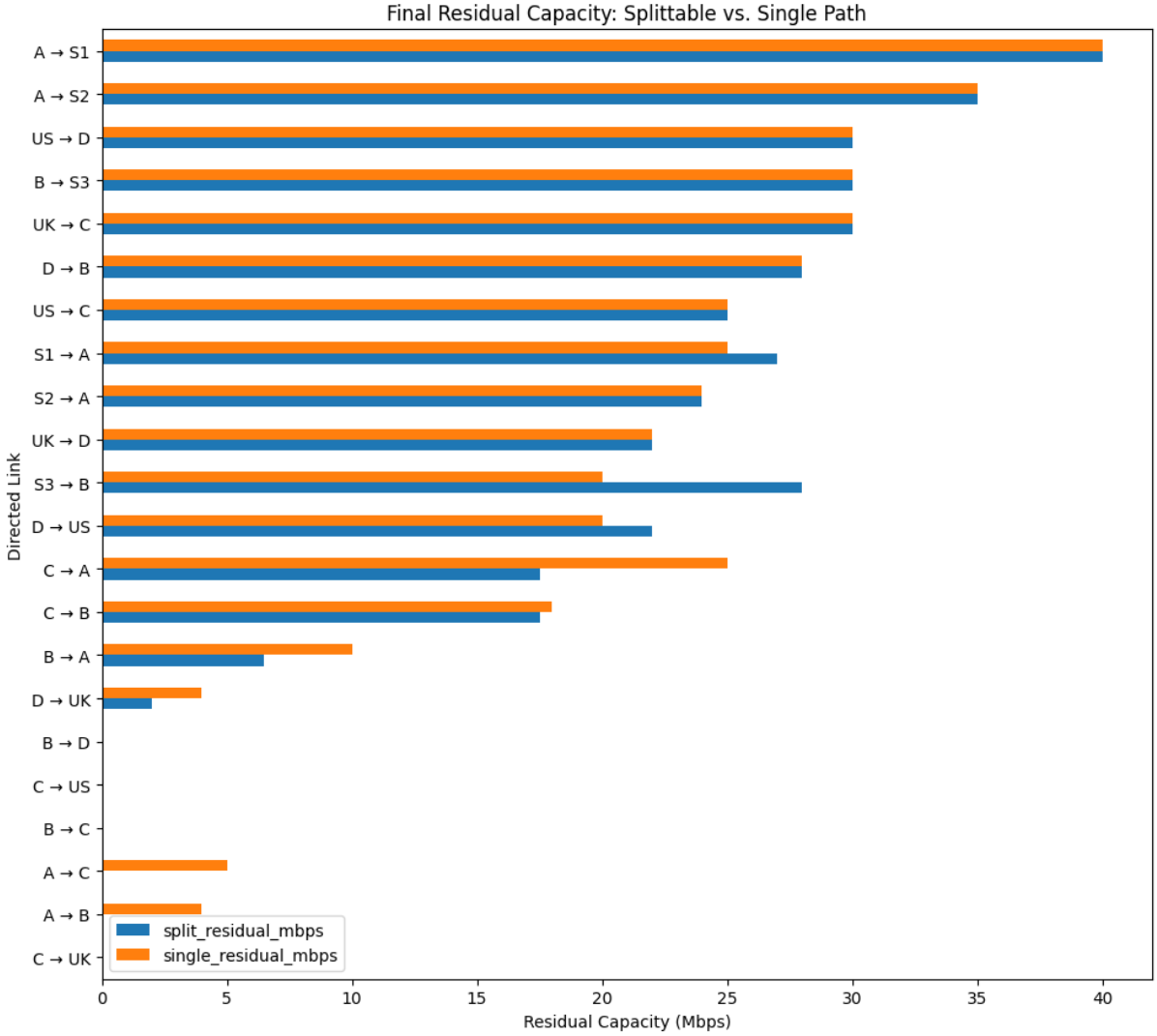

plt.xlabel(
    "Residual Capacity (Mbps)"
)

plt.ylabel(
    "Directed Link"
)

plt.title(
    "Final Residual Capacity: "
    "Splittable vs. Single Path"
)

plt.tight_layout()

plt.show()
```



40.9.1 Visualization Interpretation

Links with bars near:

$$0 \text{ Mbps}$$

have little or no remaining headroom in the corresponding final solution.

Other links may retain substantial unused capacity.

However, spare capacity can become **stranded** when there is no complete source-to-destination route that can use it without encountering another constrained link.

For example, spare capacity on one internal link does not help if:

- the next downstream link is saturated;
- the correct destination exit is already full;
- source-entry capacity or route structure prevents the flow from reaching it.

40.9.5 Business Impact

This distinction is critical for capacity planning.

A network can contain substantial total unused bandwidth while still rejecting demand.

Therefore:

$$\text{Total Spare Link Capacity} \neq \text{Additional Deliverable Throughput}.$$

End-to-end topology matters.

This is precisely why Section 41 measures the marginal value of each link through re-optimization rather than relying only on utilization or headroom.

40.10 Persistent Pressure Does Not Equal a True Bottleneck

Section 40 identifies links that are:

- highly utilized;
- saturated;
- repeatedly saturated;

- low in residual headroom.

These are useful infrastructure-pressure indicators.

However, they do not prove that additional capacity on a link will improve the network objective.

A saturated link may have zero marginal throughput value if another part of the end-to-end path remains more restrictive.

Therefore:

$$\boxed{\text{Saturated Link} \neq \text{True Throughput Bottleneck}}$$

A stronger definition of a throughput bottleneck is:

> **A link whose additional capacity causes the optimized maximum delivered throughput to increase.**

Section 41 tests exactly that definition.

40.10.1 Business Impact

This distinction prevents a common infrastructure-planning mistake:

> **expanding whichever link is currently at 100% utilization.**

A saturated link may be an important operating link without being the best place to invest.

The next section therefore evaluates capacity using direct marginal value rather than visual congestion alone.

40.11 Detailed Section Summary

Section 40 compared the physical network across four optimized operating states:

1. Split Stage 1;
2. Split Stage 2;
3. Single Stage 1;
4. Single Stage 2.

1. The Physical Network Is Held Constant

All four solutions operate on the same:

- 22 directed links;
- capacities;
- latency assumptions.

Therefore, observed utilization differences are caused by routing and admission decisions rather than changes in infrastructure.

2. Saturation Varies by Optimization State

Expected saturated-link counts are approximately:

- Split Stage 1: 3;
- Split Stage 2: 6;
- Single Stage 1: 5;
- Single Stage 2: 4.

The Stage 2 latency objective therefore materially changes infrastructure use.

3. Several Links Experience Persistent Pressure

The strongest recurring pressure appears on:

- B → D;
- C → UK;
- C → US.

These links are expected to be saturated in all four optimized states.

B → C is also highly important and is expected to be saturated in three of the four states.

4. Other Links Are Routing-Policy Dependent

Internal links such as:

- A → B;
- A → C;

change substantially depending on the selected routing formulation and objective.

5. Spare Capacity Can Be Stranded

A link may have substantial residual bandwidth while the network still rejects traffic.

This occurs because deliverability depends on an entire feasible source-to-destination path.

Therefore:

$$\text{Unused Link Capacity} \neq \text{Usable End-to-End Capacity}.$$

6. Saturation Is Not Sufficient to Identify Investment Priority

A saturated link may still have zero marginal throughput value.

Therefore:

$$\boxed{\text{Saturation} \neq \text{Marginal Capacity Value}}$$

The next section directly tests each link by adding capacity and re-solving the optimization problem.

40.12 Connection to Section 41

Section 40 identified where the optimized network appears constrained.

Section 41 now asks the stronger question:

> **Which links actually limit end-to-end throughput?**

For every directed link e, I will:

1. increase its capacity by:

$$1 \text{ Mbps};$$

2. keep all other capacities unchanged;
3. re-solve Stage 1;
4. measure the change in maximum delivered throughput.

This creates a finite-difference estimate of marginal capacity value:

$$MV_e = T^*(C_e + 1) - T^*(C_e).$$

A positive value means that one additional Mbps of capacity on that directed link

creates additional deliverable throughput.

This directly distinguishes:

- merely saturated links;
- true throughput bottlenecks.

41.1 Purpose and Definition of a True Bottleneck

41.1.1 MARKDOWN CELL — Purpose and Context

41. True Bottleneck Identification Through Marginal Capacity Analysis

41.1 Purpose and Definition of a True Bottleneck

Section 40 identified directed links that are:

- highly utilized;
- saturated;
- repeatedly saturated across optimized states;
- low in residual capacity.

Those measures describe **network pressure**.

However, they do not directly measure whether adding capacity to a link would increase the amount of traffic that can actually reach the correct destination.

Section 41 therefore introduces a stronger bottleneck test.

For directed link e, let:

$$C_e$$

represent its baseline capacity.

Let:

$$T^*(C)$$

represent the maximum Stage 1 throughput under the baseline network capacity vector.

The experiment increases only the capacity of directed link e by:

$$1 \text{ Mbps}.$$

Therefore:

$$C_e^+ = C_e + 1.$$

The network is then re-optimized.

The marginal capacity value of link e is defined as:

$$\boxed{MV_e = T^*(C_e + 1) - T^*(C_e)}$$

where:

- MV_e = marginal throughput value of additional capacity on link e;
- $T^*(C_e + 1)$ = maximum throughput after increasing link e by 1 Mbps;
- $T^*(C_e)$ = baseline maximum throughput.

If:

$$MV_e > 0,$$

then additional capacity on that link increases end-to-end deliverable throughput.

Under the current operating state, such a link can be considered a **true local throughput bottleneck**.

41.1.2 Interpretation

This analysis tests infrastructure value directly.

Instead of inferring importance from:

- utilization;
- saturation;
- topology;

the model asks:

> **If I add one Mbps of capacity to this directed link and change nothing else, does the network actually deliver more traffic?**

That is a stronger bottleneck definition than simply identifying links operating at 100% utilization.

41.1.3 Why Saturation Alone Is Not Enough

A directed link can operate at:

$$100\%$$

utilization and still have:

$$MV_e = 0.$$

This occurs when another part of the end-to-end route remains constraining.

For example, additional capacity on an internal link may provide no throughput benefit if:

- a downstream exit link is already saturated;
- another internal link remains the limiting constraint;
- no additional source-to-destination path becomes feasible.

Therefore:

$$\boxed{100\% \text{ Utilization} \not\Rightarrow MV_e > 0}$$

This distinction is central to the infrastructure analysis.

41.1.4 Business Impact

Marginal-capacity analysis converts network optimization into a more useful infrastructure-investment question.

Instead of asking:

> **Which links look congested?**

the organization can ask:

> **Where would the next unit of bandwidth actually create additional delivered service?**

That distinction is directly relevant to:

- upgrade prioritization;
- capital allocation;
- network planning;
- cost-benefit analysis.

41.2 Construct the Directed Capacity Table

T## 41.2 Construct the Directed Capacity Table

The physical network contains:

$$11$$

full-duplex physical links.

Because capacity is modeled independently in each direction, each physical link produces two directed arcs.

Therefore:

$$11 \times 2 = 22$$

directed capacity constraints.

The perturbation experiment operates on these directed arcs individually.

For example:

$$B \to D$$

and:

$$D \to B$$

In [440…

```
# ------------------------------------------------------------
# 41.2 Build directed capacity table from physical links
# ------------------------------------------------------------

forward_capacity_df = (
    physical_link_data[
        [
            "node_1",
            "node_2",
            "capacity_mbps"
        ]
    ]
    .rename(
        columns={
            "node_1": "from_node",
            "node_2": "to_node"
        }
    )
)


reverse_capacity_df = (
```

```
    physical_link_data[
        [
            "node_1",
            "node_2",
            "capacity_mbps"
        ]
    ]
    .rename(
        columns={
            "node_1": "to_node",
            "node_2": "from_node"
        }
    )
)


directed_capacity_df = pd.concat(
    [
        forward_capacity_df,
        reverse_capacity_df
    ],
    ignore_index=True
)


directed_capacity_df[
    "link"
] = (
    directed_capacity_df[
        "from_node"
    ].astype(str)
    + " → "
    + directed_capacity_df[
        "to_node"
    ].astype(str)
)


directed_capacity_df = (
    directed_capacity_df
    .sort_values(
        [
            "from_node",
            "to_node"
        ]
    )
    .reset_index(
        drop=True
    )
)
```

```
directed_capacity_df
```

Out[440…

	from_node	to_node	capacity_mbps	link
0	A	B	20	A → B
1	A	C	25	A → C
2	A	S1	40	A → S1
3	A	S2	35	A → S2
4	B	A	20	B → A
5	B	C	18	B → C
6	B	D	28	B → D
7	B	S3	30	B → S3
8	C	A	25	C → A
9	C	B	18	C → B
10	C	UK	30	C → UK
11	C	US	25	C → US
12	D	B	28	D → B
13	D	UK	22	D → UK
14	D	US	30	D → US
15	S1	A	40	S1 → A
16	S2	A	35	S2 → A
17	S3	B	30	S3 → B
18	UK	C	30	UK → C
19	UK	D	22	UK → D
20	US	C	25	US → C
21	US	D	30	US → D

In [441…

```
# ------------------------------------------------------------
# 41.2 Validate directed capacity table
# ------------------------------------------------------------

print(
    "Number of directed arcs:",
    len(
        directed_capacity_df
    )
)
```

```
print(
    "Unique directed arcs:",
    directed_capacity_df[
        [
            "from_node",
            "to_node"
        ]
    ]
    .drop_duplicates()
    .shape[0]
)


assert len(
    directed_capacity_df
) == 22


assert (
    directed_capacity_df[
        [
            "from_node",
            "to_node"
        ]
    ]
    .drop_duplicates()
    .shape[0]
    == 22
)
```

```
Number of directed arcs: 22
Unique directed arcs: 22
```

41.2.1 Expected Result

The validation should report:

```
Number of directed arcs: 22
Unique directed arcs: 22
```

41.2.2 Business Impact

The perturbation experiment is intentionally directional.

Increasing:

$$B \to D$$

by 1 Mbps does not automatically increase:

$$D \to B.$$

This allows the infrastructure analysis to identify which direction of a physical link actually carries marginal value.

41.3 Prepare the Existing Candidate-Path Table for Perturbation Analysis

The marginal-capacity experiment must use the **same candidate paths that were used in the original optimization models**.

No new paths should be generated for this experiment.

This is essential because the objective is to change only:

$$C_e \to C_e + 1.$$

All other modeling assumptions must remain unchanged, including:

- commodity demand;
- candidate routes;
- source definitions;
- destination definitions;
- path structure;
- exit-correctness rules.

The candidate-path DataFrame created earlier in the notebook may not use a fixed variable name such as:

`candidate_paths_input_df`.

Therefore, this section first identifies the existing candidate-path DataFrame already present in the notebook.

It then standardizes that table for repeated optimization.

In [442…

```
# ------------------------------------------------------------
# 41.3 Prepare candidate-path table for perturbation analysis
# ------------------------------------------------------------
#
# IMPORTANT:
# This analysis must use the SAME candidate paths that were
# used in the original optimization models.
#
# This cell searches the notebook for the existing candidate-
# path DataFrame and validates it before continuing.
```

```
# ------------------------------------------------------------


# ------------------------------------------------------------
# Step 1: Look for likely existing candidate-path DataFrames
# ------------------------------------------------------------

candidate_df_names = [
    "candidate_paths_df",
    "candidate_path_df",
    "all_candidate_paths_df",
    "path_candidates_df",
    "paths_df",
    "candidate_paths"
]


existing_candidate_df_name = None


for df_name in candidate_df_names:

    if (
        df_name in globals()
        and isinstance(
            globals()[df_name],
            pd.DataFrame
        )
    ):

        existing_candidate_df_name = df_name
        break


# ------------------------------------------------------------
# Step 2: If none of the expected names exist,
# search all DataFrames for a likely candidate-path table.
# ------------------------------------------------------------

if existing_candidate_df_name is None:

    for variable_name, variable_value in globals().items():

        if not isinstance(
            variable_value,
            pd.DataFrame
        ):
            continue

        required_basic_columns = {
            "source",
            "stream",
```

```
            "destination"
        }

        if (
            required_basic_columns
            .issubset(
                set(
                    variable_value.columns
                )
            )
            and len(variable_value) >= 20
        ):

            # Prefer tables whose variable name contains "path".
            if "path" in variable_name.lower():

                existing_candidate_df_name = (
                    variable_name
                )

                break


# ------------------------------------------------------------
# Step 3: Stop with a useful message if no table is found.
# ------------------------------------------------------------

if existing_candidate_df_name is None:

    raise NameError(
        "Could not automatically locate the existing "
        "candidate-path DataFrame. Review the candidate "
        "path table created earlier in the notebook."
    )


# ------------------------------------------------------------
# Step 4: Copy the existing candidate-path table.
# ------------------------------------------------------------

perturbation_paths_df = (
    globals()[
        existing_candidate_df_name
    ]
    .copy()
)


print(
    "Using candidate-path DataFrame:",
    existing_candidate_df_name
)
```

```
# ------------------------------------------------------------
# Step 5: Ensure commodity_id exists.
# ------------------------------------------------------------

if (
    "commodity_id"
    not in perturbation_paths_df.columns
):

    perturbation_paths_df[
        "commodity_id"
    ] = (
        perturbation_paths_df[
            "source"
        ].astype(str)
        + "_"
        + perturbation_paths_df[
            "stream"
        ].astype(str)
    )


# ------------------------------------------------------------
# Step 6: Build commodity-demand lookup from demand_df.
# ------------------------------------------------------------

commodity_demand_lookup = (
    demand_df
    .set_index(
        "commodity_id"
    )[
        "demand_mbps"
    ]
    .to_dict()
)


perturbation_paths_df[
    "demand_mbps"
] = (
    perturbation_paths_df[
        "commodity_id"
    ]
    .map(
        commodity_demand_lookup
    )
)


# ------------------------------------------------------------
```

```
# Step 7: Validate every candidate path mapped to demand.
# ------------------------------------------------------------

if (
    perturbation_paths_df[
        "demand_mbps"
    ]
    .isna()
    .any()
):

    missing_commodities = (
        perturbation_paths_df.loc[
            perturbation_paths_df[
                "demand_mbps"
            ].isna(),
            "commodity_id"
        ]
        .drop_duplicates()
        .tolist()
    )

    raise ValueError(
        "Some candidate-path commodities could not be "
        "matched to demand_df: "
        f"{missing_commodities}"
    )


# ------------------------------------------------------------
# Step 8: Reset index and create stable path-row identifier.
# ------------------------------------------------------------

perturbation_paths_df = (
    perturbation_paths_df
    .reset_index(
        drop=True
    )
)


perturbation_paths_df[
    "path_row_id"
] = (
    perturbation_paths_df.index
)


# ------------------------------------------------------------
# Step 9: Report validation information.
# ------------------------------------------------------------
```

```
print(
    "Candidate path records:",
    len(
        perturbation_paths_df
    )
)

print(
    "Unique commodities:",
    perturbation_paths_df[
        "commodity_id"
    ].nunique()
)

print(
    "Columns available:"
)

print(
    list(
        perturbation_paths_df.columns
    )
)


perturbation_paths_df.head()
```

```
Using candidate-path DataFrame: candidate_paths_df
Candidate path records: 68
Unique commodities: 20
Columns available:
['path_id', 'commodity_id', 'source', 'stream', 'destination', 'demand_
mbps', 'path', 'path_edges', 'num_links', 'path_capacity_mbps', 'path_l
atency_ms', 'is_min_latency', 'path_row_id']
```

Out [442...

	path_id	commodity_id	source	stream	destination	demand_mbps	path	p
0	0	S1_UK-L	S1	UK-L	UK	6	S1 -> A -> B -> C -> UK	
1	1	S1_UK-L	S1	UK-L	UK	6	S1 -> A -> B -> D -> UK	
2	2	S1_UK-L	S1	UK-L	UK	6	S1 -> A -> C -> B -> D -> UK	
3	3	S1_UK-L	S1	UK-L	UK	6	S1 -> A -> C -> UK	
4	4	S1_UK-T	S1	UK-T	UK	7	S1 -> A -> B -> C -> UK	

41.3.1 Expected Results

The candidate-path preparation should report approximately:

```
Candidate path records: 68
Unique commodities: 20
```

41.3.2 Interpretation

The candidate-path table has now been identified from the existing notebook state rather than recreated.

This is analytically important because the perturbation experiment should not

modify:

- available paths;
- demand;
- commodity definitions.

The only intended change in each experiment is:

$$C_e \to C_e + 1.$$

Therefore, any change in maximum throughput can be attributed to the capacity perturbation rather than a change in routing options.

41.4 Standardize the Directed Path-Edge Representation

The repeated optimization models must determine which candidate paths consume capacity on each directed link.

The solver therefore needs each candidate route represented explicitly as a sequence of directed edges.

For example:

$$S1 \to A \to C \to UK$$

should be represented internally as:

```
[
    ("S1", "A"),
    ("A", "C"),
    ("C", "UK")
]
```

In [443…

```
# ------------------------------------------------------------
# 41.4 Validate and standardize path-edge representation
# ------------------------------------------------------------
#
# The perturbation solvers need each candidate path represented
# as a list/tuple of directed edges, for example:
#
# [
#     ("S1", "A"),
#     ("A", "C"),
#     ("C", "UK")
# ]
#
# This cell first looks for an existing edge-list column.
```

```
#
# If the notebook instead stores routes as node sequences,
# it converts consecutive node pairs into directed edges.
# ------------------------------------------------------------


# ------------------------------------------------------------
# Step 1: Look for an existing edge-list column.
# ------------------------------------------------------------

possible_edge_columns = [
    "path_edges",
    "route_edges",
    "edges",
    "directed_edges",
    "edge_list"
]


existing_edge_column = None


for column_name in possible_edge_columns:

    if column_name in perturbation_paths_df.columns:

        existing_edge_column = column_name
        break


# ------------------------------------------------------------
# Step 2: If an edge-list column exists,
# standardize its name to path_edges.
# ------------------------------------------------------------

if existing_edge_column is not None:

    if existing_edge_column != "path_edges":

        perturbation_paths_df[
            "path_edges"
        ] = perturbation_paths_df[
            existing_edge_column
        ]

    print(
        "Using existing edge column:",
        existing_edge_column
    )


# ------------------------------------------------------------
```

```
# Step 3: Otherwise, look for a node-sequence column.
# ------------------------------------------------------------

else:

    possible_node_columns = [
        "path_nodes",
        "nodes",
        "route_nodes",
        "path",
        "route"
    ]


    existing_node_column = None


    for column_name in possible_node_columns:

        if column_name in perturbation_paths_df.columns:

            existing_node_column = column_name
            break


    if existing_node_column is None:

        raise ValueError(
            "Could not locate either an edge-list column "
            "or a node-sequence column in perturbation_paths_df. "
            "Review the printed 'Columns available' output "
            "from Section 41.3."
        )


    # ------------------------------------------------------------
    # Convert node sequence to directed edge sequence.
    #
    # Example:
    #
    # ["S1", "A", "C", "UK"]
    #
    # becomes:
    #
    # [
    #     ("S1", "A"),
    #     ("A", "C"),
    #     ("C", "UK")
    # ]
    # ------------------------------------------------------------

    def nodes_to_edges(
```

```
        node_sequence
    ):

        return list(
            zip(
                node_sequence[:-1],
                node_sequence[1:]
            )
        )


    perturbation_paths_df[
        "path_edges"
    ] = (
        perturbation_paths_df[
            existing_node_column
        ]
        .apply(
            nodes_to_edges
        )
    )


    print(
        "Created path_edges from node-sequence column:",
        existing_node_column
    )


# ------------------------------------------------------------
# Step 4: Validate path_edges are present and nonempty.
# ------------------------------------------------------------

missing_path_edges = (
    perturbation_paths_df[
        "path_edges"
    ]
    .isna()
    .sum()
)


empty_or_invalid_path_edges = (
    perturbation_paths_df[
        "path_edges"
    ]
    .apply(
        lambda x:
            (
                not isinstance(
                    x,
                    (list, tuple)
```

```
                )
                or len(x) == 0
            )
    )
    .sum()
)


print(
    "Missing path_edges values:",
    missing_path_edges
)

print(
    "Empty or invalid path_edges values:",
    empty_or_invalid_path_edges
)


assert missing_path_edges == 0

assert empty_or_invalid_path_edges == 0


# ------------------------------------------------------------
# Step 5: Display several paths for manual validation.
# ------------------------------------------------------------

display_columns = [
    "commodity_id",
    "source",
    "stream",
    "destination",
    "path_edges"
]


perturbation_paths_df[
    display_columns
].head(10)
```

```
Using existing edge column: path_edges
Missing path_edges values: 0
Empty or invalid path_edges values: 0
```

Out [443...

	commodity_id	source	stream	destination	path_edges
0	S1_UK-L	S1	UK-L	UK	[(S1, A), (A, B), (B, C), (C, UK)]
1	S1_UK-L	S1	UK-L	UK	[(S1, A), (A, B), (B, D), (D, UK)]
2	S1_UK-L	S1	UK-L	UK	[(S1, A), (A, C), (C, B), (B, D), (D, UK)]
3	S1_UK-L	S1	UK-L	UK	[(S1, A), (A, C), (C, UK)]
4	S1_UK-T	S1	UK-T	UK	[(S1, A), (A, B), (B, C), (C, UK)]
5	S1_UK-T	S1	UK-T	UK	[(S1, A), (A, B), (B, D), (D, UK)]
6	S1_UK-T	S1	UK-T	UK	[(S1, A), (A, C), (C, B), (B, D), (D, UK)]
7	S1_UK-T	S1	UK-T	UK	[(S1, A), (A, C), (C, UK)]
8	S1_US-L	S1	US-L	US	[(S1, A), (A, B), (B, C), (C, US)]
9	S1_US-L	S1	US-L	US	[(S1, A), (A, B), (B, D), (D, US)]

41.4.1 Expected Results

The code should either identify an existing edge column:

```
Using existing edge column: path_edges
Missing path_edges values: 0
Empty or invalid path_edges values: 0
```

41.4.2 Interpretation

The candidate-path table now contains an explicit directed-edge representation for every route.

This allows the optimization model to determine whether candidate path p consumes capacity on directed link e through the condition:

$$e \in p.$$

For example, the capacity constraint for:

$$A \to C$$

includes a route containing:

$$(A, C),$$

but does not include a route containing only:

$$(C, A).$$

This preserves the full-duplex assumption that capacity is independent by direction.

41.4.3 Business Impact

Correct directional accounting is essential for infrastructure recommendations.

A finding that:

$$B \to D$$

has positive marginal capacity value applies specifically to that direction.

It does not imply that additional capacity on:

$$D \to B$$

would provide the same benefit.

This makes the resulting bottleneck recommendations operationally actionable.

41.5 Build the Baseline Capacity Dictionary

In [444…

```
# ------------------------------------------------------------
# 41.5 Build baseline directed-capacity dictionary
# ------------------------------------------------------------

baseline_capacity_dict = {
    (
        row.from_node,
        row.to_node
    ):
        float(
            row.capacity_mbps
        )
    for row in directed_capacity_df.itertuples()
}


print(
    "Directed capacity entries:",
    len(
        baseline_capacity_dict
    )
)
```

```
assert len(
    baseline_capacity_dict
) == 22
```

```
Directed capacity entries: 22
```

41.5.1 Expected Result

The baseline capacity dictionary should contain:

$$22$$

directed capacity entries.

Each key has the form:

$$(u, v)$$

and its value is the corresponding directional capacity in Mbps.

41.6 Validate Every Candidate Path Edge Against the Physical Network

Before constructing the optimization models, every directed edge used by every candidate path should be verified against the physical-network capacity dictionary.

This provides an important schema and topology validation.

For every candidate path edge:

$$(u, v),$$

the following must hold:

$$(u, v) \in E,$$

where E is the set of 22 directed network arcs.

If a path contains an edge that is missing from the capacity dictionary, then the optimization model would not have a corresponding capacity constraint for that edge.

That could silently invalidate the bottleneck analysis.

41.6.1 CODE CELL — Edge-to-Network Validation

In [445…

```
# ------------------------------------------------------------
# 41.6 Validate every path edge against network capacities
# ------------------------------------------------------------

network_edge_set = set(
    baseline_capacity_dict.keys()
)


path_edge_set = set()


for edge_list in perturbation_paths_df[
    "path_edges"
]:

    for edge in edge_list:

        # Convert to tuple in case an edge was stored as a list.
        edge_tuple = tuple(
            edge
        )

        path_edge_set.add(
            edge_tuple
        )


unknown_path_edges = (
    path_edge_set
    -
    network_edge_set
)


print(
    "Unique directed edges used by candidate paths:",
    len(
        path_edge_set
    )
)

print(
    "Directed edges in physical network:",
    len(
        network_edge_set
    )
)

print(
    "Candidate-path edges missing from network:",
    unknown_path_edges
```

```
)


assert len(
    unknown_path_edges
) == 0
```

```
Unique directed edges used by candidate paths: 14
Directed edges in physical network: 22
Candidate-path edges missing from network: set()
```

41.6.3 Expected Results

The final line should report:

```
Candidate-path edges missing from network: set()
This confirms that every directed edge used by every candidate
route has a corresponding capacity in the physical network
model.

The network itself contains: 22 directed arcs.

The candidate paths may use only a subset of those arcs, which
is acceptable.
```

41.6.4 Interpretation

This check ensures that the route representation and the physical network are fully compatible.

Every edge consumed by a candidate path is now guaranteed to have a corresponding capacity constraint.

That is important because the Stage 1 optimization uses:

$$\sum_{p:e\in p} f_p \le C_e.$$

If an edge were missing from the capacity dictionary, the corresponding capacity limitation could be omitted from the model.

41.6.5 Business Impact

This validation prevents false bottleneck conclusions caused by route-data inconsistencies.

Infrastructure recommendations are credible only if every optimized path is

correctly mapped to the physical network assets whose capacity it consumes.

41.7 Reusable Splittable Stage 1 Solver

The splittable Stage 1 perturbation model uses one continuous flow variable for every candidate-path record:

$$f_r \geq 0.$$

For each source-stream commodity c:

$$\sum_{r \in R_c} f_r \leq D_c.$$

For each directed network link e:

$$\sum_{r : e \in r} f_r \leq C_e.$$

The objective is:

$$\max \sum_r f_r.$$

This is the same throughput-maximization structure used in the original splittable Stage 1 analysis.

Only the supplied capacity dictionary changes during perturbation testing.

41.7.2 — Splittable Solver

In [468…

```
# ------------------------------------------------------------
# 41.7 Reusable splittable Stage 1 throughput solver
# ------------------------------------------------------------

def solve_split_stage1_with_capacities(
    path_df,
    capacity_dict,
    solver_msg=False
):
    """
    Solve the splittable Stage 1 maximum-throughput LP
    for a supplied directed-capacity dictionary.

    Parameters
    ----------
    path_df : pandas.DataFrame
```

```
    Candidate-path table containing:
    - path_row_id
    - commodity_id
    - demand_mbps
    - path_edges

capacity_dict : dict
    Dictionary keyed by directed edge tuple:
    (from_node, to_node)

solver_msg : bool
    Whether CBC solver output should be displayed.

Returns
-------
dict
    status
    throughput_mbps
"""

model = pulp.LpProblem(
    "Split_Stage1_Perturbation",
    pulp.LpMaximize
)


# ------------------------------------------------------------
# One continuous flow variable per candidate path.
# ------------------------------------------------------------

flow_vars = {
    int(row.path_row_id):
        pulp.LpVariable(
            f"f_split_{int(row.path_row_id)}",
            lowBound=0,
            cat="Continuous"
        )
    for row in path_df.itertuples()
}


# ------------------------------------------------------------
# Objective:
# maximize total delivered traffic.
# ------------------------------------------------------------

model += pulp.lpSum(
    flow_vars[
        int(row.path_row_id)
    ]
    for row in path_df.itertuples()
)
```

```
# ------------------------------------------------------------
# Commodity demand constraints.
#
# Total flow assigned across all candidate paths for a
# commodity cannot exceed the offered demand.
# ------------------------------------------------------------

for commodity_id, group in (
    path_df
    .groupby(
        "commodity_id"
    )
):

    demand_value = float(
        group[
            "demand_mbps"
        ].iloc[0]
    )

    model += (
        pulp.lpSum(
            flow_vars[
                int(row.path_row_id)
            ]
            for row in group.itertuples()
        )
        <= demand_value
    )


# ------------------------------------------------------------
# Directed-link capacity constraints.
#
# Every candidate path containing edge e consumes capacity
# from that directional link.
# ------------------------------------------------------------

for edge, capacity_value in (
    capacity_dict.items()
):

    model += (
        pulp.lpSum(
            flow_vars[
                int(row.path_row_id)
            ]
            for row in path_df.itertuples()
            if edge in row.path_edges
        )
```

```
            <= float(
                capacity_value
            )
        )


    # ------------------------------------------------------------
    # Solve using CBC.
    # ------------------------------------------------------------

    solver = pulp.PULP_CBC_CMD(
        msg=solver_msg
    )


    model.solve(
        solver
    )


    status = pulp.LpStatus[
        model.status
    ]


    throughput_mbps = (
        float(
            pulp.value(
                model.objective
            )
        )
        if status == "Optimal"
        else None
    )


    return {
        "status":
            status,

        "throughput_mbps":
            throughput_mbps
    }
```

41.7.3 Interpretation

The reusable function reconstructs only the Stage 1 throughput problem.

It does not include the Stage 2 latency objective.

This is intentional.

The infrastructure question being tested is:

> **Does additional capacity increase the amount of traffic that can be delivered?**

Therefore, the relevant objective is the original Stage 1 maximum-throughput objective.

41.8 Reusable Single-Path Stage 1 Solver

The single-path perturbation model requires:

- continuous path-flow variables;
- binary path-selection variables.

For candidate path record r:

$$f_r \geq 0$$

and:

$$y_r \in \{0, 1\}.$$

The flow-selection linking constraint is:

$$f_r \leq D_c y_r.$$

For each commodity:

$$\sum_{r \in R_c} y_r \leq 1.$$

Therefore, at most one candidate route may carry positive flow for a source-stream commodity.

The objective remains:

$$\max \sum_r f_r.$$

Partial delivery is still allowed.

41.8.2 — Single-Path Solver

In [469…

```
# ------------------------------------------------------------
# 41.8 Reusable single-path Stage 1 throughput solver
# ------------------------------------------------------------
```

```
def solve_single_stage1_with_capacities(
    path_df,
    capacity_dict,
    solver_msg=False
):
    """
    Solve the single-path Stage 1 maximum-throughput MILP
    for a supplied directed-capacity dictionary.

    Parameters
    ----------
    path_df : pandas.DataFrame
        Candidate-path table containing:
        - path_row_id
        - commodity_id
        - demand_mbps
        - path_edges

    capacity_dict : dict
        Dictionary keyed by directed edge tuple:
        (from_node, to_node)

    solver_msg : bool
        Whether CBC solver output should be displayed.

    Returns
    -------
    dict
        status
        throughput_mbps
    """

    model = pulp.LpProblem(
        "Single_Stage1_Perturbation",
        pulp.LpMaximize
    )


    # ------------------------------------------------------------
    # Continuous path-flow variables.
    # ------------------------------------------------------------

    flow_vars = {
        int(row.path_row_id):
            pulp.LpVariable(
                f"f_single_{int(row.path_row_id)}",
                lowBound=0,
                cat="Continuous"
            )
        for row in path_df.itertuples()
    }
```

```
# ------------------------------------------------------------
# Binary path-selection variables.
# ------------------------------------------------------------

select_vars = {
    int(row.path_row_id):
        pulp.LpVariable(
            f"y_single_{int(row.path_row_id)}",
            lowBound=0,
            upBound=1,
            cat="Binary"
        )
    for row in path_df.itertuples()
}


# ------------------------------------------------------------
# Objective:
# maximize total delivered traffic.
# ------------------------------------------------------------

model += pulp.lpSum(
    flow_vars[
        int(row.path_row_id)
    ]
    for row in path_df.itertuples()
)


# ------------------------------------------------------------
# Commodity-level constraints.
# ------------------------------------------------------------

for commodity_id, group in (
    path_df
    .groupby(
        "commodity_id"
    )
):

    demand_value = float(
        group[
            "demand_mbps"
        ].iloc[0]
    )


    # ------------------------------------------------------
    # Total delivery cannot exceed offered demand.
    # ------------------------------------------------------
```

```
        model += (
            pulp.lpSum(
                flow_vars[
                    int(row.path_row_id)
                ]
                for row in group.itertuples()
            )
            <= demand_value
        )


        # ------------------------------------------------------
        # At most one path may be selected.
        # ------------------------------------------------------

        model += (
            pulp.lpSum(
                select_vars[
                    int(row.path_row_id)
                ]
                for row in group.itertuples()
            )
            <= 1
        )


        # ------------------------------------------------------
        # Flow-selection linking.
        #
        # Positive flow on a path is only possible when its
        # binary selection variable is active.
        # ------------------------------------------------------

        for row in group.itertuples():

            model += (
                flow_vars[
                    int(row.path_row_id)
                ]
                <=
                demand_value
                *
                select_vars[
                    int(row.path_row_id)
                ]
            )


    # ------------------------------------------------------
    # Directed-link capacity constraints.
    # ------------------------------------------------------
```

```
    for edge, capacity_value in (
        capacity_dict.items()
    ):

        model += (
            pulp.lpSum(
                flow_vars[
                    int(row.path_row_id)
                ]
                for row in path_df.itertuples()
                if edge in row.path_edges
            )
            <= float(
                capacity_value
            )
        )


    # ------------------------------------------------------------
    # Solve using CBC.
    # ------------------------------------------------------------

    solver = pulp.PULP_CBC_CMD(
        msg=solver_msg
    )


    model.solve(
        solver
    )


    status = pulp.LpStatus[
        model.status
    ]


    throughput_mbps = (
        float(
            pulp.value(
                model.objective
            )
        )
        if status == "Optimal"
        else None
    )


    return {
        "status":
            status,
```

```
        "throughput_mbps":
            throughput_mbps
    }
```

41.8.3 Interpretation

The single-path perturbation model uses exactly the same:

- commodity demands;
- candidate routes;
- directed capacities;
- Stage 1 objective;

as the splittable perturbation model.

The structural difference is the binary one-path restriction.

Therefore, any difference in marginal-capacity value between the two models reflects the routing-policy difference rather than inconsistent network assumptions.

41.8.4 Important Modeling Note

The explicit commodity-demand constraint in the single-path formulation is mathematically redundant once the following are both enforced:

$$\sum_{r \in R_c} y_r \leq 1$$

and:

$$f_r \leq D_c y_r.$$

However, the demand constraint is retained here because it:

- makes the model easier to audit;
- states the business rule explicitly;
- provides consistency with the splittable formulation.

Its inclusion does not change the intended feasible solution set.

41.9 Validate the Reusable Solvers Against the Original Baseline

Before running any perturbation analysis, the reusable solvers must reproduce the previously established baseline Stage 1 results.

Expected maximum throughput is:

$$83 \text{ Mbps}$$

for both:

- splittable routing;
- single-path routing.

This is a critical validation step.

If the reusable models do not reproduce the original baseline results, then the perturbation experiment is not equivalent to the earlier optimization and should not be interpreted.

In [471…

```
# ------------------------------------------------------------
# 41.9.1 Ensure PuLP is imported for optimization
# ------------------------------------------------------------

try:

    import pulp

    print(
        "PuLP successfully imported."
    )

    print(
        "PuLP version:",
        pulp.__version__
    )

except ImportError:

    raise ImportError(
        "PuLP is not installed in this notebook environment. "
        "Install it before continuing with Section 41."
    )
```

```
PuLP successfully imported.
PuLP version: 3.3.2
```

In [473…

```
# ------------------------------------------------------------
# 41.9 Validate reusable perturbation solvers
# ------------------------------------------------------------

split_baseline_check = (
    solve_split_stage1_with_capacities(
```

```
        perturbation_paths_df,
        baseline_capacity_dict
    )
)


single_baseline_check = (
    solve_single_stage1_with_capacities(
        perturbation_paths_df,
        baseline_capacity_dict
    )
)


print(
    "Split baseline status:",
    split_baseline_check[
        "status"
    ]
)

print(
    "Split baseline throughput:",
    split_baseline_check[
        "throughput_mbps"
    ],
    "Mbps"
)


print()


print(
    "Single baseline status:",
    single_baseline_check[
        "status"
    ]
)

print(
    "Single baseline throughput:",
    single_baseline_check[
        "throughput_mbps"
    ],
    "Mbps"
)


assert (
    split_baseline_check[
        "status"
```

```
    ]
    == "Optimal"
)


assert (
    single_baseline_check[
        "status"
    ]
    == "Optimal"
)


assert abs(
    split_baseline_check[
        "throughput_mbps"
    ]
    -
    split_stage1_throughput
) < 1e-6


assert abs(
    single_baseline_check[
        "throughput_mbps"
    ]
    -
    single_stage1_throughput
) < 1e-6
```

```
Split baseline status: Optimal
Split baseline throughput: 83.0 Mbps

Single baseline status: Optimal
Single baseline throughput: 83.0 Mbps
```

4.9.2 Results and Interpretation

Both reusable Stage 1 perturbation models successfully reproduce the original baseline optimization results.

For the splittable formulation:

$$T^*_{split} = 83 \text{ Mbps}.$$

For the single-path formulation:

$$T^*_{single} = 83 \text{ Mbps}.$$

Both models also returned an optimization status of:

`Optimal`

Therefore, the repeated-solve framework is consistent with the original Stage 1 optimization.

This is an important validation result because the marginal-capacity experiment should change only one modeling input at a time:

$$C_e \to C_e + 1.$$

The following elements remain unchanged:

- commodity demand;
- candidate paths;
- directed network topology;
- routing rules;
- destination requirements;
- Stage 1 objective.

Because the reconstructed models reproduce the original:

$$83 \text{ Mbps}$$

baseline exactly, any throughput change observed in the perturbation experiment can now be attributed to the directed-link capacity increase rather than to a change in the underlying optimization model.

41.8.3 Business Impact

This validation establishes a reliable baseline for the infrastructure-investment analysis.

The model has demonstrated that it can reproduce the original network operating point before any capacity is changed.

Therefore, when a later experiment shows that adding:

$$1 \text{ Mbps}$$

to a particular directed link increases total delivered throughput, that result can be interpreted as genuine incremental capacity value.

This makes the subsequent bottleneck ranking suitable for:

- infrastructure prioritization;
- capacity planning;

- engineering recommendations;
- cost-benefit analysis.

41.10 Run the +1 Mbps Marginal-Capacity Experiment

For every directed link e:

1. copy the baseline capacity vector;
2. increase only that directed link by:

$$1 \text{ Mbps};$$

3. keep all other capacities unchanged;
4. re-solve the splittable Stage 1 model;
5. re-solve the single-path Stage 1 model;
6. compare each result with its original 83-Mbps baseline.

For the splittable formulation:

$$MV_e^{split} = T^*_{split}(C_e + 1) - T^*_{split}(C).$$

For the single-path formulation:

$$MV_e^{single} = T^*_{single}(C_e + 1) - T^*_{single}(C).$$

All other:

- demands;
- candidate paths;
- topology assumptions;
- destination rules;

remain unchanged.

In [475…

```
# ------------------------------------------------------------
# 41.10 +1 Mbps marginal-capacity experiment
# ------------------------------------------------------------

marginal_results = []


split_baseline_throughput = float(
    split_baseline_check[
        "throughput_mbps"
    ]
```

```
)


single_baseline_throughput = float(
    single_baseline_check[
        "throughput_mbps"
    ]
)


for row in directed_capacity_df.itertuples():

    edge = (
        row.from_node,
        row.to_node
    )


    # ----------------------------------------------------------
    # Copy the complete baseline capacity dictionary.
    # ----------------------------------------------------------

    perturbed_capacity_dict = (
        baseline_capacity_dict.copy()
    )


    # ----------------------------------------------------------
    # Increase exactly ONE directed link by 1 Mbps.
    # ----------------------------------------------------------

    perturbed_capacity_dict[
        edge
    ] = (
        perturbed_capacity_dict[
            edge
        ]
        + 1.0
    )


    # ----------------------------------------------------------
    # Re-solve splittable Stage 1.
    # ----------------------------------------------------------

    split_result = (
        solve_split_stage1_with_capacities(
            perturbation_paths_df,
            perturbed_capacity_dict
        )
    )
```

```
# ------------------------------------------------------------
# Re-solve single-path Stage 1.
# ------------------------------------------------------------

single_result = (
    solve_single_stage1_with_capacities(
        perturbation_paths_df,
        perturbed_capacity_dict
    )
)


# ------------------------------------------------------------
# Store results.
# ------------------------------------------------------------

marginal_results.append(
    {
        "from_node":
            row.from_node,

        "to_node":
            row.to_node,

        "link":
            row.link,

        "baseline_capacity_mbps":
            float(
                row.capacity_mbps
            ),

        "split_status":
            split_result[
                "status"
            ],

        "split_perturbed_throughput_mbps":
            split_result[
                "throughput_mbps"
            ],

        "split_marginal_value_mbps":
            (
                split_result[
                    "throughput_mbps"
                ]
                -
                split_baseline_throughput
                if split_result[
                    "throughput_mbps"
```

```
                    ]
                    is not None
                    else None
                ),

            "single_status":
                single_result[
                    "status"
                ],

            "single_perturbed_throughput_mbps":
                single_result[
                    "throughput_mbps"
                ],

            "single_marginal_value_mbps":
                (
                    single_result[
                        "throughput_mbps"
                    ]
                    -
                    single_baseline_throughput
                    if single_result[
                        "throughput_mbps"
                    ]
                    is not None
                    else None
                )
        }
    )


marginal_capacity_df = pd.DataFrame(
    marginal_results
)


marginal_capacity_df
```

Out [475…

	from_node	to_node	link	baseline_capacity_mbps	split_status	split_pertu
0	A	B	A → B	20.0	Optimal	
1	A	C	A → C	25.0	Optimal	
2	A	S1	A → S1	40.0	Optimal	
			A			

3	A	S2	→ S2	35.0	Optimal
4	B	A	B → A	20.0	Optimal
5	B	C	B → C	18.0	Optimal
6	B	D	B → D	28.0	Optimal
7	B	S3	B → S3	30.0	Optimal
8	C	A	C → A	25.0	Optimal
9	C	B	C → B	18.0	Optimal
10	C	UK	C → UK	30.0	Optimal
11	C	US	C → US	25.0	Optimal
12	D	B	D → B	28.0	Optimal
13	D	UK	D → UK	22.0	Optimal
14	D	US	D → US	30.0	Optimal
15	S1	A	S1 → A	40.0	Optimal
16	S2	A	S2 → A	35.0	Optimal
17	S3	B	S3 → B	30.0	Optimal

18	UK	C	UK → C	30.0	Optimal
19	UK	D	UK → D	22.0	Optimal
20	US	C	US → C	25.0	Optimal
21	US	D	US → D	30.0	Optimal

41.10.1 Interpretation

Each row now answers a direct infrastructure question:

> **If this directed link receives exactly one additional Mbps of capacity and everything else remains unchanged, how much additional traffic can the optimized network deliver?**

A marginal value of:

$$1$$

means that one additional Mbps of link capacity creates one additional Mbps of end-to-end delivered traffic.

A marginal value of:

$$0$$

means the additional link capacity does not improve Stage 1 throughput under the current network state.

41.10.4 MARKDOWN CELL — Business Impact

41.10.2 Business Impact

This converts physical bandwidth into an incremental service metric.

A link with:

$$MV_e = 1$$

has demonstrated immediate throughput value.

A saturated link with:

$$MV_e = 0$$

may still be operationally important, but expanding it by itself does not create additional delivered traffic.

That distinction is important for capital prioritization.

41.11 Validate All Perturbation Solves

In [476…

```
# ------------------------------------------------------------
# 41.11 Validate perturbation solve statuses
# ------------------------------------------------------------

split_all_optimal = bool(
    (
        marginal_capacity_df[
            "split_status"
        ]
        == "Optimal"
    ).all()
)


single_all_optimal = bool(
    (
        marginal_capacity_df[
            "single_status"
        ]
        == "Optimal"
    ).all()
)


print(
    "All split perturbation solves optimal:",
    split_all_optimal
)

print(
    "All single-path perturbation solves optimal:",
    single_all_optimal
)


assert split_all_optimal
assert single_all_optimal
```

```
All split perturbation solves optimal: True
All single-path perturbation solves optimal: True
```

41.11.1 Expected Results

Both checks should return:

```
All split perturbation solves optimal: True
All single-path perturbation solves optimal: True
```

41.11.1 Interpretation

Every directed-link experiment has now been solved on the same analytical basis.

No link is being compared using:

- an infeasible solve;
- a failed solve;
- a different optimization status.

This makes the marginal-value ranking internally consistent.

41.12 Identify True Splittable-Model Bottlenecks

A directed link is classified as having positive local throughput value under the splittable model when:

$$MV_e^{split} > 0.$$

Because numerical solvers may return extremely small floating-point values, a small tolerance is used:

$$MV_e^{split} > 10^{-6}.$$

In [477…

```
# ------------------------------------------------------------
# 41.12 Identify positive marginal-value links
# for splittable Stage 1
# ------------------------------------------------------------

marginal_tolerance = 1e-6


split_true_bottleneck_df = (
    marginal_capacity_df[
        marginal_capacity_df[
            "split_marginal_value_mbps"
```

```
        ]
        > marginal_tolerance
    ]
    .sort_values(
        "split_marginal_value_mbps",
        ascending=False
    )
    .reset_index(
        drop=True
    )
)


split_true_bottleneck_df[
    [
        "link",
        "baseline_capacity_mbps",
        "split_perturbed_throughput_mbps",
        "split_marginal_value_mbps"
    ]
]
```

Out[477…

	link	baseline_capacity_mbps	split_perturbed_throughput_mbps	split_margi
0	B → D	28.0	84.0	
1	C → UK	30.0	84.0	
2	C → US	25.0	84.0	

41.12.1 Expected Results

Based on the earlier splittable Stage 1 network structure, the strongest expected positive marginal-capacity links are:

- B → D;
- C → UK;
- C → US.

Each is expected to provide approximately:

$$1 \text{ Mbps}$$

of additional Stage 1 throughput when its capacity is increased by:

$$1 \text{ Mbps}.$$

Therefore, the expected local relationship is approximately:

$$MV_e^{split} = 1$$

for those links.

The executed optimization output should be treated as the authoritative result.

41.12.2 Interpretation

These links pass the stronger bottleneck test:

$$T^*(C_e + 1) > T^*(C_e).$$

Therefore, additional capacity on these directed arcs immediately allows the splittable Stage 1 model to deliver more traffic.

They are therefore more meaningful throughput-bottleneck candidates than links identified only through high utilization.

41.12.3 Why These Links Matter

B → D

B → D provides access to the alternate D corridor.

The direct B → C corridor cannot carry all traffic that would ideally traverse it.

Additional B → D capacity therefore increases access to:

- D → UK;
- D → US.

C → UK

C → UK is a destination-facing UK link.

When this link is full, additional upstream UK-bound capacity may become unusable because traffic cannot leave the network through that exit.

C → US

C → US performs the corresponding role for US-bound traffic.

Increasing this capacity can directly increase the amount of US traffic that leaves

the network.

41.12.4 Business Impact

Under the current splittable Stage 1 network state, additional capacity on:

- B → D;
- C → UK;
- C → US;

is expected to translate directly into additional delivered throughput.

These links therefore deserve stronger consideration in infrastructure planning than links selected solely because they happen to operate at high utilization.

In [478…

```
# ------------------------------------------------------------
# 41.13 Identify positive marginal-value links
# for single-path Stage 1
# ------------------------------------------------------------

single_true_bottleneck_df = (
    marginal_capacity_df[
        marginal_capacity_df[
            "single_marginal_value_mbps"
        ]
        > marginal_tolerance
    ]
    .sort_values(
        "single_marginal_value_mbps",
        ascending=False
    )
    .reset_index(
        drop=True
    )
)


single_true_bottleneck_df[
    [
        "link",
        "baseline_capacity_mbps",
        "single_perturbed_throughput_mbps",
        "single_marginal_value_mbps"
    ]
]
```

Out [478...

	link	baseline_capacity_mbps	single_perturbed_throughput_mbps	single_ma
0	B → D	28.0	84.0	
1	C → UK	30.0	84.0	
2	C → US	25.0	84.0	

41.13.1 Interpretation

The single-path marginal values should be read directly from the optimization output.

They should not be inferred from the splittable solution.

The reason is that:

$$\mathcal{F}_{single} \subseteq \mathcal{F}_{split}.$$

The one-path restriction changes how additional capacity can be used.

Therefore, a link that has positive marginal value under splittable routing may:

- have the same value under single-path routing;
- have smaller value;
- have zero value.

Likewise, a different link may become important specifically because of the single-path restriction.

41.13.2 Why the Results May Differ

Splittable routing can use additional capacity fractionally across several routes.

Single-path routing requires each commodity to remain on one selected route.

Therefore, the usefulness of an extra:

$$1 \text{ Mbps}$$

may depend on whether that capacity is sufficient to change a discrete route-

selection decision.

The single-path marginal-value landscape can therefore behave differently from the continuous LP result.

41.13.3 Business Impact

Capacity planning should be evaluated under the routing policy that will actually be deployed.

A link that is valuable under splittable routing may not provide the same benefit under a single-path operating policy.

Infrastructure architecture and routing architecture are therefore interdependent decisions.

In [479…

```
# ------------------------------------------------------------
# 41.14 Compare marginal capacity values directly
# ------------------------------------------------------------

marginal_capacity_comparison_df = (
    marginal_capacity_df[
        [
            "link",
            "baseline_capacity_mbps",
            "split_marginal_value_mbps",
            "single_marginal_value_mbps"
        ]
    ]
    .copy()
)


marginal_capacity_comparison_df[
    "maximum_marginal_value_mbps"
] = (
    marginal_capacity_comparison_df[
        [
            "split_marginal_value_mbps",
            "single_marginal_value_mbps"
        ]
    ]
    .max(
        axis=1
    )
)


marginal_capacity_comparison_df[
```

```
    "split_minus_single_marginal_mbps"
] = (
    marginal_capacity_comparison_df[
        "split_marginal_value_mbps"
    ]
    -
    marginal_capacity_comparison_df[
        "single_marginal_value_mbps"
    ]
)


marginal_capacity_comparison_df = (
    marginal_capacity_comparison_df
    .sort_values(
        "maximum_marginal_value_mbps",
        ascending=False
    )
    .reset_index(
        drop=True
    )
)


marginal_capacity_comparison_df
```

Out[479…

	link	baseline_capacity_mbps	split_marginal_value_mbps	single_marginal_v
0	C → US	25.0	1.0	
1	B → D	28.0	1.0	
2	C → UK	30.0	1.0	
3	D → B	28.0	0.0	
4	US → C	25.0	0.0	
5	UK → D	22.0	0.0	
6	UK → C	30.0	0.0	

7	S3 → B	30.0	0.0
8	S2 → A	35.0	0.0
9	S1 → A	40.0	0.0
10	D → US	30.0	0.0
11	D → UK	22.0	0.0
12	A → B	20.0	0.0
13	A → C	25.0	0.0
14	C → B	18.0	0.0
15	C → A	25.0	0.0
16	B → S3	30.0	0.0
17	B → C	18.0	0.0
18	B → A	20.0	0.0
19	A → S2	35.0	0.0
20	A → S1	40.0	0.0
21	US →	30.0	0.0

```
D
```

41.14.1 Interpretation

The comparison table reveals whether each directed link's incremental capacity value is:

- consistent across routing policies;
- stronger under splittable routing;
- stronger under single-path routing;
- zero under both.

The field:

`split_minus_single_marginal_mbps`

provides an additional direct comparison.

A positive value means additional capacity is more immediately useful under the split routing policy.

A negative value means it is more useful under the single-path routing policy.

41.14.2Business Impact

A capacity upgrade that produces positive marginal value under both formulations is particularly robust.

Its usefulness is less dependent on the routing architecture.

A link whose value exists only under one routing policy should instead be evaluated jointly with the organization's routing strategy.

41.15 Saturation vs. True Marginal Bottleneck Classification

Section 40 identified saturated links.

Section 41 can now compare saturation with actual marginal throughput value.

A directed link can fall into four categories.

1. Saturated + Positive Marginal Value

The link is:

- fully utilized;
- throughput-limiting.

This is the strongest immediate capacity candidate.

2. Saturated + Zero Marginal Value

The link is fully utilized, but increasing it alone does not increase total throughput.

Another constraint remains binding.

3. Not Saturated + Zero Marginal Value

The link has spare capacity and additional capacity provides no immediate Stage 1 benefit.

4. Not Saturated + Positive Marginal Value

If this occurs, it should be examined carefully.

It may result from:

- alternative optima;
- discrete routing effects;
- numerical/model structure.

The perturbation result remains the direct measure of incremental value.

In [480…

```
# ------------------------------------------------------------
# 41.15 Split Stage 1:
# saturation versus marginal value
# ------------------------------------------------------------

split_saturation_vs_value_df = (
    split_stage1_link_df[
        [
            "from_node",
            "to_node",
            "utilization_percentage",
            "is_saturated"
        ]
    ]
    .copy()
)
```

```
split_saturation_vs_value_df[
    "link"
] = (
    split_saturation_vs_value_df[
        "from_node"
    ].astype(str)
    + " → "
    + split_saturation_vs_value_df[
        "to_node"
    ].astype(str)
)


split_saturation_vs_value_df = (
    split_saturation_vs_value_df
    .merge(
        marginal_capacity_df[
            [
                "link",
                "split_marginal_value_mbps"
            ]
        ],
        on="link",
        how="left"
    )
)


def classify_saturation_value(
    saturated,
    marginal_value
):

    has_value = (
        marginal_value
        > marginal_tolerance
    )

    if saturated and has_value:

        return (
            "Saturated + Positive Marginal Value"
        )

    elif saturated and not has_value:

        return (
            "Saturated + Zero Marginal Value"
        )

    elif (
```

```
        not saturated
        and has_value
    ):

        return (
            "Not Saturated + Positive Marginal Value"
        )

    else:

        return (
            "Not Saturated + Zero Marginal Value"
        )


split_saturation_vs_value_df[
    "capacity_classification"
] = [
    classify_saturation_value(
        saturated,
        marginal_value
    )
    for saturated, marginal_value
    in zip(
        split_saturation_vs_value_df[
            "is_saturated"
        ],
        split_saturation_vs_value_df[
            "split_marginal_value_mbps"
        ]
    )
]


split_saturation_vs_value_df.sort_values(
    [
        "split_marginal_value_mbps",
        "utilization_percentage"
    ],
    ascending=[
        False,
        False
    ]
)
```

Out[480…

	from_node	to_node	utilization_percentage	is_saturated	link	split_margi
0	C	UK	100.000000	True	C → UK	
1	C	US	100.000000	True	C →	

					US
2	B	D	100.000000	True	B → D
3	A	C	96.000000	False	A → C
4	D	UK	90.909091	False	D → UK
5	B	C	61.111111	False	B → C
6	S1	A	32.500000	False	S1 → A
7	S2	A	31.428571	False	S2 → A
8	D	US	26.666667	False	D → US
9	S3	B	6.666667	False	S3 → B
10	UK	D	0.000000	False	UK → D
11	US	C	0.000000	False	US → C
12	UK	C	0.000000	False	UK → C
13	C	B	0.000000	False	C → B
14	D	B	0.000000	False	D → B
15	A	S1	0.000000	False	A → S1

16	C	A	0.000000	False	C → A
17	B	A	0.000000	False	B → A
18	A	B	0.000000	False	A → B
19	B	S3	0.000000	False	B → S3
20	A	S2	0.000000	False	A → S2
21	US	D	0.000000	False	US → D

In [481…

```
# ------------------------------------------------------------
# 41.15 Single Stage 1:
# saturation versus marginal value
# ------------------------------------------------------------

single_saturation_vs_value_df = (
    single_stage1_link_df[
        [
            "from_node",
            "to_node",
            "utilization_percentage",
            "is_saturated"
        ]
    ]
    .copy()
)


single_saturation_vs_value_df[
    "link"
] = (
    single_saturation_vs_value_df[
        "from_node"
    ].astype(str)
    + " → "
    + single_saturation_vs_value_df[
        "to_node"
    ].astype(str)
)
```

```
single_saturation_vs_value_df = (
    single_saturation_vs_value_df
    .merge(
        marginal_capacity_df[
            [
                "link",
                "single_marginal_value_mbps"
            ]
        ],
        on="link",
        how="left"
    )
)


single_saturation_vs_value_df[
    "capacity_classification"
] = [
    classify_saturation_value(
        saturated,
        marginal_value
    )
    for saturated, marginal_value
    in zip(
        single_saturation_vs_value_df[
            "is_saturated"
        ],
        single_saturation_vs_value_df[
            "single_marginal_value_mbps"
        ]
    )
]


single_saturation_vs_value_df.sort_values(
    [
        "single_marginal_value_mbps",
        "utilization_percentage"
    ],
    ascending=[
        False,
        False
    ]
)
```

Out[481…

	from_node	to_node	utilization_percentage	is_saturated	link	single_marg
1	C	US	100.000000	True	C → US	

2	C	UK	100.000000	True	C → UK
4	B	D	100.000000	True	B → D
0	B	C	100.000000	True	B → C
3	A	C	100.000000	True	A → C
5	D	UK	81.818182	False	D → UK
6	S1	A	37.500000	False	S1 → A
7	D	US	33.333333	False	D → US
8	S3	B	33.333333	False	S3 → B
9	S2	A	31.428571	False	S2 → A
10	C	B	27.777778	False	C → B
11	A	B	20.000000	False	A → B
12	B	A	15.000000	False	B → A
13	A	S1	0.000000	False	A → S1
14	C	A	0.000000	False	C → A
15	D	B	0.000000	False	D →

					B
16	UK	C	0.000000	False	UK → C
17	US	C	0.000000	False	US → C
18	B	S3	0.000000	False	B → S3
19	UK	D	0.000000	False	UK → D
20	A	S2	0.000000	False	A → S2
21	US	D	0.000000	False	US → D

41.15.1 Interpretation

This comparison separates:

> **links that look congested**

from:

> **links whose additional capacity actually improves throughput.**

That is the central analytical distinction of Section 41.

A link should not be classified as a true throughput bottleneck simply because:

$$U_e = 100\%.$$

The stronger condition is:

$$MV_e > 0.$$

41.15.2 Business Impact

This distinction can materially change infrastructure-investment priorities.

Capacity spending should focus first on locations where the optimization demonstrates measurable incremental service value rather than merely high utilization.

41.16 Utilization vs. Marginal Capacity Value Visualization

In [482…

```
# ------------------------------------------------------------
# 41.16 Visualize utilization versus marginal value
# for Splittable Stage 1
# ------------------------------------------------------------

plt.figure(
    figsize=(9, 6)
)


plt.scatter(
    split_saturation_vs_value_df[
        "utilization_percentage"
    ],
    split_saturation_vs_value_df[
        "split_marginal_value_mbps"
    ]
)


for row in (
    split_saturation_vs_value_df
    .itertuples()
):

    if (
        row.split_marginal_value_mbps
        > marginal_tolerance
        or
        row.utilization_percentage
        >= 90
    ):

        plt.annotate(
            row.link,
            (
                row.utilization_percentage,
                row.split_marginal_value_mbps
            )
        )
```

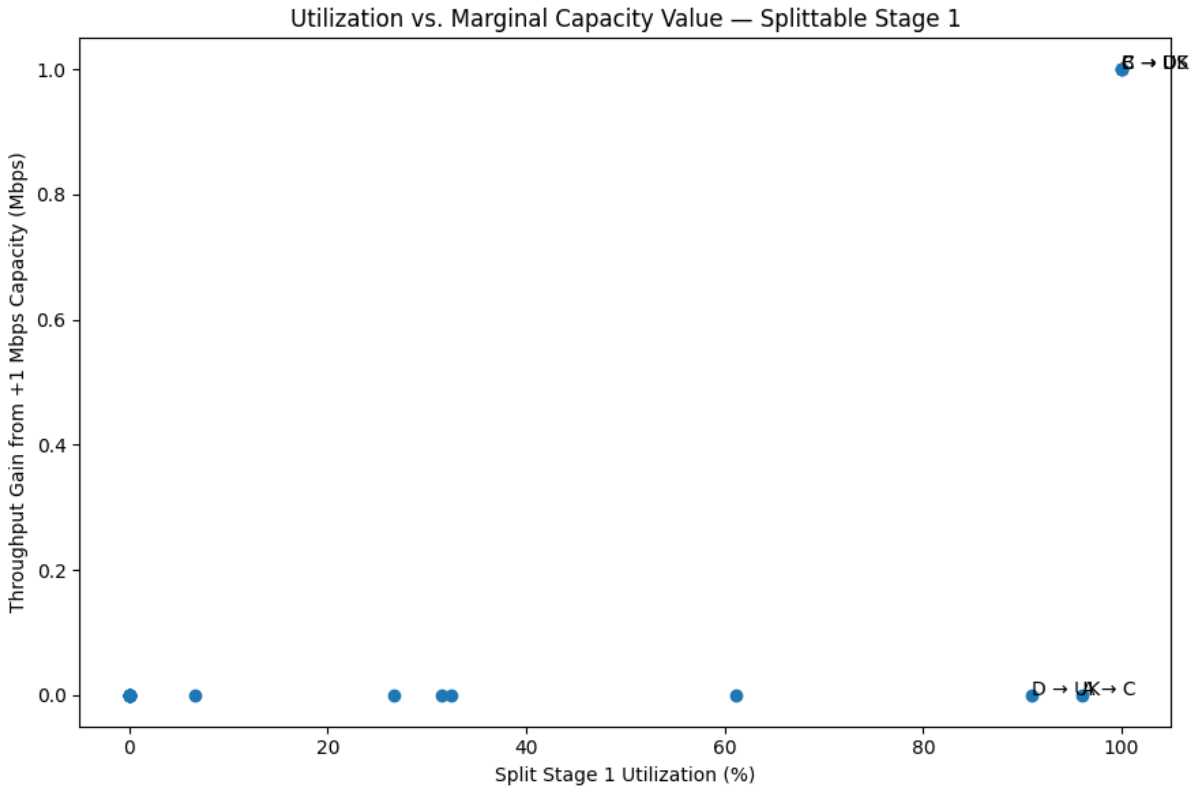
```
plt.xlabel(
    "Split Stage 1 Utilization (%)"
)

plt.ylabel(
    "Throughput Gain from +1 Mbps Capacity (Mbps)"
)

plt.title(
    "Utilization vs. Marginal Capacity Value "
    "— Splittable Stage 1"
)

plt.tight_layout()

plt.show()
```



41.16.1 Visualization Interpretation

The scatter plot directly compares two different infrastructure concepts.

The horizontal axis represents:

$$U_e,$$

the amount of current link utilization.

The vertical axis represents:

$$MV_e,$$

the amount of additional end-to-end throughput created by one additional Mbps of capacity.

Links in the upper-right portion of the figure are particularly important because they are both:

- heavily utilized;
- throughput-sensitive to additional capacity.

Links near:

$$100\%$$

utilization but:

$$MV_e = 0$$

are saturated but do not provide immediate marginal throughput value.

41.16.2 Business Impact

This is a stronger infrastructure-planning visualization than utilization alone.

It helps prevent the common assumption that:

> **the busiest link is automatically the best upgrade target.**

The stronger upgrade candidate is a link where additional bandwidth creates measurable end-to-end service.

41.17 Positive Marginal Capacity Value Visualization

In [483…

```
# ------------------------------------------------------------
# 41.17 Visualize links with positive marginal value
# under either routing policy
# ------------------------------------------------------------

positive_marginal_df = (
    marginal_capacity_comparison_df[
        (
            marginal_capacity_comparison_df[
                "split_marginal_value_mbps"
            ]
            > marginal_tolerance
```

```
        )
        |
        (
            marginal_capacity_comparison_df[
                "single_marginal_value_mbps"
            ]
            > marginal_tolerance
        )
    ]
    .copy()
)


if len(
    positive_marginal_df
) > 0:

    positive_marginal_df.set_index(
        "link"
    )[
        [
            "split_marginal_value_mbps",
            "single_marginal_value_mbps"
        ]
    ].plot(
        kind="bar",
        figsize=(10, 5)
    )

    plt.ylabel(
        "Throughput Gain from +1 Mbps Capacity (Mbps)"
    )

    plt.xlabel(
        "Directed Link"
    )

    plt.title(
        "Positive Marginal Capacity Value "
        "by Routing Formulation"
    )

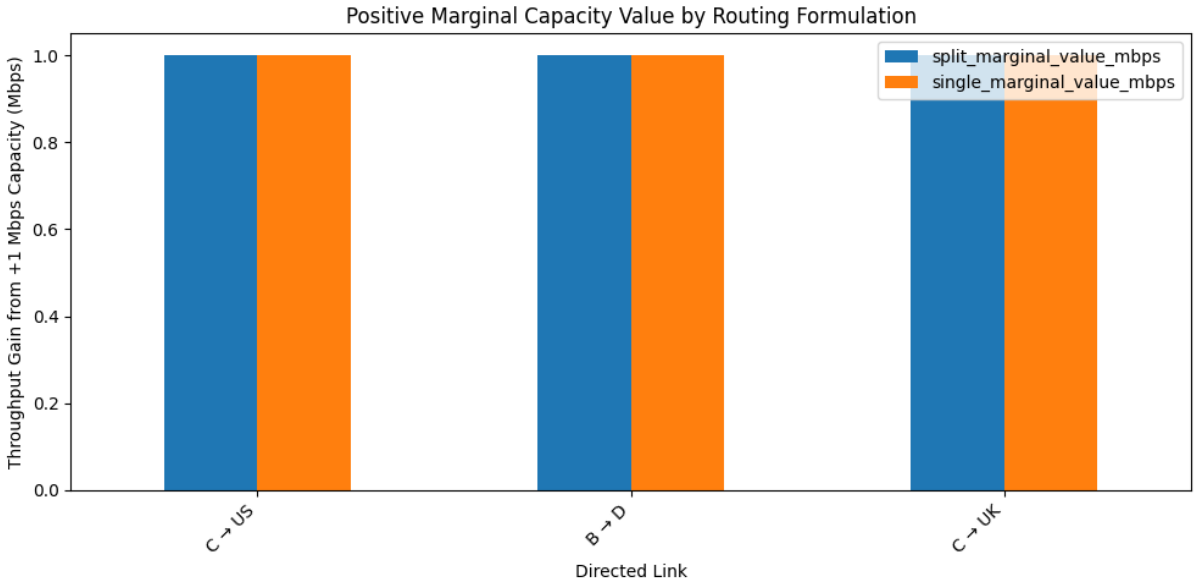
    plt.xticks(
        rotation=45,
        ha="right"
    )

    plt.tight_layout()

    plt.show()

else:
```

```
    print(
        "No directed links have positive "
        "one-Mbps marginal throughput value."
    )
```



41.17.1 Visualization Interpretation

This graph removes links with zero marginal throughput value and focuses only on capacity changes that create additional service under at least one routing policy.

The resulting shortlist is more directly relevant to infrastructure investment than the full utilization table.

Differences between the split and single-path bars indicate that the usefulness of additional physical capacity depends partly on the routing policy.

41.17.2 Business Impact

A useful infrastructure shortlist should prioritize links that combine:

1. positive marginal throughput value;
2. operational importance;
3. reasonable upgrade cost;
4. resilience value;
5. expected future demand.

The optimization provides the first and most direct technical measure: incremental delivered throughput.

41.18 Marginal Capacity Value Is Local, Not a Linear Forecast

The perturbation experiment increases capacity by only:

$$1 \text{ Mbps}.$$

Therefore, the calculated marginal value is a **local finite-difference estimate**.

If:

$$MV_e = 1,$$

the result means:

> Increasing the link from C_e to $C_e + 1$ creates approximately one additional Mbps of optimized throughput.

It does **not** imply that:

$$T^*(C_e + 10) = T^*(C_e) + 10.$$

As one capacity constraint is relaxed, another link may become the new bottleneck.

41.18.1 Why This Matters

Network constraints interact.

A possible sequence is:

1. B → D is expanded;
2. additional traffic moves through D;
3. D → UK or D → US becomes more heavily utilized;
4. an exit link eventually becomes the next binding constraint.

Therefore, the value of capacity is state dependent.

The network should be re-optimized after each meaningful capacity change.

41.18.2 Business Impact

The one-Mbps experiment answers:

> **Where does the next unit of bandwidth have value?**

It does not provide a complete multi-year capital plan.

Larger upgrades should be evaluated using:

- repeated perturbation;
- explicit scenario analysis;
- or a dedicated capacity-expansion optimization model.

41.19 Preliminary Capacity-Priority Table

The marginal-capacity results can now be converted into an initial technical upgrade shortlist.

A directed link is considered an **immediate throughput candidate** when it has:

$$MV_e > 0$$

under at least one routing formulation.

This does not mean the link should automatically be upgraded.

It means that the link has demonstrated measurable local throughput value.

In [484…

```
# ------------------------------------------------------------
# 41.19 Preliminary capacity-priority table
# ------------------------------------------------------------

capacity_priority_df = (
    marginal_capacity_comparison_df[
        (
            marginal_capacity_comparison_df[
                "split_marginal_value_mbps"
            ]
            > marginal_tolerance
        )
        |
        (
            marginal_capacity_comparison_df[
                "single_marginal_value_mbps"
            ]
            > marginal_tolerance
        )
    ]
    .copy()
)


capacity_priority_df[
    "immediate_throughput_candidate"
```

```
] = True


capacity_priority_df = (
    capacity_priority_df
    .sort_values(
        [
            "maximum_marginal_value_mbps",
            "baseline_capacity_mbps"
        ],
        ascending=[
            False,
            True
        ]
    )
    .reset_index(
        drop=True
    )
)


capacity_priority_df
```

Out[484...

	link	baseline_capacity_mbps	split_marginal_value_mbps	single_marginal_va
0	C → US	25.0	1.0	
1	B → D	28.0	1.0	
2	C → UK	30.0	1.0	

41.19.1 Interpretation

The priority table identifies links where additional capacity has demonstrated immediate local throughput value.

For the splittable model, the expected shortlist includes:

- B → D;
- C → UK;
- C → US.

The single-path candidates should be taken directly from the solved perturbation

results rather than assumed.

This table should therefore be treated as a **technical screening tool**, not a final capital recommendation.

41.19.4 MARKDOWN CELL — Business Impact

41.19.2 Business Impact

A final upgrade decision should combine marginal throughput value with:

- upgrade cost;
- cost per Mbps;
- implementation difficulty;
- expected demand growth;
- outage risk;
- resilience benefit;
- strategic importance.

A useful future economic measure is:

$$\text{Cost per Added Delivered Mbps} = \frac{\text{Upgrade Cost}}{\text{Incremental Optimized Throughput}}.$$

This would connect the network optimization directly to investment economics.

41.20 Key Difference Between Utilization and Marginal Capacity Value

The central infrastructure insight from Sections 40 and 41 is:

$$\boxed{\text{Utilization describes current use}}$$

while:

$$\boxed{\text{Marginal value describes incremental usefulness}}$$

These are different concepts.

Utilization

Answers:

> **How much of this link is currently being used?**

Marginal Capacity Value

Answers:

> **How much additional end-to-end service would one additional Mbps of this link create?**

A link can therefore be:

- highly utilized and valuable to expand;
- highly utilized but not valuable to expand alone;
- lightly utilized and irrelevant to current throughput;
- useful only under a particular routing policy.

41.20.1 ### Business Impact

Utilization dashboards are useful for day-to-day operations.

Marginal-capacity analysis is more useful for answering:

> **Where should the next dollar of network capacity be invested?**

The two metrics should therefore be used together rather than treated as substitutes.

41.21 Detailed Section Summary

Section 41 replaces utilization-based bottleneck inference with a direct perturbation test.

1. Marginal Capacity Value

For every directed link:

$$MV_e = T^*(C_e + 1) - T^*(C_e).$$

A positive value means one additional Mbps of directed-link capacity creates additional end-to-end delivered traffic.

2. The Original Candidate Paths Are Preserved

The perturbation analysis reuses the same candidate-path table used by the original optimization.

This ensures that the only intended experimental change is:

$$C_e \to C_e + 1.$$

3. Path Edges Are Standardized

Every candidate route is represented explicitly as a sequence of directed edges.

For example:

$$S1 \to A \to C \to UK$$

becomes:

```
[
    ("S1", "A"),
    ("A", "C"),
    ("C", "UK")
]
```

41.22 Business Recommendation and Transition to Section 42

41.22 Business Recommendation

The recommended infrastructure-planning sequence is:

Step 1 — Identify Persistent Pressure

Use the cross-state utilization analysis from Section 40 to identify links that repeatedly experience high utilization.

Step 2 — Test Marginal Throughput Value

Increase each directed link by a small amount and re-solve the network optimization.

Step 3 — Prioritize Positive-Value Links

Links satisfying:

$$MV_e > 0$$

should receive stronger technical priority than links that are merely saturated.

Step 4 — Incorporate Economic and Operational Information

For each candidate expansion, evaluate:

- upgrade cost;
- cost per Mbps;
- implementation difficulty;
- expected future demand;
- resilience benefit;
- operational risk.

Step 5 — Re-Optimize Larger Capacity Packages

Do not assume that a one-Mbps marginal value continues indefinitely.

As capacity changes, the identity of the bottleneck can change.

Therefore, larger infrastructure decisions should be explicitly re-optimized.

41.23 Connection to Section 42

Sections 40 and 41 now provide two complementary infrastructure signals.

Section 40

Identified:

> **Where does the network experience recurring utilization pressure?**

Section 41

Identified:

> **Where does additional capacity actually increase end-to-end throughput?**

Section 42 can now combine those findings into a physical bottleneck and infrastructure-priority view.

The next analysis will:

- visualize the highest-value links on the network topology;
- distinguish internal-corridor constraints from destination-facing constraints;
- translate marginal capacity value into practical engineering recommendations.

This creates the bridge from mathematical optimization results to network-infrastructure decision support.

42. Capacity Investment Prioritization and Physical Bottleneck Interpretation

42.1 Purpose, Context, and Connection to Section 41

Sections 40 and 41 examined the physical network from two complementary perspectives.

Section 40 — Infrastructure Pressure

Section 40 identified links that were:

- highly utilized;
- repeatedly saturated;
- low in residual headroom;
- sensitive to routing policy.

That analysis answered:

> **Where does the network appear constrained?**

Section 41 — Marginal Capacity Value

Section 41 strengthened that analysis by increasing each directed link by:

$$1\ \mathrm{Mbps}$$

and re-solving the Stage 1 optimization.

For directed link e, marginal capacity value was defined as:

$$MV_e = T^*(C_e + 1) - T^*(C_e).$$

That analysis answered:

> **Where does additional capacity actually create more delivered traffic?**

Section 42 now combines those two views.

The objective is to translate the optimization results into a practical infrastructure-planning framework by identifying:

- persistent structural constraints;
- destination-facing bottlenecks;
- internal corridor bottlenecks;
- routing-policy-dependent constraints;
- links whose utilization is high but marginal value is zero;
- links that should receive stronger consideration for capacity expansion.

The section therefore moves from:

> **optimization diagnostics**

to:

> **engineering and investment interpretation.**

41.1.2 Interpretation

Sections 40 and 41 intentionally answer different questions.

A link may be:

- highly utilized but have no immediate marginal throughput value;
- moderately utilized but strategically important under another network state;
- saturated and have positive marginal value;
- important only under one routing formulation.

Therefore, infrastructure decisions should not rely on a single metric.

Section 42 combines:

1. utilization pressure;
2. saturation persistence;
3. residual capacity;
4. marginal throughput value;
5. network position.

This produces a more complete view of physical-network importance.

41.1.3 Business Impact

The purpose of the optimization model is not merely to produce routes.

It can also support decisions such as:

- where additional bandwidth should be purchased;
- which links deserve engineering attention;
- where redundancy may have strategic value;
- which apparent bottlenecks should not receive immediate investment.

This transforms the model into an infrastructure decision-support tool.

42.2.2 Merge Infrastructure and Marginal-Value Results

In [487…

```
# ------------------------------------------------------------
# 42.2 Build unified capacity–priority table
# ------------------------------------------------------------

capacity_investment_df = (
    cross_state_link_metrics_df[
        [
            "link",
            "from_node",
            "to_node",
            "capacity_mbps",
            "average_utilization_pct",
            "maximum_utilization_pct",
            "minimum_utilization_pct",
            "average_residual_mbps",
            "minimum_residual_mbps",
            "saturation_count",
            "saturation_frequency_pct",
            "pressure_classification"
        ]
    ]
    .merge(
        marginal_capacity_comparison_df[
            [
                "link",
                "split_marginal_value_mbps",
                "single_marginal_value_mbps",
                "maximum_marginal_value_mbps"
            ]
        ],
        on="link",
```

```
        how="left"
    )
)


capacity_investment_df[
    "has_positive_marginal_value"
] = (
    capacity_investment_df[
        "maximum_marginal_value_mbps"
    ]
    > marginal_tolerance
)


capacity_investment_df = (
    capacity_investment_df
    .sort_values(
        [
            "has_positive_marginal_value",
            "maximum_marginal_value_mbps",
            "saturation_count",
            "maximum_utilization_pct"
        ],
        ascending=[
            False,
            False,
            False,
            False
        ]
    )
    .reset_index(
        drop=True
    )
)


capacity_investment_df
```

Out [487...

	link	from_node	to_node	capacity_mbps	average_utilization_pct	maximu
0	B → D	B	D	28.0	100.000000	
1	C → UK	C	UK	30.0	100.000000	
2	C → US	C	US	25.0	100.000000	
	B					

3	→ C	B	C	18.0	90.277778
4	A → C	A	C	25.0	94.000000
5	A → B	A	B	20.0	50.000000
6	D → UK	D	UK	22.0	86.363636
7	B → A	B	A	20.0	33.125000
8	S1 → A	S1	A	40.0	35.000000
9	D → US	D	US	30.0	30.000000
10	S3 → B	S3	B	30.0	20.000000
11	S2 → A	S2	A	35.0	31.428571
12	C → A	C	A	25.0	7.500000
13	C → B	C	B	18.0	7.638889
14	A → S1	A	S1	40.0	0.000000
15	A → S2	A	S2	35.0	0.000000
16	B → S3	B	S3	30.0	0.000000
17	D → B	D	B	28.0	0.000000

18	UK → C	UK	C	30.0	0.000000
19	UK → D	UK	D	22.0	0.000000
20	US → C	US	C	25.0	0.000000
21	US → D	US	D	30.0	0.000000

42.2.3 Interpretation

The combined table separates two concepts that are often incorrectly treated as equivalent.

Network Pressure

Measured using:

- utilization;
- saturation frequency;
- residual headroom.

Incremental Capacity Value

Measured using:

$$MV_e = T^*(C_e + 1) - T^*(C_e).$$

A link with both:

- recurring high utilization;
- positive marginal throughput value;

is a stronger expansion candidate than a link with high utilization alone.

42.2.4 Business Impact

This table can serve as the technical foundation for a capacity-planning scorecard.

Additional business information could later be added, including:

- expansion cost;
- implementation lead time;
- outage risk;
- vendor constraints;
- expected demand growth;
- resilience benefit.

The optimization results therefore become one input into a broader investment decision.

42.3 Infrastructure Investment Classification

To translate the numerical results into a practical engineering view, each directed link can be classified according to both utilization pressure and marginal throughput value.

Priority 1 — Immediate Throughput Candidate

A link has:

$$MV_e > 0.$$

Additional capacity has already demonstrated measurable throughput value.

Priority 2 — Persistent Pressure, No Immediate Marginal Gain

A link has:

- recurring high utilization or repeated saturation;
- zero local one-Mbps throughput value.

This means the link is operationally important, but expanding it alone does not currently increase throughput.

Priority 3 — Policy-Dependent Constraint

A link becomes highly utilized under selected routing states but is not consistently constrained.

Priority 4 — Available / Low-Pressure Capacity

A link remains relatively underutilized and has zero local marginal throughput value.

These categories are engineering interpretations, not automatic capital-approval rules.

42.3.2 Classification Function

In [489…

```
# ------------------------------------------------------------
# 42.3 Classify infrastructure investment priority
# ------------------------------------------------------------

def classify_investment_priority(row):

    # Highest priority:
    # adding capacity produces measurable throughput gain.
    if (
        row[
            "maximum_marginal_value_mbps"
        ]
        > marginal_tolerance
    ):
        return "Priority 1 – Immediate Throughput Candidate"

    # Persistent infrastructure pressure but no immediate
    # one-Mbps throughput improvement.
    elif (
        row[
            "saturation_count"
        ]
        >= 3
    ):
        return "Priority 2 – Persistent Pressure / Zero Local Gain"

    # Highly utilized only in selected states.
    elif (
        row[
            "maximum_utilization_pct"
        ]
        >= 90
    ):
        return "Priority 3 – Policy-Dependent Constraint"

    # Everything else currently has lower immediate priority.
    else:
        return "Priority 4 – Available / Lower Pressure Capacity"


capacity_investment_df[
    "investment_priority"
```

```
] = (
    capacity_investment_df
    .apply(
        classify_investment_priority,
        axis=1
    )
)


capacity_investment_df[
    [
        "link",
        "capacity_mbps",
        "maximum_utilization_pct",
        "saturation_count",
        "split_marginal_value_mbps",
        "single_marginal_value_mbps",
        "investment_priority"
    ]
]
```

Out[489...

	link	capacity_mbps	maximum_utilization_pct	saturation_count	split_marg
0	B → D	28.0	100.000000	4	
1	C → UK	30.0	100.000000	4	
2	C → US	25.0	100.000000	4	
3	B → C	18.0	100.000000	3	
4	A → C	25.0	100.000000	2	
5	A → B	20.0	100.000000	1	
6	D → UK	22.0	90.909091	0	
7	B →	20.0	67.500000	0	

	A			
8	S1 → A	40.0	37.500000	0
9	D → US	30.0	33.333333	0
10	S3 → B	30.0	33.333333	0
11	S2 → A	35.0	31.428571	0
12	C → A	25.0	30.000000	0
13	C → B	18.0	27.777778	0
14	A → S1	40.0	0.000000	0
15	A → S2	35.0	0.000000	0
16	B → S3	30.0	0.000000	0
17	D → B	28.0	0.000000	0
18	UK → C	30.0	0.000000	0
19	UK → D	22.0	0.000000	0
20	US → C	25.0	0.000000	0
21	US → D	30.0	0.000000	0

42.3.3 Interpretation

The classification prevents a link from being promoted to the highest investment category simply because it is saturated.

Priority 1 requires direct evidence that additional capacity produces more end-to-end throughput.

Priority 2 identifies an important but different case:

> **The link is repeatedly under pressure, but another network constraint prevents a one-Mbps expansion from producing immediate throughput gain.**

Such links may still matter in a coordinated multi-link expansion plan.

42.3.4 Business Impact

The distinction between Priority 1 and Priority 2 is particularly important for capital allocation.

A Priority 1 upgrade can potentially produce immediate service value.

A Priority 2 upgrade may need to be paired with another upgrade before value is realized.

Therefore:

$$\text{Individual Upgrade Value} \neq \text{Package Upgrade Value}.$$

This suggests a future extension using multi-link capacity-expansion optimization.

42.4 Identify the Highest-Value Capacity Candidates

In [490…

```
# ------------------------------------------------------------
# 42.4 Identify links with positive marginal capacity value
# ------------------------------------------------------------

immediate_capacity_candidates_df = (
    capacity_investment_df[
        capacity_investment_df[
            "maximum_marginal_value_mbps"
```

```
        ]
        > marginal_tolerance
    ]
    .copy()
)


immediate_capacity_candidates_df[
    [
        "link",
        "capacity_mbps",
        "maximum_utilization_pct",
        "saturation_count",
        "split_marginal_value_mbps",
        "single_marginal_value_mbps",
        "maximum_marginal_value_mbps"
    ]
]
```

Out[490…

	link	capacity_mbps	maximum_utilization_pct	saturation_count	split_margi
0	B → D	28.0	100.0	4	
1	C → UK	30.0	100.0	4	
2	C → US	25.0	100.0	4	

42.4.1 Results and Explanation

This table should be treated as the authoritative list of links with positive marginal capacity value.

For the splittable baseline, earlier analysis suggests that the strongest expected candidates include:

- B → D;
- C → UK;
- C → US.

However, the final investment table should use the actual solved results from Section 41, particularly for the single-path formulation.

A positive marginal value means:

$$T^*(C_e + 1) > T^*(C_e).$$

Therefore, additional capacity on that directed arc has demonstrated immediate end-to-end throughput value under at least one routing policy.

42.4.2 Business Impact

These links form the first technical shortlist for capacity expansion.

They should not yet be interpreted as a final investment ranking because the current model does not contain:

- capital cost;
- construction cost;
- implementation time;
- outage risk;
- contractual pricing;
- expected demand growth.

However, the optimization establishes that capacity on these links is operationally useful.

42.5 B → D as an Internal-Corridor Constraint

If the Section 41 perturbation output confirms positive marginal value for:

$$B \to D,$$

the link represents a structurally important **internal-corridor constraint**.

The network has two major routes from the B/C region toward the destination exits.

One corridor moves through:

$$B \to C,$$

while the alternate corridor moves through:

$$B \to D.$$

Node D then provides access to:

- D → UK;
- D → US.

The naive shortest-path analysis initially placed excessive traffic through the C corridor.

Optimized routing instead uses B → D to move substantial traffic away from:

- B → C;
- C → UK;
- C → US.

Therefore, B → D is not merely a busy link.

It provides access to an alternate exit corridor and increases the network's ability to distribute traffic across the topology.

45.5.1 Business Impact

An upgrade to B → D potentially has two forms of value.

42.5.2 Throughput Value

If:

$$MV_{B\to D} > 0,$$

additional capacity immediately increases deliverable traffic.

42.5.3 Structural / Resilience Value

Even beyond the local marginal result, a stronger B → D corridor may provide:

- greater route diversity;
- reduced dependence on B → C;
- additional ability to absorb congestion;
- improved failure tolerance.

The current optimization quantifies the first benefit.

Resilience analysis would be a valuable future extension.

42.6 Interpret C → UK and C → US as Destination-Boundary Constraints

42.6 Destination-Boundary Constraints

The directed links:

$$C \to UK$$

and:

$$C \to US$$

have a different structural role from B → D.

These links sit immediately before the destination nodes.

Therefore, they function as **destination-boundary capacity constraints**.

For traffic using the C corridor, every additional unit of UK-bound traffic must eventually traverse:

$$C \to UK,$$

and every additional unit of US-bound traffic must eventually traverse:

$$C \to US.$$

Once these arcs are saturated, unused upstream capacity cannot create additional delivery through C.

This explains why a network can simultaneously contain:

- unused internal bandwidth;
- undelivered demand.

The final destination-facing capacity still constrains end-to-end service.

42.6.3 Business Impact

Destination-boundary links are particularly important because upstream improvements may have limited value once the boundary itself is full.

This creates an infrastructure-planning principle:

> **Do not expand upstream capacity without verifying that destination-facing capacity can absorb the additional flow.**

Marginal-capacity analysis helps identify when exit-facing upgrades should accompany internal-corridor upgrades.

42.7 Identify Saturated Links with Zero

Immediate Marginal Value

In [492…

```
# ------------------------------------------------------------
# 42.7 Identify saturated / pressured links with zero
# immediate marginal throughput gain
# ------------------------------------------------------------

pressure_without_local_gain_df = (
    capacity_investment_df[
        (
            capacity_investment_df[
                "saturation_count"
            ]
            > 0
        )
        &
        (
            capacity_investment_df[
                "maximum_marginal_value_mbps"
            ]
            <= marginal_tolerance
        )
    ]
    .copy()
)


pressure_without_local_gain_df[
    [
        "link",
        "maximum_utilization_pct",
        "saturation_count",
        "minimum_residual_mbps",
        "maximum_marginal_value_mbps",
        "investment_priority"
    ]
]
```

Out [492...

	link	maximum_utilization_pct	saturation_count	minimum_residual_mbps	m
3	B → C	100.0	3	0.0	
4	A → C	100.0	2	0.0	
5	A → B	100.0	1	0.0	

42.7.1 Interpretation

These links provide one of the most important lessons in the assessment.

They may be:

- saturated in one or more optimized states;
- low in residual headroom;

yet still have:

$$MV_e = 0.$$

That means additional capacity on the link **alone** does not increase maximum Stage 1 throughput.

Possible reasons include:

- another downstream link remains binding;
- another upstream constraint limits additional traffic;
- the one-Mbps increment is insufficient to change a discrete routing decision;
- the link participates in a larger bottleneck set rather than acting independently.

42.7.2 Business Impact

These links should not automatically be ignored.

Instead, they should be interpreted as possible candidates for:

- coordinated upgrades;
- resilience investments;

- future demand growth.

However, they have weaker evidence for immediate stand-alone throughput investment than links with positive marginal value.

42.8 Visualize Capacity Investment Priority

In [493…

```
# ------------------------------------------------------------
# 42.8 Visualize saturation persistence versus
# marginal throughput value
# ------------------------------------------------------------

plt.figure(
    figsize=(10, 6)
)


plt.scatter(
    capacity_investment_df[
        "saturation_frequency_pct"
    ],
    capacity_investment_df[
        "maximum_marginal_value_mbps"
    ]
)


for row in (
    capacity_investment_df
    .itertuples()
):

    if (
        row.maximum_marginal_value_mbps
        > marginal_tolerance
        or
        row.saturation_frequency_pct
        >= 75
    ):

        plt.annotate(
            row.link,
            (
                row.saturation_frequency_pct,
                row.maximum_marginal_value_mbps
            )
        )


plt.xlabel(
```

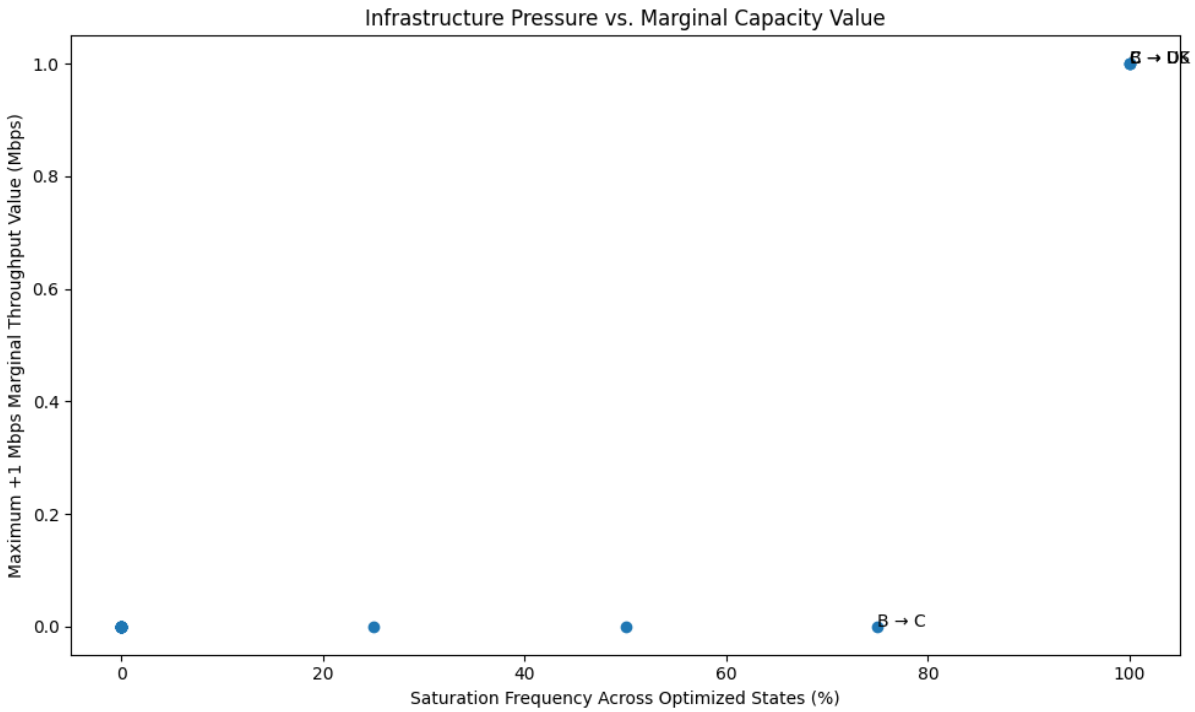
```
    "Saturation Frequency Across Optimized States (%)"
)

plt.ylabel(
    "Maximum +1 Mbps Marginal Throughput Value (Mbps)"
)

plt.title(
    "Infrastructure Pressure vs. Marginal Capacity Value"
)

plt.tight_layout()

plt.show()
```



42.8.1 Visualization Interpretation

The chart combines two infrastructure signals.

The horizontal axis measures how persistently a link experiences capacity pressure.

The vertical axis measures whether one additional Mbps produces more delivered traffic.

The most compelling near-term expansion candidates appear toward the upper-right:

- frequently constrained;

- positive marginal value.

Links toward the lower-right are also important.

They experience persistent pressure but have little immediate stand-alone marginal throughput value.

This suggests that they may belong to a broader interacting bottleneck structure.

42.8.2 Business Impact

This visualization helps separate:

- operational congestion;
- economic usefulness of incremental capacity.

That distinction is valuable when infrastructure budgets are limited.

It directs attention toward links where capacity pressure and incremental service value overlap.

42.9 Physical Network Bottleneck Visualization

Tables provide precision, but the topology is easier to interpret when the investment candidates are placed back onto the physical network.

The network visualization below labels each physical link with:

- capacity;
- maximum directed marginal throughput value.

Because each physical link is full duplex, its two directions may have different marginal values.

For a physical pair (u, v), the visualization uses:

$$MV_{physical} = \max\left(MV_{u\to v}, MV_{v\to u}\right).$$

This is used only for topology visualization.

Directional conclusions should still be taken from the detailed tables.

In [494…

```
# ------------------------------------------------------------
# 42.9 Build physical network graph for bottleneck display
# ------------------------------------------------------------
```

```
import networkx as nx


capacity_network_graph = nx.Graph()


for row in physical_link_data.itertuples():

    capacity_network_graph.add_edge(
        row.node_1,
        row.node_2,
        capacity_mbps=float(
            row.capacity_mbps
        ),
        latency_ms=float(
            row.latency_ms
        )
    )


print(
    "Physical network nodes:",
    capacity_network_graph.number_of_nodes()
)

print(
    "Physical network links:",
    capacity_network_graph.number_of_edges()
)
```

```
Physical network nodes: 9
Physical network links: 11
```

42.9.1 Expected Result

The physical topology should contain:

$$9$$

nodes and:

$$11$$

physical links.

This matches the original synthetic network definition.

42.9.2 Aggregate Directional Marginal Value to Physical Links

In [495…

```
# ------------------------------------------------------------
```

```
# 42.9 Aggregate directed marginal values for topology display
# ------------------------------------------------------------

physical_marginal_records = []


for row in physical_link_data.itertuples():

    forward_link = (
        f"{row.node_1} → {row.node_2}"
    )

    reverse_link = (
        f"{row.node_2} → {row.node_1}"
    )


    forward_matches = (
        marginal_capacity_comparison_df[
            marginal_capacity_comparison_df[
                "link"
            ]
            == forward_link
        ]
    )


    reverse_matches = (
        marginal_capacity_comparison_df[
            marginal_capacity_comparison_df[
                "link"
            ]
            == reverse_link
        ]
    )


    forward_value = (
        float(
            forward_matches[
                "maximum_marginal_value_mbps"
            ].iloc[0]
        )
        if len(
            forward_matches
        ) > 0
        else 0.0
    )


    reverse_value = (
        float(
```

```
            reverse_matches[
                "maximum_marginal_value_mbps"
            ].iloc[0]
        )
        if len(
            reverse_matches
        ) > 0
        else 0.0
    )


    physical_marginal_records.append(
        {
            "node_1":
                row.node_1,

            "node_2":
                row.node_2,

            "capacity_mbps":
                float(
                    row.capacity_mbps
                ),

            "forward_marginal_value":
                forward_value,

            "reverse_marginal_value":
                reverse_value,

            "maximum_physical_marginal_value":
                max(
                    forward_value,
                    reverse_value
                )
        }
    )


physical_marginal_df = pd.DataFrame(
    physical_marginal_records
)


physical_marginal_df
```

Out [495…

	node_1	node_2	capacity_mbps	forward_marginal_value	reverse_marginal_
0	S1	A	40.0	0.0	
1	S2	A	35.0	0.0	
2	S3	B	30.0	0.0	
3	A	B	20.0	0.0	
4	A	C	25.0	0.0	
5	B	C	18.0	0.0	
6	B	D	28.0	1.0	
7	C	UK	30.0	1.0	
8	C	US	25.0	1.0	
9	D	UK	22.0	0.0	
10	D	US	30.0	0.0	

42.9.3 Network Visualization

In [497…

```
# ------------------------------------------------------------
# 42.9 Visualize physical network with capacity and
# marginal-capacity value labels
# ------------------------------------------------------------

position = nx.spring_layout(
    capacity_network_graph,
    seed=42
)


plt.figure(
    figsize=(12, 8)
)


nx.draw_networkx_nodes(
    capacity_network_graph,
    position,
    node_size=1800
)


nx.draw_networkx_labels(
    capacity_network_graph,
    position,
```

```
    font_size=10
)


nx.draw_networkx_edges(
    capacity_network_graph,
    position,
    width=2
)


physical_edge_labels = {}


for row in (
    physical_marginal_df
    .itertuples()
):

    physical_edge_labels[
        (
            row.node_1,
            row.node_2
        )
    ] = (
        f"C={row.capacity_mbps:.0f}\n"
        f"Max MV={row.maximum_physical_marginal_value:.1f}"
    )

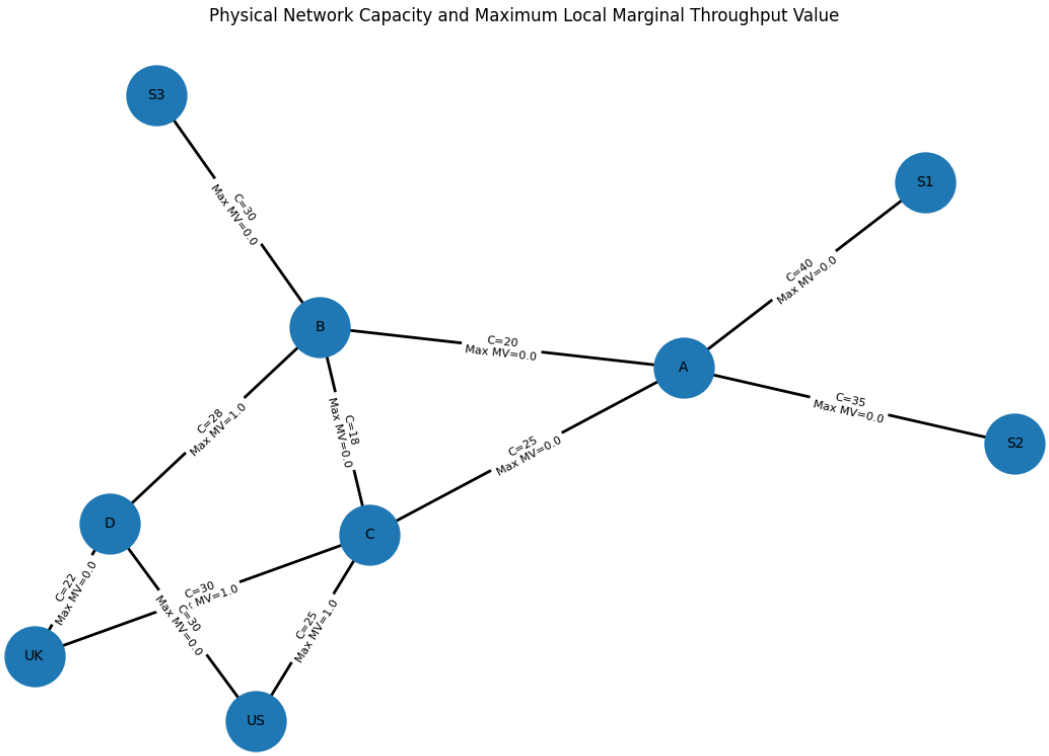

nx.draw_networkx_edge_labels(
    capacity_network_graph,
    position,
    edge_labels=physical_edge_labels,
    font_size=8
)


plt.title(
    "Physical Network Capacity and "
    "Maximum Local Marginal Throughput Value"
)

plt.axis(
    "off"
)

plt.tight_layout()

plt.show()
```



42.9.4 Visualization Interpretation

The network diagram places the marginal-capacity results back into their physical context.

This is particularly useful for understanding the difference between:

42.9.5 Internal-Corridor Constraints

For example:

$$B \to D.$$

These links influence how traffic moves through the interior of the network.

42.9.6 Destination-Boundary Constraints

For example:

$$C \to UK$$

and:

$$C \to US.$$

These links determine how much traffic can ultimately leave the network through a destination boundary.

The topology therefore helps explain why a collection of local capacity constraints can interact to create the overall:

$$83 \text{ Mbps}$$

throughput ceiling.

42.10 Local Marginal Value vs. Multi-Link Investment

The one-Mbps perturbation experiment is intentionally local.

It answers:

> **Where does the next Mbps of bandwidth have immediate value?**

However, infrastructure investments are often larger and may involve several links simultaneously.

Suppose two links belong to the same constrained corridor.

It is possible that:

$$MV_a = 0$$

and:

$$MV_b = 0,$$

when upgraded individually,

but:

$$T^*(C_a + 1, C_b + 1) > T^*(C),$$

when both are upgraded together.

This occurs because bottlenecks can exist as interacting constraint sets.

Therefore:

$$\boxed{\text{Zero Individual Marginal Value} \not\Rightarrow \text{Zero Joint Upgrade Value}}$$

42.10.1 Business Impact

The current analysis is appropriate for identifying immediate local opportunities.

For large capital planning, a future version of the model could introduce explicit capacity-investment decision variables.

For example:

$$\Delta C_e \geq 0$$

could represent purchased additional bandwidth.

A future optimization could then balance:

$$\text{Service Value} - \text{Capacity Cost}.$$

That would turn the current marginal analysis into a full capacity-expansion model.

42.11 Capacity Investment Should Consider Cost

The current model measures the **technical value** of additional capacity.

It does not contain the cost of expanding each link.

If upgrade cost data became available, a useful metric would be:

$$\text{Cost per Added Delivered Mbps} = \frac{\text{Incremental Upgrade Cost}}{\text{Incremental Delivered Throughput}}.$$

For example, two links may each produce:

$$1 \text{ Mbps}$$

of additional throughput.

If one upgrade costs substantially less, it may be the economically preferable investment.

Therefore, marginal throughput value is a necessary but not sufficient input to capital planning.

42.11.1 Business Impact

A production investment framework should combine:

- marginal throughput value;
- latency value;
- resilience value;

- implementation cost;
- upgrade lead time;
- expected demand growth.

The current optimization provides the technical foundation onto which those economic inputs can be added.

42.12 Detailed Section Summary

Section 42 translates the optimization outputs into an infrastructure-investment framework.

1. Utilization and Marginal Value Are Combined

A link is evaluated using both:

- observed infrastructure pressure;
- incremental throughput value.

2. Positive Marginal Value Provides the Strongest Immediate Technical Signal

A link satisfying:

$$MV_e > 0$$

has demonstrated that additional bandwidth creates additional end-to-end service.

3. B → D Represents an Internal-Corridor Constraint

If confirmed by the executed Section 41 results, B → D is important because it provides access to the alternate D corridor and reduces dependence on the B → C path.

4. C → UK and C → US Represent Destination-Boundary Constraints

These links limit how much traffic can leave the network through the C-based destination exits.

5. Saturated Links with Zero Marginal Value Are Still Informative

They may indicate:

- interacting bottlenecks;
- policy-dependent pressure;
- resilience concerns.

However, they have weaker evidence for immediate stand-alone throughput investment.

6. Marginal Value Is Local

The one-Mbps perturbation should not be extrapolated linearly to large upgrades.

As capacity changes, the bottleneck structure can change.

7. Final Investment Decisions Require Economic Data

A complete capital plan should also include:

- upgrade cost;
- implementation difficulty;
- future demand;
- resilience value.

Overall Conclusion

The strongest capacity candidates are not simply the most utilized links.

They are links where:

$$\boxed{\text{Infrastructure Pressure} + \text{Positive Marginal Value}}$$

provide evidence that additional capacity is both needed and useful.

42.13 Connection to Section 43

Sections 39–42 have analyzed the baseline network in considerable depth.

However, a routing policy that performs well under one network state may behave differently when:

- demand decreases;
- demand increases;
- internal capacity degrades;

- destination exits become constrained;
- usable capacity becomes fragmented.

Section 43 therefore moves from:

> **baseline optimization**

to:

> **scenario and stress testing.**

The next question is:

> **How robust are the routing conclusions when the network operating conditions change?**

This is particularly important for evaluating whether the additional flexibility of splittable routing becomes more valuable under stressed or fragmented conditions.

43. Stress Testing, Demand Scenarios, and Routing Robustness

43.1 Purpose and Analytical Framework

The baseline network produced an important result:

$$T^*_{split} = T^*_{single} = 83 \text{ Mbps}.$$

Therefore, splittable routing provides no baseline Stage 1 throughput advantage.

However, this does not prove that splitting has no throughput value under other network states.

The value of routing flexibility may change when:

- demand increases;
- demand decreases;
- internal links degrade;
- destination capacity becomes constrained;
- residual capacity becomes fragmented across multiple paths.

Section 43 therefore performs scenario analysis.

For each scenario, the notebook re-solves:

1. Splittable Stage 1;
2. Single-Path Stage 1.

The primary comparison metrics are:

- offered demand;
- split maximum throughput;
- single-path maximum throughput;
- split delivery percentage;
- single-path delivery percentage;
- absolute split throughput advantage.

The split throughput advantage is:

$$\Delta T = T^*_{split} - T^*_{single}.$$

43.1.1 Important Analytical Rule

Scenario outcomes should not be assumed in advance.

The theoretical relationship remains:

$$T^*_{single} \leq T^*_{split}.$$

Therefore:

$$\Delta T \geq 0.$$

However, the inequality may hold with:

- equality;
- strict inequality.

If a scenario produces:

$$\Delta T = 0,$$

that is a valid result.

It means commodity splitting is not required to improve maximum throughput under that particular scenario.

The analysis should not force or manufacture a split-routing advantage.

43.1.2 Business Impact

Scenario analysis tests whether a routing architecture remains appropriate as operating conditions change.

A routing policy should ideally be evaluated not only for the current network state, but also for:

- demand growth;
- degradation;
- congestion;
- capacity fragmentation.

This provides a stronger basis for production architecture decisions.

43.2 Build a Reusable Scenario Solver

The scenario framework should change only the intended scenario inputs.

For each scenario, the solver can modify:

- commodity demand;
- directed-link capacity;

while preserving:

- topology;
- candidate paths;
- destination rules;
- routing formulations.

The reusable function below accepts:

- a scenario name;
- a demand multiplier;
- optional directed-capacity multipliers.

It then solves both Stage 1 routing formulations and returns a consistent results record.

In [498…

```
# ------------------------------------------------------------
# 43.2 Reusable Stage 1 scenario solver
# ------------------------------------------------------------

def run_stage1_scenario(
```

```
    scenario_name,
    base_path_df,
    base_capacity_dict,
    demand_multiplier=1.0,
    capacity_multipliers=None,
    solver_msg=False
):
    """
    Run one Stage 1 network scenario under both:
        1. Splittable routing
        2. Single-path routing

    Parameters
    ----------
    scenario_name : str
        Descriptive scenario label.

    base_path_df : pandas.DataFrame
        Candidate-path table prepared in Section 41.

    base_capacity_dict : dict
        Baseline directed-link capacities.

    demand_multiplier : float
        Multiplier applied to every commodity demand.
        Example:
            0.60 = 60% of baseline demand
            1.50 = 150% of baseline demand

    capacity_multipliers : dict or None
        Optional dictionary:
            {(u, v): multiplier}

        Example:
            {
                ("A", "C"): 0.70,
                ("C", "A"): 0.70
            }

    solver_msg : bool
        Whether CBC solver messages should be shown.

    Returns
    -------
    dict
        Scenario summary.
    """

    # ----------------------------------------------------------
    # Copy baseline candidate-path data.
    # ----------------------------------------------------------
```

```
    scenario_path_df = (
        base_path_df
        .copy()
    )


    # ------------------------------------------------------------
    # Scale commodity demand.
    # ------------------------------------------------------------

    scenario_path_df[
        "demand_mbps"
    ] = (
        scenario_path_df[
            "demand_mbps"
        ]
        *
        float(
            demand_multiplier
        )
    )


    # ------------------------------------------------------------
    # Copy baseline capacities.
    # ------------------------------------------------------------

    scenario_capacity_dict = (
        base_capacity_dict.copy()
    )


    # ------------------------------------------------------------
    # Apply optional directional capacity multipliers.
    # ------------------------------------------------------------

    if capacity_multipliers is not None:

        for edge, multiplier in (
            capacity_multipliers.items()
        ):

            if edge not in scenario_capacity_dict:

                raise KeyError(
                    f"Scenario '{scenario_name}' references "
                    f"unknown directed edge {edge}."
                )

            scenario_capacity_dict[
                edge
            ] = (
```

```
            scenario_capacity_dict[
                edge
            ]
            *
            float(
                multiplier
            )
        )


# ------------------------------------------------------
# Calculate total offered scenario demand.
#
# Because each commodity appears on multiple candidate
# path rows, demand must be counted once per commodity.
# ------------------------------------------------------

scenario_commodity_demand_df = (
    scenario_path_df[
        [
            "commodity_id",
            "demand_mbps"
        ]
    ]
    .drop_duplicates(
        subset=[
            "commodity_id"
        ]
    )
)


offered_demand_mbps = float(
    scenario_commodity_demand_df[
        "demand_mbps"
    ]
    .sum()
)


# ------------------------------------------------------
# Solve splittable Stage 1.
# ------------------------------------------------------

split_result = (
    solve_split_stage1_with_capacities(
        scenario_path_df,
        scenario_capacity_dict,
        solver_msg=solver_msg
    )
)
```

```
# ------------------------------------------------------------
# Solve single-path Stage 1.
# ------------------------------------------------------------

single_result = (
    solve_single_stage1_with_capacities(
        scenario_path_df,
        scenario_capacity_dict,
        solver_msg=solver_msg
    )
)


# ------------------------------------------------------------
# Validate optimality.
# ------------------------------------------------------------

if (
    split_result[
        "status"
    ]
    != "Optimal"
):

    raise RuntimeError(
        f"Split model failed in scenario "
        f"'{scenario_name}'. "
        f"Status: {split_result['status']}"
    )


if (
    single_result[
        "status"
    ]
    != "Optimal"
):

    raise RuntimeError(
        f"Single-path model failed in scenario "
        f"'{scenario_name}'. "
        f"Status: {single_result['status']}"
    )


# ------------------------------------------------------------
# Extract optimized throughputs.
# ------------------------------------------------------------

split_throughput = float(
    split_result[
```

```
            "throughput_mbps"
        ]
    )


    single_throughput = float(
        single_result[
            "throughput_mbps"
        ]
    )


    # ------------------------------------------------------------
    # Compute delivery percentages.
    # ------------------------------------------------------------

    split_delivery_pct = (
        100
        * split_throughput
        / offered_demand_mbps
        if offered_demand_mbps > 0
        else 0
    )


    single_delivery_pct = (
        100
        * single_throughput
        / offered_demand_mbps
        if offered_demand_mbps > 0
        else 0
    )


    # ------------------------------------------------------------
    # Return standardized scenario results.
    # ------------------------------------------------------------

    return {
        "scenario":
            scenario_name,

        "demand_multiplier":
            float(
                demand_multiplier
            ),

        "offered_demand_mbps":
            offered_demand_mbps,

        "split_throughput_mbps":
            split_throughput,
```

```
        "single_throughput_mbps":
            single_throughput,

        "split_delivery_pct":
            split_delivery_pct,

        "single_delivery_pct":
            single_delivery_pct,

        "split_advantage_mbps":
            (
                split_throughput
                -
                single_throughput
            )
    }
```

43.2.1 Interpretation

The scenario solver preserves the same analytical relationship used throughout the notebook.

Only the scenario inputs change.

This creates a controlled comparison across operating states.

The key output is:

$$\Delta T = T^*_{split} - T^*_{single}.$$

If:

$$\Delta T > 0,$$

the additional routing flexibility of the splittable formulation has throughput value in that scenario.

If:

$$\Delta T = 0,$$

the single-path restriction is still not throughput-binding.

43.3 Validate the Scenario Framework Using the Baseline

In [500…

```
# ------------------------------------------------------------
```

```
# 43.3 Validate scenario framework using baseline conditions
# ------------------------------------------------------------

baseline_scenario_result = (
    run_stage1_scenario(
        scenario_name="Baseline",
        base_path_df=perturbation_paths_df,
        base_capacity_dict=baseline_capacity_dict,
        demand_multiplier=1.0
    )
)


baseline_scenario_result
```

Out[500…

```
{'scenario': 'Baseline',
 'demand_multiplier': 1.0,
 'offered_demand_mbps': 126.0,
 'split_throughput_mbps': 83.0,
 'single_throughput_mbps': 83.0,
 'split_delivery_pct': 65.87301587301587,
 'single_delivery_pct': 65.87301587301587,
 'split_advantage_mbps': 0.0}
```

43.3.1 Expected Results

The baseline scenario should reproduce:

$$126 \text{ Mbps}$$

of offered demand.

Both routing formulations should deliver:

$$83 \text{ Mbps}.$$

Therefore:

$$\Delta T = 83 - 83 = 0.$$

The delivery percentage should be approximately:

$$65.87\%.$$

In [501…

```
# ------------------------------------------------------------
# 43.3 Validate baseline scenario numerically
# ------------------------------------------------------------

assert abs(
    baseline_scenario_result[
        "offered_demand_mbps"
```

```
    ]
    -
    total_demand
) < 1e-6


assert abs(
    baseline_scenario_result[
        "split_throughput_mbps"
    ]
    -
    split_stage1_throughput
) < 1e-6


assert abs(
    baseline_scenario_result[
        "single_throughput_mbps"
    ]
    -
    single_stage1_throughput
) < 1e-6


print(
    "Scenario framework successfully reproduces baseline."
)
```

```
Scenario framework successfully reproduces baseline.
```

43.3.2 Interpretation

The scenario framework reproduces the original baseline exactly.

This confirms that future scenario differences are caused by the modified:

- demand;
- capacity;

rather than by changes in the optimization formulation.

43.4 Low-Demand Scenario — 60% of Baseline Demand

The first stress test is actually a low-load scenario.

Every source-stream demand is reduced to:

$$60\%$$

of baseline.

Baseline offered demand is:

$$126 \text{ Mbps}.$$

Therefore, total low-demand traffic is:

$$0.60(126) = 75.6 \text{ Mbps}.$$

This scenario asks:

> **When demand falls below the baseline maximum-throughput ceiling, can the physical network deliver all offered traffic?**

No throughput result is assumed in advance.

The optimization determines whether all:

$$75.6 \text{ Mbps}$$

can be routed feasibly.

In [502…
```
# ------------------------------------------------------------
# 43.4 Low-demand scenario: 60% of baseline demand
# ------------------------------------------------------------

low_demand_result = (
    run_stage1_scenario(
        scenario_name="Low Demand - 60%",
        base_path_df=perturbation_paths_df,
        base_capacity_dict=baseline_capacity_dict,
        demand_multiplier=0.60
    )
)


low_demand_result
```

Out[502…
```
{'scenario': 'Low Demand - 60%',
 'demand_multiplier': 0.6,
 'offered_demand_mbps': 75.6,
 'split_throughput_mbps': 75.6,
 'single_throughput_mbps': 75.6,
 'split_delivery_pct': 100.0,
 'single_delivery_pct': 100.0,
 'split_advantage_mbps': 0.0}
```

43.4.1 Interpretation

If both models deliver the entire:

$$75.6 \text{ Mbps},$$

then the physical network has sufficient capacity for this lower operating state.

That would produce:

$$100\%$$

delivery.

If the models still lose traffic, that would indicate that specific topology or directional-capacity constraints remain binding even when total demand is lower.

The executed result should determine the conclusion.

43.4.2 Business Impact

The low-demand scenario helps identify whether congestion is:

- persistent even under light load;
- primarily a peak-demand problem.

If all traffic is delivered at 60% demand, the network may have sufficient capacity during normal periods but insufficient capacity during heavier operating states.

43.5 High-Demand Scenario — 150% of Baseline Demand

The second scenario increases every commodity demand to:

$$150\%$$

of baseline.

Total offered demand becomes:

$$1.50(126) = 189 \text{ Mbps}.$$

This scenario asks:

> **How does the routing architecture behave when the network is significantly more congested than the baseline?**

The larger demand level may:

- increase undelivered traffic;
- change which physical constraints become binding;
- potentially increase the value of routing flexibility.

However, no split-routing advantage is assumed in advance.

In [503…
```
# ------------------------------------------------------------
# 43.5 High-demand scenario: 150% of baseline demand
# ------------------------------------------------------------

high_demand_result = (
    run_stage1_scenario(
        scenario_name="High Demand - 150%",
        base_path_df=perturbation_paths_df,
        base_capacity_dict=baseline_capacity_dict,
        demand_multiplier=1.50
    )
)


high_demand_result
```

Out[503…
```
{'scenario': 'High Demand - 150%',
 'demand_multiplier': 1.5,
 'offered_demand_mbps': 189.0,
 'split_throughput_mbps': 83.0,
 'single_throughput_mbps': 83.0,
 'split_delivery_pct': 43.91534391534392,
 'single_delivery_pct': 43.91534391534392,
 'split_advantage_mbps': 0.0}
```

43.5.1 Interpretation

The important metrics are:

- maximum split throughput;
- maximum single-path throughput;
- delivery percentages;
- split throughput advantage.

If:

$$\Delta T > 0,$$

splitting becomes throughput valuable under heavier congestion.

If:

$$\Delta T = 0,$$

then the physical network constraints continue to dominate both routing formulations equally.

Either result is analytically valid.

43.5.2 Business Impact

The high-demand scenario approximates future traffic growth or a peak operating period.

If delivery percentage deteriorates sharply, the network may require:

- capacity expansion;
- admission-control policies;
- traffic prioritization.

If split routing begins to outperform single path under high demand, that provides evidence that multipath capability becomes more valuable as congestion increases.

43.6 Internal-Corridor Degradation Scenario

This scenario reduces capacity on important internal links associated with the C corridor.

The physical links:

- A ↔ C;
- B ↔ C;

are reduced to:

$$70\%$$

of baseline capacity in both directions.

Because the original network defines these as full-duplex physical links, both directed capacities are reduced.

This represents conditions such as:

- partial circuit degradation;
- reduced available bandwidth;
- maintenance restrictions;
- shared-resource contention.

The scenario asks:

> **How well can the network use the alternate topology when the C-centered internal corridor becomes narrower?**

In [504…

```
# ------------------------------------------------------------
# 43.6 Internal-corridor degradation:
# A–C and B–C physical links reduced to 70%
# ------------------------------------------------------------

internal_corridor_capacity_multipliers = {
    ("A", "C"): 0.70,
    ("C", "A"): 0.70,

    ("B", "C"): 0.70,
    ("C", "B"): 0.70
}


internal_degradation_result = (
    run_stage1_scenario(
        scenario_name="Internal Corridor Degradation – 70%",
        base_path_df=perturbation_paths_df,
        base_capacity_dict=baseline_capacity_dict,
        demand_multiplier=1.0,
        capacity_multipliers=(
            internal_corridor_capacity_multipliers
        )
    )
)


internal_degradation_result
```

Out[504…

```
{'scenario': 'Internal Corridor Degradation – 70%',
 'demand_multiplier': 1.0,
 'offered_demand_mbps': 126.0,
 'split_throughput_mbps': 78.1,
 'single_throughput_mbps': 78.1,
 'split_delivery_pct': 61.98412698412697,
 'single_delivery_pct': 61.98412698412697,
 'split_advantage_mbps': 0.0}
```

43.6.1 Interpretation

The reduced A–C and B–C capacities place more pressure on the alternate D corridor.

The scenario therefore tests whether:

- B → D;
- D → UK;
- D → US;

can absorb additional traffic.

If splittable routing achieves higher throughput, the result would indicate that fractional path allocation helps use the remaining fragmented internal capacity.

If the models tie, then splitting is still not required to achieve maximum throughput under this degradation.

43.6.2 Business Impact

This scenario approximates a partial infrastructure impairment rather than increased demand.

It helps evaluate whether routing flexibility provides resilience when part of the internal topology becomes less capable.

That is a different source of value from baseline throughput.

43.7 Destination-Exit Degradation Scenario

This scenario reduces the capacities of the destination-facing directed links:

- C → UK;
- D → UK;
- C → US;
- D → US.

Each is reduced to:

$$75\%$$

of baseline capacity.

Only the destination-facing directions are modified because the scenario is specifically modeling reduced outbound capacity toward the exits.

The reverse-direction capacities remain unchanged.

The scenario asks:

> **How sensitive is total deliverability to reduced destination-**

> **boundary capacity?**

In [505…
```
# ------------------------------------------------------------
# 43.7 Destination-facing exit degradation: 75%
# ------------------------------------------------------------

exit_capacity_multipliers = {
    ("C", "UK"): 0.75,
    ("D", "UK"): 0.75,
    ("C", "US"): 0.75,
    ("D", "US"): 0.75
}


exit_degradation_result = (
    run_stage1_scenario(
        scenario_name="Destination Exit Degradation - 75%",
        base_path_df=perturbation_paths_df,
        base_capacity_dict=baseline_capacity_dict,
        demand_multiplier=1.0,
        capacity_multipliers=(
            exit_capacity_multipliers
        )
    )
)


exit_degradation_result
```

Out[505…
```
{'scenario': 'Destination Exit Degradation - 75%',
 'demand_multiplier': 1.0,
 'offered_demand_mbps': 126.0,
 'split_throughput_mbps': 69.25,
 'single_throughput_mbps': 69.25,
 'split_delivery_pct': 54.96031746031746,
 'single_delivery_pct': 54.96031746031746,
 'split_advantage_mbps': 0.0}
```

43.7.1 Interpretation

This scenario directly stresses the boundary between the internal network and the final destinations.

If throughput falls sharply, that would reinforce the Section 42 conclusion that destination-facing capacity is structurally important.

Splittable routing may have limited ability to compensate if both available exit corridors become constrained simultaneously.

This is an important reminder:

> **Routing flexibility cannot create physical destination capacity that does not exist.**

43.7.2 Business Impact

Exit-capacity degradation may represent:

- downstream provider limitations;
- destination-side congestion;
- reduced external connectivity.

If the scenario produces substantial loss under both routing policies, investment in internal routing sophistication alone would not solve the problem.

The physical destination boundary would require attention.

43.8 Fragmented-Capacity Stress Scenario

One potential advantage of splittable routing is its ability to combine smaller amounts of residual capacity across multiple paths.

This is known as **capacity fragmentation**.

Conceptually, suppose a commodity requires:

$$6 \text{ Mbps}.$$

Two feasible paths may each have only:

$$3 \text{ Mbps}$$

of residual capacity.

A single-path model may be unable to place the full 6 Mbps on either path.

A splittable model could potentially assign:

$$3 + 3 = 6 \text{ Mbps}.$$

However, because partial commodity delivery is allowed in this assessment, the throughput advantage of splitting is not guaranteed.

Therefore, the fragmented-capacity scenario below is diagnostic rather than constructed to force a desired result.

In [507…

```
# ------------------------------------------------------------
# 43.8 Fragmented-capacity stress scenario
# ------------------------------------------------------------
#
# Reduce several important internal corridors differently.
# The purpose is to create a more uneven distribution of
# residual path capacity while preserving multiple routes.
#
# This scenario is diagnostic.
# It does NOT assume that split routing must outperform
# single-path routing.
# ------------------------------------------------------------

fragmented_capacity_multipliers = {
    ("A", "C"): 0.75,
    ("C", "A"): 0.75,

    ("B", "C"): 0.65,
    ("C", "B"): 0.65,

    ("B", "D"): 0.80,
    ("D", "B"): 0.80,

    ("A", "B"): 0.85,
    ("B", "A"): 0.85
}


fragmented_capacity_result = (
    run_stage1_scenario(
        scenario_name="Fragmented Internal Capacity",
        base_path_df=perturbation_paths_df,
        base_capacity_dict=baseline_capacity_dict,
        demand_multiplier=1.0,
        capacity_multipliers=(
            fragmented_capacity_multipliers
        )
    )
)


fragmented_capacity_result
```

Out[507…

```
{'scenario': 'Fragmented Internal Capacity',
 'demand_multiplier': 1.0,
 'offered_demand_mbps': 126.0,
 'split_throughput_mbps': 72.85,
 'single_throughput_mbps': 72.85,
 'split_delivery_pct': 57.81746031746031,
 'single_delivery_pct': 57.81746031746031,
 'split_advantage_mbps': 0.0}
```

43.8.1 Interpretation

The key quantity is:

$$\Delta T = T^*_{split} - T^*_{single}.$$

If:

$$\Delta T > 0,$$

the scenario provides evidence that splittable routing can combine residual capacity more effectively when capacity becomes fragmented.

If:

$$\Delta T = 0,$$

that is still a valid result.

It means that under this particular fragmented-capacity pattern, the single-path restriction does not reduce maximum delivered throughput.

The scenario should not be repeatedly modified merely to manufacture a split-routing advantage.

A conceptual fragmented-capacity example can still explain when such an advantage could occur.

43.9.2 Business Impact

The fragmented-capacity scenario tests an operational environment that may occur when:

- several circuits are partially degraded;
- capacity is unevenly reserved;
- multiple links have small residual headroom.

If the split model performs better, this supports retaining multipath capability for stressed conditions.

If it does not, the operational simplicity of single-path routing remains attractive for this network.

43.9.3 Consolidate All Scenario Results

In [509…

```
# ------------------------------------------------------------
```

```
# 43.9 Consolidate scenario results
# ------------------------------------------------------------

scenario_results_df = pd.DataFrame(
    [
        baseline_scenario_result,
        low_demand_result,
        high_demand_result,
        internal_degradation_result,
        exit_degradation_result,
        fragmented_capacity_result
    ]
)


scenario_results_df[
    "split_lost_mbps"
] = (
    scenario_results_df[
        "offered_demand_mbps"
    ]
    -
    scenario_results_df[
        "split_throughput_mbps"
    ]
)


scenario_results_df[
    "single_lost_mbps"
] = (
    scenario_results_df[
        "offered_demand_mbps"
    ]
    -
    scenario_results_df[
        "single_throughput_mbps"
    ]
)


scenario_results_df[
    "split_advantage_pct_of_demand"
] = (
    100
    * scenario_results_df[
        "split_advantage_mbps"
    ]
    /
    scenario_results_df[
        "offered_demand_mbps"
    ]
```

```
)


scenario_results_df
```

Out[509…

	scenario	demand_multiplier	offered_demand_mbps	split_throughput_mb
0	Baseline	1.0	126.0	83.
1	Low Demand - 60%	0.6	75.6	75.
2	High Demand - 150%	1.5	189.0	83.
3	Internal Corridor Degradation - 70%	1.0	126.0	78.
4	Destination Exit Degradation - 75%	1.0	126.0	69.
5	Fragmented Internal Capacity	1.0	126.0	72.

43.9.3 Detailed Scenario Results Analysis

The completed scenario analysis provides a stronger conclusion than the baseline comparison alone.

The executed results are:

Scenario	Offered Demand	Split Throughput	Single-Path Throughput	Delivery Rate	Split Advantage
Baseline	126.0 Mbps	83.0 Mbps	83.0 Mbps	65.87%	0.0 Mbps
Low Demand - 60%	75.6 Mbps	75.6 Mbps	75.6 Mbps	100.00%	0.0 Mbps
High Demand - 150%	189.0 Mbps	83.0 Mbps	83.0 Mbps	43.92%	0.0 Mbps
Internal Corridor Degradation -	126.0 Mbps	78.1 Mbps	78.1 Mbps	61.98%	0.0 Mbps

70%					
Destination Exit Degradation - 75%	126.0 Mbps	69.25 Mbps	69.25 Mbps	54.96%	0.0 Mbps
Fragmented Internal Capacity	126.0 Mbps	72.85 Mbps	72.85 Mbps	57.82%	0.0 Mbps

Several important conclusions follow.

Result 1 — Low Demand Is Fully Serviceable

When demand is reduced to:

$$60\%$$

of baseline:

$$126(0.60) = 75.6 \text{ Mbps},$$

both routing models deliver:

$$75.6 \text{ Mbps}.$$

Therefore:

$$100\%$$

of demand is served.

This demonstrates that the network is not structurally incapable of serving all traffic under every operating state.

The service deficit emerges only once offered demand reaches levels that interact with the network's capacity constraints.

Result 2 — Higher Demand Does Not Increase the Baseline Throughput Ceiling

When offered demand increases to:

$$189 \text{ Mbps},$$

both routing models still deliver only:

$$83 \text{ Mbps}.$$

Therefore, the delivery rate falls to approximately:

$$43.92\%.$$

The network is no longer demand-limited.

It is capacity-limited.

Additional demand cannot be converted into additional service because the existing physical topology and directed capacities already constrain maximum throughput.

Result 3 — Internal-Corridor Degradation Reduces Throughput

Reducing selected internal corridor capacities to 70% lowers maximum throughput from:

$$83$$

to:

$$78.1 \text{ Mbps}.$$

The absolute reduction is:

$$83 - 78.1 = 4.9 \text{ Mbps}.$$

This confirms that the internal transport network contributes materially to the baseline capacity ceiling.

Result 4 — Destination-Exit Degradation Is More Severe

Reducing destination-facing capacity to 75% lowers throughput to:

$$69.25 \text{ Mbps}.$$

Relative to the baseline:

$$83 - 69.25 = 13.75 \text{ Mbps}$$

of additional service is lost.

Among the tested degradation scenarios, this produces the largest reduction in maximum throughput.

This strengthens the earlier infrastructure conclusion that destination-facing links are strategically important.

Result 5 — Fragmented Internal Capacity Also Produces Significant Loss

The fragmented-capacity scenario delivers:

$$72.85 \text{ Mbps}.$$

Relative to baseline:

$$83 - 72.85 = 10.15 \text{ Mbps}$$

is lost.

This demonstrates that the network is sensitive not only to one major bottleneck but also to distributed degradation across multiple internal links.

Result 6 — Splitting Does Not Improve Stage 1 Throughput in Any Tested Scenario

The most important routing-policy result is that:

$$T^*_{split} = T^*_{single}$$

in every tested scenario.

Therefore:

$$\Delta T = T^*_{split} - T^*_{single} = 0$$

across:

- baseline demand;
- low demand;
- high demand;
- internal degradation;
- destination-exit degradation;
- fragmented capacity.

This provides stronger evidence than the baseline alone.

Within the tested network and scenario set, the one-path restriction is not throughput-binding.

The additional feasible-region flexibility of splittable routing does not create additional Stage 1 Mbps.

Why This Happens

Partial delivery is allowed in the single-path formulation.

Therefore, a commodity is not required to fit its entire original demand on one path.

The model may deliver:

$$0 < z_{s,k} < D_{s,k}$$

on a single route.

This makes the single-path formulation substantially more flexible than an all-or-nothing unsplittable-flow formulation.

The current physical bottlenecks can therefore be reached without requiring individual commodities to split across multiple paths.

The dominant limitation is physical network capacity rather than path divisibility.

Business Impact

The stress-testing results materially strengthen the routing recommendation.

The evidence does **not** support deploying splittable routing because it provides additional Stage 1 throughput under the tested conditions.

It does not.

Its demonstrated baseline value instead appears in:

- lower sensitive latency;
- finer-grained routing flexibility;
- possible resilience value under future network states not represented by the current scenarios.

For Stage 1 throughput, the simpler single-path formulation remains competitive across every tested operating condition.

This supports an adaptive architecture:

> Use single-path routing when it satisfies service requirements,
> while retaining splittable-routing capability when latency, resilience,
> or future network conditions justify the additional complexity.

43.9.4 Interpretation

The consolidated table provides a consistent comparison across all operating states.

For each scenario it reports:

- offered demand;
- split throughput;
- single-path throughput;
- lost demand;
- delivery percentage;
- split throughput advantage.

This makes it possible to determine whether the baseline conclusion:

$$T^*_{split} = T^*_{single}$$

remains true under stress.

43.10 Validate the Feasible-Region Relationship Across Scenarios

The mathematical relationship established in Section 39 must remain true under every scenario as long as the two models use the same:

- scenario demand;
- scenario capacities;
- candidate paths;
- routing rules other than the one-path restriction.

Therefore:

$$T^*_{single} \leq T^*_{split}.$$

Equivalently:

$$\Delta T = T^*_{split} - T^*_{single} \geq 0.$$

The following validation checks this relationship for every scenario.

In [531…

```
# ------------------------------------------------------------
# 43.10 Validate split feasible-region dominance
# ------------------------------------------------------------
```

```
scenario_tolerance = 1e-6


scenario_results_df[
    "theoretical_relationship_valid"
] = (
    scenario_results_df[
        "split_advantage_mbps"
    ]
    >=
    -scenario_tolerance
)


display(
    scenario_results_df[
        [
            "scenario",
            "split_throughput_mbps",
            "single_throughput_mbps",
            "split_advantage_mbps",
            "theoretical_relationship_valid"
        ]
    ]
)


assert (
    scenario_results_df[
        "theoretical_relationship_valid"
    ]
    .all()
)


print(
    "Theoretical split >= single relationship "
    "holds for every scenario."
)
```

	scenario	split_throughput_mbps	single_throughput_mbps	split_advantage_
0	Baseline	83.00	83.00	
1	Low Demand - 60%	75.60	75.60	
2	High Demand - 150%	83.00	83.00	
3	Internal Corridor Degradation - 70%	78.10	78.10	
4	Destination Exit Degradation - 75%	69.25	69.25	
5	Fragmented Internal Capacity	72.85	72.85	

```
Theoretical split >= single relationship holds for every scenario.
```

Expected Result

Every scenario should satisfy:

$$T^*_{single} \leq T^*_{split}.$$

If a scenario produced:

$$T^*_{single} > T^*_{split},$$

the result should trigger a model audit rather than be interpreted as a legitimate routing advantage.

43.11 Throughput Comparison Across Scenarios

In [512…

```
# ------------------------------------------------------------
# 43.11 Compare maximum throughput across scenarios
# ------------------------------------------------------------

scenario_results_df.set_index(
    "scenario"
)[
```

```
    [
        "split_throughput_mbps",
        "single_throughput_mbps"
    ]
].plot(
    kind="bar",
    figsize=(13, 6)
)


plt.ylabel(
    "Maximum Delivered Throughput (Mbps)"
)

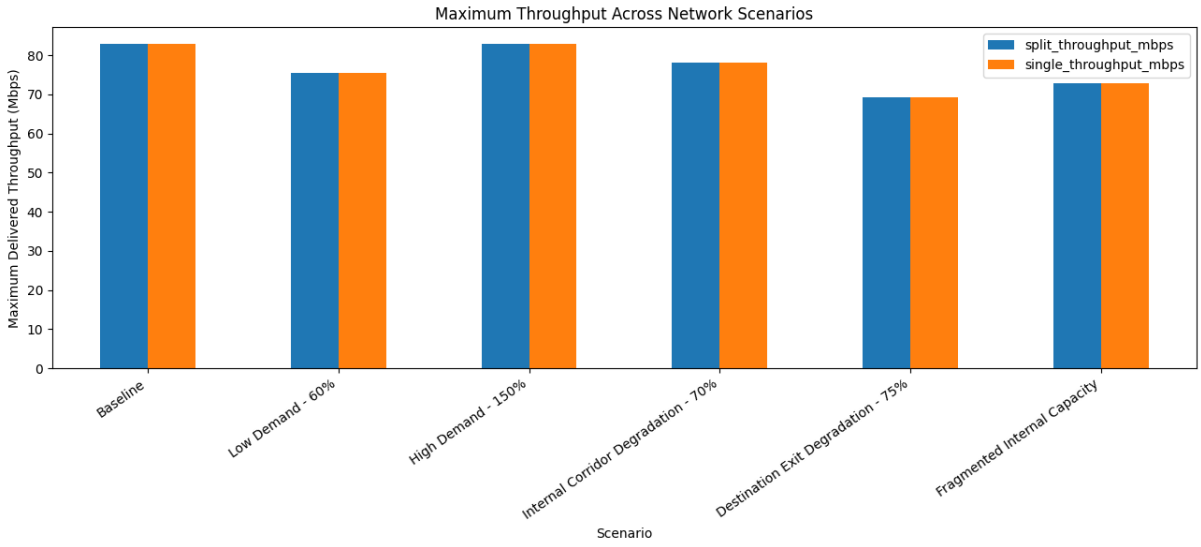
plt.xlabel(
    "Scenario"
)

plt.title(
    "Maximum Throughput Across Network Scenarios"
)

plt.xticks(
    rotation=35,
    ha="right"
)

plt.tight_layout()

plt.show()
```



Visualization Interpretation

The chart shows how maximum deliverable traffic changes as:

- demand changes;

- internal capacity degrades;
- exit capacity degrades;
- residual capacity becomes fragmented.

The separation between the two bars in each scenario represents the throughput value of splittable routing.

If the bars overlap, splitting provides no throughput advantage in that scenario.

If the split bar is higher, the single-path restriction has become throughput-binding.

Business Impact

The chart provides a useful architecture decision view.

If single-path and split routing remain tied across nearly all scenarios, the operational simplicity of single-path routing becomes more compelling.

If splitting provides material gains in stressed states, a hybrid operating strategy may be preferable:

- single path during normal operation;
- splittable routing during congestion or degradation.

43.12 Delivery Percentage Across Scenarios

In [514…

```
# ------------------------------------------------------------
# 43.12 Compare delivery percentage across scenarios
# ------------------------------------------------------------

scenario_results_df.set_index(
    "scenario"
)[
    [
        "split_delivery_pct",
        "single_delivery_pct"
    ]
].plot(
    kind="bar",
    figsize=(13, 6)
)


plt.ylabel(
    "Delivered Demand (%)"
)
```

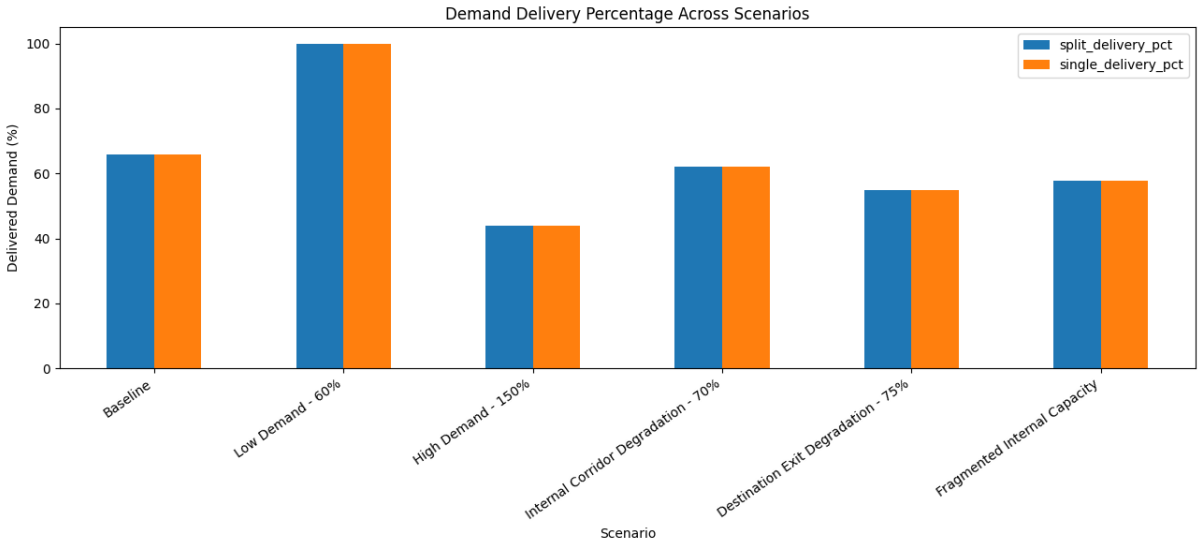
```
plt.xlabel(
    "Scenario"
)

plt.title(
    "Demand Delivery Percentage Across Scenarios"
)

plt.xticks(
    rotation=35,
    ha="right"
)

plt.tight_layout()

plt.show()
```



Visualization Interpretation

Throughput alone can be misleading when total offered demand changes.

For example:

- 83 Mbps may be strong performance when offered demand is 90 Mbps;
- the same 83 Mbps may represent much weaker service when offered demand is 189 Mbps.

Delivery percentage therefore normalizes network performance relative to demand:

$$R = 100\frac{T}{D}.$$

This provides a more comparable service metric across scenarios.

43.13 Splittable Throughput Advantage Across Scenarios

In [516…

```
# ------------------------------------------------------------
# 43.13 Visualize split-routing throughput advantage
# ------------------------------------------------------------

plt.figure(
    figsize=(12, 5)
)


plt.bar(
    scenario_results_df[
        "scenario"
    ],
    scenario_results_df[
        "split_advantage_mbps"
    ]
)


plt.axhline(
    0,
    linestyle="--"
)


plt.ylabel(
    "Splittable Throughput Advantage (Mbps)"
)

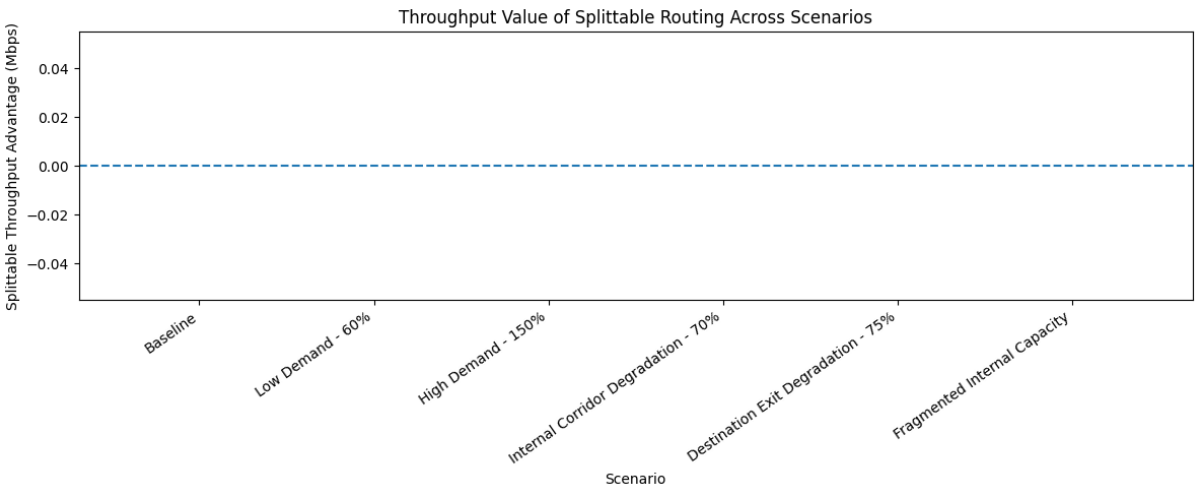
plt.xlabel(
    "Scenario"
)

plt.title(
    "Throughput Value of Splittable Routing "
    "Across Scenarios"
)

plt.xticks(
    rotation=35,
    ha="right"
)

plt.tight_layout()
```

```
plt.show()
```



Visualization Interpretation

This is the most direct visualization of the throughput value of route splitting.

For every scenario:

$$\Delta T = T^*_{split} - T^*_{single}.$$

A bar at:

$$0$$

means the one-path restriction does not reduce maximum throughput.

A positive bar means that split routing is able to use network capacity more effectively than the single-path formulation.

Because:

$$\mathcal{F}_{single} \subseteq \mathcal{F}_{split},$$

negative bars should not occur beyond numerical tolerance.

Business Impact

This chart provides a direct answer to an important architecture question:

> **Under what operating conditions is the complexity of multipath routing actually rewarded with additional throughput?**

That is more informative than comparing the two routing policies only at the baseline.

43.14 Identify the Most Stressful Scenario

In [517…

```
# ------------------------------------------------------------
# 43.14 Identify scenarios with lowest delivery percentage
# ------------------------------------------------------------

scenario_stress_ranking_df = (
    scenario_results_df[
        [
            "scenario",
            "offered_demand_mbps",
            "split_throughput_mbps",
            "single_throughput_mbps",
            "split_delivery_pct",
            "single_delivery_pct",
            "split_advantage_mbps"
        ]
    ]
    .sort_values(
        "split_delivery_pct",
        ascending=True
    )
    .reset_index(
        drop=True
    )
)


scenario_stress_ranking_df
```

Out [517...

	scenario	offered_demand_mbps	split_throughput_mbps	single_throughp
0	High Demand - 150%	189.0	83.00	
1	Destination Exit Degradation - 75%	126.0	69.25	
2	Fragmented Internal Capacity	126.0	72.85	
3	Internal Corridor Degradation - 70%	126.0	78.10	
4	Baseline	126.0	83.00	
5	Low Demand - 60%	75.6	75.60	

43.14.1 Interpretation

The lowest delivery percentages identify the network states that place the greatest pressure on service.

The cause of poor performance should then be distinguished.

For example:

- high demand represents increased load;
- internal degradation represents reduced corridor capacity;
- exit degradation represents reduced destination-boundary capacity.

Different causes may require different mitigation strategies.

43.14.2 Business Impact

Scenario ranking helps differentiate whether the highest operational risk comes from:

- demand growth;
- internal network degradation;

- exit limitations;
- fragmented capacity.

That distinction can inform:

- capacity planning;
- resilience planning;
- routing-policy design.

43.14.3 Business Interpretation of Scenario Severity

The scenario ranking reveals that different types of network stress do not have equal impact.

Among the tested capacity-degradation scenarios:

Internal Corridor Degradation

Maximum throughput:

$$78.1 \text{ Mbps.}$$

Loss relative to baseline:

$$4.9 \text{ Mbps.}$$

Fragmented Internal Capacity

Maximum throughput:

$$72.85 \text{ Mbps.}$$

Loss relative to baseline:

$$10.15 \text{ Mbps.}$$

Destination Exit Degradation

Maximum throughput:

$$69.25 \text{ Mbps.}$$

Loss relative to baseline:

$$13.75 \text{ Mbps.}$$

The destination-exit degradation therefore produces the largest throughput loss among the tested degradation scenarios.

Why This Happens

Destination-facing links sit near the final boundary of the network.

Upstream capacity has limited value if traffic cannot ultimately cross the final exit links.

This creates a form of **stranded internal capacity**.

The network may contain available bandwidth upstream while still being unable to deliver additional traffic to UK or US.

Business Impact

The result reinforces an important infrastructure-planning principle:

> **Internal network expansion should be evaluated together with destination-facing capacity.**

Investment in upstream bandwidth may generate little service value if exit capacity remains constrained.

This does not mean destination links will always be the dominant bottleneck in every network.

It means that, in this synthetic network and tested scenario set, destination-facing capacity has the strongest observed degradation impact.

43.15 Identify Scenarios Where Splitting Has Throughput Value

In [518…

```
# ------------------------------------------------------------
# 43.15 Identify scenarios where split routing provides
# positive throughput value
# ------------------------------------------------------------

split_advantage_scenarios_df = (
    scenario_results_df[
        scenario_results_df[
            "split_advantage_mbps"
        ]
        > scenario_tolerance
```

```
    ]
    .copy()
)


if len(
    split_advantage_scenarios_df
) > 0:

    display(
        split_advantage_scenarios_df[
            [
                "scenario",
                "split_throughput_mbps",
                "single_throughput_mbps",
                "split_advantage_mbps",
                "split_advantage_pct_of_demand"
            ]
        ]
    )

else:

    print(
        "No tested scenario produced a positive "
        "splittable throughput advantage."
    )
```

```
No tested scenario produced a positive splittable throughput advantage.
```

Interpretation

If the table contains one or more scenarios, those are the operating states where the single-path restriction becomes throughput-binding.

For those cases:

$$T^*_{split} > T^*_{single}.$$

That provides direct evidence that route splitting has throughput value under those conditions.

If the table is empty, the correct conclusion is:

> **No tested scenario required commodity splitting to improve maximum throughput.**

That is still a meaningful result and supports the operational simplicity of the single-path formulation for the tested network states.

Business Impact

The routing recommendation can now become conditional rather than universal.

For example:

- use single-path routing during ordinary conditions;
- activate split routing only in scenario classes where it demonstrably improves performance.

This could provide the benefits of operational simplicity without permanently giving up multipath flexibility.

43.16 Scenario Interpretation Matrix

In [519…

```
# ------------------------------------------------------------
# 43.16 Build scenario interpretation matrix
# ------------------------------------------------------------

def classify_scenario_split_value(
    advantage
):

    if advantage > scenario_tolerance:

        return (
            "Splittable Throughput Advantage"
        )

    elif abs(
        advantage
    ) <= scenario_tolerance:

        return (
            "Throughput Tie"
        )

    else:

        return (
            "Validation Review Required"
        )


scenario_interpretation_df = (
    scenario_results_df[
        [
            "scenario",
            "offered_demand_mbps",
```

```
            "split_delivery_pct",
            "single_delivery_pct",
            "split_advantage_mbps"
        ]
    ]
    .copy()
)


scenario_interpretation_df[
    "routing_interpretation"
] = (
    scenario_interpretation_df[
        "split_advantage_mbps"
    ]
    .apply(
        classify_scenario_split_value
    )
)


scenario_interpretation_df
```

Out [519…

	scenario	offered_demand_mbps	split_delivery_pct	single_delivery_pct	s
0	Baseline	126.0	65.873016	65.873016	
1	Low Demand - 60%	75.6	100.000000	100.000000	
2	High Demand - 150%	189.0	43.915344	43.915344	
3	Internal Corridor Degradation - 70%	126.0	61.984127	61.984127	
4	Destination Exit Degradation - 75%	126.0	54.960317	54.960317	
5	Fragmented Internal Capacity	126.0	57.817460	57.817460	

Interpretation

The matrix creates a concise routing-policy summary for every tested network

state.

It distinguishes scenarios where:

- both models provide identical maximum throughput;
- split routing provides a measurable throughput benefit;
- an unexpected result requires validation.

This makes the scenario analysis easier to communicate to nontechnical stakeholders.

43.17 Throughput Is Not the Only Stress Metric

Section 43 focuses primarily on Stage 1 throughput because it provides the cleanest comparison between the routing feasible regions.

However, two scenarios may produce the same throughput while differing materially in:

- sensitive latency;
- source-level service;
- traffic-class service;
- residual capacity;
- number of saturated links;
- route complexity.

Therefore:

$$\boxed{\text{Equal Scenario Throughput} \neq \text{Equal Operational Performance}}$$

A complete production stress-testing framework could repeat the Stage 2 latency optimization for each scenario.

That would allow comparison of:

- throughput robustness;
- latency robustness;

simultaneously.

Business Impact

A routing architecture that preserves throughput but causes substantial latency degradation may still be unacceptable for latency-sensitive applications.

Therefore, Stage 1 scenario analysis should be considered the first layer of robustness testing rather than the entire production resilience framework.

43.18 Static Snapshot Limitation

Every scenario in this section remains a static optimization snapshot.

The model does not simulate:

- traffic arriving over time;
- queues;
- packet loss;
- route churn;
- congestion-control behavior;
- dynamic rerouting delays.

Therefore, the scenarios answer:

> **What is the best feasible routing allocation under this network state?**

They do not answer:

> **How does the network dynamically transition into or out of this state?**

This distinction should be maintained when interpreting operational resilience.

43.19 Detailed Scenario Summary

Section 43 extends the baseline optimization into several demand and capacity states.

1. Baseline

The scenario framework must reproduce:

$$T^*_{split} = T^*_{single} = 83 \text{ Mbps}.$$

This validates the scenario solver.

2. Low Demand

Demand is reduced to:

$$60\%$$

of baseline:

$$75.6 \text{ Mbps}.$$

This tests whether congestion disappears at lower load.

3. High Demand

Demand increases to:

$$150\%$$

of baseline:

$$189 \text{ Mbps}.$$

This tests routing behavior under heavier congestion.

4. Internal-Corridor Degradation

A ↔ C and B ↔ C capacity are reduced.

This tests whether the alternate D corridor can compensate for weaker C-centered internal capacity.

5. Destination-Exit Degradation

C → UK, D → UK, C → US, and D → US are reduced.

This tests sensitivity to the network's destination boundary.

6. Fragmented Capacity

Several internal links are reduced by different amounts.

This tests whether split routing can use smaller pieces of residual capacity across multiple paths.

7. Theoretical Relationship

Every scenario should satisfy:

$$T^*_{single} \leq T^*_{split}.$$

Any violation indicates a model inconsistency.

8. Split Advantage Is Not Assumed

A scenario with:

$$\Delta T = 0$$

is a valid result.

The purpose of scenario testing is to discover when splitting helps, not to manufacture an advantage.

Overall Conclusion

Scenario analysis reveals whether the routing conclusions are robust across changing operating conditions.

The strongest routing architecture is not necessarily the one that performs best in only the baseline state.

It is the one that provides the best combination of:

- service;
- latency;
- simplicity;
- resilience;

across the operating states that matter to the business.

43.20 Business Recommendation

The scenario results should be used to determine whether the organization needs:

Always-On Splittable Routing

Appropriate if split routing provides material value across normal and stressed states.

Single-Path Routing

Appropriate if:

- throughput ties across most scenarios;
- operational simplicity is highly valued;
- latency remains acceptable.

Adaptive Routing Policy

Potentially the most practical architecture if:

- single path performs well under normal conditions;
- split routing becomes valuable only during congestion, degradation, or fragmentation.

Under an adaptive policy, the network could operate in a simpler single-path mode by default and activate multipath routing only when predefined operating conditions justify the added complexity.

Possible triggers could eventually include:

- utilization thresholds;
- delivery-loss thresholds;
- latency thresholds;
- failure states.

Those operational control rules are outside the current static optimization model but represent a natural production extension.

43.21 Connection to the Next Section

Sections 39–43 have now addressed:

- routing-policy comparison;
- physical utilization;
- true bottleneck identification;
- capacity-investment interpretation;
- demand and capacity stress testing.

One important question remains.

The Stage 1 objective treats every delivered Mbps equally.

That means the optimizer is not explicitly required to provide:

- fairness across sources;
- minimum traffic-class service;

- minimum sensitive-traffic admission;
- SLA guarantees.

The next section can therefore examine:

> **How should fairness, service priorities, and policy constraints be incorporated into the optimization model?**

This moves the analysis from:

> **What is physically optimal?**

to:

> **What operating policy should the organization actually choose?**

44. Fairness, Service Policy, and SLA-Aware Optimization

44.1 Purpose and Connection to the Previous Analysis

44.1.1 MARKDOWN CELL — Purpose and Context

44. Fairness, Service Policy, and SLA-Aware Optimization

44.1 Purpose and Connection to the Previous Analysis

Sections 39–43 focused primarily on:

- maximum network throughput;
- sensitive-traffic latency;
- routing flexibility;
- physical-link utilization;
- marginal capacity value;
- infrastructure-investment priorities;
- demand and capacity stress testing.

Those analyses answer important technical questions.

However, the Stage 1 optimization currently treats every delivered Mbps as equally valuable.

The Stage 1 objective is:

$$\max \sum_{s,k} z_{s,k},$$

where:

- s represents a source;
- k represents a traffic stream;
- $z_{s,k}$ represents delivered demand for commodity (s, k).

This objective maximizes total delivered traffic, but it does not require:

- equal treatment across sources;
- equal delivery percentages across commodities;
- minimum service levels;
- latency-sensitive admission priority;
- customer-specific SLAs;
- business-critical traffic protection.

Therefore, a solution can be mathematically throughput-optimal while still producing significantly different service outcomes across individual commodities.

This issue becomes especially important because the baseline network cannot serve all offered demand.

The network has:

$$126 \text{ Mbps}$$

of offered traffic but can deliver only:

$$83 \text{ Mbps}.$$

Therefore:

$$43 \text{ Mbps}$$

must remain undelivered.

Once scarcity exists, the optimization problem is no longer only:

> **How much traffic can the network deliver?**

It also becomes:

> **How should limited capacity be allocated across competing commodities?**

Section 44 therefore extends the analysis from physical optimization into service-policy optimization.

44.1.2 Core Analytical Distinction

Core Analytical Distinction

The earlier analysis demonstrated that:

$$\boxed{\text{Equal Total Throughput} \neq \text{Equal Service Allocation}}$$

The splittable and single-path Stage 1 models both deliver:

$$83 \text{ Mbps},$$

yet they do not necessarily serve the same:

- sources;
- commodities;
- traffic classes.

Therefore, throughput optimization determines:

> **What is physically efficient?**

but it does not automatically determine:

> **What is fair, acceptable, or aligned with business policy?**

A production optimization framework should treat those as separate decision layers.

44.2 Define Commodity Delivery Ratios

44.2.1 MARKDOWN Mathematical Definition

44.2 Commodity Delivery Ratios

For each commodity (s, k), define:

$$D_{s,k}$$

as offered demand and:

$$z_{s,k}$$

as delivered demand.

The commodity delivery ratio is:

$$r_{s,k} = \frac{z_{s,k}}{D_{s,k}}.$$

Because:

$$0 \le z_{s,k} \le D_{s,k},$$

the delivery ratio satisfies:

$$0 \le r_{s,k} \le 1.$$

Interpretation:

- $r_{s,k} = 1$ means 100% of demand is served;
- $r_{s,k} = 0.50$ means 50% is served;
- $r_{s,k} = 0$ means the commodity receives no service.

Delivery ratio is preferable to raw Mbps for fairness analysis because commodities have different offered-demand levels.

In [521…

```
# ------------------------------------------------------------
# 44.2 Calculate commodity-level delivery ratios
# ------------------------------------------------------------

# Create copies so that the original Stage 1 output tables
# remain unchanged.

split_fairness_df = (
    split_stage1_delivery_df
    .copy()
)


single_fairness_df = (
    single_stage1_delivery_df
    .copy()
)


# ------------------------------------------------------------
# Calculate normalized delivery ratio for each commodity.
#
# delivery_ratio = delivered demand / offered demand
# ------------------------------------------------------------

split_fairness_df[
```

```
    "delivery_ratio"
] = (
    split_fairness_df[
        "delivered_mbps"
    ]
    /
    split_fairness_df[
        "demand_mbps"
    ]
)


single_fairness_df[
    "delivery_ratio"
] = (
    single_fairness_df[
        "delivered_mbps"
    ]
    /
    single_fairness_df[
        "demand_mbps"
    ]
)


# Convert to percentage for easier business interpretation.

split_fairness_df[
    "delivery_percentage"
] = (
    100
    *
    split_fairness_df[
        "delivery_ratio"
    ]
)


single_fairness_df[
    "delivery_percentage"
] = (
    100
    *
    single_fairness_df[
        "delivery_ratio"
    ]
)


print(
    "Splittable Stage 1 commodity delivery ratios:"
)
```

```
display(
    split_fairness_df[
        [
            "commodity_id",
            "source",
            "stream",
            "demand_mbps",
            "delivered_mbps",
            "delivery_ratio",
            "delivery_percentage"
        ]
    ]
)


print(
    "Single-Path Stage 1 commodity delivery ratios:"
)

display(
    single_fairness_df[
        [
            "commodity_id",
            "source",
            "stream",
            "demand_mbps",
            "delivered_mbps",
            "delivery_ratio",
            "delivery_percentage"
        ]
    ]
)
```

```
Splittable Stage 1 commodity delivery ratios:
```

	commodity_id	source	stream	demand_mbps	delivered_mbps	delivery_ratio
0	S1_UK-L	S1	UK-L	6.0	6.0	1.000000
1	S1_UK-T	S1	UK-T	7.0	7.0	1.000000
2	S1_US-L	S1	US-L	5.0	0.0	0.000000
3	S1_US-T	S1	US-T	6.0	0.0	0.000000
4	S2_UK-L	S2	UK-L	5.0	0.0	0.000000
5	S2_UK-T	S2	UK-T	6.0	6.0	1.000000
6	S2_US-L	S2	US-L	5.0	5.0	1.000000
7	S2_US-T	S2	US-T	5.0	0.0	0.000000
8	S3_UK-L	S3	UK-L	5.0	0.0	0.000000
9	S3_UK-T	S3	UK-T	7.0	2.0	0.285714
10	S3_US-L	S3	US-L	6.0	0.0	0.000000
11	S3_US-T	S3	US-T	6.0	0.0	0.000000
12	B_UK-L	B	UK-L	6.0	6.0	1.000000
13	B_UK-T	B	UK-T	14.0	14.0	1.000000
14	B_US-L	B	US-L	8.0	8.0	1.000000
15	B_US-T	B	US-T	9.0	9.0	1.000000
16	C_UK-L	C	UK-L	4.0	4.0	1.000000
17	C_UK-T	C	UK-T	5.0	5.0	1.000000
18	C_US-L	C	US-L	5.0	5.0	1.000000
19	C_US-T	C	US-T	6.0	6.0	1.000000

```
Single-Path Stage 1 commodity delivery ratios:
```

	commodity_id	source	stream	demand_mbps	delivered_mbps	delivery_ratio
0	S1_UK-L	S1	UK-L	6.0	0.0	0.000000
1	S1_UK-T	S1	UK-T	7.0	4.0	0.571429
2	S1_US-L	S1	US-L	5.0	5.0	1.000000
3	S1_US-T	S1	US-T	6.0	6.0	1.000000
4	S2_UK-L	S2	UK-L	5.0	5.0	1.000000
5	S2_UK-T	S2	UK-T	6.0	6.0	1.000000
6	S2_US-L	S2	US-L	5.0	0.0	0.000000
7	S2_US-T	S2	US-T	5.0	0.0	0.000000
8	S3_UK-L	S3	UK-L	5.0	0.0	0.000000
9	S3_UK-T	S3	UK-T	7.0	7.0	1.000000
10	S3_US-L	S3	US-L	6.0	0.0	0.000000
11	S3_US-T	S3	US-T	6.0	3.0	0.500000
12	B_UK-L	B	UK-L	6.0	6.0	1.000000
13	B_UK-T	B	UK-T	14.0	14.0	1.000000
14	B_US-L	B	US-L	8.0	8.0	1.000000
15	B_US-T	B	US-T	9.0	2.0	0.222222
16	C_UK-L	C	UK-L	4.0	1.0	0.250000
17	C_UK-T	C	UK-T	5.0	5.0	1.000000
18	C_US-L	C	US-L	5.0	5.0	1.000000
19	C_US-T	C	US-T	6.0	6.0	1.000000

44.2.3 Detailed Results Analysis

Detailed Results Analysis

The commodity-level delivery ratios expose information that is hidden by the network-wide throughput result.

At the aggregate level, the baseline network delivers:

$$83 \text{ Mbps}$$

from:

$$126 \text{ Mbps}$$

of offered demand.

Therefore, the network-wide delivery ratio is:

$$\rho_{network} = \frac{83}{126} \approx 0.6587.$$

This means approximately:

$$65.87\%$$

of total offered traffic is delivered.

However, this aggregate value does **not** imply that every commodity receives approximately 65.87% of its demand.

The splittable Stage 1 solution instead produces a highly uneven admission pattern.

Earlier results showed:

- 12 commodities are fully served;
- 7 commodities receive zero service;
- 1 commodity is partially served.

Therefore, the optimizer is concentrating available bandwidth on selected commodities rather than distributing the available 83 Mbps proportionally across all 20 commodities.

That behavior is mathematically consistent with the Stage 1 objective:

$$\max \sum_{s,k} z_{s,k}.$$

Every delivered Mbps contributes exactly the same amount to the objective.

The model therefore has no inherent preference between:

- giving 50% service to two commodities;

and:

- giving 100% service to one and 0% to another;

if both alternatives produce the same total throughput.

This is an important distinction between:

$$\boxed{\text{Efficiency}}$$

and:

$$\boxed{\text{Fairness}}$$

The baseline Stage 1 model optimizes efficiency but does not explicitly optimize fairness.

44.2.4 Business Insights

Business Insights

A network-level KPI showing:

$$65.87\%$$

delivery may appear reasonable while some individual commodities receive:

$$0\%$$

service.

If commodities represent customers, locations, or applications, this could be commercially unacceptable.

A production dashboard should therefore report more than total throughput.

Recommended fairness-oriented KPIs include:

- minimum commodity delivery ratio;
- median commodity delivery ratio;
- average commodity delivery ratio;
- number of fully served commodities;
- number of partially served commodities;
- number of zero-service commodities;
- delivery ratio by source;
- delivery ratio by traffic class.

This changes the analytical question from:

> **How much traffic was delivered?**

to:

> **Who received the service and who did not?**

In [522…

```
# ------------------------------------------------------------
# 44.3 Summarize commodity-level fairness outcomes
# ------------------------------------------------------------

def summarize_fairness(
    fairness_df,
    model_name
):

    summary = {
        "model":
            model_name,

        "minimum_delivery_ratio":
            float(
                fairness_df[
                    "delivery_ratio"
                ].min()
            ),

        "median_delivery_ratio":
            float(
                fairness_df[
                    "delivery_ratio"
                ].median()
            ),

        "mean_delivery_ratio":
            float(
                fairness_df[
                    "delivery_ratio"
                ].mean()
            ),

        "fully_served_commodities":
            int(
                (
                    fairness_df[
                        "delivery_ratio"
                    ]
                    >= 1 - 1e-6
                ).sum()
            ),

        "partially_served_commodities":
            int(
                (
                    (
                        fairness_df[
                            "delivery_ratio"
                        ]
```

```
                        > 1e-6
                    )
                    &
                    (
                        fairness_df[
                            "delivery_ratio"
                        ]
                        < 1 - 1e-6
                    )
                ).sum()
            ),

        "zero_service_commodities":
            int(
                (
                    fairness_df[
                        "delivery_ratio"
                    ]
                    <= 1e-6
                ).sum()
            )
    }

    return summary


fairness_summary_df = pd.DataFrame(
    [
        summarize_fairness(
            split_fairness_df,
            "Splittable Stage 1"
        ),

        summarize_fairness(
            single_fairness_df,
            "Single-Path Stage 1"
        )
    ]
)


fairness_summary_df
```

Out [522…

	model	minimum_delivery_ratio	median_delivery_ratio	mean_delivery_ratio
0	Splittable Stage 1	0.0	1.0	0.614286
1	Single-Path Stage 1	0.0	1.0	0.627183

44.3.2 — Results Interpretation

Results Interpretation

The fairness summary provides a much richer view than total throughput alone.

In particular, the minimum delivery ratio identifies the worst-served commodity:

$$\alpha = \min_{s,k} r_{s,k}.$$

If the output shows:

$$\alpha = 0,$$

then at least one commodity receives no service.

Based on the baseline results, this is expected.

For the splittable Stage 1 solution, seven commodities receive:

$$0 \text{ Mbps}.$$

Therefore:

$$\alpha = 0.$$

This means the Stage 1 solution is:

- throughput efficient;
- not max-min fair.

The optimizer has successfully found the largest feasible aggregate throughput, but it has not protected the worst-served commodity.

44.3.3 MARKDOWN CELL — Why This Happens

Why This Happens

There is no penalty associated with setting:

$$z_{s,k} = 0$$

for one commodity if doing so allows scarce capacity to be used more effectively elsewhere.

The Stage 1 formulation does not contain:

$$z_{s,k} \geq r_{min} D_{s,k}$$

and does not optimize:

$$\min_{s,k} \frac{z_{s,k}}{D_{s,k}}.$$

Therefore, starvation is mathematically permitted.

44.3.4 MARKDOWN CELL — Business Impact

Business Impact

A minimum service ratio of zero may be unacceptable in a production environment.

The organization must decide whether:

- zero service is ever permissible;
- every commodity deserves a minimum guarantee;
- only selected priority commodities require protection.

Possible policies include:

Hard Minimum Service

$$r_{s,k} \geq r_{min}.$$

Max-Min Fairness

Maximize:

$$\alpha = \min_{s,k} r_{s,k}.$$

Priority-Based Protection

Guarantee minimum service only to selected traffic classes or customers.

The correct approach is a business-policy decision rather than a purely mathematical decision.

44.4 Visualize Commodity Delivery Ratios

44.4.1 CODE CELL

In [524…

```
# ------------------------------------------------------------
# 44.4 Compare commodity delivery ratios
# ------------------------------------------------------------

commodity_fairness_comparison_df = (
    split_fairness_df[
        [
            "commodity_id",
            "delivery_percentage"
        ]
    ]
    .rename(
        columns={
            "delivery_percentage":
                "split_delivery_percentage"
        }
    )
    .merge(
        single_fairness_df[
            [
                "commodity_id",
                "delivery_percentage"
            ]
        ]
        .rename(
            columns={
                "delivery_percentage":
                    "single_delivery_percentage"
            }
        ),
        on="commodity_id",
        how="outer"
    )
)


commodity_fairness_comparison_df = (
    commodity_fairness_comparison_df
    .sort_values(
        "commodity_id"
    )
)


commodity_fairness_comparison_df.set_index(
    "commodity_id"
)[
    [
        "split_delivery_percentage",
        "single_delivery_percentage"
```

```
    ]
].plot(
    kind="bar",
    figsize=(15, 6)
)


plt.ylabel(
    "Delivered Demand (%)"
)

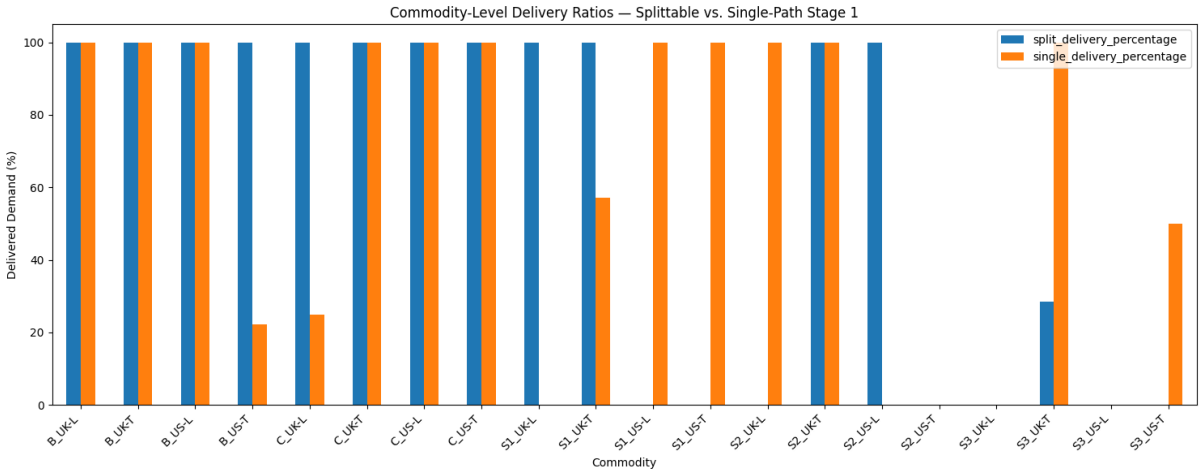
plt.xlabel(
    "Commodity"
)

plt.title(
    "Commodity-Level Delivery Ratios "
    "— Splittable vs. Single-Path Stage 1"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()

plt.show()
```



44.4.2 MARKDOWN CELL — Visualization Interpretation

Visualization Interpretation

The commodity-level bar chart makes the distributional difference between the two throughput-optimal solutions immediately visible.

Both formulations deliver:

$$83 \text{ Mbps}$$

in total.

However, individual commodity service levels may differ substantially.

This demonstrates that:

$$\boxed{\text{Equal Network Throughput} \neq \text{Equal Customer / Commodity Experience}}$$

The chart is particularly useful for identifying:

- completely dropped commodities;
- partially served commodities;
- commodities whose treatment differs significantly between routing policies.

44.4.3 Business Insight

Business Insight

This visualization is closer to the perspective a customer-facing business would need.

Executives may care about total network utilization, but customers experience service at the commodity level.

A production decision should therefore combine:

- aggregate efficiency metrics;
- distributional service metrics.

44.5 Lexicographic Fairness Optimization

44.5.1 Recommended Formulation

44.5 Lexicographic Fairness Optimization

A strong production formulation can extend the existing optimization hierarchy.

Stage 1 — Maximize Total Throughput

First solve:

$$T^* = \max \sum_{s,k} z_{s,k}.$$

This preserves the network's maximum feasible service level.

Stage 2 — Maximize Fairness

Fix:

$$\sum_{s,k} z_{s,k} = T^*,$$

and maximize the minimum commodity delivery ratio.

Define:

$$\alpha = \min_{s,k} \frac{z_{s,k}}{D_{s,k}}.$$

This can be implemented using:

$$z_{s,k} \geq \alpha D_{s,k} \qquad \forall (s,k).$$

Then solve:

$$\max \alpha.$$

Stage 3 — Minimize Sensitive Latency

Fix:

- throughput at T^*;
- fairness at α^*;

then minimize latency exposure for latency-sensitive traffic.

Stage 4 — Operational Cleanup

Optionally minimize:

- tolerant-traffic latency;
- route count;
- unnecessary path splitting;
- internal circulation;
- route churn.

The resulting hierarchy is:

$$\boxed{\text{Throughput} \rightarrow \text{Fairness} \rightarrow \text{Sensitive Latency} \rightarrow \text{Operational Cleanup}}$$

44.5.2 Deep Interpretation

The network already has a clear physical scarcity condition:

$$126 > 83.$$

Therefore, some demand must be rejected.

The real decision is not whether the network can serve all demand.

It cannot.

The real decision is:

> **How should unavoidable scarcity be allocated?**

Lexicographic optimization provides a transparent answer.

First

Do not waste physically usable capacity.

Preserve:

$$83 \text{ Mbps}.$$

Second

Among all 83-Mbps solutions, improve the worst-served commodities.

Third

Among throughput- and fairness-optimal solutions, improve sensitive latency.

This separates three fundamentally different questions:

1. How much can be served?
2. Who should receive scarce service?
3. How should admitted traffic be routed?

That decomposition improves auditability and governance.

44.5.3 MARKDOWN CELL — Business Impact

Business Impact

A lexicographic hierarchy is easier to explain to decision-makers than an arbitrary weighted objective.

For example:

1. preserve all feasible throughput;
2. avoid starving customers;
3. improve latency where possible;
4. simplify operations.

Each stage reflects a distinct organizational priority.

44.6 Minimum SLA Delivery Ratio

44.6.1 MARKDOWN CELL

44.6 Minimum SLA Delivery Ratio

Fairness can also be expressed as a hard service guarantee.

For a required minimum delivery ratio:

$$r_{min},$$

the model can enforce:

$$z_{s,k} \geq r_{min} D_{s,k} \qquad \forall (s,k).$$

For example:

$$r_{min} = 0.50$$

requires every commodity to receive at least 50% of its offered demand.

This turns fairness from a preference into a service requirement.

44.6.2 MARKDOWN CELL — Feasibility Caveat

Feasibility Caveat

A hard SLA may make the optimization infeasible.

For example, the network may not have enough end-to-end capacity to guarantee:

$$90\%$$

service to every commodity.

A useful analysis would therefore determine the largest common feasible service ratio:

$$r_{max}^{SLA}.$$

This represents the strongest uniform guarantee that the current network can physically support.

44.6.3 MARKDOWN CELL — Business Impact

Business Impact

This creates a direct connection between:

- network infrastructure;
- contractual service promises.

If the required SLA exceeds physical feasibility, the organization must change at least one of:

- capacity;
- demand;
- traffic priority;
- contractual guarantee.

44.7 Weighted Traffic Priorities

44.7.1 MARKDOWN CELL

44.7 Weighted Traffic Priorities

The organization may not consider every delivered Mbps equally valuable.

Let:

$$w_k$$

represent the business priority of stream k.

The objective could become:

$$\max \sum_{s,k} w_k z_{s,k}.$$

For example, the organization may choose:

$$w_L > w_T,$$

where:

- w_L is the weight assigned to latency-sensitive traffic;
- w_T is the weight assigned to tolerant traffic.

This would make sensitive traffic more valuable during Stage 1 admission.

44.7.2 Caveat

Weights directly encode business preferences.

Therefore, they should not be selected merely because they produce an attractive output.

They require policy justification.

Different weights can change:

- commodity admission;
- source-level allocation;
- total throughput.

The weighting scheme should therefore be documented and governed.

44.8 Why Weighted Sums Should Not Automatically Replace Lexicographic Objectives

44.8.1 Weighted Sums vs. Lexicographic Objectives

A single combined objective could be written as:

$$\max \left[\lambda_T T + \lambda_F \alpha - \lambda_L LE\right].$$

However, this creates implicit exchange rates between:

- throughput;
- fairness;
- latency.

The model might effectively decide that losing one Mbps of throughput is worthwhile for a particular latency improvement.

Unless those tradeoffs are explicitly approved by the business, they may be difficult to defend.

Therefore, lexicographic optimization is preferable when priority order matters more than smooth tradeoffs.

44.9 Traffic-Class Minimum Service Policies

44.9.1 MARKDOWN CELL

44.9 Traffic-Class Minimum Service Policies

Minimum service does not need to be identical for every stream.

For latency-sensitive traffic:

$$z_{s,k} \geq r_L D_{s,k}, \qquad k \in K_L.$$

For tolerant traffic:

$$z_{s,k} \geq r_T D_{s,k}, \qquad k \in K_T.$$

A policy could set:

$$r_L > r_T.$$

This gives latency-sensitive traffic stronger admission protection.

44.9.2 Key Insight

Stage 2 can only improve latency for traffic admitted during Stage 1.

If Stage 1 drops sensitive traffic entirely, Stage 2 cannot recover it.

Therefore:

$$\boxed{\text{Admission Policy} \neq \text{Routing Quality Policy}}$$

Both must be considered explicitly.

44.10 Source-Level Fairness

Fairness can also be measured at the source level.

Let:

$$D_s = \sum_k D_{s,k}$$

and:

$$z_s = \sum_k z_{s,k}.$$

Define:

$$r_s = \frac{z_s}{D_s}.$$

This measures the percentage of total source demand served.

Source-level fairness is useful when source nodes represent:

- customers;
- facilities;
- geographic markets;
- organizations.

In [526…

```
# ------------------------------------------------------------
# 44.10 Calculate source-level delivery ratios
# ------------------------------------------------------------

split_source_fairness_df = (
    split_stage1_source_summary
    .copy()
)


single_source_fairness_df = (
    single_stage1_source_summary
    .copy()
)


split_source_fairness_df[
    "source_delivery_ratio"
] = (
    split_source_fairness_df[
        "delivered_mbps"
    ]
    /
    split_source_fairness_df[
        "demand_mbps"
    ]
)
```

```
single_source_fairness_df[
    "source_delivery_ratio"
] = (
    single_source_fairness_df[
        "delivered_mbps"
    ]
    /
    single_source_fairness_df[
        "demand_mbps"
    ]
)


split_source_fairness_df[
    "source_delivery_percentage"
] = (
    100
    *
    split_source_fairness_df[
        "source_delivery_ratio"
    ]
)


single_source_fairness_df[
    "source_delivery_percentage"
] = (
    100
    *
    single_source_fairness_df[
        "source_delivery_ratio"
    ]
)


print(
    "Splittable source-level fairness:"
)

display(
    split_source_fairness_df
)


print(
    "Single-path source-level fairness:"
)

display(
    single_source_fairness_df
)
```

Splittable source-level fairness:

	source	demand_mbps	delivered_mbps	lost_mbps	delivery_percentage	sou
0	S1	24.0	13.0	11.0	54.166667	
1	S2	21.0	11.0	10.0	52.380952	
2	S3	24.0	2.0	22.0	8.333333	
3	B	37.0	37.0	0.0	100.000000	
4	C	20.0	20.0	0.0	100.000000	

Single-path source-level fairness:

	source	demand_mbps	delivered_mbps	lost_mbps	delivery_percentage	sou
0	S1	24.0	15.0	9.0	62.500000	
1	S2	21.0	11.0	10.0	52.380952	
2	S3	24.0	10.0	14.0	41.666667	
3	B	37.0	30.0	7.0	81.081081	
4	C	20.0	17.0	3.0	85.000000	

44.10.3 Detailed Results Analysis

The earlier Stage 1 results show that source-level service differs significantly even though both routing formulations deliver:

$$83 \text{ Mbps}.$$

One important example is S3.

Splittable Stage 1

S3 receives approximately:

$$2 \text{ Mbps}$$

from:

$$24 \text{ Mbps}$$

of offered demand.

Therefore:

$$r_{S3}^{split} = \frac{2}{24} \approx 8.33\%.$$

Single-Path Stage 1

S3 receives approximately:

$$10 \text{ Mbps}$$

from the same:

$$24 \text{ Mbps}$$

of offered demand.

Therefore:

$$r_{S3}^{single} = \frac{10}{24} \approx 41.67\%.$$

This is a substantial difference in source-level service.

Yet both models still produce exactly:

$$83 \text{ Mbps}$$

of total network throughput.

Therefore:

$$\boxed{\text{Equal Network Efficiency} \neq \text{Equal Source Experience}}$$

44.10.4 Business Insight

If S3 represented a customer or regional market, the difference between approximately:

$$8.3\%$$

and:

$$41.7\%$$

delivery would be commercially significant.

This reinforces the need to report network performance at multiple levels:

- total network;

- destination;
- traffic class;
- source;
- commodity.

A single network-wide KPI can hide substantial distributional differences.

44.11 Fairness vs. Efficiency Tradeoff

44.11.1 Fairness vs. Efficiency Tradeoff

Fairness constraints may reduce some of the optimizer's flexibility.

A fairness-oriented solution could potentially require:

- longer routes;
- more capacity consumption;
- reduced throughput;
- higher latency.

Therefore, fairness should not be treated as costless.

A useful conceptual frontier compares:

Efficiency

$$T$$

with:

Fairness

$$\alpha = \min_{s,k} r_{s,k}.$$

44.11.2 MARKDOWN CELL — Business Decision Interpretation

Business Decision Interpretation

The correct question is not:

> **Should throughput or fairness be optimized?**

The more useful question is:

> **How much aggregate throughput, if any, should the**

> **organization trade for materially better minimum service?**

For example, imagine a future result of:

- 83 Mbps with highly uneven service;
- 82 Mbps with much stronger minimum service.

Management would then need to determine whether:

$$1\ \text{Mbps}$$

of throughput is worth more than the improvement in customer-level service.

The optimization can quantify that tradeoff.

The business must decide which point is acceptable.

44.12 Recommended Production Objective Hierarchy

Based on the findings of this assessment, a production optimization hierarchy could be:

Stage 1 — Maximize Throughput

$$\max T$$

Stage 2 — Enforce Policy / Improve Fairness

Examples:

$$\max \alpha$$

or:

$$r_{s,k} \geq r_{min}.$$

Stage 3 — Minimize Sensitive Latency

$$\min LE_{sensitive}.$$

Stage 4 — Operational Cleanup

Optionally minimize:

- tolerant latency;

- unnecessary route count;
- unnecessary splitting;
- route churn;
- internal network usage.

Therefore:

$$\text{Throughput} \rightarrow \text{Policy/Fairness} \rightarrow \text{Sensitive Latency} \rightarrow \text{Operational Cleanu}$$

44.12.1 Why Stage 4 Matters

The current Stage 2 objective does not penalize tolerant-traffic latency.

As a result, tolerant traffic may use long paths when doing so helps sensitive traffic.

Those routes are mathematically valid but may be operationally undesirable.

A cleanup stage can preserve:

- maximum throughput;
- fairness;
- sensitive-latency optimum;

while reducing unnecessary routing complexity.

44.13 Governance and Auditability

Fairness rules and service priorities represent organizational policy.

They should therefore be documented explicitly.

A production optimization system should be able to report:

- offered demand;
- delivered demand;
- delivery ratio;
- priority class;
- SLA requirement;
- unmet demand;
- reason for unmet demand.

This improves explainability for:

- engineering;
- operations;
- business owners;
- governance teams.

The optimizer should not function as an unexplained black box when it is deciding which demand receives scarce capacity.

44.14 Detailed Section Summary

Section 44 extends the network problem from physical optimization into service-policy optimization.

1. Throughput Alone Does Not Encode Fairness

The Stage 1 objective maximizes:

$$\sum_{s,k} z_{s,k}$$

but does not require balanced service.

2. Commodity Delivery Ratio Provides a Normalized Metric

$$r_{s,k} = \frac{z_{s,k}}{D_{s,k}}.$$

3. The Baseline Contains Zero-Service Commodities

Therefore:

$$\alpha = \min_{s,k} r_{s,k} = 0$$

for the throughput-only solution.

4. Equal Throughput Can Produce Very Different Source Outcomes

The S3 example demonstrates that network-level equality can hide substantial source-level differences.

5. Max-Min Fairness Can Protect the Worst-Served Commodity

After throughput is fixed, maximize:

$$\alpha.$$

6. SLA Constraints Can Create Explicit Guarantees

For example:

$$r_{s,k} \geq r_{min}.$$

7. Traffic Priorities Can Be Explicitly Modeled

Latency-sensitive traffic can receive stronger admission protection.

8. A Recommended Hierarchy Is

$$\text{Throughput} \rightarrow \text{Fairness / SLA} \rightarrow \text{Sensitive Latency} \rightarrow \text{Operational Cleanu}$$

44.14.1 Business Conclusion

The baseline network does not merely have a capacity problem.

It also has an **allocation problem caused by scarcity**.

Because:

$$126 > 83,$$

some demand must remain unserved.

Therefore, a production optimizer must answer both:

> **How much demand can be delivered?**

and:

> **Who should receive the limited capacity?**

That second question is fundamentally a business-policy question.

The optimization framework should therefore distinguish:

- physical feasibility;
- organizational policy;
- quality of service;
- operational simplicity.

44.15 Connection to Section 45

The analysis has now examined the network from several perspectives:

- feasibility;
- throughput;
- route structure;
- latency;
- infrastructure utilization;
- marginal capacity value;
- stress scenarios;
- fairness;
- service policy.

Section 45 consolidates these findings into an executive-level set of conclusions and recommendations.

The objective is to answer:

> **What are the most important decisions that should be made from the analysis?**

44.16 Decision Synthesis Before Executive Findings

The preceding analysis changes the routing decision from a simple comparison of:

> **splittable vs. single-path**

into a broader network-management decision.

Several findings now need to be considered together.

Physical Capacity

The baseline network offers:

$$126 \text{ Mbps}$$

of demand but can deliver only:

$$83 \text{ Mbps}.$$

Therefore:

$$43 \text{ Mbps}$$

of baseline demand cannot be served even under optimized routing.

Routing Architecture

The splittable and single-path formulations achieve identical Stage 1 throughput in:

- the baseline;
- low demand;
- high demand;
- internal-corridor degradation;
- destination-exit degradation;
- fragmented-capacity conditions.

Therefore, commodity splitting is not required to achieve maximum Stage 1 throughput in any tested scenario.

Service Quality

Although splitting does not improve Stage 1 throughput, the final splittable solution produces lower observed sensitive latency.

Within the split pipeline, Stage 2 reduces sensitive latency exposure from:

$$850$$

to:

$$644 \text{ Mbps} \cdot \text{ms},$$

a reduction of approximately:

$$24.24\%.$$

Therefore, routing flexibility has demonstrated service-quality value even when it does not increase total delivered Mbps.

Infrastructure

Marginal-capacity analysis shows that several links have immediate end-to-end

throughput value when expanded.

The strongest observed local candidates include:

- B → D;
- C → UK;
- C → US.

This demonstrates that physical infrastructure, rather than lack of commodity splitting, is the primary baseline throughput limitation.

Fairness

Maximum throughput does not guarantee balanced service.

The fairness analysis shows that the minimum commodity delivery ratio is:

$$0.$$

Therefore, at least one commodity receives no service under each Stage 1 throughput-only solution.

This means the production problem includes both:

- a capacity-allocation problem;
- a routing problem.

Integrated Decision

The resulting business decision is not simply:

> **Which routing formulation wins?**

The more complete question is:

> **How should routing architecture, infrastructure investment, latency optimization, and service-allocation policy work together?**

The evidence supports the following hierarchy:

$$\text{Physical Feasibility} \rightarrow \text{Fairness / SLA Policy} \rightarrow \text{Sensitive Latency} \rightarrow \text{Opera}$$

This provides the analytical bridge into the executive recommendations in Section 45.

45. Executive Findings and Recommendations

45.1 Executive Overview

This assessment evaluates routing and capacity allocation on a synthetic nine-node network containing:

- 11 physical full-duplex links;
- 22 directed arcs;
- 20 source-stream commodities;
- 126 Mbps of total offered demand.

The analysis compares:

- naive independent shortest-path routing;
- splittable optimization;
- single-path optimization;
- Stage 1 throughput maximization;
- Stage 2 sensitive-latency optimization;
- infrastructure utilization;
- marginal capacity value;
- stress scenarios;
- fairness and service-policy extensions.

The central baseline result is:

$$\boxed{T^*_{split} = T^*_{single} = 83 \text{ Mbps}}$$

from:

$$126 \text{ Mbps}$$

of offered demand.

Therefore:

$$43 \text{ Mbps}$$

remains undelivered.

Baseline delivery rate is:

$$\frac{83}{126} \times 100 \approx 65.87\%.$$

45.1.1 Executive Interpretation of the Baseline Capacity Ceiling

The value:

$$83 \text{ Mbps}$$

should not be interpreted as merely a solver result.

It is the maximum amount of current demand that the network can deliver while satisfying all modeled capacity constraints.

The service deficit is:

$$126 - 83 = 43 \text{ Mbps}.$$

Therefore:

$$34.13\%$$

of baseline offered demand cannot be delivered.

This establishes that the baseline problem cannot be solved through routing improvements alone.

Even optimal routing cannot deliver the missing 43 Mbps.

The organization would need some combination of:

- additional physical capacity;
- traffic prioritization;
- demand management;
- service-policy changes.

45.2 Finding 1 — Naive Independent Shortest-Path Routing Is Infeasible

Routing commodities independently on their shortest paths produces severe link overload.

Examples include approximately:

- B → C: 61 / 18 Mbps ≈ 338.9%;

- C → US: 61 / 25 Mbps ≈ 244.0%;
- C → UK: 65 / 30 Mbps ≈ 216.7%;
- A → C: 45 / 25 Mbps = 180.0%.

Therefore:

> **Per-commodity shortest paths cannot be selected independently in a shared-capacity network.**

Traffic decisions must be optimized jointly.

Why This Happens

Shortest-path routing optimizes each commodity locally.

It does not account for the fact that many commodities compete for the same physical links.

For example:

$$B \to C$$

has only:

$$18 \text{ Mbps}$$

of capacity,

yet naive routing attempts to send approximately:

$$61 \text{ Mbps}$$

through it.

This is not a small constraint violation.

It demonstrates that local routing choices collectively create a globally infeasible plan.

Business Insight

This principle extends beyond networking.

Whenever multiple decisions share constrained resources:

> **Local optimization can produce globally infeasible outcomes.**

Centralized capacity-aware optimization therefore provides value even before more advanced objectives such as latency or fairness are introduced.

45.3 Finding 2 — Maximum Baseline Throughput Is 83 Mbps

Both routing formulations establish the same physical service ceiling:

$$T^* = 83 \text{ Mbps}.$$

This produces:

$$43 \text{ Mbps}$$

of unmet demand and a delivery rate of:

$$65.87\%.$$

This is the baseline network's maximum feasible throughput under the current topology and capacities.

Business Impact

The 83-Mbps value becomes the key baseline capacity KPI.

Any routing policy delivering less than 83 Mbps is leaving feasible service unused.

Any attempt to deliver materially more than 83 Mbps requires a physical or demand-side change rather than routing changes alone.

45.4 Finding 3 — Splitting Does Not Increase Baseline Throughput

The feasible regions satisfy:

$$\mathcal{F}_{single} \subseteq \mathcal{F}_{split}.$$

Therefore:

$$T^*_{single} \leq T^*_{split}.$$

In the baseline network:

$$T^*_{single} = T^*_{split} = 83.$$

The inequality holds with equality.

Therefore:

> **The single-path restriction is not throughput-binding in the baseline network.**

The split Stage 1 solution also does not require any commodity to use multiple positive-flow routes.

Therefore, actual commodity splitting is not necessary to reach the 83-Mbps baseline maximum.

Deeper Interpretation

The splittable model contains more routing freedom than the single-path model.

Yet that additional freedom creates:

$$0 \text{ Mbps}$$

of additional baseline throughput.

This is analytically important because it prevents unnecessary complexity.

The correct conclusion is not:

> **Splitting has no value.**

The correct conclusion is:

> **Splitting has no baseline Stage 1 throughput value.**

Its value may instead appear in:

- latency;
- degraded states;
- capacity fragmentation;
- resilience.

Business Impact

If maximum baseline throughput were the only objective, single-path routing would be sufficient.

This favors operational simplicity unless additional benefits justify the complexity

of multipath routing.

45.5 Finding 4 — Equal Throughput Does Not Mean Equal Service

Although both formulations deliver:

$$83 \text{ Mbps},$$

they do not necessarily serve the same commodities.

Therefore:

$$\boxed{\text{Equal Objective Value} \neq \text{Equal Optimization Solution}}$$

Differences appear in:

- commodity admission;
- source-level service;
- stream-level service;
- link utilization.

For example, the split Stage 1 solution delivers approximately:

$$16 \text{ Mbps}$$

of UK-L traffic,

while the single-path Stage 1 solution delivers approximately:

$$12 \text{ Mbps}.$$

45.5.2 MARKDOWN CELL — Business Impact

Business Impact

Aggregate network KPIs should be supplemented by:

- source service;
- traffic-class service;
- customer service;
- SLA attainment.

Otherwise, materially different customer outcomes may remain hidden behind the same throughput number.

45.6 Finding 5 — Splittable Stage 2 Produces Better Observed Sensitive Latency

The final split solution produces lower observed average latency for both sensitive traffic classes.

UK-L

Splittable:

$$18.25 \text{ ms}$$

Single path:

$$21.17 \text{ ms}$$

Approximate reduction:

$$13.8\%.$$

US-L

Splittable:

$$19.56 \text{ ms}$$

Single path:

$$24.78 \text{ ms}$$

Approximate reduction:

$$21.1\%.$$

Analytical Caveat

The Stage 1 admission vectors differ between the two formulations.

Therefore:

$$\text{Observed Latency Difference} = \text{Admission-Mix Effect} + \text{Routing-Flexibility}$$

The correct statement is:

> **The complete splittable optimization pipeline produces better observed sensitive-latency outcomes.**

The raw difference should not be interpreted as a perfectly controlled causal

estimate of splitting alone.

Business Interpretation

These latency improvements occur without sacrificing the 83-Mbps Stage 1 throughput ceiling.

This means routing flexibility can improve the quality of admitted service even when it cannot improve the total quantity delivered.

That distinction matters for latency-sensitive applications where response time affects:

- user experience;
- transaction completion;
- real-time control;
- SLA compliance.

45.7 Finding 6 — Stage 2 Meaningfully Improves Sensitive Latency Exposure

Within the splittable pipeline, Stage 2 preserves the exact Stage 1 commodity admissions and minimizes sensitive latency.

Sensitive latency exposure decreases from:

$$850$$

to:

$$644 \text{ Mbps} \cdot \text{ms}.$$

Absolute reduction:

$$206.$$

Relative reduction:

$$\frac{206}{850} \times 100 \approx 24.24\%.$$

Deep Results Analysis

Latency exposure combines:

- the volume of sensitive traffic;
- the latency experienced by that traffic.

Therefore, the approximately:

$$24.24\%$$

reduction is stronger evidence than simply showing that one selected path became shorter.

Importantly, Stage 2 preserves the exact Split Stage 1 admission vector.

Therefore, unlike the raw split-vs-single comparison, the Stage 1-to-Stage 2 improvement is a clean routing-quality improvement.

The same sensitive traffic is being served.

Only its routing changes.

Business Impact

The two-stage optimization architecture successfully separates:

1. how much traffic is admitted;
2. how well the admitted traffic is routed.

This is highly desirable in production because service quantity can be protected before optimizing service quality.

45.8 Finding 7 — The D Corridor Is Structurally Important

Naive shortest routing places excessive traffic through the C corridor.

Optimized routing makes substantial use of:

$$B \to D,$$

followed by:

- D → UK;
- D → US.

The D corridor therefore provides essential alternate transport capacity.

Without it, more traffic would compete for:

- B → C;
- C → UK;
- C → US.

Business Insight

Longer alternate routes may have significant strategic value.

The lowest-latency path is not necessarily the most valuable path when many commodities share constrained infrastructure.

Route diversity itself can be an important network asset.

45.9 Finding 8 — Saturation Does Not Equal Investment Priority

Several directed links operate at:

$$100\%$$

utilization in one or more optimized states.

However:

$$\boxed{\text{Saturated Link} \neq \text{True Throughput Bottleneck}}$$

Section 41 defines marginal capacity value as:

$$MV_e = T^*(C_e + 1) - T^*(C_e).$$

Only links satisfying:

$$MV_e > 0$$

demonstrate immediate local throughput value.

Investment Insight

A utilization dashboard answers:

> **Which links are busy?**

Marginal-capacity analysis answers:

> **Where would additional bandwidth create more service?**

The second question is closer to an investment decision.

A saturated link with:

$$MV_e = 0$$

may not justify immediate stand-alone expansion.

A link with:

$$MV_e > 0$$

has demonstrated measurable local throughput value.

45.10 Finding 9 — Important Capacity Candidates Have Different Structural Roles

The executed Section 41 perturbation analysis should be treated as authoritative for identifying positive marginal-value links.

The splittable baseline is expected to highlight links such as:

- B → D;
- C → UK;
- C → US.

These links represent two different bottleneck categories.

Internal-Corridor Constraint

B → D increases access to the alternate D corridor.

Destination-Boundary Constraints

C → UK and C → US determine how much traffic can leave the network through C.

Therefore, infrastructure planning must evaluate both:

- internal transport capacity;
- final exit capacity.

45.11 Finding 10 — Unused Capacity Can Be Stranded

The network may contain spare link capacity while still losing demand.

This is not contradictory.

Useful network capacity must exist along a complete feasible source-to-destination route.

Therefore:

$$\boxed{\text{Total Spare Capacity} \neq \text{Additional Deliverable Throughput}}$$

Unused capacity on an isolated link may be unusable because:

- the next link is full;
- the destination exit is full;
- another corridor remains binding.

Business Insight

Capacity planning should be end-to-end.

The organization should avoid purchasing bandwidth simply because one local metric shows headroom or congestion.

The correct question is:

> **Does the complete path have enough capacity to convert the upgrade into delivered service?**

45.12 Finding 11 — Splittable Routing Does Not Increase Stage 1 Throughput in Any Tested Scenario

The scenario analysis extends the baseline split-vs-single comparison across several materially different network states.

The tested scenarios include:

- baseline demand;
- 60% demand;
- 150% demand;
- 70% internal-corridor capacity;
- 75% destination-exit capacity;

- fragmented internal capacity.

In every scenario:

$$T^*_{single} = T^*_{split}.$$

Therefore:

$$\Delta T = T^*_{split} - T^*_{single} = 0.$$

This is a stronger result than the baseline comparison alone.

It demonstrates that the one-path restriction is not throughput-binding across any of the tested operating states.

Why This Matters

The splittable formulation has the larger feasible region:

$$\mathcal{F}_{single} \subseteq \mathcal{F}_{split}.$$

However, the additional feasible points do not create a higher Stage 1 objective value in the tested scenarios.

The dominant throughput limitation is therefore the physical network rather than commodity path divisibility.

Business Impact

The current evidence does not justify multipath routing on the basis of Stage 1 throughput improvement.

The strongest demonstrated value of the splittable formulation is instead:

- lower sensitive latency;
- additional routing flexibility.

Operationally, this strengthens the case for single-path routing as the normal mode when:

- throughput requirements are satisfied;
- latency remains acceptable;
- simplicity is valuable.

Splittable routing should be retained as an optional capability rather than assumed to be universally superior.

45.13 Finding 12 — Fairness Must Be Explicitly Modeled

The baseline Stage 1 objective values every delivered Mbps equally.

Therefore, the optimizer may completely drop some commodities while still maximizing throughput.

A production model should consider:

- minimum delivery ratios;
- source fairness;
- traffic-class guarantees;
- SLA constraints;
- priority weights.

A recommended optimization hierarchy is:

$$\text{Throughput} \rightarrow \text{Policy/Fairness} \rightarrow \text{Sensitive Latency} \rightarrow \text{Operational Cleanu}$$

45.12.1 Finding — Destination-Facing Capacity Is the Most Severe Tested Degradation

The stress analysis shows materially different effects across capacity-degradation scenarios.

Baseline throughput is:

$$83 \text{ Mbps}.$$

Internal-corridor degradation reduces throughput to:

$$78.1 \text{ Mbps}.$$

Fragmented internal capacity reduces throughput to:

$$72.85 \text{ Mbps}.$$

Destination-exit degradation reduces throughput to:

$$69.25 \text{ Mbps}.$$

Therefore, among the tested capacity-degradation scenarios, the destination-exit scenario creates the largest throughput reduction:

$$83 - 69.25 = 13.75 \text{ Mbps}.$$

Interpretation

This reinforces the earlier conclusion that destination-facing capacity is structurally important.

Upstream links may retain available bandwidth, but that capacity cannot create end-to-end service when the final UK or US boundary is constrained.

Business Impact

Capacity planning should therefore evaluate the entire end-to-end corridor.

Increasing internal bandwidth without corresponding destination-facing capacity may create stranded infrastructure.

In this network, protecting and expanding exit capacity should receive strong consideration alongside the internal B → D corridor.

45.14 Recommended Production Routing Strategy

The results do not support declaring either routing formulation universally superior.

Single-Path Routing Is Attractive When

- maximum throughput is the primary requirement;
- operational simplicity matters;
- deterministic routes are desirable;
- baseline conditions dominate.

Splittable Routing Is Attractive When

- sensitive latency matters;
- capacity becomes fragmented;
- congestion increases;
- degradation creates a need for finer-grained routing flexibility.

Therefore, the strongest production architecture may be:

> **adaptive routing.**

Use single-path routing when it satisfies service requirements.

Retain splittable-routing capability for operating states where the additional flexibility produces measurable value.

Recommended Operating Model

Normal Conditions

Prefer single-path routing when:

- throughput requirements are satisfied;
- latency remains acceptable;
- network stress is low.

Benefits include:

- simpler monitoring;
- easier troubleshooting;
- deterministic routes;
- lower operational complexity.

Performance-Critical or Stressed Conditions

Enable split-routing flexibility when:

- sensitive latency becomes unacceptable;
- congestion increases;
- capacity becomes fragmented;
- infrastructure degradation occurs.

This balances performance with operational simplicity.

45.15 Recommended Production Optimization Hierarchy

A production optimizer could use:

Step 1 — Maximize Total Throughput

$$\max T.$$

Step 2 — Apply Fairness and SLA Policy

Examples:

$$\max \alpha$$

or:

$$r_{s,k} \geq r_{min}.$$

Step 3 — Minimize Sensitive Latency

Preserve throughput and policy outcomes while minimizing:

$$LE_{sensitive}.$$

Step 4 — Operational Cleanup

Minimize:

- tolerant latency;
- unnecessary active routes;
- route churn;
- internal circulation.

Step 5 — Monitor Infrastructure

Track:

- utilization;
- residual headroom;
- marginal capacity value.

Step 6 — Re-Optimize Under Stress

Evaluate:

- demand growth;
- degradation;
- failures;
- fragmentation.

45.16 Executive Scorecard

In [532…
```
# ------------------------------------------------------------
```

```
# Update / expand executive scorecard with completed
# scenario-analysis findings
# ------------------------------------------------------------

executive_scorecard_df = pd.DataFrame([
    {
        "dimension":
            "Total Offered Demand",
        "result":
            "126 Mbps",
        "business_implication":
            "Baseline demand exceeds feasible network capacity."
    },

    {
        "dimension":
            "Maximum Baseline Throughput",
        "result":
            "83 Mbps",
        "business_implication":
            "43 Mbps remains undelivered even under optimized routing.
    },

    {
        "dimension":
            "Baseline Delivery Rate",
        "result":
            "65.87%",
        "business_implication":
            "Approximately one-third of baseline demand cannot be serv
    },

    {
        "dimension":
            "Naive Minimum-Latency Routing",
        "result":
            "Infeasible",
        "business_implication":
            "Shared-capacity routing decisions must be optimized joint
    },

    {
        "dimension":
            "Split vs Single Baseline",
        "result":
            "83 vs 83 Mbps",
        "business_implication":
            "Splitting provides no baseline Stage 1 throughput advanta
    },

    {
        "dimension":
```

```
            "Split vs Single Stress Scenarios",
        "result":
            "Throughput tie in all tested scenarios",
        "business_implication":
            "Single-path restriction was not throughput-binding in the
    },

    {
        "dimension":
            "Low Demand Scenario",
        "result":
            "75.6 / 75.6 Mbps delivered",
        "business_implication":
            "The network can serve 100% of demand when load is suffici
    },

    {
        "dimension":
            "High Demand Scenario",
        "result":
            "83 / 189 Mbps delivered",
        "business_implication":
            "The network becomes capacity-limited rather than demand-l
    },

    {
        "dimension":
            "Most Severe Tested Capacity Degradation",
        "result":
            "Destination exits: 69.25 Mbps",
        "business_implication":
            "Destination-facing capacity is a major structural constra
    },

    {
        "dimension":
            "Sensitive Latency Exposure",
        "result":
            "850 to 644 Mbps-ms",
        "business_implication":
            "Split Stage 2 reduces sensitive latency exposure by appro
    },

    {
        "dimension":
            "Local Capacity Candidates",
        "result":
            "B->D, C->UK, C->US",
        "business_implication":
            "Additional capacity on these links increases optimized th
    },
```

```
    {
        "dimension":
            "Fairness",
        "result":
            "Minimum commodity delivery ratio = 0",
        "business_implication":
            "Throughput maximization alone can completely starve indiv
    },

    {
        "dimension":
            "Recommended Architecture",
        "result":
            "Adaptive / policy-aware",
        "business_implication":
            "Use simpler routing when sufficient; retain split capabil
    }
])


executive_scorecard_df
```

Out [532...

	dimension	result	business_implication
0	Total Offered Demand	126 Mbps	Baseline demand exceeds feasible network capac...
1	Maximum Baseline Throughput	83 Mbps	43 Mbps remains undelivered even under optimiz...
2	Baseline Delivery Rate	65.87%	Approximately one-third of baseline demand can...
3	Naive Minimum-Latency Routing	Infeasible	Shared-capacity routing decisions must be opti...
4	Split vs Single Baseline	83 vs 83 Mbps	Splitting provides no baseline Stage 1 through...
5	Split vs Single Stress Scenarios	Throughput tie in all tested scenarios	Single-path restriction was not throughput-bin...
6	Low Demand Scenario	75.6 / 75.6 Mbps delivered	The network can serve 100% of demand when load...
7	High Demand Scenario	83 / 189 Mbps delivered	The network becomes capacity-limited rather th...
8	Most Severe Tested Capacity Degradation	Destination exits: 69.25 Mbps	Destination-facing capacity is a major structu...
9	Sensitive Latency Exposure	850 to 644 Mbps-ms	Split Stage 2 reduces sensitive latency exposu...
10	Local Capacity Candidates	B->D, C->UK, C->US	Additional capacity on these links increases o...
11	Fairness	Minimum commodity delivery ratio = 0	Throughput maximization alone can completely s...
12	Recommended Architecture	Adaptive / policy-aware	Use simpler routing when sufficient; retain sp...

Deep Analysis of the Executive Scorecard

The scorecard demonstrates that no single decision solves the entire network problem.

Capacity Problem

The network delivers:

$$83/126 = 65.87\%$$

of baseline demand.

Routing Problem

Naive shortest routing is infeasible.

Coordinated optimization is required.

Architecture Problem

Single-path and split routing tie on baseline throughput.

Therefore, multipath complexity should not be justified by baseline Mbps alone.

Service-Quality Problem

Splittable Stage 2 improves sensitive latency.

Therefore, routing flexibility can provide quality-of-service value.

Investment Problem

Saturation does not necessarily indicate where added capacity creates service.

Marginal capacity value provides a stronger signal.

Governance Problem

Maximum throughput does not guarantee fair service.

Fairness and SLAs require explicit policy.

These are separate decision layers and should be managed separately.

Executive Business Insight

The optimization should be viewed as a decision stack.

The recommended sequence is:

1. determine what the physical network can support;
2. determine how scarce capacity should be allocated;
3. optimize quality of service;
4. simplify the operating plan;
5. identify where infrastructure expansion creates value.

This structure makes each decision easier to explain, govern, and audit.

45.17 Final Executive Recommendation

The baseline network should not use independent minimum-latency routing because shared-capacity interactions make that approach infeasible.

A centralized optimization framework should instead:

1. maximize feasible throughput;
2. apply fairness and service-policy requirements;
3. optimize latency-sensitive routing;
4. simplify the final operating plan;
5. monitor marginal capacity value;
6. re-optimize when demand or infrastructure conditions change.

Routing Architecture Recommendation

For the tested network:

$$T^*_{single} = T^*_{split}$$

under every Stage 1 scenario evaluated.

Therefore, the current evidence does **not** support splittable routing as a means of increasing total delivered Stage 1 throughput.

Single-path routing remains attractive as the normal operating mode because it offers:

- equivalent tested throughput;
- simpler route interpretation;
- simpler monitoring;
- lower operational complexity.

However, splittable-routing capability should be retained.

Its demonstrated value appears primarily in:

- lower sensitive latency;
- finer-grained routing flexibility.

Within the splittable pipeline, Stage 2 reduces sensitive latency exposure by approximately:

$$24.24\%.$$

Therefore, routing flexibility should be activated when its measurable service-quality value justifies additional operational complexity.

Capacity Recommendation

The remaining throughput limitation is primarily physical.

Capacity investment should therefore prioritize links based on:

$$MV_e = T^*(C_e + 1) - T^*(C_e)$$

rather than utilization alone.

The strongest tested local capacity candidates include:

- B → D;
- C → UK;
- C → US.

Stress testing also shows that destination-facing capacity deserves particular attention because the tested exit-degradation scenario produces the largest throughput reduction among the evaluated capacity-degradation cases.

Service-Policy Recommendation

Maximum throughput does not guarantee acceptable service allocation.

The minimum commodity delivery ratio under the throughput-only solutions is:

$$0.$$

Therefore, production deployment should explicitly incorporate:

- fairness;
- SLA minimums;
- traffic-class policy;
- business priorities.

Recommended Production Hierarchy

The final recommended optimization architecture is:

$$\text{Throughput} \rightarrow \text{Fairness / SLA Policy} \rightarrow \text{Sensitive Latency} \rightarrow \text{Operational C}$$

This structure separates:

- what the physical network can support;
- how scarce service should be allocated;
- how admitted traffic should be routed;
- how operational complexity should be controlled.

The result is a routing framework that is technically efficient, policy-aware, and operationally defensible.

45.18 Connection to Section 46

The conclusions above are based on a deliberately simplified optimization model.

Before treating them as production recommendations, the assumptions and limitations must be stated explicitly.

Section 46 therefore documents:

- modeling assumptions;
- analytical limitations;
- interpretation boundaries;
- production extensions;
- final conclusions.

This establishes exactly where the results are strong and where additional modeling would be required.

46. Assumptions, Limitations, Production Extensions, and Final Conclusion

46.1 Purpose

Optimization results are valid only within the assumptions of the model that produced them.

The current assessment intentionally simplifies real network behavior in order to isolate the core routing and capacity-allocation problem.

Section 46 documents those assumptions explicitly.

The purpose is not to weaken the conclusions.

Instead, it establishes the correct interpretation boundary:

> **What does this model demonstrate reliably, and what would require additional modeling before production deployment?**

Interpretation Boundary

The assumptions do not invalidate the results.

They define what the results mean.

For example:

- 83 Mbps is optimal under the modeled static network;
- 18.25 ms is a fixed-link-latency routing result;
- marginal capacity value is a local one-Mbps effect.

Therefore:

> **The model provides rigorous conclusions inside its defined abstraction.**

Production deployment would require additional layers of realism.

46.2 Static Network Snapshot

The model represents one fixed point in time.

Demand and capacity remain constant during each optimization solve.

There is no:

- time-indexed traffic;
- queue carryover;
- traffic forecasting inside the optimization;
- dynamic demand evolution.

Therefore, the model answers:

> **What is the best feasible routing allocation for this network snapshot?**

It does not simulate how traffic changes dynamically over time.

46.3 No Queuing Delay

Path latency is modeled as the sum of fixed link latencies:

$$L_p = \sum_{e \in p} L_e.$$

Latency does not increase as utilization approaches capacity.

Therefore, a link at:

$$100\%$$

utilization has the same modeled latency as the same link at lower utilization.

Real networks may behave differently because queueing delay often increases as links become heavily loaded.

Business Implication

The modeled latency results may be optimistic near highly utilized links.

Real networks operating near capacity may experience:

- queue buildup;
- variable latency;
- jitter;
- packet loss.

Therefore, modeled latency should be interpreted as:

> **routing-path latency under fixed link-delay assumptions**

rather than a guaranteed production SLA.

Production deployment should pair optimization with real telemetry.

Production Extension

A more realistic model could include:

- utilization-dependent delay;
- piecewise-linear queueing approximations;
- empirical latency curves;
- nonlinear congestion models.

This would link congestion directly to service quality.

46.4 Lossless Transmission

The model assumes all routed traffic successfully traverses each selected link.

It does not include:

- packet loss;
- retransmission;
- corruption;
- congestion drop;
- reliability probabilities.

Delivered flow is therefore treated as successful end-to-end throughput.

46.5 Unlimited Node Processing Capacity

Only link capacities constrain traffic.

Nodes themselves have no explicit limit on:

- processing;
- switching;
- CPU;
- memory;
- forwarding rate.

In a real network, intermediate nodes may become bottlenecks even when adjacent links retain spare capacity.

46.6 Full-Duplex Independent Directional Capacity

Each physical link is represented using two directed arcs.

Capacity in one direction is independent of capacity in the reverse direction.

For example:

$$C_{A \to C}$$

and:

$$C_{C\to A}$$

are separate capacity resources.

This is consistent with the stated full-duplex network assumption.

46.7 Candidate Paths Are Pre-Enumerated

The optimizer does not generate arbitrary routes during the solve.

Instead, it selects from a predefined candidate-path set.

The current analysis contains approximately:

$$68$$

candidate path-stream records.

Therefore, optimization is exact relative to the candidate routes that were supplied.

If a better route was never generated, the solver cannot discover it.

Deep Analysis

The optimizer may be globally optimal over the candidate-path set while still missing a potentially useful route that was excluded from the enumeration.

Therefore:

$$\text{Optimization Quality} \leq \text{Quality of Candidate Path Set}.$$

For this small synthetic network, path enumeration is manageable.

For larger networks, candidate-path generation becomes an important part of model quality.

Production Extension

Larger systems could use:

- k-shortest paths;
- column generation;
- dynamic path generation;
- decomposition methods.

These approaches improve scalability and reduce dependence on a fixed candidate-route list.

46.8 Destination Nodes Are Terminals

UK-bound traffic must terminate at UK.

US-bound traffic must terminate at US.

Candidate paths do not cross the wrong destination and continue elsewhere.

This prevents unrealistic paths such as:

$$S1 \to \cdots \to UK \to \cdots \to US.$$

46.9 Demand Is Source Specific

Each source-stream pair is modeled as an independent commodity.

For example:

- S1 UK-L;
- S2 UK-L;
- B UK-L;

are distinct commodities.

Demand is not pooled across sources.

This makes source-level service and fairness analysis possible.

46.10 Partial Delivery Is Allowed

For every commodity:

$$0 \leq z_{s,k} \leq D_{s,k}.$$

Therefore, a commodity may be partially served.

This applies under both:

- splittable routing;
- single-path routing.

Under single-path routing, partial delivery means:

> **The commodity may receive less than full demand, but all positive delivered flow must use one route.**

Why Partial Delivery Matters

This assumption materially affects the split-vs-single comparison.

The current single-path formulation is more flexible than an all-or-nothing unsplittable-flow model.

A commodity can still receive:

$$0 < z_{s,k} < D_{s,k}$$

while using one route.

Therefore, the baseline result:

$$T^*_{single} = T^*_{split}$$

should be interpreted in the context of partial delivery.

If commodities were required to be either:

- fully served;
- fully rejected;

the single-path throughput result could differ.

46.11 No Hard Latency Threshold

46.11.1 MARKDOWN CELL

46.11 No Hard Latency Threshold

Latency-sensitive traffic is optimized by minimizing latency exposure.

There is no hard constraint such as:

$$L_p \leq 30 \text{ ms}.$$

Therefore, a higher-latency route remains feasible if required.

A production SLA may require explicit latency caps.

46.12 No Sensitive-First Admission Priority

Stage 1 treats every delivered Mbps equally.

It does not prioritize:

- UK-L;
- US-L;

over tolerant streams.

Therefore, sensitive traffic can be dropped during Stage 1 throughput maximization.

Stage 2 can improve latency only for sensitive traffic that Stage 1 already admitted.

This is why Section 44 distinguishes:

- admission policy;
- routing-quality optimization.

46.13 Tolerant Traffic Latency Is Unpriced in Stage 2

Stage 2 minimizes latency only for:

- UK-L;
- US-L.

Tolerant traffic latency is not part of the objective.

Therefore, tolerant traffic may use long routes when doing so improves sensitive latency while preserving throughput.

Those routes are mathematically valid but may be operationally undesirable.

Production Extension

A later cleanup stage could minimize:

- tolerant latency;
- number of active routes;

- unnecessary splitting.

This could preserve:

- throughput;
- fairness;
- sensitive-latency optimum;

while improving operational simplicity.

46.14 Throughput Degeneracy

Several different admission patterns can produce the same:

$$83 \text{ Mbps}$$

Stage 1 optimum.

Therefore, the Stage 1 solution is not necessarily unique.

This explains why the split and single-path models can produce different:

- commodity admission;
- source service;
- stream service;
- link utilization;

while achieving the same objective value.

Business Significance

Degeneracy is not merely a mathematical curiosity.

It means the solver may have several equally efficient 83-Mbps solutions that produce very different business outcomes.

One solution may favor:

- one source;
- another traffic class;
- another destination.

Therefore, production deployment should use secondary objectives deliberately.

Possible secondary objectives include:

- fairness;
- customer priority;
- sensitive admission;
- latency;
- operational simplicity.

46.15 Split-vs-Single Latency Comparison Is Not Fully Controlled

The final split and single-path Stage 2 models preserve different Stage 1 admission vectors.

Therefore:

$$\text{Observed Latency Difference} = \text{Admission-Mix Effect} + \text{Routing-Flexibility}$$

The comparison is valid for evaluating the complete pipelines.

However, it should not be interpreted as a perfectly controlled causal estimate of splitting alone.

Deep Interpretation

For UK-L:

- the split model serves approximately 16 Mbps;
- the single-path model serves approximately 12 Mbps.

Therefore, the average-latency calculations are based on different admitted traffic populations.

The result:

$$18.25 < 21.17$$

is still operationally meaningful.

It shows:

> **The complete split pipeline delivers more UK-L traffic while also producing lower observed average UK-L latency.**

However, it does not prove that route splitting alone explains the full difference.

Future Controlled Experiment

A stronger causal comparison would:

1. define one common admission vector;
2. fix that vector under both routing models;
3. optimize sensitive latency;
4. compare resulting latency metrics.

Any remaining difference would more directly isolate routing flexibility.

46.16 Marginal Capacity Value Is Local

Section 41 increases each directed link by:

$$1 \text{ Mbps}.$$

Therefore:

$$MV_e = T^*(C_e + 1) - T^*(C_e)$$

is a local finite-difference measure.

A result such as:

$$MV_e = 1$$

does not imply that adding:

$$10 \text{ Mbps}$$

will create:

$$10 \text{ Mbps}$$

of additional throughput.

Business Interpretation

A positive one-Mbps marginal value means:

> **This is where the next unit of bandwidth has value.**

It does not mean:

> **Buy unlimited bandwidth here.**

Once a bottleneck is expanded, another link may become limiting.

Therefore, capacity planning should be iterative:

1. propose an upgrade;
2. re-optimize;
3. identify the new bottleneck;
4. evaluate the next increment.

46.17 Joint Bottlenecks

The one-link-at-a-time perturbation analysis measures individual local value.

However, some bottlenecks may require coordinated expansion.

It is possible that:

$$MV_a = 0$$

and:

$$MV_b = 0$$

individually,

while:

$$T^*(C_a + 1, C_b + 1) > T^*(C).$$

Therefore:

$$\boxed{\text{Zero Individual Marginal Value} \not\Rightarrow \text{Zero Joint Upgrade Value}}$$

46.18 No Failure Probabilities or Reliability Model

The scenario analysis reduces selected capacities but does not model:

- failure probabilities;
- mean time between failures;
- recovery times;
- correlated outages.

Therefore, resilience is tested deterministically rather than probabilistically.

46.19 No Route-Churn Penalty

The optimization can change routes freely between solves.

There is no explicit penalty for:

- changing paths;
- updating forwarding rules;
- migrating traffic;
- transient disruption.

A production environment may prefer route stability unless a performance improvement justifies the operational change.

Production Extension

A future model could include:

$$\text{Route Change Cost}$$

as an objective or constraint.

This would create a tradeoff between:

- mathematical optimality;
- operational stability.

46.20 Deterministic Demand and Capacity

All demand and capacity values are treated as known constants.

Real networks contain uncertainty in:

- traffic demand;
- available capacity;
- latency;
- failure states.

A production model could therefore use:

- stochastic programming;
- robust optimization;
- chance constraints;
- scenario-weighted optimization.

Why Uncertainty Matters

A solution optimized perfectly for one deterministic point may be fragile if operating conditions change.

For example:

- demand may spike;
- effective bandwidth may fall;
- latency may increase;
- a link may degrade.

A mature optimization system should therefore optimize not only expected performance but also robustness.

Section 43 provides an initial deterministic stress-testing framework.

46.21 Synthetic Data

The assessment uses synthetic network and demand data.

Therefore, numerical values such as:

$$83 \text{ Mbps}$$

apply only to the constructed assessment network.

However, the methodological conclusions are more general.

Examples include:

- independent shortest paths can violate shared capacity;
- splittable routing has a superset feasible region;
- equal objective value does not imply equal service;
- saturation does not equal marginal capacity value;
- fairness must be explicitly modeled.

46.22 Scalability Considerations

The current problem is relatively small:

- 9 nodes;
- 22 directed arcs;

- 20 commodities;
- approximately 68 candidate path-stream records.

The splittable models are LPs.

The single-path models are MILPs and introduce binary route-selection variables.

As the network grows, computational complexity can increase rapidly with:

- number of commodities;
- number of candidate paths;
- number of time periods;
- number of scenarios.

Scalability Business Insight

The two routing policies create an interesting implementation tradeoff.

Splittable routing is:

- operationally more complex;
- mathematically continuous and potentially easier to optimize at scale.

Single-path routing is:

- operationally simpler;
- mathematically discrete and potentially harder to optimize at large scale.

Therefore, production architecture decisions should consider both:

- network operations complexity;
- optimization-compute complexity.

Production Extension

Large networks may require:

- decomposition;
- column generation;
- hierarchical optimization;
- rolling-horizon optimization;
- distributed optimization.

46.23 Validation Boundaries

The notebook contains several internal validation checks:

- demand totals;
- directed-link counts;
- candidate-path counts;
- capacity consistency;
- path-edge validity;
- solver optimality;
- baseline reproduction;
- theoretical relationship:

$$T^*_{single} \leq T^*_{split}.$$

These checks increase confidence in internal model consistency.

However, they do not constitute external production validation.

Production Validation

A production implementation should also compare model outputs with:

- network telemetry;
- actual route behavior;
- measured latency;
- historical congestion;
- failure incidents;
- engineering acceptance tests.

This distinguishes:

> **model correctness**

from:

> **real-world predictive validity.**

46.24 Recommended Future Work

Several extensions would strengthen the model.

1. Fairness and SLA Optimization

Add:

- minimum delivery ratios;
- source guarantees;
- traffic-class priorities.

2. Real Telemetry Validation

Calibrate with observed:

- traffic;
- utilization;
- latency.

3. Queue-Aware Latency

Model congestion-related delay.

4. Failure and Resilience Modeling

Evaluate:

- single-link failures;
- node failures;
- correlated outages.

5. Controlled Split-vs-Single Latency Experiment

Use identical admission vectors.

6. Route-Churn Penalties

Reduce unnecessary operational changes.

7. Capacity-Expansion Optimization

Introduce:

$$\Delta C_e$$

as decision variables with upgrade costs.

8. Stochastic / Robust Optimization

Model uncertainty.

9. Adaptive Routing Controller

Determine when split routing should be activated.

10. Time-Indexed Optimization

Model demand and routing across time.

Prioritized Future Work

Not every extension has equal immediate value.

A practical implementation roadmap would be:

Priority 1 — Fairness and SLA Policy

The current model already shows materially different service allocations.

Therefore, fairness is the most immediate modeling gap.

Priority 2 — Real Telemetry Validation

Confirm that the abstraction reflects actual operating behavior.

Priority 3 — Queue-Aware Latency

This becomes important because several optimized links operate near capacity.

Priority 4 — Failure and Resilience Analysis

Evaluate whether alternate corridors remain usable under failures.

Priority 5 — Capacity Cost Modeling

Convert technical marginal value into:

$$\text{Cost per Incremental Delivered Mbps}.$$

Priority 6 — Dynamic / Time-Based Optimization

Move from static snapshots toward operational control.

This sequence prioritizes extensions most likely to materially change business decisions.

46.X Scenario Generalization Limitation

The stress-testing results show no Stage 1 throughput advantage for splittable routing in any scenario evaluated.

This is a strong result for the tested synthetic network, but it should not be generalized to all possible network topologies.

The theoretical relationship remains:

$$T^*_{single} \leq T^*_{split}.$$

A different network may contain:

- narrower parallel paths;
- different source placement;
- different demand granularity;
- different all-or-nothing service requirements;
- more severe capacity fragmentation;

such that:

$$T^*_{single} < T^*_{split}.$$

Therefore, the correct conclusion is:

> **Splittable routing did not improve Stage 1 throughput in the baseline or any tested stress scenario for this network.**

The correct conclusion is not:

> **Splittable routing can never improve throughput.**

This distinction preserves the validity of the theoretical feasible-region relationship while accurately reporting the empirical results of the assessment.

46.25 Final Technical Conclusion

The assessment demonstrates that network routing must be treated as a joint optimization problem rather than a collection of independent shortest-path decisions.

The baseline network contains:

$$126 \text{ Mbps}$$

of demand but can deliver only:

$$83 \text{ Mbps}$$

under the current topology and capacity configuration.

The resulting delivery rate is:

$$65.87\%.$$

Both routing formulations achieve the same baseline throughput:

$$T^*_{split} = T^*_{single} = 83.$$

Therefore, the inability to split individual commodities is not the primary baseline throughput limitation.

However, splittable routing provides additional flexibility that produces better observed sensitive-latency outcomes.

The infrastructure analysis also demonstrates that:

$$\boxed{\text{Saturation} \neq \text{Marginal Capacity Value}}$$

Finally, maximum throughput does not guarantee acceptable service distribution.

A production system should explicitly incorporate:

- fairness;
- SLAs;
- traffic priorities;
- latency requirements;
- operational simplicity.

Integrated Technical Interpretation

The analysis reveals four distinct constraint layers.

Layer 1 — Physical Capacity

The network cannot deliver all baseline demand.

$$126 - 83 = 43 \text{ Mbps}$$

remains undelivered even under optimal routing.

Layer 2 — Routing Architecture

The single-path restriction does not reduce baseline throughput.

Therefore, routing architecture is not the primary baseline capacity constraint.

Layer 3 — Service Quality

Routing architecture does affect latency.

The final split solution produces better observed sensitive-latency outcomes.

Layer 4 — Allocation Policy

Neither throughput formulation guarantees fair service distribution.

Therefore, allocation policy must be explicitly modeled.

These layers should not be collapsed into one objective because each answers a different decision question.

46.26 Final Business Conclusion

The most practical recommendation is an **adaptive, policy-aware routing architecture**.

Under Normal Conditions

Use the simplest routing strategy that satisfies service requirements.

For the baseline analyzed here, single-path routing achieves the same maximum throughput as splittable routing.

When Service Quality or Network Stress Requires Additional Flexibility

Retain the ability to activate splittable routing when it creates measurable value through:

- lower sensitive latency;
- improved use of fragmented capacity;
- improved degraded-network performance.

For Infrastructure Planning

Prioritize upgrades using:

$$\text{Marginal Capacity Value}$$

rather than saturation alone.

For Service Governance

Use an explicit hierarchy:

$$\boxed{\text{Throughput} \rightarrow \text{Fairness / SLA} \rightarrow \text{Sensitive Latency} \rightarrow \text{Operational Simplic}}$$

Business Insights From the Complete Analysis

The complete assessment produces several business-level conclusions.

1. Optimization Cannot Replace Missing Infrastructure

The optimizer improves use of existing capacity, but it cannot deliver the:

$$43 \text{ Mbps}$$

of baseline demand that the current network cannot physically support.

2. More Sophisticated Routing Is Not Automatically Better

Splittable routing is mathematically more flexible, yet it does not improve baseline throughput.

Technical complexity should therefore be justified by measurable value.

3. The Baseline Value of Splitting Is Primarily Service Quality

Its strongest demonstrated baseline advantage is lower sensitive latency rather than additional delivered Mbps.

4. Scarcity Requires Governance

Because not all demand can be served, the organization must define how limited capacity should be distributed.

That is a business-policy decision.

5. Infrastructure Spending Should Be Marginal-Value Driven

Capacity investment should focus on links where additional bandwidth increases end-to-end service rather than simply selecting the most heavily utilized asset.

6. Architecture Should Be Adaptive

The strongest design is likely neither permanently single-path nor permanently splittable.

Instead:

- use simpler routing when conditions permit;
- activate additional flexibility when measurable value exists.

This balances performance with operational complexity.

46.27 Final Assessment Closing Statement

This analysis began with a relatively small synthetic network, but the underlying decision problem is broadly applicable.

The key lesson is that network performance cannot be understood from one metric alone.

Throughput, latency, fairness, routing complexity, and capacity investment interact.

A rigorous optimization framework should therefore:

1. identify what is physically feasible;
2. separate throughput from service policy;
3. protect latency-sensitive traffic;
4. validate whether added routing complexity creates measurable value;
5. distinguish utilization from true bottleneck value;
6. test conclusions under stressed conditions;
7. document assumptions and limitations explicitly.

That combination of optimization, validation, interpretation, and policy design provides a stronger foundation for real-world network decision-making.

Final Decision Framework

The completed analysis suggests that network decisions should follow a disciplined hierarchy.

Question 1 — What Is Physically Possible?

Solve maximum throughput.

Question 2 — How Should Scarce Capacity Be Allocated?

Apply fairness, priority, and SLA policy.

Question 3 — How Should Admitted Traffic Be Routed?

Optimize sensitive latency and operational quality.

Question 4 — Where Should Infrastructure Be Expanded?

Use marginal-capacity analysis.

Question 5 — Will the Decision Remain Good Under Stress?

Run demand, degradation, and fragmentation scenarios.

This produces the integrated decision framework:

$$\text{Physical Feasibility} \rightarrow \text{Business Policy} \rightarrow \text{Service Quality} \rightarrow \text{Infrastructure}$$

The most important conclusion from the assessment is therefore not one routing path, one objective value, or one bottleneck.

It is the recognition that:

> **Network optimization is a layered decision problem involving capacity, allocation, quality of service, operational complexity, and infrastructure investment.**

Treating those layers explicitly produces decisions that are more technically defensible, operationally practical, and aligned with business objectives.

In [ ]: